台達電子工業股份有限公司

DELTA CONFIDENTIAL

PCI-DMC-x01&02 Series

高速通訊型 PCI 介面 12/6 軸運動控制卡

程式開發手冊

版本:16.01.28

關於此手冊...

使用者須知

請將本手冊置於安全的地方。

由於可能因產品種類的新增、技術的改善與異動、或是文案數據和表的變更，敝公司將保留本手冊更新的權利，恕不另行通知。

未經台達電子公司書面同意，請勿複印、複製本手冊的任一章節內文。

商標

Windows XP/7, Visual Studio, Visual C++, Visual BASIC 的註冊商標屬於Microsoft所有。

BCB (Borland C++ Builder) 的註冊商標屬於Borland公司所有。

其它產品名稱均僅作識別之用，其註冊商標均屬該原公司所有。

技術支持與服務

假若您需要技術支援、服務與相關訊息，或者您對於在使用本產品過程中有任何問題，歡迎您連結至我們的網站(http://www.delta.com.tw/industrialautomation)或與我們聯絡。我們期待能夠依照您的需求，提供我們最的支持與服務給您。可以透過以下方式聯絡我們。

ASIA
DELTA ELECTRONICS, INC.
Taoyuan Plant 1
31-1, XINGBANG ROAD,
GUISHAN INDUSTRIAL ZONE,
TAOYUAN COUNTY 33370, TAIWAN, R.O.C.
TEL: 886-3-362-6301
FAX: 886-3-362-7267

NORTH/SOUTH AMERICA
DELTA PRODUCTS CORPORATION (USA)
Raleigh Office
P.O. BOX 12173
5101 DAVIS DRIVE,
RESEARCH TRIANGLE PARK, NC 27709, U.S.A.
TEL: 1-919-767-3813
FAX: 1-919-767-3969

JAPAN
DELTA ELECTRONICS (JAPAN), INC.
Tokyo Office
DELTA SHIBADAIMON BUILDING
2-1-14 SHIBADAIMON, MINATO-KU,
TOKYO, 105-0012, JAPAN
TEL: 81-3-5733-1111
FAX: 81-3-5733-1211

EUROPE
DELTRONICS (THE NETHERLANDS) B.V.
Eindhoven Office
DE WITBOGT 15, 5652 AG EINDHOVEN,
THE NETHERLANDS
TEL: 31-40-259-2850
FAX: 31-40-259-2851

DELTA CONFIDENTIAL

第一章 API 函式庫介紹----------1
1.1 函式庫的使用----------1
1.2 開始編輯新的專案----------1

第二章 命令傳回值與訊息描述----------1
2.1 錯誤傳回值一覽----------1

第三章 操作原理----------10
3.1 軸卡初始化----------10
3.2 讀取/寫入驅動器參數----------5
3.3 CANopen 協定的操作----------8
3.4 原點復歸操作運動控制----------10
3.5 轉矩運動控制----------15
3.6 速度運動控制----------20
3.7 單軸運動控制----------25
3.8 雙軸運動控制----------37
3.9 三軸運動控制----------53
3.10 多軸運動控制----------62
3.11 同步運動控制----------67
3.12 Speed Continue----------74
3.13 螺旋補間 Helix 應用 Normal_Follow----------80
3.14 電子凸輪----------92
3.15 間隙補償----------99
3.16 背隙補償----------106
3.17 軟體極限操作----------113
3.18 遠端擴充 IO 模組 Input 應用(RM32MN、RM64MN)操作----------120
3.19 遠端擴充 IO 模組 Output 應用(RM32NT、RM64NT)操作----------124
3.20 遠端擴充 IO 模組 In-Out 應用(RM32PT、GA+GE16MN/NT、RM32 MN/NT)操作 --127
3.21 遠端擴充 IO 模組應用-手輪操作----------133
3.22 遠端擴充脈波介面模組應用-RM04PI Mode 1 操作----------139
3.23 遠端擴充脈波介面模組應用-RM04PI Mode 2 操作----------148
3.24 中斷操作 Interrupt----------155
3.25 主控軸卡安全機制操作 Master Security----------162
3.26 遠端擴充類比輸入模組應用-RM04AD----------169
3.27 遠端擴充類比輸出模組應用-RM04DA----------172
3.28 位置比較 Compare----------176
3.29 Axis Group----------183
3.30 DDA 應用 DDA Axes Move----------187
3.31 位置比較 B02_Compare----------191
3.32 Table Motion 範例----------201

DELTA CONFIDENTIAL

3.33 B02 Latch 範例 ----208
3.34 Motion Buffer 範例 ----221
3.99 歷程記錄器 ----227

第四章 控制 API----2
4.1 資料型別與範圍----2
4.2 函數描述----3

第五章 硬體初始化 API----1
5.1 _DMC_01_open----1
5.2 _DMC_01_close----1
5.3 _DMC_01_get_CardNo_seq----2
5.4 _DMC_01_pci_initial ----3
5.5 _DMC_01_get_card_version----4
5.6 _DMC_01_get_fpga_version----5
5.7 _DMC_01_get_model_type ----6

第六章 軸卡操作 API----1
6.1 _DMC_01_initial_bus ----1
6.2 _DMC_01_start_ring ----2
6.3 _DMC_01_get_device_table----3
6.4 _DMC_01_get_node_table----4
6.5 _DMC_01_reset_card ----5
6.6 _DMC_01_check_nodeno ----6
6.7 _DMC_01_get_master_connect_status----7
6.8 _DMC_01_get_mailbox_Error ----8
6.9 _DMC_01_get_mailbox_cnt----9
6.10 _DMC_01_get_dsp_cnt---- 10
6.11 _DMC_01_set_dio_output ---- 11
6.12 _DMC_01_get_dio_output ---- 12
6.13 _DMC_01_get_dio_input ---- 13
6.14 _DMC_01_set_dio_output_DW---- 14
6.15 _DMC_01_set_dio_single_output_DW ---- 15
6.16 _DMC_01_get_dio_output_DW ---- 16
6.17 _DMC_01_get_dio_single_output_DW ---- 17
6.18 _DMC_01_get_dio_input_DW---- 18
6.19 _DMC_01_get_dio_single_input_DW ---- 19
6.20 _DMC_01_set_dio_output_DW_error_handle ---- 20
6.21 _DMC_01_get_dio_output_DW_error_handle ---- 21
6.22 _DMC_01_get_cycle_time ---- 22
6.23 _DMC_01_initial_bus2 ---- 23

DELTA CONFIDENTIAL

6.24 _DMC_01_motion_cnt ---- 24
6.25 _DMC_01_pdo_Rx_error ---- 25
6.26 _DMC_01_pdo_Tx_error ---- 26
6.27 _DMC_01_get_motion_error_code ---- 27
6.28 _DMC_01_get_motion_error_cnt ---- 28
6.29 _DMC_01_get_DLL_path ---- 29
6.30 _DMC_01_get_DLL_version ---- 30
6.31 _DMC_01_get_DLL_path_Single ---- 1
6.32 _DMC_01_get_DLL_version_Single ---- 2
6.33 _DMC_01_set_dio_single_output_DW_trigger_time ---- 3
6.34 _DMC_01_set_dio_single_output_DW_one_shot ---- 5
6.35 _DMC_01_get_dio_single_output_DW_trigger_status ---- 6
6.36 _DMC_01_set_dio_single_input_DW_trigger_mode ---- 8
6.37 _DMC_01_get_dio_single_input_DW_trigger_mode ---- 10
6.38 _DMC_01_get_dio_single_input_DW_trigger_flag ---- 12
6.39 _DMC_01_set_dio_single_output ---- 13
6.40 _DMC_01_get_dio_single_output ---- 14
6.41 _DMC_01_get_dio_single_input ---- 15
6.42 _DMC_01_set_dio_single_output_DW_trigger_cycle ---- 16
6.43 _DMC_01_set_dio_single_output_DW_trigger_enable ---- 18

第七章 伺服驅動器參數讀取/寫入 API ---- 19
7.1 _DMC_01_read_servo_parameter ---- 20
7.2 _DMC_01_write_servo_parameter ---- 21

第八章 使用 SDO 協定 API ---- 1
8.1 CANopen SDO 協定格式 ---- 2
8.2 _DMC_01_check_canopen_lock ---- 5
8.3 _DMC_01_get_canopen_ret ---- 6
8.4 _DMC_01_set_pdo_mode ---- 7
8.5 _DMC_01_send_message ---- 8
8.6 _DMC_01_send_message2 ---- 10
8.7 _DMC_01_send_message3 ---- 11
8.8 _DMC_01_send_message4 ---- 12
8.9 _DMC_01_read_message ---- 13
8.10 _DMC_01_read_message2 ---- 14
8.11 _DMC_01_get_message ---- 16
8.12 _DMC_01_reset_sdo_choke ---- 17
8.13 _DMC_01_get_sdo_retry_history ---- 1

第九章 點對點運動控制使用封包協定 API ---- 2

DELTA CONFIDENTIAL

9.1 _DMC_01_set_sdo_driver_speed_profile ---- 1
9.2 _DMC_01_start_sdo_driver_r_move ---- 2
9.3 _DMC_01_start_sdo_driver_a_move ---- 3
9.4 _DMC_01_start_sdo_driver_new_position_move ---- 4

第十章 原點復歸運動控制使用封包協定 API ---- 1
10.1 _DMC_01_set_home_config ---- 1
10.2 _DMC_01_set_home_move ---- 6
10.3 _DMC_01_set_home_config2 ---- 7
10.4 _DMC_01_set_home_config_offset2 ---- 8
10.5 _DMC_01_set_home_move2 ---- 9
10.6 _DMC_01_escape_home_move ---- 1
10.7 _DMC_01_set_home_config_by_torque_limit ---- 2

第十一章 速度運動控制使用封包協定 API ---- 4
11.1 _DMC_01_set_velocity_mode ---- 1
11.2 _DMC_01_set_velocity ---- 2
11.3 _DMC_01_set_velocity_stop ---- 3
11.4 _DMC_01_get_rpm ---- 5
11.5 _DMC_01_set_velocity_torque_limit ---- 1

第十二章 轉矩運動控制使用封包協定 API ---- 2
12.1 _DMC_01_set_torque_mode ---- 1
12.2 _DMC_01_set_torque ---- 2
12.3 _DMC_01_set_torque_stop ---- 3
12.4 _DMC_01_set_torque_velocity_limit ---- 4

第十三章 使用 PDO 協定 API ---- 1
13.1 _DMC_01_ipo_set_svon ---- 1
13.2 _DMC_01_get_buffer_length ---- 2
13.3 _DMC_01_buf_dwell ---- 3
13.4 _DMC_01_command_buf_clear ---- 4
13.5 _DMC_01_set_group ---- 1
13.6 _DMC_01_set_dmc_mode ---- 3

第十四章 停止運動控制 API ---- 4
14.1 _DMC_01_emg_stop ---- 5
14.2 _DMC_01_sd_stop ---- 6
14.3 _DMC_01_sd_abort ---- 7
14.4 _DMC_01_set_sld_mode ---- 8
14.5 _DMC_01_set_sld_stop ---- 9

DELTA CONFIDENTIAL

14.6 _DMC_01_set_sd_time ---- 10
14.7 _DMC_01_set_scurve_rate ---- 11
14.8 _DMC_01_set_dio_input_level_emg ---- 1
14.9 _DMC_01_set_dio_input_trigger_level ---- 2
14.10 _DMC_01_set_dio_input_level_emg_clear ---- 3
14.11 _DMC_01_get_dio_input_level_emg_status ---- 4

第十五章 運動狀態 API ---- 5
15.1 _DMC_01_motion_done ---- 6
15.2 _DMC_01_motion_status ---- 8
15.3 _DMC_01_set_01_ph_inposition ---- 10
15.4 _DMC_01_get_01_ph_inposition ---- 11

第十六章 運動計數值 API ---- 1
16.1 _DMC_01_get_command ---- 1
16.2 _DMC_01_set_command ---- 2
16.3 _DMC_01_get_position ---- 3
16.4 _DMC_01_set_position ---- 4
16.5 _DMC_01_get_target_pos ---- 5
16.6 _DMC_01_get_torque ---- 6
16.7 _DMC_01_get_current_speed ---- 7
16.8 _DMC_01_get_current_speed_rpm ---- 1

第十七章 軟體極限 API ---- 2
17.1 _DMC_01_set_soft_limit ---- 3
17.2 _DMC_01_enable_soft_limit ---- 4
17.3 _DMC_01_disable_soft_limit ---- 5
17.4 _DMC_01_get_soft_limit_status ---- 1

第十八章 單軸運動控制 API ---- 2
18.1 _DMC_01_start_tr_move ---- 1
18.2 _DMC_01_start_sr_move ---- 3
18.3 _DMC_01_start_ta_move ---- 5
18.4 _DMC_01_start_sa_move ---- 6
18.5 _DMC_01_p_change ---- 7
18.6 _DMC_01_v_change ---- 8
18.7 _DMC_01_start_tr_move_2seg ---- 10
18.8 _DMC_01_start_sr_move_2seg ---- 13
18.9 _DMC_01_start_ta_move_2seg ---- 15
18.10 _DMC_01_start_sa_move_2seg ---- 17
18.11 _DMC_01_start_tr_move_2seg2 ---- 18

DELTA CONFIDENTIAL

18.12 _DMC_01_start_sr_move_2seg2 ---- 20
18.13 _DMC_01_start_ta_move_2seg2 ---- 21
18.14 _DMC_01_start_sa_move_2seg2 ---- 23
18.15 _DMC_01_feedrate_overwrite ---- 25
18.16 _DMC_01_start_v3_move ---- 28
18.17 _DMC_01_multi_table_V3_move ---- 30
18.18 _DMC_01_feedrate_overwrite_read ---- 32

第十九章 二軸線性補間運動控制 API ---- 1
19.1 _DMC_01_start_tr_move_xy ---- 2
19.2 _DMC_01_start_sr_move_xy ---- 4
19.3 _DMC_01_start_ta_move_xy ---- 5
19.4 _DMC_01_start_sa_move_xy ---- 6
19.5 _DMC_01_start_v3_move_xy ---- 7

第二十章 二軸圓弧補間運動控制 API ---- 1
20.1 _DMC_01_start_tr_arc_xy ---- 3
20.2 _DMC_01_start_sr_arc_xy ---- 5
20.3 _DMC_01_start_ta_arc_xy ---- 6
20.4 _DMC_01_start_sa_arc_xy ---- 7
20.5 _DMC_01_start_tr_arc2_xy ---- 8
20.6 _DMC_01_start_sr_arc2_xy ---- 10
20.7 _DMC_01_start_ta_arc2_xy ---- 11
20.8 _DMC_01_start_sa_arc2_xy ---- 12
20.9 _DMC_01_start_tr_arc3_xy ---- 13
20.10 _DMC_01_start_sr_arc3_xy ---- 15
20.11 _DMC_01_start_ta_arc3_xy ---- 17
20.12 _DMC_01_start_sa_arc3_xy ---- 19
20.13 _DMC_01_start_spiral_xy ---- 21
20.14 _DMC_01_start_spiral2_xy ---- 23
20.15 _DMC_01_start_v3_arc_xy ---- 25
20.16 _DMC_01_start_v3_arc2_xy ---- 27
20.17 _DMC_01_start_v3_arc3_xy ---- 29
20.18 _DMC_01_start_v3_spiral_xy ---- 31
20.19 _DMC_01_start_v3_spiral2_xy ---- 33
20.20 _DMC_01_start_multiarc ---- 35
20.21 _DMC_01_Start_Move_Spiral_Circle ---- 38
20.22 _DMC_01_Start_Move_Spiral2_Circle ---- 40
20.23 _DMC_01_Start_Move_Spiral2_Circle ---- 42
20.24 _DMC_01_Start_Arc_follow ---- 44

DELTA CONFIDENTIAL

第二十一章　三軸線性補間運動控制 API ---- 1
21.1 _DMC_01_start_tr_move_xyz ---- 1
21.2 _DMC_01_start_sr_move_xyz ---- 3
21.3 _DMC_01_start_ta_move_xyz ---- 5
21.4 _DMC_01_start_sa_move_xyz ---- 7
21.5 _DMC_01_start_v3_move_xyz ---- 9

第二十二章　三軸螺旋補間運動控制 API ---- 2
22.1 _DMC_01_start_tr_heli_xy ---- 1
22.2 _DMC_01_start_sr_heli_xy ---- 4
22.3 _DMC_01_start_ta_heli_xy ---- 6
22.4 _DMC_01_start_sa_heli_xy ---- 8
22.5 _DMC_01_start_v3_heli_xy ---- 10
22.6 _DMC_01_start_v3_sphere_xyz ---- 13
22.7 _DMC_01_start_v3_heli_multi_axes_move ---- 16

第二十三章　速度運動控制 API ---- 1
23.1 _DMC_01_tv_move ---- 2
23.2 _DMC_01_sv_move ---- 4

第二十四章　同步運動控制 API ---- 1
24.1 _DMC_01_sync_move ---- 2
24.2 _DMC_01_sync_move_config ---- 1

第二十五章　遠端模組控制 API ---- 2
25.1 _DMC_01_get_rm_input_value ---- 1
25.2 _DMC_01_get_rm_input_single_value ---- 2
25.3 _DMC_01_set_rm_input_filter ---- 4
25.4 _DMC_01_set_rm_input_filter_enable ---- 6
25.5 _DMC_01_set_rm_output_value ---- 7
25.6 _DMC_01_set_rm_output_single_value ---- 8
25.7 _DMC_01_set_rm_output_value_error_handle ---- 9
25.8 _DMC_01_get_rm_output_value ---- 10
25.9 _DMC_01_get_rm_output_single_value ---- 11
25.10 _DMC_01_get_rm_output_value_error_handle ---- 13
25.11 _DMC_01_set_rm_output_active ---- 1

第二十六章　電子手輪與 JOG 操作 API ---- 2
26.1 _DMC_01_set_rm_mpg_axes_enable ---- 4
26.2 _DMC_01_set_rm_mpg_axes_enable2 ---- 6

DELTA CONFIDENTIAL

26.3 _DMC_01_set_rm_mpg_axes_enable3 ---- 9
26.4 _DMC_01_set_rm_jog_axes_enable ---- 12
26.5 _DMC_A02_set_mpg_axes_enable ---- 14
26.6 _DMC_A02_mpg_get_io_state_enable ---- 16
26.7 _DMC_A02_mpg_fsm_get_lock_state ---- 17
26.8 _DMC_A02_mpg_get_poisition ---- 18
26.9 _DMC_A02_mpg_axis_get_io_state ---- 19
26.10 _DMC_A02_mpg_axis_get_ratio_state ---- 20
26.11 _DMC_A02_set_jog_axes_enable ---- 21
26.12 _DMC_A02_get_jog_io_state ---- 23
26.13 _DMC_A02_set_mpg_axes_enable2 ---- 24
26.14 _DMC_A02_get_mpg_axes_enable2 ---- 26
26.15 _DMC_A02_set_mpg_axes_extend ---- 27
26.16 _DMC_A02_get_mpg_axes_extend ---- 28
26.17 _DMC_A02_mpg_axis_get_io_state2 ---- 29
26.18 _DMC_A02_mpg_axis_get_ratio_state2 ---- 30
26.19 _DMC_A02_set_mpg_axes_enable3 ---- 31
26.20 _DMC_01_Set_MPG_AMF ---- 33
26.21 _DMC_Qep_set_mpg_axis_enable ---- 34
26.22 _DMC_Qep_get_mpg_axis_enable ---- 36
26.23 _DMC_Qep_get_mpg_axis_cnt ---- 38
26.24 _DMC_Qep_set_mpg_clear ---- 39
26.25 _DMC_MP1_set_mpg_axis_enable ---- 40
26.26 _DMC_MP1_get_mpg_axis_cnt ---- 42
26.27 _DMC_MP1_set_mpg_axis_clear ---- 43
26.28 _DMC_MP1_get_mpg_axis_enable ---- 44

第二十七章 四組脈波介面專用 API ---- 1
27.1 _DMC_01_set_rm_04pi_ipulser_mode ---- 1
27.2 _DMC_01_set_rm_04pi_opulser_mode ---- 2
27.3 _DMC_01_set_rm_04pi_svon_polarity ---- 3
27.4 _DMC_01_set_rm_04pi_DO2 ---- 4
27.5 _DMC_01_set_rm_04pi_homing_ratio ---- 5
27.6 _DMC_01_rm_04PI_get_buffer ---- 7
27.7 _DMC_01_set_rm_04pi_md2_feedback_mode ---- 8
27.8 _DMC_01_set_rm_04pi_home_config_Tdec ---- 9
27.9 _DMC_01_set_rm_04pi_Home_counter_Delay_time ---- 10
27.10 _DMC_01_set_ref_counter ---- 11

第二十八章 四組脈波介面（模式 1）運動控制 API ---- 1
28.1 _DMC_01_rm_04pi_md1_start_move ---- 1

DELTA CONFIDENTIAL

28.2 _DMC_01_rm_04pi_md1_v_move ---- 3
28.3 _DMC_01_rm_04pi_md1_start_line2 ---- 4
28.4 _DMC_01_rm_04pi_md1_start_line3 ---- 6
28.5 _DMC_01_rm_04pi_md1_start_line4 ---- 8
28.6 _DMC_01_rm_04pi_md1_start_arc ---- 10
28.7 _DMC_01_rm_04pi_md1_start_arc2 ---- 12
28.8 _DMC_01_rm_04pi_md1_start_arc3 ---- 14
28.9 _DMC_01_rm_04pi_md1_start_heli ---- 16
28.10 _DMC_01_rm_04pi_md1_p_change ---- 18
28.11 _DMC_01_rm_04pi_md1_v_change ---- 19
28.12 _DMC_01_rm_04pi_md1_set_gear ---- 20
28.13 _DMC_01_rm_04pi_md1_set_soft_limit ---- 21
28.14 _DMC_01_rm_04pi_md1_get_soft_limit_status ---- 22
28.15 _DMC_01_rm_04pi_md1_set_sld ---- 24
28.16 _DMC_01_rm_04pi_md1_get_mc_error_code ---- 25
28.17 _DMC_01_set_rm_04pi_ref_counter ---- 1

第二十九章 四組類比輸出遠端擴充模組 API ---- 2
29.1 _DMC_01_rm_04da_set_output_value ---- 1
29.2 _DMC_01_rm_04da_get_output_value ---- 2
29.3 _DMC_01_rm_04da_get_return_code ---- 3
29.4 _DMC_01_rm_04da_set_output_range ---- 5
29.5 _DMC_01_rm_04da_set_output_enable ---- 6
29.6 _DMC_01_rm_04da_set_output_overrange ---- 7
29.7 _DMC_01_rm_04da_set_output_error_clear ---- 8
29.8 _DMC_01_rm_04da_set_output_error_handle ---- 9
29.9 _DMC_01_rm_04da_set_output_offset_value ---- 10
29.10 _DMC_01_rm_04da_get_output_offset_value ---- 12
29.11 _DMC_01_rm_04da_read_data ---- 2
29.12 _DMC_01_get_04da_Output_range ---- 3

第三十章 四組類比輸入遠端擴充模組 API ---- 4
30.1 _DMC_01_set_04ad_input_range ---- 1
30.2 _DMC_01_get_04ad_input_range ---- 2
30.3 _DMC_01_set_04ad_conversion_time ---- 3
30.4 _DMC_01_get_04ad_conversion_time ---- 4
30.5 _DMC_01_get_04ad_data ---- 5
30.6 _DMC_01_set_04ad_average_mode ---- 6
30.7 _DMC_01_get_04ad_average_mode ---- 7
30.8 _DMC_01_set_04ad_input_enable ---- 8

DELTA CONFIDENTIAL

第三十一章 Slave 資訊 API ---- 1
31.1 _DMC_01_get_devicetype ---- 2
31.2 _DMC_01_get_slave_version ---- 1
31.3 _DMC_01_get_slave_subversion ---- 2

第三十二章 監控參數 API ---- 3
32.1 _DMC_01_set_monitor ---- 4
32.2 _DMC_01_get_monitor ---- 8
32.3 _DMC_01_get_monitor_I32 ---- 9
32.4 _DMC_01_get_servo_command ---- 11
32.5 _DMC_01_get_servo_DI ---- 12
32.6 _DMC_01_get_servo_DO ---- 1

第三十三章 警報訊息 API ---- 2
33.1 _DMC_01_set_ralm ---- 1
33.2 _DMC_01_get_alm_code ---- 2
33.3 _DMC_01_master_alm_code ---- 3
33.4 _DMC_01_slave_error ---- 4
33.5 _DMC_01_set_ralm_servo_all ---- 5
33.6 _DMC_01_set_limit_auto_ralm ---- 6

第三十四章 多軸運動控制 API ---- 1
34.1 _DMC_01_multi_axes_move ---- 1
34.2 _DMC_01_liner_speed_master ---- 3
34.3 _DMC_01_start_v3_multi_axes ---- 5
34.4 _DMC_01_start_v3_msbrline_move ---- 7
34.5 _DMC_01_p_change_multi_axes ---- 1
34.6 _DMC_01_Start_Multi_Arc ---- 3

第三十五章 緩衝區操作 API ---- 7
35.1 _DMC_01_set_trigger_buf_function ---- 8

第三十六章 Interrupt API ---- 1
36.1 _DMC_01_set_int_factor ---- 2
36.2 _DMC_01_int_enable ---- 3
36.3 _DMC_01_int_disable ---- 4
36.4 _DMC_01_get_int_count ---- 5
36.5 _DMC_01_get_int_status ---- 6
36.6 _DMC_01_link_interrupt ---- 7
36.7 _DMC_01_set_dmc_cycle_int_count ---- 8

DELTA CONFIDENTIAL

36.8 _DMC_01_set_dio_input_trigger_mode ---- 1
36.9 _DMC_01_set_dio_input_int_enable ---- 2
36.10 _DMC_01_get_dio_input_int_counter ---- 3
36.11 _DMC_01_set_dio_input_int_counter_clear ---- 4
36.12 _DMC_01_get_dio_input_int_status ---- 5

第三十七章 Security API ---- 6
37.1 _DMC_01_read_security ---- 7
37.2 _DMC_01_read_security_status ---- 8
37.3 _DMC_01_write_security ---- 9
37.4 _DMC_01_write_security_status ---- 10
37.5 _DMC_01_check_userpassword ---- 11
37.6 _DMC_01_write_userpassword ---- 12
37.7 _DMC_01_check_verifykey ---- 13
37.8 _DMC_01_write_verifykey ---- 14
37.9 _DMC_01_read_serialno ---- 15
37.10 _misc_slave_check_userpassword ---- 16
37.11 _misc_slave_write_userpassword ---- 17
37.12 _misc_slave_get_serialno ---- 18
37.13 _misc_security ---- 19
37.14 _misc_slave_write_verifykey ---- 20
37.15 _misc_slave_check_verifykey ---- 21
37.16 _misc_slave_user_data_buffer_read ---- 22
37.17 _misc_slave_user_data_buffer_write ---- 23
37.18 _misc_slave_user_data_to_flash ---- 24

第三十八章 極限反向 API ---- 1
38.1 _DMC_01_rm_04pi_set_MEL_polarity ---- 2
38.2 _DMC_01_rm_04pi_get_MEL_polarity ---- 3
38.3 _DMC_01_rm_04pi_set_PEL_polarity ---- 4
38.4 _DMC_01_rm_04pi_get_PEL_polarity ---- 5
38.5 _DMC_01_set_01ph_alm_polarity ---- 6
38.6 _DMC_01_set_01ph_org_polarity ---- 7

第三十九章 B01 Compare API ---- 8
39.1 _DMC_01_set_compare_channel_position ---- 10
39.2 _DMC_01_get_compare_channel_position ---- 11
39.3 _DMC_01_set_compare_ipulser_mode ---- 12
39.4 _DMC_01_set_compare_channel_direction ---- 13
39.5 _DMC_01_set_compare_channel_trigger_time ---- 14
39.6 _DMC_01_set_compare_channel_one_shot ---- 15

DELTA CONFIDENTIAL

39.7 _DMC_01_set_compare_channel_source ---- 16
39.8 _DMC_01_channel0_position_cmp ---- 17
39.9 _DMC_01_channel1_output_enable ---- 18
39.10 _DMC_01_channel1_output_mode ---- 19
39.11 _DMC_01_channel1_get_io_status ---- 21
39.12 _DMC_01_channel1_set_gpio_out ---- 22
39.13 _DMC_01_channel1_position_compare_table ---- 23
39.14 _DMC_01_channel1_position_compare_table_level ---- 24
39.15 _DMC_01_channel1_position_compare_table_cnt ---- 26
39.16 _DMC_01_set_compare_channel_polarity ---- 27
39.17 _DMC_01_channel0_position_cmp_by_gpio ---- 28
39.18 _DMC_01_channel1_position_re_compare_table ---- 29
39.19 _DMC_01_channel1_position_re_compare_table_level ---- 30
39.20 _DMC_01_set_compare_channel_enable ---- 31
39.21 _DMC_01_channel1_get_fifo_status ---- 32
39.22 _DMC_01_channel1_get_fifo_counter ---- 33

第四十章 連續速度 API ---- 1
40.1 _DMC_01_speed_continue ---- 2
40.2 _DMC_01_speed_continue_mode ---- 3
40.3 _DMC_01_speed_continue_combine_ratio ---- 5

第四十一章 特殊運動控制 API ---- 1
41.1 _DMC_01_enable_electcam ---- 3
41.2 _DMC_01_set_cam_enable ---- 5
41.3 _DMC_01_set_cam_pitch ---- 6
41.4 _DMC_01_set_cam_org ---- 7
41.5 _DMC_01_set_cam_mode ---- 8
41.6 _DMC_01_set_cam_table ---- 9
41.7 _DMC_01_set_cam_table2 ---- 11
41.8 _DMC_01_set_backlash ---- 13
41.9 _DMC_01_set_backlash_info ---- 14
41.10 _DMC_01_set_ecam_repeat ---- 1

第四十二章 軸卡特殊應用 API ---- 1
42.1 _misc_app_get_circle_endpoint ---- 4
42.2 _misc_app_get_circle_center_point ---- 5
42.3 _misc_set_record_debuging ---- 6
42.4 _misc_open_record_debuging_file ---- 7
42.5 _DMC_01_enable_dda_mode ---- 8
42.6 _DMC_01_set_dda_data ---- 9

DELTA CONFIDENTIAL

42.7 _DMC_01_get_dda_cnt ---------- 10
42.8 _DMC_01_clear_dda_data ---------- 11
42.9 _DMC_01_enable_dda_mode_group ---------- 12
42.10 _DMC_01_set_dda_data_group ---------- 13
42.11 _DMC_01_get_dda_cnt_group ---------- 14
42.12 _DMC_01_clear_dda_data_group ---------- 15
42.13 _DMC_01_save_device_info ---------- 16
42.14 _DMC_01_compare_device_info ---------- 17
42.15 _DMC_01_enable_record_mode ---------- 18
42.16 _DMC_01_get_record_data ---------- 19
42.17 _DMC_01_get_record_cnt ---------- 20
42.18 _DMC_01_set_mc_fifo_mode ---------- 21
42.19 _DMC_01_set_function_time ---------- 22
42.20 _DMC_01_get_fifo_cnt ---------- 23
42.21 _DMC_01_set_gear ---------- 24
42.22 _DMC_01_get_gear ---------- 25
42.23 _DMC_01_set_position_ref_source ---------- 26
42.24 _DMC_01_get_position_ref_source ---------- 27
42.25 _DMC_01_set_virtual_command ---------- 28
42.26 _DMC_01_get_virtual_command ---------- 29
42.27 _DMC_01_set_virtual_position ---------- 30
42.28 _DMC_01_disable_virtual_pos ---------- 31
42.29 _DMC_01_set_stepping_move_rate ---------- 32
42.30 _DMC_01_MRAM_read_word ---------- 33
42.31 _DMC_01_MRAM_write_word ---------- 34
42.32 _DMC_01_MRAM_read_dword ---------- 35
42.33 _DMC_01_MRAM_write_dword ---------- 36
42.34 _DMC_01_get_watchdog_cnt ---------- 37
42.35 _DMC_01_set_watchdog_cnt ---------- 38
42.36 _DMC_01_get_watchdog_value ---------- 39
42.37 _DMC_01_set_watchdog_value ---------- 40
42.38 _DMC_01_Security_Check_UserPassword ---------- 1
42.39 _DMC_01_Security_Get_Check_UserPassword_State ---------- 2
42.40 _DMC_01_Security_Write_UserPassword ---------- 4
42.41 _DMC_01_Security_Get_Write_UserPassword_State ---------- 5
42.42 _DMC_01_Security_Check_Verifykey ---------- 6
42.43 _DMC_01_Security_Get_Check_Verifykey_State ---------- 7
42.44 _DMC_01_Security_Write_Verifykey ---------- 8
42.45 _DMC_01_Security_Get_Write_Verifykey_State ---------- 9
42.46 _DMC_01_buffer_correlation_master_slave ---------- 10
42.47 _DMC_01_buffer_correlation_status ---------- 12
42.48 _DMC_01_GPIO_Set_Type ---------- 14

DELTA CONFIDENTIAL

42.49 _DMC_01_Get_GPIO_Input_Value---------- 15
42.50 _DMC_01_Get_GPIO_Output_Value ---------- 16
42.51 _DMC_01_Set_GPIO_Output_Value---------- 17
42.52 _DMC_01_Set_GPIO_Output_error_handle ---------- 18
42.53 _DMC_01_Get_GPIO_Output_error_handle---------- 19
42.54 _DMC_01_get_mp1_dio_input---------- 20
42.55 _DMC_01_get_mp1_dio_single_input ---------- 21

第四十三章 B02 Compare---------- 2
43.1 _DMC_B02_qep_set_mode(2D 共用) ---------- 4
43.2 _DMC_B02_qep_set_inverse(2D 共用) ---------- 5
43.3 _DMC_B02_qep_get_reload_counter(2D 共用) ---------- 6
43.4 _DMC_B02_qep_set_reload_counter(2D 共用) ---------- 7
43.5 _DMC_B02_qep_get_counter(2D 共用)---------- 8
43.6 _DMC_B02_gpio_software_control_status(2D 共用) ---------- 9
43.7 _DMC_B02_gpio_software_control_setting(2D 共用) ---------- 10
43.8 _DMC_B02_ipcmp_initial ---------- 11
43.9 _DMC_ B02_ipcmp_oneshot_time---------- 14
43.10 _DMC_ B02_ipcmp_software_trigger ---------- 16
43.11 _DMC_ B02_ipcmp_compare_position ---------- 17
43.12 _DMC_ B02_ipcmp_compare_trigger_count ---------- 20
43.13 _DMC_ B02_ipcmp_compare_trigger_count_clear ---------- 21
43.14 _DMC_B02_tpcmp_initial---------- 22
43.15 _DMC_B02_tpcmp_oneshot_time---------- 25
43.16 _DMC_B02_tpcmp_software_trigger---------- 27
43.17 _DMC_B02_tpcmp_software_trigger_level ---------- 28
43.18 _DMC_B02_tpcmp_compare_position_table_reset---------- 29
43.19 _DMC_B02_tpcmp_compare_position_table---------- 30
43.20 _DMC_B02_tpcmp_compare_position_table_level ---------- 34
43.21 _DMC_B02_tpcmp_compare_trigger_count(2D 共用)---------- 39
43.22 _DMC_B02_tpcmp_compare_trigger_count_clear(2D 共用) ---------- 40
43.23 _DMC_B02_ext_io_get_input_value---------- 41
43.24 _DMC_B02_ext_io_get_output_value---------- 42
43.25 _DMC_B02_ext_io_set_output_value---------- 43
43.26 _DMC_B02_tpcmp_initial_2D---------- 44
43.27 _DMC_B02_tpcmp_oneshot_time_2D ---------- 46
43.28 _DMC_B02_tpcmp_software_trigger_2D ---------- 47
43.29 _DMC_B02_tpcmp_software_trigger_level_2D---------- 48
43.30 _DMC_B02_tpcmp_compare_position_table_2D ---------- 49
43.31 _DMC_B02_tpcmp_compare_position_table_reset_2D ---------- 50
43.32 _DMC_B02_tpcmp_compare_position_table_level_2D---------- 51
43.33 _DMC_B02_tpcmp_compare_axis_dist_2D---------- 52

DELTA CONFIDENTIAL

43.34 _DMC_B02_get_tpcmp_compare_axis_dist_2D ---- 53

第四十四章 TableMotion ---- 2
44.1 _DMC_01_start_tablemotion ---- 4
44.2 _DMC_01_start_tablemotion2 ---- 10
44.3 _DMC_01_start_tablemotion3 ---- 13
44.4 _DMC_01_start_tablemotion_cyclemode ---- 20
44.5 _DMC_01_start_tablemotion_buffer ---- 21
44.6 _DMC_01_start_tablemotion_buffer2 ---- 24
44.7 _DMC_01_start_tablemotion_buffer3 ---- 28
44.8 _DMC_01_get_tablemotion_length ---- 32
44.9 _DMC_01_tablemotion_feedrate_overwrite ---- 33
44.10 _DMC_01_tablemotion_feedhold ---- 35
44.12 _DMC_01_set_tablemotion_blending_ratio ---- 36
44.13 _DMC_01_start_tablemotion_profile_planning_para ---- 38
44.14 _DMC_01_start_tablemotion_profile_planning ---- 40
44.15 _DMC_01_start_tablemotion_profile_planning2 ---- 48
44.16 _DMC_01_start_tablemotion_profilemode ---- 51
44.17 _DMC_01_set_axis_normalamf_enable ---- 52
44.18 _DMC_01_set_normalamf_enable ---- 53
44.19 _DMC_01_set_tablemotion_ioctl_mapping ---- 54
44.20 _DMC_01_get_tablemotion_current_speed ---- 56
44.21 _DMC_01_Execute_Single_Step ---- 57
44.22 _DMC_01_Get_TableMotion_Now_MotionNo ---- 58
44.23 _DMC_01_TableMotion_Single_Step ---- 59
44.24 _DMC_01_start_tableMotion_profileMode_data_flying_photography ---- 60

第四十五章 PVT 模式 ---- 63
45.1 _DMC_01_Start_PVTComplete ---- 64
45.2 _DMC_01_Start_PVT ---- 66
45.3 _DMC_01_get_PVT_cnt ---- 68

第四十六章 B02 Latch ---- 1
46.1 _DMC_B02_ext_io_latch_initial ---- 2
46.2 _DMC_B02_ext_io_latch_function ---- 5
46.3 _DMC_B02_ext_io_latch_enable ---- 7
46.4 _DMC_B02_latch_fifo_reset ---- 8
46.5 _DMC_B02_latch_fifo_status ---- 9
46.6 _DMC_B02_latch_fifo_size ---- 10
46.7 _DMC_B02_latch_fifo_get_qep_counter ---- 11
46.8 _DMC_B02_latch_fifo_get_qep_counter2 ---- 13

DELTA CONFIDENTIAL

46.9 _DMC_B02_latch_fifo_dsp_get_data_counter ---------- 15
46.10 _DMC_B02_latch_fifo_dsp_data_counter_reset ---------- 16
46.11 _DMC_B02_latch_fifo_dsp_data_clear ---------- 17

第四十七章 Motion Buffer ---------- 1
47.1 _DMC_01_MotionBuf_Set_Crd_Parameters ---------- 8
47.2 _DMC_01_MotionBuf_Get_Crd_Parameters ---------- 11
47.3 _DMC_01_MotionBuf_Line_MultiAxis ---------- 13
47.4 _DMC_01_MotionBuf_Line_MultiAxis_G00 ---------- 15
47.5 _DMC_01_MotionBuf_Arc_Radius ---------- 17
47.6 _DMC_01_MotionBuf_Arc_CenterPos ---------- 20
47.7 _DMC_01_MotionBuf_Set_LookAhead_Parameters ---------- 23
47.8 _DMC_01_MotionBuf_Start ---------- 27
47.9 _DMC_01_MotionBuf_Stop ---------- 29
47.10 _DMC_01_MotionBuf_Get_Position ---------- 31
47.11 _DMC_01_MotionBuf_Get_Velocity ---------- 33
47.12 _DMC_01_MotionBuf_Feedrate_Override ---------- 35
47.13 _DMC_01_MotionBuf_Get_Motion_Status ---------- 37
47.14 _DMC_01_MotionBuf_Set_Stop_Deceleration ---------- 39
47.15 _DMC_01_MotionBuf_Get_Stop_Deceleration ---------- 41
47.16 _DMC_01_MotionBuf_SetUserSegNum ---------- 43
47.17 _DMC_01_MotionBuf_GetUserSegNum ---------- 45
47.18 _DMC_01_MotionBuf_Clear_Buffer ---------- 47
47.19 _DMC_01_MotionBuf_Single_Step ---------- 49
47.20 _DMC_01_MotionBuf_BufGear ---------- 51
47.21 _DMC_01_MotionBuf_BufMove ---------- 53
47.22 _DMC_01_MotionBuf_IOControl ---------- 57
47.23 _DMC_01_MotionBuf_Advanced_IOControl ---------- 60
47.24 _DMC_01_MotionBuf_Delay ---------- 63
47.25 _DMC_01_MotionBuf_Get_Remain_Buffer_Size ---------- 65
47.26 _DMC_01_MotionBuf_SoftLimit_On ---------- 67
47.27 _DMC_01_MotionBuf_SoftLimit_Off ---------- 69
47.28 _DMC_01_MotionBuf_Clear_Crd ---------- 71
47.29 _DMC_01_MotionBuf_Disable ---------- 73
47.30 _DMC_01_MotionBuf_Helix ---------- 75
47.31 _DMC_01_MotionBuf_Line_MultiAxis2 ---------- 78
47.32 _DMC_01_MotionBuf_Line_MultiAxis2_G00 ---------- 81
47.33 _DMC_01_MotionBuf_Arc_MidPos ---------- 83
47.34 _DMC_01_MotionBuf_MultiAxis_Arc1 ---------- 86
47.35 _DMC_01_MotionBuf_MultiAxis_Arc2 ---------- 89
47.36 _DMC_01_MotionBuf_MultiAxis_Arc3 ---------- 92
47.37 _DMC_01_MotionBuf_MultiAxis_Arc4 ---------- 95

DELTA CONFIDENTIAL

47.38 _DMC_01_MotionBuf_Arc_follow ---------- 98
47.39 _DMC_01_MotionBuf_Set_G00_Check----------101
47.40 _DMC_01_MotionBuf_Get_G00_Check ----------103
47.41 _DMC_01_MotionBuf_Get_G00_Check_Flag----------105
47.42 _DMC_01_MotionBuf_Get_G00_Check_match----------107

DELTA CONFIDENTIAL

第一章 API 函式庫介紹

PCI-DMC-A01/B01/F01 提供可操作的函式庫與動態連結(DLL). 您可以藉由呼叫這些函式庫來完成您所需完成的各項功能。以下各小節將告訴您如何將這些函式庫導入您的開發環境。

1.1 函式庫的使用

當您將安裝程式安裝完成，您能在「lib」的資料夾中找到 4 個函式庫，該函式庫分別是給 Visual Studio C 與 Borland 開發環境下使用。

表 1.1

PCI_DMC_01.lib	Visual Studio C 函式庫 (A01/B01/A02/B02 專用)
BCBPCI_DMC_01.lib	Borland C 函式庫(A01/B01 專用)
PCI_DMC.lib	Visual Studio C 函式庫(所有卡通用)
BCBPCI_DMC.lib	Borland C 函式庫(所有卡通用)

1.2 開始編輯新的專案

1.2.1 使用 VC

1. 在您的專案中加入以下指令:
   # include "..\inc\VC\PCI_DMC.h"
   # include "..\inc\VC\PCI_DMC_Err.h"
2. 在 Visual C 開發環境下選擇 Project / Setting / Link
   在 Object / Library modules 中鍵入"..\lib\PCI_DMC.lib"
3. 設置完成，你可以開始使用 API 來操作 PCI-DMC 軸卡。

DELTA CONFIDENTIAL

1.2.2 使用 Borland C

1. 在您的專案中加入以下指令:

   # include “..\inc\BCB \PCI_DMC.h”

   # include “..\inc\BCB\PCI_DMC_Err.h”

2. 在 Borland C++ Build 的開發環境下選擇 View/ Project Manger

   並在新專案中加入“..\lib\BCBPCI_DMC.lib”此函式庫在您的新專案中。

3. 設置完成，你可以開始使用 API 來操作 PCI-DMC 軸卡。

1.2.3 使用 VB

在安裝目錄“..\inc\VB”中，您可將“PCI_DMC.bas”與 “PCI_DMC_Err.bas”兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.4 使用 Delphi

在安裝目錄“..\ inc\Delphi” 中，您可將“PCI_DMC.pas” 加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.5 使用 VB.Net

在安裝目錄“..\inc\VB.Net” 中，您可將“PCI_DMC.vb”與 “PCI_DMC_ERR.vb”兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.6 使用 C#

在安裝目錄“..\ inc\C#” 中，您可將“PCI_DMC.cs”與 “PCI_DMC_ERR.cs”兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

DELTA CONFIDENTIAL

第二章　命令傳回值與訊息描述

2.1　錯誤傳回值一覽

當您在使用 PCI-DMC 的 API 函式庫時，API 函式多半會回傳一個錯誤代碼，如表 2.1。

若該 API 函式傳回的值為 0 時，這表示此 API 函式是成功地被執行；反之，若此 API 函式回傳其它的錯誤代碼，這表示可能在操作程序或者是硬體連結方面產生問題。請依該錯誤代碼的描述進行問題排解。

表 2.1

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_A/B01 錯誤代碼表		
0	ERR_NoError	API 成功運作
3	ERR_CardNoError	卡號錯誤請確認卡片上的 DIP Switch 調整的號碼
5	ERR_bootmodeErr	無法啟動 DSP 數位程序
6	ERR_downloadcode	DSP 記憶體程序讀/寫錯誤
7	ERR_downloadinit	DSP 記憶體資料讀/寫錯誤
8	ERR_PCI_boot_first	“_DMC_01_pci_initial” API 函式必須先運作
9	ERR_FeedRate_updata	FeedRate overwrite 的值相同
11	ERR_AxisNoError	軸數目設置錯誤(過大)
12	ERR_IPO_First	需要為 IPO 模式
13	ERR_Target_reach	Mode 1 運行時,需 Target 到達
14	ERR_Servo_on_first	需要先 servo on
15	ERR_MPG_Mode	在手輪模式下無法清除位置
16	ERR_PDO_TG	使用 PDO 模式下指令給模組,無法回傳收到
17	ERR_ConfigFileOpenError	創建存放除錯資訊的檔案錯誤
18	ERR_Ctrl_value	使用控制指令錯誤
19	ERR_Security_Fifo	使用 Security Fpga write Error
20	ERR_Security_Fifo_busy	使用 Security Fpga busy
21	ERR_SpeedLimitError	設置的速度大於最大速度
22	ERR_Security_Page	使用 Security page 需要小於 16
23	ERR_Slave_Security_op	使用 Security slave_operate 指令無效
24	ERR_channel_no	channel no 錯誤
25	ERR_start_ring_first	“_DMC_01_start_ring” API 函式必須先運作

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
26	ERR_NodeIDError	NodeID 不存在
27	ERR_MailBoxErr	DSP 忙碌，指令無法下達
28	ERR_SdoData	SDO 資料有送出但無回應
29	ERR_IOCTL	作業系統無法處理此 IRP
30	ERR_SdoSvonFirst	使用 SDO 操作軸控時,需先 Servo On
31	ERR_SlotIDError	Slave 模組(GA or RM)無此 Slot 號碼
32	ERR_PDO_First	使用 PDO 協定操作，必須先啟動 PDO 模式
33	ERR_Protocal_build	Protocal,電氣尚未建立
34	ERR_Maching_TimeOut	模組配對 time Out
35	ERR_Maching_NG	模組配對 NG
36	ERR_Group_Num	設定群組最大值為 6 組
37	ERR_Master_Alarm	故障發生(通訊不良,Driver Alm)
38	ERR_Alarm_reset	Reset Alarm Fail
40	ERR_Master_Security_Wr	使用 Security Master Write 指令無效
41	ERR_Master_Security_Rd	使用 Security Master Read 指令無效
42	ERR_Master_Security_Pw	需要先輸入正確的 password
50	ERR_NonSupport_CardVer	Master Card 的版本錯誤,請連繫代理商,購買正確 Master Card
51	ERR_Compare_Source	Ver Type : B Compare Source 選擇錯誤
52	ERR_Compare_Direction	Compare 的方向錯誤 dir 需為 1 或 0 1:ccw,0:cw
60	ERR_GetDLLPath	Dll 路徑錯誤
61	ERR_GetDLLVersion	擷取 Dll 版本資料發生錯誤
70	ERR_04PISTOP_Timeout	對 04PI 下達 Stop 指令逾時未成功
71	ERR_ServoSTOP_Timeout	對伺服下達 Stop 指令逾時未成功
72	ERR_04PISTOP_status	04PI Stop MC_done not to Error
73	ERR_04PIHoming_err	04PI 於回 Home 模式狀態錯誤
74	ERR_04PISdo_trans	04PI 下達 SDO 指令，但資料回覆錯誤
75	ERR_Qqp_Index	B02 - Qep 索引值錯誤
76	ERR_Gpio_Output_Size	B02 - GPIO 索引值錯誤或者超出索引範圍
77	ERR_Ipcmp_Index	B02 - ipcmp 索引值錯誤或者超出索引範圍
79	ERR_Tpcmp_Fifo_Index	B02 – tpcmp FIFO 索引值錯誤或者超出索引範圍
80	ERR_Tpcmp_Qep_Source	B02 – tpcmp qep 來源索引錯誤

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
81	ERR_Tpcmp_Table_No_Data	B02 – tpcmp 內無 Table 資料
82	ERR_Tpcmp_Table_Out_Of_Range	B02 - PositionTable 需 Compare 的值超過 255 或填寫的內容錯誤
83	ERR_Tpcmp_Table_Overflow	B02 – FIFO 內容可填寫的內容值,超過容量大小
84	ERR_Extio_Range	B02 – 指定的外部 IO 索引值錯誤或者超出索引範圍
86	ERR_LATCH_FIFO_EMPTY	B02 – 檢查 FIFO 為空值則拋出此錯誤
87	ERR_Latch_Fifo_Invalid_Data	B02 – 檢查 FIFO 內容錯誤
100	ERR_GPIO_PORTNO	GPIO 連接的阜號錯誤
101	ERR_JOG_AXISNO	JOG 軸數錯誤
102	ERR_A02_MPG_Enable	A02 手輪已致能,無法重覆致能
103	ERR_A02_JOG_Enable	A02 JOG 已致能,無法重覆致能
104	ERR_API_Parameter_Range	參數超過範圍
105	ERR_MPG_Enable	手輪已致能 ,無法重覆致能
106	ERR_API_Parameter_Point	指標型態參數不可為空

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
112	ERR_RangeError	設定軸的號碼錯誤
114	ERR_MotionBusy	Motion 的命令重疊
116	ERR_SpeedError	設置最大速度為 0
117	ERR_AccTimeError	加/減速時大於 1000 秒
124	ERR_PitchZero	操作螺旋位移，參數「pitch」的值為 0
127	ERR_BufferFull	Motion 命令緩衝區已滿
128	ERR_PathError	Motion 命令錯誤
130	ERR_NoSupportMode	不支持速度變化
132	ERR_FeedHold_support	Feedhold Stop 啟動，無法接收新的命令
133	ERR_SDStop_On	執行減速停止命令，無法接收新的命令
134	ERR_VelChange_supper	無法執行速度變化功能(Feddhold,同動與減速)
135	ERR_Command_set	無法連續執行 FeedHold 的功能
136	ERR_sdo_message_choke	Sdo 指令回傳有誤，請檢查網路線接線是否接妥
137	ERR_VelChange_buff_feed	Feedhold 功能須先啟動，無法速度變化

	hold	
138	ERR_VelChange_sync_ move	正在等待同動命令，無法速度變化
139	ERR_VelChange_SD_On	正在等待減速命令，無法速度變化
140	ERR_P_Change_Mode	單軸點對點模式，加速段,速度等於 0,非單軸點對點模式
141	ERR_BufferLength	當模式在 _Path_p_change,_Path_velocity_change_onfly, _Path_Start_Move_2seg 時 Buffer Length 需要為 0
142	ERR_2segMove_Dist	距離需要同向
143	ERR_CenterMatch	經過反向運算圓心要一致
144	ERR_EndMatch	經過反向運算圓心要一致
145	ERR_AngleCalcu	經過計算角度錯誤
146	ERR_RedCalcu	經過半徑錯誤
147	ERR_GearSetting	Gear 的分子或分母為 0
148	ERR_CamTable	Table Setting First Arrary Point Error 設定 table 不能到負值 table[-1]沒有這種設定
149	ERR_AxesNum	多軸設定值必須要兩軸以上
150	ERR_SpiralPos	最終位置會到達螺旋圓心
151	ERR_SpeedMode_Slave	在速度連續時使用的 Slave 軸,無法執行 Motion 指令
152	ERR_SpeedMode_SlaveSet	設定軸的虛擬必須在軸的前半部
154	ERR_Backlash_step	accstep+contstep+decstep 需小於 100
155	ERR_Backlash_status	設定時 motion done 及 buffer length 必須都為 0
156	ERR_DistOver	輸入 Dist 超過 TotalDist
160	ERR_PVTDataError	PVT motion 輸入的 Data 有誤
161	ERR_PVTModeError	不支援 PVT 模式
162	ERR_PVTChannel	channel 必需小於支援 PVT 的軸數
163	ERR_TM_DATA_Axis	Table Motion 資料軸數超過最大值
164	ERR_TM_CYCLE_DATA	Table Motion 循環次數不等於 1 時，起始點和末點不同
165	ERR_TM_DATA_SIZE	資料個數超過上限
166	ERR_AMFNum	濾波超過 2 道或小於 1 道
167	ERR_TM_INACTIVE	Table Motion 啟動中
168	ERR_TM_BUFFER_FULL	此次新增資料會超過 Buffer 大小
190	ERR_TP_CIRCLE_DATA	3 點走圓，資料輸入錯誤(三點共線)
193	ERR_AMF_ENABLE	AMF 已啟動
194	ERR_AMF_Time	濾波時間超出範圍
195	ERR_SPEED_RATIO	TableMotion 預改變的速度百分比太大
196	ERR_MOTION_DONE	已 MotionDone，不能在加入資料

197	ERR_TM_SD_STOP	TableMotion SD STOP 中

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
198	ERR_TM_FEEDRATE_OVERWRITE	TableMotion FeedRateOverwrite 中
199	ERR_IO_NUMBER	超過最大 IO 數量
200	ERR_PROFILE_MODE_ENABLE	TableMotion 速度規劃模式已啟動
201	ERR_NOT_TM_CYCLE_MODE	不是 TableMotion Cycle
202	ERR_NONE_ANALYSIS	Profilemode 尚未分析
203	ERR_OVER_TABLE_NUM	超過最 Table 數目
204	ERR_ACCDEC_LIMIT	加速度限制為 0
205	ERR_SOFTLIMIT	軟體極限負的位置大於正的位置
206	ERR_TM_ARCBIT	TableMotion ArcBit 錯誤(超過三軸),ProfileMOde(超過二軸)
207	ERR_TM_COLLINEAR	TableMotion ArcBit 標記的軸三點共線
208	ERR_TM_ARC_ANOTHER_AXIS_POS	TableMotion ArcBit 標記的其他軸座標不一樣
209	ERR_TM_PROFILE_PARAMETERS	TableMotion ProfileMode 參數設置錯誤。開啟曲線偵測時，ArcRefSpeed
210	ERR_TM_PROFILE_RINGBUFFER_STATUS	TableMotion ProfileMode 尚未重新造表
234	ERR_MB_FIFO_FULL	Motion Buffer FIFO 已滿
235	ERR_MB_ITP_START	Motion Buffer 運動已啟動
236	ERR_FIFO_0_START	Motion Buffer FIFO 0 已啟動
237	ERR_GEAR_AXISNO	Motion Buffer BufGear 軸號和坐標系的軸號衝突
238	ERR_GEAR_SYNCHRONIZE_NUM	Motion Buffer BufGear 同步運動已達上限
239	ERR_MB_IO_ADVANCE	Motion Buffer IO 提前輸出時間超過坐標系設置時間
240	ERR_MB_NONE_INITIAL	Motion Buffer 坐標系尚未建立
241	ERR_MB_CLEAR_BUFFER	Motion Buffer 狀態在運動中，無法清除資料
242	ERR_MB_ARC_PARAMETER	Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
243	ERR_MB_SETCOMMAND	Motion Buffer 狀態下無法使用 set command
244	ERR_MB_GEAR_CONTROL	Motion Buffer BufGear 後面不可接緩存區指令

245	ERR_MB_DISABLE_COMMAND	Motion Buffer 已使用 Disable command, 後續指令無作用
601	Compare_Cards_Not_Equal	比對結果ㄧ軸卡資訊(卡號、數量)不符
602	Compare_Nodes_Not_Equal	比對結果ㄧ站號資訊(數量)不符
603	Compare_Node_ID_Not_Equal	比對結果ㄧ站號資訊(站號)不符
604	Compare_Node_Device_Type_Not_Equal	比對結果ㄧ模組資訊(模組第二分類)不符
605	Compare_Node_Identity_Object_Not_Equal	比對結果ㄧ模組資訊(模組第一分類)不符
606	Compare_File_Path_NULL	檔案路徑錯誤
607	Compare_File_Open_Fail	檔案開啟失敗(請確定路徑輸入正確)
608	Compare_File_Not_Exist	檔案不存在

錯誤傳回代碼(十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_F01 錯誤代碼表		
0	ERR_NoError	API 成功運作
301	ERR_NoCardFound	無此卡號或尚未 Initial
302	ERR_CardInitial	卡片 Initial 失敗
303	ERR_MemoryAccess_Failed	記憶體讀寫失敗
304	ERR_MemoryOutOfRange	記憶體使用超過 Range
305	ERR_UartTxIsBusy	Uart Tx Is Busy
306	ERR_UartRxError	Uart Rx 讀取錯誤
307	ERR_UartRxIsNotReady	Uart Rx 尚未準備完成
308	ERR_NotSupportFunc	不支援此 Function
309	ERR_NoNodeFound	Slave 站號設置錯誤
310	ERR_APIInputError	API 參數輸入錯誤(超出限定值)
311	ERR_SDOFailed	SDO 傳送失敗
312	ERR_SDOBusy	SDO 忙碌中或 SDO 同時有兩個指令被寫入
313	ERR_APITypeErr	此模組不支援此 API
314	ERR_ScaleFailed	AD 校正失敗
315	ERR_F_BufferFull	MailBox_Buffer 已滿
316	ERR_ConnectErr	通訊連線異常
317	ERR_MBWordChFailed	MailBox 同時有兩個指令寫入
318	ERR_MailBoxFailed	MailBox 傳送失敗

319	ERR_CantResetCard	無法 ResetCard
320	ERR_PDOFailed	PDO 未回應

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
321	ERR_MBCmding	MailBox 正在處理指令
322	ERR_SVOFF	此動作需 Servo On 方可執行
323	ERR_DriverError	此動作因 DriverErr 無法執行，請先執行 Rest Alrm
324	ERR_ConnReset_Failed	初始化重置失敗
325	ERR_F01SlotIDError	此裝置不支援 Slot 或輸入的 Slot 編號超出範圍
326	ERR_UartData_NoMatch	Download Code 讀取 Uart 的資料時，回傳無法符合正確值
327	ERR_SVON	此動作需先將伺服 Servo Off 方可執行
328	ERR_Mpg_Already_On	此軸已經致能手輪、Jog 或 DDA 功能，其餘功能暫時無法使用
329	ERR_MpgNumber_Over	手輪或 Jog 致能數量已達上限(最大 3 組)
330	ERR_Mpg_Data_Failed	手輪或 Jog 資料傳遞失敗
331	ERR_DDA_Buffer_Full	DDA Buffer 已滿
332	ERR_F_Slave_Security_op	Slave Secutiry 寫入失敗
333	ERR_F_Security_Page	Page 超過設置
334	ERR_F_GetDLLPath	找不到 DLL 路徑
335	ERR_F_GetDLLVersion	找不到 DLL 版本訊息
336	F_Compare_File_Open_Fail	檔案開啟失敗(請確定路徑輸入正確)
337	F_Compare_File_Not_Exist	檔案不存在
338	F_Compare_Cards_Not_ Equal	比對結果─軸卡資訊(卡號、數量)不符
339	F_Compare_File_Path_NULL	檔案路徑錯誤
340	F_Compare_Node_ID_Not_ Equal	比對結果─站號資訊(站號)不符
341	F_Compare_Node_Device_ Type_Not_Equal	比對結果─模組資訊(模組第二分類)不符
342	F_Compare_Node_Identity_ Object_Not_Equal	比對結果─模組資訊(模組第一分類)不符
343	F_Compare_Nodes_Not_ Equal	比對結果─站號資訊(數量)不符
344	ERR_SecurityNoRet	Security 傳送結果未回傳
345	ERR_SDORetTimeOut	SDO 回傳時間過長

346	ERR_Uart_Connect_Fail	Uart 通訊失敗
399	ERR_UseGetError	系統型錯誤, 需使用 _DMC_01_get_card_error_code 讀取錯誤碼

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_A01/B01/F01 General Error Code		
901	ERR_SameCardNumber	卡號重複(所有軸卡卡號不能相同)
902	ERR_CardType	此軸卡不支援此 API
903	ERR_CantFindDLL	此 API 與其對應的 DLL 連結失敗(沒有 DLL 或是 DLL 函式庫中沒有此函式)
904	ERR_SerchErrorCode	此 API 的錯誤需使用 _DMC_01_get_cardtype_errorcode 來取得
905	ERR_NoCardFound	使用 API 當下找不到此軸卡。
910	ERR_DLLinUse	合併版 PCI_DMC.dll 正被使用中。
911	ERR_CantFindAB01	找不到 PCI_DMC_01.dll。
912	ERR_CantFindF01	找不到 PCI_DMC_F01.dll。
913	ERR_CantFindDMCDLL	找不到 PCI_DMC.dll。

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
BMC_General Error Code(For PAC Only)		
400	ERR_NO_ERROR	裝置作用正常
401	ERR_NO_DEVICE_FOUND	找不到裝置
402	ERR_DEVICE_NOT_INITIALIZED	裝置尚未初始化
403	ERR_MEMORY_READ	記憶體讀取錯誤
404	ERR_MEMORY_WRITE	記憶體寫入錯誤
405	ERR_UART_TX_BUSY	通訊阜 TX 忙碌中
406	ERR_UART_RX_ERROR	通訊阜 RX 錯誤
407	ERR_UART_RX_NOT_READY	通訊阜 RX 尚未準備完成
408	ERR_MODEL_TYPE	模組型別錯誤
409	ERR_INVALID_PARAMETER,	非法參數設置
410	ERR_SEMAPHORE_CREATE_FAILED,	Semaphore 信號建立時發生錯誤
411	ERR_SEMAPHORE_OPEN_FAILED,	Semaphore 信號開啟時發生錯誤
412	ERR_SEMAPHORE_NOT_ENOUGH,	Semaphore 信號無法再容許其他程序存取

413	ERR_PORT_IO_READ,	PORT IO 讀取錯誤
414	ERR_PORT_IO_WRITE,	PORT IO 寫入錯誤
415	ERR_IST_INITIAL	中斷服務程序初始化錯誤
416	ERR_IST_ENABLE,	中斷服務程序啟動失敗
417	ERR_IST_EVENT_LINK,	中斷服務程序事件連結錯誤
418	ERR_MAC_SEND_FRAME_PDO	PDO 封包發送失敗
419	ERR_MAC_SEND_FRAME_SDO,	SDO 封包發送失敗
420	ERR_MAC_RECV_FRAME	接收封包失敗

DELTA CONFIDENTIAL

第三章　操作原理

3.1　軸卡初始化

3.1.1　函式列表

表 3.1

函式名稱
_DMC_01_open
_DMC_01_get_CardNo_seq
_DMC_01_reset_card
_DMC_01_close
_DMC_01_pci_initial
_DMC_01_initial_bus
_DMC_01_start_ring
_DMC_01_get_device_table
_DMC_01_get_node_table

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.1.2 流程圖

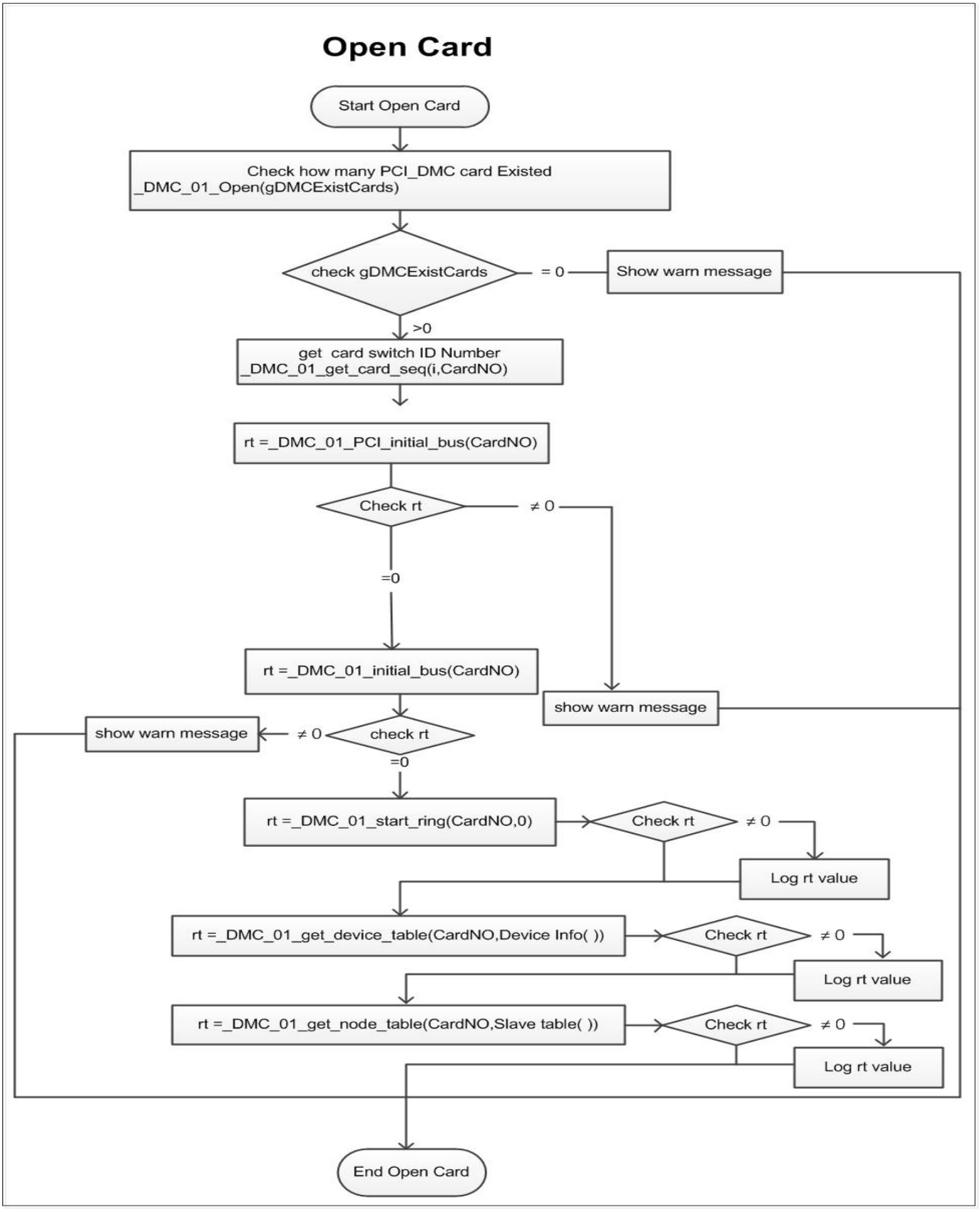


圖 3.010

3.1.3 應用範例

程式外觀

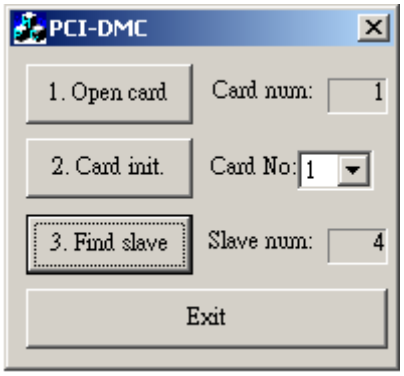


圖 3.011

1) 開啟軸卡

圖 3.012

按壓圖 3.012 中的「Open card」鍵開始執行以下程序：

```
/* gDMCExistCards 變數會被填入 PC 上 PCI-DMC-01 的數量*/
rt = _DMC_01_open(&gDMCExistCards);
```

2) 初始化軸卡

圖 3.013

按壓圖 3.013 中的「Card init」鍵開始執行以下程序：

```
for(i=0; i<gDMCExistCards; i++)
{
/*  取得 PC 上第 i 張軸卡號，卡號即 Switch 指撥開關對應的數值*/
  rt = _DMC_01_get_CardNo_seq(i, &CardNo);
  gpDMCCardNoList[i] = CardNo;
```

DELTA CONFIDENTIAL

```
/*進行軸卡初始化 */
rt = _DMC_01_pci_initial(gpDMCCardNoList[i]);   // 初始化軸卡
if(rt != 0) AfxMessageBox("Can't boot PCI_DMC Master Card!");
rt = _DMC_01_initial_bus(gpDMCCardNoList[i]); // 通訊協定初始化
  gbpDSPBoot[gpDMCCardNoList[i]] = true;
}
```

// PCI_DMC_F01 在進行 _DMC_01_initial_bus 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode

的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請原卡原卡號再做一次_DMC_01_initial_bus)。

3) 建立通訊

圖 3.014

按壓圖 3.014 中的「Find slave」鍵開始執行以下程序：

```
rt = _DMC_01_start_ring(gDMCCardNo, 0); // 開始進行通訊
rt = _DMC_01_get_device_table(gDMCCardNo,
&gpDeviceInfo[gDMCCardNo]);
rt = _DMC_01_get_node_table(gDMCCardNo,
&gpSlaveTable[0][gDMCCardNo]);
```

上述程序執行完成後，顯示找尋到的 Slave 裝置於「Slave num」欄位。

DELTA CONFIDENTIAL

NOTE

_DMC_01_get_device_table → 取得使用 PDO 協定的 Slot ID。

例如：gpDeviceInfo[gDMCCardNo]為一資料型別為 “WORD”的變數，若其值為 7，以二進制表示為” 0000 0000 0000 0111 “，即表示存在的 Slot ID 為”1”, “2”, “3” 的 Slave 裝置。

_DMC_01_get_node_table → 取得使用 SDO 定的 Node ID。

例如：gpSlaveTable[0][gDMCCardNo] 為一資料型別為 “DWORD”的變數，若其值為 7，以二進制表示為” 0000 0000 0000 0000 0000 0000 0000 0111 “，，即表示存在的 Node ID 為”1”, “2”, “3” 的 Slave 裝置。

※您可以使用下列的演算法來找出 SDO 的站號(Node ID)

```
lMask = 0x1;
for(i=0; i<32; i++)
{
/*  第 i 個 bit 值為 1 時,條件式成立.*/
if((gpSlaveTable[0][gDMCCardNo]>>i) & lMask) {
/*  取得的 i 值+1 即為站號(Node ID)，對應驅動器參數「P3-00」  */
gpNodeID[gNodeNum] = (unsigned short)(i+1);
gNodeNum++;
}
}
```

4) 離開程序

圖 3.015

按壓圖 3.015 中的「Exit」鍵開始執行以下程序：

```
for(i=0; i<gDMCExistCards; i++) {
        rt = _DMC_01_reset_card(gpDMCCardNoList[i]);     // 將軸卡重置
}
_DMC_01_close();   // 終止 PCI-DMC-01 運行
```

DELTA CONFIDENTIAL

3.2 讀取/寫入驅動器參數

3.2.1 函式列表

表 3.2

函式名稱
_DMC_01_read_servo_parameter
_DMC_01_write_servo_parameter

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.2.2 應用範例

程式外觀

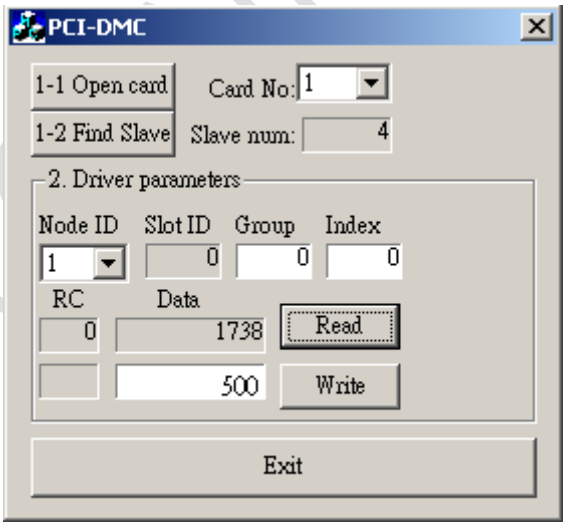


圖 3.020

1) 軸卡開啟與初始化

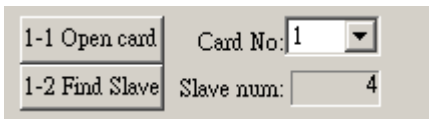


圖 3.021

按壓圖 3.021 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.021 中的「Find Slave」鍵開始搜尋串接模組。

DELTA CONFIDENTIAL

關於初始化詳細說明，請參閱 3.1.3 節「開啟軸卡」到「建立通訊」的函式運用

2) 填入欲變更的驅動器 (包含驅動器站號、參數群組號,與索引值)

圖 3.022

以例子來說，您可填入以下數值如圖 3.022.

第 1 個欄位“Node ID” ：若值填入 1，表示將操作站號為 1 的驅動器。

第 2 個欄位“Slot ID”：此欄位無法變更，它會顯示目前 Slave 裝置(驅動器的 Slot ID 值為 0)。

第 3 個欄位”Group” ：指該裝置(通常為驅動器)的群組編號，關於該編號功能詳細的說明，請參閱”ASDA-A2 使用手冊。如圖 3.022 群組值設置為“0”，表示將設置”P0-xx”的驅動器參數值(xx 的值請見下方 index 的說明)。

第 4 個欄位”Index”：如上述，此數值需與 Group 的值搭配使用。如圖 3.022，index 的值為 0，是故，此圖範例將會對 Node ID 為 1 的驅動器的”P0-00”參數（驅動器版本）進行讀取/寫入。

3) 讀取驅動器參數

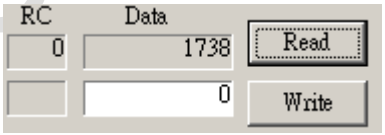


圖 3.023

按壓圖 3.023 中的「Read」鍵開始執行以下程序：

```
rt = _DMC_01_read_servo_parameter(gDMCCardNo, NodeID, SlotID, group, idx,
&data);
// 將傳回 data 值，該值即為此驅動器的驅動器參數目前設置的值。
// rt 的值將顯示在「RC」欄位，data 的值將顯示在「Data」欄位。
```

DELTA CONFIDENTIAL

4) 寫入驅動器參數

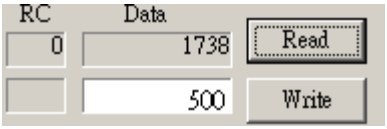


圖 3.024

如圖 3.024，若您欲在驅動器中寫入參數值，您必須在 edit box 的欄位中填入欲寫入

的值，接著按壓「Write」鍵開始執行以下程序：

```
rt = _DMC_01_write_servo_parameter(gDMCCardNo, NodeID, SlotID, group, idx, data);
// 該值會寫入您設置的驅動器群組參數中，詳細設置請參閱上述說明。
```

5) 離開程序

按壓圖 3.020 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.3 CANopen 協定的操作

3.3.1 函式列表

表 3.3

函式名稱
_DMC_01_send_message
_DMC_01_read_message
_DMC_01_get_message

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.3.2 應用範例

程式外觀

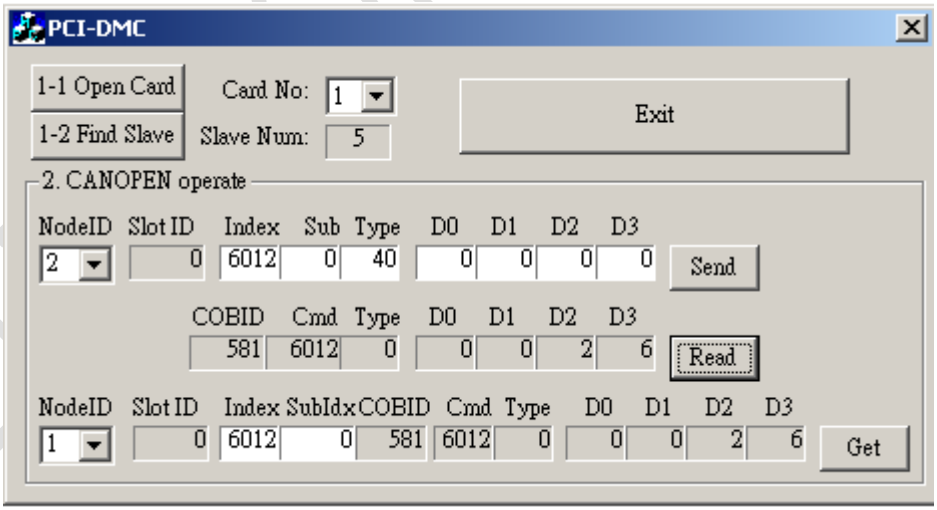


圖 3.030

1) 軸卡開啟與初始化
按壓圖 3.030 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.030 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) 送出 SDO 協定的命令

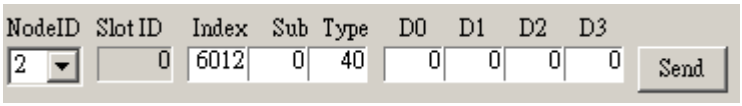


圖 3.031

您必須填入“NodeID”的值，該值會參照到您欲設置的 slave 裝置。關於“Index”、“Sub”與“Type”設定的值，請參考 CANopen 手冊(DS 402)。“D0”到”D3”的值，這些值是用來鍵入您欲設定的 SDO 命令資料(有效資料：Byte). 當您完成上述資料的填入動作，按壓圖 3.031 中的「Send」鍵即開始執行以下 API 函式：

```
rt = _DMC_01_send_message(gDMCCardNo, NodeID, SlotID, Index, SubIdx, DataType, Value0, Value1, Value2, Value3);
```

3) 讀取 SDO 協定的命令

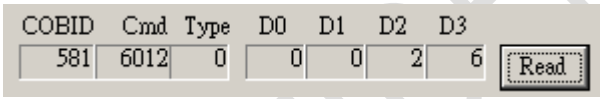


圖 3.032

按壓圖 3.032 中的「Read」鍵開始執行以下程序：

```
/* 取得您送出最近一筆 SDO 命令所傳回的資料 */
rt = _DMC_01_read_message(gDMCCardNo, & COBID, &Cmd , &DataType, &Value0, &Value1, &Value2, &Value3);
```

您可以建立一些變數來存放這些 SDO 命令回傳的資料，關於此傳回資料的詳細說明，請參閱第 8.7 節敘述。

4) 取得 SDO 協定的命令

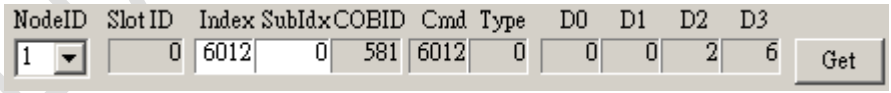


圖 3.033

您需填入對應的 Node ID、Object Dictionary(OD) index 與 sub-index，以取得您欲知道 CANopen 介面協定的相關資訊。當您完成資料的填入，按壓「Get」鍵執行：

```
Rt = _DMC_01_get_message(gDMCCardNo,NodeID,SlotID,Index,SubIndex, &COBID, &Cmd , &DataType, &Value0, &Value1, &Value2, &Value3);
```

您可以建立一些變數來存放這些 SDO 命令回傳的資料，關於此傳回資料的詳細說明，請參閱第 8.9 節敘述。

5) 離開程序

按壓圖 3.030 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個

DELTA CONFIDENTIAL

API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.4 原點復歸操作運動控制

3.4.1 概述

大部分的應用程式利用增量式編碼器(incremental encoder)來取得 position feedback。在運行任何正確地運動控制前，進行原點復歸的操作是必要的。當電源被開啟後，機台上的 position 的狀況將出現 3 種可能， 其一，position 停在一般原復歸設置的位置上等待下一個命令；其二，position 停在 ORG 感測器的位置；其三，position 停留在 ORG 與 Limit Switch(PEL 與 MEL)之間的位置。請參考下方圖 3.040。

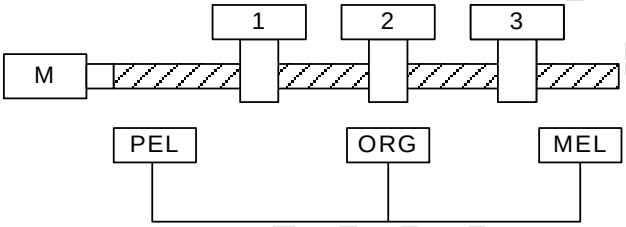


圖 3.040

PCI-DMC 針對上述條件有不同函式的支持。在一般 position 原點復歸的情況下，PCI-DMC 提供多達 35 種原點復歸模式的支持(包含預留的部分)。使用者僅需使用軟件進行相關設置，該硬件即可完成使用者指定之原點復歸的操作。當完成原點復歸的動作時，對應之命令(command)與反饋位置(feed back position) 將會清除為零值，但目標位置(target position)並不會被清除為零。下圖示意原點復歸執行條件：

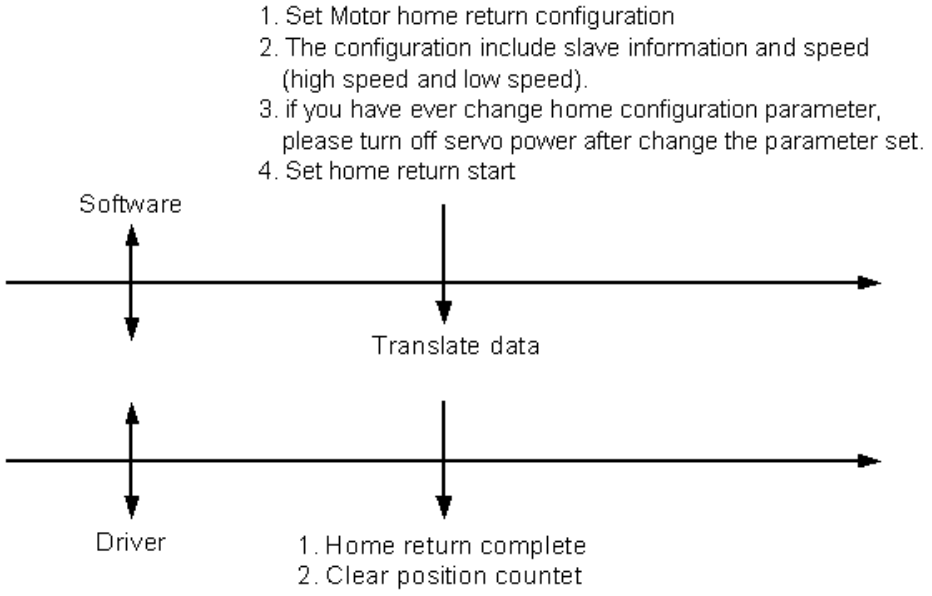


圖 3.041

3.4.2 函式列表

表 3.4

函式名稱
_DMC_01_set_home_config
_DMC_01_set_home_move
_DMC_01_escape_home_move

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.4.3 流程圖

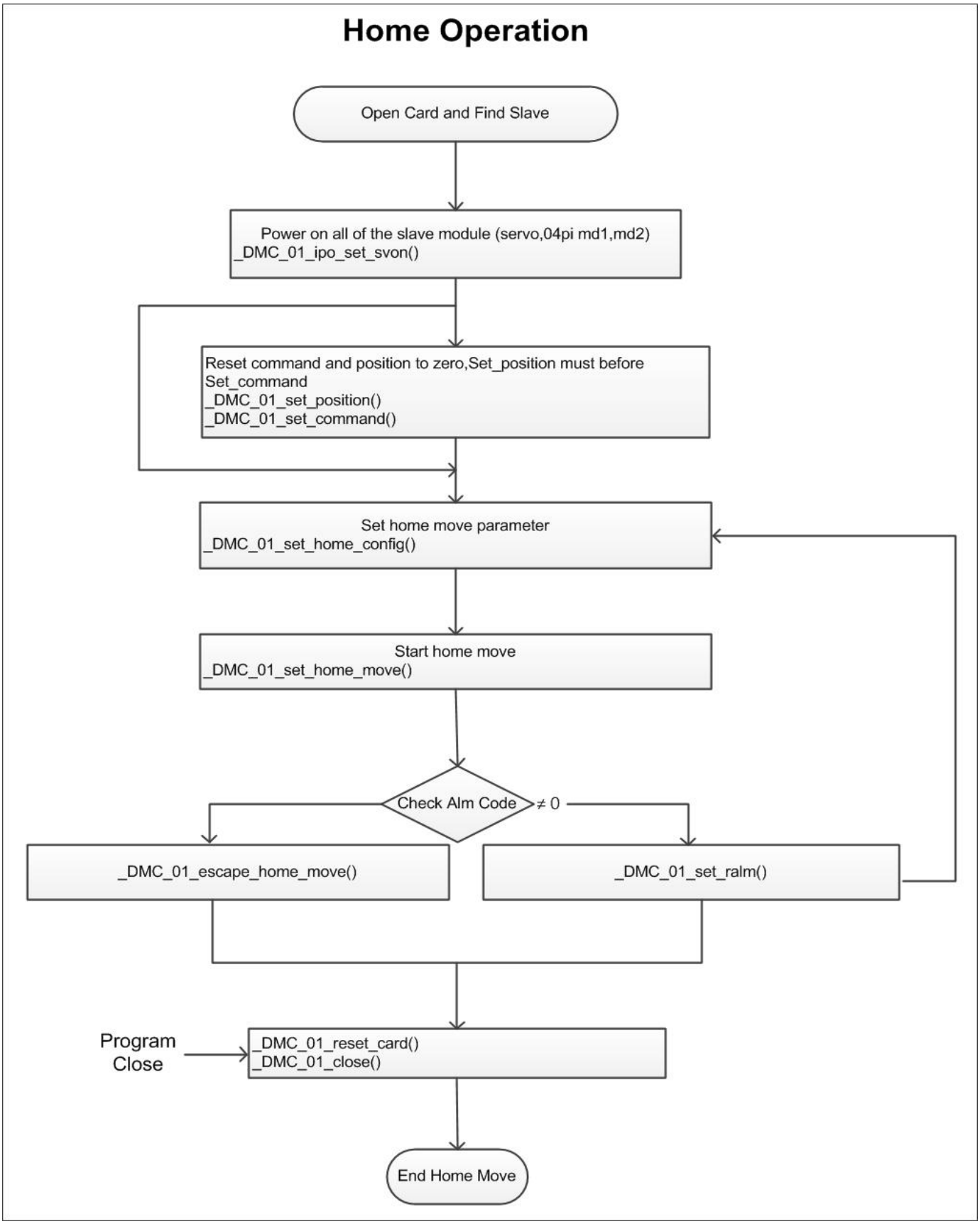


圖 3.042

3.4.4 應用範例

程式外觀

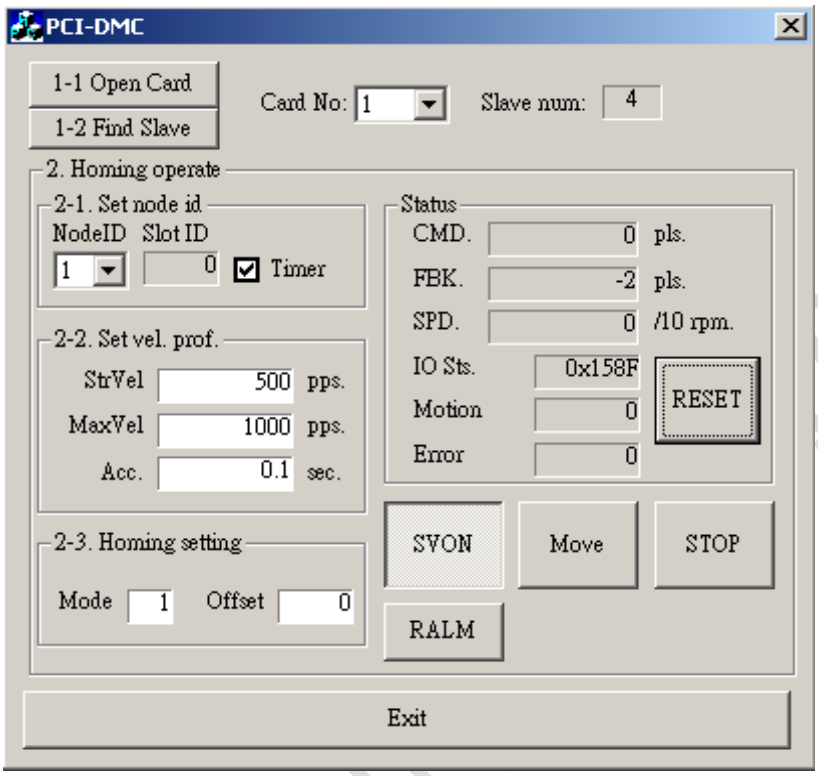


圖 3.043

1) 軸卡開啟與初始化

按壓圖 3.043 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.043 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 填入運動控制的引數內容值

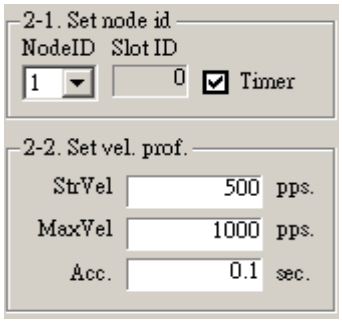


圖 3.044

NodeID 項目：API 函式的引數變數 “NodeID”。
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。
StrVel.項目：二段速，每秒送出的脈衝值。API 函式的引數變數 “StrVel”。
MaxVel.項目：一段速，每秒送出的脈衝值。API 函式的引數變數 “MaxVel”。
Acc.項目：達到最大速度所花費時間。API 函式的引數變數 “acc”。

DELTA CONFIDENTIAL

3) 設置原點復歸配置參數(原點復歸模式與偏移量)

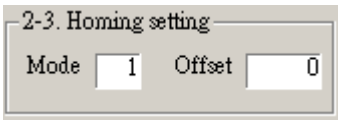


圖 3.045

Mode 項目：原點復歸模式 1 至 35。API 函式的引數變數 “home_mode”。
Offset 項目：原點復歸偏移量。API 函式的引數變數 “home_offset”。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

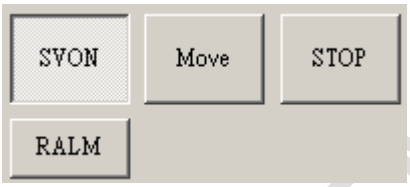


圖 3.046

按壓圖 3.046 中的「SVON」鍵開始執行以下程序：

```
rt =  _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 原點復歸操作

如圖 3.046。按壓「Move」鍵開始執行以下程序：

```
/* 設置原點復歸模式：1~35、偏移量與速度參數 */
rt = _DMC_01_set_home_config(gDMCCardNo, NodeID, SlotID, home_mode, home_offset, StrVel, MaxVel, acc);
/* 啟動原點復歸運動 */
rt = _DMC_01_set_home_move(gDMCCardNo, NodeID, SlotID);
```

6) 停止原點復歸運動

當您欲停止原點復歸運動操作，您必須按壓圖 3.046 中的「STOP」鍵執行以下程序：

```
/* 中斷原點復歸運動 */
rt = _DMC_01_escape_home_move(gDMCCardNo, NodeID, SlotID);
```

7) 離開程序

按壓圖 3.043 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.5 轉矩運動控制

3.5.1 函式列表

表 3.5

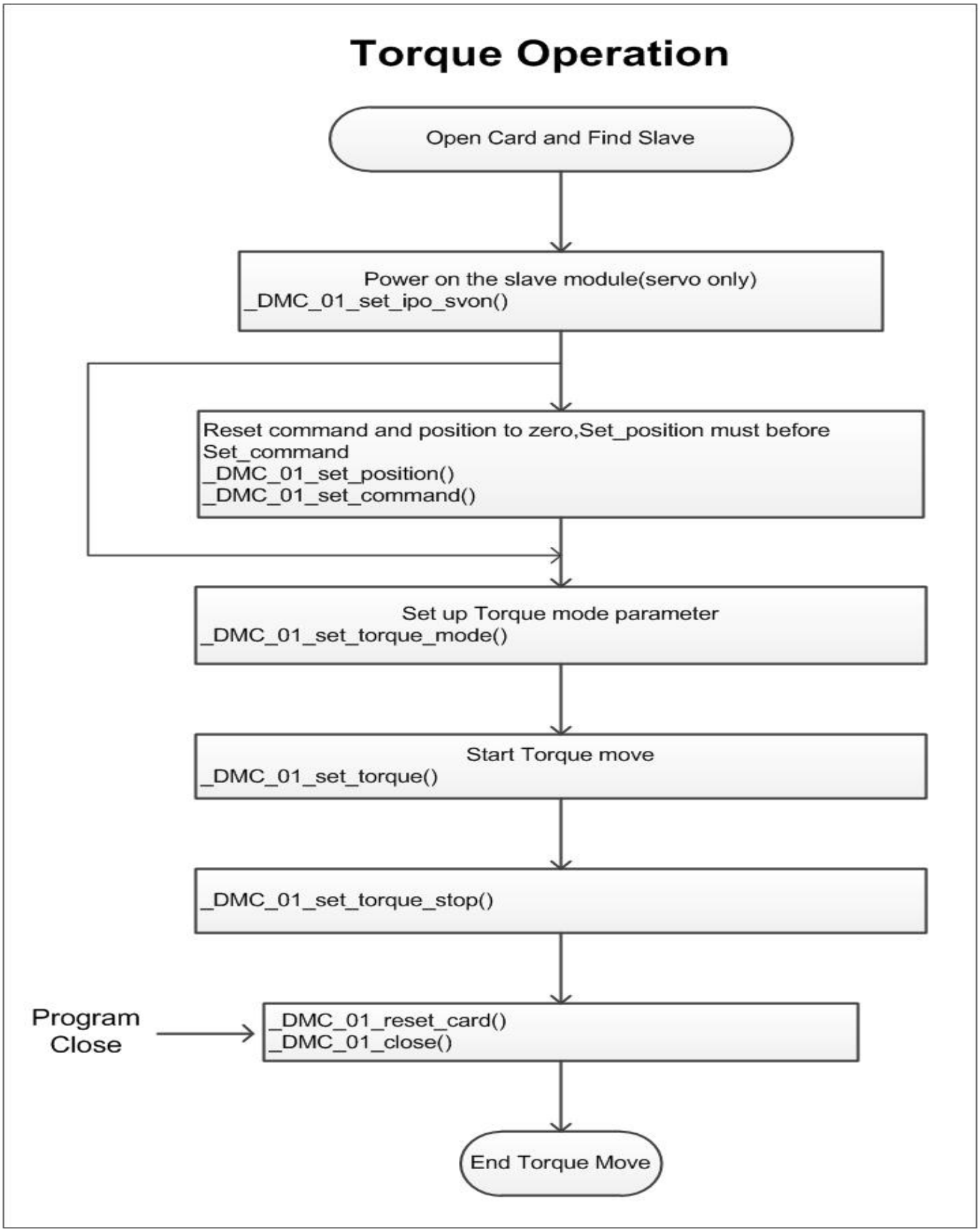
函式名稱
_DMC_01_set_torque_mode
_DMC_01_set_torque
_DMC_01_set_torque_stop
_DMC_01_get_torque

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.5.2 流程圖


Torque Operation
Open Card and Find Slave
Power on the slave module(servo only)
_DMC_01_set_ipo_svon()
Reset command and position to zero,Set_position must before Set_command
_DMC_01_set_position()
_DMC_01_set_command()
Set up Torque mode parameter
_DMC_01_set_torque_mode()
Start Torque move
_DMC_01_set_torque()
_DMC_01_set_torque_stop()
Program Close
_DMC_01_reset_card()
_DMC_01_close()
End Torque Move

3.050

3.5.3 應用範例

程式外觀

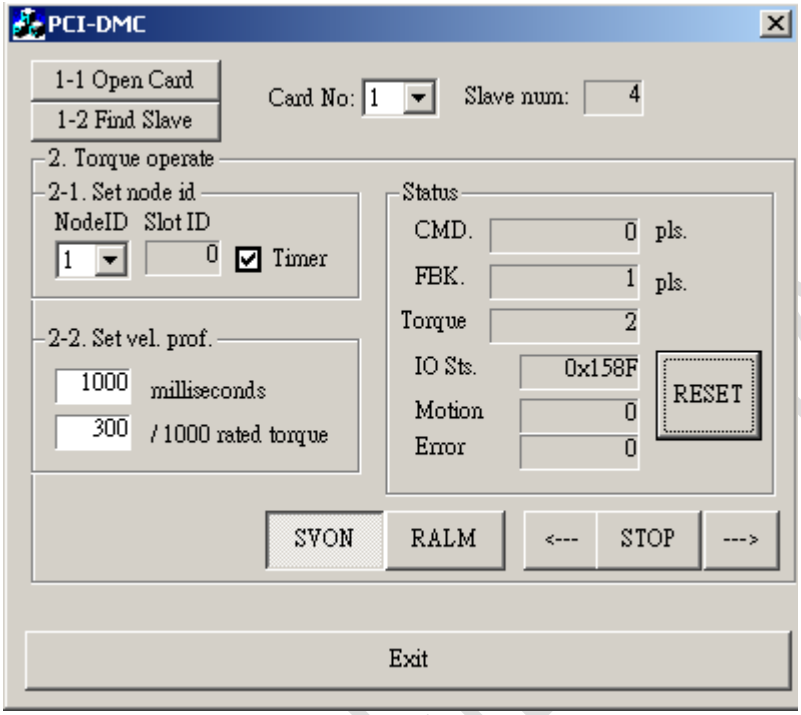


圖 3.051

1) 軸卡開啟與初始化
按壓圖 3.051 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.051 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

圖 3.052

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函式的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入 slope 與 ratio 值

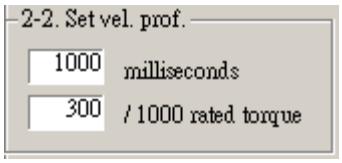


圖 3.053

Slop 項目：由 0 至 100%額定扭力所需時間。(單位：毫秒)

Ratio 項目：額定扭力值千分比。舉例來說，若此值為 300 時表示 30% 額定扭力。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

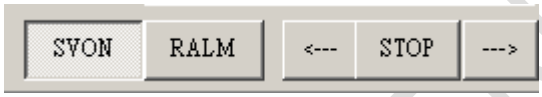


圖 3.054

按壓圖 3.054 中的「SVON」鍵開始執行以下程序：

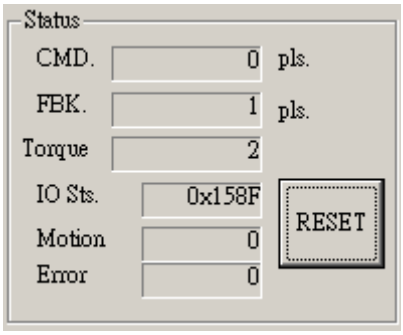
```
rt =  _DMC_01_ipo_set_svon (gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 轉矩運動控制

按壓圖 3.054 中的「→」或「←」鍵執行以下程序：

```
/* 配置轉矩參數(slope 值) */
rt = _DMC_01_set_torque_mode(gDMCCardNo, NodeID, SlotID, slope);
 /* 啟動轉矩運動 */
rt = _DMC_01_set_torque(gDMCCardNo, NodeID, SlotID, ratio);
// 若 ratio 值大於 0，表示馬達將正轉運動；值小於 0 則是反轉運動。
Press the「STOP」button to excute torque stop or not.
/* 馬達轉矩運動停止與否，當 stop 值為 1 時，表示轉矩運動停止。 */
rt = _DMC_01_set_torque_stop(gDMCCardNo, NodeID, SlotID, stop);
```

6) 重置運動計數值及監控顯示



DELTA CONFIDENTIAL

圖 3.055

按壓圖 3.055 的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

7) Alm_Reset

當圖 3.055 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.054 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

8) 離開程序

按壓圖 3.051 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.6 速度運動控制

3.6.1 函式列表

表 3.6

函式名稱
_DMC_01_set_velocity_mode
_DMC_01_set_velocity
_DMC_01_set_velocity_stop
_DMC_01_get_rpm

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.6.2 流程圖

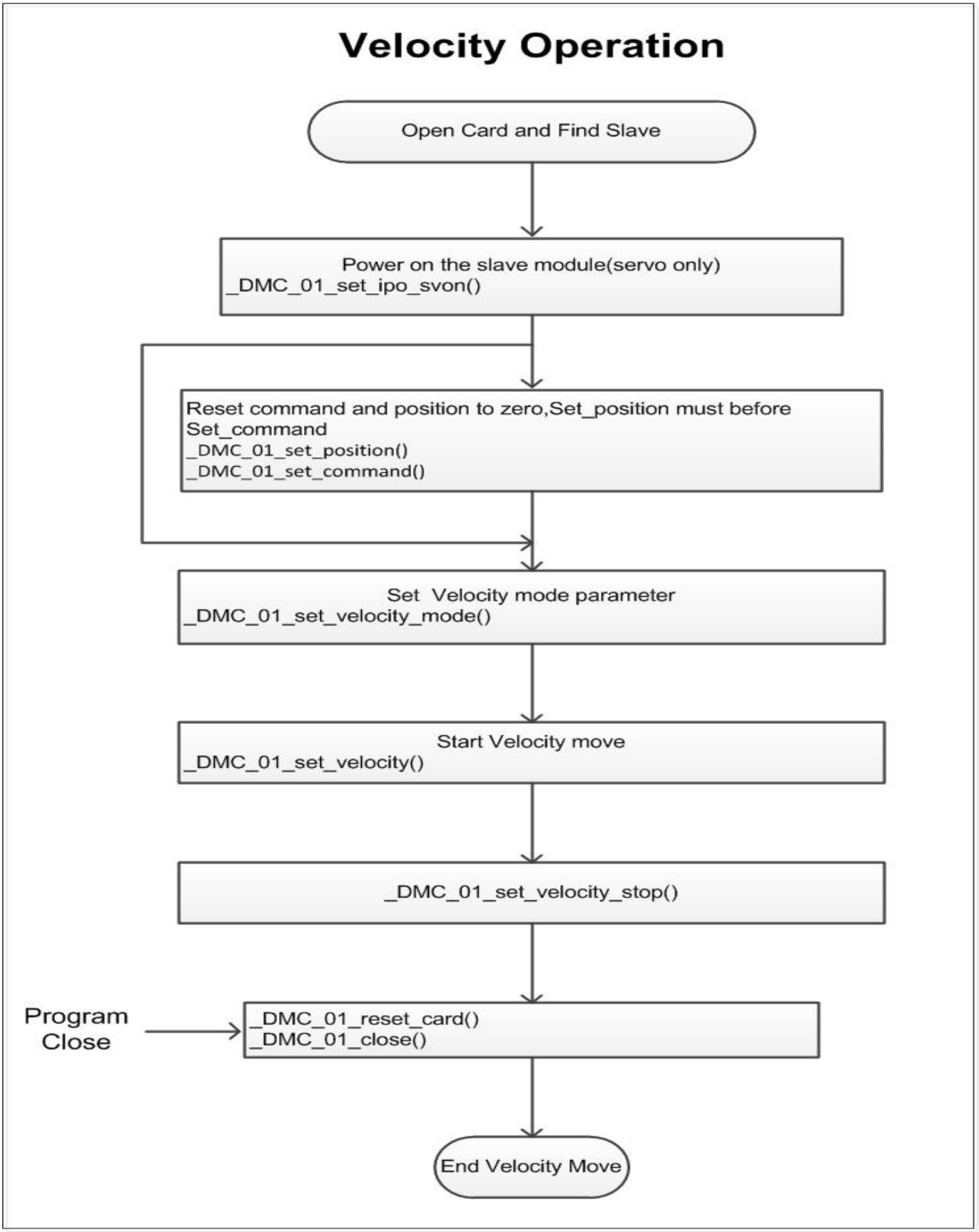


圖 3.060

3.6.3 應用範例

程式外觀

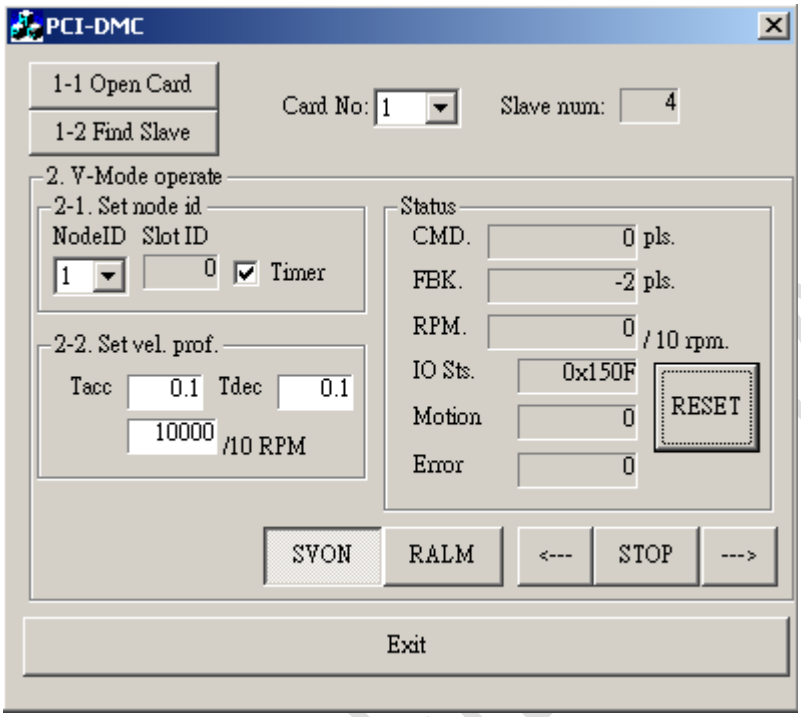


圖 3.061

1) 軸卡開啟與初始化

按壓圖 3.061 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.061 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

圖 3.062

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函式的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入加減速時間的值和每分鐘旋轉速度(RPM)值

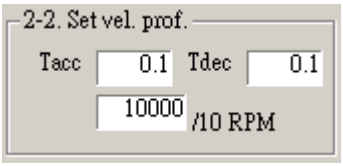


圖 3.063

Tacc 項目：API 函式的引數變數 “Tacc”.
Tdec 項目: API 函式的引數變數 “Tdec”.
RPM 項目: API 函式的引數變數 “rpm”.
※實際 RPM 值為變數 rpm 的 0.1 倍。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.064

按壓圖 3.064 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 速度運動控制

按壓圖 3.064 中的「→」或「←」鍵執行以下程序：

```
/* 配置速度模式參數(加速時間與減速時間的值) */
rt = _DMC_01_set_velocity_mode(gDMCCardNo, NodeID, SlotID, Tacc, Tdec);
 /* 啟動速度模式運動 */
rt = _DMC_01_set_velocity(gDMCCardNo, NodeID, SlotID, rpm);
// 若 rpm 的值大於 0 時，驅動器馬達正轉；反之，值小於 0 為反轉。
```

Press the「STOP」button to excute velocity stop or not.

```
/* 設置是否停止速度運動控制。若 stop 的值為 1，速度運動停止 */
rt = _DMC_01_set_velocity_stop(gDMCCardNo, NodeID, SlotID, stop);
```

DELTA CONFIDENTIAL

6) 重置運動計數值及監控顯示

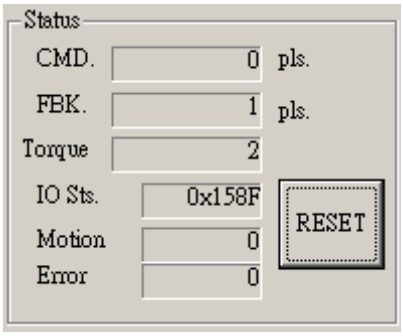


圖 3.065

按壓圖 3.065 的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

7) Alm_Reset

當圖 3.065 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.064 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

8) 離開程序

按壓圖 3.061 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.7 單軸運動控制

3.7.1 函式列表

表 3.7

函式名稱
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_p_change
_DMC_01_v_change
_DMC_01_feedrate_overwrite
_DMC_01_start_sa_move_2seg
_DMC_01_start_ta_move_2seg
_DMC_01_start_sr_move_2seg
_DMC_01_start_tr_move_2seg
_DMC_01_start_sa_move_2seg2
_DMC_01_start_ta_move_2seg2
_DMC_01_start_sr_move_2seg2
_DMC_01_start_tr_move_2seg2
_DMC_01_tv_move
_DMC_01_sv_move
_DMC_01_start_v3_move
_DMC_01_set_position
_DMC_01_set_command
_DMC_01_get_command
_DMC_01_get_position
_DMC_01_get_current_speed
_DMC_01_motion_status
_DMC_01_motion_done
_DMC_01_get_alm_code
_DMC_01_emg_stop

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡

CONFIDENTIAL

●	●

3.7.2 應用範例

程式外觀

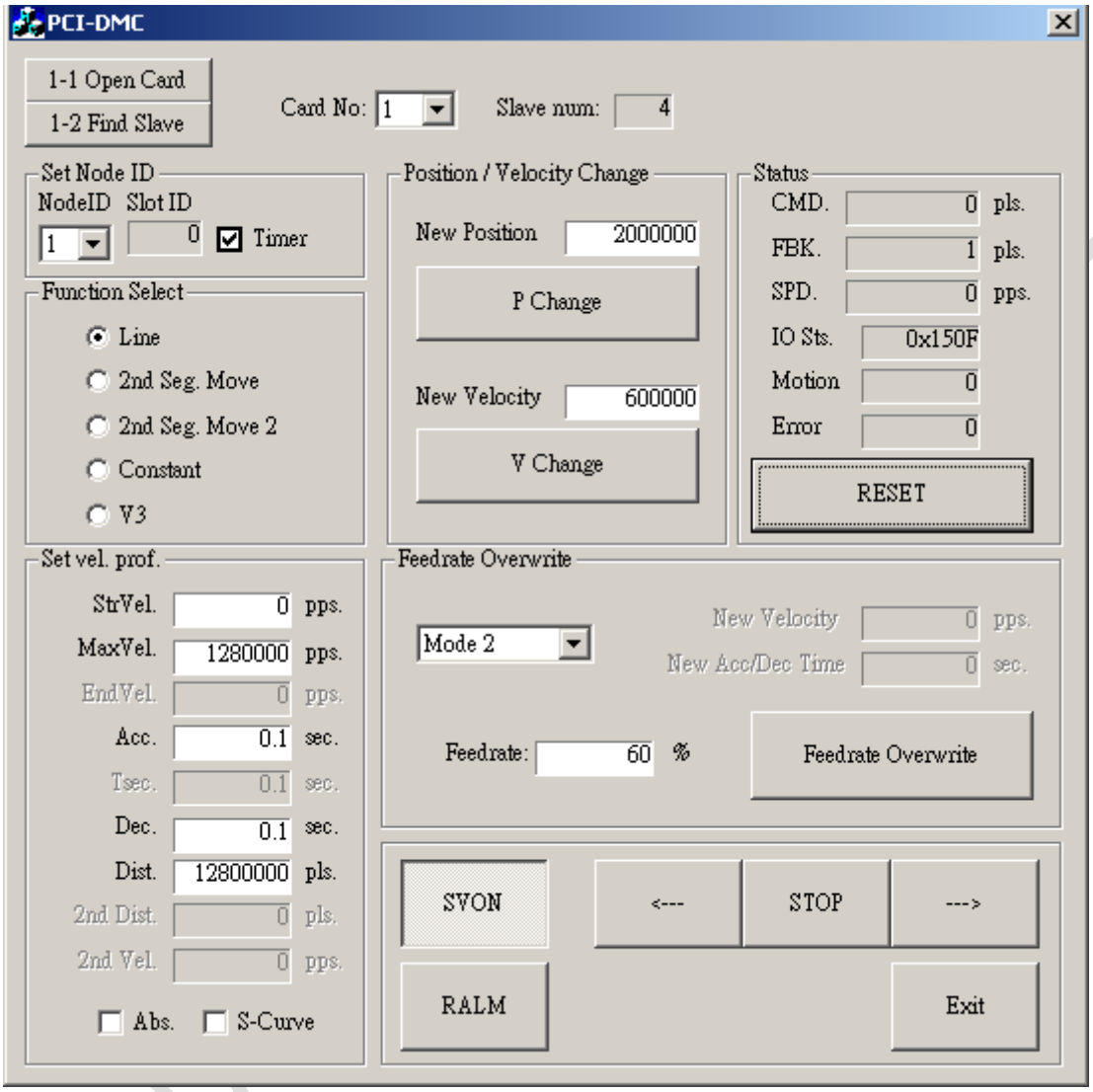


圖 3.070

1) 軸卡開啟與初始化

按壓圖 3.070 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.070 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

圖 3.071

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 功能選擇

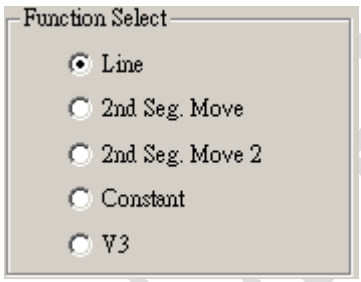


圖 3.072

Line 項目：點對點運動。

2nd Seg. Move 項目：二段式運動模式一。

2nd Seg. Move2 項目：二段式運動模式二。

Constant 項目：直線運動模式。

V3 項目：V3 運動模式。

4) 選擇 Line 項目時運動控制的引數內容值

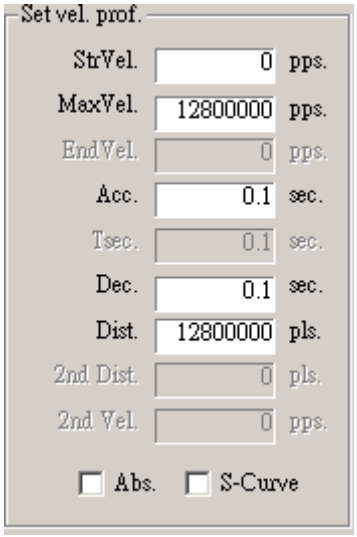


圖 3.073

DELTA CONFIDENTIAL

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

5) 選擇 Line 項目時設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.074

按壓圖 3.074 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 選擇 Line 項目時開始進行速度運動控制

按壓圖 3.074 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

DELTA CONFIDENTIAL

7) 選擇 Line 項目時開始進行位置變換或速度變換運動控制

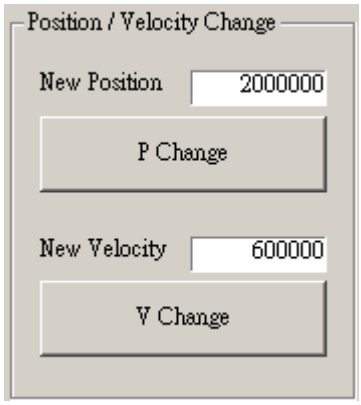


圖 3.075

當按壓圖 3.074 中的「→」或「←」鍵執行 Line 運動控制時，若您欲將運動中的位置

置換為新的位置，您必須按壓圖 3.075 中的「P change」執行以下程序：

```
rt = _DMC_01_p_change (CardNo, NodeID, SlotID, NewPos);
 // 以新的位置值置換目前的位置
```

若您欲將運動中的速度置換為新的速度，您必須按壓圖 3.075 中的「V change」執行

以下程序：

```
rt = _DMC_01_v_change (CardNo, NodeID, SlotID, NewSpeed, sec);
 // 以新的速度值置換目前的速度
```

8) 選擇 Line 項目時開始進行 FeedrateOverwrite 運動控制

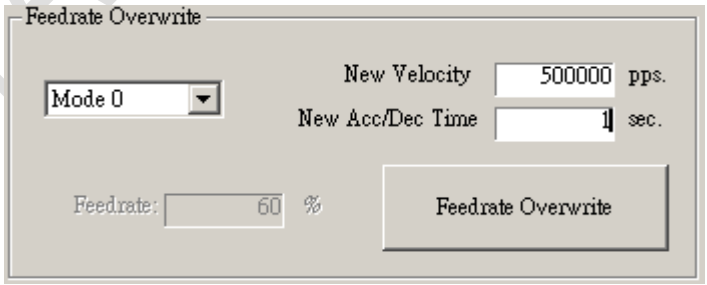


圖 3.076

DELTA CONFIDENTIAL

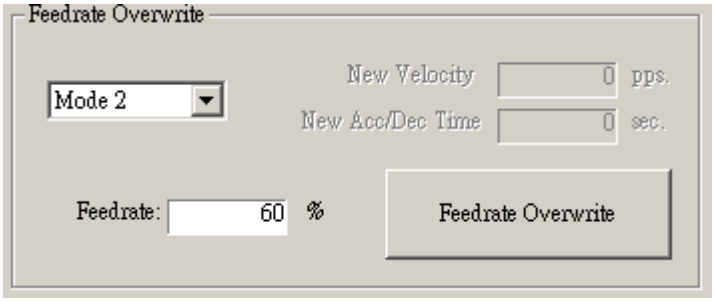


圖 3.077

當按壓圖 3.074 中的「→」或「←」鍵執行 Line 運動控制時，若您欲進行 Mode0、1

的 FeedrateOverwrite，必須按壓圖 3.076 中的「Feedrate Overwrite」執行以下程序：

```
rt = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID ,Mode,Feedrate,
NewAccDecT); // Mode=0，1
```

若您欲進行 Mode2 的 FeedrateOverwrite，您必須按壓圖 3.077 中的「Feedrate Overwrite」執行以下程序：

```
rt = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID ,Mode,Feedrate,
NewAccDecT); // Mode=2
```

9) 選擇 2nd Seg. Move 或 2nd Seg. Move 2 項目時，畫面如圖 3.078 所示：

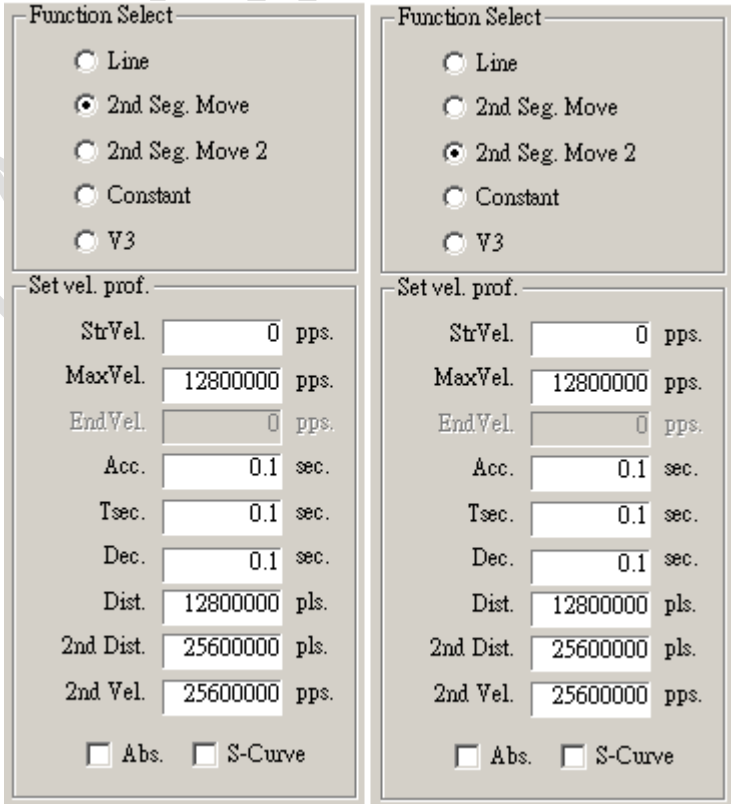


圖 3.078

DELTA CONFIDENTIAL

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。

MaxVel 項目：第一段行程中設置的最大速度。API 函數的引數變數 “MaxVel”。

Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。

Tsec 項目：第一段行程切換至第二段行程的加/減速時間。API 函數引數變數 “Tsec”。

Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

Dist.項目：指定的第一段絕對座標行程。API 函數的引數變數 “Dist”。

2ndDist.項目：指定的第二段絕對座標行程。API 函數的引數變數 “Dist2”。

2ndVel 項目：第二段行程中設置的最大速度。API 函數的引數變數 “MaxVel2”。

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 2nd Seg. Move 或 2nd Seg. Move 2 時，

將執行以下程序：

```
rt = _DMC_01_start_sa_move_2seg (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的絕對座標進行二段速型式一運動位移
rt = _DMC_01_start_ta_move_2seg (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的絕對座標進行二段速型式一運動位移
rt = _DMC_01_start_sr_move_2seg (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的相對座標進行二段速型式一運動位移
rt = _DMC_01_start_tr_move_2seg (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
 //參照 T-curve 速度剖面下的相對座標進行二段速型式一運動位移

rt = _DMC_01_start_sa_move_2seg2 (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的絕對座標進行二段速型式二運動位移
rt = _DMC_01_start_ta_move_2seg2 (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的絕對座標進行二段速型式二運動位移
rt = _DMC_01_start_sr_move_2seg2 (CardNo, NodeID, SlotID, Dist,
```

DELTA CONFIDENTIAL

```
Dist2,StrVel,
    MaxVel, MaxVel2, Tacc, Tsec, Tdec);
    //參照 S-curve 速度剖面下的相對座標進行二段速型式二運動位移
    rt = _DMC_01_start_tr_move_2seg2 (CardNo, NodeID, SlotID, Dist,
Dist2,StrVel,
    MaxVel, MaxVel2, Tacc, Tsec, Tdec);
     //參照 T-curve 速度剖面下的相對座標進行二段速型式二運動位移
```

DELTA CONFIDENTIAL

10) 選擇 Constant 項目時，畫面如圖 3.079 所示：

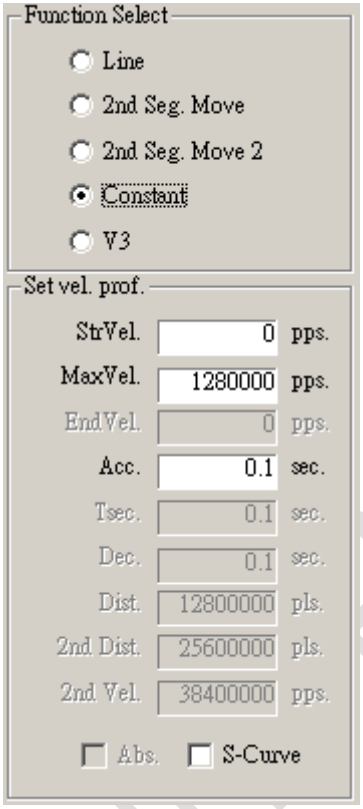


圖 3.079

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：行程中設置的最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 Constant 時，將執行以下程序：

```
rt = _DMC_01_tv_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
//S-curve 速度曲線
rt = _DMC_01_sv_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
//T-curve 速度曲線
```

DELTA CONFIDENTIAL

11) 選擇 V3 項目時，畫面如圖 3.07A 所示：

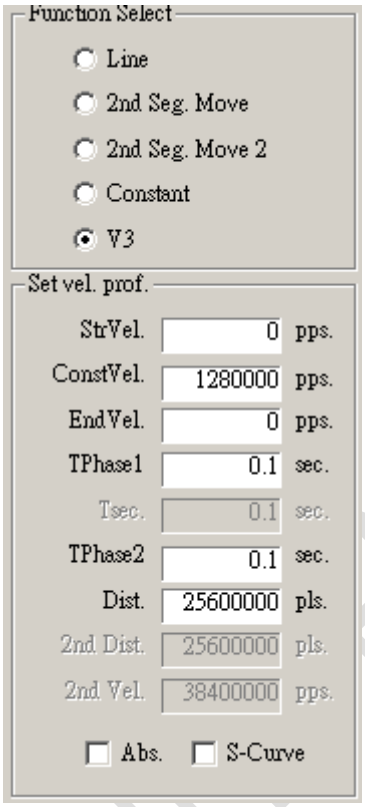


圖 3.07A

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。

ConstVel 項目：行程中的常態速度。API 函數的引數變數 “ConstVel”。

EndVel 項目：運動結束速度。API 函數的引數變數 “EndVel”。

TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。

TPhase2 項目：ConstVel 到 EndVel 所花費時間。API 函數的引數變數 “TPhase2”。 Dist.項目：指定的運動行程。API 函數的引數變數 “Dist”。

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 V3 運動時，將執行以下程序：

```
rt = _DMC_01_start_v3_move (CardNo, NodeID, SlotID, Dist, StrVel, ConstVel,
EndVel, TPhase1, TPhase2, m_curve, m_r_a);
//增加 EndVel 的單軸運動位移
```

DELTA CONFIDENTIAL

12) 重置運動位移計數值(Command 與 Feedback)及監控顯示

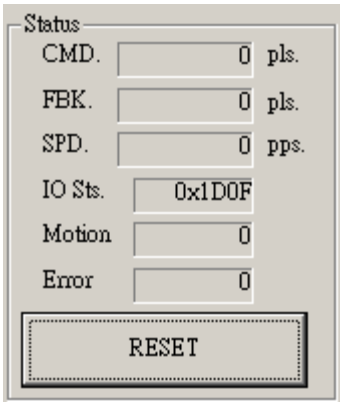


圖 3.07B

按壓圖 3.07B 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

DELTA CONFIDENTIAL

13) 停止運動及 Alm_Code 重置

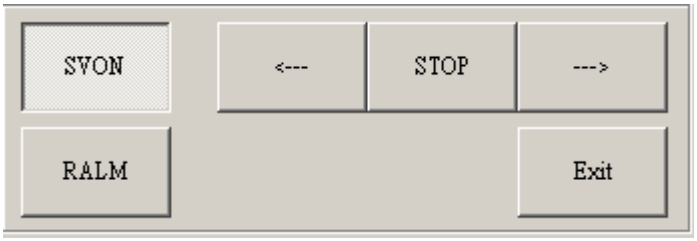


圖 3.07C

不論正在執行哪一項運動，當按壓圖 3.07C 中的「STOP」鍵開始執行緊急停止：

rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.07B 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.07C 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

14) 離開程序

按壓圖 3.07C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.8 雙軸運動控制

3.8.1 函式列表

表 3.8

函式名稱
_DMC_01_start_sa_move_xy
_DMC_01_start_ta_move_xy
_DMC_01_start_sr_move_xy
_DMC_01_start_tr_move_xy
_DMC_01_start_v3_move_xy
_DMC_01_start_sa_arc_xy
_DMC_01_start_ta_arc_xy
_DMC_01_start_sr_arc_xy
_DMC_01_start_tr_arc_xy
_DMC_01_start_v3_arc_xy
_DMC_01_start_sa_arc2_xy
_DMC_01_start_ta_arc2_xy
_DMC_01_start_sr_arc2_xy
_DMC_01_start_tr_arc2_xy
_DMC_01_start_v3_arc2_xy
_DMC_01_start_sa_arc3_xy
_DMC_01_start_ta_arc3_xy
_DMC_01_start_sr_arc3_xy
_DMC_01_start_tr_arc3_xy
_DMC_01_start_v3_arc3_xy
_DMC_01_start_spiral_xy
_DMC_01_start_v3_spiral_xy
_DMC_01_start_spiral2_xy
_DMC_01_start_v3_spiral2_xy

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

ADLINK CONFIDENTIAL

3.8.2 應用範例

程式外觀

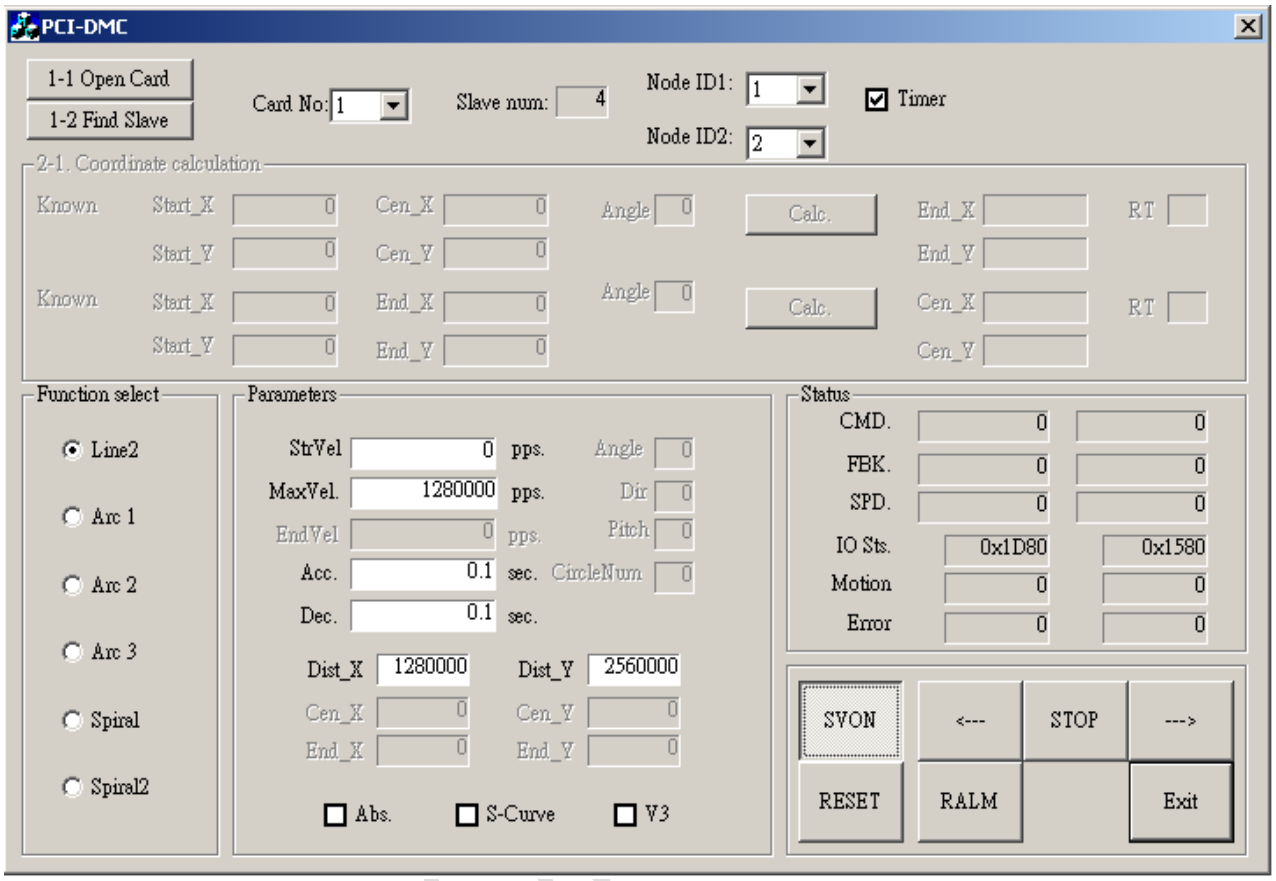


圖 3.080

1) 軸卡開啟與初始化

按壓圖 3.080 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.080 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

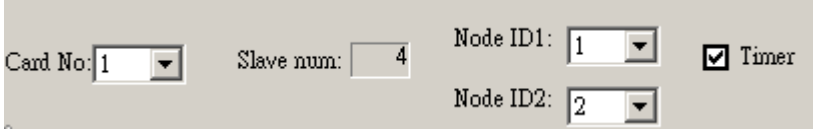


圖 3.081

選取 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 功能選擇

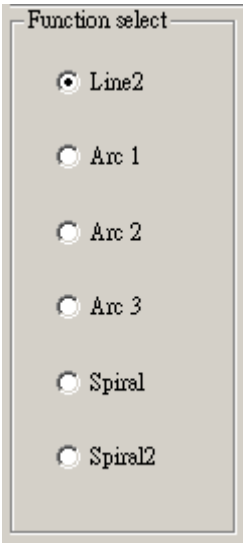


圖 3.082

Line2 項目：線性補間運動控制。
Arc1 項目 ：圓弧補間運動控制型式一。
Arc2 項目 ：圓弧補間運動控制型式二。
Arc3 項目 ：圓弧補間運動控制型式三。
Spiral 項目 ：螺旋補間運動控制型式一。
Spiral2 項目 ：螺旋補間運動控制型式二。

DELTA CONFIDENTIAL

4) 選擇 Line2 運動控制的引數內容值如圖 3.083 所示:

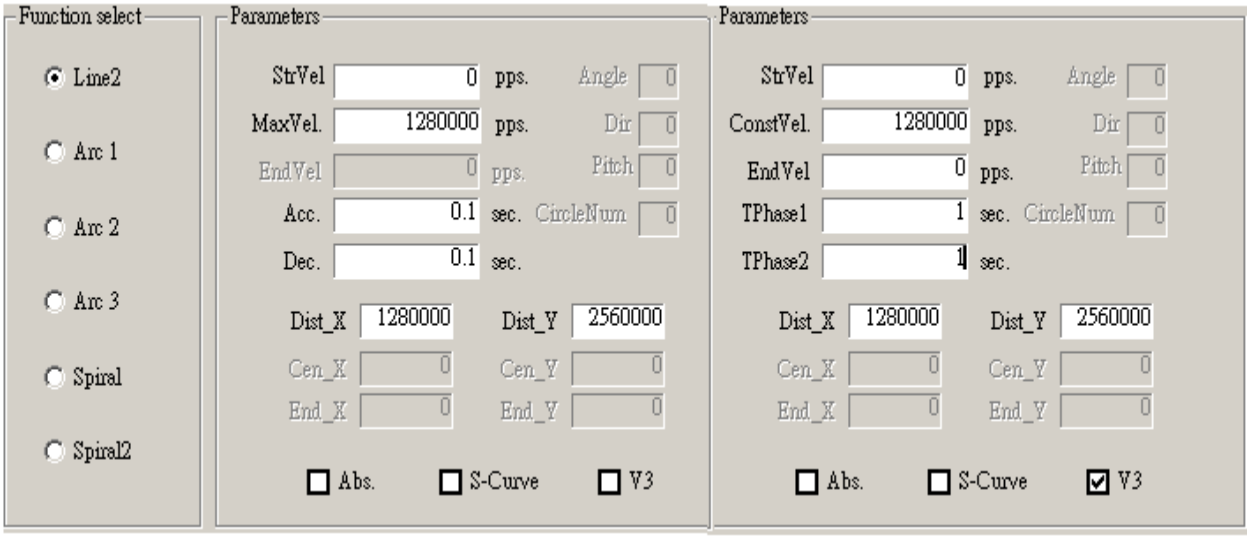


圖 3.083

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist_X 項目: X 軸的運動行程參數。API 函數的引數變數 “DisX”。
Dist_Y 項目: Y 軸的運動行程參數。API 函數的引數變數 “DisY”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.083 右部的圖示選項。
StrVel 項目　　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDEN

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

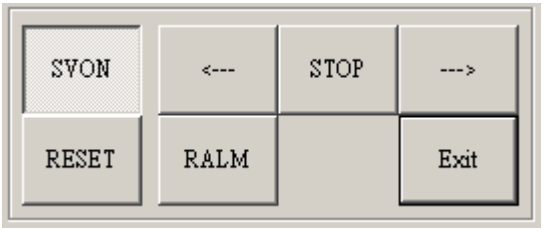


圖 3.084

按壓圖 3.084 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 當按壓圖 3.084 中的「→」或「←」鍵執行 Line2(未勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_sa_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
     DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸線性補間運動
rt = _DMC_01_start_ta_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
     DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸線性補間運動
rt = _DMC_01_start_sr_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
     DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸線性補間運動
rt = _DMC_01_start_tr_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
     DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸線性補間運動
```

7) 當按壓圖 3.084 中的「→」或「←」鍵執行 Line2(勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
     DistY, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸線性補間運動
```

DELTA CONFIDENTIAL

8) 選擇功能 Arc1、Arc2 及 Arc3 將會出現圖 3.085「Arc 計算功能」，填入已知條件並選擇使用的計算方式，將可得出使用該選擇 Arc 的參數設定資料。

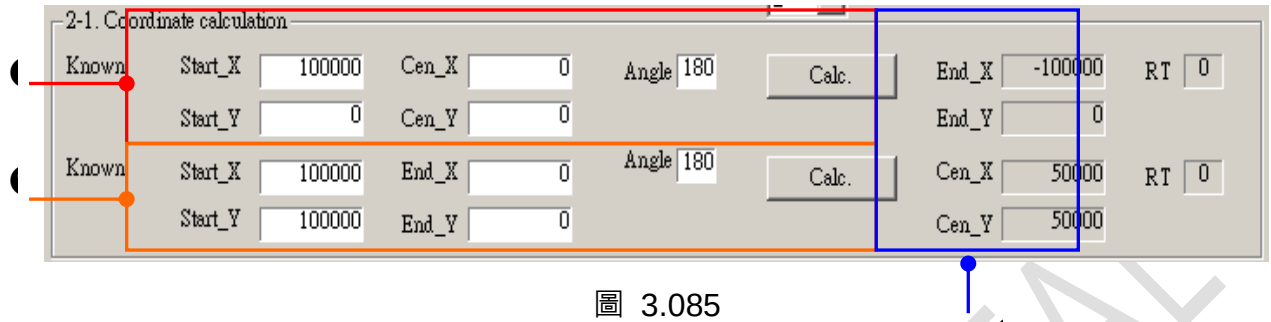


圖 3.085

若您已知起始座標 XY 值、中心座標 XY 值和對應的角度值，然後即可藉由計算式取得終點座標 XY 值。故當您按壓「Calc.」鍵開始執行以下程序：

```
/* 取得 X 軸與 Y 軸的終點座標值 */
rt = _misc_app_get_circle_endpoint(Start_X, Start_Y , Center_X, Center_Y, Angle, &End_X, &End_Y);
```

若您已知起始座標 XY 值、終點座標 XY 值和對應的角度值，然後即可藉由計算式取得中心座標 XY 值。故當您按壓「Calc.」鍵開始執行以下程序：

```
/* 取得 X 軸與 Y 軸的中心座標值 */
rt = _misc_app_get_circle_center_point(Start_X, Start_Y , End_X, End_Y, Angle, &Center_X, &Center_Y);
```

3. 當您完成執行程序 1 或程序 2，計算式計算出來的結果將顯示在區塊❸。

DELTA CONFIDENTIAL

9) 選擇 Arc1 運動控制的引數內容值如圖 3.086 所示:

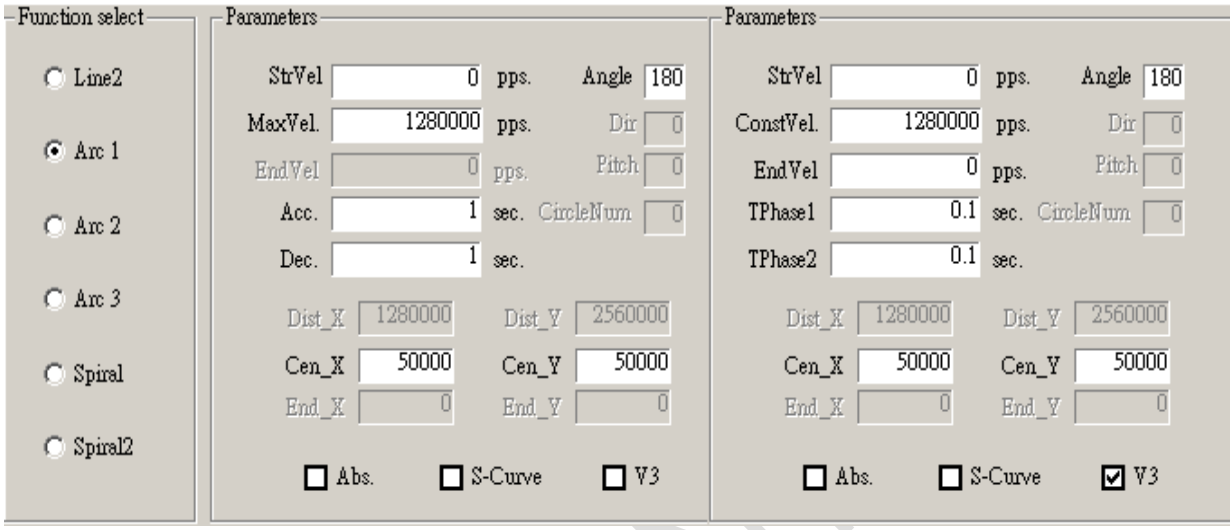


圖 3.086

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。
Angle 項目: 圓弧的角度設置。API 函數的引數變數 “Angle”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.086 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDENTIAL

10) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc1(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_ta_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_sr_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_tr_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式一)
```

11) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc1 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve,
m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式一)
```

DELTA CONFIDENTIAL

12) 選擇 Arc2 運動控制的引數內容值如圖 3.087 所示:

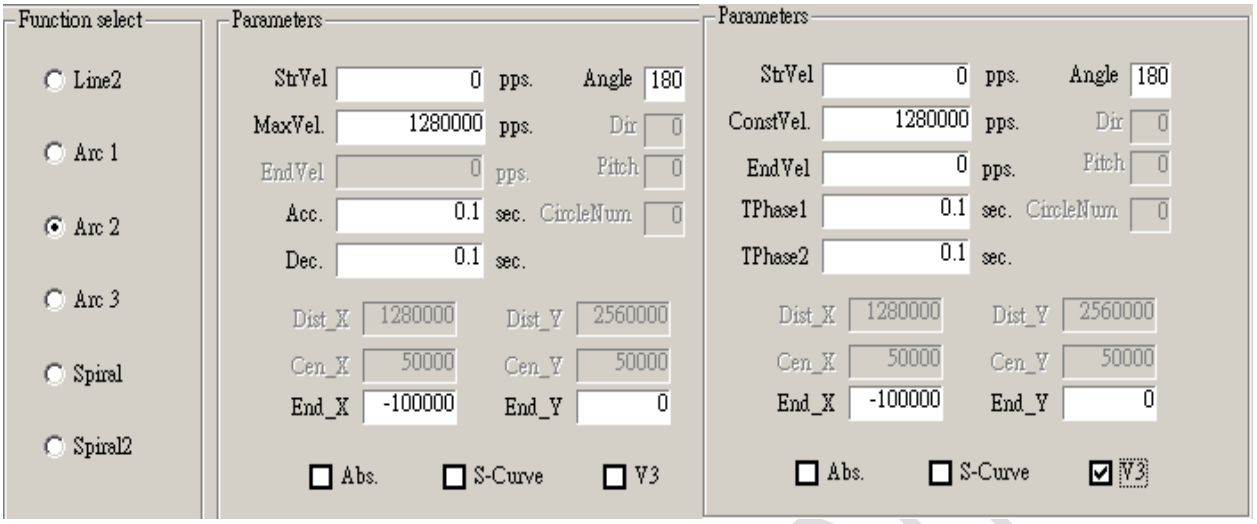


圖 3.087

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Angle 項目: 圓弧的角度設置。API 函數的引數變數 “Angle”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.087 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDEN

13) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc2(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End_X,
    End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_ta_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_sr_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_tr_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式二)
```

14) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc2 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
End _Y, Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve,
m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式二)
```

DELTA CONFIDENTIAL

15) 選擇 Arc3 運動控制的引數內容值如圖 3.088 所示:

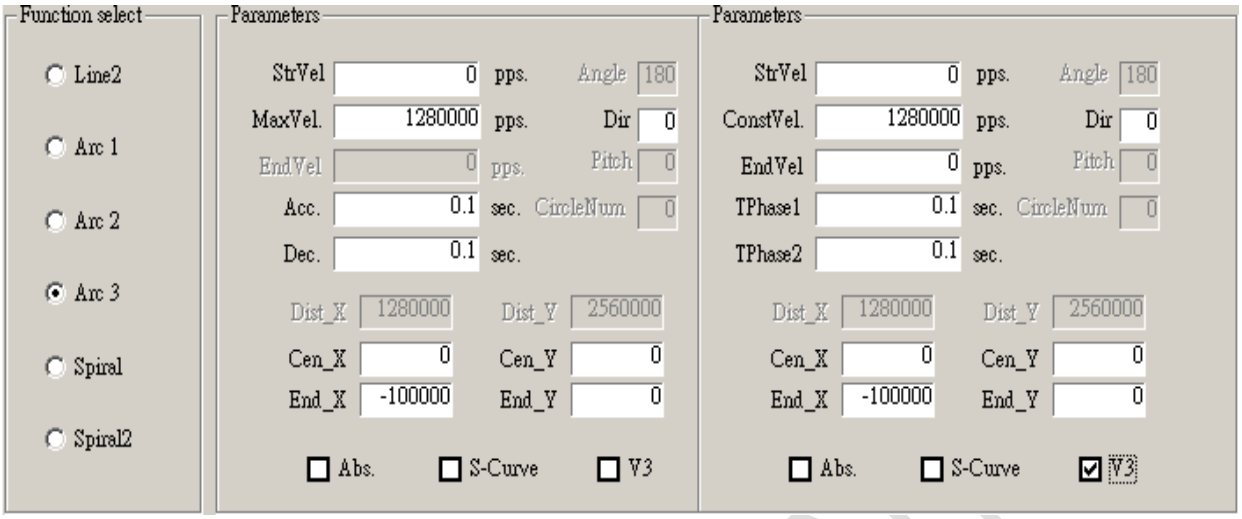


圖 3.088

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Dir 項目: 指定的方向(0:順時針方向 1:逆時針方向)。API 函數的引數變數 “Dir”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.088 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDEN

16) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc3(未勾選 V3)功能時，

將執行以下程序：

```
rt = _DMC_01_start_sa_arc3_xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_ta_arc3_xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_sr_arc3_xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_tr_arc3_xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式三)
```

17) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc3 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc3_xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2,
m_curve, m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式三)
```

DELTA CONFIDENTIAL

18) 選擇 Spiral 運動控制的引數內容值如圖 3.089 所示:

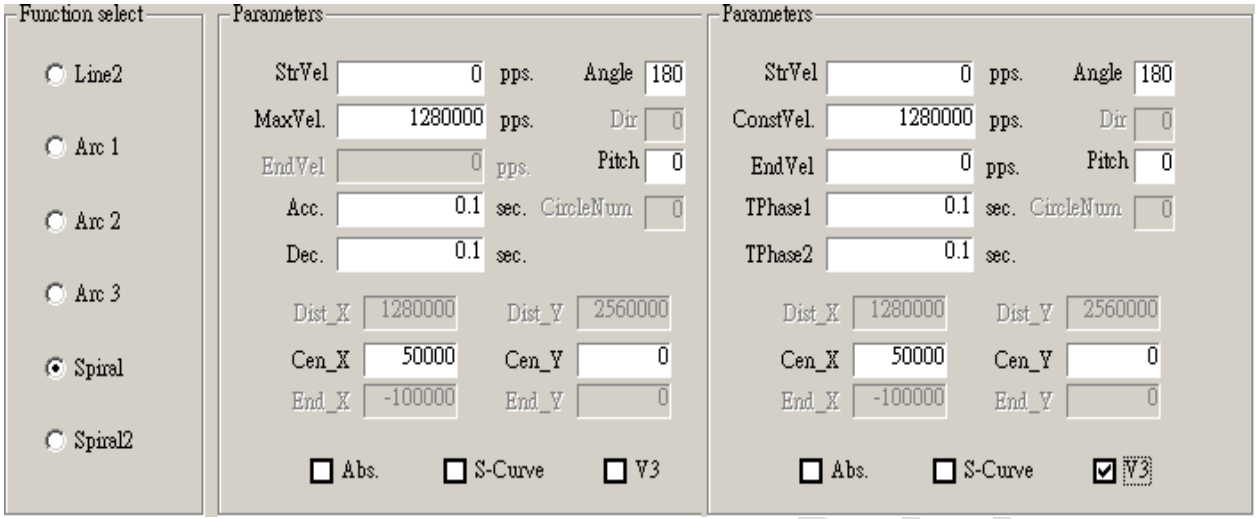


圖 3.089

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應中心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應中心位置 Y。API 函數的引數變數 “Center_Y”。
Angle 項目: 螺旋運動之總角度。API 函數的引數變數 “spiral_angle”。
Pitch 項目: 螺旋之間相對性位置距離。API 函數的引數變數 “spiral_interval”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.089 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

19) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_spiral_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Spiral_interval, Spiral_angle, StrVel, MaxVel, Tacc, Tdec, m_curve,
M_r_a);//進行兩軸螺旋運動(型式一)
```

DELTA CONFIDEN

20) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_spiral _xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Spiral_interval, Spiral_angle, StrVel, ConstVel, EndVel, TPhase1,
TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸螺旋運動(型式一)
```

21) 選擇 Spiral2 運動控制的引數內容值如圖 3.08A 所示:

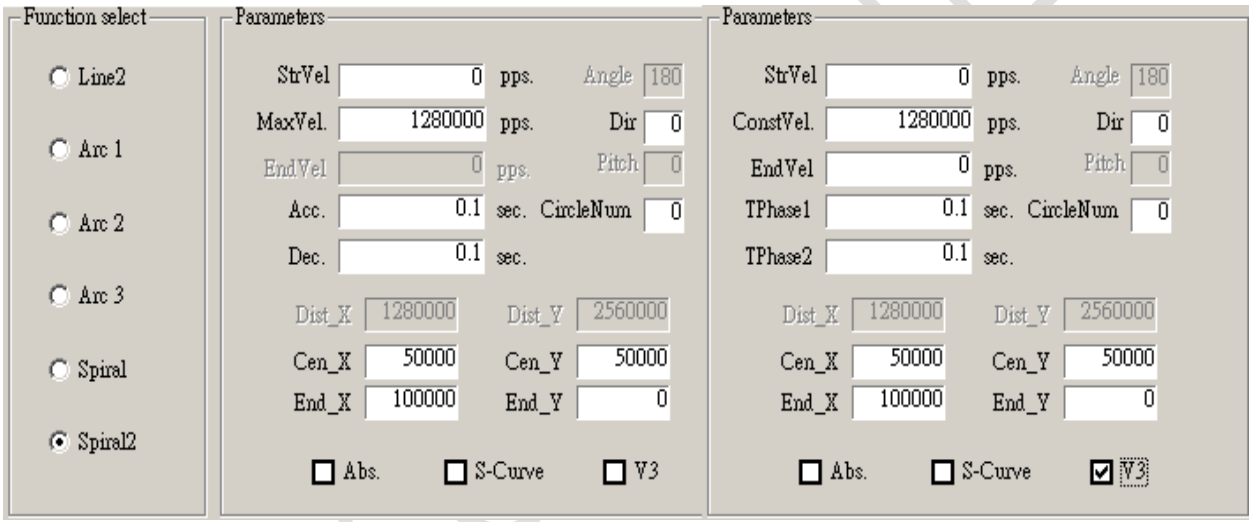


圖 3.08A

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應中心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應中心位置 Y。API 函數的引數變數 “Center_Y”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Dir 項目: 指定的方向(0:順時針方向 1:逆時針方向)。API 函數的引數變數 “dir”。
CircleNum 項目: 螺旋運動的圈數。API 函數的引數變數 “circlenum”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.08A 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。

ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。

EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。

TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。

TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

22) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral2(未勾選 V3)功能時，

將執行以下程序：

```
rt = _DMC_01_start_spiral2_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X, Center_Y, End_X, End_Y, dir, circelnum, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);//進行兩軸螺旋運動(型式二)
```

23) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral2(勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_spiral2 _xy(gDMCCardNo, NodeIDArray, SlotID, Center_X, Center_Y, End_X, End_Y, dir, circelnum, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸螺旋運動(型式二)
```

24) 監控顯示

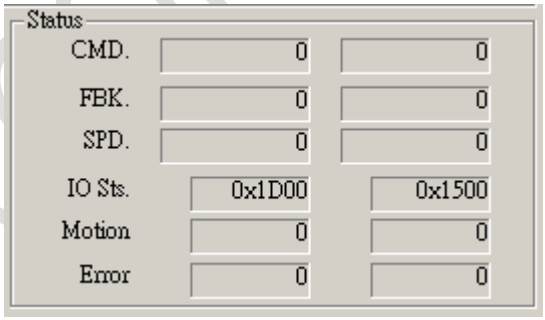


圖 3.08B

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
```

DELTA CONFIDENTIAL

```
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

25) 重置運動計數值、停止運動及 Alm_Reset

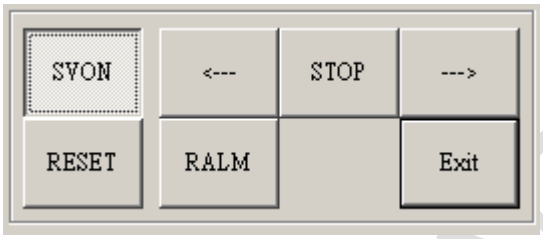


圖 3.08C

按壓圖 3.08C 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.08C 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.08B 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.08C 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

26) 離開程序

按壓圖 3.08C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.9 三軸運動控制

3.9.1 函式列表

表 3.9

函式名稱
_DMC_01_start_sa_move_xyz
_DMC_01_start_ta_move_xyz
_DMC_01_start_sr_move_xyz
_DMC_01_start_tr_move_xyz
_DMC_01_start_v3_move_xyz
_DMC_01_start_sa_heli_xy
_DMC_01_start_ta_heli_xy
_DMC_01_start_sr_heli_xy
_DMC_01_start_tr_heli_xy
_DMC_01_start_v3_heli_xy

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.9.2 應用範例

程式外觀

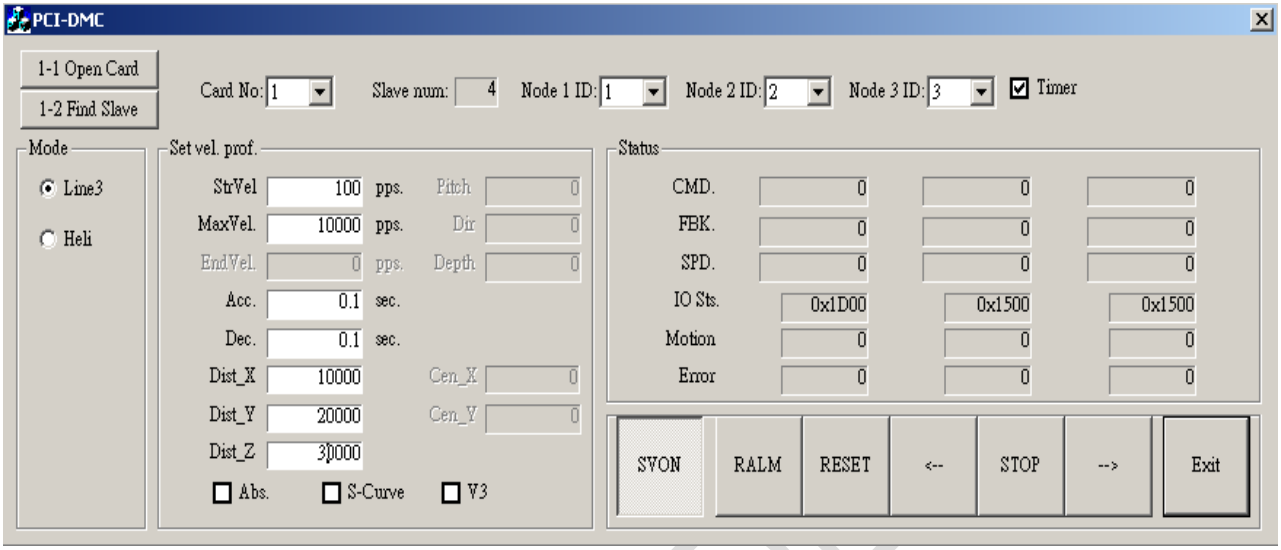


圖 3.090

1) 軸卡開啟與初始化

按壓圖 3.090 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.090 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

圖 3.091

勾選「Timer」核選方塊啟用顯示運動狀態。
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 功能選擇

圖 3.092

Line3 項目：線性補間運動控制。
Heli 項目 ：螺旋補間運動控制。

DELTA CONFIDENTIAL

4) 選擇 Line3 運動控制的引數內容值如圖 3.093 所示:

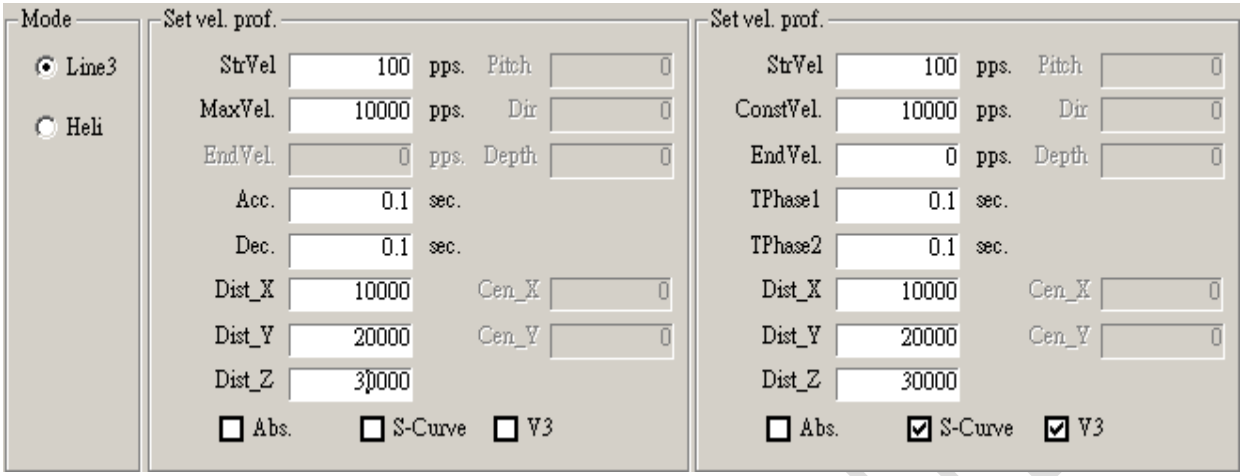


圖 3.093

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist_X 項目: 對應 X 軸站號運動的路徑參數。API 函數的引數變數 “DisX”。
Dist_Y 項目: 對應 Y 軸站號運動的路徑參數。API 函數的引數變數 “DisY”。
Dist_Z 項目: 對應 Z 軸站號運動的路徑參數。API 函數的引數變數 “DisZ”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.093 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDENTIAL

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.094

按壓圖 3.094 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 當按壓圖 3.094 中的「→」或「←」鍵執行 Line3(未勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_sa_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行三軸線性補間運動
rt = _DMC_01_start_ta_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行三軸線性補間運動
rt = _DMC_01_start_sr_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行三軸線性補間運動
rt = _DMC_01_start_tr_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行三軸線性補間運動
```

7) 當按壓圖 3.094 中的「→」或「←」鍵執行 Line3 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的三軸線性補間運動
```

DELTA CONFIDENTIAL

8) 選擇 Heli 運動控制的引數內容值如圖 3.095 所示:

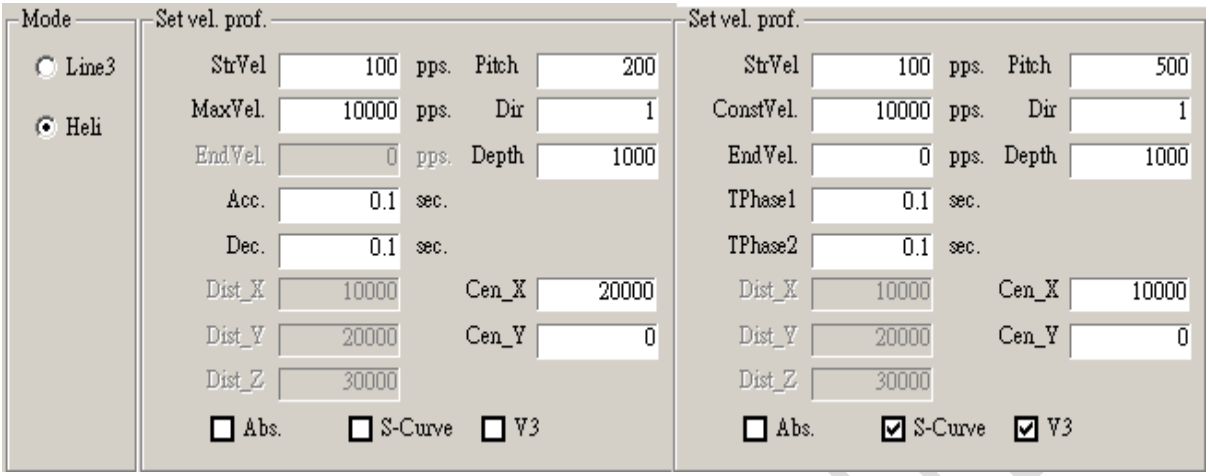


圖 3.095

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。

MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。

Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。

Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

Pitch 項目: 兩螺旋之間的高度。API 函數的引數變數 “Pitch”。

Dir 項目: 螺旋圓弧運動的方向 (0:順時針方向 1:逆時針方向)。
API 函數的引數變數 “Dir”。

Depth 項目: 指定軸對應之位置深度(整個 Z 方向高度)API 函數的引數變數 “Depth”。

Cen_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。

Cen_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

示意圖請參閱圖 3.096

若勾選 V3，則會出現圖 3.095 右部的圖示選項。

StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。

ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。

EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。

TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。

TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

DELTA CONFIDENTIAL

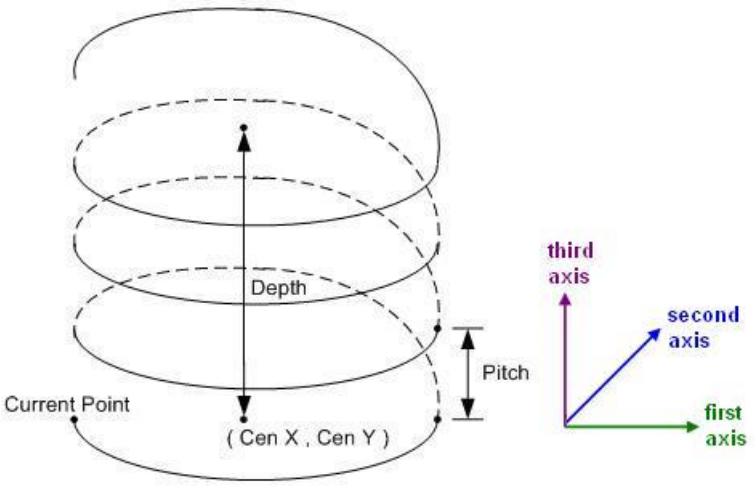


圖 3.096

9) 當按壓圖 3.094 中的「→」或「←」鍵執行 Heli(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行三軸螺旋補間運動
rt = _DMC_01_start_ta_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行三軸螺旋補間運動
rt = _DMC_01_start_sr_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行三軸螺旋補間運動
rt = _DMC_01_start_tr_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行三軸螺旋補間運動
```

10) 當按壓圖 3.094 中的「→」或「←」鍵執行 Heli (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Depth, Pitch, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2,
m_curve, m_r_a);
// 增加 EndVel 的三軸螺旋補間運動
```

11) 監控顯示

DELTA CONFIDENTIAL

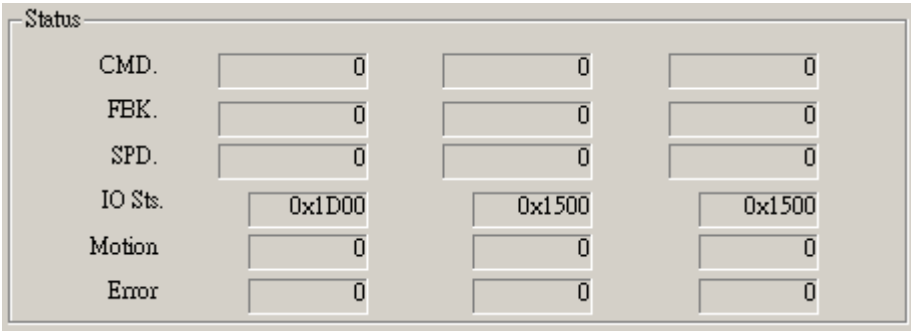


圖 3.097

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

DELTA CONFIDENTIAL

12) 重置運動計數值、停止運動及 Alm_Reset

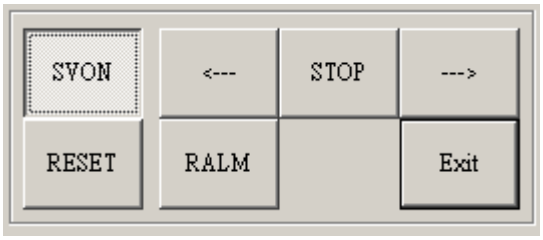


圖 3.098

按壓圖 3.098 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.098 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.097 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.098 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

13) 離開程序

按壓圖 3.098 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.10 多軸運動控制

3.10.1 函式列表

表 3.10

函式名稱
_DMC_01_multi_axes_move
_DMC_01_start_v3_multi_axes
_DMC_01_liner_speed_master

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.10.2 應用範例

程式外觀

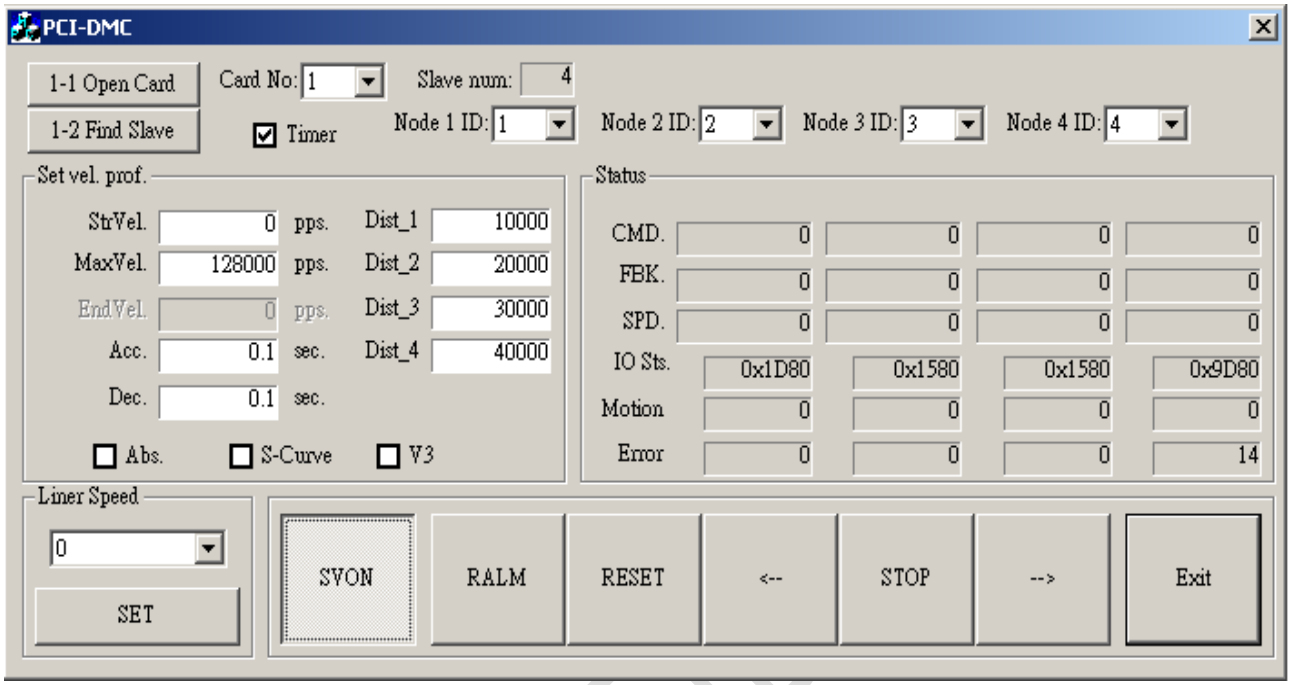


圖 3.100

1) 軸卡開啟與初始化

按壓圖 3.100 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.100 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

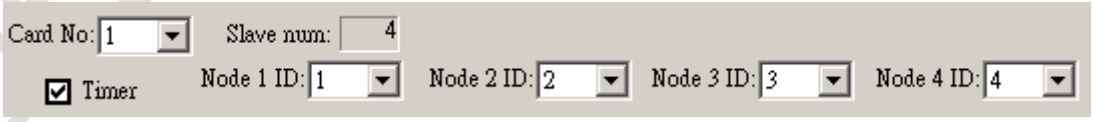


圖 3.101

勾選「Timer」核選方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入運動控制的引數內容值

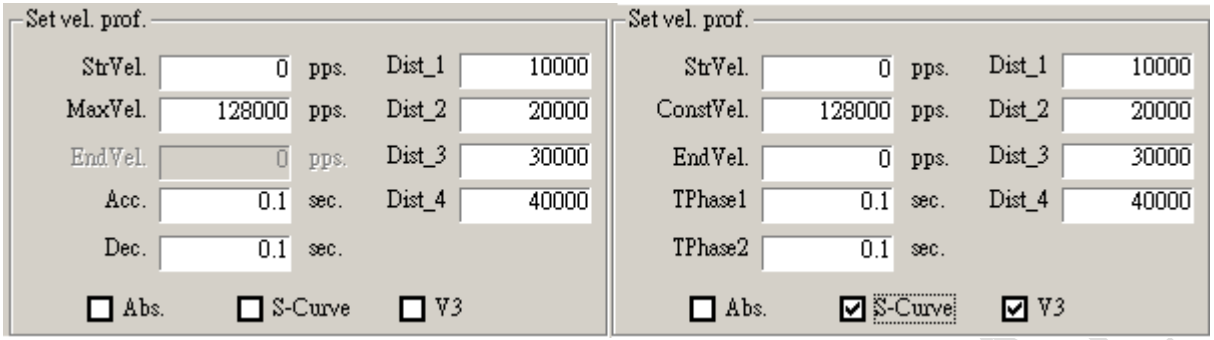


圖 3.102

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.102 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.103

按壓圖 3.103 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

DELTA CONFIDENTIAL

5) 開始進行多軸(未勾選 V3)運動控制

按壓圖 3.103 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_multi_axes_move(gDMCCardNo, AxisNum, NodeIDArry, SlotIDArry,
    DistArry, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
// 設置一軸(含)以上的運動控制。
```

6) 開始進行多軸(勾選 V3)運動控制

按壓圖 3.103 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_v3_multi_axes(gDMCCardNo, AxisNum, NodeIDArry,
    SlotIDArry, DistArry, StrVel, ConstVel, EndVel, Tacc, Tdec, m_curve, m_r_a);
// 設置增加 EndVel 一軸(含)以上的運動控制。
```

7) 設定 Liner Speed Master 功能

圖 3.104

Mode 項目：模式設定(0：向量速度，1：距離最遠軸的最大分量速度)。
API 函數的引數“Mode”。

按壓圖 3.104 中的「SET」鍵開始執行以下程序：

```
rt = _DMC_01_liner_speed_master(gDMCCardNo, NodeID, SlotID , Mode);
// 0：向量速度 1：距離最遠軸的最大分量速度 2：指定第一軸分量速度
```

8) 監控顯示

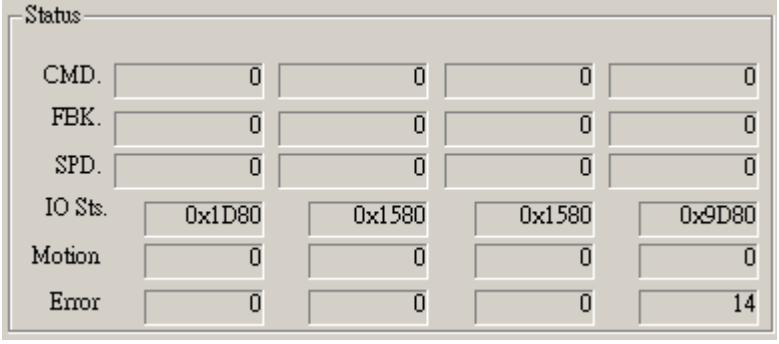


圖 3.105

DELTA CONFIDENTIAL

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

9) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.103 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.103 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.105 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.103 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

10) 離開程序

按壓圖 3.103 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.11 同步運動控制

3.11.1 函式列表

表 3.11

函式名稱
_DMC_01_sync_move
_DMC_01_sync_move_config

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.11.2 流程圖

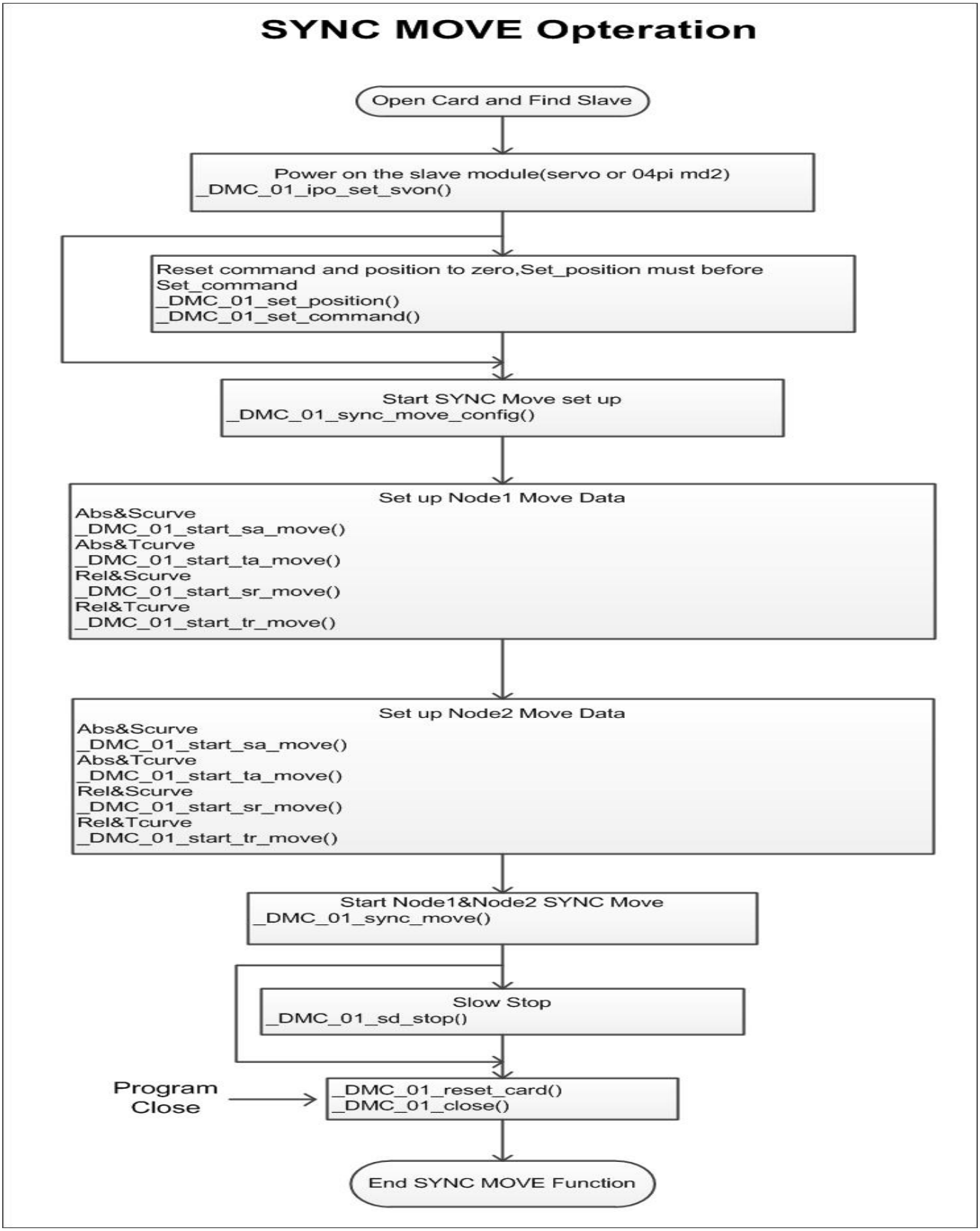


圖 3.110

3.11.3 應用範例

程式外觀

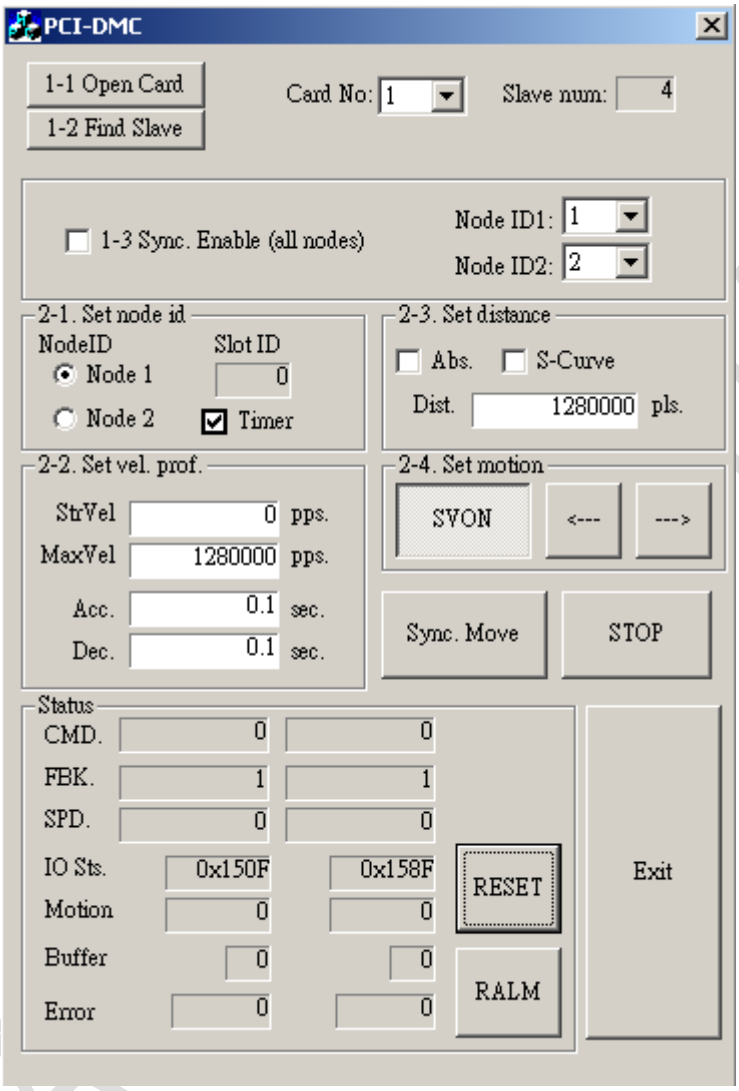


圖 3.111

1) 軸卡開啟與初始化

按壓圖 3. 111 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 111 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

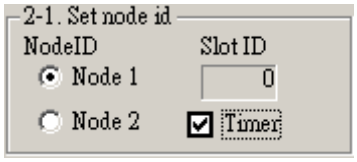


圖 3.112

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

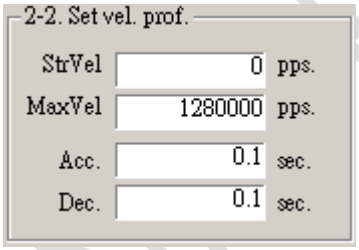


圖 3.113

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

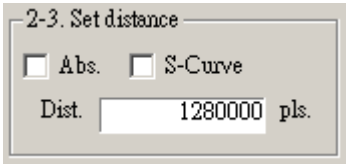


圖 3.114

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。

DELTA CONFIDENTIAL

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

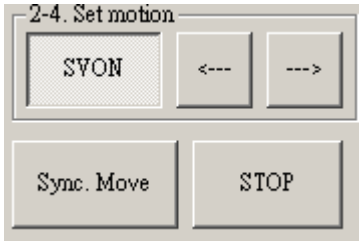


圖 3.115

按壓圖 3. 115 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 選擇是否啟動同步運動控制

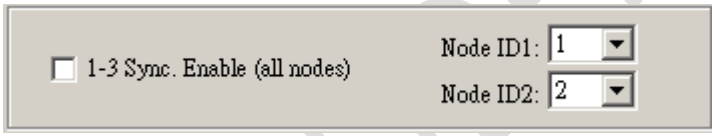


圖 3.116

Sync. Enable 核取方塊：啟動馬達運動同步與否。

勾選圖 3. 116 中的 Sync. Enable，您可以執行以下程序啟用同步運動：

```
rt = _DMC_01_sync_move_config(gDMCCardNo, gpNodeID[i], SlotID, enable);
// 若 enable 引數值為 1，啟動同步運動
```

7) 按壓圖 3. 115 中的「→」或「←」鍵開始執行點對點單軸運動命令。由於您僅設置一軸作運動，此時 motion 仍尚未動作，您必須回到 2) 設置驅動器的 Node ID 的步驟開始設置其它軸的運動命令，待設置完成後，按壓圖 3. 115 中「Sync. Move」鍵開始執行同動命令。

此同動命令如下列函數動作執行：

```
rt = _DMC_01_sync_move(gDMCCardNo);
// 當同步運動完成，同動的設置即會回復到關閉的狀態。如您欲再次使用同動命令，則必須重新啟動設置同動命令。
```

DELTA CONFIDENTIAL

8) 重置運動位移計數值、Alm_Reset 及監控顯示

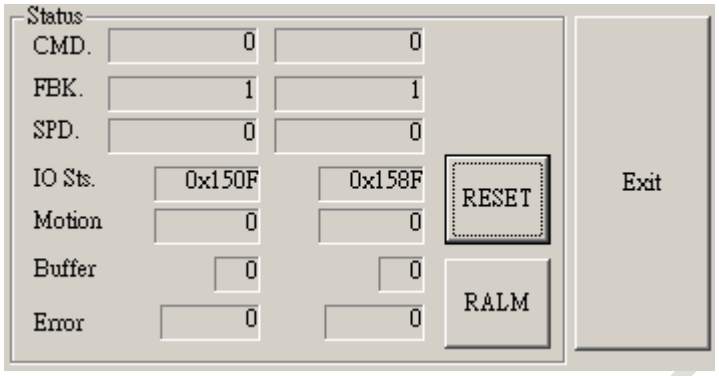


圖 3.117

按壓圖 3.117 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

當圖 3.117 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.117 中的「RALM」鍵開始

執行 Alm_Code 重置動作。

DELTA CONFIDENTIAL

9) 停止運動

不論正在執行哪一項運動，當按壓圖 3.115 中的「STOP」鍵開始執行緊急停止：

rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

10) 離開程序

按壓圖 3.117 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.12 Speed Continue

3.12.1 函式列表

表 3.12

函式名稱
_DMC_01_speed_continue
_DMC_01_speed_continue_mode
_DMC_01_speed_continue_ratio
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_start_sr_move
_DMC_01_start_tr_move

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.12.2 應用範例

程式外觀

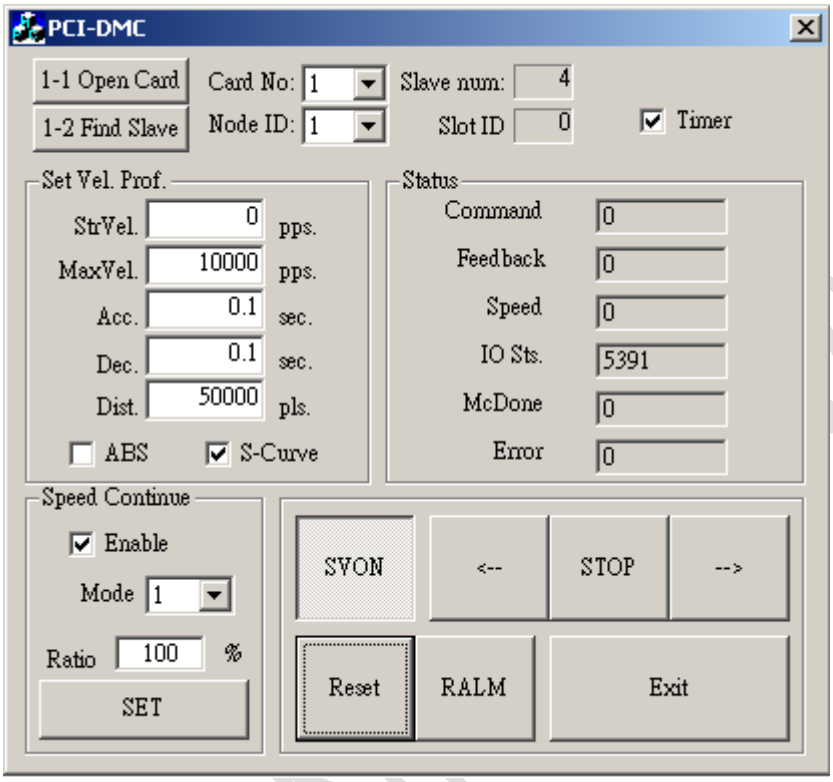


圖 3.120

1) 軸卡開啟與初始化

按壓圖 3. 120 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 120 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

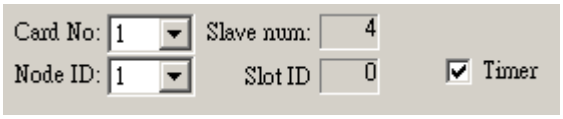


圖 3.121

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入運動控制的引數內容值

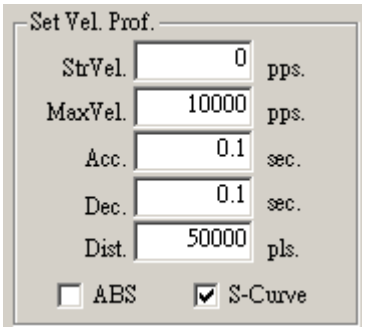


圖 3.122

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。
ABS 核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定 Speed Continue 相關參數

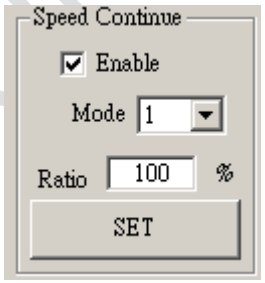


圖 3.123

Enable 核取方塊：若欲啟動 Speed Continue 功能，必須勾選它，勾選執行以下程序：

```
rt = _DMC_01_speed_continue(gDMCCardNo, NodeID, SlotID , enable);
 //0：Disable 1：Enable
```

Mode 項目：設定連續速度模式。API 函數的引數變數 “Mode”。
Ratio 項目：設定連續速度合成百分比。API 函數的引數變數 “Ratio”。
按壓圖 3.123 中的「SET」鍵開始執行以下程序：

```
rt = _DMC_01_speed_continue_mode(gDMCCardNo, NodeID, SlotID , mode);
 //0：等加速模式 1：加、減速模式 2：最大速度模式
rt = _DMC_01_speed_continue_ratio(gDMCCardNo, NodeID, SlotID , ratio);
 //0~100
```

DELTA CONFIDENTIAL

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.124

按壓圖 3.124 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 按壓圖 3.124 中的「→」或「←」鍵開始執行連續運動並執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

DELTA CONFIDENTIAL

7) 監控顯示

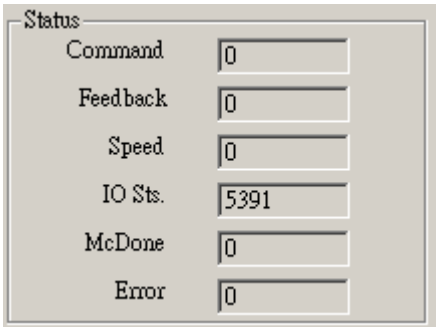


圖 3.125

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.124 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.124 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.125 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.124 中的「RALM」

DELTA CONFIDENTIAL

鍵

開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.124 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.13 螺旋補間 Helix 應用 Normal_Follow

3.13.1 函式列表

表 3.13

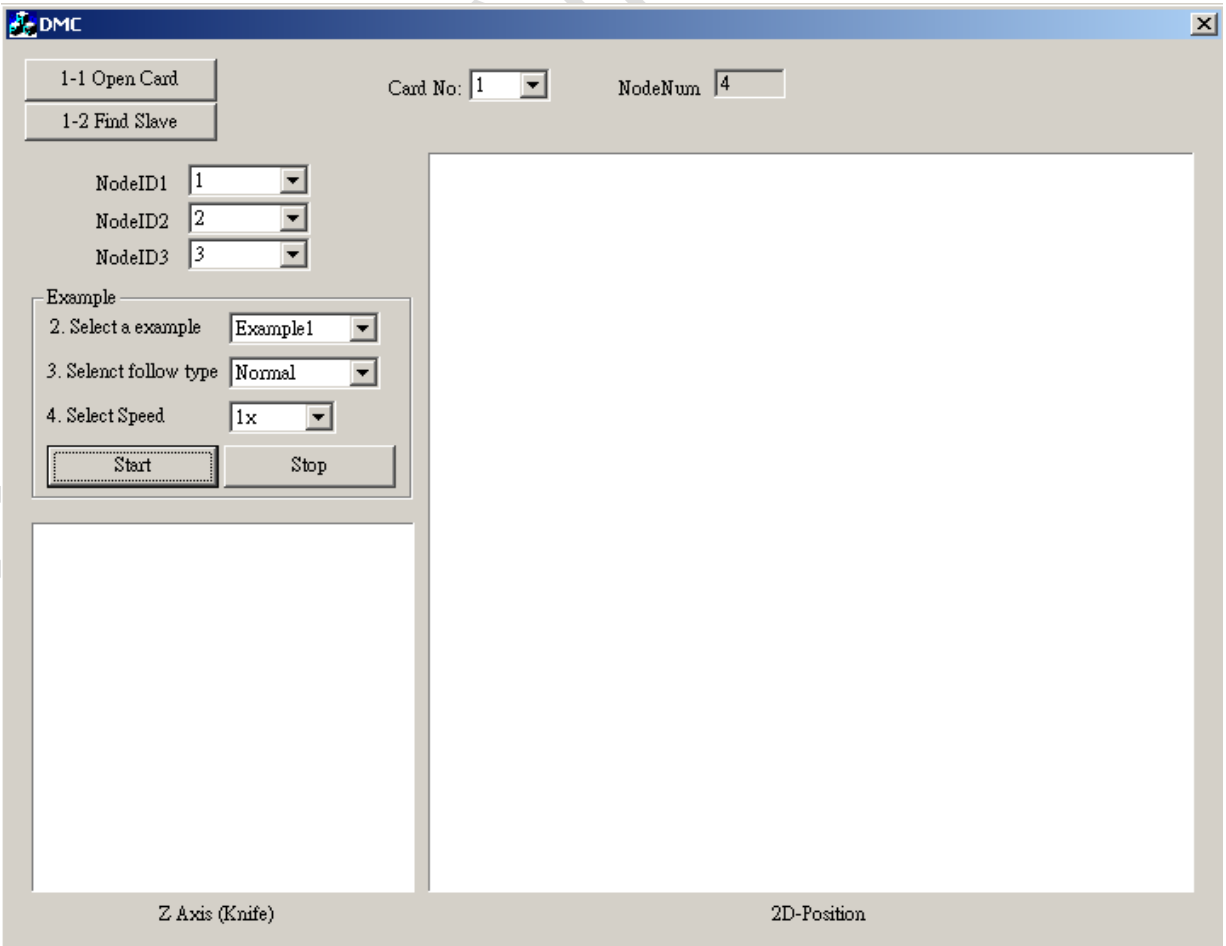
函式名稱
_DMC_01_start_tr_heli_xy
_DMC_01_start_v3_msbrline_move
_DMC_01_start_v3_multi_axes

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.13.2 應用範例

程式外觀



CONFIDENTIAL

圖 3.130

DELTA CONFIDENTIAL

1) 軸卡開啟與初始化

按壓圖 3.130 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.130 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 卡號、站數顯示:

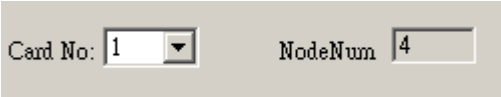


圖 3.131

Card num 項目：顯示卡片張數。

Slave num 項目：顯示站數。

3) Example 參數設定及操作:

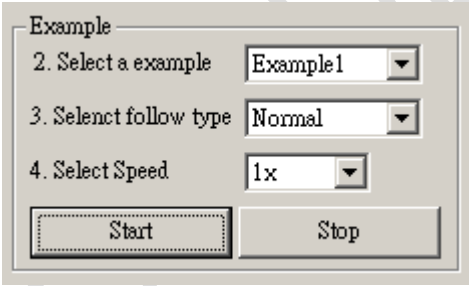


圖 3.132

Select a example 項目：有 Example1 、Example2 及 MSBRline 可供選擇。

Select follow type 項目：設有 Normal 及 Tangent 可供選擇。

Speed 項目：選擇預定速度倍率。

Start 項目：當點選"Start"鍵將執行以下設定。

_DMC_01_start_tr_heli_xy(CardNo,NodeIDArry,SlotIDArry,CenterX,CenterY, Depth,Pitch, Dir, StartVel, MaxVel, Tacc, Tdec);

參數：

CardNo：卡號。

NodeIDArry：3 軸站號的陣列。

SlotIDArry：3 軸號的陣列。

CenterX：X,Y 軸圓弧動作時的圓心(X 座標)。

CenterY：X,Y 軸圓弧動作時的圓心(Y 座標)。

Depth：Z 軸移動的距離(等於圓弧的角度除以 360 再乘上 Pitch)。

Pitch：Z 軸旋轉一圈的距離淨量(恆為正數)。

DELTA CONFIDENTIAL

Dir：圓弧動作的方向，0 為順時針，1 為逆時針。

備註：若是 Z 軸走 Pitch 的距離為順時針旋轉一圈，則如果 Dir = 0，Depth 也為正數，Dir = 1 則 Depth 為負數；若 Z 軸走 pitch 的距離為逆時針旋轉一圈，則反之。

當選擇 Example1 及 Normal 時，執行結果如圖 3.133:

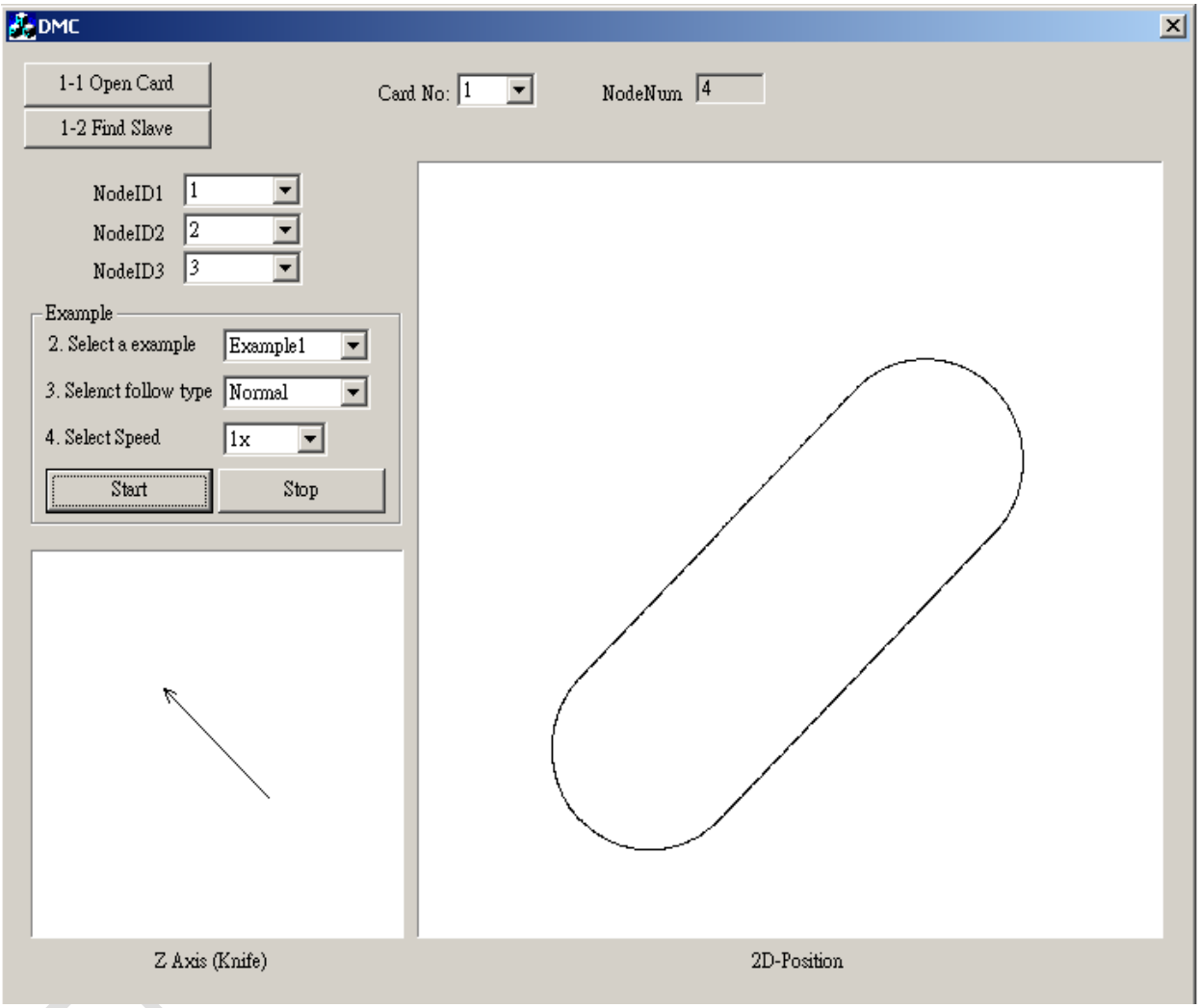


圖 3.133

當選擇 Example2 及 Tangent 時，執行結果如圖 3.134:

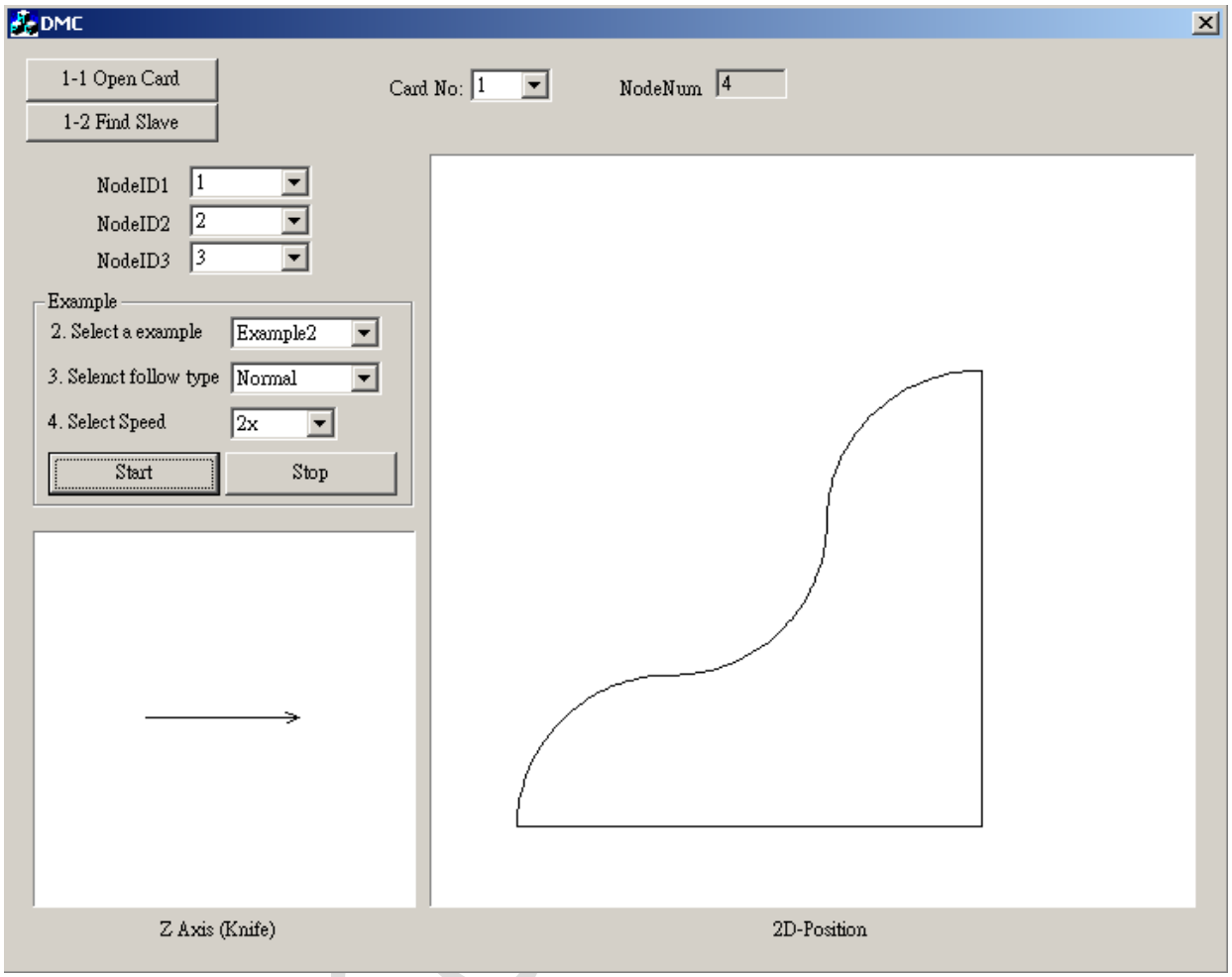


圖 3.134

DELTA C

當選擇 MSBRline 及 Normal 時，執行結果如圖 3.135:

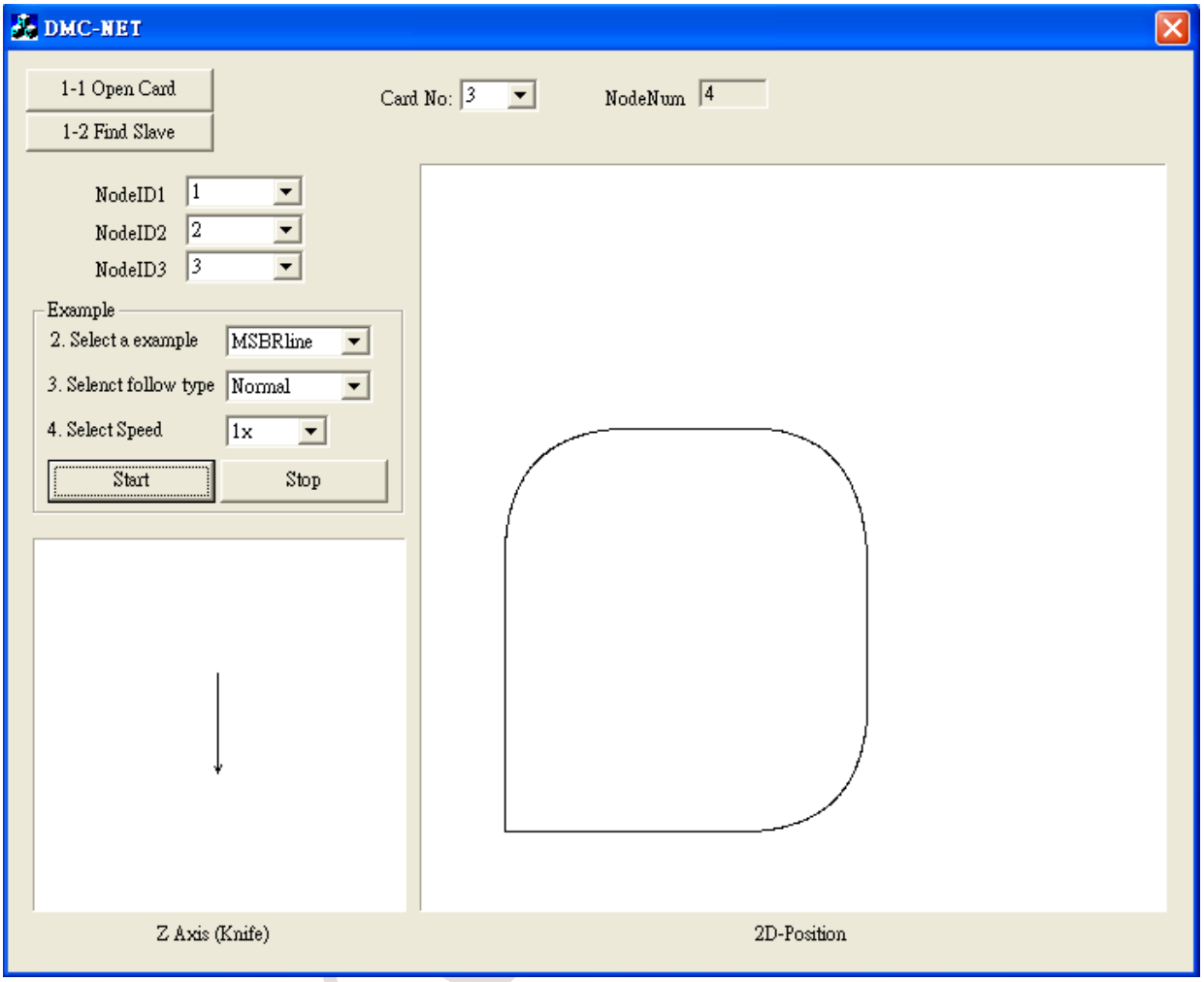


圖 3.135

DELTA

例子 1：此例是透過 X 軸與 Y 軸，使得工具繞圓角及直線移動，並且讓 Z 軸的工具始終與圖形外緣垂直(Normal)或切線(Tangent)如圖 3.136。

(以下解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

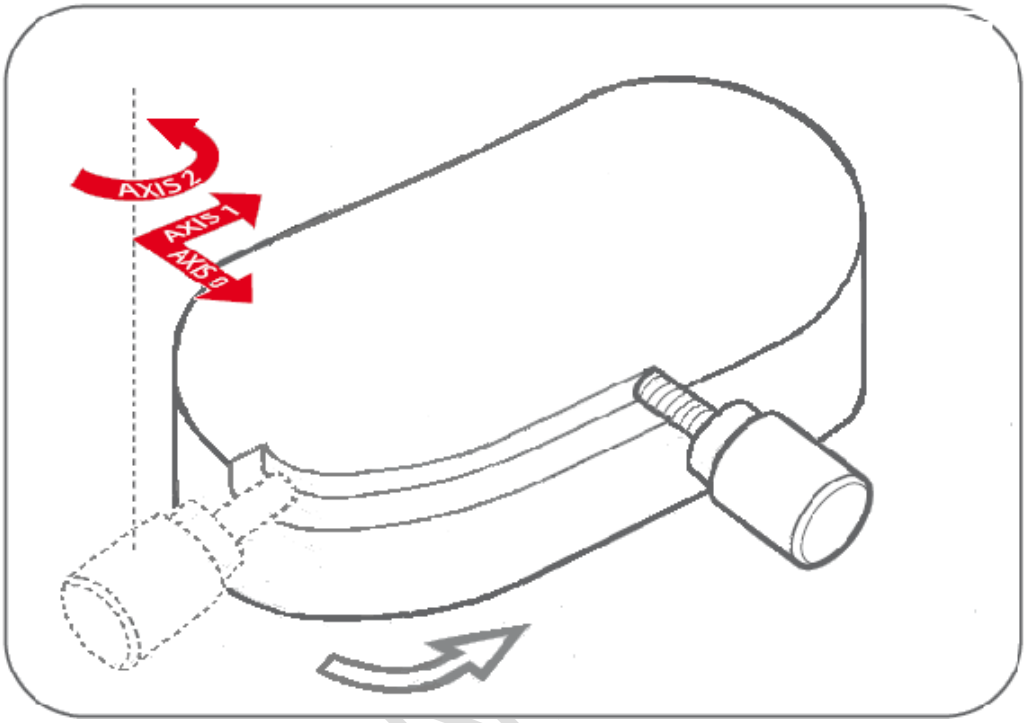


圖 3.136

DELTA CONFIDENTIAL

```
/* MotionStep1.為將三軸平移至加工起始點 */

        _DMC_01_start_sa_move(CardNo, NodeID3[0], SlotID3[0], 0, 0, MaxVel, 0.1, 0.1);
        _DMC_01_start_sa_move(CardNo, NodeID3[1], SlotID3[1], 0, 0, MaxVel, 0.1, 0.1);
        _DMC_01_start_sa_move(CardNo, NodeID3[2], SlotID3[2], -13500, 0, MaxVel, 0.1,
0.1);

/* MotionStep2.等待 Motion Done，並確定 Step1 完成 */
        for(i=0;i<3;i++)
        {
            rt[i] = _DMC_01_get_command(CardNo, NodeID3[i], SlotID3[i], &Cmd[i]);
        }
        if((rt[0] + rt[1] + rt[2] == 0) && (Cmd[0] == 0) && (Cmd[1] == 0) && (Cmd[2] == -13500))

        {
            MotionStep = 3;
        }

         /* MotionStep3. 開始加工 */
        _DMC_01_start_tr_move_xy(CardNo, NodeID2, SlotID2, 40000, 40000, MaxVel/2, MaxVel, 0.1,
0.1);
                                _DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, -10000,
10000, -Pitch*(180.0/360),
                                Pitch,1, MaxVel/2, MaxVel, 0.1, 0.1);
                                  _DMC_01_start_tr_move_xy(CardNo, NodeID2, SlotID2, -40000,
                                -40000, MaxVel/2, MaxVel,
0.1,0.1);
                                  _DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 10000,
                                -10000, - Pitch *(180.0/360), Pitch, 1, MaxVel/2, MaxVel,0.1,0.1);
```

DELTA CONFIDENTIAL

例子 2：此例是透過 X 軸與 Y 軸，使得工具沿圓角及直線移動，並且讓 Z 軸的工具始終與圖形保持垂直(Normal)或切線(Tangent)。如圖 3.137 (下例解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

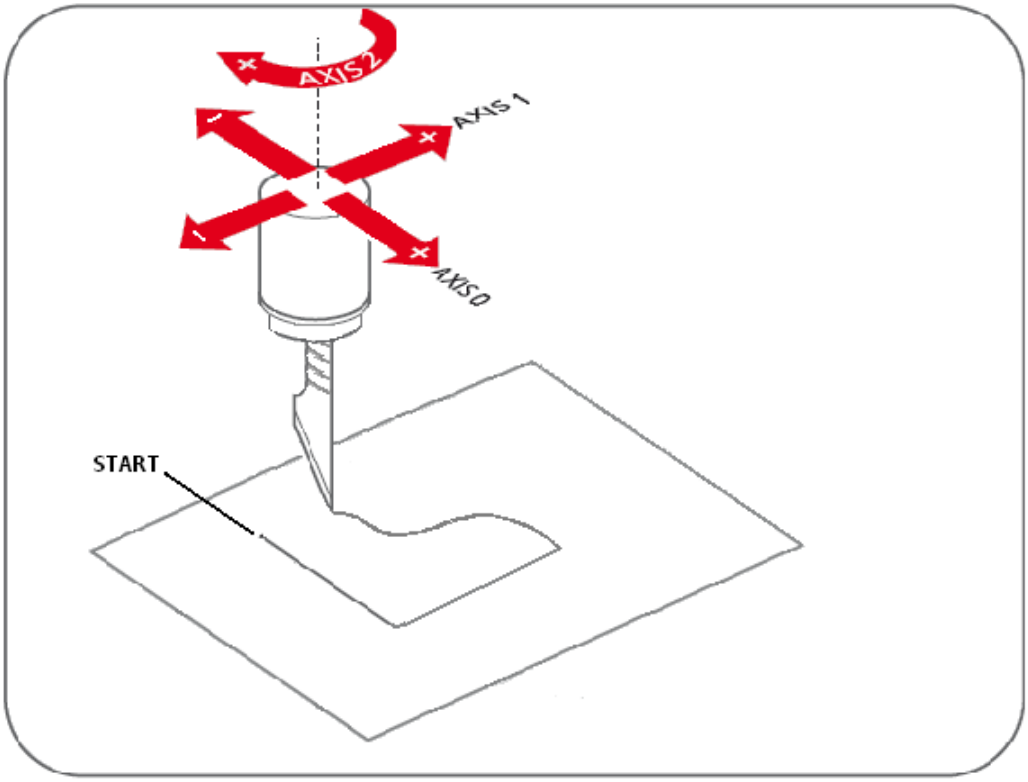


圖 3.137

```
/* MotionStep1.為將三軸平移至加工起始點 */
        _DMC_01_start_sa_move(CardNo, NodeID3[0], SlotID3[0], 0, 0, MaxVel, 0.1, 0.1);
        _DMC_01_start_sa_move(CardNo, NodeID3[1], SlotID3[1], 0, 0, MaxVel, 0.1, 0.1);
        _DMC_01_start_sa_move(CardNo, NodeID3[2], SlotID3[2], 0, 0, MaxVel, 0.1, 0.1);

        /* MotionStep2.等待 Motion Done，並確定 Step1 完成 */
        for(i=0;i<3;i++)
        {
            rt[i] = _DMC_01_get_command(CardNo, NodeID3[i], SlotID3[i], &Cmd[i]);
        }

        if((rt[0] + rt[1] + rt[2] == 0) && (Cmd[0] == 0) && (Cmd[1] == 0) && (Cmd[2] == 0))
        {
            MotionStep = 3;
        }
```

```
/* MotionStep3. 開始加工第一步(直線) */
_DMC_01_start_ta_move_xy(CardNo, NodeID2, SlotID2, 30000, 0, MaxVel/2, MaxVel, 0.1, 0.1);

/* MotionStep4. 等候加工第一步執行結束 */

rt[0] = _DMC_01_get_command(CardNo, NodeID3[0], SlotID3[0], &Cmd[0]);
rt[1] = _DMC_01_get_command(CardNo, NodeID3[1], SlotID3[1], &Cmd[1]);
if ((rt[0] + rt[1] == 0) && (Cmd[0] == 30000) && (Cmd[1] == 0))
{
    MotionStep = 5;
}

/* MotionStep5. 旋轉 Z 軸使工具保持切線(或法線) */
                    rt[0] = _DMC_01_start_tr_move(CardNo, NodeID3[2],
                SlotID3[2], -Pitch*(90.0/360.0), MaxVel/2, MaxVel, 0.1, 0.1);

/* MotionStep6. 等候 Z 軸工具定位(動作結束) */
rt[2] = _DMC_01_get_command(CardNo, NodeID3[2], SlotID3[2], &Cmd[2]);
if ((rt[2] == 0) && (Cmd[2] == TempSaveData))
{
    MotionStep = 7;
}

/* MotionStep7. 開始加工第二步(直線) */
                    _DMC_01_start_ta_move_xy(CardNo, NodeID2,
                SlotID2, 30000, 30000, MaxVel/2, MaxVel, 0.1, 0.1);

/* MotionStep8. 等候加工第二步執行結束 */
rt[0] = _DMC_01_get_command(CardNo, NodeID3[0], SlotID3[0], &Cmd[0]);
rt[1] = _DMC_01_get_command(CardNo, NodeID3[1], SlotID3[1], &Cmd[1]);
if ((rt[0] + rt[1] == 0) && (Cmd[0] == 30000) && (Cmd[1] == 30000))
{
    MotionStep = 9;
}

/* MotionStep9. 旋轉 Z 軸使工具保持切線(或法線) */
                    _DMC_01_start_tr_move(CardNo, NodeID3[2], SlotID3[2],
            -Pitch*(90.0/360.0), MaxVel/2, MaxVel, 0.1, 0.1);
```

DELTA CONFIDENTIAL

```
/* MotionStep10. 等候 Z 軸工具定位(動作結束) */
rt[2] = _DMC_01_get_command(CardNo, NodeID3[2], SlotID3[2], &Cmd[2]);
if ((rt[2] == 0) && (Cmd[2] == TempSaveData))
{
        MotionStep = 11;
}

/* MotionStep11. 開始加工(連續圓弧) */
                    _DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 0,
                -10000, -Pitch*(90.0/360.0), Pitch, 1, MaxVel/2, MaxVel, 0.1,
                0.1);
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, -10000, 0, Pitch *(90.0/360.0),
                                    Pitch,
                0, MaxVel/2, MaxVel, 0.1, 0.1);
                    _DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 0,
                -10000, - Pitch *(90.0/360.0), Pitch, 1, MaxVel/2, MaxVel, 0.1,
                0.1)
```

DELTA CONFIDENTIAL

例子 3：此例是透過 X 軸與 Y 軸，使用 Rline 運動指令下達轉折 90° 路徑座標。透過調整 Parameter 參數來決定轉折的平滑程度。Parameter 越大，移動路徑中距離轉捩點座標的最小距離越遠。並透過將起始速度與上一筆運動的結束速度相同，來達到運動指令銜接時向量速度連續的效果。

(下例解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

```
/* MotionStep20. 開始加工(連續圓弧) */

I32 Table1[7][12]={
{40000,      0,        0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{40000, 20000, -9000, 0, 0, 0, 0, 0, 0 ,0, 0, 0},
{40000, 40000, -9000, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{20000, 40000,-18000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 40000,-18000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 20000,-27000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 00000,-27000, 0, 0, 0, 0, 0, 0, 0, 0,0}
};
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3, NodeID3, SlotID3, 0, Table1[0], Table1[1], 0,
                                       Parameter, 1, 0, 0, 0,   MaxVel, MaxVel, 1, 0, 2, 1);
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3,   NodeID3, SlotID3,   0,   Table1[2], Table1[3], 0,
                                       Parameter,   1, 0, 0, MaxVel, MaxVel, MaxVel, 0, 0, 2, 1);
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3, NodeID3, SlotID3, 0, Table1[4], Table1[5], 0,
                                       Parameter, 1,   0, 0, MaxVel, MaxVel, MaxVel, 0, 0, 2, 1);
rt[0] = _DMC_01_start_v3_multi_axes(gDMCCardNo, 3, NodeID3, SlotID3, Table1[6], MaxVel, MaxVel, 0, 0, 1, 1,
                                       1);

/* MotionStep21. 等待運動指令結束*/
rt[0] = _DMC_01_get_buffer_length(gDMCCardNo, gpNodeID[0], 0, &bufferLength);
if (bufferLength==0)
{
    rt[0] = _DMC_01_motion_done(gDMCCardNo, gpNodeID[0], 0, &i);
    if(i==0)
    {
        MotionStep = 0;
    }
}
```

DELTA CONFIDENTIAL

3.14 電子凸輪

3.14.1 函式列表

表 3.14

函式名稱
_DMC_01_enable_electcam

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.14.2 流程圖

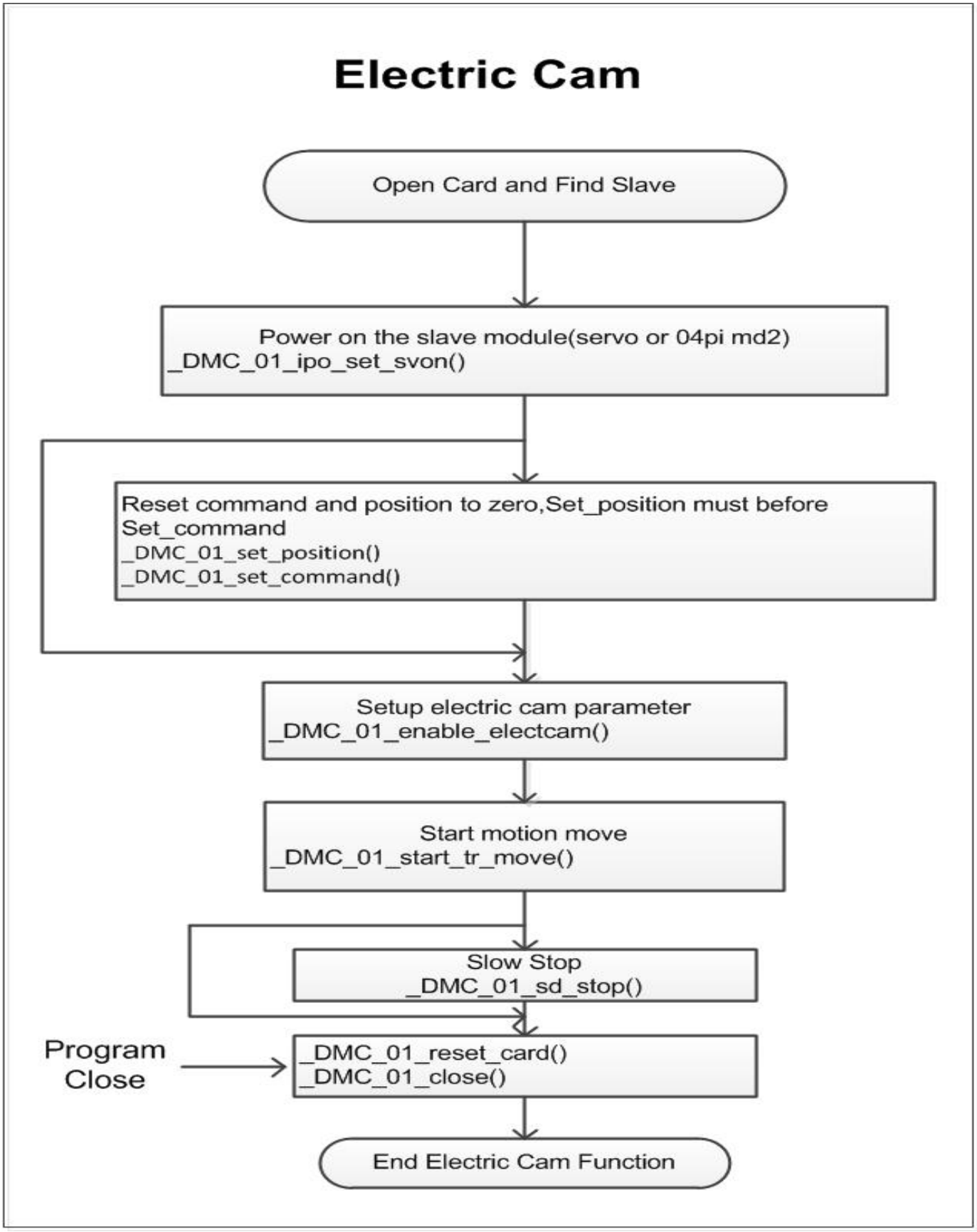


圖 3.140

3.14.3 應用範例

程式外觀

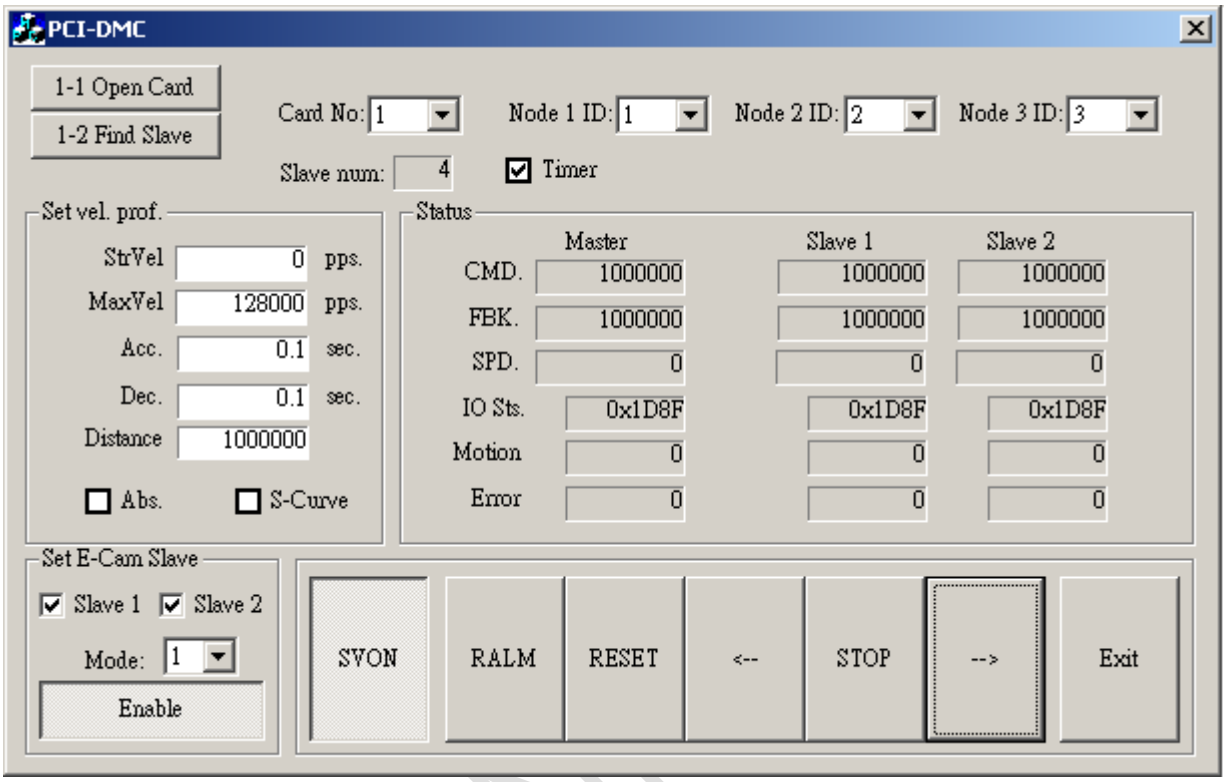


圖 3.141

1) 軸卡開啟與初始化

按壓圖 3.141 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.141 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

圖 3.142

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入運動控制的引數內容值

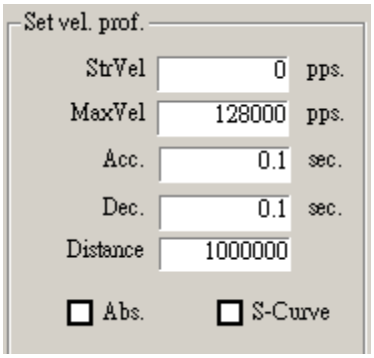


圖 3.143

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定 Electric Cam 相關參數

圖 3.144

Slave1 核取方塊：設定第一跟隨軸資料。API 函數的引數變數 “axisbit”。
Slave2 核取方塊：設定第二跟隨軸資料。API 函數的引數變數 “axisbit”。
Mode 項目：設定跟隨主軸的 Command 或 Position。API 函數的引數變數 “mode”。

按壓圖 3.144 中的「Enable」鍵開始執行以下程序：

```
rt = _DMC_01_enable_electcam(gDMCCardNo, NodeID, SlotID , enable,
axisbit,
mode); //mode 0：跟隨主軸 Command  1：跟隨主軸 Position
```

DELTA CONFIDENTIAL

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.145

按壓圖 3.145 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 按壓圖 3.145 中的「→」或「←」鍵開始執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

DELTA CONFIDENTIAL

7) 監控顯示

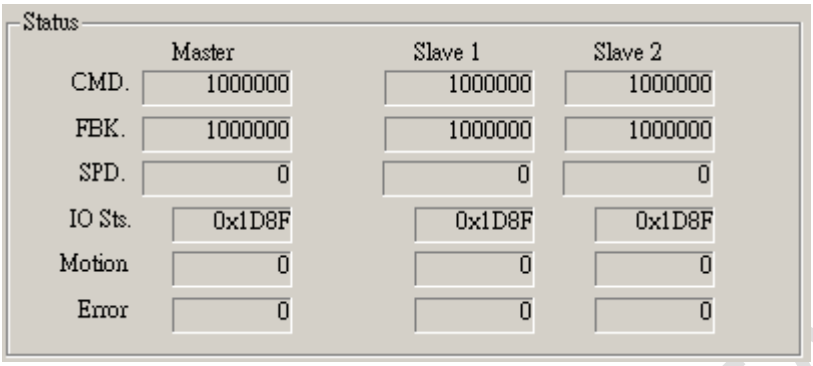


圖 3.146

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.145 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.145 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.146 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.145 中的「RALM」

DELTA CONFIDENTIAL

鍵

開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.145 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.15 間隙補償

3.15.1 函式列表

表 3.15

函式名稱
_DMC_01_set_cam_enable
_DMC_01_set_cam_mode
_DMC_01_set_cam_pitch
_DMC_01_set_cam_org
_DMC_01_set_cam_table

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.15.2 流程圖

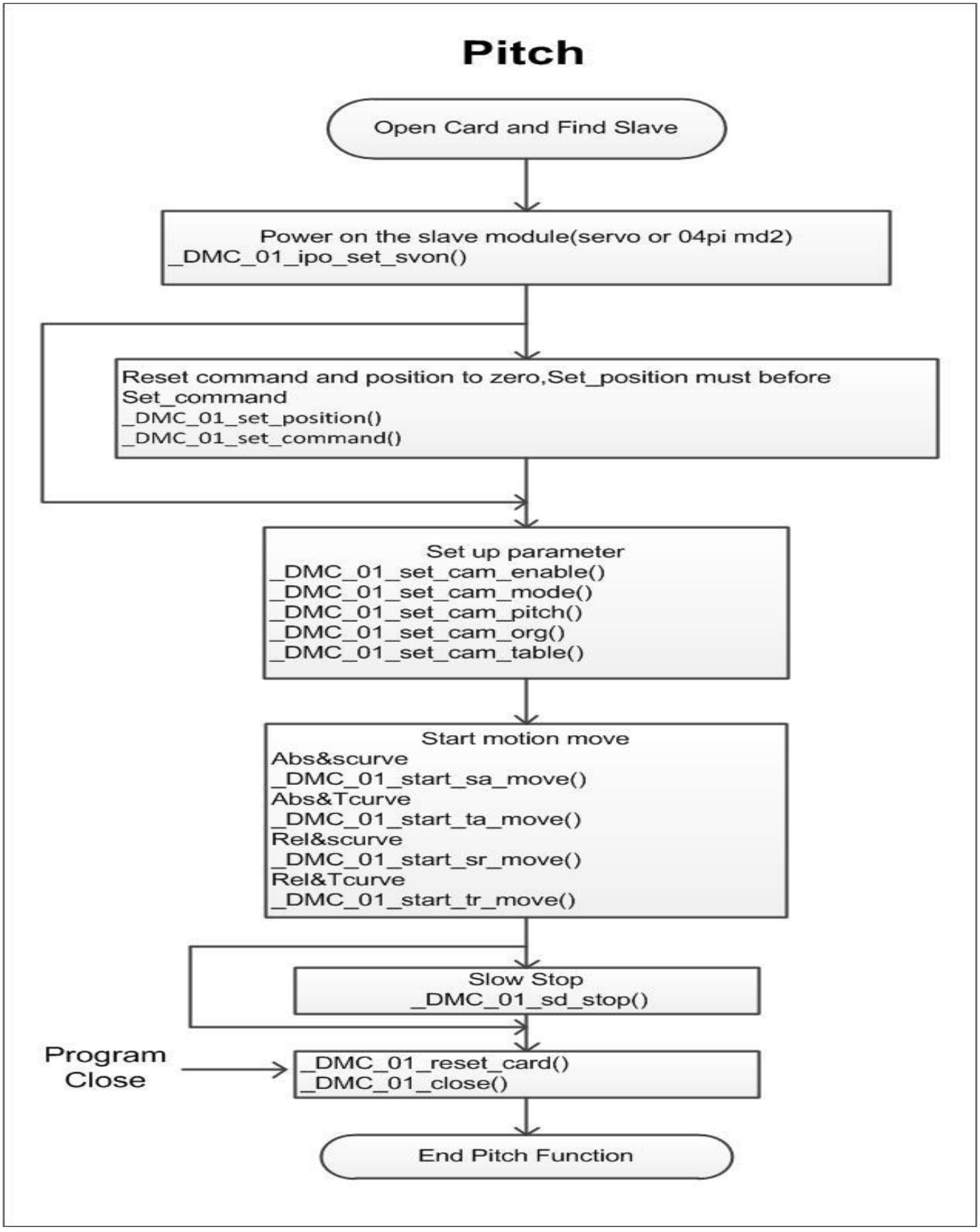


圖 3.150

3.15.3 應用範例

程式外觀

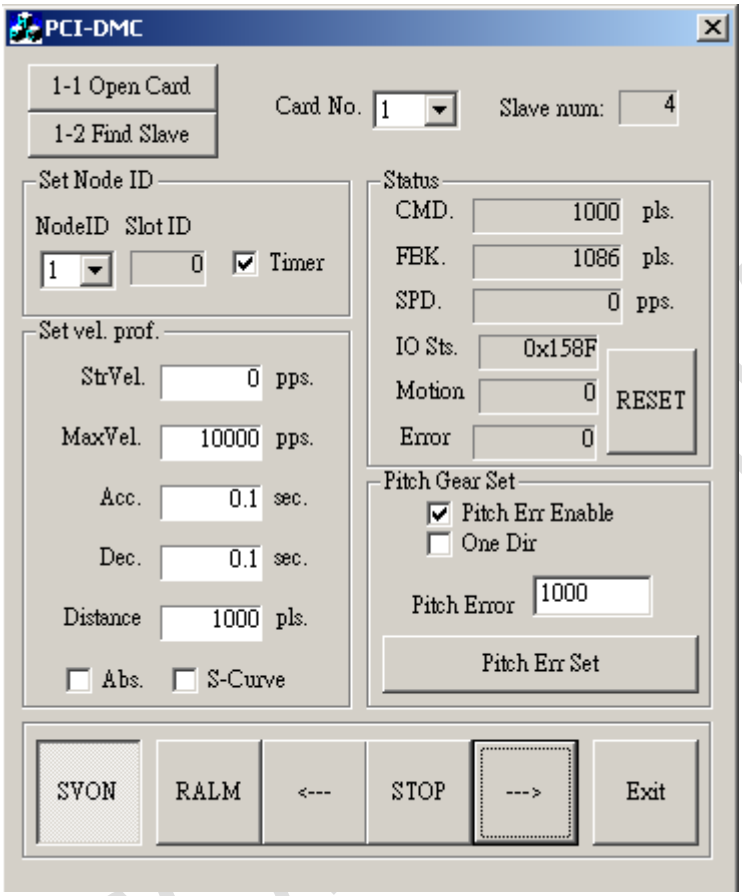


圖 3.151

1) 軸卡開啟與初始化

按壓圖 3.151 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.151 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

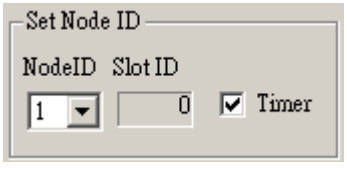


圖 3.152

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

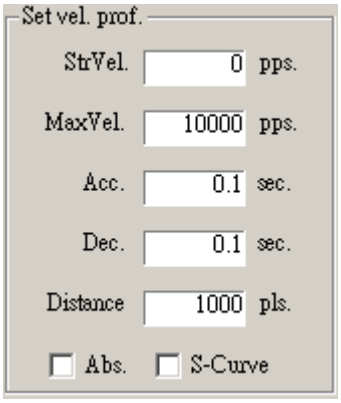


圖 3.153

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.154

按壓圖 3.154 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);

// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

DELTA CONFIDENTIAL

5) 設定動態補償參數

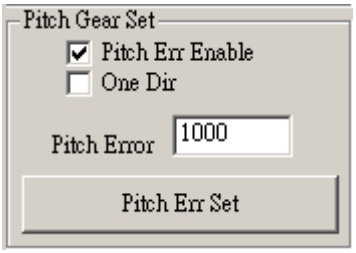


圖 3.155

Pitch Err Enable 項目：啟動/關閉間隙補償功能。API 函數的引數變數 “on_off”。

One Dir 項目：間隙補償模式(單向/雙向)。API 函數的引數變數 “Mode”。

Pitch Err 項目：間隙補償間隔(Pulse)。API 函數的引數變數 “pitch”。

按壓圖 3.155 中的「Pitch Err Set」鍵開始執行以下程序，並且在 C 槽留下記錄間隙補償資料表的檔案(PitchErrLog.txt):

```
rt = _DMC_01_set_cam_enable(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 關閉間隙補償；1 –啟動間隙補償
rt = _DMC_01_set_cam_mode(gDMCCardNo, NodeID, SlotID , Mode);
// Mode: 0 – 設定雙向表格，來回用不同表格做間隙補償；
         1 – 設定單向表格，來回都用同一表格做間隙補償。
rt = _DMC_01_set_cam_pitch(gDMCCardNo, NodeID, SlotID , pitch);
// pitch: 設定間隙補償間隔(Pulse)
rt = _DMC_01_set_cam_org(gDMCCardNo, NodeID, SlotID, Dir, OrgPos);
// Dir: 0 – 設定正向間隙補償原點；1 –設定負向間隙補償原點
//OrgPos: 間隙補償原點起始位置
rt = _DMC_01_set_cam_table(gDMCCardNo, NodeID, SlotID ,Dir, table);
// Dir: 0 – 設定正向間隙補償；1 –設定負向間隙補償
// table: 間隙補償參數資料
```

6) 開始進行運動控制

按壓圖 3.154 中的「→」或「←」鍵執行以下程序並根據設定進行間隙補償：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
```

DELTA CONFIDENTIAL

```
MaxVel, acc, dec);
    // 參照 S-curve 速度剖面下的相對座標進行運動位移
    rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
    // 參照 T-curve 速度剖面下的相對座標進行運動位
```

7) 重置運動計數值及監控顯示

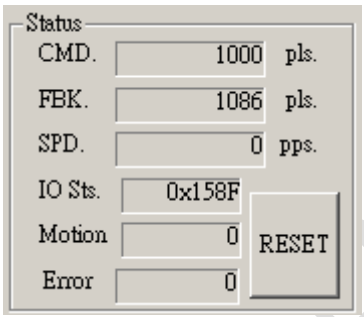


圖 3.156

按壓圖 3.156 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 停止運動及 Alm_Reset

DELTA CONFIDENTIAL

不論正在執行哪一項運動，當按壓圖 3.154 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.156 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.154 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.154 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.16 背隙補償

3.16.1 函式列表

表 3.16

函式名稱
_DMC_01_set_backlash_info
_DMC_01_set_backlash

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.16.2 流程圖

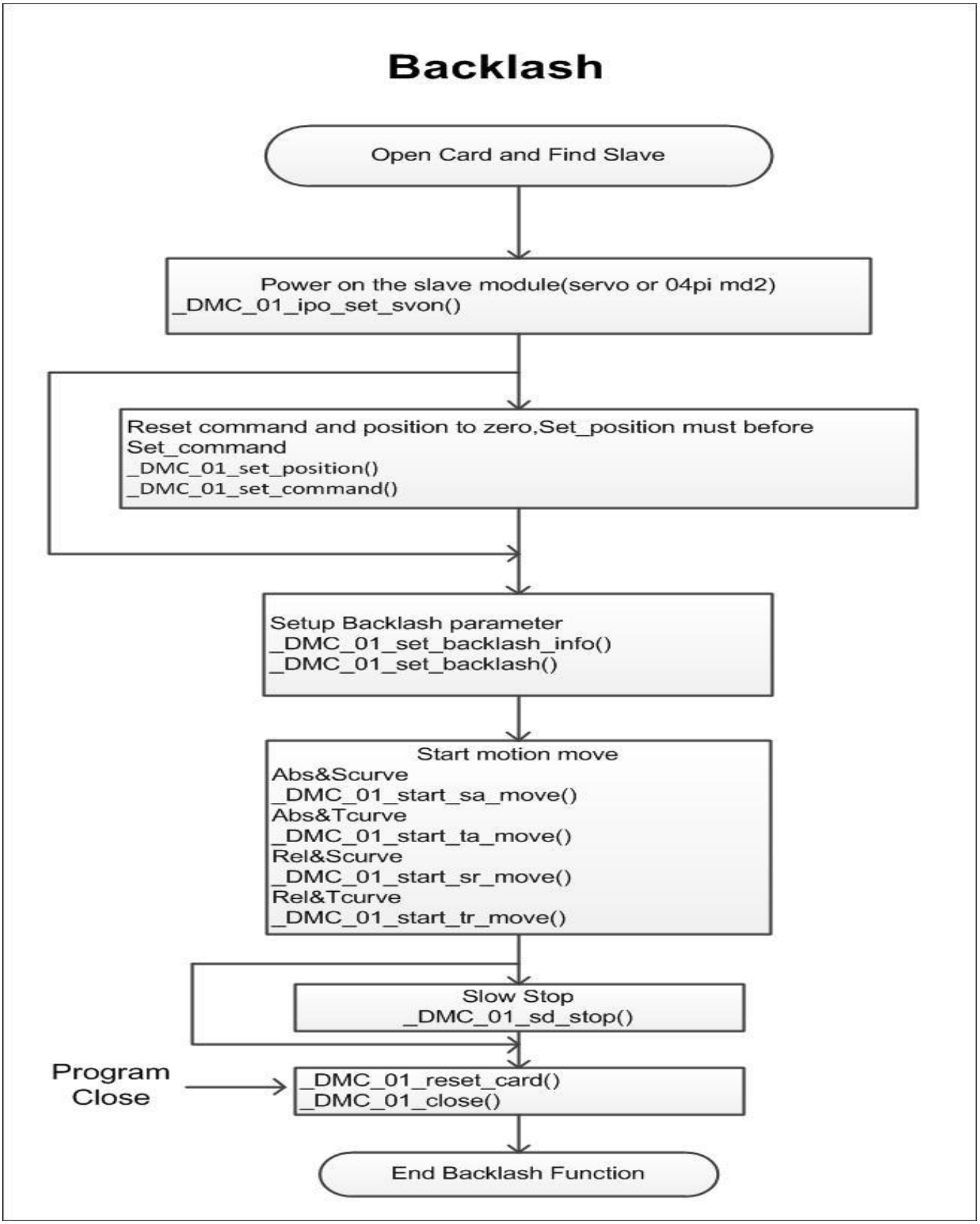


圖 3.160

3.16.3 應用範例

程式外觀

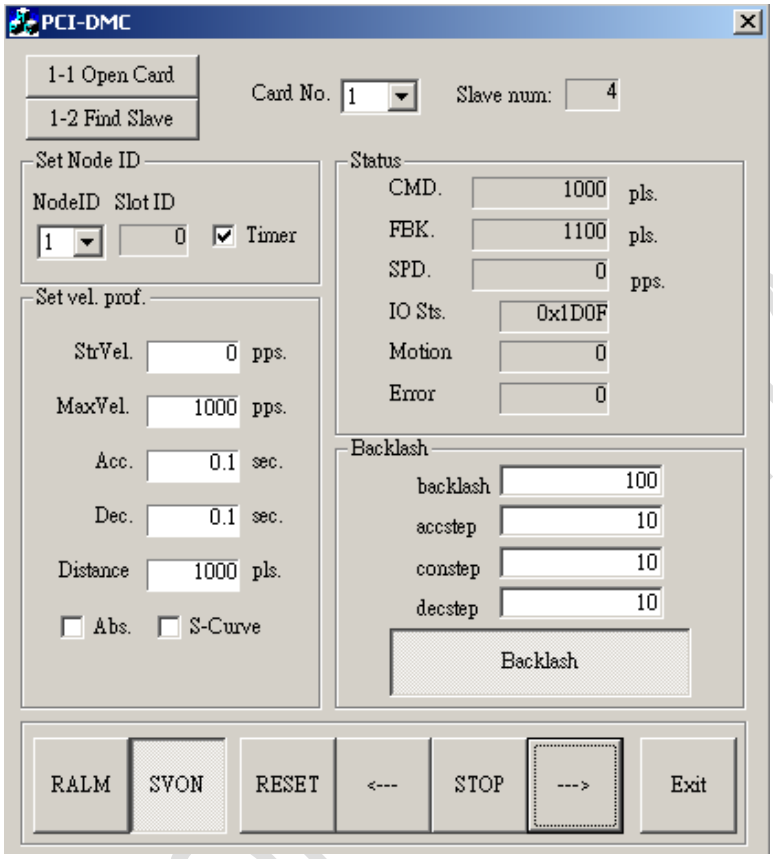


圖 3.161

1) 軸卡開啟與初始化

按壓圖 3.161 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.161 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

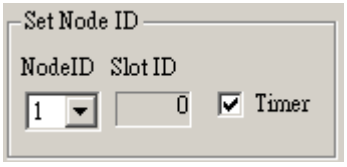


圖 3.162

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 填入運動控制的引數內容值

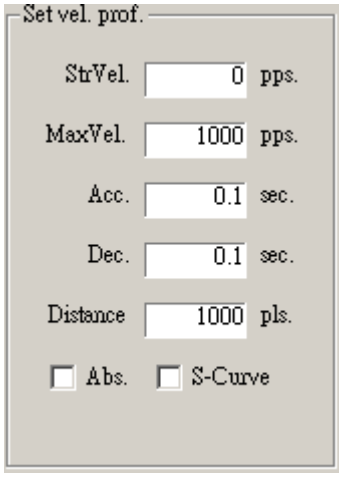


圖 3.163

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.164

按壓圖 3.164 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

DELTA CONFIDENTIAL

5) 設定背隙補償參數

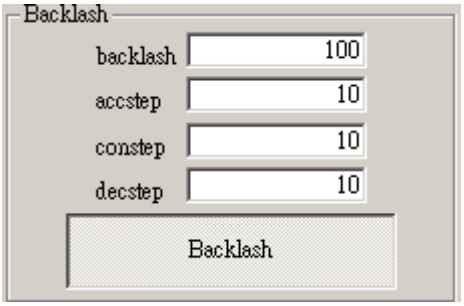


圖 3.165

按壓圖 3.165 中的「BackLash」鍵開始執行以下程序：

```
rt = _DMC_01_set_bachlash_info(gDMCCardNo, NodeID, SlotID , BackLash, accstep, constep, decstep);
// BackLash:  單位距離所要背隙補償之 Pulse 量
rt = _DMC_01_set_backlash(gDMCCardNo, NodeID, SlotID , enable);
// enable: 0 –  關閉背隙補償功能；1 –啟動背隙補償功能
```

6) 開始進行運動控制，按壓圖 3.164 中的「→」或「←」鍵執行以下程序並依據設定進行背隙補償：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移

rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位
```

DELTA CONFIDENTIAL

7) 監控顯示

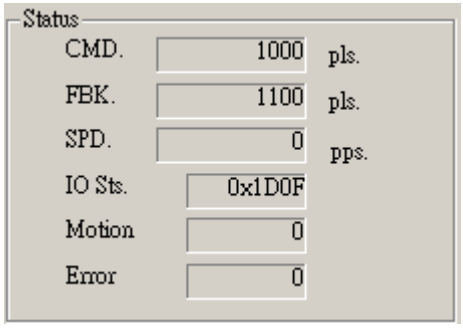


圖 3.166

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

DELTA CONFIDENTIAL

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.164 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.164 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.166 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.164 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.164 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.17 軟體極限操作

3.17.1 函式列表

表 3.17

函式名稱
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_set_soft_limit
_DMC_01_enable_soft_limit
_DMC_01_disable_soft_limit
_DMC_01_get_soft_limit_status

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.17.2 流程圖

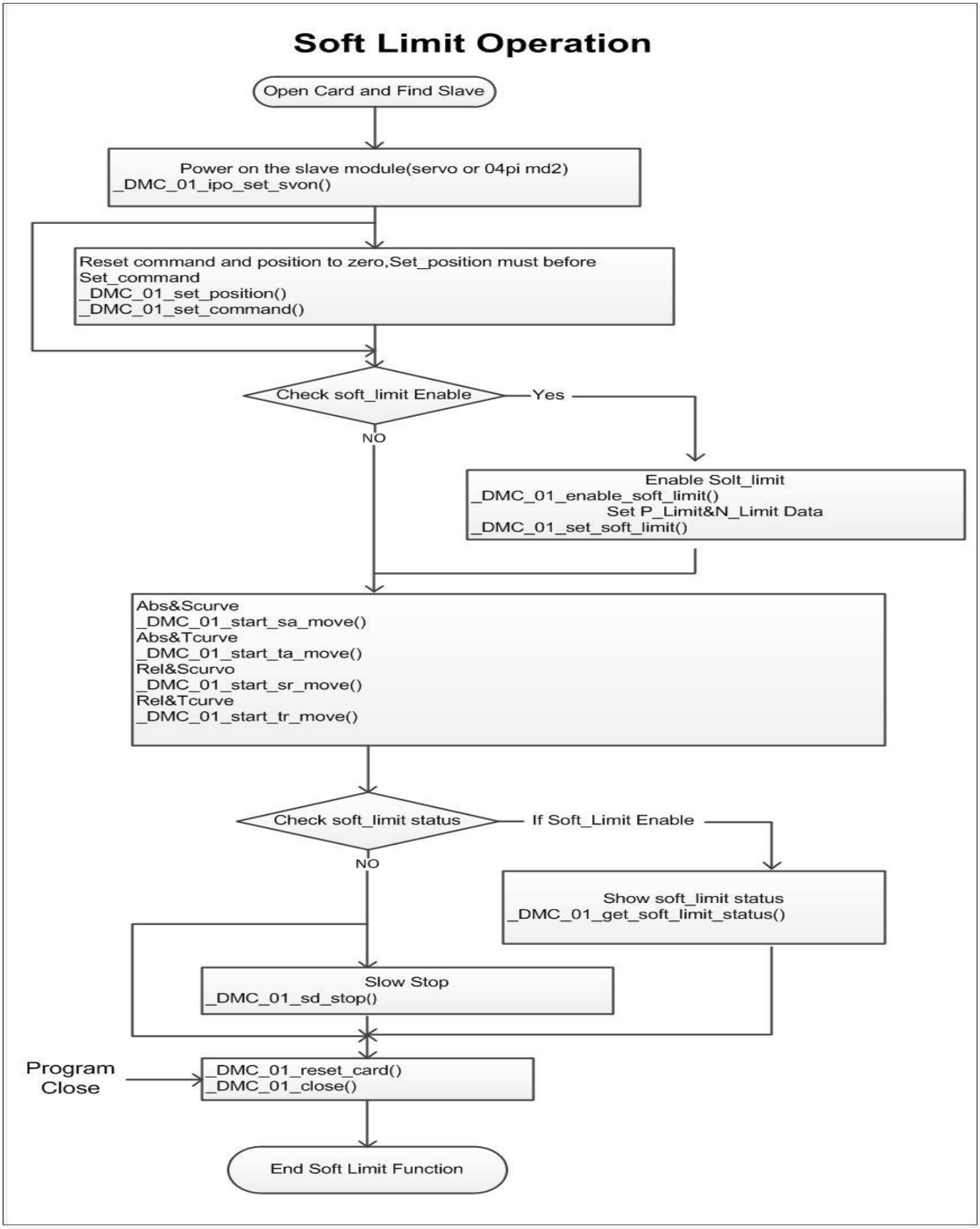


圖 3.170

3.17.3 應用範例

程式外觀

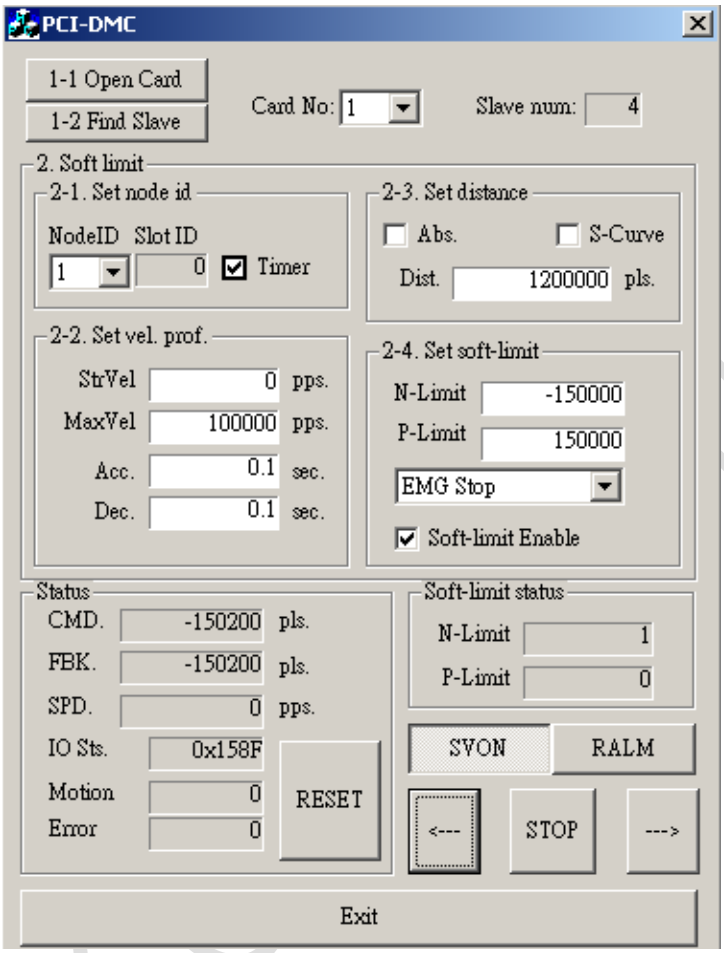


圖 3.171

1) 軸卡開啟與初始化

按壓圖 3.171 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.171 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

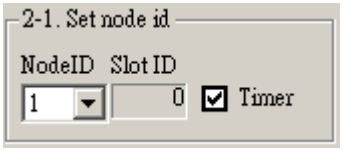


圖 3.172

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

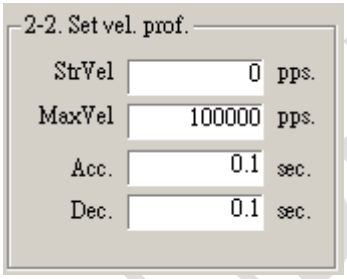


圖 3.173

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

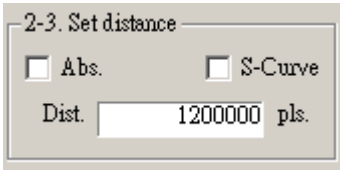


圖 3.174

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。

DELTA CONFIDENTIAL

5) 設定正/負極限值、停止運動模式與是否啟用軟體極限

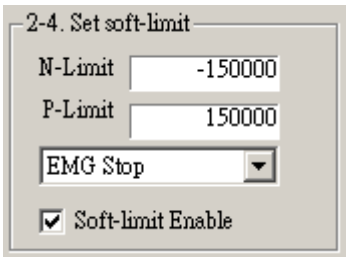


圖 3.175

N-Limit 項目：負極限值。API 函數的引數變數 “NLimit”。
P-Limit 項目：正極限值。API 函數的引數變數 “PLimit”。
Stop mode 下拉選單：停止的方式。API 函數的引數變數 “action”。
Soft-limit 核取方塊：啟動/關閉軟體極限。

6) 若您欲啟用軟體極限功能，您必須將圖 3.175 中的「Soft-limit Enable」核取方塊勾選，

並執行以下程序：

```
/* 啟動設定軟體極限 */
rt = _DMC_01_enable_soft_limit(gDMCCardNo, NodeID, SlotID, action);
  // action 引數表示碰觸到極限時，使用的停止方式。值為 1 時，表示立即停止；值為 2 時，表示使用減速停止
/* 正極限與負極限的設定值 */
rt = _DMC_01_set_soft_limit(gDMCCardNo, NodeID, SlotID, PLimit, NLimit);
// PLimit 引數表示正極限的設定值；NLimit 則為負極限的設定值
```

7) 若您欲關閉軟體極限功能時，您必須使用以下程序關閉軟體極限：

```
rt = _DMC_01_disable_soft_limit(gDMCCardNo, NodeID, SlotID);
```

8) 設定伺服馬達動力 ON/OFF(servo on/servo off)

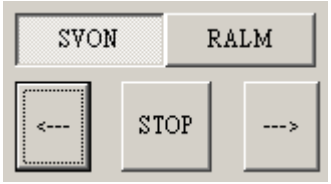


圖 3.176

按壓圖 3.176 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

DELTA CONFIDENTIAL

9) 開始進行運動控制(此處以點對點運動為例)

按壓圖 3.176 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

10) 若您欲觀察進行點對點運動時，碰觸正/負極限的狀態，您能執行以下程序進行觀察：

```
/* 觀察碰觸正/負極限所回應的狀態 */
rt = _DMC_01_get_soft_limit_status(gDMCCardNo, NodeID, SlotID, &PLimit_sts, &NLimit_sts);
```

如圖 3.177 中的所示，左圖表示此運動期間尚未碰觸到正極限與負極限；中間的圖表示此運動期間有碰觸到正極限；而右圖表示此運動期間有碰觸到負極限。

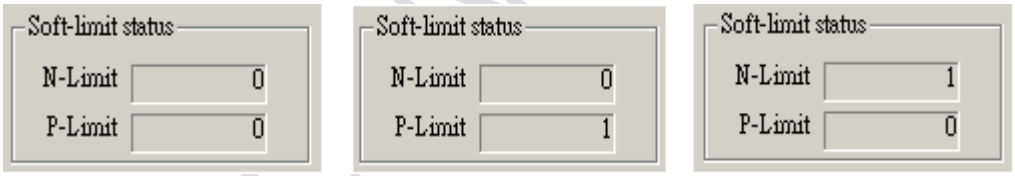


圖 3.177

11) 重置運動計數值及監控顯示

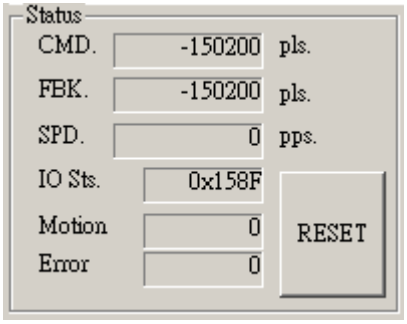


圖 3.178

DELTA CONFIDENTIAL

按壓圖 3.178 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

12) 停止運動及 Alm_Reset

不論正在執行哪一項運動，當按壓圖 3.176 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.178 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.176 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

13) 離開程序

按壓圖 3.171 中「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.18 遠端擴充 IO 模組 Input 應用(RM32MN、RM64MN)操作

3.18.1 函式列表

表 3.18

函式名稱
_DMC_01_set_rm_input_filter
_DMC_01_set_rm_input_filter_enable
_DMC_01_get_rm_input_value

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.18.2 應用範例

程式外觀

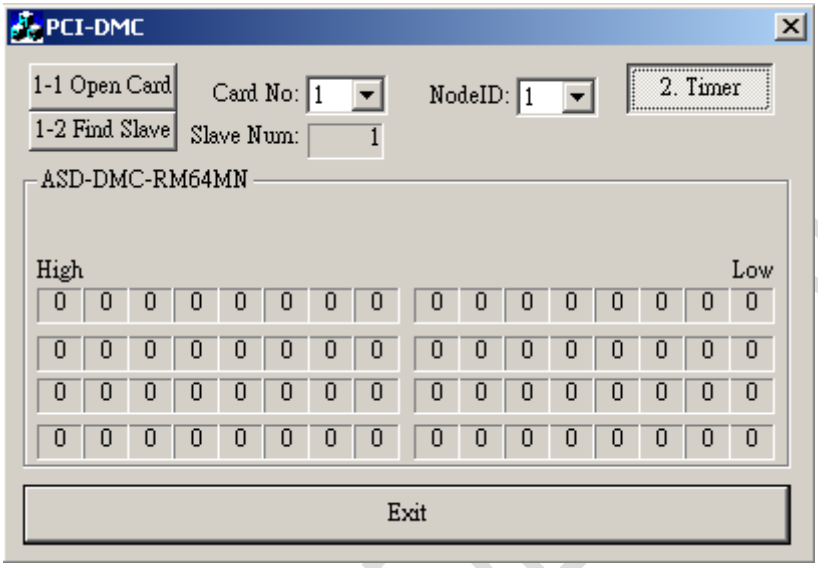


圖 3.180

1) 軸卡開啟與初始化
按壓圖 3. 180 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3. 180 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化軸卡的詳細說明，請參閱第 3.1.2 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

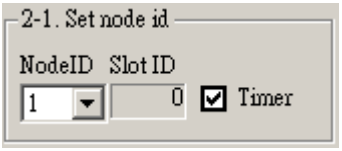


圖 3.181

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) ASD-DMC-RM32MN/ RM64MN 模組初始化參數設定。您必須執行以下程序：

```
rt = _DMC_01_set_rm_input_filter(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter);
// port 變數表示可設置 remote 模組上的 2 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1。 filter 變數是設定軟體濾波器的等級，當
```

DELTA CONFIDENTIAL

您將值設置為 0 時，軟體濾波時間為 1ms；值為 1 時，濾波時間為 2ms，依此類推。

```
rt = _DMC_01_set_rm_input_filter_enable(gDMCCardNo,
gpNodeID[gNodeNum], 0, port, filter_enable);
// filter_enable 變數的值範圍為 0~0xFFFF，祺是用來控制 Port 上 0 到 15 的軟體
濾波遮罩。
```

4) 數位輸入(DI)

當您欲從數位輸入端取得您在數位輸出模組所送出的資料，您需要使用 ASD-DMC-RM32MN/RM64MN 模組，並執行以下程序：

```
/* 取得數位輸入 Port 0 的值 RM32MN/RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 0,
&input_value[0]);
// Input_value[0]變數將會回覆從 Port 0 的 bit 0 到 bit 15 所取得的資料

/* 取得數位輸入 Port 1 的值 RM32MN/RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 1,
&input_value[1]);
// Input_value[1] 變數將會回覆從 Port 1 的 bit 0 到 bit 15 所取得的資料

/* 取得數位輸入 Port 2 的值 RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 2,
&input_value[1]);
// Input_value[1] 變數將會回覆從 Port 2 的 bit 0 到 bit 15 所取得的資料

/* 取得數位輸入 Port 3 的值 RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 3,
&input_value[1]);
// Input_value[1] 變數將會回覆從 Port 3 的 bit 0 到 bit 15 所取得的資料
```

如圖 3.182 表示 ASD-DMC-RM64MN 模組上 Port 0、1、2 無取得任何資料；而 Port 3 的 bit 0 為 ON，其餘的 bit 為 OFF。

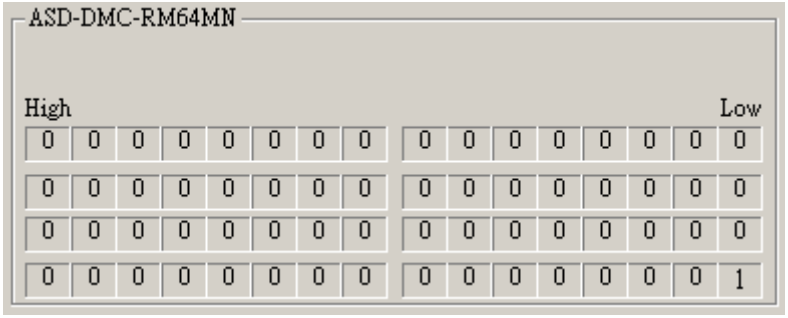


圖 3.182

DELTA CONFIDENTIAL

5) 離開程序

按壓圖 3.180 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.19 遠端擴充 IO 模組 Output 應用(RM32NT、RM64NT)操作

3.19.1 函式列表

表 3.19

函式名稱
_DMC_01_set_rm_output_active
_DMC_01_set_rm_output_value
_DMC_01_set_rm_output_value_error_handle

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.19.2 應用範例

程式外觀

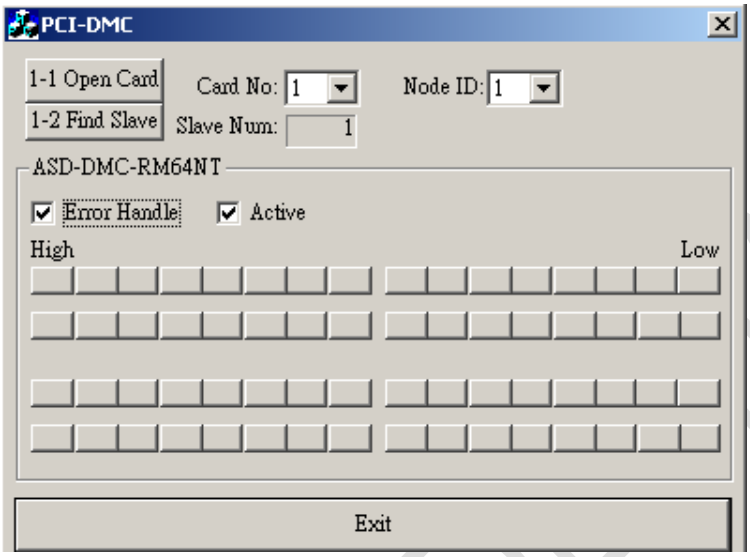


圖 3.190

1) 軸卡開啟與初始化

按壓圖 3.190 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.190 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 數位輸出(DO)操作

當您欲操作數位輸出時，首先勾選圖 3.190 中的 Active 選項並執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。
```

再來使用 ASD-DMC-RM32NT/RM64NT 模組，並執行以下程序：

```
/* 設定數位輸出 Port 0 的值 RM32NT/RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 0,
output_value[0]);
// Output_value[0]變數將存放 Port 0 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 1 的值 RM32NT/RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 1,
output_value[1]);
// Output_value[1]變數將存放 Port 1 的 bit 0 到 bit 15 欲輸出的值
```

DELTA CONFIDENTIAL

```
/* 設定數位輸出 Port 2 的值 RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 2,
output_value[1]);
// Output_value[1]變數將存放 Port 2 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 3 的值 RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 3,
output_value[1]);
// Output_value[1]變數將存放 Port 3 的 bit 0 到 bit 15 欲輸出的值
```

如圖 3.191 表示 ASD-DMC-RM64NT 模組上 Port 0、1、2、3 無輸出任何資料。

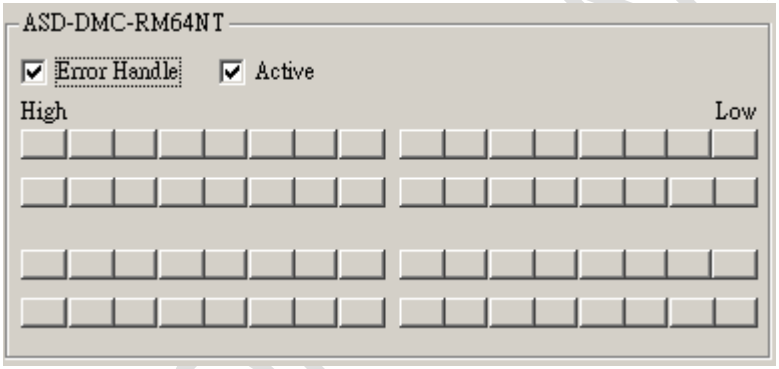


圖 3.191

3) 保留輸出埠的輸出狀態（值）

當您欲將當前的輸出，在模組電源尚未關閉前持續保留，必須將圖 3.191 中的 Error Handle 選項勾選並執行下列程序：

```
rt = _DMC_01_set_rm_output_value_error_handle(gDMCCardNo,
gpNodeID[gNodeNum], 0, port, gErrorHandle);
// port 變數表示可設置 remote 模組上的 2 組 Port，當其值為 0，將會操作模組上
的 Port 0；值若為 1，表示將操作在 Port 1。而 gErrorHandle 變數的值是設置當錯誤
發生時，系統是否保留該錯誤輸出的值。當其值為 1 時，系統將保留該輸出值，直到
模組的電源關閉或移除；若其值為 0，則錯誤輸出值將被重置為 0。
```

4) 離開程序

按壓圖 3.190 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.20 遠端擴充IO模組In-Out應用(RM32PT、GA+GE16MN/NT、RM32 MN/NT)操作

3.20.1 函式列表

表 3.20

函式名稱
_DMC_01_set_rm_input_filter
_DMC_01_set_rm_input_filter_enable
_DMC_01_set_rm_output_value_error_handle
_DMC_01_get_slave_version
_DMC_01_get_devicetype
_DMC_01_set_rm_output_value
_DMC_01_get_rm_input_value
_DMC_01_get_rm_input_single_value
_DMC_01_set_rm_output_single_value
_DMC_01_get_rm_output_single_value

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.20.2 應用範例

程式外觀

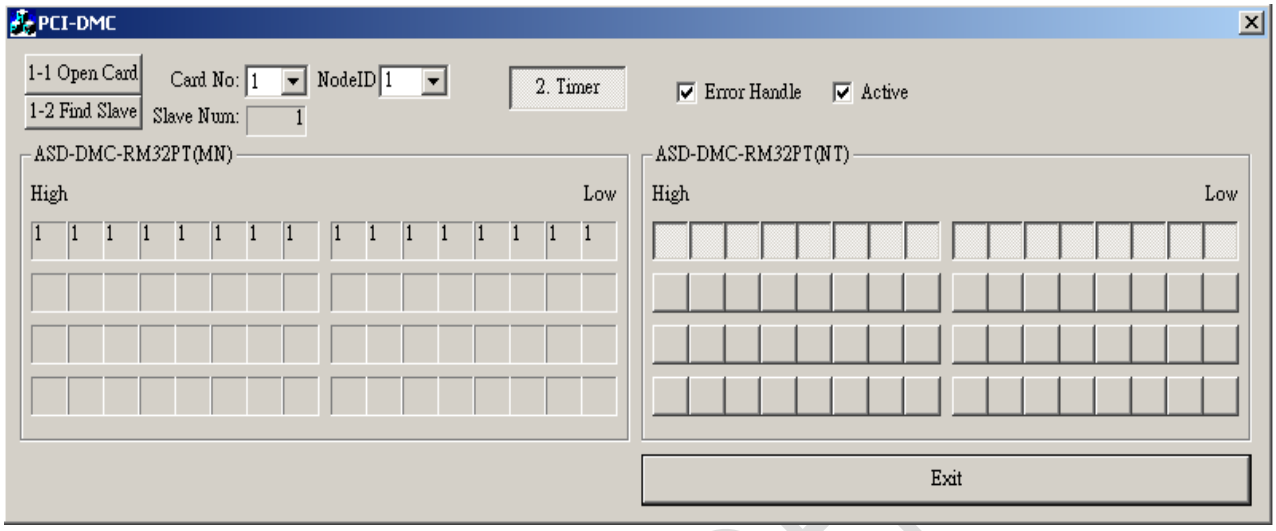


圖 3. 200

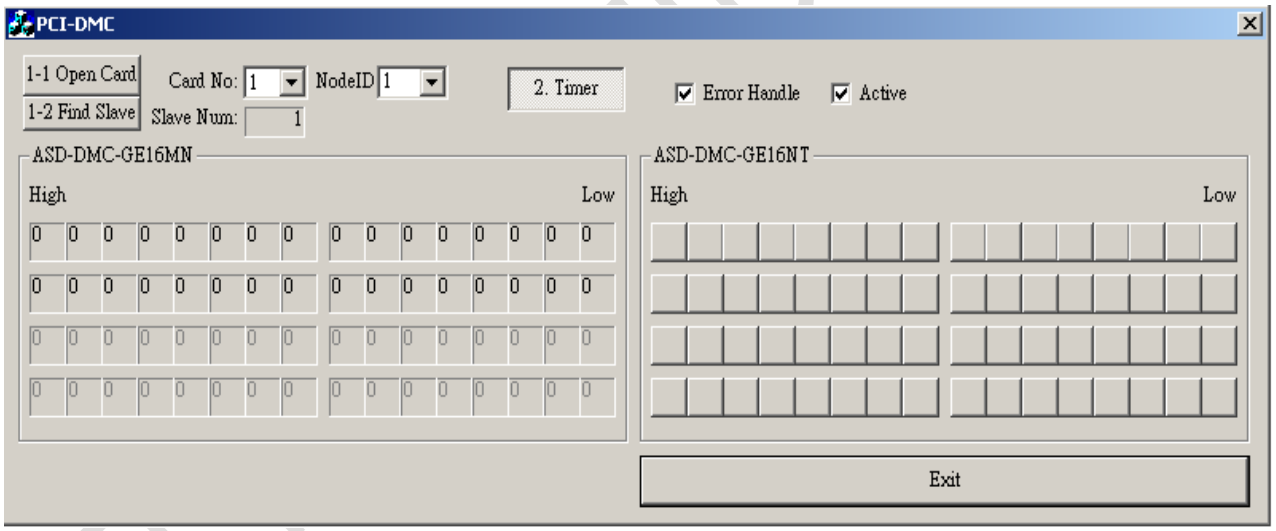


圖 3.201

1) 軸卡開啟與初始化

按壓圖 3.200 或圖 3.201 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.200 或圖 3.201 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) ASD-DMC-RM32PT/GA+GE16MN/NT 模組初始化參數設定。您可以執行以下程序：

```
rt = _DMC_01_set_rm_input_filter(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter);
// port 變數表示可設置 remote 模組上的 1~4 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1，以此類推。 filter 變數是設定軟體濾波器的等級，當您將值設置為 0 時，軟體濾波時間為 1ms；值為 1 時，濾波時間為 2ms，
依此類推。

rt = _DMC_01_set_rm_input_filter_enable(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter_enable);
// filter_enable 變數的值範圍為 0~0xFFFF，祺是用來控制 Port 上 0 到 15 的軟體濾波遮罩。
```

3) 數位輸入(DI)與數位輸出(DO)操作

當您欲操作數位輸出時，首先勾選圖 3.200 或圖 3.201 中的 Active 選項並執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。
```

再來使用 ASD-DMC-RM32PT/GA+GE16NT 模組，並執行以下程序：

```
/* 設定數位輸出 Port 0 的值 RM32PT/GA+GE16NT(最多 4 組) */
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 0, output_value[0]);
// Output_value[0]變數將存放 Port 0 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 1 的值 RM32PT/ GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 1, output_value[1]);
// Output_value[1]變數將存放 Port 1 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 2 的值 GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 2, output_value[1]);
// Output_value[1]變數將存放 Port 2 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 3 的值 GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 3, output_value[1]);
```

DELTA CONFIDENTIAL

// Output_value[1]變數將存放 Port 3 的 bit 0 到 bit 15 欲輸出的值

當您欲從數位輸入端取得您在數位輸出模組所送出的資料，您需要使用 ASD-DMC-RM32PT/GA+GE16MN 模組，並執行以下程序：

```
/*  取得數位輸入 Port 0 的值  RM32PT/GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 0, &input_value[0]);
// Input_value[0]變數將會回覆從 Port 0 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 1 的值  RM32PT/ GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 1, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 1 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 2 的值  GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 2, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 2 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 3 的值  GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 3, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 3 的 bit 0 到  bit 15 所取得的資料
```

4) 保留輸出埠的輸出狀態（值）

當您欲將當前的輸出，在模組電源尚未關閉前持續保留，必須將圖 3.200 或圖 3.201 中的 Error Handle 選項勾選並執行下列程序：

```
rt = _DMC_01_set_rm_output_value_error_handle(gDMCCardNo, gpNodeID[gNodeNum], 0, port, gErrorHandle);
// port 變數表示可設置 remote 模組上的 1~4 組 Port，當其值為 0，將會操作模組上
的 Port 0；值若為 1，表示將操作在 Port 1，以此類推。而 gErrorHandle 變數的值是
設置當錯誤發生時，系統是否保留該錯誤輸出的值。當其值為 1 時，系統將保留該輸
出值，直到模組的電源關閉或移除；若其值為 0，則錯誤輸出值將被重置為 0。
```

5) 離開程序

按壓圖 3.200 或圖 3.201 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個

DELTA CONFIDENTIAL

API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.20.3 Single_value 函式應用範例

程式外觀

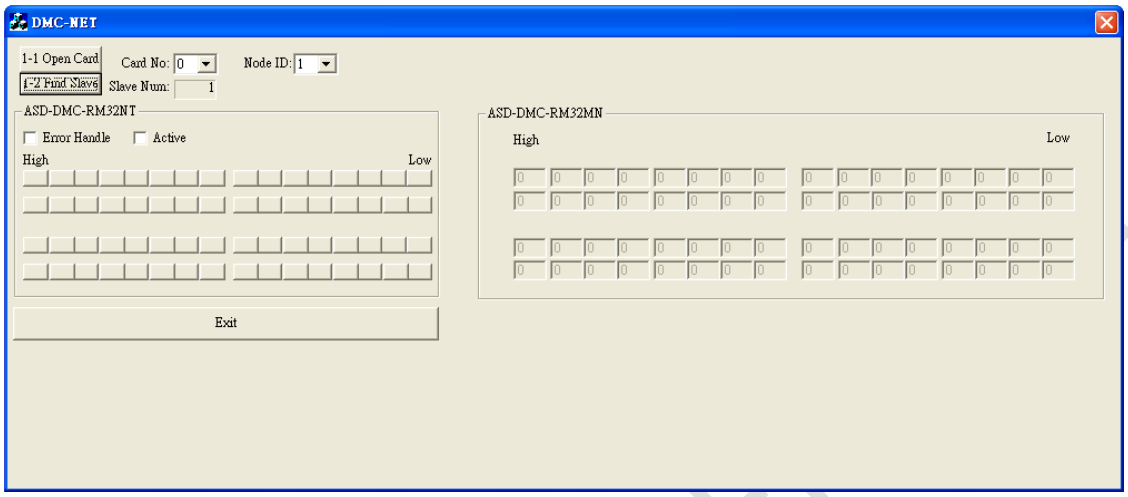


圖 3. 202

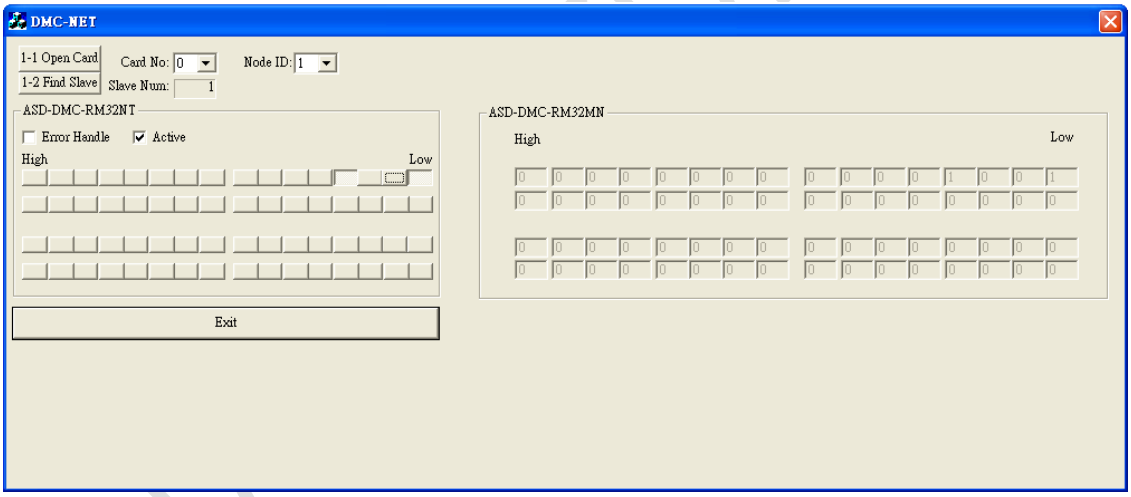


圖 3. 203

1) 軸卡開啟與初始化

按壓圖 3.202 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.202 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 數位輸出(DO)操作

當您欲操作數位輸出時，首先勾選圖 3.203 中的 Active 選項並執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。
```

再來使用 ASD-DMC-RM32NT 模組，並執行以下程序：

```
/*  設定數位輸出 Port 0 的 Bit0 與 Bit3 輸出值 RM32NT */
rt = _DMC_01_set_rm_output_single_value(U16 CardNo, U16 NodeID, U16 SlotID, 0, 0,1)
rt = _DMC_01_set_rm_output_single_value(U16 CardNo, U16 NodeID, U16 SlotID, 0, 3,1)
// value 變數將設定 Port 0 的 Bit 0 與 Bit3 欲輸出的值
```

3) 數位輸入(DI)操作

使用 ASD-DMC-RM32MN 模組，並執行以下程序：

```
/*  取得數位輸入值 RM32MN */
_DMC_01_get_rm_input_single_value(U16 CardNo, U16 NodeID, U16 SlotID, 0, 0,U16 *value);
_DMC_01_get_rm_input_single_value(U16 CardNo, U16 NodeID, U16 SlotID, 0, 3,U16 *value);
// value 變數將存放 Port 0 的 Bit 0 與 Bit3 欲輸出的值
```

4) 保留輸出埠的輸出狀態（值）

當您欲將當前的輸出，在模組電源尚未關閉前持續保留，必須將圖 3.200 或圖 3.201 中的 Error Handle 選項勾選並執行下列程序：

```
rt = _DMC_01_set_rm_output_value_error_handle(gDMCCardNo, gpNodeID[gNodeNum], 0, port, gErrorHandle);
// port 變數表示可設置 remote 模組上的 1~4 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1，以此類推。而 gErrorHandle 變數的值是設置當錯誤發生時，系統是否保留該錯誤輸出的值。當其值為 1 時，系統將保留該輸出值，直到模組的電源關閉或移除；若其值為 0，則錯誤輸出值將被重置為 0。
```

5) 離開程序

按壓圖 3.200 或圖 3.201 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.21 遠端擴充 IO 模組應用-手輪操作

3.21.1 函式列表

表 3.21

函式名稱
_DMC_01_get_rm_input_value
_DMC_01_set_rm_mpg_axes_enable2
_DMC_01_set_rm_mpg_axes_enable3

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.21.2 流程圖

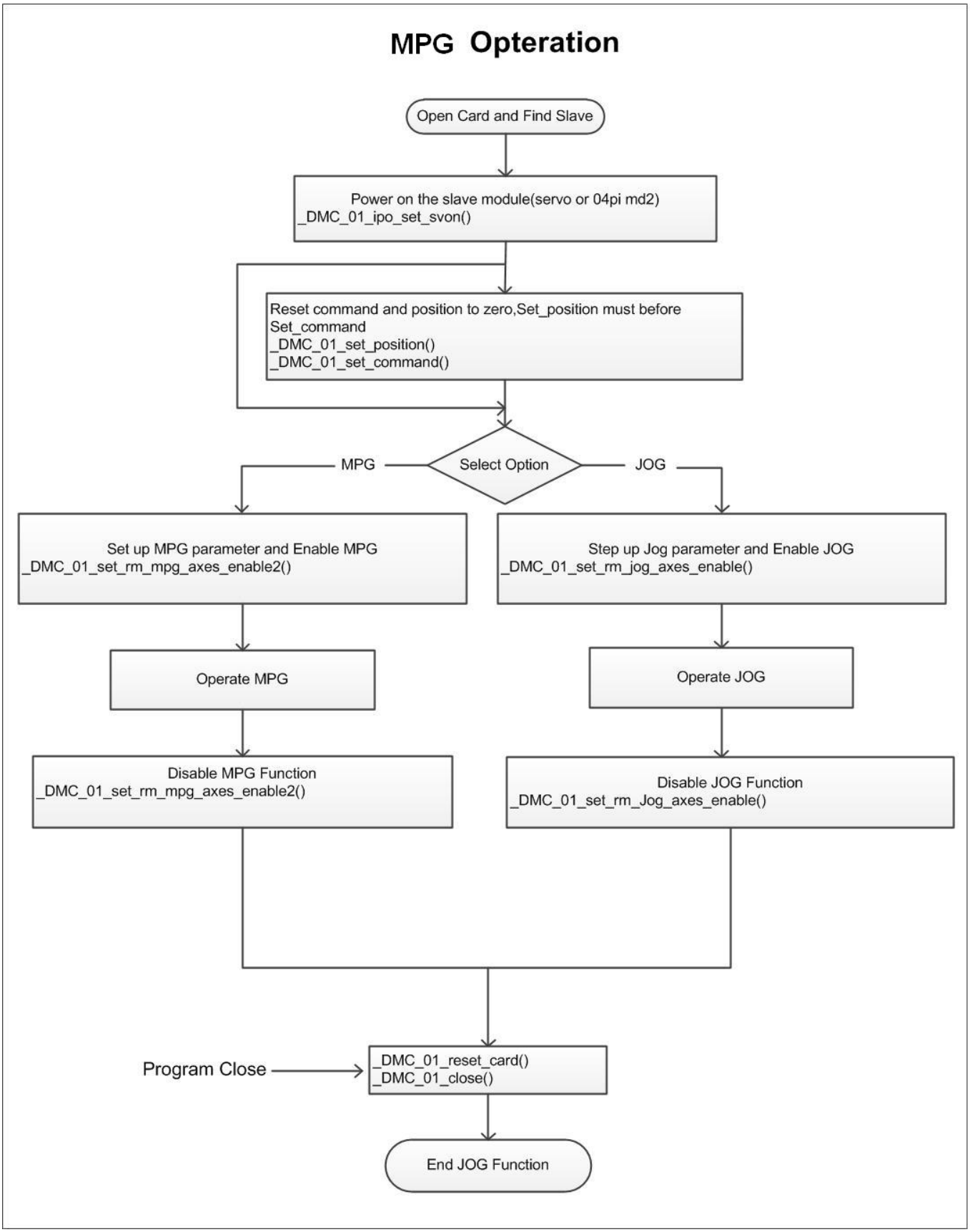


圖 3.210

3.21.3 應用範例

程式外觀

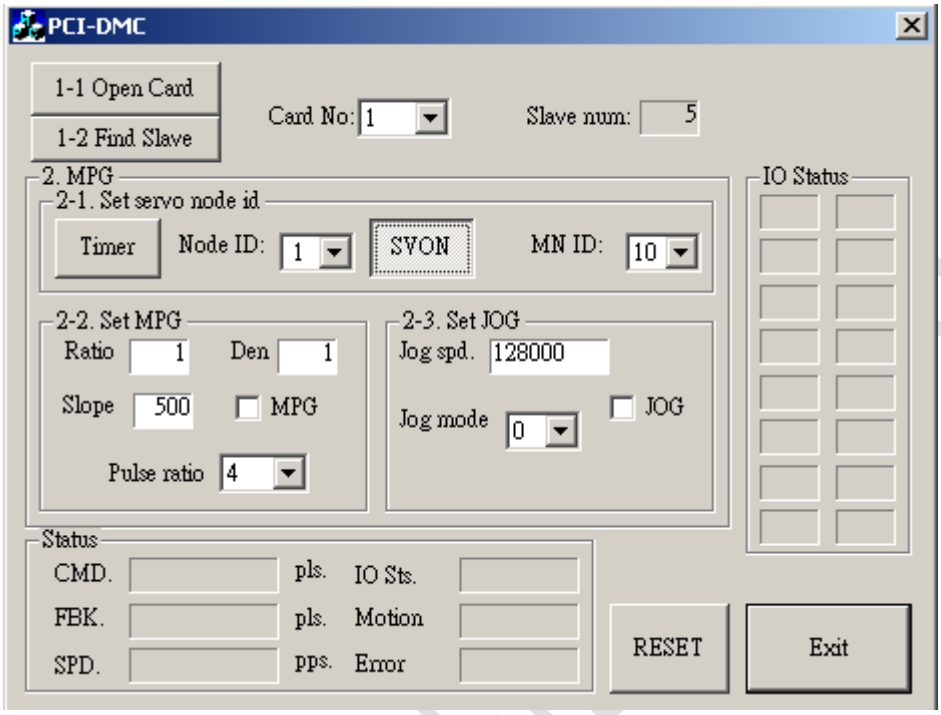


圖 3.211

1) 軸卡開啟與初始化

按壓圖 3. 211 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 211 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID、伺服馬達動力 ON/OFF 及啟用顯示運動狀態

圖 3.212

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

MN ID 項目：手輪站號設定。API 函數的引數變數 “MNNodeID”。

按壓圖 3.212 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

DELTA CONFIDENTIAL

3) 填入手輪控制的參數內容值

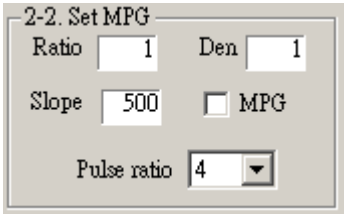


圖 3.213

Ratio 項目：手輪每轉馬達圈數比率。API 函數的引數變數 “ratio”。

Den 項目：手輪每轉馬達圈數分母。API 函數的引數變數 denominator”。

Slope 項目：手輪速度。API 函數的引數變數 “slop”。

Pulse ratio 項目：手輪每格輸出脈波數比率值。API 函數的引數變數 “pulser_ratio”。

MPG 核取方塊：要啟動手輪功能須勾選它(Ratio、Den、Slope 及 Pulse Ratio 的

參數修改，都必須要在 MPG 勾選前完成，否則不能完成參數變動)。

4) 若您欲執行「手輪運動控制」時，您必須勾選圖 3.213 中的「MPG」並執行以下程序：

```
rt = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
// 取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
rt = _DMC_01_set_rm_mpg_axes_enable2(CardNo, MasterNodeID, MasterSlotID,
NodeID, SlotID, enable, pulser_ratio, ratio, slop, denominator);
// 手輪運動控制。ratio 為每轉馬達圈數比率分子；denominator 為每轉馬達圈數比分母；藉由這兩個參數值的設定，可任意調整手輪輸出值。
```

此時，即可操作手輪。

5) 停止手輪運動控制

若您欲停止手輪的操作，請將圖 3.213 中的「MPG」核取方塊勾選取消即可停止手輪運動控制。

DELTA CONFIDENTIAL

6) 重置運動計數值

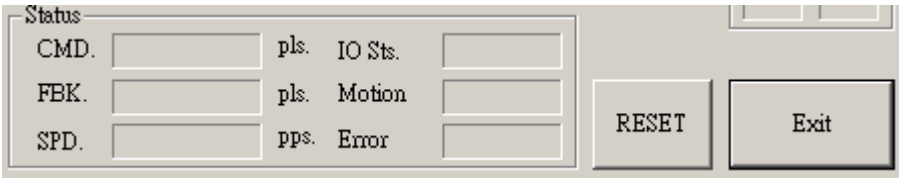


圖 3.214

若您欲將圖 3.214 中的 Command 及 Position 清除，必須將圖 3.213 中的 MPG 核取方塊勾選取消後，按壓圖 3.214 中的「RESET」鍵，執行以下程序：

```
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//將 Command 清為零
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//將 Position 清為零
```

7) 手輪 Jog 功能參數設定

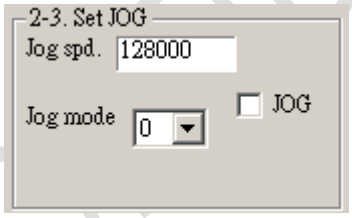


圖 3.215

Jog mode 項目：JOG 模式選擇。API 函數的引數變數 “jog_mode”。
Jog spd 項目：JOG 速度設定。API 函數的引數變數 “jog_speed”。
JOG 核取方塊：要啟動手輪 JOG 功能須勾選它(Jog mode 及 Jog speed 的參數修改，都必須要在 JOG 勾選前完成，否則不能完成參數變動)。

8) 若您欲執行「手輪 JOG 運動控制」時，您必須勾選圖 3.215 中的「JOG」並執行以下程序：

```
rt = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
// 取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
rt = _DMC_01_set_rm_jog_axes_enable(CardNo, MasterNodeID, MasterSlotID,
NodeID, SlotID, enable, jog_mode,jog_speed, sec);
```

此時，即可操作手輪 JOG 功能。

DELTA CONFIDENTIAL

9) 停止手輪運動 JOG 控制

若您欲停止手輪的操作，請將圖 3.215 中的「JOG」核取方塊勾選取消即可停止手輪運動 JOG 控制。

10) 重置運動計數值

若您欲將圖 3.214 中的 Command 及 Position 清除，必須將圖 3.215 中的 JOG 核取方塊勾選取消後，按壓圖 3.214 中的「RESET」鍵，執行以下程序：

```
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//將 Command 清為零
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//將 Position 清為零
```

11) 離開程序

按壓圖 3.214 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.22 遠端擴充脈波介面模組應用-RM04PI Mode 1 操作

3.22.1 函式列表

表 3.22

函式名稱
_DMC_01_get_buffer_length
_DMC_01_set_monitor
_DMC_01_get_monitor
_DMC_01_rm_04pi_md1_get_soft_limit_status
_DMC_01_set_rm_04pi_ipulser_mode
_DMC_01_set_rm_04pi_opulser_mode
_DMC_01_rm_04pi_md1_set_soft_limit
_DMC_01_set_rm_04pi_svon_polarity
_DMC_01_rm_04pi_md1_start_move
_DMC_01_rm_04pi_md1_v_move
_DMC_01_rm_04pi_set_MEL_polarity
_DMC_01_rm_04pi_set_PEL_polarity
_DMC_01_set_home_config
_DMC_01_set_home_move
_DMC_01_rm_04pi_md1_p_change
_DMC_01_rm_04pi_md1_v_change
_DMC_01_set_rm_04pi_DO2

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.22.2 流程圖

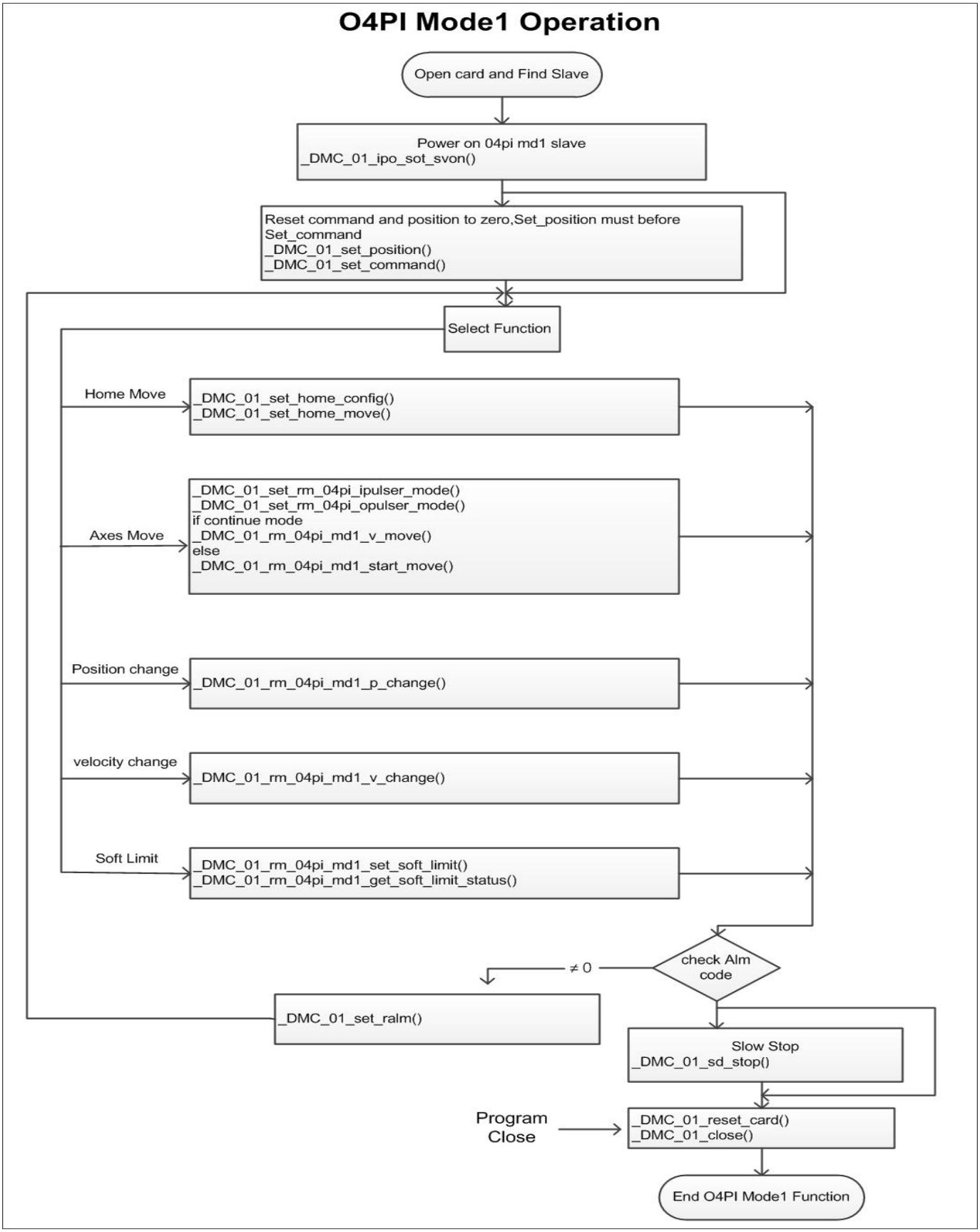


圖 3.220

3.22.3 應用範例

程式外觀

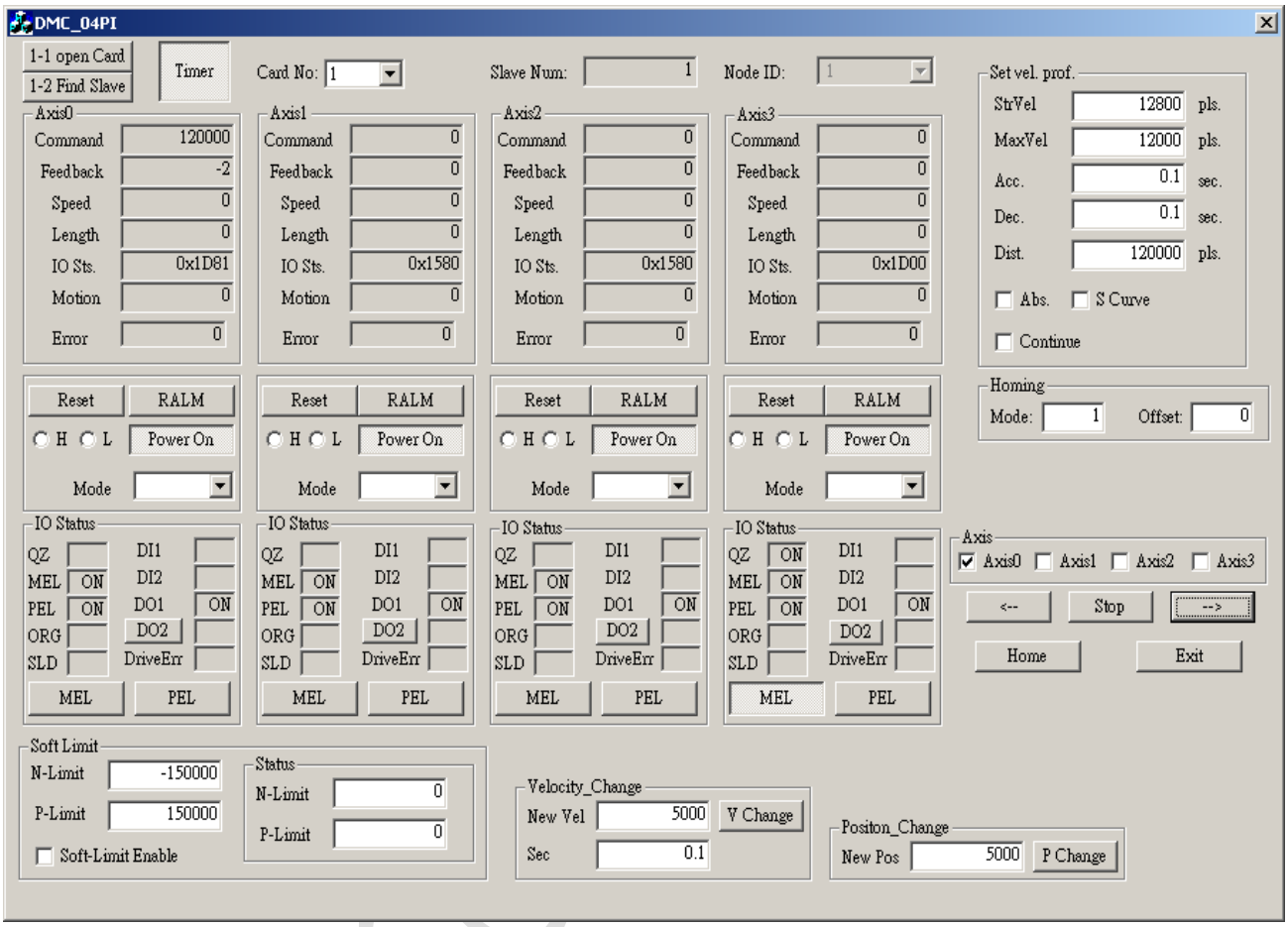


圖 3.221

1) 軸卡開啟與初始化

按壓圖 3. 221 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 221 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

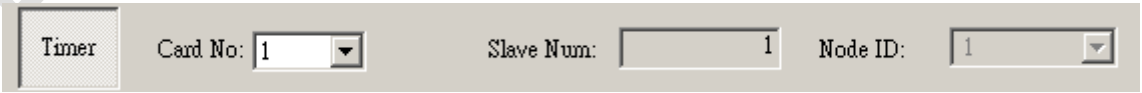


圖 3.222

填入 Node ID 並勾選「Timer」核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

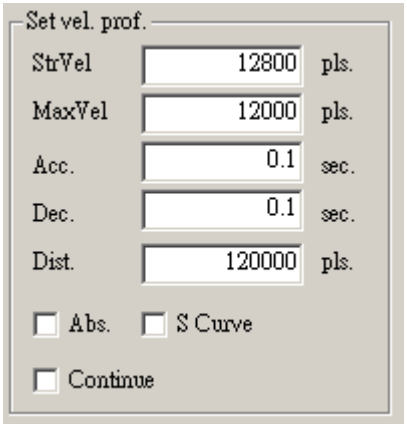


圖 3.223

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist 項目：設定運動距離。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
Continue 核取方塊：若您欲使用 Continue 運動模式，必須勾選它。

4) 選擇運動模式與設置運動行程

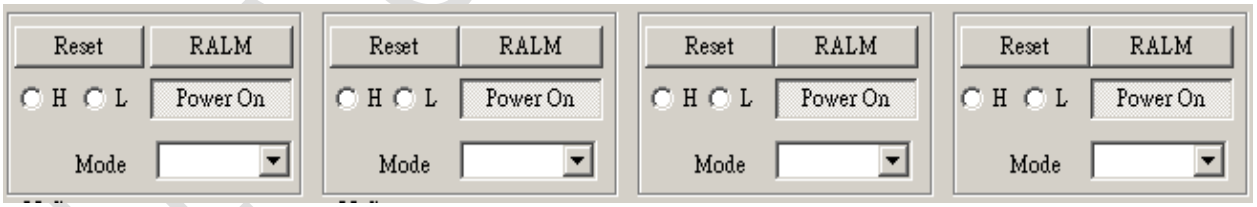


圖 3.224

Power On 核取方塊：按 Power On 啟動該軸電源。
H/L 核取方塊：選擇運行電位。
Mode 項目：輸出相位分 ABphase 及 CW/CCW 兩種。
RALM 項目：可以清除運行過程中產生得異常代碼。
Reset 項目：可以清除 Command 及 feedback 等相關顯示資料。

DELTA CONFIDENTIAL

5) 狀態顯示及正、負極性反相功能

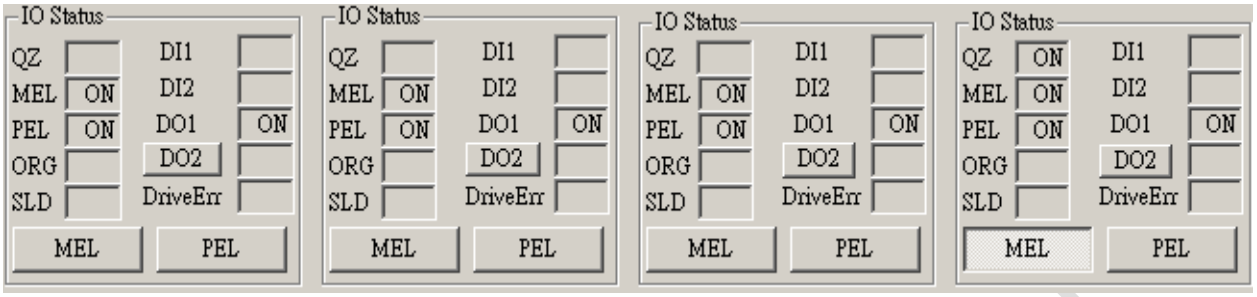


圖 3.225

MEL 核取方塊：設定將負極限反向。

PEL 核取方塊：設定將正極限反向。

按壓圖 3.225 中的「MEL」鍵開始執行以下程序：

rt = _DMC_01_rm_04pi_set_MEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);

按壓圖 3.225 中的「PEL」鍵開始執行以下程序：

rt = _DMC_01_rm_04pi_set_PEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);

6) 設定回 Home 運動模式的類別及位移植:

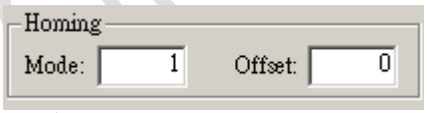


圖 3.226

Mode 項目：選擇回 Home 運動模式的類別。API 函數的引數變數 “home_mode”。

Offset 項目：設定回 Home 運動的位移植。API 函數的引數變數 “home_offset”。

7) 設定正/負極限值、是否啟用軟體極限與顯示是否碰觸軟體極限:

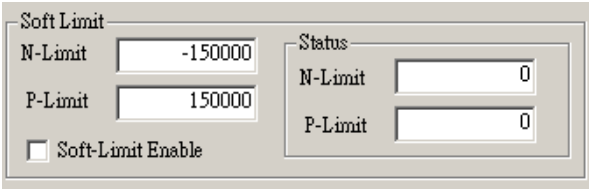


圖 3.227

P-Limit 項目：正極限值。API 函數的引數變數 “PLimit”。

DELTA CONFIDENTIAL

N-Limit 項目：負極限值。API 函數的引數變數 “NLimit”。
Soft-limit Enable 核取方塊：啟動/關閉軟體極限。
P- Limit 項目：碰觸軟體正極限狀態顯示。
N- Limit 項目：碰觸軟體負極限狀態顯示。

8) 設定欲置換的新位置值及置換啟動:

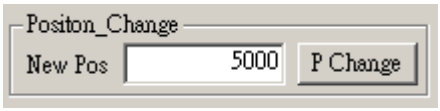


圖 3.228

New Pos 項目：新位置值。API 函數的引數變數 “NewPos”。
P Change 項目：置換啟動。

9) 設定欲置換的新速度值、加、減速時間及置換啟動:

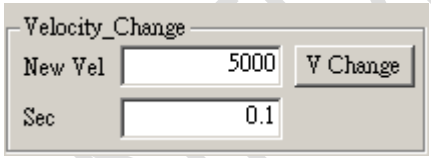


圖 3.229

New Vel 項目：設定新速度值。API 函數的引數變數 “NewSpeed”。
sec 項目：設定加、減速時間。API 函數的引數變數 “sec”。
V Change 項目：置換啟動。

10) 設定伺服馬達動力 ON/OFF(servo on/servo off)

按壓圖 3.224 中的「Power On」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

11) 選定軸號，開始進行運動控制(此處以點對點運動為例)

圖 3.22A

按壓圖 3.22A 中的「→」或「←」鍵執行以下程序：

DELTA CONFIDENTIAL

rt = _DMC_01_rm_04pi_md1_start_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a); //進行運動位移

12) 啟用軟體極限功能

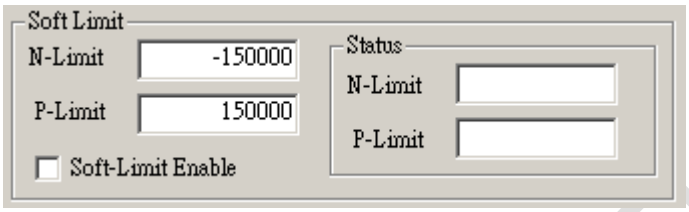


圖 3.22B

若您欲使用軟體極限功能時，必須先設定圖 3.22B 中的「P-Limit」與「N-Limit」的值，

並勾選「Soft-limit Enable」核取方塊，執行以下程序：

/* 啟動設定軟體極限 */

rt = _DMC_01_rm_04pi_md1_set_soft_limit(CardNo, NodeID, SlotID, PLimit, NLimit, Enable); // PLimit 引數表示正極限的設定值；NLimit 則為負極限的設定值

/* 軟體極限狀態顯示 */

rt = _DMC_01_rm_04pi_md1_get_soft_limit_status(CardNo, NodeID, SlotID, &NLimit_status, &PLimit_status);

※關於軟體極限範例的詳細敘述，請參閱「3.13 軟體極限」一節。

13) Countine 運動控制

若您欲進行 Countine 運動時，請勾選圖 3.223 中的「Countine」核取方塊，再按壓

圖 3.22A 中的「→」或「←」鍵執行以下程序：

/* Countine 運動 */

rt = _DMC_01_rm_04pi_md1_v_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Tdec, dir, m_curve);

DELTA CONFIDENTIAL

14) 原點復歸運動控制

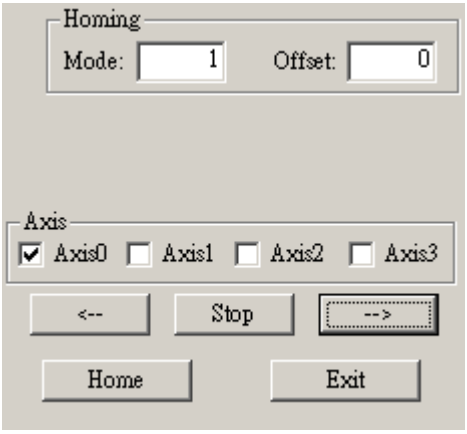


圖 3.22C

若您欲進行回 Home 運動時，請在圖 3.22C 中的 Homing 填入「Mode」及「Offset」並按壓「Home」鍵執行以下程序：

```
/* 設置原點復歸模式：1~35、偏移量與速度參數 */
rt = _DMC_01_set_home_config(gDMCCardNo, NodeID, SlotID, home_mode, home_offset, StrVel, MaxVel, acc);
/* 啟動原點復歸運動 */
rt = _DMC_01_set_home_move(gDMCCardNo, NodeID, SlotID);
```

15) 位置置換功能

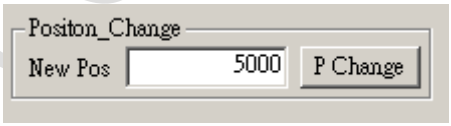


圖 3.22D

將設定的新位置值替換目前的位置，您必須按壓圖 3.22D 中的「P change」鍵執行以下程序:

```
/* P Change */
rt = _DMC_01_rm_04pi_md1_p_change(CardNo, NodeID, SlotID, NewPos);
```

DELTA CONFIDENTIAL

16) 速度置換功能

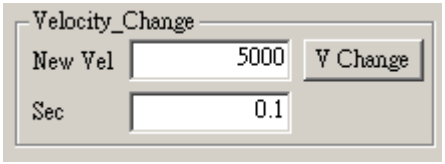


圖 3.22E

將設定的新速度值替換目前的速度，您必須按壓圖 3.22E 中的「V change」鍵執行以下程序:

```
/* P Change */
    rt = _DMC_01_rm_04pi_md1_v_change(CardNo, NodeID, SlotID,
NewsPeed,sec);
```

17) 停止運動及 Alm_Reset

不論正在執行哪一項運動，當按壓圖 3.22C 中的「Stop」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.221 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.224 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

18) 離開程序

按壓圖 3.22C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.23 遠端擴充脈波介面模組應用-RM04PI Mode 2 操作

3.23.1 函式列表

表 3.23

函式名稱
_DMC_01_start_sa_move
_DMC_01_start_ta_mov
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_set_rm_04pi_ipulser_mode
_DMC_01_set_rm_04pi_opulser_mode
_DMC_01_set_rm_04pi_svon_polarity
_DMC_01_set_monitor
_DMC_01_get_monitor
_DMC_01_send_message

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.23.2 流程圖

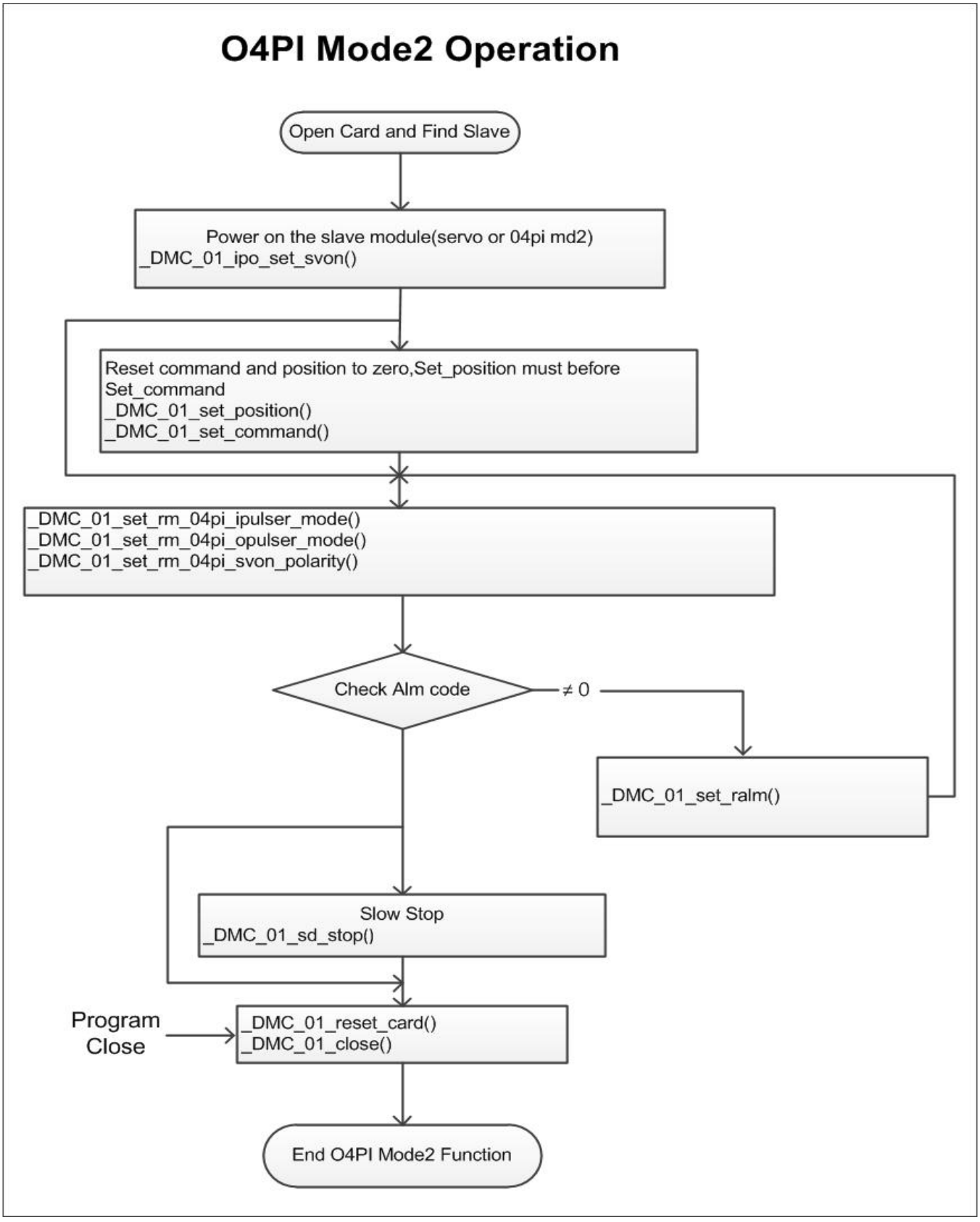


圖 3.230

3.23.3 應用範例

程式外觀

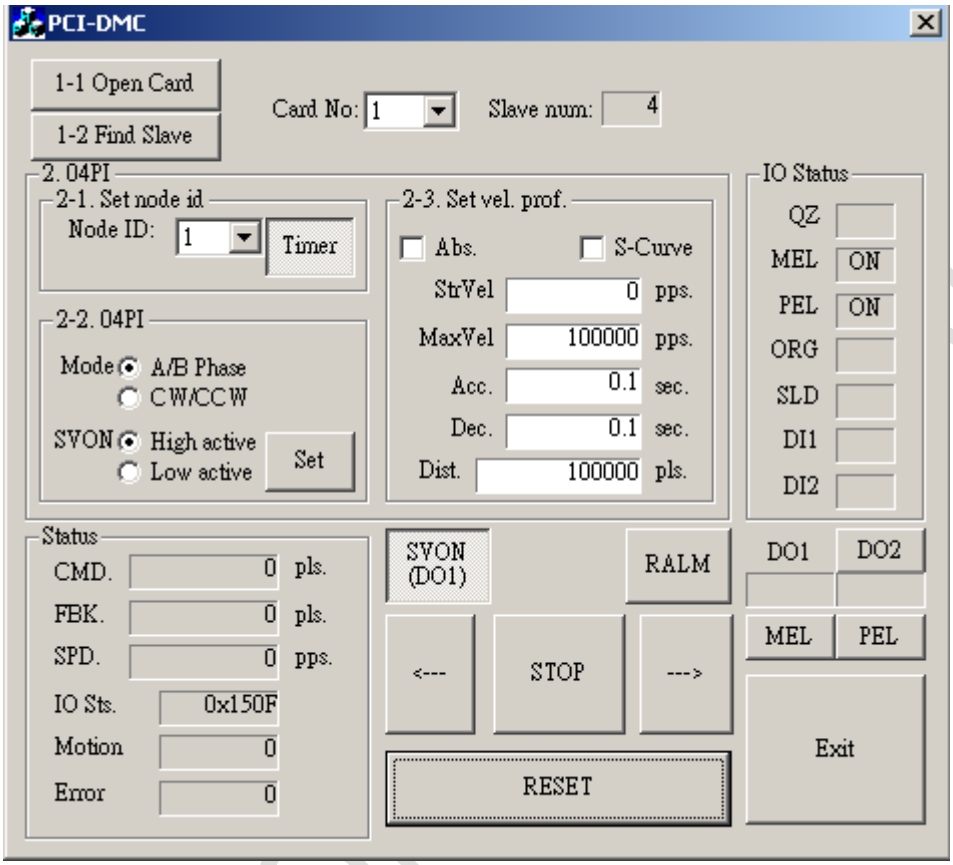


圖 3.231

1) 軸卡開啟與初始化

按壓圖 3. 231 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 231 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

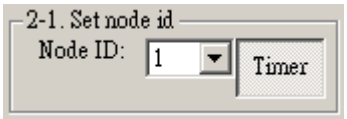


圖 3.232

填入 Node ID 並勾選「Timer」核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

DELTA CONFIDENTIAL

3) 選擇控制的引數內容值

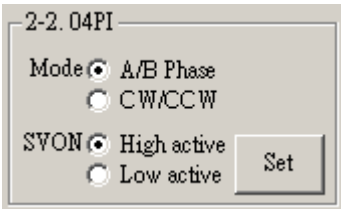


圖 3.233

AB phase 項目： 輸入/輸出模式為 AB phase 模式。
CW/CCW 項目： 輸入/輸出模式為 CW/CCW 模式。
High active 項目： 高準位時觸發。
Low active 項目： 低準位時觸發。

4) 填入運動控制的引數內容值

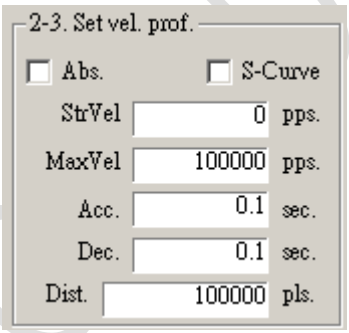


圖 3.234

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：設定運動距離。API 函數的引數變數 “Distance”。

DELTA CONFIDENTIAL

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

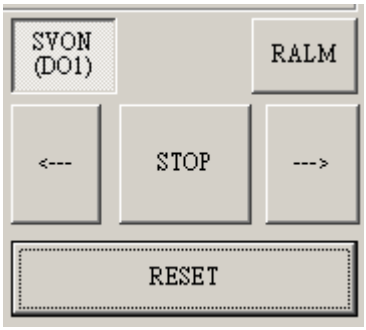


圖 3.235

按壓圖 3.235 中的「SVON(DO1)」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 狀態顯示及正、負極性反相功能

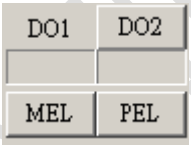


圖 3.236

MEL 核取方塊：設定將負極限反向。

PEL 核取方塊：設定將正極限反向。

按壓圖 3.236 中的「MEL」鍵開始執行以下程序：

```
rt = _DMC_01_rm_04pi_set_MEL_polarity(gDMCCardNo,NodeID,SlotID,
Inverse);
```

按壓圖 3.236 中的「PEL」鍵開始執行以下程序：

```
rt = _DMC_01_rm_04pi_set_PEL_polarity(gDMCCardNo,NodeID,SlotID,
Inverse);
```

7) 若設定了圖 3.233 中的 AB Phase、CW/CCW 及 High active、Low active 後按壓

「Set」鍵開始執行以下程序：

```
rt =_DMC_01_set_rm_04pi_ipulser_mode(CardNo, NodeID, SlotID, mode);
//設定脈波介面模組輸入相位模式
rt = _DMC_01_set_rm_04pi_opulser_mode(CardNo, NodeID, SlotID, mode);
//設定脈波介面模組輸出相位模式
rt = _DMC_01_set_rm_04pi_svon_polarity(CardNo, NodeID, SlotID, polarity);
//設定 POWER ON（SVON）的準位
```

DELTA CONFIDENTIAL

8) 開始進行運動控制(此處以點對點運動為例)

按壓圖 3.235 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

9) 監控顯示

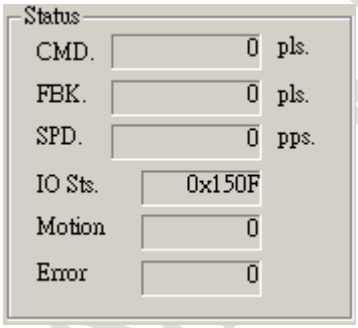


圖 3.237

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

DELTA CONFIDENTIAL

10) 數值重置、停止運動及 Alm_Reset

按壓圖 3.235 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.235 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.237 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.235 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

11) 離開程序

按壓圖 3.231 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.24 中斷操作 Interrupt

3.24.1 函式列表

表 3.24

函式名稱
_DMC_01_int_enable
_DMC_01_int_disable
_DMC_01_set_int_factor
_DMC_01_get_int_count
_DMC_01_get_int_status
_DMC_01_link_interrupt

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.24.2 流程圖

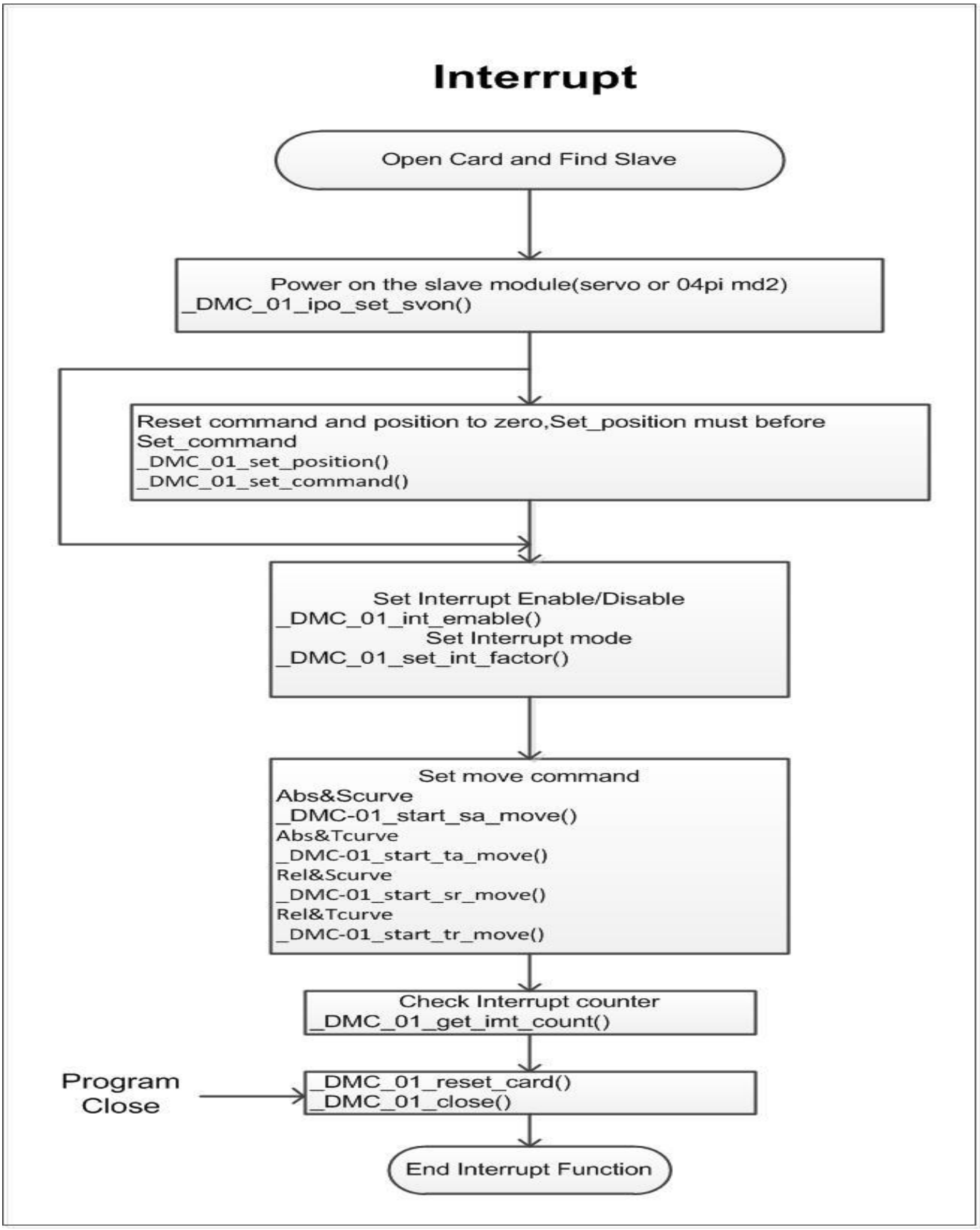


圖 3.240

3.24.3 應用範例

程式外觀

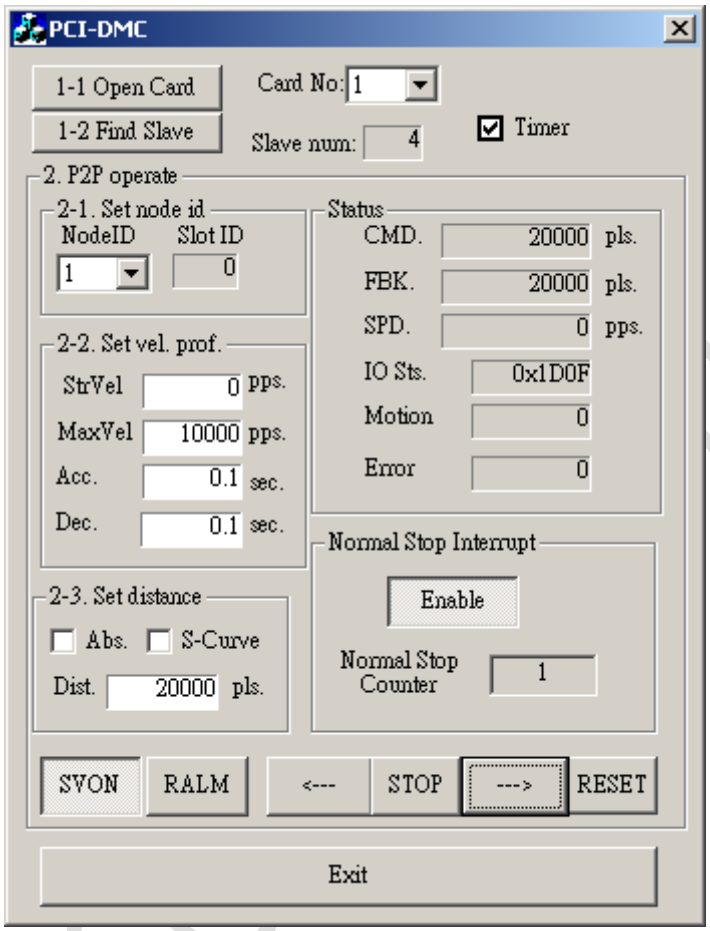


圖 3.241

1) 軸卡開啟與初始化

按壓圖 3.241 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.241 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

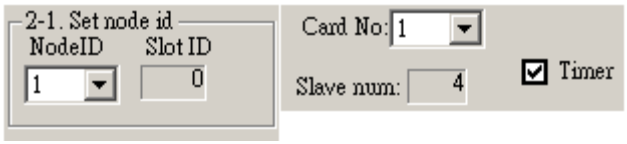


圖 3.242

勾選 NodeID 項目並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

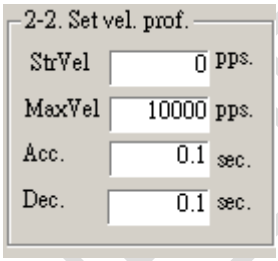


圖 3.243

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

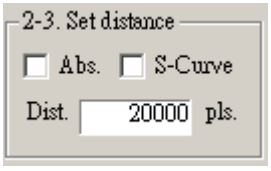


圖 3.244

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它，Default 為 T-curve。

Dist.項目：運動行程。API 函數的引數變數 “Distance”。

DELTA CONFIDENTIAL

5) 設定 Normal Stop Interrupt 是否啟用

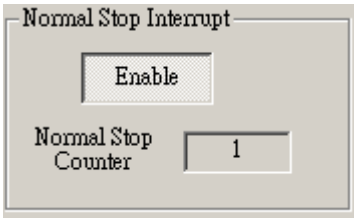


圖 3.245

Enable 項目：啟動/關閉 Normal Stop Interrupt。
Normal Stop Counter 項目：啟動 Normal Stop Interrupt 後累計觸發數。

6) 若您欲啟用 Normal Stop Interrupt 功能，您必須將圖 3.245 中的 “Enable” 鍵按下，

並執行以下程序：

```
/* 啟動 Interrupt 功能*/
rt = _DMC_01_int_enable(gDMCCardNo, NodeID);
/* 關閉 Interrupt 功能*/
rt = _DMC_01_int_disable(gDMCCardNo, NodeID);
/* 設定 Interrupt 功能為 Normal Stop Interrupt*/
rt = _DMC_01_set_int_factor(gDMCCardNo, NodeID,1);
/* 1: Normal Stop：任一運動結束後會觸發(Mode2)  */
/* 2: Next Buffer：執行 Buffer 動作時會觸發(Mode2)  */
/* 3: Acceleration End：在加速結束時觸發(Mode2)  */
/* 4: Deceleration Start：在減速開始時觸發(Mode2)  */
/* 5: Sdo Finish：（此功能尚未開放）  */
/* 6: DMC Cycle Start：進入 DMC Cycle 時觸發  */
/* 7: RM04PI-FIFO：進入 04PI FIFO 時觸發(Mode1)  */
/* 8: User define：（此功能尚未開放）  */
```

7) 設定伺服馬達動力 ON/OFF(servo on/servo off)

圖 3.246

按壓圖 3.246 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
```

DELTA CONFIDENTIAL

// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON

8)開始進行運動控制(此處以點對點運動為例)

按壓圖 3.246 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

9) 設定 Interrupt 功能為 Normal Stop Interrupt：

當進行運動的距離完成時或於中途按下 Stop 鍵，用下列 API 去讀取 Int_Count 值會較之前增加 1。如上圖 3.245 Normal Stop Counter 所示。

```
rt = _DMC_01_get_int_counter(gDMCCardNo, NodeID, &Int_Count);
```

10) 監控顯示

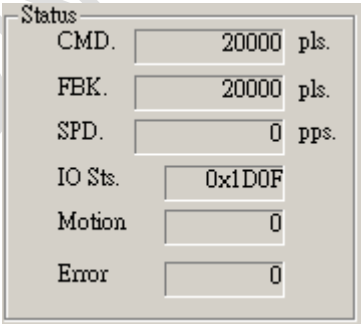


圖 3.247

如圖 3.247 中所示，將使用下列程序取得監控資訊:

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

DELTA CONFIDENTIAL

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

11) 數值重置、停止運動及 Alm_Reset

按壓圖 3.246 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.246 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.247 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.246 中的「RALM」鍵

開始執行 Alm_Code 重置動作。

12) 離開程序

按壓圖 3.246 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.25 主控軸卡安全機制操作 Master Security

3.25.1 函式列表

表 3.25

函式名稱
_DMC_01_check_userpassword
_DMC_01_write_userpassword
_DMC_01_read_serialno
_DMC_01_write_verifykey
_DMC_01_check_verifykey
_DMC_01_read_security
_DMC_01_read_security_status
_DMC_01_write_security
_DMC_01_write_security_status
_misc_security

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.25.2 流程圖

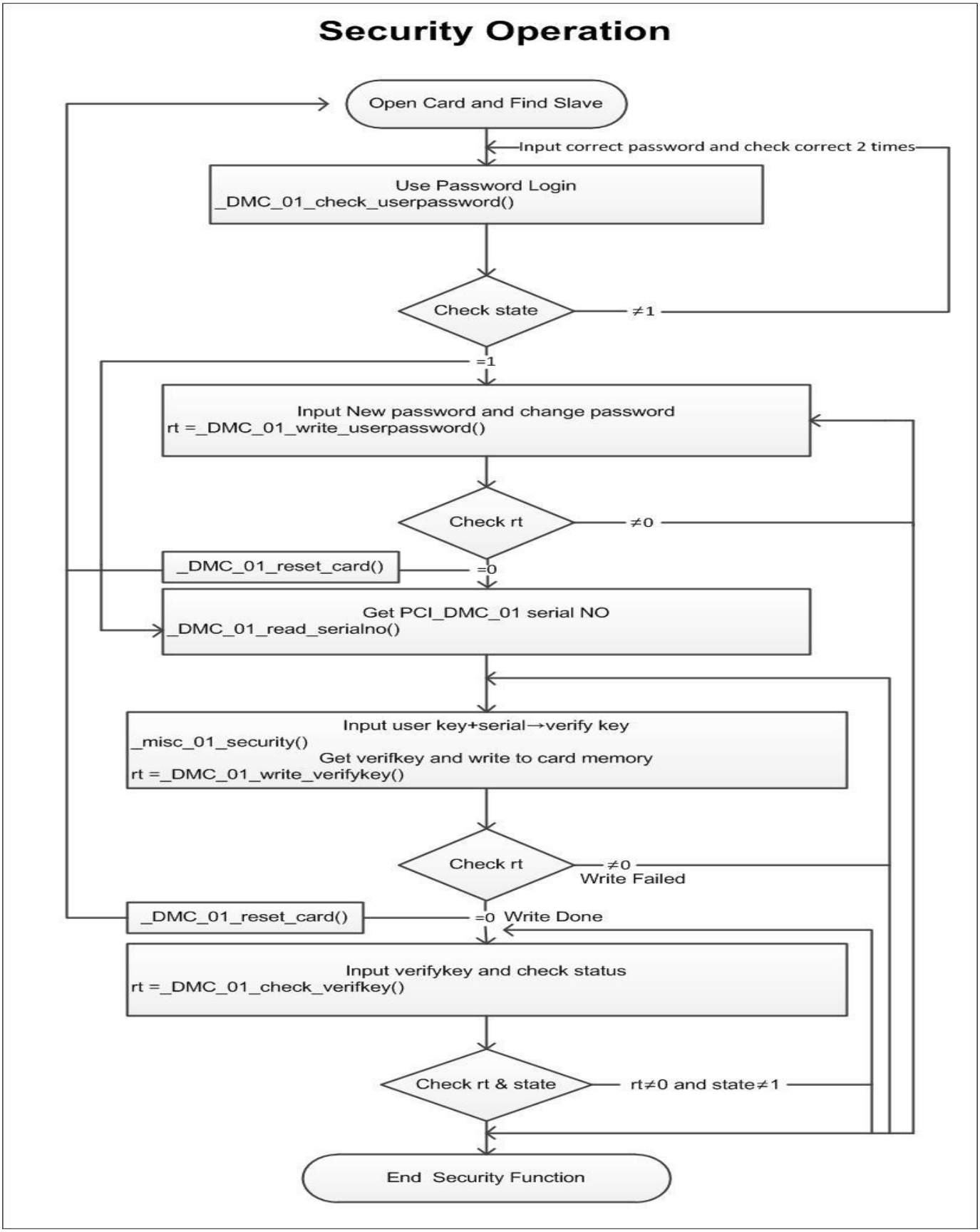


圖 3.250

3.25.3 應用範例

程式外觀

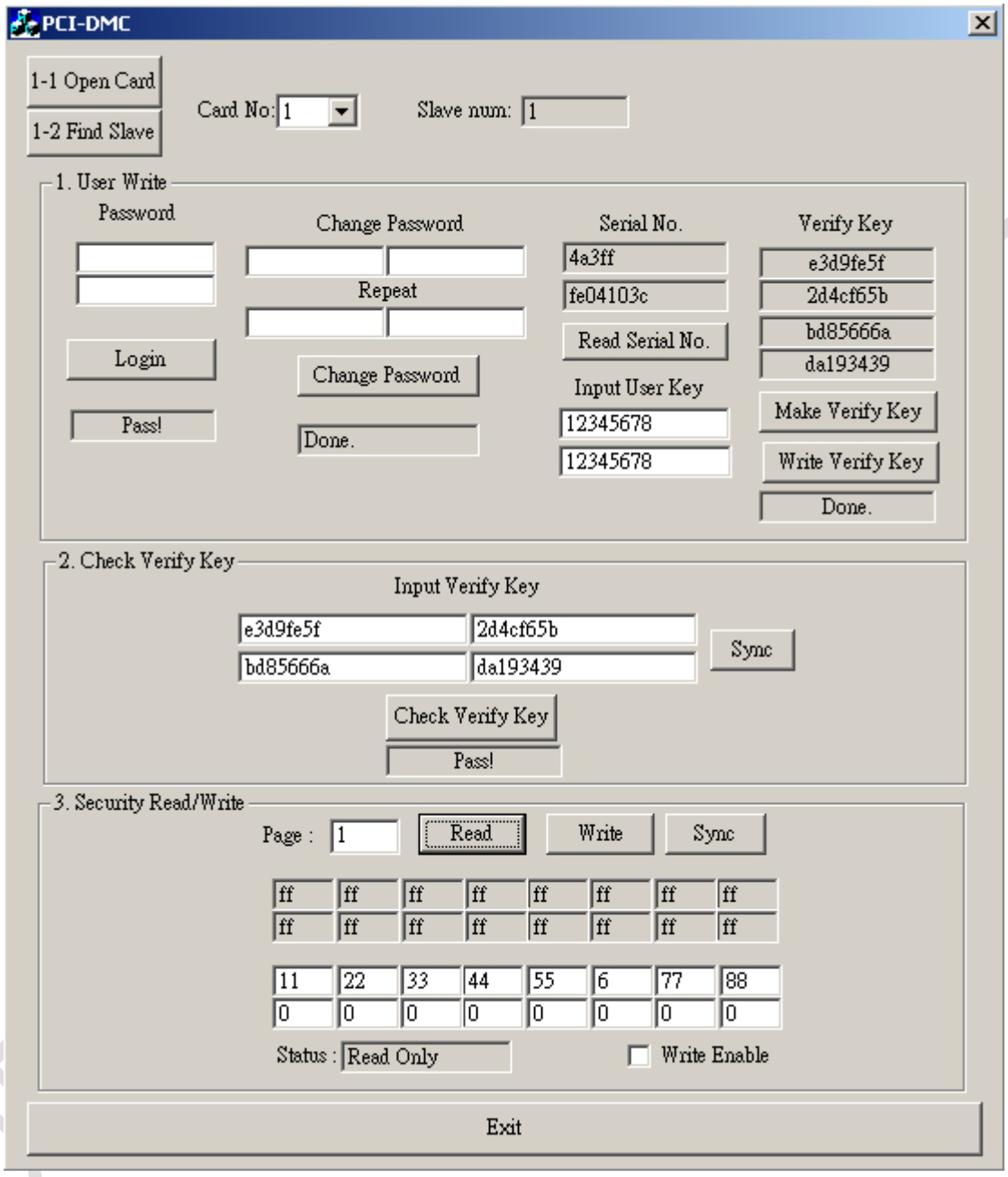


圖 3.251

1) 軸卡開啟與初始化

按壓圖 3.251 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.251 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) 進行軸卡登入動作。(SerialNo、Verifykey、Memory Read/Write 均需要 Check
Password Pass 才能顯示運作)。

圖 3.252

Password 項目：輸入兩組 1~8 位 0~F 的 16 進制值，軸卡預設密碼
(Password1:FFFFFFFF Password2:FFFFFFFF)。
Login 項目：進行密碼確認動作。
狀態列項目：如果密碼正確，圖 3.252 下方狀態列出現「Pass!」，如果密碼錯誤，
狀態列出現「Error!」。

3) 變更軸卡密碼:

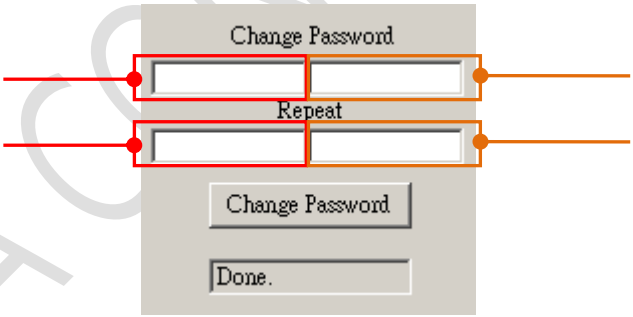


圖 3.253

Change Password 項目：輸入兩組 1~8 位 0~F 的 16 進制值(❶及❸)。
Repeat 項目：重複輸入兩組 1~8 位 0~F 的 16 進制值(❷及❹)
(與 Change Password 輸入值相同)。
Change 項目：按下「Change Password」鍵則進行密碼變更。
狀態列項目：如果 Change Password 與 Repeat 輸入的密碼是相同的，則按下圖 3.253
「Change Password」鍵後，變更密碼成功，狀態列出現「Done.」，
如果 Change Password 與 Repeat 輸入的密碼是不同的，狀態列會出現
「Please rewrite it.」，請重新進行密碼變更動作。

DELTA CONFIDENTIAL

4) 建立 Verifykey 動作

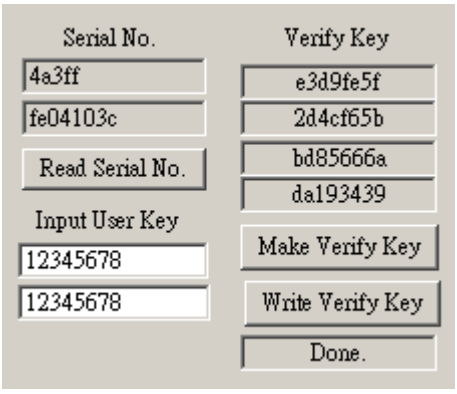


圖 3.254

SerialNo.顯示：軸卡出廠的製造序號，顯示於此。

Read Serial No 項目：進行讀取軸卡製造序號的動作。

Input User Key 項目：輸入客戶端自訂的兩組 1~8 位 0~F 的 16 進制的 Key 值。

Verify key 顯示：由系統產生的 verifykey 值顯示於此。

Make Verify Key 項目：將 SerialNo 及客戶輸入的 Userkey，經加密程序，產生成四組 VerifyKey。

Write Verify Key 項目：將產生的 Verifykey 值寫入軸卡中。

狀態列項目：如果 Verifykey 寫入成功，狀態列出現「Done.」，寫入失敗，出現「Error!」。

5) Verify key 是否正確

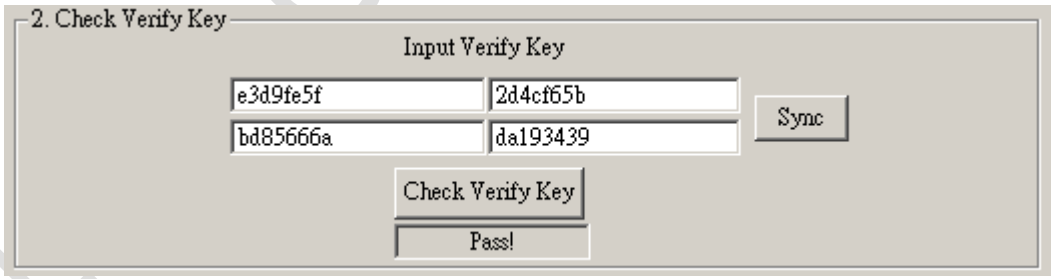


圖 3.255

Input Verify key 項目：輸入要檢查軸卡的 Verify key 值。

Sync 項目：可將圖 3.254 程序產生的 Verify key 值複製於 Input Verify key 欄位。

Check Verify key 項目：按下圖 3.255 中的「Check Verify Key」，若 Key 值正確，

狀態列顯示「Pass.」Key 值錯誤，狀態列顯示「Error!」。

DELTA CONFIDENTIAL

6) 記憶體讀/寫功能(PCI_DMC_F01 無此功能)

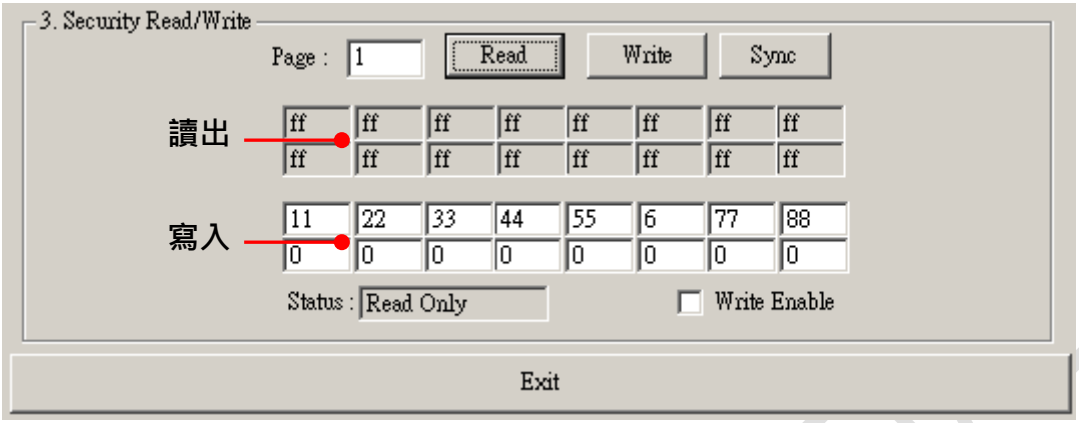


圖 3.256

Page 項目：輸入要讀取或寫入的 Memory Page(0~9)值。

Read 項目：執行讀取指定 Memory Page 動作，並顯示於「讀取 」位置。

Write Enable 項目：在執行寫入動作前，需將 Write Enable 選項打勾，這時於 Status 狀態列會出現「Write/Read」。

Sync 項目：按 Sync 鍵可以將「讀取」位置的值複製到「寫入」位置。

Write 項目：執行將「寫入」位置的值寫入 Memory 中。

Status 項目：Default 值為「Read Only」，Write Enable 選項打勾，顯示「Write/Read」。

7) 若按下圖 3.252 的「login」鍵則執行下列程序：

```
/* 檢查 Password 功能*/
rt = _DMC_01_check_userpassword(gDMCCardNo, Password(0),state);
```

若按下圖 3.253 的「Change Password」鍵則執行下列程序：

```
/* 進行變更密碼動作 */
rt = _DMC_01_write_userpassword(gDMCCardNo, chpassword(0));
```

若按下圖 3.254 的「Read Serial No.」鍵則執行下列程序：

```
/* 讀取 Serial No 動作*/
rt = _DMC_01_read_serialno(gDMCCardNo, serial(0));
```

若按下圖 3.254 的「Make Verify Key」鍵則執行下列程序：

```
/* 進行加密製作 Verify Key 動作*/
rt =
_misc_security(userkey(0),userkey(1),SerialNo(0),SerialNo(1),verifykey(0) ,
verifykey(1) ,verifykey(2) ,verifykey(3));
```

若按下圖 3.254 的「Write Verify Key」鍵則執行下列程序：

```
/* 將 Verify Key 寫入 Memory 中*/
rt =_DMC_01_write_verifykey(gDMCCardNo,verifykey(0));
```

DELTA CONFIDENTIAL

若按下圖 3.255 的「Check Verify Key」鍵則執行下列程序：

```
/* 檢查 Verify Key 值是否正確*/
rt =_DMC_01_check_verifykey(gDMCCardNo,verifykey(0),state);
```

若按下圖 3.256 的「Read」鍵則執行下列程序：

```
/* 讀取指定 Page 的資料顯示於「讀取」位置*/
rt =_DMC_01_read_security(gDMCCardNo,page,ary(0));
```

若勾選圖 3.256 的「WriteEnable」則執行下列程序：

```
/* 將「讀/寫」狀態改變*/
rt =_DMC_01_write_security_status(gDMCCardNo,on_off);
```

若按下圖 3.256 的「Write」鍵則執行下列程序：

```
/* 將「寫入」位置的資料寫入指定 Page 的 Memory 中*/
rt =_DMC_01_write_security(gDMCCardNo,page,ary(0));
/* 執行完這個 API 後，Dll 會將讀/寫狀態自動改為 Read Only，若需要再寫入其他
   資料，須重新設定「讀/寫」狀態(Write_security_status)*/
```

8) 離開程序

按壓圖 3.256 的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.26 遠端擴充類比輸入模組應用-RM04AD

3.26.1 函式列表

表 3.26

函式名稱
_DMC_01_set_04ad_input_range
_DMC_01_get_04ad_input_range
_DMC_01_set_04ad_conversion_time
_DMC_01_get_04ad_conversion_time
_DMC_01_get_04da_data
_DMC_01_set_04ad_average_mode
_DMC_01_get_04ad_average_mode
_DMC_01_set_04ad_input_enable
_DMC_01_set_04ad_input_range

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.26.2 應用範例

程式外觀

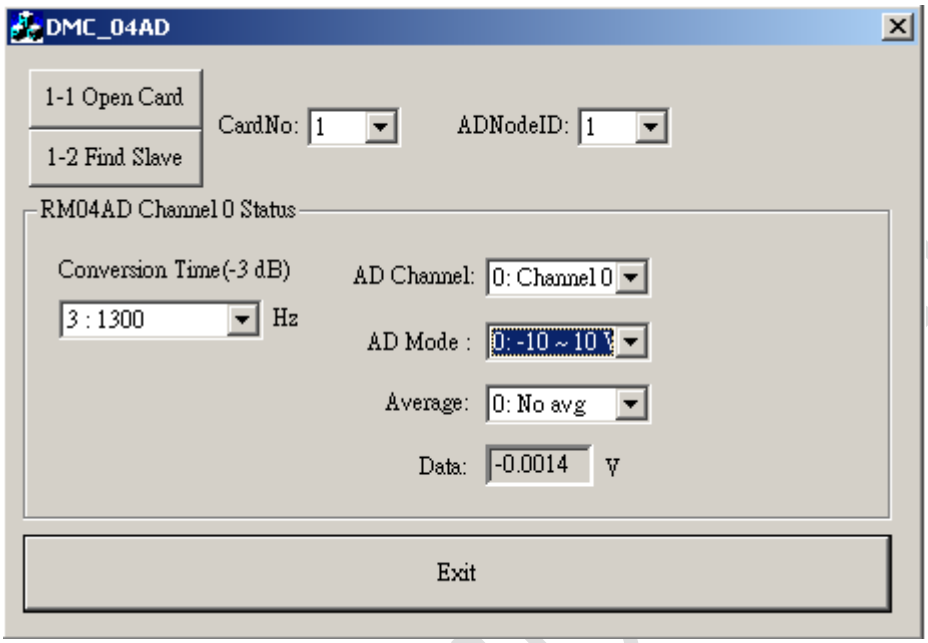


圖 3.260

1) 軸卡開啟與初始化

按壓圖 3.260 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.260 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2) AD Conversion Time 模式選擇:

圖 3.261

Conversion Time 項目：選擇 Conversion Time 的模式。

DELTA CONFIDENTIAL

3) 選擇 AD Channel / 顯示模式 / Average 等相關資料:

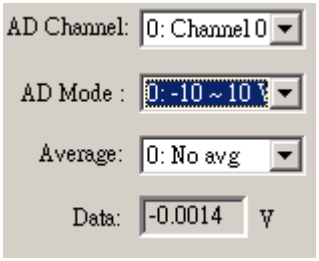


圖 3.262

Channel 項目：選擇 AD 致能輸入之 Channel 位置。
AD Mode 項目：選擇 AD 顯示範圍模式。
Average 項目：選擇波形顯示計算頻率。
Data 項目：輸入 AD 之電壓或電流值顯示位置。

4) 若選擇圖 3.261 的 Conversion Time 選項，則執行下列程序：

```
/* 設定 Conversion Time 模式*/
rt
=_DMC_01_set_04ad_conversion_time(gDMCCardNo,ADNodeID,SlotID,mode);
```

若選擇圖 3.262 的 Mode 選項，則執行下列程序：

```
/* 設定 AD 輸入範圍模式*/
rt
=_DMC_01_set_04ad_input_range(gDMCCardNo,ADNodeID,SlotID,ChannelAD,
mode);
```

若選擇圖 3.262 的 Average 選項，則執行下列程序：

```
/* 設定 AD 波形輸出計算頻率*/
rt =_DMC_01_set_04ad_average_mode(gDMCCardNo,ADNodeID,SlotID,
ChannelAD,mode);
```

圖 3.262 的 Data 顯示，則執行下列程序：

```
/* 顯示 AD 輸入電壓或電流值 */
rt
=_DMC_01_get_04ad_data(gDMCCardNo,ADNodeID,SlotID,ChannelAD,data);
```

5) 離開程序

按壓圖 3.260 中的「Exit」鍵開始執行程序的離開與關閉。
欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.27 遠端擴充類比輸出模組應用-RM04DA

3.27.1 函式列表

表 3.27

函式名稱
_DMC_01_rm_04da_set_output_value
_DMC_01_rm_04da_get_output_value
_DMC_01_rm_04da_get_return_code
_DMC_01_rm_04da_read_data
_DMC_01_rm_04da_set_output_range
_DMC_01_rm_04da_set_output_enable
_DMC_01_rm_04da_set_output_overrange
_DMC_01_rm_04da_set_output_error_clear
_DMC_01_rm_04da_set_output_error_handle
_DMC_01_rm_04da_set_output_offset_value
_DMC_01_rm_04da_get_output_offset_value

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.27.2 應用範例

程式外觀

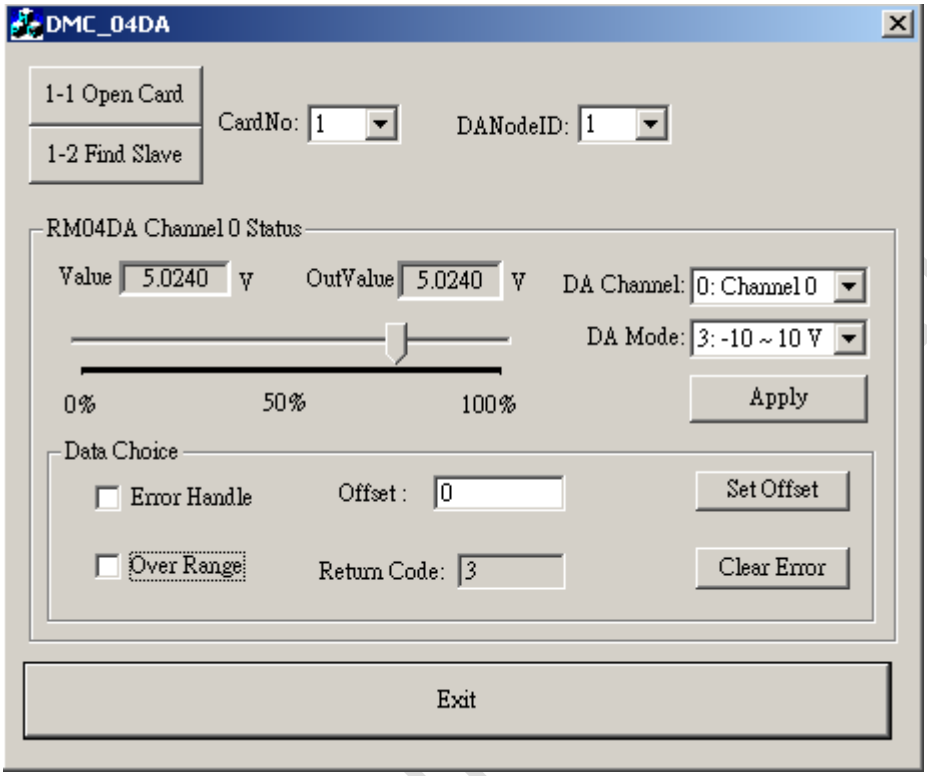


圖 3.270

1) 軸卡開啟與初始化

按壓圖 3.270 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.270 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) DA Channel 選擇及顯示模式選擇:

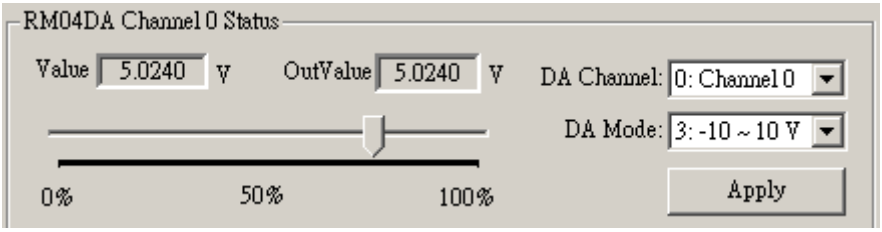


圖 3.271

DA Channel 項目：輸入欲使用之 DA Channel 編號(0~3)。

DA Mode 項目：選擇 DA 顯示範圍模式，API 函數的引數變數 “mode”。

Value 項目: 根據選擇範圍模式及拉把位置顯示預估電壓或電流值。

OutValue 項目: 顯示實際輸出的電壓或電流值。

Apply 項目：按壓 Apply 鍵，會將 Value 的預估電流或電壓值轉為實際輸出。

3) DA Data Choice:

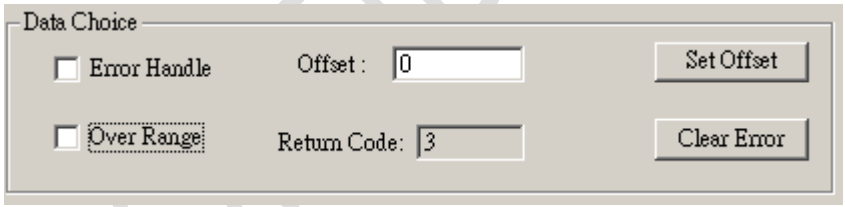


圖 3.272

Error Handle 項目：勾選本項目，可在重新執行本程式時保留上次離開時的狀態。

Over Range 項目：勾選本項目，可使電壓或電流輸出值增加 10%”。

Set Offset 項目：設定 DA 的偏差量。

ClearError 項目：將 Rtn Code 所顯示之 Error 狀態清除。

4) 若選擇圖 3.271 的「DA Mode」選項，則執行下列程序：

```
/* 先將 Offset 值歸 0 */
rt =
_DMC_01_rm_04da_set_output_offset_value(gDMCCardNo,DANodeID,SlotID,
ChannelDA,0);
/* 再將 Output Disable */
rt = _DMC_01_rm_04da_set_output_enable(gDMCCardNo,DANodeID,SlotID,
ChannelDA,0);
/* 設定 DA 選擇模式 */
```

DELTA CONFIDENTIAL

```
rt = _DMC_01_rm_04da_set_output_range(gDMCCardNo,DANodeID,SlotID,
ChannelDA,DAmode);
```

若選擇圖 3.271 的「Apply」選項，則執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。

/* 設定 DA 輸出值 */
rt = _DMC_01_rm_04da_set_output_value(gDMCCardNo,DANodeID,SlotID,
ChannelDA,AppValue);
```

若勾選圖 3.272 的 Error Handle 選項，則執行下列程序：

```
/* Enable Error Handle 功能*/
rt
=_DMC_01_rm_04da_set_output_error_handle(gDMCCardNo,DANodeID,SlotID,
ChannelDA,1);
```

若勾選圖 3.272 的 Over Range 選項，則執行下列程序：

```
/* Enable Over Range 功能*/
rt
=_DMC_01_rm_04da_set_output_overrange(gDMCCardNo,DANodeID,SlotID,
ChannelDA,1);
```

若點選圖 3.272 的「Set Offset」選項，則執行下列程序：

```
/* 將輸入之 Offset 值設定進去 DA */
rt
=_DMC_01_rm_04da_set_output_offset_value(gDMCCardNo,DANodeID,SlotID,
ChannelDA,offset);
```

若點選圖 3.272 的「Clear Error」選項，則執行下列程序：

```
/* 將 Error 狀態清除 */
rt
=_DMC_01_rm_04da_set_output_error_clear(gDMCCardNo,DANodeID,SlotID,
ChannelDA,1);
```

5) 離開程序

按壓圖 3.270 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.28 位置比較 Compare

3.28.1 函式列表

表 3.28

函式名稱
_DMC_01_set_compare_channel_position
_DMC_01_get_compare_channel_position
_DMC_01_set_compare_ipulser_mode
_DMC_01_set_compare_channel_direction
_DMC_01_set_compare_channel_enable
_DMC_01_set_compare_channel_trigger_time
_DMC_01_set_compare_channel_one_shot
_DMC_01_set_compare_channel_source
_DMC_01_channel0_position_cmp
_DMC_01_channel1_output_enable
_DMC_01_channel1_output_mode
_DMC_01_channel1_get_io_status
_DMC_01_channel1_get_gpio_out
_DMC_01_channel1_position_compare_table
_DMC_01_channel1_position_compare_table_level
_DMC_01_channel1_position_compare_table_cnt
_DMC_01_set_compare_channel_polarity

■ 屬性

PCI_DMC_B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.28.2 流程圖

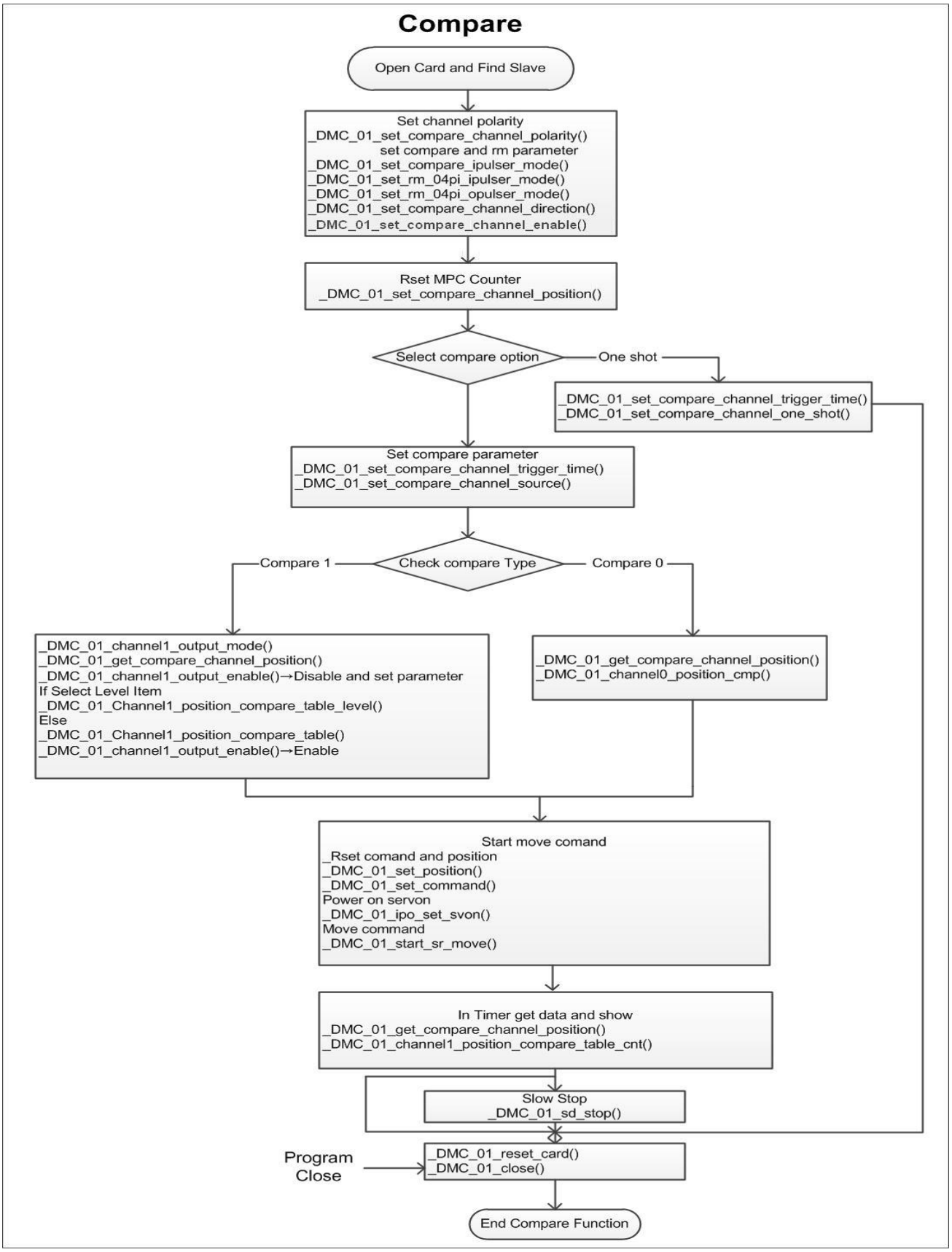


圖 3.280

3.28.3 應用範例

程式外觀

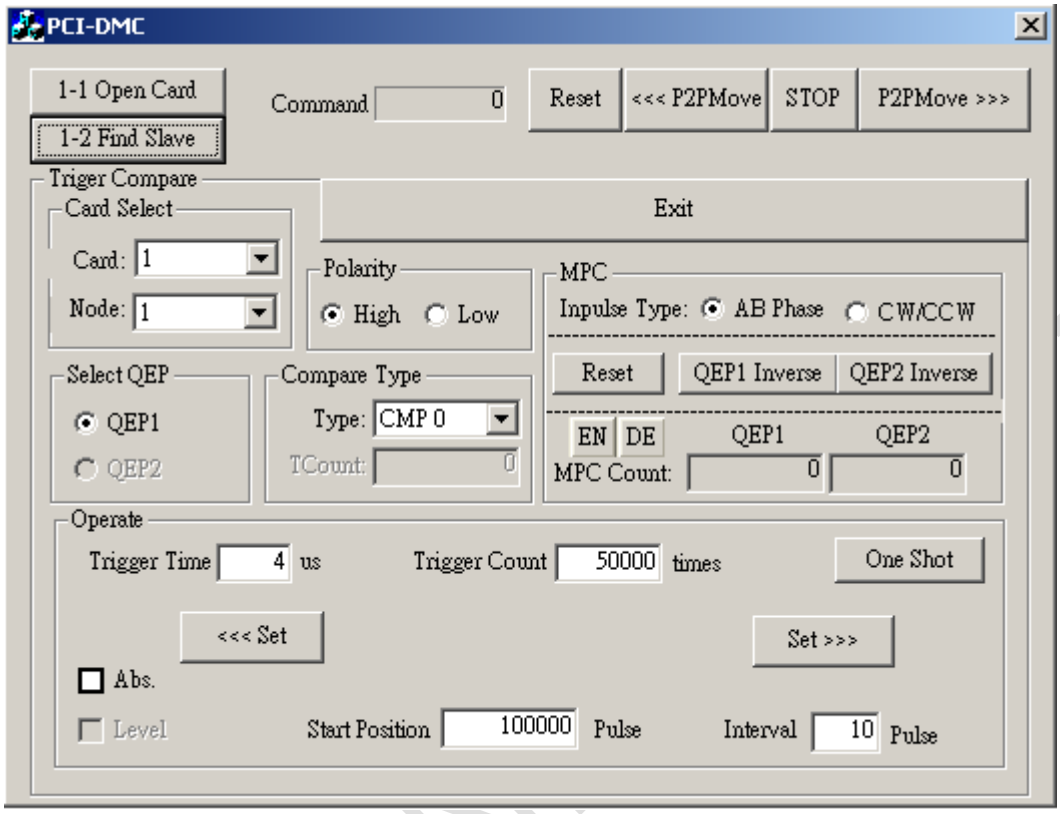


圖 3.281

1) 軸卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-B01 軸卡。

按壓圖 3.281 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.281 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) 卡號、站號及 QEP 選擇:

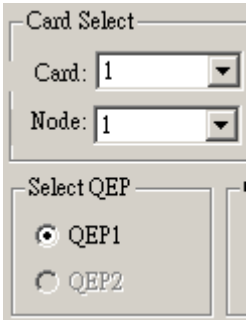


圖 3.282

Card 項目：輸入欲使用之 PCI_DMC_B01 卡片編號。

Node 項目：選擇使用站號，應與 QEP1 與 QEP2 選擇一致。

QEP1 項目: 例如:選擇使用 Channel0，站號應選擇 Node1。

QEP2 項目: 例如:選擇使用 Channel1，站號應選擇 Node2。

3) Compare Type Select 及 Polarity 設定:

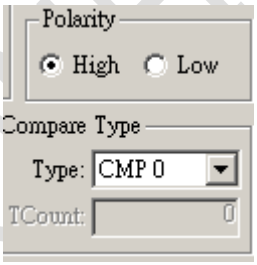


圖 3.283

High 項目：勾選本項目，執行以下設定: 0: High

```
rt = _DMC_01_set_compare_channel_polarity(CpCardNo,0);
```

Low 項目：勾選本項目，執行以下設定: 1: Low

```
rt = _DMC_01_set_compare_channel_polarity(CpCardNo,1);
```

Type 項目：選擇 Comapre1 或是 Compare2 功能。

```
rt =
_DMC_01_set_compare_channel_source(CpCardNo,Compare_Type,CpQEP);
//Compare_type:0=COMP1,1=COMP2
```

TCount 項目：Trigger 實際執行次數。

DELTA CONFIDENTIAL

4) MPC 參數設定:

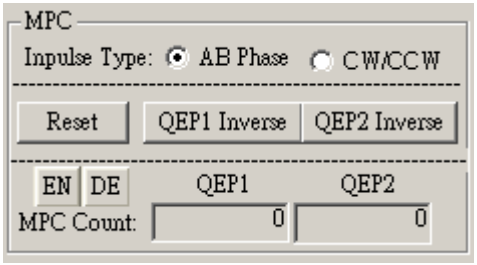


圖 3.284

EN/DE 項目：開啟/關閉 Compare 功能，執行以下設定。

```
rt = _DMC_01_set_compare_channel_enable(CpCardNo,channel,1); //1：開啟
```

Inpulse Type 項目：選擇 AB Phase 或是 CW/CCW 的模式。執行以下設定:

```
rt = _DMC_01_set_compare_ipulser_mode(CpCardNo,mode);
// 0: AB Phase   1: CW/CCW
```

Reset 項目：按下此鍵，將 QEP1 及 QEP2 的 MPC Count 清除,執行以下設定。

```
rt = _DMC_01_set_compare_channel_position(CpCardNo,channel,0);
```

QEP1 Inverse 項目：反向，按下此鍵，執行以下設定:

```
rt = _DMC_01_set_compare_channel_direction(CpCardNo,0,dir); // dir:0 or 1
```

QEP2 Inverse 項目：反向，按下此鍵，執行以下設定:

```
rt = _DMC_01_set_compare_channel_direction(CpCardNo,1,dir); // dir:0 or 1
```

5) Operate 相關資料設定:

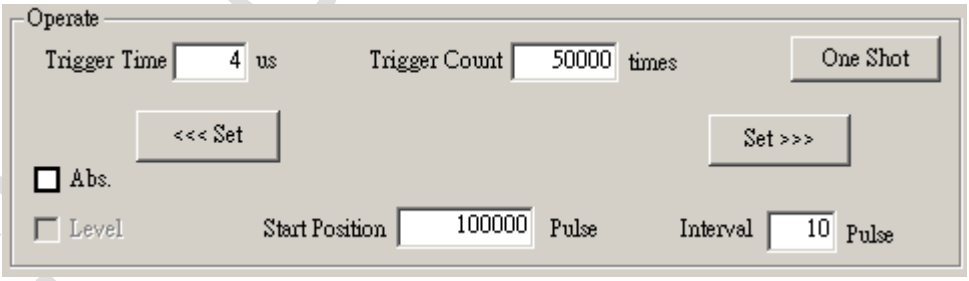


圖 3.285

Trigger Time 項目：輸入一個 Trigger 致能時間。

Trigger count 項目：輸入 Trigger 總致能次數。

Start Position 項目：輸入 Trigger 致能起始位置。

Interval 項目：輸入 Trigger 致能頻率，例如:輸入 10 表示每 10 個 Pulse 致能一次。

Abs 核取方塊：若您欲參照絕對座標的方式進行 Trigger 致能，必須勾選它。

Level 核取方塊：勾選它，執行以下設定:

DELTA CONFIDENTIAL

rt = _DMC_01_channel1_output_mode(CpCardNo,mode);
// mode: 0 表示 Normal 型態， 1 表示自訂型態(詳見 39 章 Compare API 說明)
One Shot 項目：按下此鍵，只打一個 Trigger，執行以下設定:
rt =
_DMC_01_set_compare_channel_trigger_time(CpCardNo,compare_channel,
time_us); // time_us=Trigger time
rt =
_DMC_01_set_compare_channel_one_shot(CpCardNo,compare_channel);

Set 項目：可選擇>>>及<<<這兩個方向，按下此鍵，執行以下設定:
rt = _DMC_01_get_compare_channel_position(CpCardNo,compare_channel,
position);
rt =
_DMC_01_set_compare_channel_trigger_time(CpCardNo,compare_channel,
time_us); // time_us = Trigger time
If CompareType=CMP1 //Compare1
rt = _DMC_01_channel0_position_cmp(CpCardNo,start,dir,interval);
// dir->0:CMP1，1:CMP2
else //Compare2
先將 output disable
rt = _DMC_01_channel1_output_enable(CpCardNo,0); // 0:off 1:on
若 Level 核取方塊是勾選的，則執行下列設定:
rt =
_DMC_01_channel1_position_compare_table(CpCardNo,pos_table,tab_size);
若未勾選，則執行下列設定:
rt =
_DMC_01_channel1_position_compare_table_level(CpCardNo,pos_table,
level_table,tab_size);
最後將 output enable
rt = _DMC_01_channel1_output_enable(CpCardNo,1); // 0:off 1:on

6) Command 顯示及測試操作區:

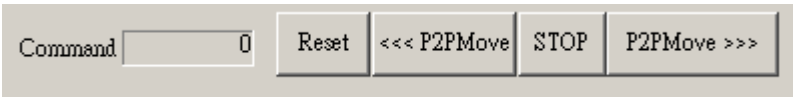


圖 3.286

Reset 項目：按下此鍵會進行 Motion 歸零動作。
P2PMove 項目：按下此鍵會進行 Motion 正向或負向動作。
STOP 項目：按下此鍵會將 Motion 動作停止。

DELTA CONFIDENTIAL

Command 項目：顯示 Motion 動作目前的位置。

7) 離開程序

按壓圖 3.281 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.29 Axis Group

3.29.1 函式列表

表 3.29

函式名稱
_DMC_01_set_group

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

DELTA CONFIDENTIAL

3.29.2 流程圖

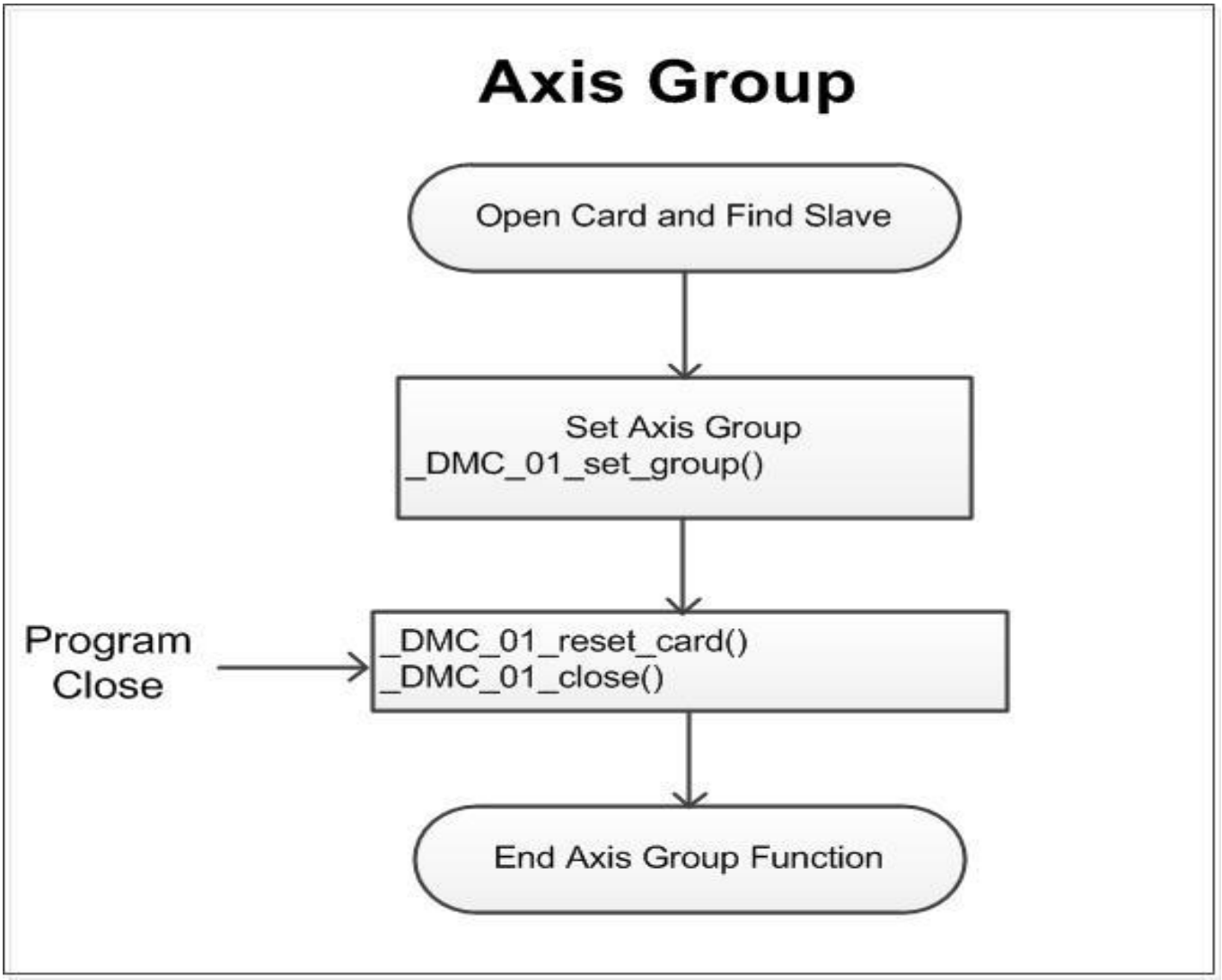


圖 3.290

DELTA

3.29.3 應用範例

程式外觀

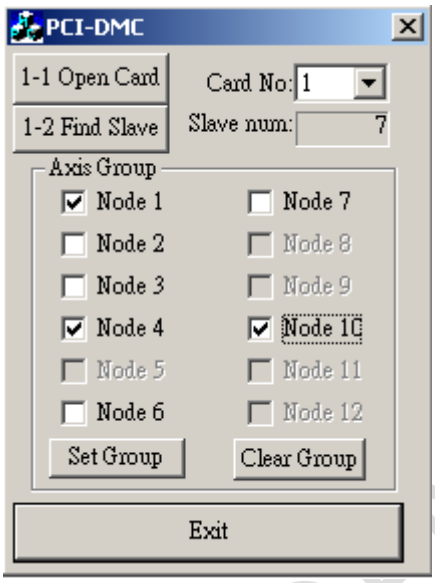


圖 3.291

1) 軸卡開啟與初始化

按壓圖 3.291 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.291 中的「Find Slave」鍵開始搜尋串接模組。

關於此按鍵的相關敘述，可參考第 3.1.3 節「軸卡初始化與切換操作模式」說明。

2) 卡號、站數顯示:

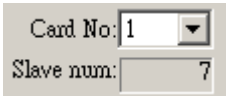


圖 3.292

Card num 項目：顯示卡片張數。

Slave num 項目：顯示站數。

DELTA CONFIDENTIAL

3) Group 設定及解除:

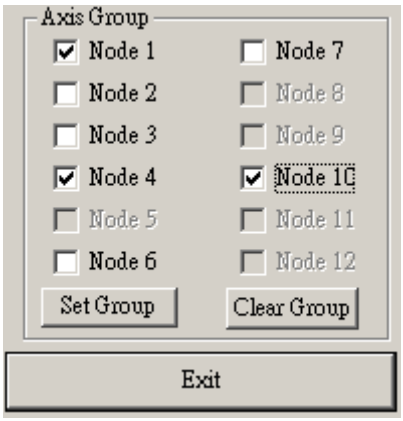


圖 3.293

Nodexx 項目：顯示站號詳細資料。

Set Group 項目：1.一張卡最多可以設定六個群組。

2.假設設定 Node1，Node4，Node10 為一個群組，表示當連線是

在 Node4 發生異常，那麼 Node1，4，10 會一起停止，其他

Node6，7 則不受影響，可維持正常運作。

3.當選好要設為同一群組的 Node 目標時，點選"Set Group"將執

行以下設定。

rt = _DMC_01__DMC_01_set_group (gDMCCardNo, NodeID, SlotID, NodeNum,

1); //1:表示將所選擇的 Node 設定為同一群組。

Clear All Group 項目：當點選"Clear All Group"將執行以下設定。

rt = _DMC_01_set_group(gDMCCardNo, (U16*)1, 0, 1, 0);

//0:表示將本卡所有的群組設定都清除

4) 離開程序

按壓圖 3.293 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

DELTA CONFIDENTIAL

3.30 DDA 應用 DDA Axes Move

3.13.1 函式列表

表 3.30

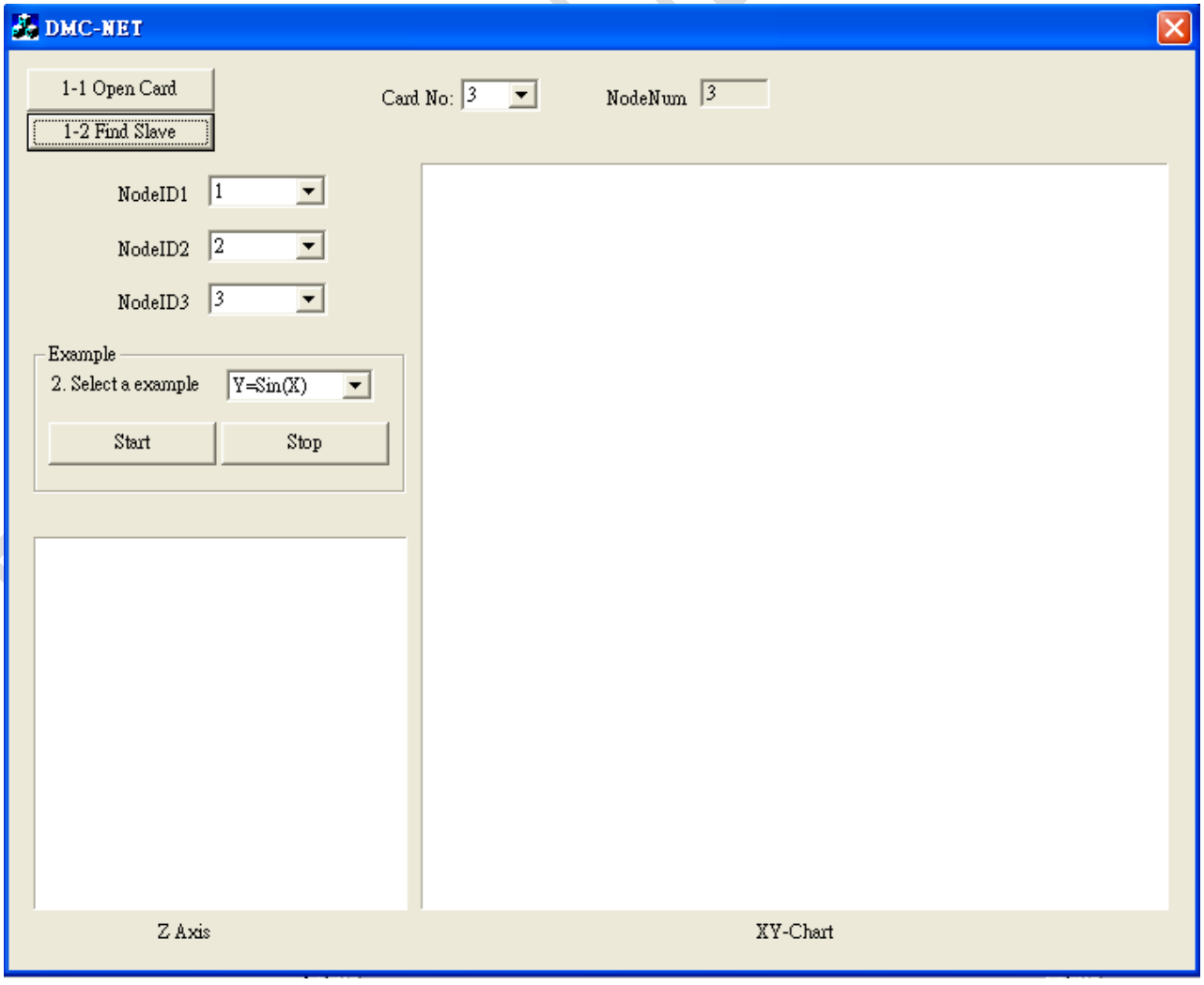
函式名稱
_DMC_01_enable_dda_mode
_DMC_01_set_dda_data

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.30.2 應用範例

程式外觀



CONFIDENTIAL

圖 3.300

6)軸卡開啟與初始化

按壓圖 3.130 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3.130 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

7)卡號、站數顯示:

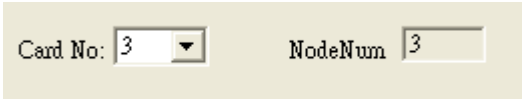


圖 3.301

Card num 項目：顯示卡片張數。

Slave num 項目：顯示站數。

8)Example 參數設定及操作:

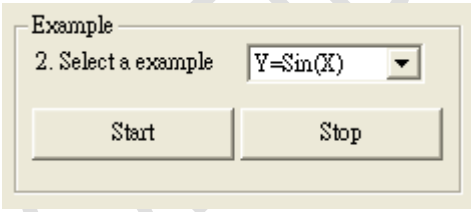


圖 3.302

Select a example 項目：有 Y=Sin(X)可供選擇。

Start 項目：當點選”Start”鍵將執行以下設定。

_DMC_01_enable_dda_mode(CardNo, enable)

_DMC_01_set_dda_data(CardNo, * abs_pos)

參數：

CardNo：卡號。

enable ：bit0~12 代表軸 1~12，為 1 時表示該軸啟用 DDA 功能。

abs_pos：1X12 陣列，代表軸 1~12 的絕對位置。

DELTA CONFIDENTIAL

此例是以每 1ms 為時間單位計算出 X 軸需行走 t*pi/720(pps)的距離，Y 軸行走 Sin(X)的距離。並且讓 Z 軸的始終保持切線方向，如圖 3.303。

(以下解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

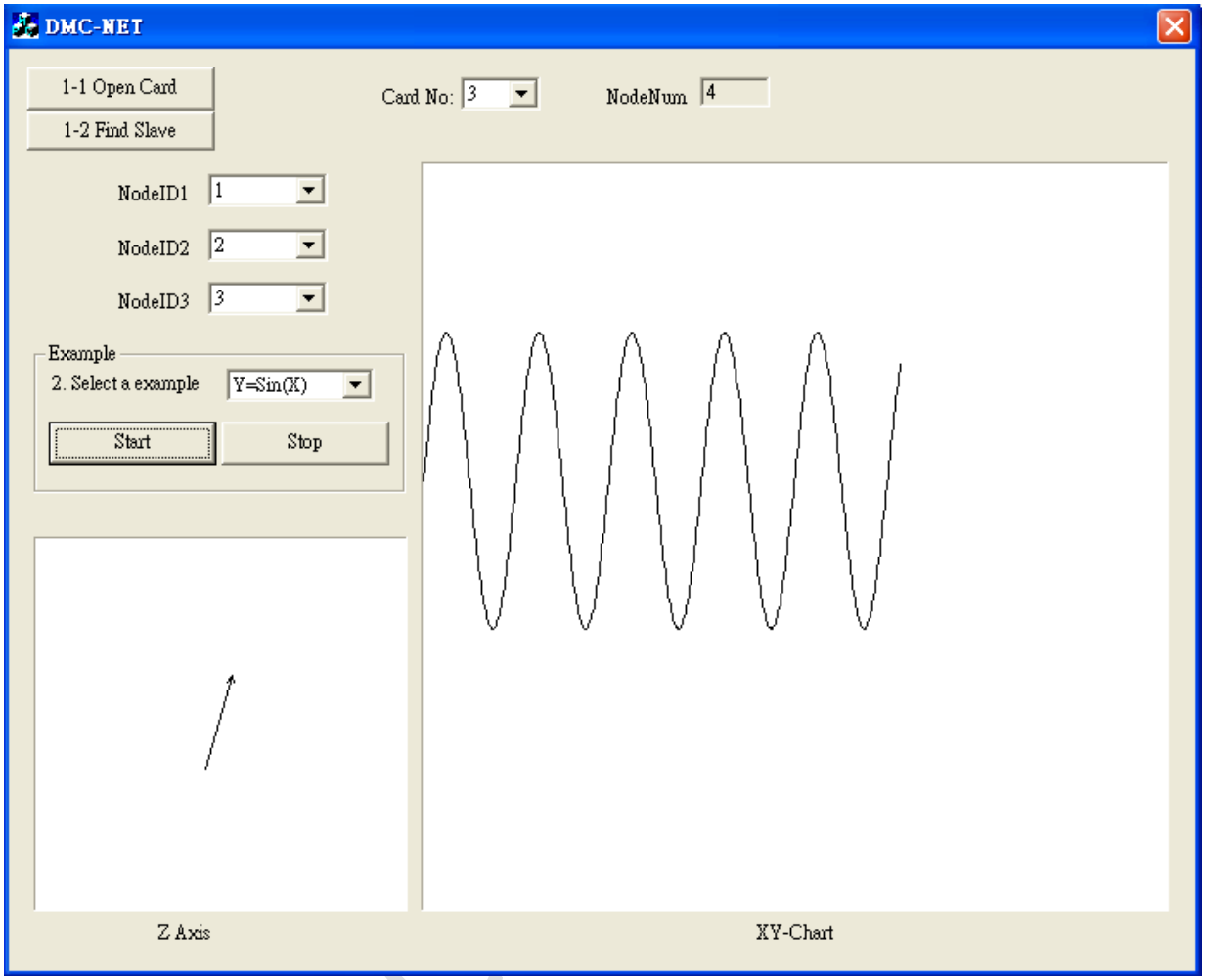


圖 3.303

```
/* MotionStep1.預先填入 100ms 的座標資料到 DDA Buffer 內 */
rc= _DMC_01_get_dda_cnt(gDMCCardNo, &i);
if (i < 100)
{
for (i=0;i<100;i++)
{
      X=(i*PI/720);
      Y=32000*sin(X);
      Z=9000*sin(X);
      DDA_Table[i][gpNodeID[2]-1]=Z;
      DDA_Table[i][gpNodeID[1]-1]=Y;
      DDA_Table[i][gpNodeID[0]-1]=1280000*X/360;
}
X_Counter = i;
rc= rc+_DMC_01_enable_dda_mode(gDMCCardNo, DDA_Enable);
}
if(rc == 0)
{     MotionStep = 2;     }

/* MotionStep2.若 DDA Buffer 內資料低於 100 筆，產生的後 100ms 的座標資料到 DDA Buffer
內*/
rc= _DMC_01_get_dda_cnt(gDMCCardNo, &i);
if (i < 100)
{
for (i=0;i<100;i++)
{
      X=((X_Counter+i)*PI/720);
      Y=32000*sin(X);
      Z=9000*sin(X);
      DDA_Table[i][gpNodeID[2]-1]=Z;
      DDA_Table[i][gpNodeID[1]-1]=Y;
      DDA_Table[i][gpNodeID[0]-1]=1280000*X/360;
      rc= _DMC_01_set_dda_data(gDMCCardNo, DDA_Table[i]);
}
X_Counter=X_Counter+i;
}
/*填入 7.5 秒的座標資料後結束*/
if (X_Counter > 7500)
{     MotionStep =0 ;     }
```

DELTA CONFIDENTIAL

3.31 位置比較 B02_Compare

3.31.1 函式列表

表 3.31

函式名稱
_DMC_B02_qep_set_mode
_DMC_B02_qep_set_inverse
_DMC_B02_qep_get_reload_counter
_DMC_B02_qep_set_reload_counter
_DMC_B02_qep_get_counter
_DMC_B02_gpio_software_control_status
_DMC_B02_gpio_software_control_setting
_DMC_B02_ipcmp_initial
_DMC_B02_ipcmp_oneshot_time
_DMC_B02_ipcmp_software_trigger
_DMC_B02_ipcmp_compare_position
_DMC_B02_ipcmp_compare_trigger_count
_DMC_B02_ipcmp_compare_trigger_count_clear
_DMC_B02_tpcmp_initial
_DMC_B02_tpcmp_oneshot_time
_DMC_B02_tpcmp_software_trigger
_DMC_B02_tpcmp_software_trigger_level
_DMC_B02_tpcmp_compare_position_table_reset
_DMC_B02_tpcmp_compare_position_table
_DMC_B02_tpcmp_compare_position_table_level
_DMC_B02_tpcmp_compare_trigger_count
_DMC_B02_tpcmp_compare_trigger_count_clear
_DMC_B02_ext_io_get_input_value
_DMC_B02_ext_io_get_output_value
_DMC_B02_ext_io_set_output_value

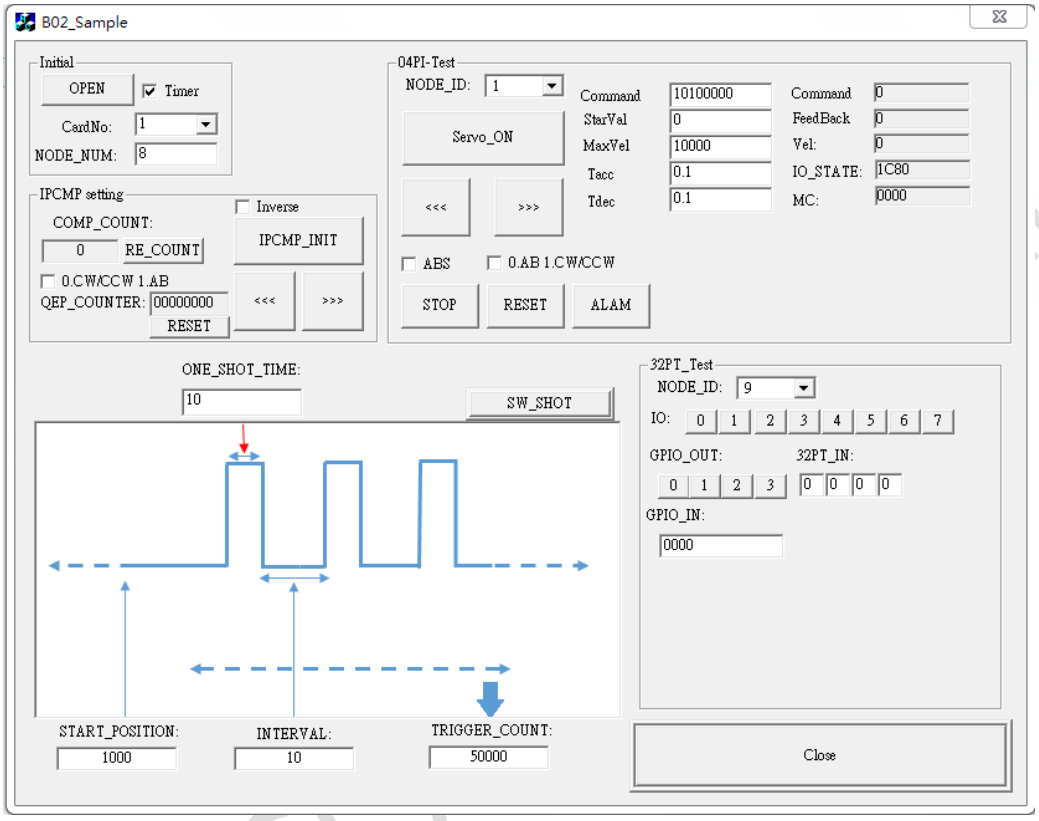
■ 屬性

PCI_DMC_B02 軸卡
●

DELTA CONFIDENTIAL

3.31.2 應用範例一:IPCMP

程式外觀



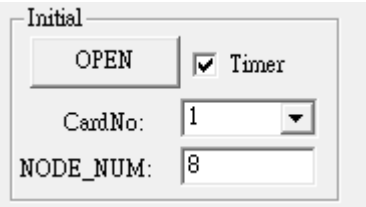
1)軸卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-B02 軸卡。

按壓圖 3.31.2 中的「Open」鍵開始執行軸卡初始化與自動搜尋 04PI 與 32PT 模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2)卡號、站號及開啟 Timer:

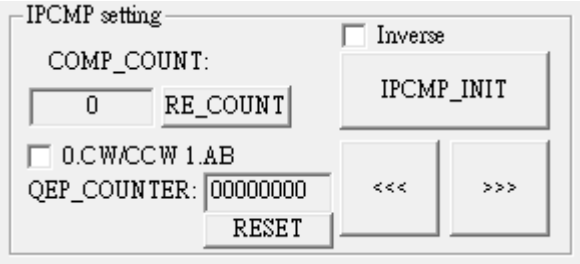


取得初始化參數與連接的 NOID 數，

Card No 項目：顯示卡片張數。

Slave num(NODE_NUM)項目：顯示站數。

3)IPCMP 相關設定:



IPCMP_INIT :初始化 IPCMP。43.8 節

Inverse :脈波反向設定。43.2 節

rt= _DMC_B02_ipcmp_oneshot_time(l_nCardNo, l_nIpcmpIndex, l_nPulseWidth);

```
rt=  _DMC_B02_ipcmp_initial(l_nCardNo,
      l_nIpcmpIndex,                    //CMPINDEX: 0~3
      l_nQepIndex,                      //QEP: 0~2
      l_nGpioPin,                       //GPIO: 0 ~ 9
      l_Linverse);                      //脈波準位  Inverse
```

0:CW/CCW 1.AB :設定脈波模式此處請打勾選差動(1.AB phase)。43.1 節

rt = _DMC_B02_qep_set_mode(l_nCardNo, l_nQepIndex, nMode);

RE_COUNT :重置 COMP_COUNT。43.13 節

rt= _DMC_B02_ipcmp_compare_trigger_count_clear(l_nCardNo, l_nIpcmpIndex);

RESET :重置 QEP_COUNTER。43.4 節

rt = _DMC_B02_qep_set_reload_counter(l_nCardNo, l_nQepIndex, 0);

DELTA CONFIDENTIAL

>>>　　:設定正向 Compare 設定。43.11 節

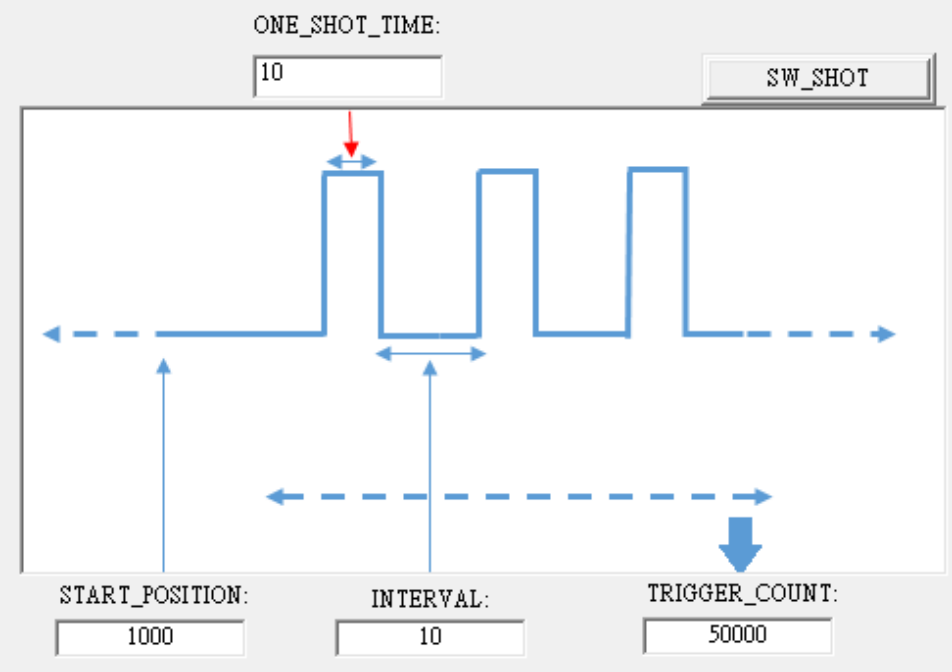
<<<　　:設定反向 Compare 設定。43.11 節

```
int l_nStartPosition = 0;
unsigned short l_nInterval = 100;
unsigned long l_nTriggerCount =10000;
//設定正向
rt=  _DMC_B02_ipcmp_compare_position(l_nCardNo, l_nIpcmpIndex, l_nStartPosition, l_nInterval,
     l_nTriggerCount, 0);
//設定反向
rt=  _DMC_B02_ipcmp_compare_position(l_nCardNo, l_nIpcmpIndex, l_nStartPosition, l_nInterval,
     l_nTriggerCount, 1);
```

相關 API 內部操作請參考範例 B02_Sample 或 43.8 章節 B02 相關 API。

DELTA CONFIDENTIAL

4)IPCMP 相關設定:



ONE_SHOT_TIME :初始化 IPCMP。43.9 節

```
//   nPulseWidth -> 0~4095(us)
rt=   _DMC_B02_ipcmp_oneshot_time(l_nCardNo, l_nIpcmpIndex, l_nPulseWidth);
```

SW_SHOT :發送一次 TRIGGER 脈波。43.10 節

```
unsigned char l_nIpcmpIndex = g_nIpcmpIndex;
//針對某個腳位執行一次 Trigger
rt = _DMC_B02_ipcmp_software_trigger(l_nCardNo, l_nIpcmpIndex);
```

```
//下面兩個部分已經在 ipcmp_initial 設定完成
```

START_POSITION :設定起始位置(cw->1000,ccw->-1000) 。43.11 節

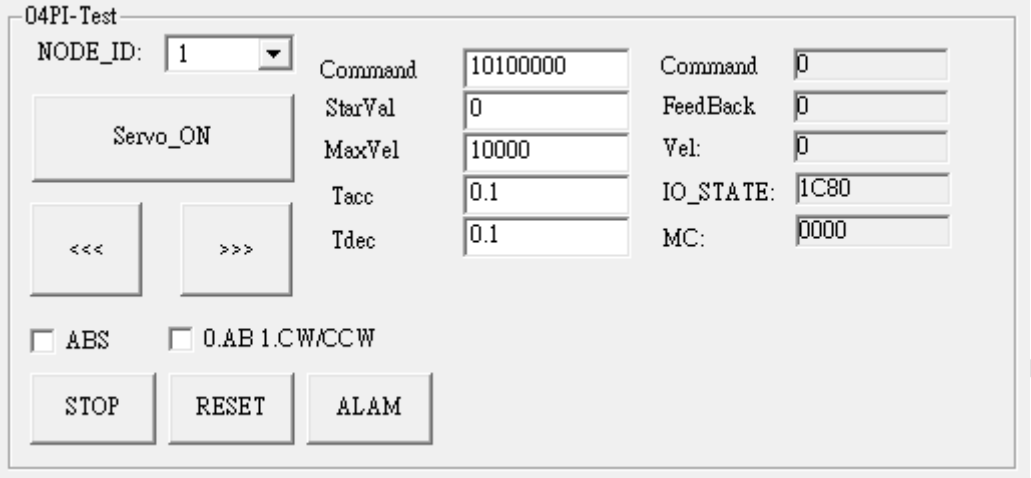
INTERVAL :設定間格寬度。43.11 節

TRIGGER_COUNT :設定 TRIGGER 次數(請確認行程內是否能夠打完) 。43.11 節

相關 API 內部操作請參考範例 B02_Sample 或 43.8 章節 B02 相關 API。

DELTA CONFIDENTIAL

5)04PI 相關設定:



本範例使用 04pi 作為測試模組,實際請使用者依照使用模組設定相關 api

NODE_ID:站號。

AB 1.CW/CCW:針對模組脈波設定,此處 04PI 為 CW/CCW，針對客戶當前模組設定。

```
//Mode:[0]AB phase    [1]CW/CCW
if( GetCheckValue(IDC_CHK_04PI_MODE))
{
      mode = 1;  //CW/CCW
}
else
{
      mode = 0;  //AB Phase
}
   rti = _DMC_01_set_rm_04pi_ipulser_mode(l_nCardNo, lNode_id, 0, mode);
   rto = _DMC_01_set_rm_04pi_opulser_mode(l_nCardNo, lNode_id, 0, mode);
```

其他相關操作一律與 04PI 運作相同。

操作步驟:

1. Servo_ON 後
2. 選擇剛剛設定的 IPCMP 設定的”>>>”或”<<<”執行運動指令
3. 緊接著 compare 會跟據 04PI 的 CW 與 CCW 將根據單向訊號使 Compare 腳位發送出相對應的 counter 數。

4.此時可在剛剛的 COMP_COUNT 看到打出的 COUNT 數。

DELTA CONFIDENTIAL

6)32PT 對應 GPIO 範例:

NODE_ID:站號。
IO: 使用 32PT-output 送出訊號給 B02 input。

GPIO_OUT：使用 B02 送出 output 43.25 節, 32PT 抓取 B02output 訊號,輸出在 32PT_IN。43.24 節

```
//設定被選到的 OUT 輸出
rt =_DMC_B02_ext_io_set_output_value(l_nCardNo, l_nOutputValue);
```

GPIO_IN:選擇 EXTIO 腳位元,選擇 GPIO 的模式,L2H or H2L 設定對應的 API 後則可觀察輸入欄位是否有訊號變化。43.23 節

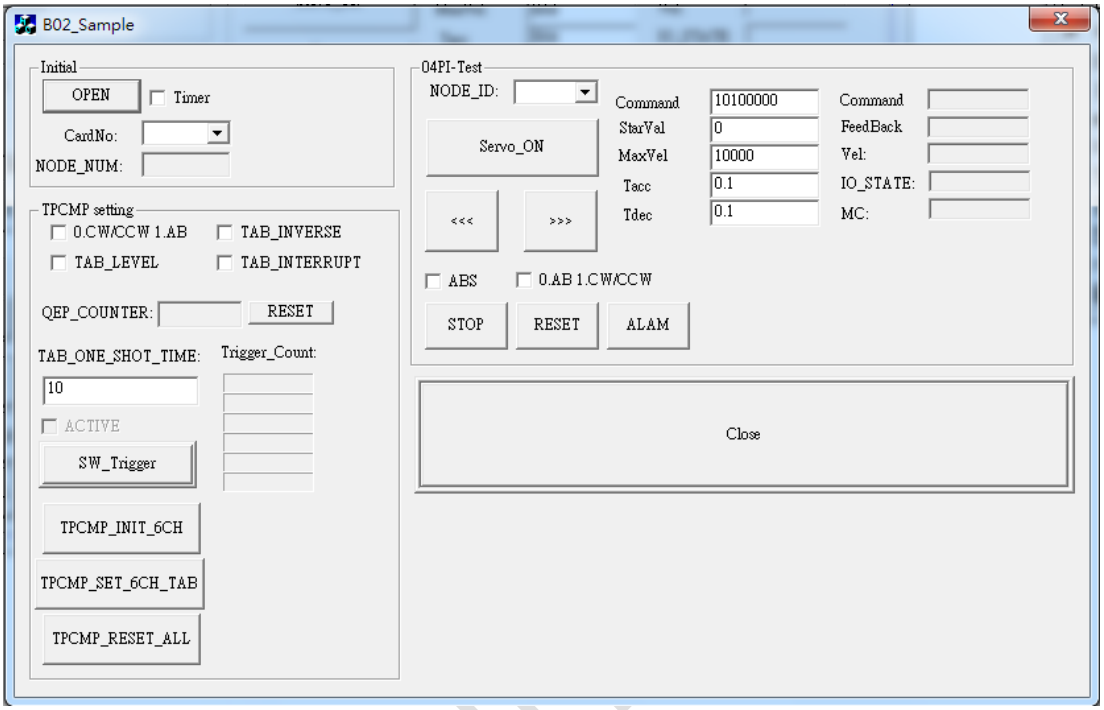
```
//此部分是使用 32PT 配合測試，使用者根據使用模組修改
rt =_DMC_B02_ext_io_get_input_value(l_nCardNo, &l_nIoState);
```

相關 API 內部操作請參考範例 B02_Sample 或 43.23 章節 B02-GPIO 相關 API。

DELTA CONFIDENTIAL

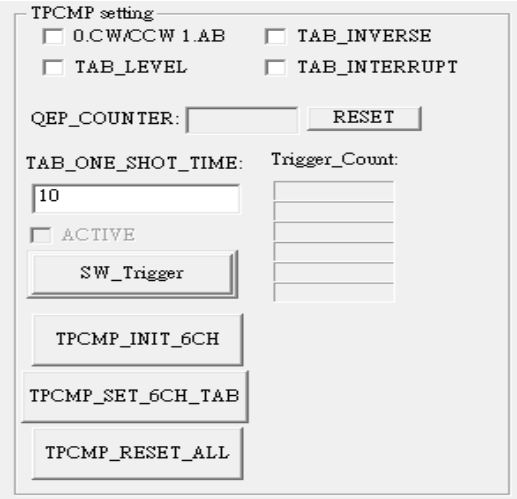
3.31.3 應用範例二:TPCMP

程式外觀



DELTA CONFIDENTIAL

7)設定 TPCMP 相關範例:



NODE_ID:站號。

0.CW/CCW 1.AB:設定脈波模式。43.1 節

```
//設定比對訊號模式;
rt = _DMC_B02_qep_set_mode(l_nCardNo, l_nQepIndex, nMode);
```

TAB_ONE_SHOT_TIME:一次脈波時間。43.15 節

```
//設定一次脈波時間; nPulseWidth -> 0~4095(us)
rt =_DMC_B02_tpcmp_oneshot_time(l_nCardNo, l_nTpcmpIndex, m_nOneshotTime);
```

TAB_LEVEL:設定是否啟用 tab_level。43.17 節

TAB_INVERSE: 設定是否啟用 tab_inverse。43.14 節

TAB_INTERRUPT:暫無使用。43.14 節

SW_TRIGGER: 打出一次脈波與 level 並用可決定起使脈波准位。43.16 節

TPCMP_INIT_6CH:一次設定 6 個 channel。43.14 節

```
//初始化 Table 設定
rt =_DMC_B02_tpcmp_initial(l_nCardNo,
                           l_nTpcmpIndex,
                           l_nQepIndex,
                           l_nGpioPin,
                           m_bInverse,
                           m_bInterrupt);
```

DELTA CONFIDENTIAL

TPCMP_SET_6CH_TAB:產生 table 設定。43.19 節

```
        //填入自定義的 Table 表資料 放入 l_nPositionTable 陣列內
for (unsigned long i=0; i<l_nTableSize; ++i)
    {
        if (l_nPositionTable != NULL)
        {
            l_nPositionTable[i] = 10*(i+1)+j;
        }
        if (l_nPositionLevel != NULL)
        {
            l_nPositionLevel[i] = (i%2)?1:0;
        }
    }

    if (GetCheckValue(IDC_CHK_TAB_LEVEL))
    {
    //Level 準位觸發
        rt =  _DMC_B02_tpcmp_compare_position_table_level(l_nCardNo, nFifoIndex,
          l_nPositionTable, l_nPositionLevel, l_nTableSize);
    }
    else
    {
    //一般準位觸發
        rt =   _DMC_B02_tpcmp_compare_position_table(l_nCardNo,nFifoIndex,
          l_nPositionTable, l_nTableSize);
    }
```

TPCMP_RESET_ALL:清除所有 table 設定。43.22 節

```
    // 清除 TPCMP 的 COUNT
    rt =_DMC_B02_tpcmp_compare_trigger_count_clear(l_nCardNo, nFifoIndex);
    // 清除 TPCMP 的 TABLE
    rt =_DMC_B02_tpcmp_compare_position_table_reset(l_nCardNo, nFifoIndex);
```

相關 API 內部操作請參考範例 B02_Sample 或 43.14 章節 B02 相關 API。

DELTA CONFIDENTIAL

3.32 Table Motion 範例

3.32.1 函式列表

表 3.32

函式名稱
_DMC_01_start_tablemotion
_DMC_01_start_tablemotion_cyclemode
_DMC_01_start_tablemotion_buffer
_DMC_01_get_tablemotion_length
_DMC_01_tablemotion_feedrate_overwrite
_DMC_01_tablemotion_feedhold
_DMC_01_set_tablemotion_blending_ratio
_DMC_01_start_tablemotion_profile_planning_para
_DMC_01_start_tablemotion_profile_planning
_DMC_01_start_tablemotion_profilemode
_DMC_01_set_axis_normalamf_enable
_DMC_01_set_normalamf_enable

- **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

PCI_DMC_A02/B02 軸卡
●

DELTA CONFIDENTIAL

3.32.2 應用範例

程式外觀

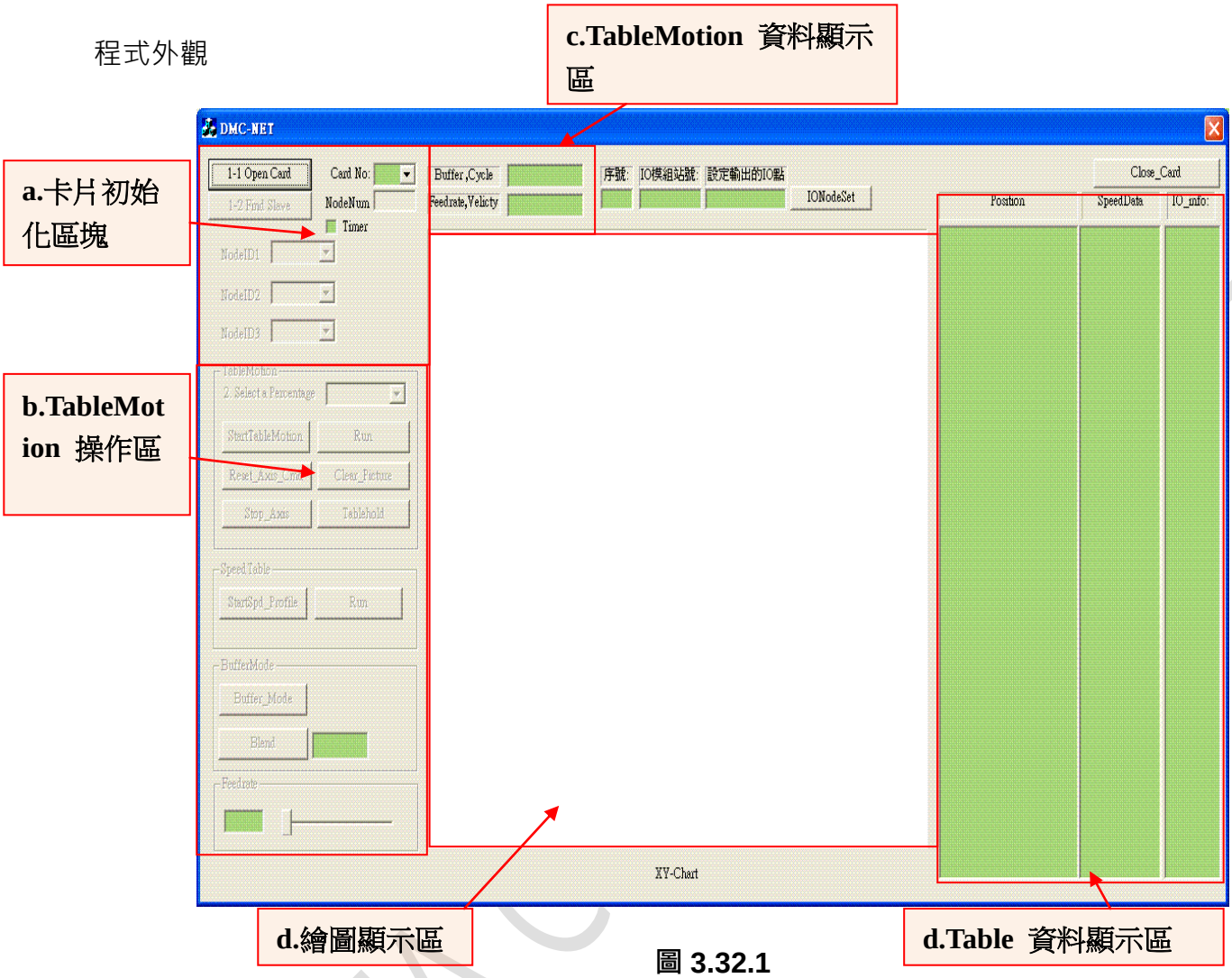


圖 3.32.1

1)軸卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-A or B 軸卡。

按壓圖 3.31.2 中的「Open」鍵開始執行軸卡初始化與自動搜尋驅動器，請連接三台(三軸)驅動器。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2)設定 TableMotion 相關畫面:

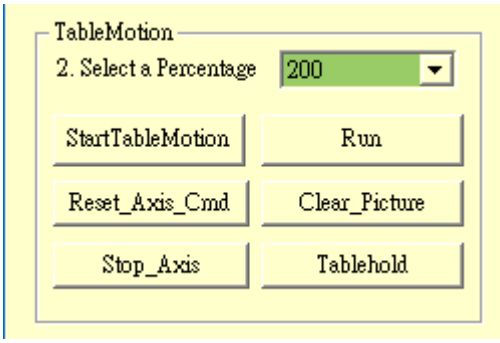


圖 3.32.2

StartTableMotion 項目：啟動 TableMoion 相關設定。

```
U16 slavearray[3]={1,2,0};
U16 slaveSlotID[3]={0,0,0};
U32 Speed =32000;
F64 acc =0.1;
I16 amfno =1;
```

//設定卡號、NodeID 陣列、SlotID 陣列、TableNo 表號碼、點位資料陣列、資料大小、軸數、最大速度、加速度、後加減處理道數、執行次數、IO 資料陣列。

```
rt = _DMC_01_start_tablemotion(gDMCCardNo, slavearray, slaveSlotID, TableNo, Positon_Table, 1201, 2, Speed, acc, amfno, 1, IOinfo_Table);
```

Run 項目：執行 TableMotion Cyclemode。

```
rt = _DMC_01_start_tablemotion_cyclemode(gDMCCardNo, TableNo);
```

Reset _Axis_Cmd 項目：執行所以軸 CMD 重新設定。

Clear_Picture 項目：執行清除繪圖區。

Stop_Axis 項目：執行軸運動停止。

Tablehold 項目：執行 **Tablehold**。

U16 value_hold =1;開啟 FeedHold

DELTA CONFIDENTIAL

```
rt= _DMC_01_tablemotion_feedhold (gDMCCardNo, TableNo, value_hold, 0.1 );
```

3)Speed Mode 操作畫面:

圖 3.32.3

StartSpd_Profile 項目：執行 TableMotion Cyclemode。

```
U16 rt;
U16 slavearray[3]={1,2,0};
U16 slaveSlotID[3]={0,0,0};
I32 accdec[12] ={0};

//設定各軸運動速度限制
for(int i=0;i<12;i++)
{
    accdec[i] =1280000/2;
}

I16 TableNo = 0;

//設定速度規劃模式的相關參數。最大速須注意是否超過 1ms 運動時所能負擔的最大 Pulse 數
例如:350000000/1000(1ms)=350000 (最大速)
rt = _DMC_01_start_tablemotion_profile_planning_para(gDMCCardNo, TableNo, 500, 350000 *
0.3, 0, 4000, 15, 50000, 10000);

//根據_DMC_01_start_tablemotion_profile_planning_para 設定的參數，計算出各運動間的加減
速時間以及各運動間適當的轉角速度。
rt = _DMC_01_start_tablemotion_profile_planning(gDMCCardNo, slavearray, slaveSlotID,
TableNo, accdec, Positon_Table, 1201, 2, Speed_Table, IOinfo_Table);
```

Run 項目：執行 TableMotion Cyclemode。

```
//執行 ProfileMode。
I16 TableNo =0;
```

DELTA CONFIDENTIAL

```
rt = _DMC_01_start_tablemotion_profilemode(gDMCCardNo, TableNo);
```

4)Buffer Mode 操作畫面:

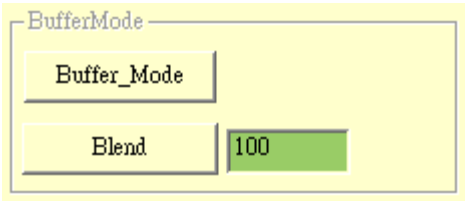


圖 3.32.4

Buffer_Mode 項目：執行 **Buffer_Mode**。

```
//使用此 API 可以一直新增資料,但是以下參數在第一次使用_DMC_01_start_tablemotion_buffer 時就固定了 ： 軸數、最大速、濾波時間、濾波次數。新增資料當下必須檢查 Buffer 是否超出剩餘可用數量，最多 2000 筆。
    U16 rt;
    U16 slavearray[3]={1,2,0};
    U16 slaveSlotID[3]={0,0,0};

    U32 Speed =32000;
    F64 acc =0.1;
    I16 amfno =1;

//注意在填入 Buffer 資料時，當前運動狀態下的資料筆數與欲填入資料筆數相加不可大於 2000 筆
//DataSize = Current Buffer length + Add Buffer Size
    rt = _DMC_01_start_tablemotion_buffer(gDMCCardNo, slavearray, slaveSlotID, TableNo, Positon_Table, 1201, 2, Speed, acc, amfno, IOinfo_Table);
```

Blend 項目：執行 **Blend 運動混和百分比(運動與運動之間的等待時間)**。

```
//混合百分比:
//Ratio = 100 --> 100%，運動之間沒有等待，和原路徑誤差大。
//Ratio = 0--> 0%，會等這個單節結束下個運動才起動，基本上會疊合到運動間的端點位置。
U32 Ratio =100;//100%
    rt=   _DMC_01_set_tablemotion_blending_ratio (gDMCCardNo, TableNo, Ratio);
```

DELTA CONFIDENTIAL

5)Feedrate 操作畫面:

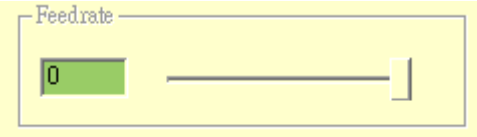


圖 3.32.5

SLIDER 項目：控制 Feedrate 大小最大 500。

// current slider value

I32 pos = pSlidCtrl->GetPos();//這裡是由取得當前 **SLIDER** 元件的設定值來作為 FeedRate 參數。

rt=_DMC_01_tablemotion_feedrate_overwrite (gDMCCardNo, 0, pos, 0.1);
rt=_DMC_01_tablemotion_feedrate_overwrite (gDMCCardNo, 1, pos, 0.1);

6)IO 操作畫面:

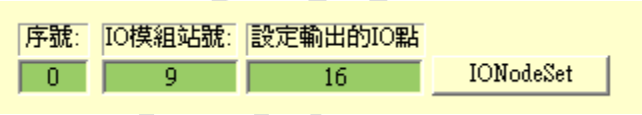


圖 3.32.6

序號：設定點位的排列號碼 0~12。

IO 模組站號：要輸出的 IO, NodeID。

設定輸出的 IO 點：輸出的 IO 點 bit 值。

//先填入 Table 在執行 IONodeSet:

IONodeSet：設定 IO 資料。

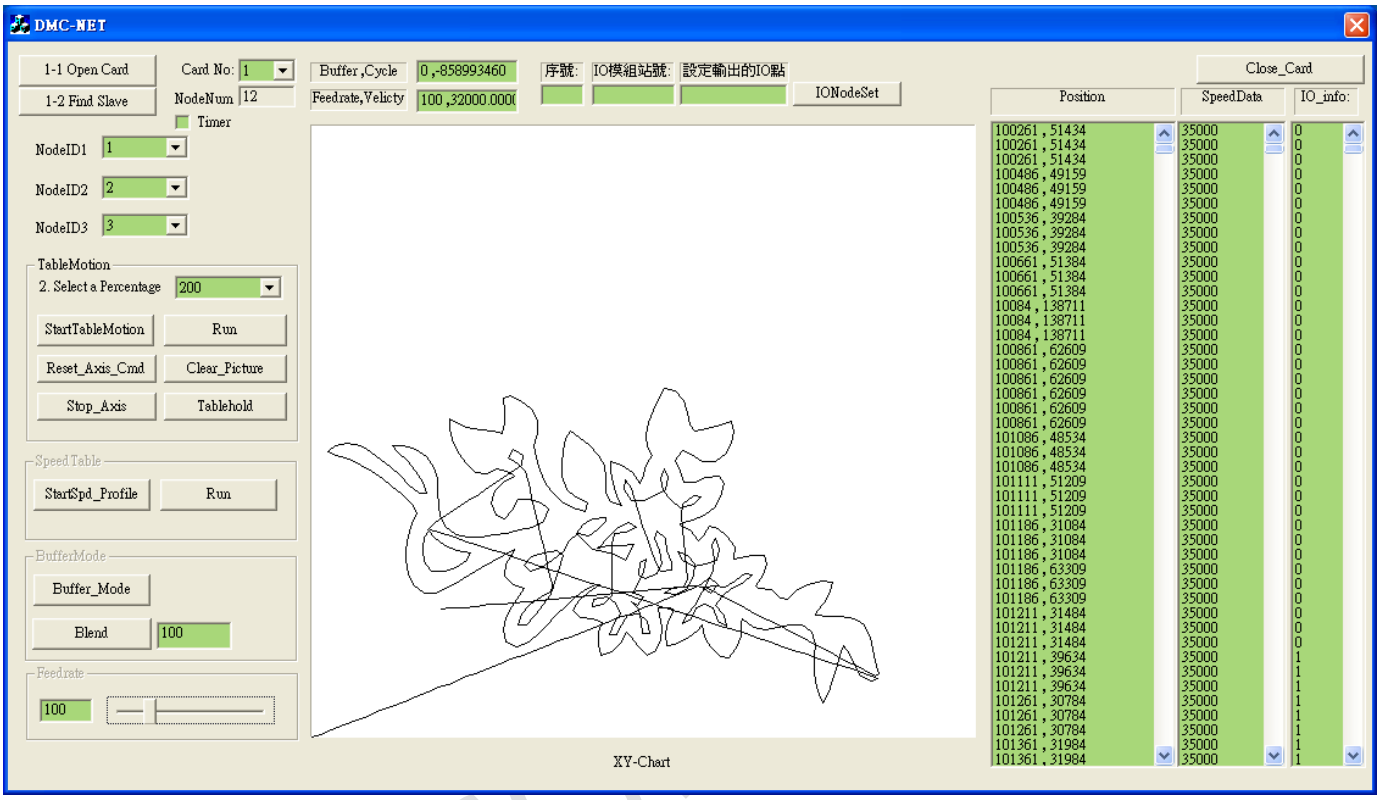
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，設置的模組輸出值才會從輸出埠輸出。

//設定 IO 資料表。控制每個總內線的 IO 模組狀態。定義 Table 運動過程中每個運動間 IO 的輸出入狀態。
U16 rt;
rt = _DMC_01_set_tablemotion_ioctl_mapping (gDMCCardNo, TableNo, Serial, RmNodeid,

DELTA CONFIDENTIAL

0,OuputIONum);

6)本範例結果:



DELTA CONFIDENTIAL

3.33 B02 Latch 範例

3.33.1 函式列表

表 3.33

函式名稱
_DMC_B02_ext_io_latch_initial
_DMC_B02_ext_io_latch_function
_DMC_B02_ext_io_latch_enable
_DMC_B02_latch_fifo_reset
_DMC_B02_latch_fifo_status
_DMC_B02_latch_fifo_size
_DMC_B02_latch_fifo_get_qep_counter

- **屬性**

PCI_DMC_B02 軸卡
●

DELTA CONFIDENTIAL

3.33.2 應用範例

程式外觀

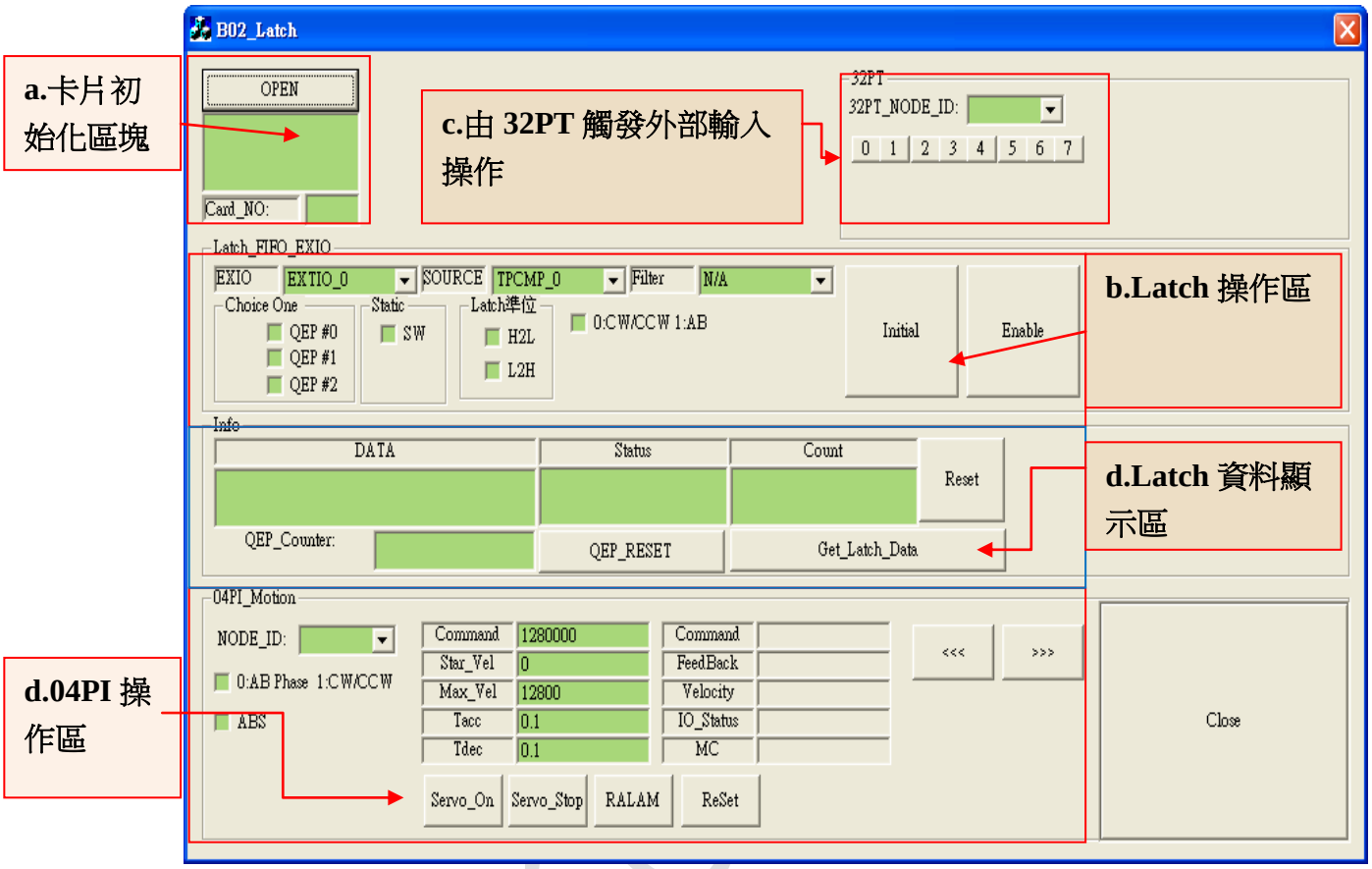


圖 3.33.2.A

1)軸卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-B02 軸卡。

按壓圖 3.31.2 中的「Open」鍵開始執行軸卡初始化與自動搜尋 04PI 與 32PT 模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2)卡號、站號及開啟 Timer:

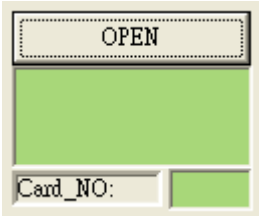


圖 3.33.2.b

Open 項目：開啟卡片取得卡號(Card_NO)與卡的版本。

3)設定 Latch　IO 開啟條件的相關設定:

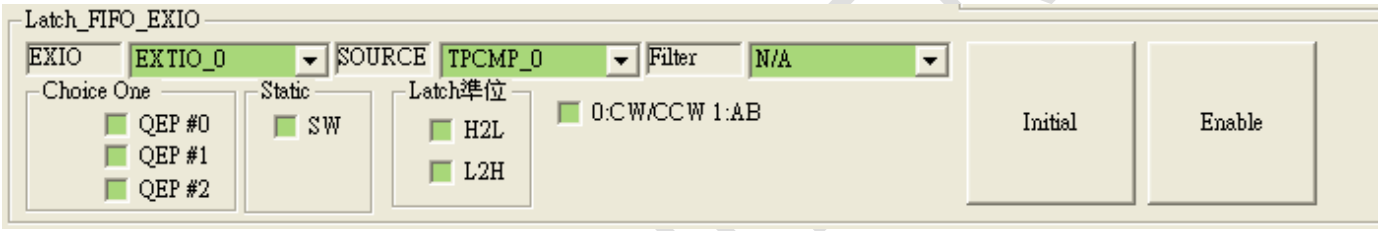


圖 3.33.2.c

Latch_FIFO_EXIO 項目：設定 Latch 的外部觸發訊號源，來決定 Latch 的觸發條件。

ChoiceOne 項目：根據外部實體佈線(QA、QB)與 B02 軸卡連接的 EnCoder 來決定(參照 UserGuide-B02 軸卡腳位)訊號來源。

Static 項目:內部測試用項目，勾選此項目 Source 選擇(SW_EN)操作順序 1.Initial->Enable

2.取消勾選 SW->Initial->Enable

此時按下 Get_Latch_Data(圖 3.33.d)來查看當下 Latch 的位置，驗證 Latch 功能正確性。

DELTA CONFIDENTIAL

圖 3.33.2.d

Latch 準位項目：設定來源觸發的準位高低，H2L 算一次 Latch 成功或是 L2H 算一次 Latch 成功。

CW/CCW or AB：可選擇外部訊號 Type。 (例:勾選為 AB)

EXIO 項目：可以選擇當下 Latch 的位置應輸入對應的 IO 腳位是哪隻腳 EXTIO0~EXTIO7。

SOURCE:1.TPCMP 、2.GPIO_IN 、3.SW_EN

TPCMP ->表示使用 B02 內部 Table Compare(參閱四十三章)作為 Latch 訊號來源。

GPIO_IN->使用外部訊號連接到對應的 B02 (GPIO_INput)

SW_EN->則為軟體觸發使用。

Initial 程式流程如下:

```
//初始化 Latch 相關設定
rt= _DMC_B02_ext_io_latch_initial(l_nCardNo, l_nIoPin,
        g_ExtioInformation[l_nIoPin].m_nSource,
        g_ExtioInformation[l_nIoPin].m_nFilter);

//決定 Latch 的腳位與訊號關聯
rt=   _DMC_B02_ext_io_latch_function(l_nCardNo, l_nIoPin,
        g_ExtioInformation[l_nIoPin].m_bQep0,
        g_ExtioInformation[l_nIoPin].m_bQep1,
        g_ExtioInformation[l_nIoPin].m_bQep2,
        g_ExtioInformation[l_nIoPin].m_bSw,
        g_ExtioInformation[l_nIoPin].m_bH2L,
        g_ExtioInformation[l_nIoPin].m_bL2H);
```

Enable 程式流程如下:

```
//啟用 Latch 當下設定的 Pin 腳
g_ExtioInformation[l_nIoPin].m_bEnable   = 1;
```

DELTA CONFIDENTIAL

```
rt = _DMC_B02_ext_io_latch_enable(l_nCardNo, l_nIoPin,
g_ExtioInformation[l_nIoPin].m_bEnable);
```

4)Latch 資料顯示區:

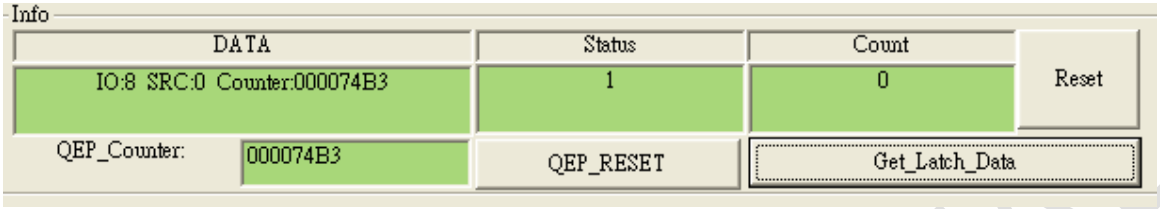


圖 3.33.2.e

Info 項目:顯示 Latch 點資訊。DATA:顯示擷取的資料點。IO:為代表 Bit 位置、Status:為 1 表示無資料狀態、0 表示 FIFO 有紀錄值、Count: 代表現在的紀錄資料筆數(如圖 3.33.e)。

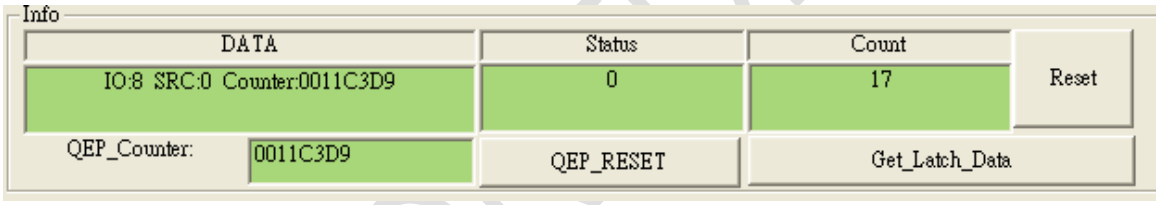


圖 3.33.2.f

DELTA CONFIDENTIAL

QEP_Counter 項目:顯示軸卡現在 Pluse 紀錄位置。

QEP_RESET 項目:表示重置軸卡紀錄的 EnCoder 資料。

```
//清除當下指定的 QEP_Count 值
unsigned char l_nQepIndex = g_nQepIndex;
rt = _DMC_B02_qep_set_reload_counter(l_nCardNo, l_nQepIndex, 0);
```

Get_Latch_Data 項目:代表從當前 FIFO 取回一筆 Latch 資料顯示在 DATA 區。

```
long l_nQepCounter = 0;
unsigned char l_nIoType = 0xFF;
unsigned char l_nQepSource = 0xFF;
//取出 FIFO 內 Latch 的內容值:有 QEP 計數值(Pluse)、腳位資訊、QEP 來源
rt =_DMC_B02_latch_fifo_get_qep_counter(l_nCardNo, &l_nQepCounter, &l_nIoType,
&l_nQepSource);
```

Reset 項目:可清空 FIFO 內全部資料。

```
//清除 FIFO 內所有 Latch 資訊
rt =_DMC_B02_latch_fifo_reset(l_nCardNo, true);
rt =_DMC_B02_latch_fifo_reset(l_nCardNo, false);
```

5)04PI 控制區:

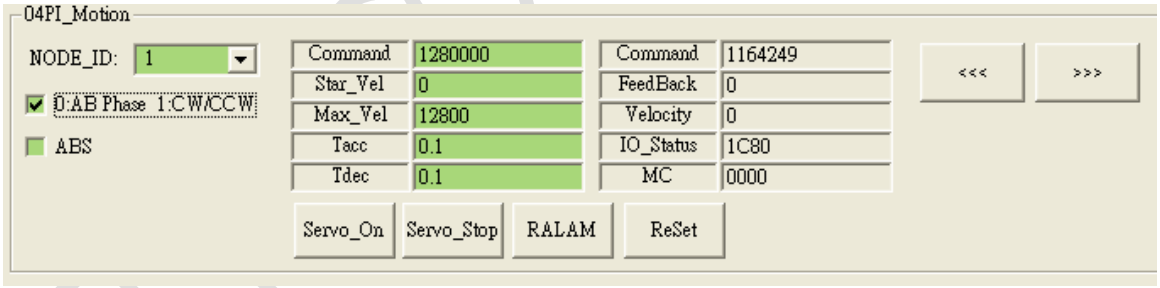


圖 3.33.2.g

0:AB Phase 1.CW/CCW 項目:勾選後為 CW/CCW 模式。
Servo_On:啟動馬達或伺服。
Servo_Stop:停止啟動馬達或伺服。
RALAM:清除異警。
ReSet:重置伺服狀態。
<<< >>>項目:控制運動方向。

DELTA CONFIDENTIAL

6)32PT 觸發控制區:

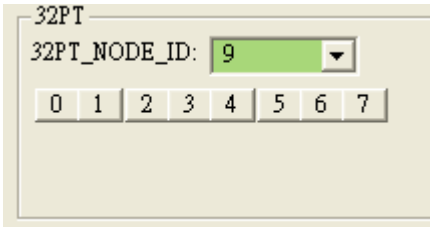


圖 3.33.2.h

32PT 項目:區域內部 7 個 buttom 按鈕都為 OUTPUT 輸出。

當(圖 3.33.2.c)Source 選為 EXTIN 則表示訊號由外部輸入。

此範例使用 32PT-output Y0~Y7 作為訊號輸出。連接到 B02 軸卡的輸入端 31~38 腳位。

例:當 EXTIO_0 被 Enable 起來表示 EXTIO_0 當下可以 Latch 住 B02 軸卡 31 腳位的訊號。

以此類推。

如下圖設定

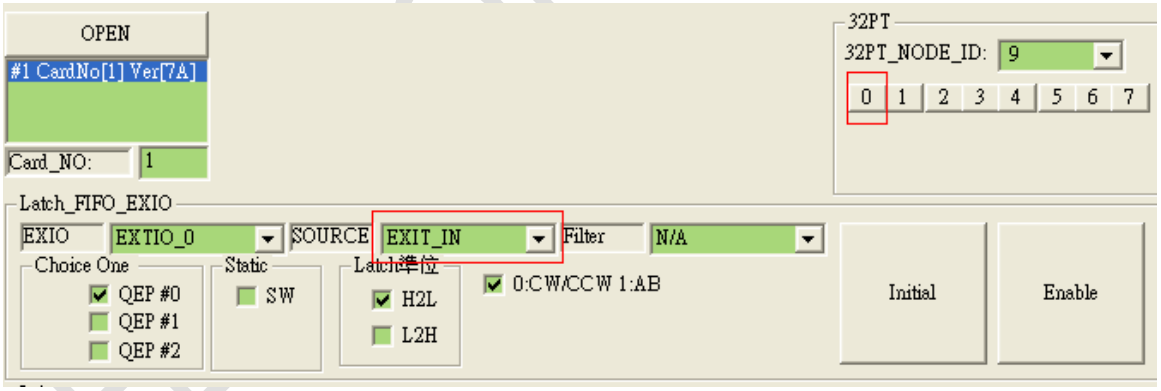


圖 3.33.2.i

所以按下 UI 上第 0 個位置表示觸發外部訊號輸入，此時為 H2L 所以第 0 個位置必須被按壓一次後在次解除下壓，代表訊號已經由 low->high->low 被觸發一次，則當下觸發的 QEP 位置則會被紀錄下來。

DELTA CONFIDENTIAL

3.33.3 應用範例

程式外觀

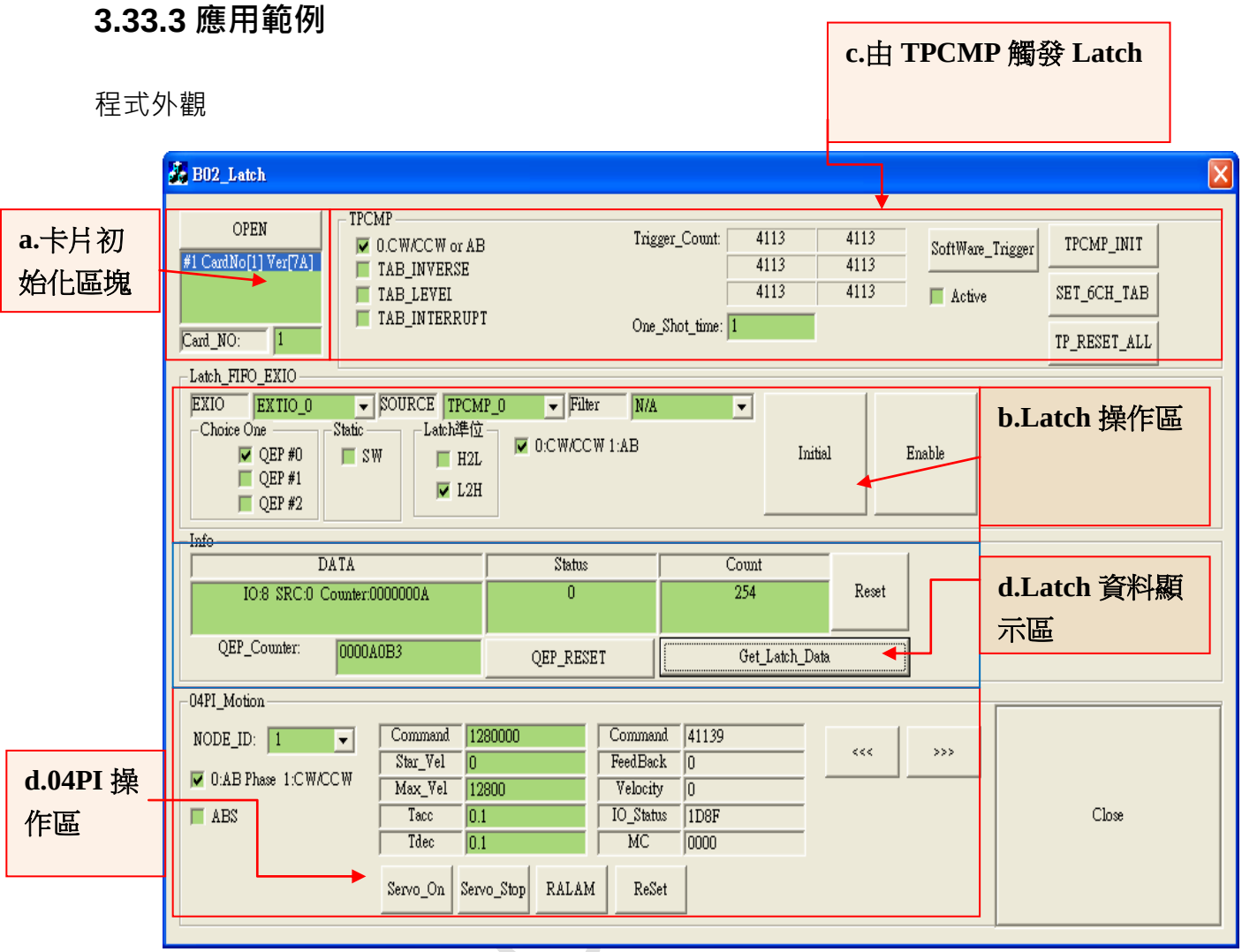


圖 3.33.3.A

1)軸卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-B02 軸卡。

按壓圖 3.31.2 中的「Open」鍵開始執行軸卡初始化與自動搜尋 04PI 與 32PT 模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

2)卡號、站號及開啟 Timer:

圖 3.33.3.b

Open 項目：開啟卡片取得卡號(Card_NO)與卡的版本。

3)設定 Latch　IO 開啟條件的相關設定:

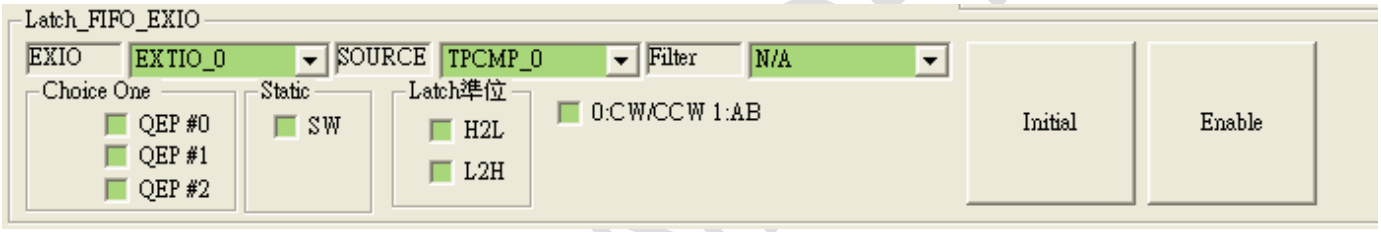


圖 3.33.3.c

Latch_FIFO_EXIO 項目：設定 Latch 的外部觸發訊號源，來決定 Latch 的觸發條件。

ChoiceOne 項目：根據外部實體佈線(QA、QB)與 B02 軸卡連接的 EnCoder 來決定(參照 UserGuide-B02 軸卡腳位)訊號來源。

Static 項目:內部測試用項目，勾選此項目 Source 選擇(SW_EN)操作順序 1.Initial->Enable

2.取消勾選 SW->Initial->Enable

此時按下 Get_Latch_Data(圖 3.33.d)來查看當下 Latch 的位置，驗證 Latch 功能正確性。

DELTA CONFIDENTIAL

圖 3.33.3.d

Latch 準位項目：設定來源觸發的準位高低，H2L 算一次 Latch 成功或是 L2H 算一次 Latch 成功。

CW/CCW or AB：可選擇外部訊號 Type。 (例:勾選為 AB)

EXIO 項目：可以選擇當下 Latch 的位置應輸入對應的 IO 腳位是哪隻腳 EXTIO0~EXTIO7。

SOURCE:1.TPCMP 、2.GPIO_IN 、3.SW_EN

TPCMP ->表示使用 B02 內部 Table Compare(參閱四十三章)作為 Latch 訊號來源。

GPIO_IN->使用外部訊號連接到對應的 B02 (GPIO_INput)

SW_EN->則為軟體觸發使用。

Initial 程式流程如下:

```
//初始化 Latch 相關設定
rt= _DMC_B02_ext_io_latch_initial(l_nCardNo, l_nIoPin,
        g_ExtioInformation[l_nIoPin].m_nSource,
        g_ExtioInformation[l_nIoPin].m_nFilter);

//決定 Latch 的腳位與訊號關聯
rt=   _DMC_B02_ext_io_latch_function(l_nCardNo, l_nIoPin,
            g_ExtioInformation[l_nIoPin].m_bQep0,
            g_ExtioInformation[l_nIoPin].m_bQep1,
            g_ExtioInformation[l_nIoPin].m_bQep2,
            g_ExtioInformation[l_nIoPin].m_bSw,
            g_ExtioInformation[l_nIoPin].m_bH2L,
            g_ExtioInformation[l_nIoPin].m_bL2H);
```

DELTA CONFIDENTIAL

Enable 程式流程如下:

```
//啟用 Latch 當下設定的 Pin 腳
g_ExtioInformation[l_nIoPin].m_bEnable   = 1;

rt = _DMC_B02_ext_io_latch_enable(l_nCardNo, l_nIoPin,
g_ExtioInformation[l_nIoPin].m_bEnable);
```

4)Latch 資料顯示區:

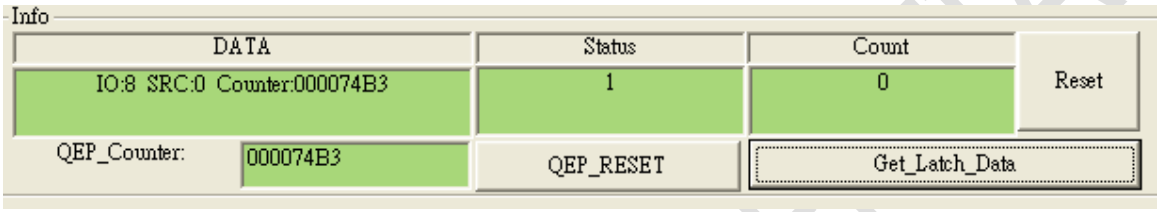


圖 3.33.3.e

Info 項目:顯示 Latch 點資訊。DATA:顯示擷取的資料點。IO:為代表 Bit 位置、Status:為 1 表示無資料狀態、0 表示 FIFO 有紀錄值、Count: 代表現在的紀錄資料筆數(如圖 3.33.e)。

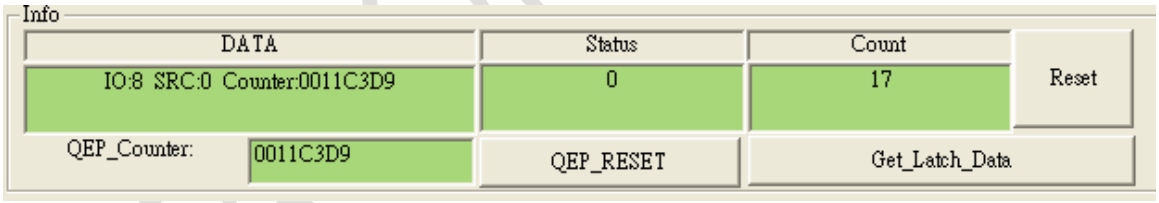


圖 3.33.3.f

QEP_Counter 項目:顯示軸卡現在 Pluse 紀錄位置。

QEP_RESET 項目:表示重置軸卡紀錄的 EnCoder 資料。

```
//清除當下指定的 QEP_Count 值
unsigned char l_nQepIndex = g_nQepIndex;
rt = _DMC_B02_qep_set_reload_counter(l_nCardNo, l_nQepIndex, 0);
```

Get_Latch_Data 項目:代表從當前 FIFO 取回一筆 Latch 資料顯示在 DATA 區。

```
long l_nQepCounter = 0;
unsigned char l_nIoType = 0xFF;
unsigned char l_nQepSource = 0xFF;
```

DELTA CONFIDENTIAL

```
//取出 FIFO 內 Latch 的內容值:有 QEP 計數值(Pluse)、腳位資訊、QEP 來源
rt =_DMC_B02_latch_fifo_get_qep_counter(l_nCardNo, &l_nQepCounter, &l_nIoType,
&l_nQepSource);
```

Reset 項目:可清空 FIFO 內全部資料。

```
//清除 FIFO 內所有 Latch 資訊
rt =_DMC_B02_latch_fifo_reset(l_nCardNo, true);
rt =_DMC_B02_latch_fifo_reset(l_nCardNo, false);
```

5)04PI 控制區:

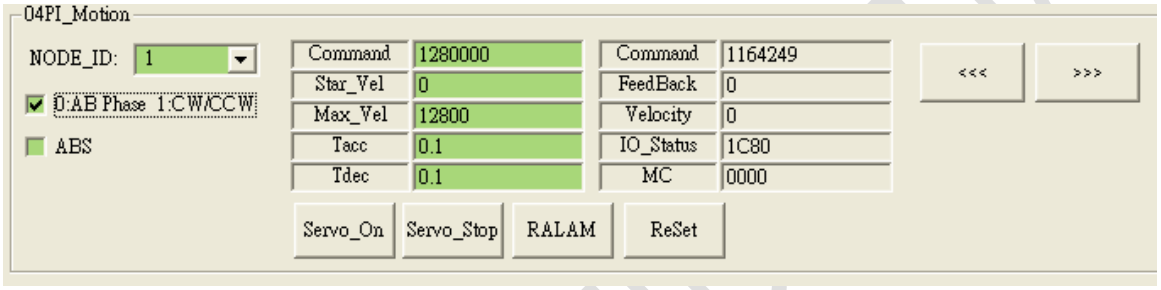


圖 3.33.3.g

0:AB Phase 1.CW/CCW 項目:勾選後為 CW/CCW 模式。
Servo_On:啟動馬達或伺服。
Servo_Stop:停止啟動馬達或伺服。
RALAM:清除異警。
ReSet:重置伺服狀態。
<<< >>>項目:控制運動方向。

DELTA CONFIDENTIAL

6)TPCMP 觸發控制區:

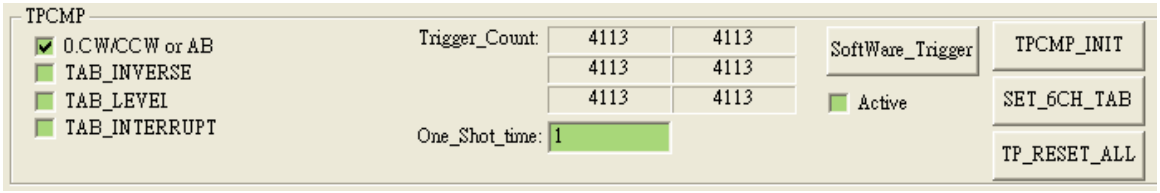


圖 3.33.3.h

TPCMP 項目:則使用 Table Compare 來作為來源端使用。

設定方式同範例 3.31.3 設定圖 3.33.3.d 內 選擇 Source 為 TPCMP_0，代表 Latch 的腳位訊號是從 Table 的第一支 CMP 腳位 B02 的 7 號腳位送出脈波。勾選 H2L 或 L2H 完成後勾選模式選擇，按下 Initial 與 Enable 後啟用此腳位 Latch 功能。

本範例為同時開啟 6 個 Table Compare 腳位輸出，所以當有其中一支腳位被開啟則會啟用 Latch 功能。

啟動 Table Compare 如同範例 3.31.3 先清除狀態 TP_RESET_ALL--->初始化 Table 設定 (TPCMP_INIT) --->設定 6 個 Table 內容(SET_6CH_TAB)

接者進入 04PI 控制區啟動 7 號腳位連接的 CMP 位置，QAQB 則是連接在對應的 NODE 站號上，啟動實體線路連接的站號，下指令控制移動。

DELTA CONFIDENTIAL

3.34 Motion Buffer 範例

3.34.1 函式列表

表 3.34

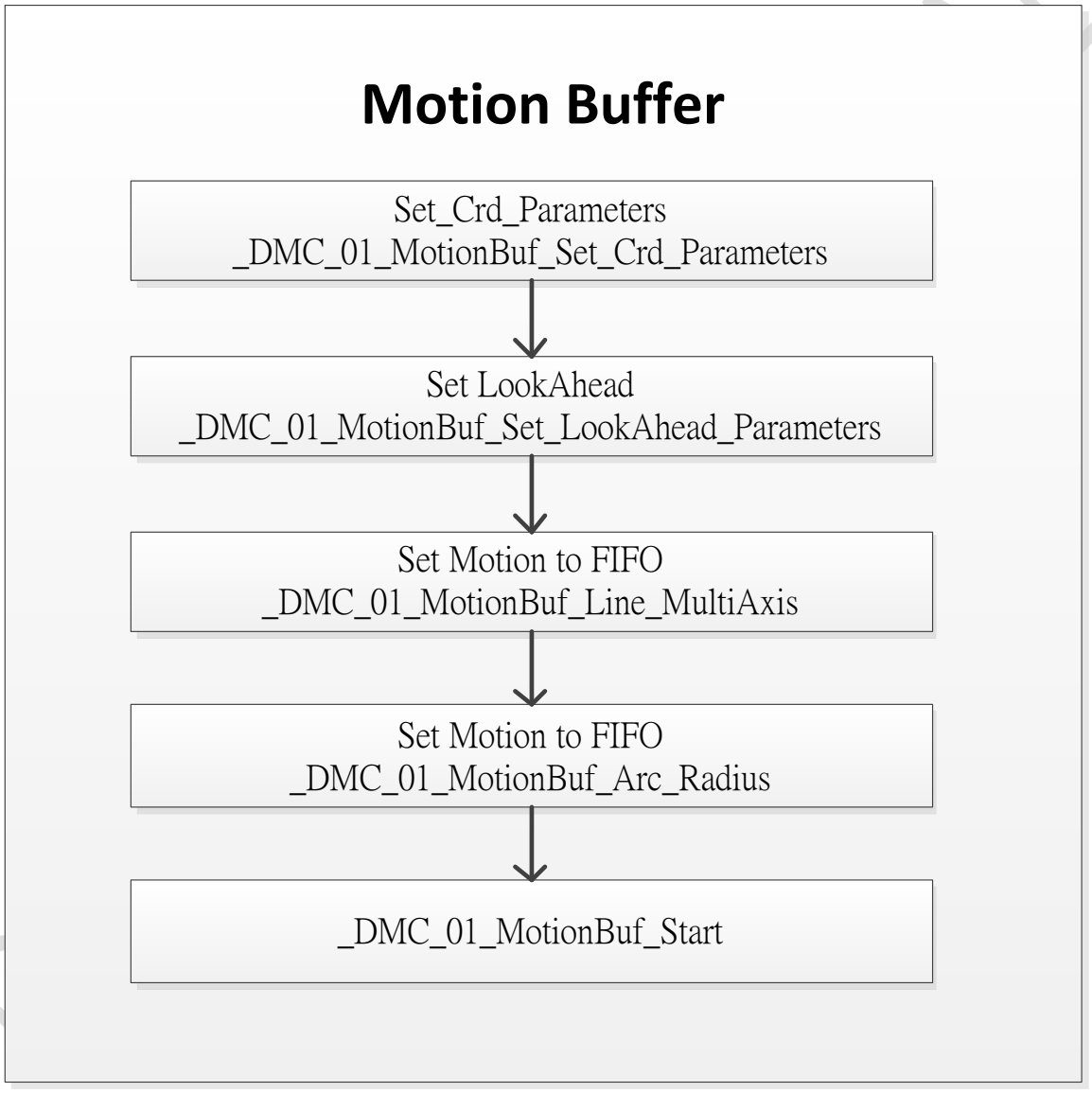
函式名稱
_DMC_01_MotionBuf_Set_Crd_Parameters
_DMC_01_MotionBuf_Get_Crd_Parameters
_DMC_01_MotionBuf_Line_MultiAxis
_DMC_01_MotionBuf_Line_MultiAxis_G00
_DMC_01_MotionBuf_Arc_Radius
_DMC_01_MotionBuf_Arc_CenterPos
_DMC_01_MotionBuf_Set_LookAhead_Parameters
_DMC_01_MotionBuf_Start
_DMC_01_MotionBuf_Stop
_DMC_01_MotionBuf_Get_Position
_DMC_01_MotionBuf_Get_Velocity
_DMC_01_MotionBuf_Feedrate_Override
_DMC_01_MotionBuf_Get_Motion_Status
_DMC_01_MotionBuf_Set_Stop_Deceleration
_DMC_01_MotionBuf_Get_Stop_Deceleration
_DMC_01_MotionBuf_SetUserSegNum
_DMC_01_MotionBuf_GetUserSegNum
_DMC_01_MotionBuf_Clear_Buffer
_DMC_01_MotionBuf_Single_Step
_DMC_01_MotionBuf_BufGear
_DMC_01_MotionBuf_BufMove
_DMC_01_MotionBuf_IOControl
_DMC_01_MotionBuf_Advanced_IOControl
_DMC_01_MotionBuf_Delay
_DMC_01_MotionBuf_Get_Remain_Buffer_Size
_DMC_01_MotionBuf_SoftLimit_On
_DMC_01_MotionBuf_SoftLimit_Off
_DMC_01_MotionBuf_Clear_Crd
_DMC_01_MotionBuf_Disable
_DMC_01_MotionBuf_Helix

DELTA CONFIDENTIAL

■ 屬性

PCI_DMC_A/B 軸卡
●

3.34.2 流程圖



3.34.3 應用範例

程式外觀

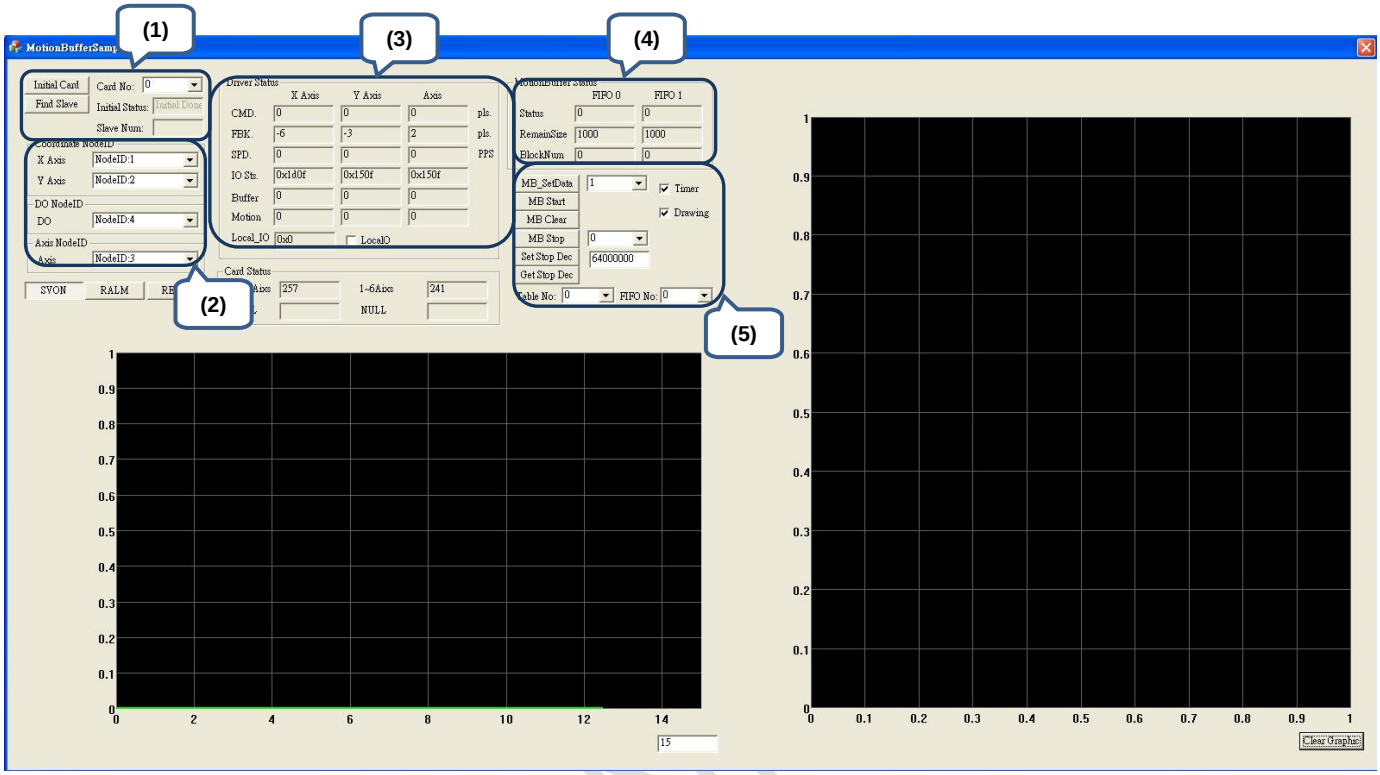


圖 3.34.3.A

1)軸卡開啟與初始化

按壓圖 3.34.3.b 中的「Open card」鍵開始執行軸卡初始化。
按壓圖 3.34.3.b 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

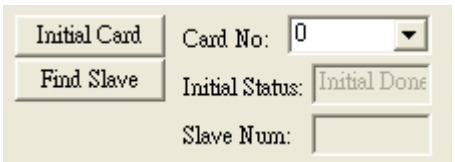


圖 3.34.3.b

DELTA CONFIDENTIAL

2)設定站號:

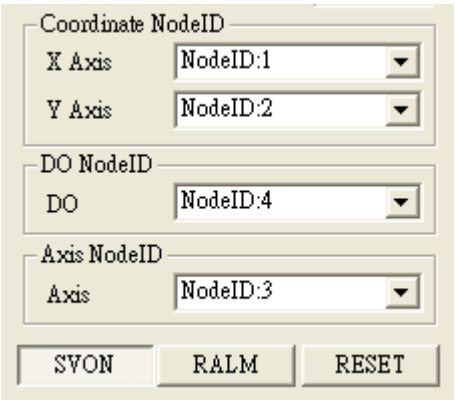


圖 3.34.3.c

Coordinate NodeID 項目：插補運動軸號
DO NodeID 項目：Motion Buffer IO 控制站號
Axis NodeID 項目：跟隨軸站號

3):從站狀態顯示

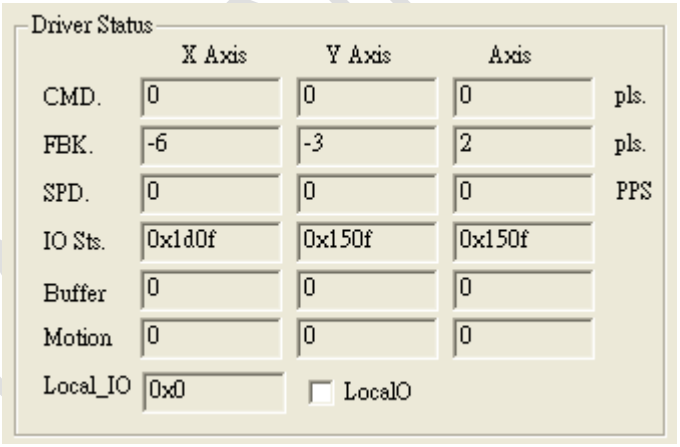


圖 3.34.3.d

Local IO 項目：勾選 Local IO，可控制 PCI-DMC-A02/F02 軸卡，並顯示 Output 狀態。

DELTA CONFIDENTIAL

4)Motion Buffer: 插補運動狀態

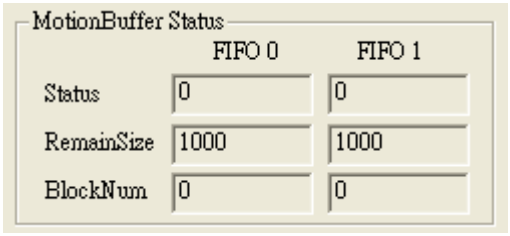


圖 3.34.3.e

顯示 Motion Buffer FIFO 0、FIFO 1 狀態，
Status 項目:插補運動狀態:0:未啟動 1:啟動
RemainSize 項目:回傳插補緩存區剩餘空間
BlockNum 項目:回傳當前已經完成的插補段數

5)Motion Buffer: 插補運動設定與啟動

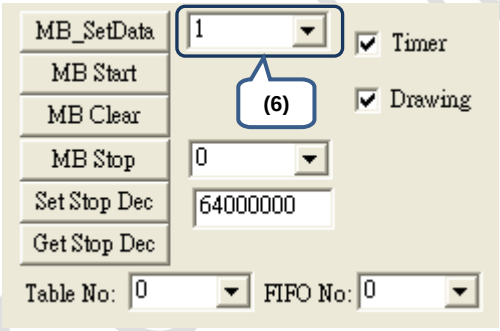


圖 3.34.3.f

MB_SetData 項目:設置 Motion Buffer 插補運動功能 Case
MB Start 項目:啟動插補運動
MB Clear 項目:清除插補緩存區內的插補數據
MB Stop 項目:停止插補運動
Set Stop Dec 項目:設置插補運動平滑停止、急停合成加速度
Get Stop Dec 項目:取得插補運動平滑停止、急停合成加速度
Table No 項目:選擇 Motion Buffer Table
FIFO No 項目:選擇 Motion Buffer FIFO
Timer 項目:勾選啟動顯示 4)插補運動狀態
Drawing 項目:開啟繪圖

啟動/停止 MotionBuffer 運動:
1.選取 6) case
2.按 MB_SetData 鍵
3.按 MB Start 鍵

DELTA CONFIDENTIAL

4.按 MB Stop 鍵，可選擇 Hold、平滑停止、緊急停止

6)Motion Buffer: 插補運動 Case

Case1 項目:Line_MultiAxis、Arc_Radius
Case2 項目:Arc_CenterPos(絕對模式)
Case3 項目:Arc_CenterPos(相對模式)
Case4 項目:刀向跟隨功能，啟動某個軸跟隨運動
Case5 項目:刀向跟隨功能，啟動某個軸點位運動(非模態指令)
Case6 項目:刀向跟隨功能，啟動某個軸點位運動(模態指令)
Case7 項目: Single_Step 單步功能
Case8 項目:Delay 延遲功能
Case9 項目: MultiAxis_G00 功能
Case10 項目: LookAhead_Parameters (轉角降速度)
Case11 項目: LookAhead_Parameters (圓弧速度修正)
Case12 項目: FIFO 0 切換 FIFO 1 運動
Case13 項目: Feedrate_Override 功能
Case14 項目: Set_Stop_Deceleration 功能
Case15 項目: IOControl 輸出功能
Case16 項目: Advanced_IOControl(延遲輸出)
Case17 項目: Advanced_IOControl(提前輸出)
Case18 項目: SetUserSegNum 插補段號功能
Case19 項目: SoftLimit_On/Off 軟體極限功能

DELTA CONFIDENTIAL

3.99 歷程記錄器

3.99.1 函式列表

表 3.99

函式名稱
_misc_set_record_debuging
_misc_open_record_debuging_file

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

DELTA CONFIDENTIAL

3.99.2 流程圖

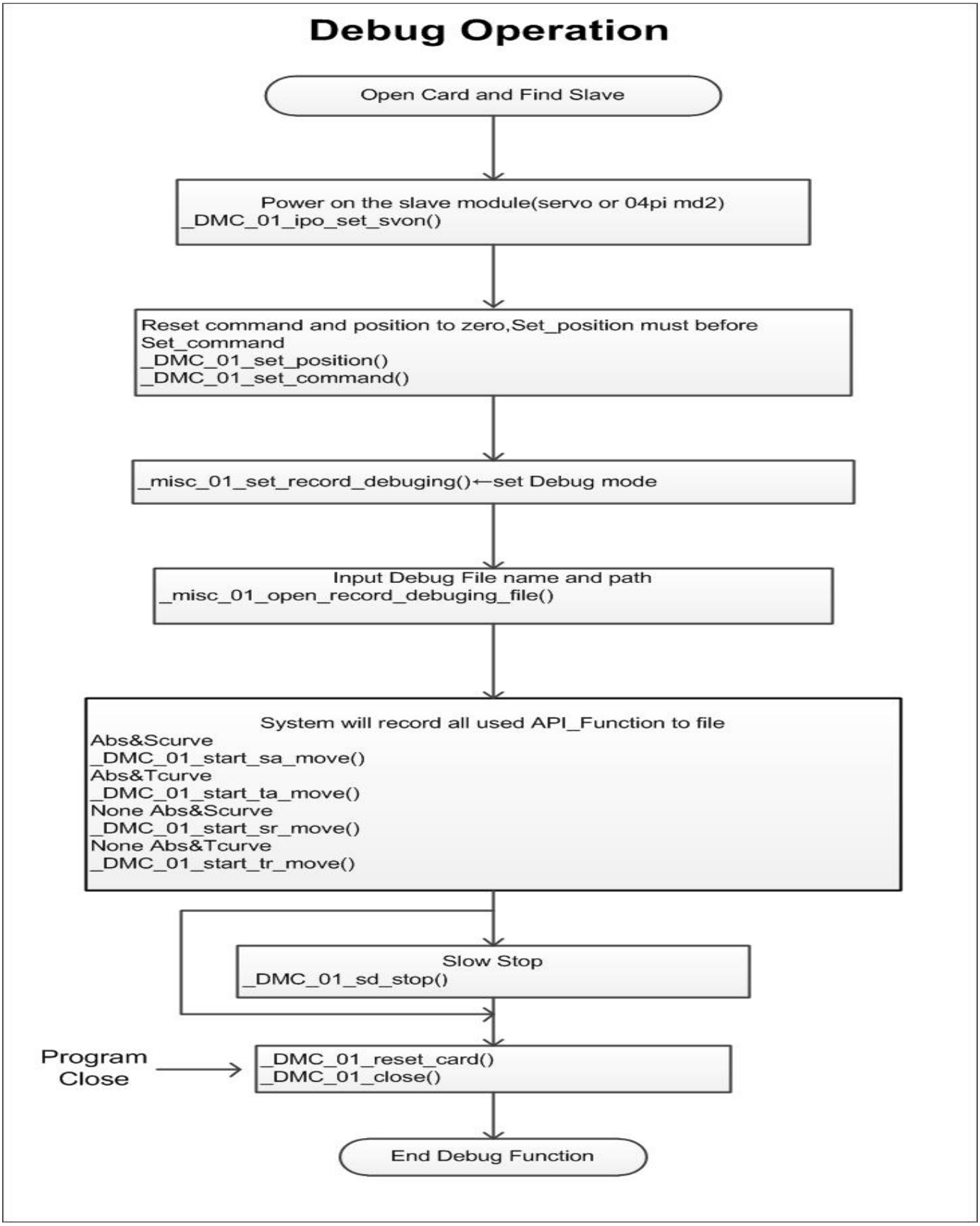


圖 3.990

3.99.3 應用範例

程式外觀

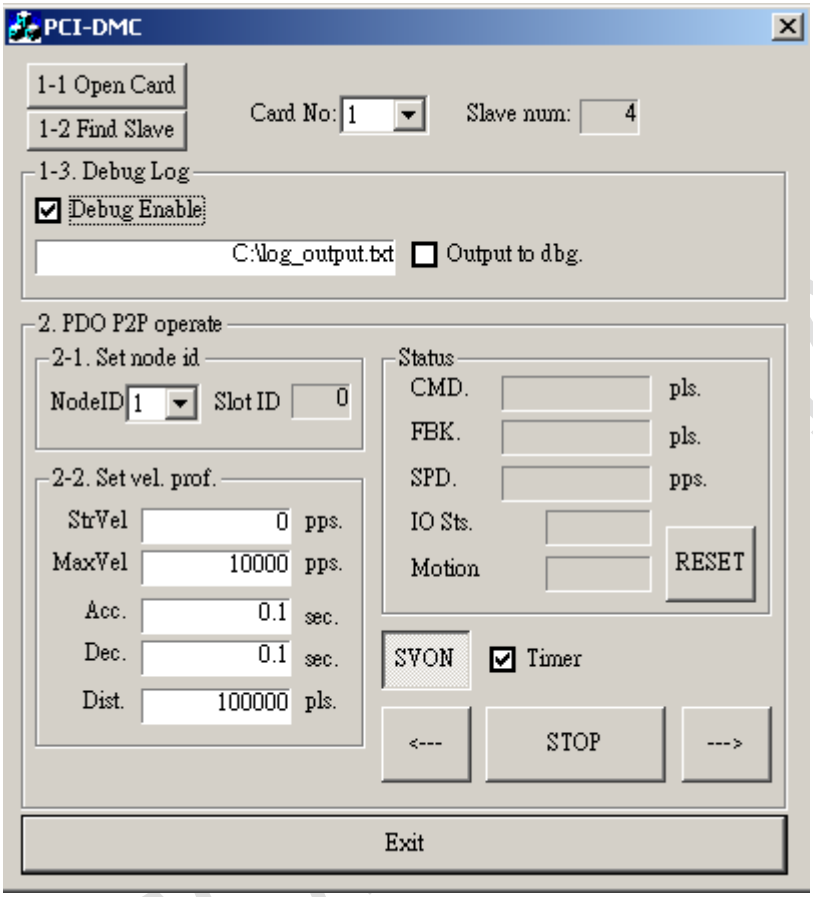


圖 3.991

1) 軸卡開啟與初始化

按壓圖 3. 991 中的「Open card」鍵開始執行軸卡初始化。

按壓圖 3. 991 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化軸卡的詳細說明，請參閱第 3.1.3 節「開啟軸卡」和「初始化軸卡」。

DELTA CONFIDENTIAL

2) Debug 的功能啟動與否

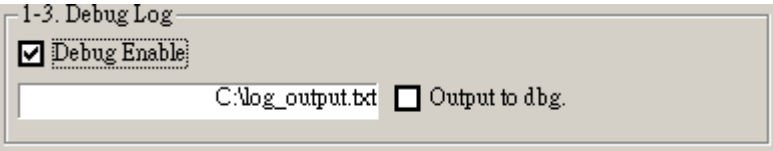


圖 3.992

欲啟動 debug 功能，必須勾選圖 3.992 中的 Debug Enable 選項並執行以下程序：

```
rt = _misc_set_record_debuging(enable);
// enable 引數值為 1 時，表示 debug 功能開啟
```

3) 填入記錄表存放的路徑，並選擇該記錄的輸出方式

當您欲觀察運動控制產生的歷程記錄，您必須輸入圖 3.992 中的檔案存放路徑及檔名並執行以下程序：

```
rt = _misc_open_record_debuging_file(gDMCCardNo ,file_name, open);
// “file_name”為一個用來存放 debug 產生的記錄輸出的檔案路徑的字元陣列變數。而另一個“open”變數則是用來決定該 debug 記錄是否輸出至您所設置檔案路徑的文件裡。即當”open”的值為 1 時，debug 記錄將會輸出存放至您設定的檔案路徑文件中；反之，若其值為 0，則該記錄僅會顯示在 visual studio 6 集成開發環境下的 debug 窗口。
```

DELTA CONFIDENTIAL

4) 開始操作任一在 PDO 模式下運行的運動控制

5) 歷程記錄的輸出

圖 3.993 表示運動控制所使用的函數與值變化的歷程顯示在 visual studio 6 集成開發環境的 debug 窗口。另外，圖 3.994 表示該函數與值變化的歷程記錄將輸出到"log_output.txt"的文件中。

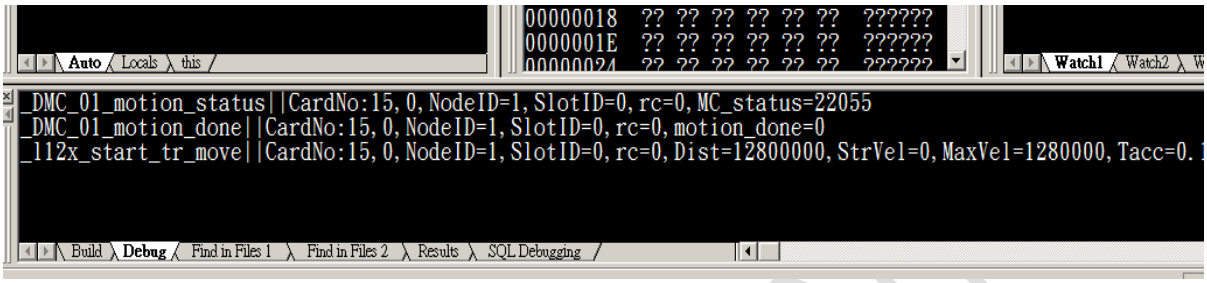


圖 3.993

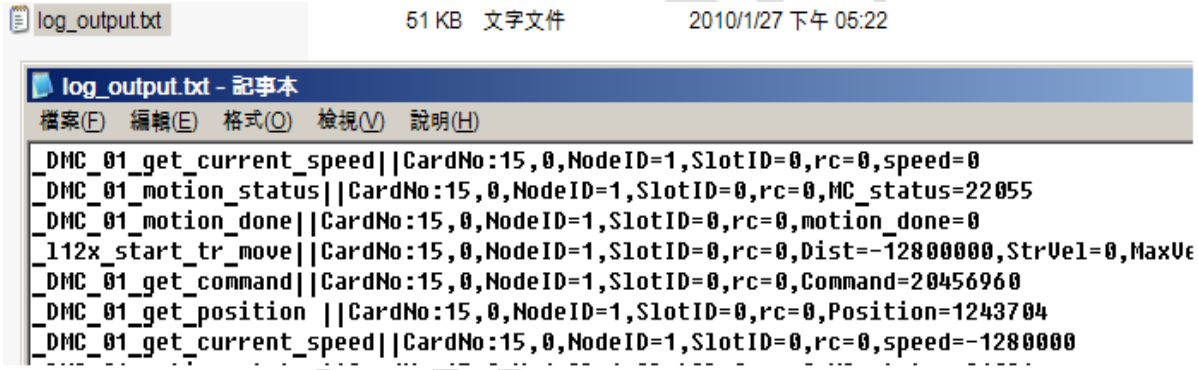


圖 3.994

6) 停止運動

按壓圖 3.991 中的「STOP」鍵開始執行一般停止：

```
rt = _DMC_F01_sd_stop(gDMCCardNo, NodeID, SlotID, 0, dec);
```

此例中使用一般停止的方式停止運動位移，此方式是設定減速時間設，於設定減速時間內停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

7) 離開程序

按壓圖 3.991 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

第四章 控制 API

4.1 資料型別與範圍

在安裝目錄“inc\VC\”資料夾中，存放一“TYPE_DEF.H”檔案，此檔案定義了一般的資料型別。

如下表 4.1，其標示該檔案中定義的型別、名稱、表示的意義、與值的範圍。

表 4.1

名稱	描述	值的範圍
U8	8-bit ASCII 字符	0 到 255
I16	16-bit 有號整數	-32768 到 32767
U16	16-bit 無號整數	0 到 65535
I32	32-bit 有號長整數	-2147483648 到 2147483647
U32	32-bit 無號長整數	0 到 4294967295
F32	32-bit 單精度浮點數	-3.402823E38 到 3.402823E38
F64	64-bit 雙精度浮點數	-1.797683134862315E308 到 1.797683134862315E309
Boolean	布林值	TRUE, FALSE

DELTA CONFIDENTIAL

4.2 函數描述

表 4.2

註	硬體初始化 API	
	_DMC_01_open	當應用程式啟動時，初始化系統資源
	_DMC_01_close	關閉 DMC 系統，釋放記憶體空間
	_DMC_01_get_CardNo_seq	取得主機上所有 PCI-DMC-軸卡卡號
	_DMC_01_pci_initial	初始此 PCI 軸卡
	_DMC_01_get_card_version	取得軸卡版本
	_DMC_01_get_fpga_version	取得軸卡 FPGA 版號
	_DMC_01_get_model_type	取得 MP1/MH1 裝置主機板號碼
	軸卡操作 API	
	_DMC_01_initial_bus	初始化外部匯流排 (單一軸卡)
	_DMC_01_start_ring	開啟 ring 的通訊
	_DMC_01_get_device_table	取得裝置列表
	_DMC_01_get_node_table	取得站號列表
	_DMC_01_reset_card	重置該軸卡
	_DMC_01_check_nodeno	查驗該站號是否已存在
	_DMC_01_get_master_connect_status	取得軸卡與擴充模組目前的連線狀態
	_DMC_01_get_mailbox_Error	取得 MailBox 傳送錯誤的次數
	_DMC_01_get_mailbox_cnt	取得 MailBox 計數值
	_DMC_01_get_dsp_cnt	取得 DSP 中斷訊號計數值
	_DMC_01_set_dio_output	設置軸卡上輸出腳位狀態
	_DMC_01_get_dio_output	取得軸卡上輸出腳位狀態
	_DMC_01_get_dio_input	取得軸卡上輸入腳位狀態
	_DMC_01_set_dio_output_DW	設置 GPIO 輸出腳位狀態(for A02/F02)
	_DMC_01_set_dio_single_output_DW	設置 GPIO 單一 bit 輸出腳位狀態(for A02/F02)
	_DMC_01_get_dio_output_DW	取得 GPIO 輸出腳位狀態(for A02/F02)
	_DMC_01_get_dio_single_output_DW	取得 GPIO 單一 bit 輸出腳位狀態(for A02/F02)
	_DMC_01_get_dio_input_DW	取得 GPIO 輸入腳位狀態(for A02/F02)
	_DMC_01_get_dio_single_input_DW	取得 GPIO 單一 bit 輸入腳位狀態(for A02/F02)
	_DMC_01_set_dio_output_DW_error_handle	設定 GPIO 保持輸出狀態
	_DMC_01_get_dio_output_DW_error_handle	取得 GPIO 保持輸出狀態
	_DMC_01_get_cycle_time	取得當前查找裝置的週期時間
	_DMC_01_initial_bus2	初始化外部所有匯流排 (多卡使用)
	_DMC_01_motion_cnt	取得 MailBox 及 DSP 計數值

PRELIMINARY

	_DMC_01_pdo_Rx_error	取得 PDO 收封包錯誤記錄
	_DMC_01_pdo_Tx_error	取得 PDO 送封包錯誤記錄
	_DMC_01_get_motion_error_code	取得運動控制命令錯誤碼
	_DMC_01_get_motion_error_cnt	取得運動控制命令錯誤次數
	_DMC_01_get_DLL_path	取得 PCI_DMC.dll 檔案詳細路徑資料
	_DMC_01_get_DLL_version	取得 PCI_DMC.dll 檔案版本。
	_DMC_01_get_DLL_path_Single	取得 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料
	_DMC_01_get_DLL_version_Single	取得 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 版本。
	_DMC_01_set_dio_single_output_DW_trigger_time	設定 DO 輸出狀態反向延遲功能 (for A02/F02)
	_DMC_01_set_dio_single_output_DW_one_shot	執行一次 DO 輸出狀態反向延遲功能 (for A02/F02)
	_DMC_01_get_dio_single_output_DW_trigger_status	取得 DO 輸出狀態反向延遲功能狀態 (for A02/F02)
	_DMC_01_set_dio_single_input_DW_trigger_mode	設定 A02/F02 單點 DI 觸發 DO 輸出功能 (for A02/F02)
	_DMC_01_get_dio_single_input_DW_trigger_mode	取得 A02/F02 單點 DI 觸發 DO 輸出功能狀態 (for A02/F02)
	_DMC_01_get_dio_single_input_DW_trigger_flag	取得觸發狀態
	_DMC_01_set_dio_output	設置 A01/B01 GPIO 輸出單一 bit 腳位狀態
	_DMC_01_get_dio_single_output	取得 A01/B01 GPIO 輸出單一 bit 腳位狀態
	_DMC_01_get_dio_single_input	取得 A01/B01 GPIO 輸入單一 bit 腳位狀態
	_DMC_01_set_dio_single_output_DW_trigger_cycle	設定 A02/F02 DO 輸出模式
	_DMC_01_set_dio_single_output_DW_trigger_enable	啟動 A02/F02 DO 輸出
	伺服驅動器參數讀取/寫入 API	
	_DMC_01_read_servo_parameter	讀取伺服驅動器參數資料
	_DMC_01_write_servo_parameter	寫入伺服驅動器參數資料
	使用 SDO 協定 API	
	_DMC_01_check_canopen_lock	SDO 模式下，檢查是否可以再執行下一筆命令
	_DMC_01_get_canopen_ret	取得 CANOpen 傳回的資訊(SDO 相關數據)
	_DMC_01_set_pdo_mode	設定 CANOpen 模式(PDO 或 SDO)

■	_DMC_01_send_message	發送 SDO 命令信息至資料緩衝區，會等待資料送達伺服端且讀回資料相符才結束
■	_DMC_01_send_message2	發送 SDO 命令信息至資料緩衝區，只確認資料送達伺服端就結束
■	_DMC_01_send_message3	發送 SDO 命令信息至資料緩衝區，但發出命令後立即結束
■	_DMC_01_send_message4	發送 SDO 命令信息至資料緩衝區，但發出命令後立即結束
	_DMC_01_read_message	讀取最後一筆 SDO 命令信息至資料緩衝區
	_DMC_01_read_message2	讀取最後一筆 SDO 命令信息至資料緩衝區，並傳回已完成的次數值
■	_DMC_01_get_message	取得 SDO 命令信息至資料緩衝區
■	_DMC_01_reset_sdo_choke	當 SDO 命令因故阻塞時，重置 SDO
	_DMC_01_get_sdo_retry_history	取得 SDO 重新發送的次數
	點對點運動控制使用 API(SDO 僅限台達 ASDA-A2-F 使用)	
▲	_DMC_01_set_sdo_driver_speed_profile	使用 SDO 送速度參數
▲	_DMC_01_start_sdo_driver_r_move	相對性運動命令
▲	_DMC_01_start_sdo_driver_a_move	絕對性運動命令
▲	_DMC_01_start_sdo_driver_new_position_move	改變運動位置命令
	原點復歸運動控制使用 API(SDO 僅限台達 ASDA-A2-F 使用)	
	_DMC_01_set_home_config	設置原點復歸各項配置
	_DMC_01_set_home_move	開始運行原點復歸運動
	_DMC_01_set_home_config2	設置原點復歸各項配置
	_DMC_01_set_home_config2_offset2	重設原點復歸 offset 值
	_DMC_01_set_home_move2	開始運行原點復歸運動
■	_DMC_01_escape_home_move	停止原點復歸運動
	速度模式控制使用 API (SDO 僅限台達 ASDA-A2-F 使用)	
■	_DMC_01_set_velocity_mode	速度運動控制參數配置
■	_DMC_01_set_velocity	設置開始運行速度運動
■	_DMC_01_set_velocity_stop	停止速度運動控制
■	_DMC_01_set_velocity_torque_limit	在速度模式時設定最大扭矩限制
	扭矩模式控制使用 API (SDO 僅限台達 ASDA-A2-F 使用)	
■	_DMC_01_set_torque_mode	扭矩控制參數配置
■	_DMC_01_set_torque	設置開始運行轉矩運動
■	_DMC_01_set_torque_stop	停止扭矩模式
■	_DMC_01_set_torque_velocity_limit	在扭矩模式時設定最大速度限制
	使用 PDO 協定 API	
	_DMC_01_ipo_set_svon	設置 Servo ON/OFF

	_DMC_01_get_buffer_length	取得未實行的運動命令行數
	_DMC_01_command_buf_clear	清除間隔時間(buffer dwell)
▲	_DMC_01_buf_dwell	設定兩個運動命令之間的間隔時間
▲	_DMC_01_set_group	設定關連性群組
	_DMC_01_set_dmc_mode	切換成 DMC_NET 模式
	停止運動控制 API	
■	_DMC_01_emg_stop	緊急停止(立即停止執行 Buffer 內所有運動控制指令)
■	_DMC_01_sd_stop	減速停止(立即停止執行 Buffer 內所有運動指令，馬達減速停止)
■	_DMC_01_sd_abort	中斷當前正在執行的運動指令，減速時間停止後，繼續執行 buffer 內運動控制指令
	_DMC_01_set_sld_mode	設定 SLD 運動模式(SLD 為外部訊號觸發的減速停止模式)。
	_DMC_01_set_sld_stop	啟動/關閉 SLD 模式設定
	_DMC_01_set_sd_time	設定減速時間。
	運動狀態 API	
	_DMC_01_motion_done	傳回軸卡目前實行的命令執行狀態
	_DMC_01_motion_status	傳回從站目前運動狀態
	_DMC_01_set_01_ph_inposition	設定 01PH 到位訊號的區間範圍
	_DMC_01_get_01_ph_inposition	取得目前 01PH 到位訊號設定的區間範圍
	運動控制數值 API	
	_DMC_01_get_command	取得軸卡 command 數值
	_DMC_01_set_command	設置軸卡新的 command 數值
	_DMC_01_get_position	取得從站模組目標位置數值
■	_DMC_01_set_position	設置從站模組目標位置數值
	_DMC_01_get_target_pos	取得軸卡目的位置的設定值
	_DMC_01_get_torque	取得馬達目前的轉矩數值(單位:電流百分比)
	_DMC_01_get_current_speed	取得軸卡速度命令值(單軸分量)
	_DMC_01_get_current_speed_rpm	取得馬達目前轉速 單位:0.1RPM)
	軟體極限 API	
	_DMC_01_set_soft_limit	設置軟體正/負極限的數值
	_DMC_01_enable_soft_limit	設置開啟軟體極限和碰觸極限後的停止方式
	_DMC_01_disable_soft_limit	關閉軟體極限
	_DMC_01_get_soft_limit_status	取得運動期間軟體正/負極限的狀態
	單軸運動控制 API	
	_DMC_01_start_tr_move	相對運動，使用 T curve
	_DMC_01_start_sr_move	相對運動，使用 S curve

	_DMC_01_start_ta_move	絕對運動，使用 T curve
	_DMC_01_start_sa_move	絕對運動，使用 S curve
	_DMC_01_p_change	改變目標位置
	_DMC_01_v_change	改變運動速度
	_DMC_01_start_tr_move_2seg	兩段相對運動，使用 T curve 達成速度連續
	_DMC_01_start_sr_move_2seg	兩段相對運動，使用 S curve 達成速度連續
	_DMC_01_start_ta_move_2seg	兩段絕對運動，使用 T curve 達成速度連續
	_DMC_01_start_sa_move_2seg	兩段絕對運動，使用 S curve 達成速度連續
	_DMC_01_start_tr_move_2seg2	兩段相對運動，使用 T curve 達成速度連續
	_DMC_01_start_sr_move_2seg2	兩段相對運動，使用 S curve 達成速度連續
	_DMC_01_start_ta_move_2seg2	兩段絕對運動，使用 T curve 達成速度連續
	_DMC_01_start_sa_move_2seg2	兩段絕對運動，使用 S curve 達成速度連續
	_DMC_01_feedrate_overwrite	改變運動速度或速度比率
	_DMC_01_start_v3_move	設定運動末速的單軸位移運動
	_DMC_01_feedrate_overwrite_read	取得改變運動向量速度或向量速度比率設定值
	二軸線性補間運動控制 API	
	_DMC_01_start_tr_move_xy	相對座標進行兩軸線性補間運動(T curve)
	_DMC_01_start_sr_move_xy	相對座標進行兩軸線性補間運動(S curve)
	_DMC_01_start_ta_move_xy	參絕對座標進行兩軸線性補間運動(T curve)
	_DMC_01_start_sa_move_xy	絕對座標進行兩軸線性補間運動(S curve)
	_DMC_01_start_v3_move_xy	設定運動末速的兩軸線性補間運動
	二軸圓弧補間運動控制 API	
	_DMC_01_start_tr_arc_xy	相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度)
	_DMC_01_start_sr_arc_xy	相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度)
	_DMC_01_start_ta_arc_xy	絕對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度)
	_DMC_01_start_sa_arc_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度)
	_DMC_01_start_tr_arc2_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_sr_arc2_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_ta_arc2_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_sa_arc2_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_tr_arc3_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、終點座標)
	_DMC_01_start_sr_arc3_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、終點座標)
	_DMC_01_start_ta_arc3_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：圓心座標、終點座標)

	_DMC_01_start_sa_arc3_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_spiral_xy	兩軸螺旋運動(已知條件：X、Y 軸圓心座標)
	_DMC_01_start_spiral2_xy	兩軸螺旋運動(已知條件：X、Y 軸圓心座標，X、Y 軸終點座標)
	_DMC_01_start_v3_arc_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：圓心座標、角度)
	_DMC_01_start_v3_arc2_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_v3_arc3_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_v3_spiral_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標)
	_DMC_01_start_v3_spiral2_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
	_DMC_01_start_multiarc	兩軸圓弧補間運動，支援多軸、arc、arc2、arc3，三種已知條件模式
	_DMC_01_Start_Move_Spiral_Circle	兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標)。
	_DMC_01_Start_Move_Spiral2_Circle	兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。
	_DMC_01_Start_Arc_follow	可指定任意兩軸作為圓弧運動的X和Y軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動。
	三軸線性補間運動控制 API	
	_DMC_01_start_tr_move_xyz	參照 T-curve 速度剖面相對座標進行三軸線性補間運動
	_DMC_01_start_sr_move_xyz	參照 S-curve 速度剖面相對座標進行三軸線性補間運動
	_DMC_01_start_ta_move_xyz	參照 T-curve 速度剖面絕對座標進行三軸線性補間運動
	_DMC_01_start_sa_move_xyz	參照 S-curve 速度剖面絕對座標進行三軸線性補間運動
	_DMC_01_start_v3_move_xyz	增加 EndVel 的三軸線性補間運動
	三軸螺旋補間運動控制 API	
	_DMC_01_start_tr_heli_xy	參照 T-curve 速度剖面相對座標進行三軸螺旋補間運動
	_DMC_01_start_sr_heli_xy	參照 S-curve 速度剖面相對座標進行三軸螺旋補間運動
	_DMC_01_start_ta_heli_xy	參照 T-curve 速度剖面絕對座標進行三軸螺旋補間運動
	_DMC_01_start_sa_heli_xy	參照 S-curve 速度剖面絕對座標進行三軸螺旋補間運動
	_DMC_01_start_v3_heli_xy	增加 EndVel 的三軸螺旋補間運動
	_DMC_01_start_v3_sphere_xyz	三點成圓
	速度運動控制 API	
	_DMC_01_tv_move	使用 T-curve 的速度運動控制

	_DMC_01_sv_move	參照 S-curve 速度剖面下的速度運動控制
	同步運動控制 API	
	_DMC_01_sync_move	設置開始運行同步運動
	_DMC_01_sync_move_config	設置同步運動功能是否啟動
	遠端擴充模組控制 API	
	_DMC_01_get_rm_input_value	取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
	_DMC_01_set_rm_input_filter	設定遠端擴充模組輸入埠的軟體濾波等級
	_DMC_01_set_rm_input_filter_enable	設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否
	_DMC_01_set_rm_output_value	設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值
	_DMC_01_set_rm_output_value_error_handle	設定遠端擴充模組發生錯誤時保持輸出狀態
	_DMC_01_get_rm_output_value	取得遠端擴充模組的輸出值
	_DMC_01_get_rm_output_value_error_handle	取得遠端擴充模組在發生錯誤時，系統是否將該值保留或重置的設定值
	_DMC_01_set_rm_output_active	設定遠端擴充模組輸出
	_DMC_01_get_rm_input_single_value	取得遠端擴充模組輸入埠單一 Bit 的值。
	_DMC_01_set_rm_output_single_value	設定遠端擴充模組輸出埠單一 Bit 的值。
	_DMC_01_get_rm_output_single_value	充模組的輸出埠單一 Bit 的值。
	電子手輪與 JOG 操作 API	
▲ ■	_DMC_01_set_rm_mpg_axes_enable	設定手輪運動控制
■	_DMC_01_set_rm_mpg_axes_enable2	手輪運動控制（可設定馬達圈數比例分母值）
■	_DMC_01_set_rm_mpg_axes_enable3	設定手輪運動控制(可設定馬達圈數比例之分母值）。
■	_DMC_01_set_rm_jog_axes_enable	設定 JOG 運動控制
	_DMC_A02_set_mpg_axes_enable	設定手輪運動控制。
	_DMC_A02_mpg_get_io_state_enable	取得手輪的開和關狀態。
	_DMC_A02_mpg_fsm_get_lock_state	取得手輪的計數器是否為啟動狀態。
	_DMC_A02_mpg_get_poisition	取得目前手輪位置計數值。
	_DMC_A02_mpg_axis_get_io_state	取得目前所要控制的軸號。
	_DMC_A02_mpg_axis_get_ratio_state	取得手輪目前選擇的倍率。
	_DMC_A02_set_jog_axes_enable	設定手輪 JOG 硬體的控制模式。
	_DMC_A02_get_jog_io_state	讀取 JOG 輸出入狀態資料。
	_DMC_A02_set_mpg_axes_enable2	設定手輪運動控制(軟體模式)
	_DMC_A02_get_mpg_axes_enable2	取得手輪運動控制(軟體模式)設定狀態
	_DMC_A02_set_mpg_axes_extend	設定手輪運動控制(軟體模式)選擇 4 軸或 10 軸
	_DMC_A02_get_mpg_axes_extend	取得手輪運動控制(軟體模式)選擇 4 軸或 10 軸
	_DMC_A02_mpg_axis_get_io_state2	取得目前所控制的軸號
	_DMC_A02_mpg_axis_get_ratio_state2	取得手輪目前選擇的倍率

	_DMC_A02_set_mpg_axes_enable3	設定手輪運動控制(軟體模式)
	_DMC_01_Set_MPG_AMF	設定手輪濾波
	_DMC_Qep_set_mpg_axis_enable	設定手輪運動控制(軟體模式 MP1-A12D/PCI-DMC-B01 適用)
	_DMC_Qep_get_mpg_axis_enable	取得手輪運動控制設定(軟體模式 MP1-A12D/PCI-DMC-B01 適用)
	_DMC_Qep_get_mpg_axis_cnt	取得手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)
	_DMC_Qep_set_mpg_clear	清除手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)
	_DMC_MP1_set_mpg_axis_enable	設定手輪運動控制(MP1-P10D-15 專用)
	_DMC_MP1_get_mpg_axis_cnt	取得手輪計數值 (MP1-P10D-15 專用)
	_DMC_MP1_set_mpg_axis_clear	清除手輪計數值 (MP1-P10D-15 專用)
	_DMC_MP1_get_mpg_axis_enable	取得手輪運動控制設定(MP1-P10D-15 專用)
	RM04PI 專用 API	
	_DMC_01_set_rm_04pi_ipulser_mode	設定 RM04PI 脈波輸入模式
	_DMC_01_set_rm_04pi_opulser_mode	設定 RM04PI 脈波輸出模式
	_DMC_01_set_rm_04pi_svon_polarity	設定 POWER ON（SVON）的準位
	_DMC_01_set_rm_04pi_DO2	設定 DO2 埠啟動
	_DMC_01_set_rm_04pi_homing_ratio	設置原點復歸速度增量比
	_DMC_01_rm_04PI_get_buffer	取得仍未執行的運動命令
	RM04PI（模式 1）運動控制 API	
	_DMC_01_rm_04pi_md1_start_move	RM04PI 在模式 1 下，進行單軸運動位移
	_DMC_01_rm_04pi_md1_v_move	RM04PI 在模式 1 下，進行速度運動控制
	_DMC_01_rm_04pi_md1_start_line2	RM04PI 在模式 1 下，進行兩軸線性補間運動
	_DMC_01_rm_04pi_md1_start_line3	RM04PI 在模式 1 下，進行三軸線性補間運動
	_DMC_01_rm_04pi_md1_start_line4	RM04PI 在模式 1 下，進行四軸線性補間運動
	_DMC_01_rm_04pi_md1_start_arc	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、角度)
	_DMC_01_rm_04pi_md1_start_arc2	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：終點座標、角度)
	_DMC_01_rm_04pi_md1_start_arc3	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、終點座標)
	_DMC_01_rm_04pi_md1_start_heli	RM04PI 在模式 1 下，進行三軸螺旋補間運動
	_DMC_01_rm_04pi_md1_p_change	RM04PI 在模式 1 下，運動中改變目標位置
	_DMC_01_rm_04pi_md1_v_change	RM04PI 在模式 1 下，運動中改變速度
	_DMC_01_rm_04pi_md1_set_gear	RM04PI 在模式 1 下，設定 Gear 功能啟動與其參數設置
	_DMC_01_rm_04pi_md1_set_soft_limit	RM04PI 在模式 1 下，設定是否啟動軟體極限

	_DMC_01_rm_04pi_md1_get_soft_limit_status	RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態
	_DMC_01_rm_04pi_md1_set_sld	設定 SLD 埠（DI3）開啟與其相關配置
	_DMC_01_rm_04pi_md1_get_mc_error_code	當 error code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息
■	_DMC_01_set_rm_04pi_ref_counter	RM04PI 在模式 1 下，選擇重新連線參考的位置
	RM04DA(類比輸出)遠端擴充模組 API	
	_DMC_01_rm_04da_set_output_value	設定類比輸出輸出數值
	_DMC_01_rm_04da_get_output_value	讀取類比輸出輸出數值
	_DMC_01_rm_04da_get_return_code	讀取類比輸出狀態值
■	_DMC_01_rm_04da_set_output_range	設定類比輸出輸出範圍
	_DMC_01_rm_04da_set_output_enable	設定腳位輸出開啟/關閉
■	_DMC_01_rm_04da_set_output_overrange	設定輸出範圍增加 10%
	_DMC_01_rm_04da_set_output_error_clear	清除錯誤狀態
■	_DMC_01_rm_04da_set_output_error_handle	斷線保持原本設定狀態
■	_DMC_01_rm_04da_set_output_offset_value	設定類比輸出 offset 數值
	_DMC_01_rm_04da_get_output_offset_value	讀取類比輸出 offset 數值
	RM04AD(四組類比輸入遠端擴充模組 API	
■	_DMC_01_set_04ad_input_range	設定類比輸入的 Input 範圍
	_DMC_01_get_04ad_input_range	取得目前 AD 的 Input 設定範圍
■	_DMC_01_set_04ad_conversion_time	設定 AD 轉換時間
	_DMC_01_get_04ad_conversion_time	取得目前 AD 所設定的轉換時間
	_DMC_01_get_04ad_data	讀取輸入之電壓數值
■	_DMC_01_set_04ad_average_mode	設定類比輸入平均功能之模式
	_DMC_01_get_04ad_average_mode	讀取類比輸入平均功能之模式
■	_DMC_01_set_04ad_input_enable	設定 AD Channel 是否致能
	Slave 資訊	
	_DMC_01_get_devicetype	獲取 slave 裝置類型
	_DMC_01_get_slave_version	獲取 slave 裝置韌體版本
	監控參數 API	
	_DMC_01_set_monitor	設定欲監控的參數
	_DMC_01_get_monitor	取得監控參數的數值
	_DMC_01_get_monitor_I32	取得監控參數的數值
	_DMC_01_get_servo_command	取得伺服驅動器發送的命令值
	_DMC_01_get_servo_DI	取得伺服驅動器 DI 訊號的值
	_DMC_01_get_servo_DO	取得伺服驅動器 DO 訊號的值
	警報訊息 API	

	_DMC_01_set_ralm	輸出的伺服驅動器警示訊息重置
	_DMC_01_get_alm_code	取得 Slave 的錯誤資訊
	_DMC_01_master_alm_code	取得 Master Card 連線的錯誤資訊
	_DMC_01_slave_error	取得 slave 通訊中連續錯誤次數
	多軸運動控制 API	
	_DMC_01_multi_axes_move	設置 2 軸以上的運動控制
	_DMC_01_liner_speed_master	多軸直線運動運動速度設定
	_DMC_01_start_v3_multi_axes	增加 EndVel 的多軸(2 軸以上)的運動控制
●	_DMC_01_p_change_multi_axes	改變目標位置(能對多軸運動指令下達)。
▲	_DMC_01_start_v3_msbrline_move	指定 EndVel 的多軸(2 軸以上)運動控制。
▲	_DMC_01_Start_Multi_Arc	多軸圓弧直線相對/絕對運動，可指定任意兩軸作為圓弧運動的 X 和 Y 軸，其餘軸做多軸直線運動。可選擇隨速(跟隨圓弧)或合速模式。
	緩衝區操作 API	
	_DMC_01_set_trigger_buf_function	使用伺服驅動器 DI3（SLD）觸發 Motion 指令
	Interrupt API	
▲	_DMC_01_set_int_factor	設定中斷模式，共有 8 種模式。
▲	_DMC_01_int_enable	中斷設定致能
▲	_DMC_01_int_disable	中斷設定解除
▲	_DMC_01_get_int_count	中斷累計次數
▲	_DMC_01_get_int_status	查詢目前設定中斷的模式
▲	_DMC_01_Link_interrupt	連結一處理程序，當設定之中斷發生就導入處理。
	Security API	
▲	_DMC_01_read_security	讀取記憶體中的安全驗證資料
▲	_DMC_01_read_security_status	獲取目前的記憶體是處於讀/寫的狀態
▲	_DMC_01_write_security	將指定的安全資料寫入指定的記憶體區塊
▲	_DMC_01_write_security_status	寫入安全資料前，將記憶體寫入功能致能
	_DMC_01_check_userpassword	確認使用者對記憶體讀/寫的權限
	_DMC_01_write_userpassword	變更密碼
	_DMC_01_check_verifykey	驗證碼驗證
	_DMC_01_write_verifykey	寫入驗證碼
	_DMC_01_read_serialno	讀取軸卡產品序號
	_misc_slave_check_userpasswaord	Slave(04PI)確認使用者對記憶體讀/寫的權限
	_misc_slave_write_userpasswaord	Slave(04PI)變更密碼
	_misc_slave_get_serialno	Slave(04PI)讀取 04PI 產品序號
	_misc_security	產生 04PI 驗證碼
	_misc_slave_write_verifykey	Slave(04PI)寫入驗證碼

	_misc_slave_check_verifykey	Slave(04PI)驗證碼驗證
	_misc_slave_user_data_buffer_read	Slave(04PI)讀取記憶體中的資料
	_misc_slave_user_data_buffer_write	Slave(04PI)將指定的資料寫入記憶體
	_misc_slave_user_data_to_flash	Slave(04PI)將資料由 Bufeer 寫入快閃記憶體中
	極限反向 API	
	_DMC_01_rm_04pi_set_MEL_polarity	設定 RM04PI 負極限反向
	_DMC_01_rm_04pi_get_MEL_polarity	取得 RM04PI 負極限目前的狀態
	_DMC_01_rm_04pi_set_PEL_polarity	設定 RM04PI 正極限反向
	_DMC_01_rm_04pi_get_PEL_polarity	取得 RM04PI 正極限目前的狀態
	_DMC_01_set_01ph_alm_polarity	設定 GE01PH ALM 準位反向
	_DMC_01_set_01ph_org_polarity	設定 RM04PI/GE01PH ORG 準位反向
	Compare API	
●	_DMC_01_set_compare_channel_position	設定該 Channel 新的 Position 數值
●	_DMC_01_get_compare_channel_position	讀取該 Channel 目前的 Position 數值
●	_DMC_01_set_compare_ipulser_mode	設定脈波輸入模式
●	_DMC_01_set_compare_channel_direction	設定該 Channel 脈波方向
●	_DMC_01_set_compare_channel_trigger_time	設定觸發訊號時間
●	_DMC_01_set_compare_channel_one_ shot	設定為執行一次觸發
●	_DMC_01_set_compare_channel_source	設定訊號來源
●	_DMC_01_channel0_position_cmp	執行固定區間的比對模式
●	_DMC_01_channel1_output_enable	不等區間的比對模式開啟/關閉
●	_DMC_01_channel1_output_mode	設定不等區間的比對模式輸出模式
●	_DMC_01_channel1_get_io_status	讀取不等區間的比對模式狀態
●	_DMC_01_channel1_set_gpio_out	設置 DMC-B01 軸卡輸出腳位狀態
●	_DMC_01_channel1_position_compare_table	設定不等距區間的位置比對資料
●	_DMC_01_channel1_position_compare_ table_level	自定義輸出準位表格(Level 模式)
●	_DMC_01_channel1_position_compare_ table_cnt	讀取已觸發次數
●	_DMC_01_set_compare_channel_polarity	設定觸發準位
●	_DMC_01_channel0_position_cmp_by_ gpio	設定由外部 DI 啟動等距比較功能
●	_DMC_01_channel1_position_re_compare_ table	重新執行一次 Channel1 的 Compare
●	_DMC_01_channel1_position_re_compare_	重新執行一次 Channel1(Level 模式)的

	table_level	Compare
●	_DMC_01_set_compare_channel_enable	設定 Compare 功能開啟/關閉
◇	_DMC_B02_qep_set_mode	設定 QEP Index(編碼器來源索引)的脈波相位模式
◇	_DMC_B02_qep_set_inverse	設定 QEP Index(編碼器來源索引)脈波是否反向。
◇	_DMC_B02_qep_get_reload_counter	讀取 QEP Index(編碼器來源索引)所設定的 Counter
◇	_DMC_B02_qep_set_reload_counter	設定 QEP Index(編碼器來源索引)的 Counter
◇	_DMC_B02_qep_get_counter	讀取 QEP Index(編碼器來源索引)實際的 Counter
◇	_DMC_B02_gpio_software_control_status	讀取 CMP1~10 是否被設成 GPIO 輸出
◇	_DMC_B02_gpio_software_control_setting	設置 CMP1~10 為 GPIO 輸出
◇	_DMC_B02_ipcmp_initial	設置 ipcmp(interval compare)的初始設定
◇	_DMC_B02_ipcmp_oneshot_time	設置 ipcmp(interval compare)執行一次 trigger 致能持續時間(for oneshot and Interval use) 一旦完成上述設定即為 ipcomp_software_trigger 與 ipcmp_compare_position 都是依據此 nPulseWidth 執行
◇	_DMC_B02_ipcmp_software_trigger	執行 ipcmp(interval compare)一次 Trigger
◇	_DMC_B02_ipcmp_compare_position	執行 ipcmp(interval compare)
◇	_DMC_B02_ipcmp_compare_trigger_count	讀取 ipcmp(interval compare)已 Trigger 的次數
◇	_DMC_B02_ipcmp_compare_trigger_count_clear	清除 ipcmp(interval compare)已 Trigger 的次數
◇	_DMC_B02_tpcmp_initial	設置 tpcmp(table compare)的初始設定
◇	_DMC_B02_tpcmp_oneshot_time	設置 tpcmp(table compare)執行一次 trigger 致能持續時間
◇	_DMC_B02_tpcmp_software_trigger	執行 tpcmp(table compare)一次 Trigger
◇	_DMC_B02_tpcmp_software_trigger_level	執行 tpcmp(table compare)設定 Trigger Level
◇	_DMC_B02_tpcmp_compare_position_table_reset	清除 compareTable 內的所有資料
◇	_DMC_B02_tpcmp_compare_position_table	執行 tpcmp(table compare)
◇	_DMC_B02_tpcmp_compare_position_table_level	執行 tpcmp(table level compare)
◇	_DMC_B02_tpcmp_compare_trigger_count	讀取 tpcmp(table compare)已 Trigger 的次數
◇	_DMC_B02_tpcmp_compare_trigger_count	清除 tpcmp(table compare)已 Trigger 的次數

	_clear	
	_DMC_B02_ext_io_get_input_value	取得 EXT GPIO 輸入腳位元狀態
	_DMC_B02_ext_io_get_output_value	取得 EXT GPIO 輸出腳位狀態
	_DMC_B02_ext_io_set_output_value	設置 EXT GPIO 輸出腳位狀態
	_DMC_B02_tpcmp_initial_2D	設置 2D tpcmp(table compare)的初始設定
	_DMC_B02_tpcmp_oneshot_time_2D	設置 2D tpcmp(table compare)執行一次 trigger 致能持續時間
	_DMC_B02_tpcmp_software_trigger_2D	執行 2D tpcmp(table compare)一次 Trigger
	_DMC_B02_tpcmp_software_trigger_level_2D	執行 2D tpcmp(table compare)設定 Trigger Level
	_DMC_B02_tpcmp_compare_position_table_2D	執行 2D tpcmp(table compare)，填入自訂 Table 表相關資料
	_DMC_B02_tpcmp_compare_position_table_reset_2D	清除 2D compare table 內的所有資料
	_DMC_B02_tpcmp_compare_position_table_level_2D	2D tpcmp(table compare)設定 Trigger Level
	_DMC_B02_tpcmp_compare_axis_dist_2D	設置 2D tpcmp(table level compare)，X Y 軸誤差範圍
	_DMC_B02_get_tpcmp_compare_axis_dist_2D	讀取設置的 2D tpcmp(table level compare)
	TableMotion 循環模式運動資料 API	
▲	_DMC_01_start_tablemotion	設置 TableMotion 循環模式的運動資料 (各點間最大速度一致)
▲	_DMC_01_start_tablemotion2	設置 TableMotion 循環模式的運動資料 (可設定各點間最大速度，支援單步執行功能)
▲	_DMC_01_start_tablemotion3	設置 TableMotion 循環模式的運動資料 (可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
▲	_DMC_01_start_tablemotion_cyclemode	啟動 TableMotion 的循環模式
▲	_DMC_01_start_tablemotion_buffer	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (各點間最大速度一致)
▲	_DMC_01_start_tablemotion_buffer2	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (可設定各點間最大速度，支援單步執行功能)
▲	_DMC_01_start_tablemotion_buffer3	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (可設定各點間最大速度，支援單步執行、絕對

		(相對)模式選擇、三點成圓路徑等功能)
▲	_DMC_01_get_tablemotion_length	取得 Buffer 內可用的資料數量。
▲	_DMC_01_tablemotion_feedrate_overwrite	設定 Table Motion federate 百分比
▲	_DMC_01_tablemotion_feedhold	設定 FeedHold 是否開啟。
▲	_DMC_01_set_tablemotion_blending_ratio	設定 Table Motion 運動間混合百分比。
▲	_DMC_01_start_tablemotion_profile_planning_para	設定速度規劃模式的相關參數。
▲	_DMC_01_start_tablemotion_profile_planning	根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。 (可設定各點間最大速度，支援單步執行功能)
▲	_DMC_01_start_tablemotion_profile_planning2	根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。 (可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
▲	_DMC_01_start_tablemotion_profilemode	啟動 TableMotion 速度規劃模式。
▲	_DMC_01_set_axis_normalamf_enable	設定指定軸的後加減 FilterTime 與是否開啟
▲	_DMC_01_set_normalamf_enable	啟用後加減功能
▲	_DMC_01_get_tablemotion_current_speed	設定取得指定 Table 當前速度
▲	_DMC_01_Execute_Single_Step	觸發啟動單步運轉功能
▲	_DMC_01_Get_TableMotion_Now_MotionNo	取得當群運動的編號
▲	_DMC_01_TableMotion_Single_Step	設定啟動單步運轉功能
▲	_DMC_01_start_tableMotion_profileMode_data_flying_photography	兩軸過點飛拍功能，此 API 會根據設定的飛拍參數計算出新的運動軌跡及速度。
	PVT 模式 API	
▲	_DMC_01_Start_PVTComplete	啟動 PVT Complete 運動
▲	_DMC_01_Start_PVT	啟動 PVT 運動
▲	_DMC_01_get_PVT_cnt	取得 PVT 點位個數
	連續速度 API	
▲	_DMC_01_speed_continue	開啟連續速度功能
▲	_DMC_01_speed_continue_mode	設定連續速度模式
▲	_DMC_01_speed_continue_combine_ratio	設定連續速度疊合比率
	其他運動控制 API	
▲	_DMC_01_enable_electcam	開啟/關閉跟隨功能
▲	_DMC_01_set_cam_enable	開啟/關閉間隙補償功能
▲	_DMC_01_set_cam_pitch	設定間隙補償間隔(單位:Pulse)
▲	_DMC_01_set_cam_org	設定間隙補償開始原點位置

▲	_DMC_01_set_cam_mode	設定間隙補償模式(單向/雙向)
▲	_DMC_01_set_cam_table	建立間隙補償參數表
▲	_DMC_01_set_cam_table2	建立間隙補償參數表資料並設定補償數量
▲	_DMC_01_set_backlash	開啟/關閉背隙補償功能
▲	_DMC_01_set_backlash_info	設定背隙補償參數
▲	_DMC_01_set_ecam_repeat	開啟/關閉電子凸輪重複功能
	其它 API	
	_misc_app_get_circle_endpoint	獲取圓弧補間所需之終點座標(X, Y)
	_misc_app_get_circle_center_point	獲取圓弧補間所需之中心座標(X, Y)
	_misc_set_record_debuging	Debug 記錄功能是否啟動
	_misc_open_record_debuging_file	設置輸出 Debug 記錄存放位置
	_DMC_01_enable_dda_mode	啟動 DDA 填表功能
	_DMC_01_set_dda_data	填入 DDA Table 資料
	_DMC_01_get_dda_cnt	取得目前 DDA Table 尚存的資料筆數
	_DMC_01_clear_dda_data	將 DDA Table 中尚存的資料清除。
	_DMC_01_save_device_info	將模組資料存為文字檔。
	_DMC_01_compare_device_info	將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。
	_DMC_01_enable_record_mode	開啟/關閉擷取 DMCNET 所下命令的功能。
	_DMC_01_get_record_data	讀取以毫秒為單位，Monitor 所下參數的 12 軸資料
	_DMC_01_get_record_cnt	讀取以毫秒為單位擷取 Monitor 所下參數資料筆數
▲	_DMC_01_set_mc_fifo_mode	FIFO 參數設定。
▲	_DMC_01_set_function_time	設定每 1ms 最小的通訊時間以確保通訊不會異常。(詳細請參照 API 內說明)
	_DMC_01_set_position_ref_source	設定座標的參考來源
	_DMC_01_get_position_ref_source	取得目前座標的參考來源
	_DMC_01_set_virtual_command	Set position 的快速模式
	_DMC_01_get_virtual_command	取得實際 command、position 與 virtual_command 的差值
	_DMC_01_set_virtual_position	將實際 Position 值設定為 Virtual Position 的值
	_DMC_01_disable_virtual_pos	設定 Command 與 Position 的連動性
	_DMC_01_Set_Stepping_Move_rate	設定脈波輸出解析放大的比率
◆	_DMC_01_MRAM_read_word	讀取 MH1(MP1) MRAM 的值(單

		位:word)
◆	_DMC_01_MRAM_write_word	寫值到 MH1(MP1)MRAM(單位:word)
◆	_DMC_01_MRAM_read_dword	讀取 MH1(MP1)MRAM 的值 (單位:double word)
◆	_DMC_01_MRAM_write_dword	寫值到 MH1(MP1)MRAM (單位:double word)
	_DMC_01_get_watchdog_cnt	讀取 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
	_DMC_01_set_watchdog_cnt	修改 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
	_DMC_01_get_watchdog_value	讀取 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
	_DMC_01_set_watchdog_value	修改 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
	_DMC_01_set_rm_04pi_md2_feedback_mode	設定脈波模組 Feedback 的連接狀態
	_DMC_01_Security_Check_UserPassword	Master 在讀/寫資料前，確認使用者對 Memory 讀/寫的權限登入
	_DMC_01_Security_Get_Check_UserPassword_State	讀取目前 Msater 檢查權限登入的狀態
	_DMC_01_Security_Write_UserPassword	變更權限登入的密碼
	_DMC_01_Security_Get_Write_UserPassword_State	檢查變更權限登入密碼的狀態
	_DMC_01_Security_Check_Verifykey	Master 將驗證碼寫入驗證是否相符
	_DMC_01_Security_Get_Check_Verifykey_State	讀取目前 Msater 檢查驗證的狀態
	_DMC_01_Security_Write_Verifykey	變更驗證的密碼
	_DMC_01_Security_Get_Write_Verifykey_State	檢查變更驗證密碼的狀態
	_DMC_01_buffer_correlation_master_slave	設定總線內伺服軸的 MotionBuffer 關連群組
	_DMC_01_buffer_correlation_status	讀取總線內伺服軸的 MotionBuffer 關連群組設定狀態
	_DMC_01_GPIO_Set_Type	設定 RS-422/485 串列埠
	_DMC_01_Get_GPIO_Input_Value	取得 MP1 GPIO 輸入腳位狀態
	_DMC_01_Get_GPIO_Output_Value	取得 MP1 GPIO 輸出腳位狀態
	_DMC_01_Set_GPIO_Output_Value	設定 MP1 GPIO 輸出腳位狀態
	_DMC_01_Set_GPIO_Output_error_handle	設定 MP1 GPIO 保持輸出狀態
	_DMC_01_Get_GPIO_Output_error_handle	取得 MP1 GPIO 保持輸出設定值
	_DMC_01_get_mp1_dio_input	取得 MP1-P10D-15 8PIN 輸入腳位狀態 (MP1-P10D-15 專用)
	_DMC_01_get_mp1_dio_single_input	取得 MP1-P10D-15 單一 bit 輸入腳位狀態(MP1-P10D-15 專用)

ADLINK CONFIDENTIAL

	B02 Latch	
◇	_DMC_B02_ext_io_latch_initial	初始化 B02 Latch 功能
◇	_DMC_B02_ext_io_latch_function	設定 B02 Latch 參數規則
◇	_DMC_B02_ext_io_latch_enable	啟動 B02 Latch 功能
◇	_DMC_B02_latch_fifo_reset	清除 Latch 資料功能
◇	_DMC_B02_latch_fifo_status	取得 Latch FIFO 資訊值
◇	_DMC_B02_latch_fifo_size	取得 FIFO 內 Latch 的資料筆數
◇	_DMC_B02_latch_fifo_get_qep_counter	取得 FIFO 內 Latch 的 Encoder 資料(通用版適用)
◇	_DMC_B02_latch_fifo_get_qep_counter2	取得 FIFO 內 Latch 的 Encoder 資料。(僅支援 2D 版本)
◇	_DMC_B02_latch_fifo_dsp_get_data_counter	取得 Latch 的 Encoder 資料筆數。(僅支援 2D 版本)
◇	_DMC_B02_latch_fifo_dsp_data_counter_reset	重置 Latch 的 Encoder 資料筆數為 0。(僅支援 2D 版本)
◇	_DMC_B02_latch_fifo_dsp_data_clear	清除已從 FIFO 內存取的 Latch 資料。(僅支援 2D 版本)
	Motion Buffer API	
▲	_DMC_01_MotionBuf_Set_Crd_Parameters	設置坐標系參數
▲	_DMC_01_MotionBuf_Get_Crd_Parameters	查詢坐標系參數
▲	_DMC_01_MotionBuf_Line_MultiAxis	緩存區指令，多軸直線運動
▲	_DMC_01_MotionBuf_Line_MultiAxis_G00	緩存區指令，多軸直線運動，終點速度始終為 0 (與SNC定義不同)
▲	_DMC_01_MotionBuf_Arc_Radius	緩存區指令，2軸平面圓弧插補(以終點位置和半徑為輸入參數)
▲	_DMC_01_MotionBuf_Arc_CenterPos	緩存區指令，2軸平面圓弧插補(以終點位置和中心點為輸入參數)
▲	_DMC_01_MotionBuf_Set_LookAhead_Parameters	設置插補前瞻功能參數
▲	_DMC_01_MotionBuf_Start	啟動插補運動
▲	_DMC_01_MotionBuf_Stop	停止插補運動
▲	_DMC_01_MotionBuf_Get_Position	查詢指定表格坐標系的當前坐標位置值

DELTA CONFIDENTIAL

▲	_DMC_01_MotionBuf_Get_Velocity	查詢指定表格坐標系的合成速度值
▲	_DMC_01_MotionBuf_Feedrate_Override	設置插補運動目標合成速度倍率
▲	_DMC_01_MotionBuf_Get_Motion_Status	查詢插補運動狀態
▲	_DMC_01_MotionBuf_Set_Stop_Deceleration	設置插補運動平滑停止、急停合成加速度
▲	_DMC_01_MotionBuf_Get_Stop_Deceleration	查詢插補運動平滑停止、急停合成加速度
▲	_DMC_01_MotionBuf_SetUserSegNum	設置自定義插補段段號
▲	_DMC_01_MotionBuf_GetUserSegNum	讀取自定義插補段段號
▲	_DMC_01_MotionBuf_Clear_Buffer	清除插補緩存區內的插補數據
▲	_DMC_01_MotionBuf_Single_Step	單步執行功能
▲	_DMC_01_MotionBuf_BufGear	實現刀向跟隨功能，啟動某個軸跟隨運動
▲	_DMC_01_MotionBuf_BufMove	實現刀向跟隨功能，啟動某個軸點位運動
▲	_DMC_01_MotionBuf_IOControl	緩存區指令，IO 輸出功能
▲	_DMC_01_MotionBuf_Advanced_IOControl	緩存區指令，IO 提前輸出功能
▲	_DMC_01_MotionBuf_Delay	緩存區指令，延時功能
▲	_DMC_01_MotionBuf_Get_Remain_Buffer_Size	查詢插補緩存區剩餘空間
▲	_DMC_01_MotionBuf_SoftLimit_On	緩存區指令，緩存區內軟體極限開啟
▲	_DMC_01_MotionBuf_SoftLimit_Off	緩存區指令，緩存區內軟體極限關閉
▲	_DMC_01_MotionBuf_Clear_Crd	清除坐標系
▲	_DMC_01_MotionBuf_Disable	緩存區指令，使用此功能可以在流程中關閉 Motion Buffer 功能。
▲	_DMC_01_MotionBuf_Helix	緩存區指令,3軸立體螺旋插補(以螺旋深

DELTA CONFIDENTIAL

		度跟間距與中心點為輸入參數)
▲	_DMC_01_MotionBuf_Line_MultiAxis2	緩存區指令，多軸直線運動，可指定運動軸數。
▲	_DMC_01_MotionBuf_Line_MultiAxis2_G00	緩存區指令，多軸直線運動，終點速度始終為 0(與 SNC 定義不同)，可指定運動軸數。
▲	_DMC_01_MotionBuf_Arc_MidPos	緩存區指令，平面圓弧插補(以圓弧經過的中間點與目標點為輸入參數)
▲	_DMC_01_MotionBuf_MultiAxis_Arc1	緩存區指令，圓弧插補(以圓弧的圓心與角度為輸入參數)做多軸同動
▲	_DMC_01_MotionBuf_MultiAxis_Arc2	緩存區指令，圓弧插補(以圓弧的終點位置與角度為輸入參數)做多軸同動
▲	_DMC_01_MotionBuf_MultiAxis_Arc3	緩存區指令，圓弧插補(以圓弧的終點位置與圓心位置為輸入參數)做多軸同動
▲	_DMC_01_MotionBuf_MultiAxis_Arc4	緩存區指令，圓弧插補(以圓弧經過的中間點與目標點為輸入參數)做多軸同動
▲	_DMC_01_MotionBuf_Arc_follow	緩存區指令，圓弧插補(可指定任意兩軸作為圓弧運動的X和Y軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動。)
▲	_DMC_01_MotionBuf_Set_G00_Check	可設定比對 MultiAxis_G00 API 運行完成時驅動器 Feedback 是否達成目標範圍條件，達成條件後才執行下一筆指令，否則停止。(需建立坐標系後才能設定)
▲	_DMC_01_MotionBuf_Get_G00_Check	取得比對條件設定值
▲	_DMC_01_MotionBuf_Get_G00_Check_Flag	取得比對狀態
▲	_DMC_01_MotionBuf_Get_G00_Check_match	取得比對成功時的 Feedback

NOTE

表 4.2 中，有 ■ 符號標記的 API 表示在使用此 API 時，會比一般 API 耗費較長時間，程式撰寫中不適合無限制的重複下達此 API。

有 ▲ 符號標記的 API 表示 PCI_DMC_F01 軸卡不提供此功能。

◆ 符號標記為 PAC 使用。

◇ 符號標記為 B02 使用。

● 符號標記為 B01 使用。

DELTA CONFIDENTIAL

DELTA CONFIDENTIAL

第五章　硬體初始化 API

此章節說明了如何使用硬體初始化應用程式介面(API)，針對硬體資源初始化、取得軸卡卡號、序號、版本等資訊。

表 5.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_open	當應用程式啟動時，初始化系統資源
A/B/F	_DMC_01_close	釋放所有系統資源
A/B/F	_DMC_01_get_CardNo_seq	取得主機上所有 PCI-DMC 軸卡號
A/B/F	_DMC_01_pci_initial	初始此 PCI 軸卡
A/B/F	_DMC_01_get_card_version	取得軸卡版號
A/B/F	_DMC_01_get_fpga_version	取得軸卡 FPGA 版號
A/B	_DMC_01_get_model_type	取得 MP1/MH1 裝置主機板號碼

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

5.1 _DMC_01_open

■ **格式**

I16 PASCAL _DMC_01_open (I16* existcard)

■ **目的**

當應用程式啟動時，初始化系統資源。系統資源為軸卡與電腦端的系統資源配置，此資源必須開啟，方能接續取得卡號與裝置列表內容。

■ **參數**

名稱	資料型別	單位	描述
Existcard	I16*	數值單位	取得系統中掛載之該軸卡數量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 existCards;
I16 status = _DMC_01_open (&existCards);
```

DELTA CONFIDENTIAL

5.2 _DMC_01_close

■ **格式**

Void PASCAL _DMC_01_close ()

■ **目的**

釋放所有系統資源。關閉系統資源為軸卡與電腦端的系統資源釋放。

■ **參數**

無引用的參數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
_DMC_01_close();
```

DELTA CONFIDENTIAL

5.3 _DMC_01_get_CardNo_seq

■ **格式**

I16 PASCAL _DMC_01_get_CardNo_seq(U16 CardNo_seq,U16* CardNo)

■ **目的**

取得主機上所有 PCI-DMC 軸卡卡號。系統資源配置完畢，則會取得裝置號碼，根據裝置號碼則可以得到對應的軸卡卡號。

■ **參數**

名稱	資料型別	單位	描述
CardNo_seq	U16	編號單位	序列數
CardNo	U16*	編號單位	軸卡號 CardNo 為 0~15，藉由 SW1 設置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

1. 一張 PCI-DMC 安裝於主機時：

```
U16 CardNo_seq = 0;
U16 CardNo;
I16 status = _DMC_01_get_CardNo_seq(CardNo_seq, &CardNo);
```

2. 兩張或多張 PCI-DMC 安裝於主機時：

```
U16 CardNo_seq = 0;
U16 CardNo;
I16 status;
I16 existCards; //此 existCards 的值參閱 _DMC_01_open 的 API，詳述參閱程式手冊第 5.1 節。

for(CardNo_seq=0; CardNo_seq <existCards; CardNo_seq ++)
        status = _DMC_01_get_CardNo_seq(CardNo_seq, &CardNo);
```

NOTE

當僅一張 PCI-DMC 於主機上，且 SW1 設置為 3，此時取得該軸卡號為 3。

當兩張或多張 PCI-DMC 接於主機時，軸卡號將依系統掃描到之第 1 張 PCI-DMC 軸卡上 SW1 的值，並依序取得其它軸卡的卡號。

SW1 的設置，請參閱 PCI-DMC 使用手冊-第 1.5.5 節敘述。

DELTA CONFIDENTIAL

5.4 _DMC_01_pci_initial

■ **格式**

I16 PASCAL _DMC_01_pci_initial (U16 CardNo)

■ **目的**

初始此 PCI-DMC 軸卡。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;

I16 status = _DMC_01_pci_initial (CardNo);

DELTA CONFIDENTIAL

5.5　_DMC_01_get_card_version

■　**格式**

I16 PASCAL _DMC_01_get_card_version (U16 CardNo, U16 *ver)

■　**目的**

取得軸卡版號。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Ver	U16*	數值單位	軸卡版號資料。 0X60~69 是 PCI_DMC_A01 0X6A~6F 是 PCI_DMC_A02 0X70~79 是 PCI_DMC_B01 0X7A~7F 是 PCI_DMC_B02 0X80~8F 是 PCI_DMC_F01 0X8A 是 PCI_DMC_F02

※　PCI_DMC_A01 軸卡版號 0x64 開始提供 GPIO 為 Input x 8/Output x 4。
PCI_DMC_A02 軸卡提供 GPIO 均為 Input x 32/Output x 24。
PCI_DMC_B01 軸卡提供 GPIO 均為 Input x 1/Output x 1。
PCI_DMC_B02 軸卡提供 GPIO 均為 Input x 8/Output x 4。
PCI_DMC_F01 軸卡提供 GPIO 均為 Input x 8/Output x 4。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 ver;

I16 status = _DMC_01_get_card_version (CardNo, &ver);
```

DELTA CONFIDENTIAL

5.6 _DMC_01_get_fpga_version

■ **格式**

I16 PASCAL _DMC_01_get_fpga_version (U16 CardNo, U16 *ver)

■ **目的**

取得軸卡 FPGA 版號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Ver	U16*	數值單位	軸卡 FPGA 版號資料格式為 0x△△□□ △高位元組，□低位元組 ex. Ver = 0x0102 為 1.2 版

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 ver;

I16 status = _DMC_01_get_fpga_version(CardNo, &ver);
```

DELTA CONFIDENTIAL

5.7　_DMC_01_get_model_type

■　**格式**

I16 PASCAL _DMC_01_get_model_type (U16 CardNo, U16* nModelType)

■　**目的**

取得 MP1/MH1 裝置主機板號碼。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nModelType	U16*	數值單位	nModelType 裝置號碼可對照驅動包中 ModelType.h 定義 ex. nModelType=0x0223 為 MH1-C50D nModelType=0x0122 為 MP1-A12D_15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16* nModelType;

I16 status = _DMC_01_get_model_type (CardNo, &nModelType);
```

DELTA CONFIDENTIAL

第六章　軸卡操作 API

此章節說明了如何使用軸卡應用程式介面(API)，控制軸卡 GPIO、運動錯誤碼、MailBox、DSP、收送 PDO 指令、軸卡動態連結檔路徑等相關操作。

表 6.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_initial_bus	初始外部匯流排(External Bus)
A/B/F	_DMC_01_start_ring	起始 ring 的通訊
A/B/F	_DMC_01_get_device_table	取得裝置列表
A/B/F	_DMC_01_get_node_table	取得站號列表
A/B/F	_DMC_01_reset_card	重置該軸卡
A/B/F	_DMC_01_check_nodeno	查驗該站號是否已存在
A/B/F	_DMC_01_get_master_connect_status	取得 Master Card 與擴充模組目前的連線狀態
A/B/F	_DMC_01_get_mailbox_Error	取得 MailBox 報錯的次數
A/B/F	_DMC_01_get_mailbox_cnt	取得 MailBox 計數值
A/B/F	_DMC_01_get_dsp_cnt	取得中斷計數值
A/B/F	_DMC_01_set_dio_output	設置 GPIO 輸出腳位狀態
A/B/F	_DMC_01_get_dio_output	取得 GPIO 輸出腳位狀態
A/B/F	_DMC_01_get_dio_input	取得 GPIO 輸入腳位狀態
A/B/F	_DMC_01_set_dio_output_DW	設置 GPIO 輸出腳位狀態(for A02)
A/B/F	_DMC_01_set_dio_single_output_DW	設置 GPIO 單一 bit 輸出腳位狀態(for A02)
A/B/F	_DMC_01_get_dio_output_DW	取得 GPIO 輸出腳位狀態(for A02)
A/B/F	_DMC_01_get_dio_single_output_DW	取得 GPIO 單一 bit 輸出腳位狀態(for A02)
A/B/F	_DMC_01_get_dio_input_DW	取得 GPIO 輸入腳位狀態(for A02)
A/B/F	_DMC_01_get_dio_single_input_DW	取得 GPIO 單一 bit 輸入腳位狀態(for A02)
A/B/F	_DMC_01_set_dio_output_DW_error_handle	設定 GPIO 保持輸出狀態
A/B/F	_DMC_01_get_dio_output_DW_error_handle	取得 GPIO 保持輸出狀態
A/B/F	_DMC_01_get_cycle_time	取得當前查找裝置的循環時間
A/B/F	_DMC_01_initial_bus2	初始外部所有匯流排 (External Bus)

DELTA CONFIDENTIAL

A/B/F	_DMC_01_motion_cnt	取得 MailBox 及 DSP 計數值
A/B/F	_DMC_01_pdo_Rx_error	取得 PDO 收封包錯誤記錄
A/B/F	_DMC_01_pdo_Tx_error	取得 PDO 送封包錯誤記錄
A/B/F	_DMC_01_get_motion_error_code	取得 Motion 動作的錯誤回傳值
A/B/F	_DMC_01_get_motion_error_cnt	取得 Motion 動作的錯誤次數
A/B/F	_DMC_01_get_DLL_path	得到 PCI_DMC.dll 詳細路徑資料(合併版)
A/B/F	_DMC_01_get_DLL_version	得到 PCI_DMC.dll 版本資料(合併版)
A/B/F	_DMC_01_get_DLL_path_Single	得到 PCI_DMC.dll 詳細路徑資料(A/B/F01)
A/B/F	_DMC_01_get_DLL_version_Single	得到 PCI_DMC.dll 版本資料(A/B/F01)
A02/F02	_DMC_01_set_dio_single_output_DW_trigger_time	設定 DO 輸出狀態反向延遲功能
A02/F02	_DMC_01_set_dio_single_output_DW_one_shot	執行一次 DO 輸出狀態反向延遲功能
A02/F02	_DMC_01_get_dio_single_output_DW_trigger_status	取得 DO 輸出狀態反向延遲功能狀態
A02/F02	_DMC_01_set_dio_single_input_DW_trigger_mode	設定 A02/F02 單點 DI 觸發 DO 輸出功能
A02/F02	_DMC_01_get_dio_single_input_DW_trigger_mode	取得 A02/F02 單點 DI 觸發 DO 輸出功能狀態
A02/F02	_DMC_01_get_dio_single_input_DW_trigger_flag	取得觸發狀態
A/B	_DMC_01_set_dio_output	設置 A01/B01 GPIO 輸出單一 bit 腳位狀態
A/B	_DMC_01_get_dio_single_output	取得 A01/B01 GPIO 輸出單一 bit 腳位狀態
A/B	_DMC_01_get_dio_single_input	取得 A01/B01 GPIO 輸入單一 bit 腳位狀態
A/B	_DMC_01_set_dio_single_output_DW_trigger_cycle	設定 A02/F02 DO 輸出模式
A/B	_DMC_01_set_dio_single_output_DW_trigger_enable	啟動 A02/F02 DO 輸出

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA INTERNAL

6.1　_DMC_01_initial_bus

■ **格式**

I16 PASCAL _DMC_01_initial_bus (U16 CardNo)

■ **目的**

初始化單一軸卡外部匯流排(External Bus)，建立電腦與軸卡的通訊。

■ **參數**

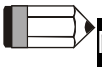
名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_initial_bus (CardNo);
```



NOTE

PCI_DMC_F01 在進行 _DMC_01_initial_bus 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode 的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請原卡原卡號再做一次 _DMC_01_initial_bus)。

DELTA CONFIDENTIAL

6.2 _DMC_01_start_ring

■ **格式**

I16 PASCAL _DMC_01_start_ring (U16 CardNo, U8 RingNo)

■ **目的**

建立軸卡上的 Ring 與模組的通訊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
RingNo	U8	編號單位	欲運行的 Ring 編號

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U8 RingNo = 0;

I16 status = _DMC_01_start_ring (CardNo, RingNo);
```

NOTE

當使用_DMC_01_start_ring 功能出現錯誤的時候，會一直重複 ring，在重複 ring 的過程中會記錄錯誤次數與錯誤代碼並回傳，直到成功為止，只要 API 能執行完畢就表示_DMC_01_start_ring 成功。

_DMC_01_start_ring 回傳訊息分為高位元與低位元，分別代表不同的意義，若是 status 不等於 0 ，例如收到的訊息是 4124 (十進制) ，將 4124 轉為 16 進制得到 101C，將訊息拆解兩部分，10 與 1C，10(十六進制)代表錯誤次數為 16 次，1C(十六進制)代表錯誤回傳代碼 28 (SDO 資料有送出但無回應)。

DELTA CONFIDENTIAL

6.3 _DMC_01_get_device_table

■ **格式**

I16 PASCAL _DMC_01_get_device_table (U16 CardNo, U16* value)

■ **目的**

取得 Ring 上已連線的 DMCNET 模組列表。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
value	U16*	編號單位	裝置編號列表

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 value=0;

I16 status = _DMC_01_get_device_table (CardNo, &value);
```

DELTA CONFIDENTIAL

6.4 _DMC_01_get_node_table

■ **格式**

I16 PASCAL _DMC_01_get_node_table (U16 CardNo, U32* NodeIDTable)

■ **目的**

取得 Ring 上已連線的 DMCNET 模組站號列表。
(必須宣告 NodeIDTable[4], 四個 U32 陣列元素)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDTable	U32*	編號單位	站號列表

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 NodeIDTable[4];

I16 status = _DMC_01_get_node_table( CardNo, &NodeIDTable);
```

DELTA CONFIDENTIAL

6.5 _DMC_01_reset_card

■ **格式**

I16 PASCAL _DMC_01_reset_card (U16 CardNo)

■ **目的**

重置軸卡。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;

I16 status = _DMC_01_reset_card (CardNo);
```

DELTA CONFIDENTIAL

6.6 _DMC_01_check_nodeno

■ **格式**

I16 PASCAL _DMC_01_check_nodeno(U16 CardNo, U16 NodeID,U16 SlotID,U16* exist)

■ **目的**

查驗 Ring 上已連線的 DMCNET 模組站號是否已存在。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
exist	U16*	選項單位	0：站號不存在 1：站號已存在

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 exist ;

I16 status = _DMC_01_check_nodeno (CardNo, NodeID, SlotID, &exist);
```

DELTA CONFIDENTIAL

6.7 _DMC_01_get_master_connect_status

■ **格式**

I16 PASCAL _DMC_01_get_master_connect_status (U16 CardNo, U16* Protocol)

■ **目的**

取得軸卡與模組目前的連線狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Protocol	U16*	選項單位	1：未接 Node1 模組 2：與模組連線失敗 3：與模組 Node1 連線正常 4：與模組 Node1 斷線

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 Protocol ;

I16 status = _DMC_01_get_master_connect_status (CardNo, &Protocol);
```

DELTA CONFIDENTIAL

6.8 _DMC_01_get_mailbox_Error

■ **格式**

I16 PASCAL _DMC_01_get_mailbox_Error (U16 CardNo, U32* error_cnt)

■ **目的**

MailBox 的存在是為了模組與電腦之間的資料同步存取而建立。此資料結構定義了模組與電腦間的通訊相關資料的紀錄。

取得 MailBox 報錯的次數。

MailBox: 軸卡與電腦之間的指令交握次數計算出指令執行錯誤的計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
error_cnt	U32*	編號單位	MailBox 報錯的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 error_cnt ;

I16 status = _DMC_01_get_mailbox_Error (CardNo, &error_cnt);
```

DELTA CONFIDENTIAL

6.9 _DMC_01_get_mailbox_cnt

■ **格式**

I16 PASCAL _DMC_01_get_mailbox_cnt (U16 CardNo, U32* PC_cnt, U32* DSP_cnt)

■ **目的**

取得 MailBox 計數值。

MailBox: 1.軸卡與電腦之間的指令交握次數計算出指令執行的計數值。

2.模組則是根據 MCU 每週期回傳指令計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
PC_cnt	U32*	次數	由 PC 端取得的 MailBox 計數值
DSP_cnt	U32*	次數	由 DSP 端取得的 MailBox 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U32 PC_cnt , DSP_cnt ;

I16 status = _DMC_01_get_mailbox_cnt (CardNo, &PC_cnt, &DSP_cnt);
```

DELTA CONFIDENTIAL

6.10 _DMC_01_get_dsp_cnt

■ **格式**

I16 PASCAL _DMC_01_get_dsp_cnt (U16 CardNo, U32* int_cnt, U32* main_cnt)

■ **目的**

取得 DSP 中斷訊號計數值。此 DSP 中斷訊號計數值使用在讀取軸卡中斷次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
int_cnt	U32*	次數	當通訊建構時發出的中斷計數值
main_cnt	U32*	次數	當 DSP 建構時發出的中斷計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 int_cnt , main_cnt;

I16 status = _DMC_01_get_dsp_cnt (CardNo, &int_cnt, &main_cnt);
```

DELTA CONFIDENTIAL

6.11 _DMC_01_set_dio_output

■ **格式**

I16 PASCAL _DMC_01_set_dio_output (U16 CardNo, U16 On_Off)

■ **目的**

設置軸卡 GPIO 輸出腳位狀態。根據此 API

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U16	選項單位	0：關閉 1：開啟 PCI_DMC_A01:4output(ex:0xF, =bit0~bit3 為 ON 表示輸出)。 PCI_DMC_B01: 1output(ex:0x1, =bit0 為 ON 表示輸出) 。 PCI_DMC_F01: 4output(ex:0xF, =bit0~bit3 為 ON 表示輸出) 。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 On_Off =1;

I16 status = _DMC_01_set_dio_output (CardNo, On_Off);
```

DELTA CONFIDENTIAL

6.12　_DMC_01_get_dio_output

■　**格式**

I16 PASCAL _DMC_01_get_dio_output (U16 CardNo, U16* On_Off)

■　**目的**

取得軸卡 GPIO 輸出腳位狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U16*	選項單位	0：關閉 1：開啟 PCI_DMC_A01:4output(ex:0xF, = bit0~bit3 狀態為 ON 表示腳位有輸出)。 PCI_DMC_B01: 1output(ex:0x1, = bit0 狀態為 ON 表示腳位有輸出) 。 PCI_DMC_F01: 4output(ex:0xF, = bit0~bit3 狀態為 ON 表示腳位有輸出) 。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 On_Off ;

I16 status = _DMC_01_get_dio_output (CardNo, &On_Off);
```

DELTA CONFIDENTIAL

6.13 _DMC_01_get_dio_input

■ **格式**

I16 PASCAL _DMC_01_get_dio_input (U16 CardNo, U16* On_Off)

■ **目的**

取得軸卡 GPIO 輸入腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U16*	選項單位	0：關閉 1：開啟 PCI_DMC_A01:8Input(ex:0xFF, =bit0~bit7 為 ON 表示腳位有輸入)。 PCI_DMC_B01: 1Input(ex:0x1, =bit0 為 ON 表示腳位有輸入) 。 PCI_DMC_F01: 8Input(ex:0xFF, =bit0~bit7 為 ON 表示腳位有輸入) 。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 On_Off ;

I16 status = _DMC_01_get_dio_input (CardNo, &On_Off);
```

DELTA CONFIDENTIAL

6.14 _DMC_01_set_dio_output_DW

■ **格式**

I16 PASCAL _DMC_01_set_dio_output_DW (U16 CardNo, U32 On_Off)

■ **目的**

設置 A02/F02 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U32	選項單位	0：關閉 1：開啟 PCI_DMC_A02:24output(ex:0xFFFFFF,bit0~bit23 為 ON 表示輸出) 。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 On_Off =0xF;

I16 status = _DMC_01_set_dio_output_DW (CardNo, On_Off);
```

DELTA CONFIDENTIAL

6.15　_DMC_01_set_dio_single_output_DW

■　**格式**

I16 PASCAL _DMC_01_set_dio_single_output_DW(U16 CardNo, U16 bitno, U32 On_Off)

■　**目的**

設置 A02/F02 GPIO 輸出單一 bit 腳位狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U32	選項單位	0：關閉 1：開啟

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 bitno = 0;
U32 On_Off =1;

I16 status = _DMC_01_set_dio_single_output_DW (CardNo, bitno, On_Off);
```

DELTA CONFIDENTIAL

6.16　_DMC_01_get_dio_output_DW

■ **格式**

I16 PASCAL _DMC_01_get_dio_output_DW (U16 CardNo, U32* On_Off)

■ **目的**

取得 A02/F02 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U32*	選項單位	0：關閉 1：開啟 PCI_DMC_A02:24output(ex:0xFFFFFF ,bit0~bit 23 狀態為 ON 表示腳位有輸出) 。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 On_Off ;

I16 status = _DMC_01_get_dio_output_DW (CardNo, &On_Off);
```

DELTA CONFIDENTIAL

6.17　_DMC_01_get_dio_single_output_DW

■　**格式**

I16 PASCAL _DMC_01_get_dio_single_output_DW (U16 CardNo, U16 bitno, U32* On_Off)

■　**目的**

取得 A02/F02 GPIO 輸出單一 bit 腳位狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U32*	選項單位	0：關閉 1：開啟

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 bitno=0;
U32 On_Off ;

I16 status = _DMC_01_get_dio_single_output_DW (CardNo,bitno, &On_Off);
```

DELTA CONFIDENTIAL

6.18　_DMC_01_get_dio_input_DW

■ **格式**

I16 PASCAL _DMC_01_get_dio_input_DW (U16 CardNo, U32* On_Off)

■ **目的**

取得 A02/F02 GPIO 輸入腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_Off	U32*	選項單位	0：關閉 1：開啟 PCI_DMC_A02:32Input(ex:0xFFFFFFFF,bit0~bit 31 為 ON 表示腳位有輸入) 。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 On_Off ;

I16 status = _DMC_01_get_dio_input_DW (CardNo, &On_Off);
```

DELTA CONFIDENTIAL

6.19 _DMC_01_get_dio_single_input_DW

■ **格式**

I16 PASCAL _DMC_01_get_dio_single_input_DW (U16 CardNo,U16 bitno,U32* On_Off)

■ **目的**

取得 A02/F02 GPIO 輸入單一 bit 腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U32*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 bitno=0;
U32 On_Off ;

I16 status = _DMC_01_get_dio_input_DW (CardNo, bitno, &On_Off);
```

DELTA CONFIDENTIAL

6.20　_DMC_01_set_dio_output_DW_error_handle

■　**格式**

I16 PASCAL _DMC_01_set_dio_output_DW_error_handle(U16 CardNo,U16 value)

■　**目的**

設定 A02/F02 GPIO 保持輸出狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
value	U16	選項單位	0：關閉 1：開啟

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 value = 1;

I16 status = _DMC_01_set_dio_output_DW_error_handle(CardNo,value);
```

DELTA CONFIDENTIAL

6.21 _DMC_01_get_dio_output_DW_error_handle

■ **格式**

I16 PASCAL _DMC_01_get_dio_output_DW_error_handle(U16 CardNo,U16 *value)

■ **目的**

取得 A02/F02 GPIO 保持輸出狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
value	U16*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 value;

I16 status = _DMC_01_get_dio_output_DW_error_handle (CardNo, &value);
```

DELTA CONFIDENTIAL

6.22　_DMC_01_get_cycle_time

■　**格式**

I16 PASCAL _DMC_01_get_cycle_time (U16 CardNo, I32 *time)

■　**目的**

取得當前模組通訊的循環時間，該時間必須小於 1000us，檢測模組是否通訊穩定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
time	I32*	數值單位	該 time 的值必須小於 1000

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
I32 time;

/*  取得模組的循環時間, time 的值必須小於 1000    */
I16 status = _DMC_01_get_cycle_time (CardNo, &time);
```

DELTA CONFIDENTIAL

6.23 _DMC_01_initial_bus2

■ **格式**

I16 PASCAL _DMC_01_initial_bus2 ( )

■ **目的**

初始化電腦上所有軸卡外部匯流排(External Bus)，建立電腦與軸卡的通訊。

■ **參數**

此函式無任何參數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 status = _DMC_01_initial_bus2( );
```

NOTE

PCI_DMC_F01 在進行 _DMC_01_initial_bus2 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode 的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請使用原卡，卡號再做一次 _DMC_01_initial_bus2)。

。

DELTA CONFIDENTIAL

6.24　_DMC_01_motion_cnt

■　**格式**

I16 PASCAL _DMC_01_motion_cnt (U16 CardNo, U16 NodeID, U16 SlotID,
U16 *pc_mc_cnt, U16 *dsp_mc_cnt)

■　**目的**

取得 MailBox 及 DSP 計數值。

MailBox: 1.軸卡與電腦之間的指令交握次數計算出運動指令執行的計數值。

2.模組則是根據 MCU 每週期回傳運動指令計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pc_mc_cnt	U16*	數值單位	PC 端入 MailBox Command 總數值。
dsp_mc_cnt	U16*	數值單位	DSP 由 MailBox 取得 Command 總數值。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 pc_mc_cnt;
U16 dsp_mc_cnt;

I16 status = _DMC_01_motion_cnt (CardNo, NodeID, SlotID, &pc_mc_cnt, &pc_mc_cnt);
```

DELTA CONFIDENTIAL

6.25 _DMC_01_pdo_Rx_error

■ **格式**

I16 PASCAL _DMC_01_pdo_Rx_error (U16 CardNo, U16 NodeID,U16 SlotID,
U32 *Rx_error1,U32 *Rx_error2,U16 set_get)

■ **目的**

取得伺服驅動器的回授封包，PDO 接收封包的錯誤記錄。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Rx_error1	U32*	數值單位	PDO　A Channel Rx Error Counters
Rx_error2	U32*	數值單位	PDO　B Channel Rx Error Counters
set_get	U16	選項單位	0：read Rx_error1,Rx_error2 value 1：write Rx_error1,Rx_error2 value

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Rx_error1;
U32 Rx_error2;
U16 set_get=0;


I16 status = _DMC_01_pdo_Rx_error (CardNo, NodeID, SlotID, &Rx_error1, &Rx_error2,
set_get);
```

DELTA CONFIDENTIAL

6.26　_DMC_01_pdo_Tx_error

■　**格式**

I16 PASCAL _DMC_01_pdo_Tx_error (U16 CardNo, U16 NodeID, U16 SlotID, U32 *Tx_error1, U32 *Tx_error2, U16 set_get)

■　**目的**

取得伺服驅動器命令封包，PDO 傳送封包的錯誤記錄。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tx_error1	U32*	數值單位	PDO　A Channel Tx Error Counters
Tx_error2	U32*	數值單位	PDO　B Channel Tx Error Counters
set_get	U16	選項單位	0：read Tx_error value 1：write Tx_error value

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Tx_error1,Tx_error2;
U16 ser_get=0;

I16 status = _DMC_01_pdo_Tx_error (CardNo, NodeID, SlotID, &Tx_error1, &Tx_error2,
set_get);
```

DELTA CONFIDENTIAL

6.27 _DMC_01_get_motion_error_code

■ **格式**

I16 PASCAL _DMC_01_get_motion_error_code (U16 CardNo, U16 NodeID,U16 SlotID, I16* error_code)

■ **目的**

取得運動控制命令錯誤碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_code	I16*	數值單位	Motion 動作的錯誤回傳值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 error_code;

I16 status = _DMC_01_get_motion_error_code (CardNo, NodeID, SlotID, & error_code);
```

DELTA CONFIDENTIAL

6.28 _DMC_01_get_motion_error_cnt

■ **格式**

I16 PASCAL _DMC_01_get_motion_error_cnt (U16 CardNo, U16 NodeID, U16 SlotID, I16* error_cnt)

■ **目的**

取得運動控制命令錯誤次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_cnt	I16*	數值單位	Motion 動作的錯誤次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 error_cnt;

I16 status = _DMC_01_get_motion_error_cnt (CardNo, NodeID, SlotID, & error_cnt);
```

DELTA CONFIDENTIAL

6.29　_DMC_01_get_DLL_path

■ **格式**

I16 PASCAL _DMC_01_get_DLL_path (char *lpFilePath, U32 nSize, U32 *nLength)

■ **目的**

得到 PCI_DMC.dll 詳細路徑資料(合併版)。

■ **參數**

名稱	資料型別	單位	描述
lpFilePath	Char*	字元陣列單位	PCI_DMC.dll 詳細路徑資料字元
nSize	U32	數值單位	路徑資料擷取長度
nLength	U32*	數值單位	傳回路徑資料實際長度

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
Char lpFilePath =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_path ( &lpFilePath, nSize, &nLength);
```

DELTA CONFIDENTIAL

6.30 _DMC_01_get_DLL_version

■ **格式**

I16 PASCAL _DMC_01_get_DLL_version (char *lpBuf, U32 nSize, U32 *nLength)

■ **目的**

得到 PCI_DMC.dll 版本資料(合併版)。

■ **參數**

名稱	資料型別	單位	描述
lpBuf	Char*	字元陣列單位	PCI_DMC.dll 詳細版本資料字元
nSize	U32	數值單位	版本資料擷取長度
nLength	U32*	數值單位	傳回版本資料實際長度

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
Char lpBuf =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_version ( & lpBuf, nSize, &nLength);
```

DELTA CONFIDENTIAL

6.31　_DMC_01_get_DLL_path_Single

■　格式

I16 PASCAL _DMC_01_get_DLL_path_Single (U16 CardNo, char *lpFilePath, U32 nSize, U32 *nLength)

■　目的

得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料。

■　參數

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
lpFilePath	Char*	字元陣列單位	PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料字元
nSize	U32	數值單位	路徑資料擷取長度
nLength	U32	數值單位	傳回路徑資料實際長度

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　範例

```
U16 CardNo= 0;
Char lpFilePath =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_path_Single (CardNo , &lpFilePath, nSize, &nLength);
```

DELTA CONFIDENTIAL

6.32 _DMC_01_get_DLL_version_Single

■ **格式**

I16 PASCAL _DMC_01_get_DLL_version_Single (U16 CardNo, char *lpBuf, U32 nSize, U32 *nLength)

■ **目的**

得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 版本資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
lpBuf	Char*	字元陣列單位	PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細版本資料字元
nSize	U32	數值單位	版本資料擷取長度
nLength	U32	數值單位	傳回版本資料實際長度

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
Char lpBuf =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_version_Single (CardNo , &lpBuf, nSize, &nLength);
```

DELTA CONFIDENTIAL

6.33　_DMC_01_set_dio_single_output_DW_trigger_time

■　**格式**

I16 PASCAL _DMC_01_set_dio_single_output_DW_trigger_time(U16 CardNo, U32 bitno, F64 time_s, U16 Enable)

■　**目的**

設定 A02/F02 DO 輸出狀態反向延遲功能。

開啟此功能後，以 6.34　one_shot　API 進行觸發，會將 DO 目前狀態反向輸出 (0->1 或 1->0)，並依設定時間保持反向輸出後，再將 DO 輸出狀態復原。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
time_s	F64	數值單位	觸發 DO 狀態反轉時間單位 S(秒) (設定時間的最小單位為 1ms)
Enable	U16	選項單位	0：關閉 1：開啟

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 biton=0;//設定 Bit 0 輸出
F64 time_s=1;//設定 DO 反向輸出時間保持 1 秒
U16 Enable=1;//開啟功能

I16 status = _DMC_01_set_dio_single_output_DW_trigger_time(CardNo, biton, time_s, Enable);
```

DELTA CONFIDENTIAL

NOTE

※此功能限 A02/F02 軸卡使用

DELTA CONFIDENTIAL

6.34　_DMC_01_set_dio_single_output_DW_one_shot

■ **格式**

I16 PASCAL _DMC_01_set_dio_single_output_DW_one_shot(U16 CardNo)

■ **目的**

執行一次 DO 輸出狀態反向延遲功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;

I16 status = _DMC_01_set_dio_single_output_DW_one_shot(CardNo);
```

DELTA CONFIDENTIAL

6.35　_DMC_01_get_dio_single_output_DW_trigger_status

■　**格式**

I16 PASCAL _DMC_01_get_dio_single_output_DW_trigger_status(U16 CardNo, U16* bitno, F64* time_s, U16* trigger_flag, U16* Enable)

■　**目的**

取得 DO 輸出狀態反向延遲功能狀態

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16*	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
time_s	F64*	數值單位	觸發 DO 狀態反轉時間單位 S(秒) (設定時間的最小單位為 1ms)
trigger_flag	U16*	數值單位	0：DO 為原狀態 1：DO 為反向輸出狀態中
Enable	U16*	選項單位	0：功能為關閉狀態 1：功能為開啟狀態

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16* biton;
F64* time_s;
U16* trigger_flag;
U16* Enable;

I16 status = _DMC_01_get_dio_single_output_DW_trigger_status(CardNo, bitno,
time_s, trigger_flag, Enable);
```

DELTA CONFIDENTIAL

NOTE

使用 6.34 _DMC_01_set_dio_single_output_DW_one_shot(CardNo) 執行 DO 輸出狀態反向延遲功能，當 DO 反向延遲輸出時 trigger_flag = 1，下一次執行 one_shot 需在 trigger_flag = 0，才能再執行。

DELTA CONFIDENTIAL

6.36　_DMC_01_set_dio_single_input_DW_trigger_mode

■ **格式**

I16 PASCAL _DMC_01_set_dio_single_input_DW_trigger_mode (U16 CardNo, U16 DI_bitno, U16 DO_bitno, U16 DO_polarity, U16 trigger_mode, U16 Enable)

■ **目的**

設定 A02/F02 單點 DI 觸發 DO 輸出功能

可指定一個 DI 點為觸發源，當觸發條件(負緣、正緣)達成，指定一個 DO 輸出所設定狀態(0、1)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
DI_bitno	U16	選項單位	欲操作的 Bit(0~31) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
DO_bitno	U16	選項單位	欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
DO_polarity	U16	選項單位	設定 DO 輸出狀態 (1 或 0)
trigger_mode	U16	選項單位	0: 負緣觸發 Falling edge 1: 正緣觸發 Rising edge
Enable	U16	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

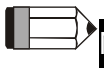
■ **範例**

```
U16 CardNo=0;
U16 DI_bitno=0;//設定 DI 0 為觸發源
U16 DO_bitno=0;//設定 DO 0 為輸出點
```

DELTA CONFIDENTIAL

U16 DO_polarity=1;//設定當觸發條件達成時，DO 0 輸出狀態 1
U16 trigger_mode=1;//設定正緣觸發模式
U16 Enable=1;//開啟功能

I16 status = _DMC_01_set_dio_single_input_DW_trigger_mode (CardNo, DI_bitno, DO_bitno, DO_polarity, trigger_mode, Enable)


NOTE

※此功能限 A02/F02 軸卡使用。
此功能只能指定一個 DI 對應一個 DO。

DELTA CONFIDENTIAL

6.37　_DMC_01_get_dio_single_input_DW_trigger_mode

■ **格式**

I16 PASCAL_DMC_01_get_dio_single_input_DW_trigger_mode(U16 CardNo, U16* DI_bitno, U16* DO_bitno, U16* DO_polarity, U16* trigger_mode, U16* Enable)

■ **目的**

取得 A02/F02 單點 DI 觸發 DO 輸出功能狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
DI_bitno	U16*	選項單位	取得觸發源 欲操作的 Bit(0~31) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
DO_bitno	U16*	選項單位	取得欲操作的 Bit(0~23) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
DO_polarity	U16*	選項單位	取得設定 DO 輸出狀態 (1 或 0)
trigger_mode	U16*	選項單位	0: 負緣觸發 Falling edge 1: 正緣觸發 Rising edge
Enable	U16*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16* DI_bitno;
U16* DO_bitno;
U16* DO_polarity;
```

DELTA CONFIDENTIAL

U16* trigger_mode;
U16* Enable;

I16 status = _DMC_01_get_dio_single_input_DW_trigger_mode (CardNo, DI_bitno, DO_bitno, DO_polarity, trigger_mode, Enable)

DELTA CONFIDENTIAL

6.38　_DMC_01_get_dio_single_input_DW_trigger_flag

■ **格式**

I16 PASCAL _DMC_01_get_dio_single_input_DW_trigger_flag (U16 CardNo, U16* DI_trigger_flag)

■ **目的**

取得觸發狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
DI_trigger_flag	U16*	數值單位	初始值為 0，當觸發條件達成時，DI_trigger_flag 狀態反向。 第一次觸發 DI_trigger_flag = 1 第二次觸發 DI_trigger_flag = 0 第三次觸發 DI_trigger_flag = 1 第四次觸發 DI_trigger_flag = 0 …… 以此類推

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16* DI_trigger_flag;

I16 status = _DMC_01_get_dio_single_input_DW_trigger_flag(CardNo, DI_trigger_flag);
```

DELTA CONFIDENTIAL

6.39　_DMC_01_set_dio_single_output

■ **格式**

I16 PASCAL _DMC_01_set_dio_single_output(U16 CardNo, U16 bitno, U16 On_Off)

■ **目的**

設置 A01/B01 GPIO 輸出單一 bit 腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U16	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 bitno = 0;
U16 On_Off =1;

I16 status = _DMC_01_set_dio_single_output (CardNo, bitno, On_Off);
```

DELTA CONFIDENTIAL

6.40 _DMC_01_get_dio_single_output

■ **格式**

I16 PASCAL _DMC_01_get_dio_single_output(U16 CardNo, U16 bitno, U16 *On_Off)

■ **目的**

取得 A01/B01 GPIO 輸出單一 bit 腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U16*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 bitno=0;
U16 On_Off ;

I16 status = _DMC_01_get_dio_single_output (CardNo,bitno, &On_Off);
```

DELTA CONFIDENTIAL

6.41 _DMC_01_get_dio_single_input

■ **格式**

I16 PASCAL _DMC_01_get_dio_single_input(U16 CardNo, U16 bitno, U16 *On_Off)

■ **目的**

取得 A01/B01 GPIO 輸入單一 bit 腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U16	選項單位	如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U16*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 bitno=0;
U32 On_Off ;

I16 status = _DMC_01_get_dio_single_input (CardNo, bitno, &On_Off);
```

DELTA CONFIDENTIAL

6.42 _DMC_01_set_dio_single_output_DW_trigger_cycle

■ **格式**

I16 PASCAL _DMC_01_set_dio_single_output_DW_trigger_cycle(U16 CardNo, U32 bitno, U16 On_time, U16 Off_time, U16 times, U16 trigger_mode)

■ **目的**

設定 A02/F02 DO 輸出模式

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
bitno	U32	選項單位	如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_time	U16	數值單位	設定 DO 狀態 1 持續時間 ,單位:ms Min:1 Max:65535
Off_time	U16	數值單位	設定 DO 狀態 0 持續時間 ,單位:ms Min:1 Max:65535
times	U16	數值單位	輸出週期次數: 定義 0->1 或 1->0 為一次輸出週期。 Min:1 Max:65535
trigger_mode	U16	選項單位	0:狀態輸出 0->1, 持續輸出直到 disable 1:狀態輸出 0->1, 依據 times 設定次數輸出 2:狀態輸出 1->0, 持續輸出直到 disable 3:狀態輸出 1->0, 依據 times 設定次數輸出

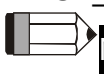
■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 bitno=0;
U16 On_time=1, Off_time=1, times=10, trigger_mode=1;
```

I16 status = _DMC_01_set_dio_single_output_DW_trigger_cycle(CardNo, bitno, On_time, Off_time, times, trigger_mode);


NOTE

※此功能限 A02/F02 軸卡使用

DELTA CONFIDENTIAL

6.43　_DMC_01_set_dio_single_output_DW_trigger_enable

■ **格式**

I16 PASCAL _DMC_01_set_dio_single_output_DW_trigger_enable(U16 CardNo, U16 enable)

■ **目的**

啟動 A02/F02 DO 輸出(需先設定 6.42)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
enable	U16	選項單位	0：關閉 1：啟動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 enable =1;

I16 status = _DMC_01_set_dio_single_output_DW_trigger_enable(CardNo, enable)
```

NOTE

※此功能限 A02/F02 軸卡使用

※ 若 6.42 模式設定為 1 或 3，達成輸出次數後保持最後輸出狀態並關閉。

DELTA CONFIDENTIAL

第七章　伺服驅動器參數讀取/寫入 API

此章節說明了如何使用伺服驅動器應用程式介面(API)，針對伺服驅動器的參數寫入與讀取，一旦設定參數群組與 P 碼即可操作伺服驅動器資料。

表 7.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_read_servo_parameter	讀取伺服驅動器參數資料
A/B/F	_DMC_01_write_servo_parameter	寫入伺服驅動器參數資料

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

7.1 _DMC_01_read_servo_parameter

■ **格式**

I16 PASCAL _DMC_01_read_servo_parameter (U16 CardNo, U16 NodeID, U16 SlotID, U16 group,U16 idx,U32* data)

■ **目的**

讀取伺服驅動器參數資料(屬於 SDO 指令，執行時間較長，不建議放在常態掃描程式區)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
group	U16	數值	模組(驅動器)群組編號
idx	U16	數值	驅動器參數群組的索引值
data	U32*	數值	該群組索引所傳回的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 group = 3;
U16 idx = 0;        // 當您設定此例的群組與索引值，表示您欲讀取驅動器參數”P3-00”
U32 data;

I16 status = _DMC_01_read_servo_parameter(CardNo, NodeID, SlotID, group, idx, &data);
```

DELTA CONFIDENTIAL

7.2 _DMC_01_write_servo_parameter

■ **格式**

I16 PASCAL _DMC_01_write_servo_parameter (U16 CardNo, U16 NodeID, U16 SlotID, U16 group,U16 idx,U32 data)

■ **目的**

寫入伺服驅動器參數資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
group	U16	數值	模組(驅動器)群組編號
idx	U16	數值	驅動器參數群組的索引值
data	U32	數值	欲寫入該群組索引的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 group = 3;
U16 idx = 0;        // 當您設定此例的群組與索引值，表示您將把資料寫入驅動器參數”P3-00”
U32 data = 1;

I16 status = _DMC_01_write_servo_parameter(CardNo, NodeID, SlotID, group, idx, data);
```

DELTA CONFIDENTIAL

第八章　使用 SDO 協定 API

此章節說明了如何使用 SDO 應用程式介面(API)，操作 SDO 對伺服端資料緩衝區命令發送與接收，以及排除發送接收的錯誤機制。

表 8.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_check_canopen_lock	SDO 模式下，檢查是否可以再執行下一筆命令
A/B/F	_DMC_01_get_canopen_ret	取得 CANopen 傳回的資訊(SDO 相關數據)
A/B/F	_DMC_01_set_pdo_mode	設置使用的 CANopen 協定(PDO 或 SDO)
A/B/F	_DMC_01_send_message	發送 SDO 命令信息至資料緩衝區，會等待資料送達伺服端且讀回資料相符才結束
A/B	_DMC_01_send_message2	發送 SDO 命令信息至資料緩衝區，只確認資料送達伺服端就結束
A/B	_DMC_01_send_message3	發送 SDO 命令信息至資料緩衝區，但發出命令後立即結束
F	_DMC_01_send_message4	發送 SDO 命令信息至資料緩衝區，但發出命令後立即結束
A/B/F	_DMC_01_read_message	讀取最後一筆 SDO 命令信息至資料緩衝區
A/B/F	_DMC_01_read_message2	讀取指定的站號最後一筆 SDO 命令信息至資料緩衝區，並傳回已完成的次數值
A/B/F	_DMC_01_get_message	取得模組插槽 SDO 命令信息至資料緩衝區
A/B/F	_DMC_01_reset_sdo_choke	當 SDO 命令因故阻塞時，重置 SDO
A/B/F	_DMC_01_get_sdo_retry_history	取得 SDO 重新發送的次數

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

說明：本章節是特別針對有需要直接對伺服做存取運用所做的說明，若您無此需求，

可直接跳過本章節。

DELTA CONFIDENTIAL

8.1　CANopen SDO 協定格式

※CANopen 的 SDO 封包格式範列 1 – Index 讀取/寫入成功

當讀取 Index 為「成功」，該封包格式傳回值如下表：

表 8.2

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x64	0x60	0x00	0x00	0x00	0x00	0x00
回傳	0x43	0x64	0x60	0x00	0x11	0x22	0x33	0x44

表格說明：

(1)　讀取 Feedback PUU(此時 CANopen 的 Index 值填入 0x6064，SubIdx 值填入 0x00)。

(2)　當 Datatype 值填入 0x40，表示讀取。

(3)　隨後回傳的 Datatype 值為 0x43，

若回傳之 Datatype 值為 0x43 表示收到的資料為 32bit [Data1~Data4 均為有效]，

值為 0x4b 表示收到的資料為 16bit [Data1~Data2 有效]，

值為 0x4f 表示收到的資料為　8bit [Data1 有效]。

(4)　Data1~Data4 的欄位為低 Word 優先讀取，而無效的高位 Data 則為 0，

因此上述表格陳列之回傳值表示收到的 Feedback PUU 為 0x44332211。

當寫入 Index 為「成功」，該封包格式傳回值如下表：

表 8.3

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2b	0x40	0x60	0x00	0x08	0x00	0x00	0x00
回傳	0x60	0x40	0x60	0x00	0x00	0x00	0x00	0x00

表格說明：

(1)　寫入控制碼 ControlWord(此時 CANopen 的 Index 值填入 0x6040，SubIdx 的值填入 0x00)。

(2)　Datatype 填入 0x2b，表示寫入。

(3)　若填入 Datatype 的值為 0x23 表示寫入的資料為 32bit [Data1~Data4 均為有效]，

填入值為 0x2b 表示寫入的資料為 16bit [Data1~Data2 有效]，

填入值為 0x2f 表示寫入的資料為　8bit [Data1 有效]。

(4)　其回傳的 Datatype 值為 0x60，表示寫入成功；此時回傳的 Data1~Data4 無意義。

(5)　Data1~Data4 的欄位為低 Word 優先，而無效的高位 Data 請以 0 遞補。

因此上述陳列之回傳值表示寫入的控制碼 ControlWord 為 0x00000008。

(6)　該 Index 的資料長度請參考台達 ASDA-A2 CANopen Technical Guide。

DELTA CONFIDENTIAL

※CANopen 的 SDO 封包格式範列 2 – Index 讀取/寫入失敗

當讀取 Index 為「失敗」(即查無此索引)，該封包格式傳回值如下表：

表 8.4

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x59	0x60	0x00	0x00	0x00	0x00	0x00
回傳	0x80	0x59	0x60	0x00	0x00	0x00	0x02	0x06

表格說明：

(1) 讀取一個無效的索引(此時 CANopen 的 Index 值填入 0x6059，SubIdx 的值填入 0x00)。

(2) 當 Datatype 值填入 0x40，表示讀取。

(3) 當驅動器內無此索引，故此讀取失敗，隨後回傳的 Datatype 值為 0x80，表示回傳讀取失敗。

(4) 上表格欄位表示回傳之 Data1~Data4 組成的資料為 0x06020000[無此索引]錯誤碼。

(5) 詳細說明，請參考台達 ASDA-A2 CANopen Technical Guide 中的 SDO Abort Code。

當寫入 Index 為「失敗」(即寫入的數值超出設置範圍)，該封包格式傳回值如下表：

表 8.5

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2f	0x60	0x60	0x00	0x0F	0x00	0x00	0x00
回傳	0x80	0x60	0x60	0x00	0x30	0x00	0x09	0x06

表格說明：

(1) 寫入操作模式(此時 CANopen 的 Index 值填入 0x6060，SubIdx 的值填入 0x00)。

(2) Datatype 填入 0x2f，表示寫入的資料為 8bit [Data1 有效]。

(3) 因操作模式的範圍為 0x00~0x07，故此命令寫入失敗，隨後回傳的 Datatype 值為 0x80，表示回傳寫入失敗。

(4) 上表格欄位表示回傳之 Data1~Data4 組成的資料為 0x06090030[寫入數值超出範圍]錯誤碼。

(5) 該 index 的資料長度請參考台達 ASDA-A2 CANopen Technical Guide。

其它詳細說明，請參考台達 ASDA-A2 CANopen Technical Guide 中的 SDO Abort Code。.

DELTA CONFIDENTIAL

※CANopen 的 SDO 封包格式範列 3 – 讀取/寫入 ASDA-A2 參數

當欲讀取 ASDA-A2 參數，該封包格式傳回值如下表：

表 8.6

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x00	0x23	0x00	0x00	0x00	0x00	0x00
回傳	0x4b	0x00	0x23	0x00	0x7F	0x00	0x00	0x00

表格說明：

(1) 可用標準 CANopen SDO 格式讀取 A2 參數。

(2) P3-00[站號] (CANopen 的 Index 值填入 0x2300 SubIdx 的值填入 0x00)。
格式：P△-○○ [驅動器參數] ⟷ 0x2△□□ [Index 值] (□□為○○的 16 進制表示)。
即若 Index 值設置為 0x2300，表示讀取 P3-00 的參數
若 Index 值設置為 0x212D，表示讀取 P1-45 的參數(2D 為 45 的 16 進制表示)

(3) SubIdx 的值均為 0x00。

當欲寫入 ASDA-A2 參數，該封包格式傳回值如下表：

表 8.7

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2b	0x00	0x23	0x00	0x7F	0x00	0x00	0x00
回傳	0x60	0x00	0x23	0x00	0x00	0x00	0x00	0x00

表格說明：

(1) 可用標準 CANopen SDO 格式寫入 A2 參數。

(2) P3-00[站號] (CANopen 的 Index 值填入 0x2300 SubIdx 的值填入 0x00)。
格式：P△-○○ [驅動器參數] ⟷ 0x2△□□ [Index 值] (□□為○○的 16 進制表示)。

(3) SubIdx 的值均為 0x00。

(4) ASDA-A2 參數的資料長度為 16bit 或 32bit

(5) 您可參考 ASDA-A2 的 EDS 檔，尋找對應參數 Index 的 DataType。
若 DataType 為 3，此時資料長度為 16bit，命令寫入的 DataType 應填入值為 0x2b
若 DataType 為 4，此時資料長度為 32bit，命令寫入的 DataType 應填入值為 0x23

DELTA CONFIDENTIAL

8.2 _DMC_01_check_canopen_lock

■ **格式**

I16 PASCAL _DMC_01_check_canopen_lock (U16 CardNo, U16 *lock)

■ **目的**

在 SDO 模式下，檢查是否可以再執行下一筆命令。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
lock	U16*	選項單位	0：未栓鎖(傳送成功)，可執行下一筆命令。 1：已栓鎖，須等候目前命令執行完成。 2：傳送失敗。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 lock;

I16 status = _DMC_01_check_canopen_lock (CardNo, &lock);
```

DELTA CONFIDENTIAL

8.3 _DMC_01_get_canopen_ret

■ **格式**

I16 PASCAL _DMC_01_get_canopen_ret (U16 CardNo, U16* COBID, U8* value)

■ **目的**

取得 CANopen 傳回的資訊(SDO 相關數據)。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
COBID	U16*	數值	CAN 物件的 ID
value	U8*	數值	物件的數據內文 Value[0]：SDO 信息回覆 數據內文長度為 32-bit，值為 43 數據內文長度為 16-bit，值為 4b 數據內文長度為 8-bit，值為 4f 數據寫入成功，值為 60 報錯，值為 80 Value[1]：Index(低位元組) Value[2]：Index(高位元組) Value[3]：Subindex Value[4]：Data low word(低位元組) Value[5]：Data low word(高位元組) Value[6]：Data high word(低位元組) Value[7]：Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 COBID=0;
U8 value[8]={0};

I16 status = _DMC_01_get_canopen_ret (CardNo, &COBID, value);
```

DELTA CONFIDENTIAL

8.4 _DMC_01_set_pdo_mode

■ **格式**

I16 PASCAL _DMC_01_set_pdo_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 Enable)

■ **目的**

設定 CANOpen 模式(PDO 或 SDO)。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0: 關閉 PDO，使用 SDO 1: 開啟 PDO

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Enable=1;

I16 status = _DMC_01_set_pdo_mode (CardNo, NodeID, SlotID, Enable);
```

DELTA CONFIDENTIAL

8.5 _DMC_01_send_message

■ **格式**

I16 PASCAL _DMC_01_send_message (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■ **目的**

發送 SDO 命令信息至資料緩衝區。(此 API 函式會等待資料送達伺服端且讀回資料相符才結束)
(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

DELTA CONFIDENTIAL

■　範例

```
/*  相關參數說明
    CardNo：Card No；NodeID：站號 ID；SlotID：插槽 ID；  Index：SDO 索引值；  SubIdx：
    SDO 子索引值；  DataType：(讀取命令)讀取數據資料，填入值 0x40；   (寫入命令)寫入
    8-bit 資料，填入值 0x2f；(寫入命令)寫入 16-bit 資料，填入值 0x2b；(寫入命令)寫入 32-bit
    資料，填入值為 0x23；
      Value0：SDO data low word(低位元組),   Value1：SDO data low word(高位元組)；
      Value2：SDO data high word(低位元組),   Value3：SDO data high word(高位元組)；
*/
  U16 CardNo=0;
  U16 NodeID=1;
  U16 SlotID=0;
  U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

  I16 status = _DMC_01_send_message (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
               value0, value1, value2, value3);
```

NOTE

_DMC_01_send_message、_DMC_01_send_message2、_DMC_01_send_message3 與 _DMC_01_send_message4 的比較。

命令名稱	比較說明
_DMC_01_send_message	於 SDO 訊息送出(TX)，等待目的地 RX 回應後，再發送訊息至目的地將資料讀出，與送出訊息做比對是否正確。 (PCI_DMC_01 與 PCI_DMC_F01 都支援此 API)
_DMC_01_send_message2	於 SDO 訊息送出(TX)，等待目的地 RX 回應後，就完成此 API 離開。下達參數與_DMC_01_send_message 完全相同。 (PCI_DMC_01 與 PCI_DMC_F01 都支援此 API)
_DMC_01_send_message3	於 SDO 訊息送出(TX)，不等待目的地的 RX 回應馬上離開此 API(一次對 12 軸下達)。 (僅 PCI_DMC_01 支援此 API)
_DMC_01_send_message4	於 SDO 訊息送出(TX)，不等待目的地的 RX 回應馬上離開 此 API，下達參數與_DMC_01_send_message3 完全 相同。 (僅 PCI_DMC_F01 支援此 API) 可以使用_DMC_01_check_canopen_lock (CardNo, &lock);這個 API 的回傳值 Lock 來判斷傳送的狀態，0 代表傳送成功，1 代表 SDO 處理中，2 代表傳送失敗。

DELTA CONFIDENTIAL

8.6 _DMC_01_send_message2

■ **格式**

I16 PASCAL _DMC_01_send_message2 (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■ **目的**

發送 SDO 命令信息至資料緩衝區。 (此 API 函式只確認資料送達伺服端就結束) (台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_send_message2 (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
             value0, value1, value2, value3);
```

DELTA CONFIDENTIAL

8.7　_DMC_01_send_message3

■　**格式**

I16 _DMC_01_send_message3 (U16 CardNo, U16 Index,U16 SubIdx,U16 DataType, U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■　**目的**

發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區(僅限 PCI_DMC_01 支援此 API，且一次對 12 軸下達相同訊息)。(台達限定)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■　**範例**

```
U16 CardNo=0 , status = 0, lock = 0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
         value2, value3);
do{
        status = _DMC_01_check_canopen_lock (CardNo, &lock);
} while(lock);
value0=0x2;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
          value2, value3);
```

DELTA CONFIDENTIAL

8.8 _DMC_01_send_message4

■ **格式**

I16 PASCAL _DMC_01_send_message4 (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■ **目的**

發送 SDO 命令信息至資料緩衝區，但發出命令後立即結束。(僅限 PCI_DMC_F01 支援此 API)
(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_send_message4 (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
             value0, value1, value2, value3);
```

DELTA CONFIDENTIAL

8.9　_DMC_01_read_message

■　**格式**

I16 PASCAL _DMC_01_read_message (U16 CardNo, U16* COBID, U16* Cmd,
U16* SubIdx, U16* Value0,U16* Value1,U16* Value2,U16* Value3)

■　**目的**

使用者設定想要讀取的物件內容，讀取 SDO 命令信息至資料緩衝區。(台達限定)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
COBID	U16*	數值	CAN 物件 ID(0x580 +目前的 Node ID)
Cmd	U16*	數值	物件字典的索引值
SubIdx	U16*	數值	物件字典的子索引值
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■　**範例**

```
/*  相關參數說明
   CardNo：Card No；COBID : CAN 物件 ID(0x580 +目前的 Node ID); NodeID：站號 ID；
   Cmd：SDO 索引值；  SubIdx：SDO 子索引值；
   Value0：SDO data low word(低位元組),   Value1：SDO data low word(高位元組)；
   Value2：SDO data high word(低位元組),   Value3：SDO data high word(高位元組)；
*/
U16 CardNo=0,
U16 COBID, Cmd, SubIdx, Value0, Value1, Value2, Value3;

I16 status = _DMC_01_read_message(CardNo, &COBID, &Cmd, & SubIdx,
             &Value0, &Value1, &Value2, &Value3);
```

DELTA CONFIDENTIAL

8.10 _DMC_01_read_message2

■ **格式**

I16 PASCAL _DMC_01_read_message2 (U16 CardNo, U16 NodeID, U16 *Cmd,
U16 *COBID ,U16 * SubIdx, U16 *Value0,U16 *Value1,U16 *Value2,U16 *Value3,U16 *cnt)

■ **目的**

讀取 SDO 命令信息至資料緩衝區，並記錄傳送已完成的次數。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
Cmd	U16*	數值	物件字典的索引值
COBID	U16*	數值	CAN 物件 ID(0x580 +目前的 Node ID)
SubIdx	U16*	數值	物件字典的子索引值
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value2	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)
cnt	U16*	數值	SDO 訊息傳送完成次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0, Cmd, COBID, Value0, Value1, Value2, Value3, cnt, temp;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_read_message2(CardNo, NodeID , &Cmd, &COBID, & SubIdx,
              &Value0, &Value1, &Value2, &Value3, &cnt)
temp = cnt;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
              value2, value3);
status = _DMC_01_read_message2(CardNo, NodeID , &Cmd, &COBID, & SubIdx, &Value0,
             &Value1, &Value2, &Value3, &cnt)
if(cnt-temp)
{
        status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0,
                value1,value2, value3);
}
```

DELTA CONFIDENTIAL

8.11 _DMC_01_get_message

■ **格式**

I16 PASCAL _DMC_01_get_message (U16 CardNo, U16 NodeID, U16 SlotID, U16 Index, U16 SubIdx,U16* Cmd, U16* COBID, U16* DataType, U16* Value0, U16* Value1, U16* Value2, U16* Value3)

■ **目的**

可從 Slot，取得 SDO 命令信息至資料緩衝區，耗時較長。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
Cmd	U16*	數值	物件字典的索引值
COBID	U16*	數值	CAN 物件 ID
DataType	U16*	數值	物件字典的資料類型
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

U16 CardNo=0, NodeID=1,　SlotID=0, Index=0x6060, SubIdx1=0;
U16 Cmd, COBID, SubIdx2, Value0, Value1, Value2, Value3;

I16 status = _DMC_01_get_message(CardNo, NodeID, SlotID, Index, SubIdx1, &Cmd, &COBID, & SubIdx2, &Value0, &Value1, &Value2, &Value3);

DELTA CONFIDENTIAL

8.12　_DMC_01_reset_sdo_choke

■　**格式**

I16 PASCAL _DMC_01_reset_sdo_choke (U16 CardNo)

■　**目的**

當 SDO 命令因故阻塞時，重置 SDO 狀態。(台達限定)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo=0;

I16 status = _DMC_01_reset_sdo_choke (CardNo);

DELTA CONFIDENTIAL

8.13 _DMC_01_get_sdo_retry_history

■ **格式**

I16 PASCAL _DMC_01_get_sdo_retry_history (U16 CardNo, U32* cnt)

■ **目的**

取得 SDO 重新發送的信息次數。(台達限定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
cnt	U32*	次數	傳回 SDO 重新發送信息的次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 cnt=0;

I16 status = _DMC_01_get_sdo_retry_history (CardNo , &cnt);
```

DELTA CONFIDENTIAL

第九章　點對點運動控制使用封包協定 API

此章節說明了如何使用點對點運動應用程式介面(API)，設定伺服驅動器(PP)的運動控制，設定點對點操作參數。

表 9.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_sdo_driver_speed_profile	使用 SDO 送速度參數
A/B	_DMC_01_start_sdo_driver_r_move	相對性運動命令
A/B	_DMC_01_start_sdo_driver_a_move	絕對性運動命令
A/B	_DMC_01_start_sdo_driver_new_position_move	改變運動位置命令

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

9.1 _DMC_01_set_sdo_driver_speed_profile

■ **格式**

I16 PASCAL _DMC_01_set_sdo_driver_speed_profile (U16 CardNo, U16 NodeID, U16 SlotID, U32 MaxVel, F64 acc, F64 dec)

■ **目的**

使用封包協定的速度參數配置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MaxVel	U32	脈波數/秒	最大速度參數
acc	F64	秒	指定的加速時間
dec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 MaxVel =1000000;
F64 acc=0.1;
F64 dec=0.1;

I16 status = _DMC_01_set_sdo_driver_speed_profile (CardNo, NodeID, SlotID, MaxVel, acc,
             dec);
```

DELTA CONFIDENTIAL

9.2　_DMC_01_start_sdo_driver_r_move

■ **格式**

I16 PASCAL _DMC_01_start_sdo_driver_r_move (U16 CardNo, U16 NodeID, U16 SlotID, I32 Distance)

■ **目的**

起始一個相對性的運動位移。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Distance	I32	脈波數	相對性的運動行程

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Distance =10000000;

I16 status = _DMC_01_start_sdo_driver_r_move (CardNo, NodeID, SlotID, Distance);
```

DELTA CONFIDENTIAL

9.3 _DMC_01_start_sdo_driver_a_move

■ **格式**

I16 PASCAL _DMC_01_start_sdo_driver_a_move (U16 CardNo, U16 NodeID, U16 SlotID, I32 Position)

■ **目的**

起始一個絕對性的運動位移。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Position	I32	脈波數	絕對性的運動位置

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Position =10000000;

I16 status = _DMC_01_start_sdo_driver_a_move (CardNo, NodeID, SlotID, Position);
```

DELTA CONFIDENTIAL

9.4 _DMC_01_start_sdo_driver_new_position_move

■ **格式**

I16 PASCAL _DMC_01_start_sdo_driver_new_position_move (U16 CardNo, U16 NodeID, U16 SlotID, I32 Position,U16 abs_rel)

■ **目的**

變換新的 position 值進行運動位移。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Position	I32	數值	新的 position 值
abs_rel	U16	選項單位	0：參考絕對座標 1：參考相對座標

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

U16 CardNo=0;
U16 NodeID =1;
I32 Position =20000000;
U16 abs_rel =1; //參考相對座標進行運動位移

I16 status = _DMC_01_start_sdo_driver_r_move (CardNo, NodeID, SlotID, Position, abs_rel);

DELTA CONFIDENTIAL

第十章　原點復歸運動控制使用封包協定 API

此章節說明了如何使用原點復歸運動應用程式介面(API)，控制伺服驅動器回原點模式下的指令操作。原點模式的配置、參數設定。

表 10.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_home_config	設置原點復歸各項配置
A/B/F	_DMC_01_set_home_move	開始運行原點復歸運動
A/B/F	_DMC_01_set_home_config2	設置原點復歸各項配置
A/B/F	_DMC_01_set_home_config2_offset2	重設原點復歸 offset 值
A/B/F	_DMC_01_set_home_move2	開始運行原點復歸運動
A/B/F	_DMC_01_escape_home_move	停止原點復歸運動
A/B/F	_DMC_01_set_home_config_by_torque_limit	DMC 模式底下，開始運行原點復歸運動，配合 Torque_limit。

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

10.1 _DMC_01_set_home_config

■ **格式**

I16 PASCAL _DMC_01_set_home_config (U16 CardNo,U16 NodeID,U16 SlotID ,
U16 Mode,I32 offset,U16 lowSpeed,U16 highSpeed,F64 acc)

■ **目的**

設置原點復歸各項配置(回 Home 前，請關閉軟體極限設定)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考後頁 note 說明)
offset	I32	脈波數	原點復歸偏移量
lowSpeed	U16	轉速/分	歸回零所使用的速度參數
highSpeed	U16	轉速/分	尋找各極限位置所使用的速度參數
acc	F64	秒	原點復歸參照的加速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

※ lowSpeed 與 highSpeed 參數的單位, 會因連結的 Slave 模組有所區別。
伺服驅動器（ASDA-A2F）➔轉速/分
伺服驅動器（ASDA-A3F）➔0.1*轉速/分
脈波介面模組（RM04PI, GE01PH）➔Pulse/Sec
脈波模組回原點最高速為 65535Pulse/Sec，若要運用更高的速度，需搭配 27.5 節的 DMC_01_set_rm_04pi_homing_ratio(詳見)。
線性馬達➔um/sec。

注意:_DMC_01_Set_home_config 與_DMC_01_set_home_move 是一組回 Home 完整的參數設定，回 Home 進行中，不可重複下達_DMC_01_Set_home_config。

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;

/* 使用原點復歸模式*/
 I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed,
          highSpeed, acc);
```

NOTE

#1. 以負極限與索引脈波信息之原點復歸模式

使用此方法，若負極限開關為無效(為低電位)時，初始方向為向左進行位移運動。此原點位置為當觸碰負極限開關時即往右方之索引脈衝位移。

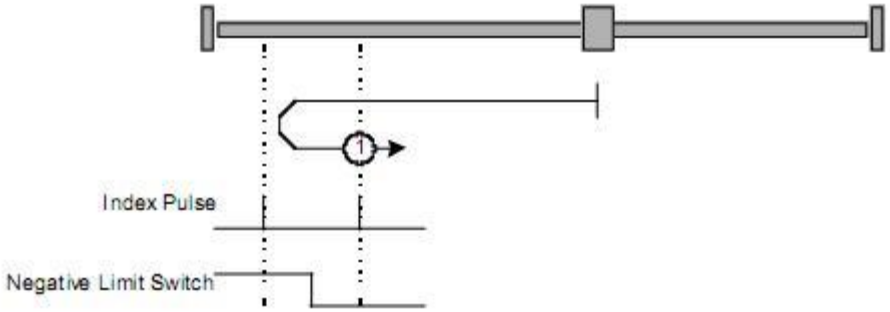


圖 10.1

#2. 以正極限與索引脈波信息之原點復歸模式

使用此方法，若正極限開關為無效(為低電位)時，初始方向為向右進行位移運動。此原點位置為當觸碰正極限開關時即往左方之索引脈衝位移。

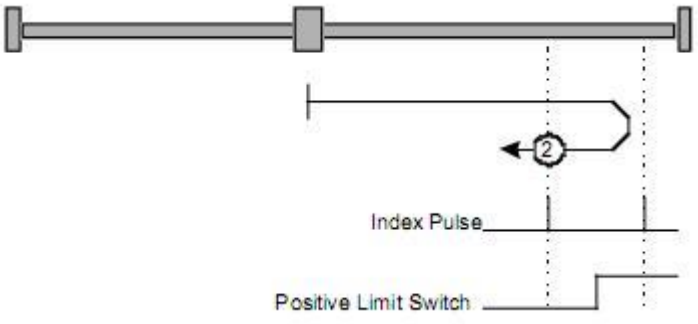


圖 10.2

DELTA CONFIDENTIAL

#3 and 4. 以原點開關正向電位與索引脈波信息之原復歸模式

使用方法 3 或方法 4，其初始位移的方向是依循原點開關當前的狀態。此原點位置取決於該原點開關改變狀態時，向左或向右任一方的索引脈衝。若進行原點復歸的初始位置為索引脈衝，將必須迴轉位移的方向。該點在迴轉位移方向後發生的任何變化，取決於原點開關當前的狀態。

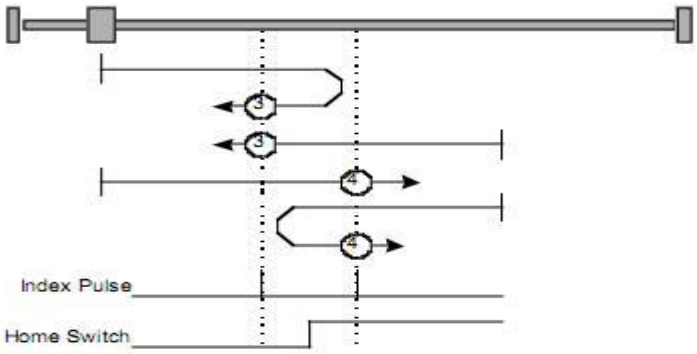


圖 10.3

#5 and 6. 以原點開關負向電位與索引脈波信息之原復歸模式

使用方法 5 或方法 6，其初始位移的方向是依循原點開關當前的狀態。此原點位置取決於該原點開關改變狀態時，向左或向右任一方的索引脈衝。若進行原點復歸的初始位置為索引脈衝，將必須迴轉位移的方向。該點在迴轉位移方向後發生的任何變化，取決於原點開關當前的狀態。

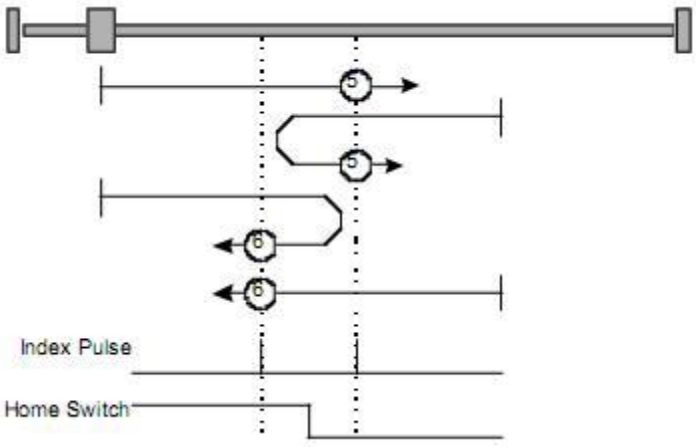


圖 10.4

DELTA CONFIDENTIAL

#7 to 14.基於索引脈波與原點開關之原點復歸模式

這些方法使用一個僅在位移中的一部分啟用的原點開關。事實上，在該軸 posistion 通過此開關，其可'瞬間'被啟用。

使用方法 7 到方法 10，其初始位移的方向是向右，使用方法 11 到方法 14，其初始位移的方向是向左。此外， 若原點開關在位移起始處已為啟用狀態。此時其初始位移的方向是依找尋到的邊緣(edge)而定。此歸回原點的位置為在正緣或負緣任一方的原點開關上的索引脈衝信號。於下方兩張圖顯示，若初始位移方向並未觸碰到原點開關時，此作動需觸碰到極限開關時，方才迴轉位移方向。

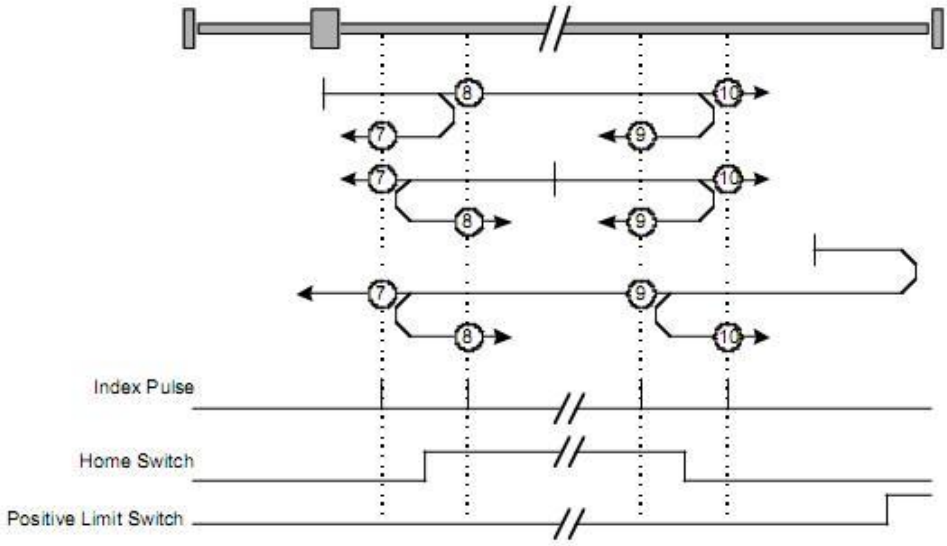


圖 10.5

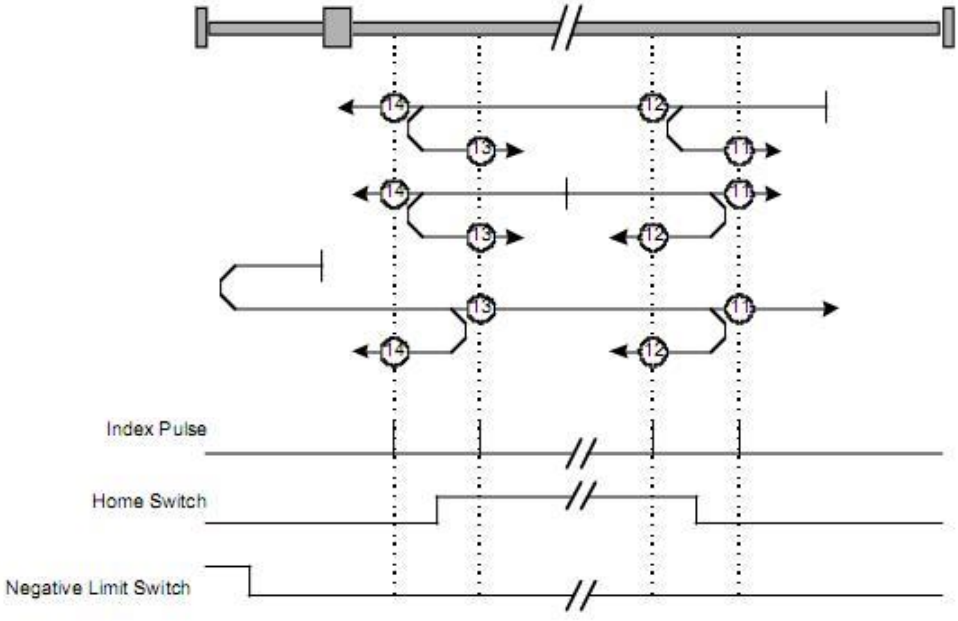


圖 10.6

#15 和 16. 保留

此為保留給原點復歸模式未來擴充使用。

#17 to 30. 與索引脈波無關之原點復歸模式

這些方法類似方法 1 到 14，兩造的區別在於此類歸回原點位置不需依賴索引脈衝。其僅依賴與原點或極限開關的切換來進行原復歸。例：方法 19 與方法 20 類似方法 3 和方法 4 之原點復歸模式，如下圖 11.7 所示。

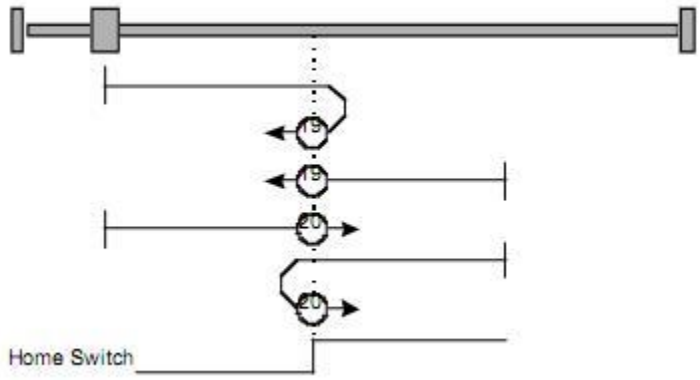


圖 10.7

#31 和 32. 保留

此為保留給原點復歸模式未來擴充使用。

#33 and 34. 與索引脈波相關之原點復歸模式

使用方法 33 或方法 34，其歸回原點正向或負向的區別，在於其歸原回原點選擇的方向位置取決於查找到的索引脈衝。

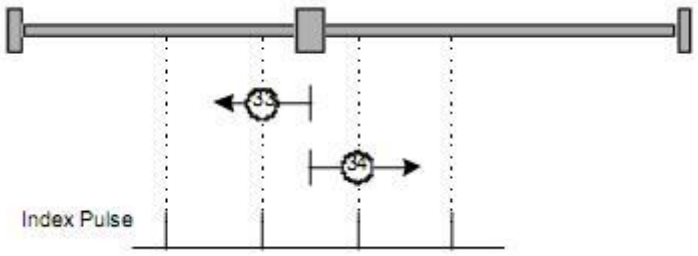


圖 10.8

#35. 依當前 positin 位置進行原點復歸的模式

模式 35 是將目前位置作為原點復歸的位置。

DELTA CONFIDENTIAL

10.2 _DMC_01_set_home_move

■ **格式**

I16 PASCAL _DMC_01_set_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

設置模組開始運行原點復歸運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;

I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

NOTE

※如何判斷回 Home 已經完成:

在下達 _DMC_01_set_home_move 後，可運用 _DMC_01_motion_done 的 MC_Status =0 (代表回 Home 動作已停止)且 _DMC_01_motion_status 回傳的 MC_status 之 MDS0=1、Target=1、Mode1=1,Mode2=1(回 Home 模式)等條件來判斷回 Home 是否已經完成，詳細說明可參考 15.1 及 15.2 章節說明。

DELTA CONFIDENTIAL

10.3　_DMC_01_set_home_config2

■　**格式**

I16 PASCAL _DMC_01_set_home_config2 (U16 CardNo,U16 NodeID,U16 SlotID ,
U16 Mode,I32 offset,U16 lowSpeed,U16 highSpeed,F64 acc)

■　**目的**

設置原點復歸各項配置(回 Home 前，請關閉軟體極限設定)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考後頁 note 說明)
offset	I32	脈波數	原點復歸偏移量
lowSpeed	U16	轉速/分	歸回零所使用的速度參數
highSpeed	U16	轉速/分	尋找各極限位置所使用的速度參數
Acc	F64	秒	原點復歸參照的加速時間

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　範例

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
I16 status = _DMC_01_set_home_config2 (CardNo, NodeID, SlotID, Mode, offset, lowSpeed,
        highSpeed, acc);
```

NOTE

此 API 與_DMC01_set_home_config 的不同在於，_DMC_01_set_home_config2 的命令是直接下到 Server，_DMC_01_set_home_config 是下到 Memory 他會等_DMC01_set_home_move 的指令下達後，一起進到 Server 並執行所以使用 _DMC_01_set_home_config2 與 _DMC_01_set_home_move2 的搭配，可以省去一些時間；但請注意，_DMC_01_set_home_config 必須搭配 _DMC_01_set_home_move 而 _DMC_01_set_home_config2 必須搭配_DMC_01_set_home_move2，如果混搭，將會發生錯亂現象。

DELTA CONFIDENTIAL

10.4 _DMC_01_set_home_config_offset2

■ **格式**

I16 PASCAL _DMC_01_set_home_config_offset2 (U16 CardNo,U16 NodeID,U16 SlotID , ,I32 offset)

■ **目的**

重新設置原點復歸 offset 值(回 Home 前，請關閉軟體極限設定)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
offset	I32	脈波數	原點復歸偏移量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;

I16 status = _DMC_01_set_home_config_offset2 (CardNo, NodeID, SlotID, offset);
```

NOTE

如果在下達_DMC_01_set_home_config2 的命令後，在_DMC_01_set_home_move2 指令未下達前，有執行到_DMC_01_set_position 這個指令，那麼之前下達的 offset 值會被清除，必須在執行_DMC_01_set_home_move2 之前重新使用_DMC_01_set_home_config_offset2 這個 API 給 offset 值。

DELTA CONFIDENTIAL

10.5 _DMC_01_set_home_move2

■ **格式**

I16 PASCAL _DMC_01_set_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

設置開始運行原點復歸運動。說明

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考 10.1 節 note 說明)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 , Mode=1;

I16 status = _DMC_01_set_home_move2 (CardNo, NodeID, SlotID, Mode);

DELTA CONFIDENTIAL

10.6 _DMC_01_escape_home_move

■ **格式**

I16 PASCAL _DMC_01_escape_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

停止原點復歸運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;

I16 status = _DMC_01_escape_home_move (CardNo, NodeID, SlotID);
```

DELTA CONFIDENTIAL

10.7　_DMC_01_set_home_config_by_torque_limit

■ **格式**

I16 PASCAL _DMC_01_set_home_config_by_torque_limit(U16 CardNo,U16 NodeID,U16 SlotID ,U16 Mode,I32 offset,U16 lowSpeed,U16 highSpeed,F64 acc,U16 torque_ratio,U16 qualified_ms,U16 max_torque_ratio)

■ **目的**

DMC 模式底下，開始運行原點復歸運動，配合 Torque_limit。(SDO 封包傳送，下達後需等待一段時間，依照客戶使用情況不一)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	編號單位	模式切換。(模式 36~39 為 DMC Torque 特規使用)
offset	I32	數值	配合模式 37 與 39 設定回 home 偏移量。
lowSpeed	U16	數值	初速
highSpeed	U16	數值	最大速
acc	F64	數值	加速度
torque_ratio	U16	數值	扭矩設定(%)
qualified_ms	U16	數值	反應時間(最大 2s)
max_torque_ratio	U16	數值	最大扭矩限制 = 扭矩設定+最大扭矩限制(%)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 Mode = 36;
I32 offset = 0;
U16 lowSpeed = 0;
U16 highSpeed = 128000;
F64 acc = 0;
```

DELTA CONFIDENTIAL

```
U16 torque_ratio = 10;          //10%
U16 qualified_ms = 2000;        //2s
U16 max_torque_ratio = 1;       //1%   ,10+1 =11%
```

I16 status = _DMC_01_set_home_config_by_torque_limit(CardNo, NodeID, SlotID , Mode, offset, lowSpeed, highSpeed, acc, torque_ratio, qualified_ms, max_torque_ratio);

status = _DMC_01_set_home_move(l_nCardNo, NodeID, SlotID);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_NoError	Rt = 0，執行正常。
ERR_SdoData	Rt = 28，SDO Data 送出但無回應。

NOTE

此 API 對應使用的 home_config 為 _DMC_01_set_home_move ，
現下並無支援 _DMC_01_set_home_ move2 使用上必須注意。
另外 DMC 針對 Torque 回 home 新增額外 4 種回 home 模式(**台達伺服驅動器 A2F、A2M 限定**)，描述如下:

Home Mode 36: 正轉扭力到達反轉。
Home Mode 37: 正轉扭力到達反轉後，走到 Offset。
Home Mode 38: 反轉扭力到達正轉。
Home Mode 39: 反轉扭力到達正轉後，走到 Offset。

搭配的台達驅動器版本為:1746 -307。

DELTA CONFIDENTIAL

第十一章　速度運動控制使用封包協定 API

此章節說明了如何使用速度運動應用程式介面(API)，控制伺服驅動器速度運動的相關參數配置，以及取得速度模式(PV)下扭矩的設定限制。

表 11.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_velocity_mode	速度運動控制參數配置
A/B/F	_DMC_01_set_velocity	設置開始運行速度運動
A/B/F	_DMC_01_set_velocity_stop	停止速度運動控制
A/B/F	_DMC_01_get_rpm	取回目前運動的 rpm 值，實際轉速為此變數(rpm)值的 1/10
A/B/F	_DMC_01_set_velocity_torque_limit	在速度模式時設定最大扭矩限制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

11.1 _DMC_01_set_velocity_mode

■ **格式**

I16 PASCAL _DMC_01_set_velocity_mode (U16 CardNo, U16 NodeID, U16 SlotID,
F64 Tacc,F64 Tdec)

■ **目的**

針對伺服驅動器的速度運動參數設定。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
F64 Tacc=0.1, Tdec=0.1;

/* 設定速度模式參數(加速時間與減速時間的值) */

I16 status = _DMC_01_set_velocity_mode (CardNo, NodeID, SlotID, Tacc, Tdec);

DELTA CONFIDENTIAL

11.2 _DMC_01_set_velocity

■ **格式**

I16 PASCAL _DMC_01_set_velocity (U16 CardNo, U16 NodeID, U16 SlotID,I32 rpm)

■ **目的**

設置開始運行速度運動的速度比例，實際轉速為此變數(rpm)值的 1/10。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I32	數值	實際轉速為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I32 rpm =-1000; //此例速度運動將會是反轉運動，該實際轉速的值為|-1000| / 10 => 100RPM

I16 status = _DMC_01_set_velocity (CardNo, NodeID, SlotID, rpm);
```

DELTA CONFIDENTIAL

11.3 _DMC_01_set_velocity_stop

■ 格式

I16 PASCAL _DMC_01_set_velocity_stop (U16 CardNo,U16 NodeID,U16 SlotID,U16 stop)

■ 目的

設定停止速度運動的停止模式。

■ 參數

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
stop	U16	選項單位	0：速度運動操作保持目前的運動狀態 1：停止速度運動

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ 範例

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 stop=1;

I16 rpm=2000; //  此例速度運動將會是正轉運動，該實際轉速的值為|2000| / 10 => 200RPM


/*  啟動速度運動的操作  */

I16 status = _DMC_01_set_velocity (CardNo, NodeID, SlotID, rpm);
```

DELTA CONFIDENTIAL

```
/* 停止目前運行的速度運動 */
status = _DMC_01_set_velocity_stop (CardNo, NodeID, SlotID, stop);
```

DELTA CONFIDENTIAL

11.4 _DMC_01_get_rpm

■ **格式**

I16 PASCAL _DMC_01_get_rpm (U16 CardNo, U16 NodeID, U16 SlotID, I16 *rpm)

■ **目的**

取回目前運動的 rpm 值，實際轉速為此變數(rpm)值的 1/10。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I16	數值	實際轉速為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 rpm;

I16 status = _DMC_01_get_rpm (CardNo, NodeID, SlotID, &rpm);
```

DELTA CONFIDENTIAL

11.5 _DMC_01_set_velocity_torque_limit

■ **格式**

I16 PASCAL _DMC_01_set_velocity_torque_limit (U16 CardNo, U16 NodeID, U16 SlotID, U32 torque_limit)

■ **目的**

在 velocity 模式時設定最大 torque 限制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
torque_limit	U32	數值	轉矩最大限制量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 torque_limit =50;//ratio 值為 50, 50 除以 1000，表示額定扭力值為 5%

I16 status = _DMC_01_set_velocity_torque_limit (CardNo, NodeID, SlotID, torque_limit);

DELTA CONFIDENTIAL

第十二章　轉矩運動控制使用封包協定 API

此章節說明了如何使用轉矩運動應用程式介面(API)，控制伺服轉矩運動的相關參數配置，配合轉矩的速度設定限制。

表 12.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_torque_mode	扭矩控制參數配置
A/B/F	_DMC_01_set_torque	設置開始運行轉矩運動
A/B/F	_DMC_01_set_torque_stop	停止扭矩模式
A/B/F	_DMC_01_set_torque_velocity_limit	在扭矩模式時設定最大速度限制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

12.1 _DMC_01_set_torque_mode

■ **格式**

I16 PASCAL _DMC_01_set_torque_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 slope)

■ **目的**

轉矩運動控制的轉矩運轉時間參數設定(slope 的值)。必須在多少轉矩時間內達到額定扭力。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
slope	U16	毫秒	0 至 100%額定扭力所需時間

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ 範例

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 slope=500; // 500 毫秒

I16 status = _DMC_01_set_torque_mode (CardNo, NodeID, SlotID, slope);
```

DELTA CONFIDENTIAL

12.2 _DMC_01_set_torque

■ **格式**

I16 PASCAL _DMC_01_set_torque (U16 CardNo, U16 NodeID, U16 SlotID, I16 ratio)

■ **目的**

設置轉矩運動的運轉方向與扭力值千分比。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	Number Uint	插槽 ID
ratio	I16	數值	額定扭力值千分比(值小於 0 反轉，大於 0 正轉)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 ratio=-50; // ratio 值為-50, -50 除以 1000，表示額定扭力值為 5%且馬達反轉

/* 設置 ratio 值並開始進行轉矩運動 */
I16 status =_DMC_01_set_torque(CardNo, NodeID, SlotID, ratio);
// 若 ratio 的值小於 0 時，馬達反轉；值大於 0 時，馬達正轉。
```

DELTA CONFIDENTIAL

12.3 _DMC_01_set_torque_stop

■ **格式**

I16 PASCAL _DMC_01_set_torque_stop (U16 CardNo, U16 NodeID, U16 SlotID, U16 stop)

■ **目的**

設定停止轉矩運動的停止模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
stop	U16	選項單位	0：等待當前運動結束後，停止目前的運動狀態。 1：立即停止轉矩運動。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 stop=1;
I16 ratio=100; // ratio 值為 100，100 除以 1000，表示額定扭力值為 10%且馬達正轉

/*設置 ratio 值並開始進行轉矩運動 */
I16 status =_DMC_01_set_torque(CardNo, NodeID, SlotID, ratio);
/* 停止轉矩運動 */
status = _DMC_01_set_torque_stop(CardNo, NodeID, SlotID, stop);
```

DELTA CONFIDENTIAL

12.4 _DMC_01_set_torque_velocity_limit

■ **格式**

I16 PASCAL _DMC_01_set_torque_velocity_limit (U16 CardNo, U16 NodeID, U16 SlotID, U32 velocity_limit)

■ **目的**

在扭矩模式時設定最大的速度限制。最大不得大於 30000 。
(實際轉速為此變數(rpm)值的 1/10)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
velocity_limit	U32	數值	速度最大限制量(rpm)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 velocity_limit =2000;

/*  設置 Velocity 最大限制量, velocity_limit 的值不得大於 30000 */
 I16 status =_DMC_01_set_torque_velocity_limit (CardNo, NodeID, SlotID, velocity_limit);
```

DELTA CONFIDENTIAL

第十三章　使用 PDO 協定 API

此章節說明了如何使用 PDO 協定應用程式介面(API)，相關 PDO 命令如下表 13.1。使用 PDO 命令主要針對必須快速回應的狀態以及設定的 API 使用。

表 13.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_ipo_set_svon	設置 Servo ON/OFF
A/B/F	_DMC_01_get_buffer_length	取得未實行的運動命令行數
A/B/F	_DMC_01_command_buf_clear	清除間隔時間(buffer dwell)
A/B	_DMC_01_buf_dwell	設定兩個運動命令之間的間隔時間
A/B	_DMC_01_set_group	設定關連性群組
A/B/F	_DMC_01_set_dmc_mode	切換成 DMC_NET 模式

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

13.1 _DMC_01_ipo_set_svon

■ **格式**

I16 PASCAL _DMC_01_ipo_set_svon (U16 CardNo, U16 NodeID, U16 SlotID, U16 ON_OFF)

■ **目的**

在”PDO 協定”下伺服驅動器的啟動與關閉設定(所有模組的啟動與關閉都可以運用此 API 下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ON_OFF	U16	選項單位	0：Servo OFF 1：Servo ON

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 ON_OFF=1;

I16 status = _DMC_01_ipo_set_svon (CardNo, NodeID, SlotID, ON_OFF);
```

NOTE

ERROR Code = 128，為 sdo 下達造成 time out。請檢查配接線。

DELTA CONFIDENTIAL

13.2 _DMC_01_get_buffer_length

■ **格式**

I16 PASCAL _DMC_01_get_buffer_length (U16 CardNo, U16 NodeID, U16 SlotID,
U16* bufferLength)

■ **目的**

取得緩衝區內仍未執行的運動命令資料數量(緩衝區(Buffer)最大為 20 筆資料)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
bufferLength	U16*	個數	未實行的運動命令個數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 bufferLength;
I16 status;

/* 啟用同步運動命令 */

status = _DMC_01_sync_move(CardNo);

/* 取得各個站號中尚未執行的運動命令 */

status = _DMC_01_get_buffer_length (CardNo, NodeID, SlotID, &bufferLength);
```

DELTA CONFIDENTIAL

13.3 _DMC_01_buf_dwell

■ **格式**

I16 PASCAL _DMC_01_buf_dwell (U16 CardNo, U16 NodeID, U16 SlotID, I32 dwell_cnt)

■ **目的**

兩個運動命令之間的間隔時間。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
dwell_cnt	I32	次數	Dewell buffer 間隔 (延遲時間為 2 x dwell_cnt + 2)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0; U16 NodeID =1; U16 SlotID=0;

I32 dwell_cnt=3; //若 dwell_cnt 值為 1，延遲時間為 4ms；此例的值為 3，延遲時間為 2*3+2=8ms

I16 status =_DMC_01_buf_dwell (CardNo, NodeID, SlotID, dwell_cnt);

DELTA CONFIDENTIAL

13.4 _DMC_01_command_buf_clear

■ **格式**

I16 PASCAL _DMC_01_command_buf_clear (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

清除間隔時間(緩衝區 dwell(bufferdwell) 計數值)。注意:括號內為專有名詞

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0; U16 NodeID =1; U16 SlotID=0; I32 dwell_cnt=3;

//已經設定了間隔時間。

I16 status = _DMC_01_buf_dwell(CardNo, NodeID, SlotID, dwell_cnt); //設定 dwell buffer 間隔

//清除_DMC_01_buf_dwell 所指定的時間。

status = _DMC_01_command_buf_clear (CardNo, NodeID,SlotID); //清除 dweel buffer 間隔

DELTA CONFIDENTIAL

13.5 _DMC_01_set_group

■ **格式**

I16 PASCAL _DMC_01_set_group (U16 CardNo, U16* NodeID,U16* SlotID,
U16 NodeID_Num, U16 enable)

■ **目的**

設定總線內伺服軸的關連性稱為群組(Group)(群組中若有任一軸發生重大問題而緊急停止時，該群組內所有站號將一同緊急停止)。群組的相關設定限制請參考下方 NOTE。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NodeID_Num	U16	數值單位	群組站數
Enable	U16	編號單位	0：解除群組設定。 1：設定為群組。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID[4] ={1,3,5};
U16 SlotID[4]={0,0,0};

U16 NodeID_Num=3; //設定為同一群組個數為 3 個。

U16 enable=1;

I16 status =_DMC_01_set_group (CardNo, NodeID, SlotID, NodeID_Num, enable);
```

NOTE

1. 預設值為所有站號為同一個 Group。

DELTA CONFIDENTIAL

2. 例如:一但設定 2、3、4 為同一個 Group，則其餘站號都視為獨立個體。

3. 所有站號最多可設定 6 個 Group。

4. 同一站號不要在一個以上的 Group 中設定。

5. 通訊斷線時，只有群組內的 Node 會全部停止

6. 若無設置群組時，任一軸通訊斷線則所有 Node 停止。

7. set_group 判斷條件只有 AL013 緊急停止會讓群組內的 Node 會全部停止，極限開關 AL014 AL015 不在判斷條件中。

8. 目前沒有任一軸發生重大問題的定義。

DELTA CONFIDENTIAL

13.6 _DMC_01_set_dmc_mode

■ **格式**

I16 PASCAL _DMC_01_set_dmc_mode (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

將模式切成 DMC_NET 模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
I16 status;

I16 status = _DMC_01_set_dmc_mode (CardNo, NodeID, SlotID);
```

DELTA CONFIDENTIAL

第十四章　停止運動控制 API

此章節說明了如何使用停止運動應用程式介面(API)控制運動結束時，設定停止的減速狀態、SLD 模式的設定、以及調整減速時間。

表 14.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_emg_stop	緊急停止(立即停止執行 Buffer 內所有運動控制指令)
A/B/F	_DMC_01_sd_stop	減速停止(立即停止執行 Buffer 內所有運動指令，馬達減速停止)
A/B/F	_DMC_01_sd_abort	中斷當前正在執行的運動指令，減速時間停止後，繼續執行 buffer 內運動控制指令
A/B/F	_DMC_01_set_sld_mode	設定 SLD 運動模式(SLD 為外部訊號觸發的減速停止模式)。
A/B/F	_DMC_01_set_sld_stop	啟動/關閉 SLD 模式設定
A/B/F	_DMC_01_set_sd_time	設定減速時間。
A/B/F	_DMC_01_set_scurve_rate	設定加、減速段，S_Curve 模式的百分比。

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

14.1 _DMC_01_emg_stop

■ **格式**

I16 PASCAL _DMC_01_emg_stop (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

目前執行的運動指令將立即停止，且緩衝區(Buffer)內的所有未執行的等待指令將被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0;U16 NodeID =1; U16 SlotID=0;

I16 status = _DMC_01_emg_stop (CardNo, NodeID, SlotID);

DELTA CONFIDENTIAL

14.2 _DMC_01_sd_stop

■ **格式**

I16 PASCAL _DMC_01_sd_stop (U16 CardNo, U16 NodeID,U16 SlotID,F64 Tdec)

■ **目的**

目前執行的運動指令將依減速時間減速停止，且緩衝區(Buffer)內所有未執行的等待指令將被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 Tdec=0.1;

I16 status = _DMC_01_sd_stop (CardNo, NodeID, SlotID, Tdec);

DELTA CONFIDENTIAL

14.3 _DMC_01_sd_abort

■ **格式**

I16 PASCAL _DMC_01_sd_abort (U16 CardNo, U16 NodeID, U16 SlotID, F64 Tdec)

■ **目的**

當前正在執行的運動指令將依減速時間停止，但緩衝區(Buffer)內所有未執行的等待指令不會被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 Tdec=0.1;

I16 status = _DMC_01_sd_abort (CardNo, NodeID, SlotID, Tdec);

DELTA CONFIDENTIAL

14.4 _DMC_01_set_sld_mode

■ **格式**

I16 PASCAL _DMC_01_set_sld_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

設定 SLD 訊號觸發的運動模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	SLD 模式設定。 0、1：緊急停止。 2：依設定之減速時間停止(DMC_01_set_sd_time) 3：變速度(Change to start Velocity)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, U16 NodeID =1, SlotID=0;
U16 mode=1;  //緊急停止。

I16 status = _DMC_01_set_sld_mode (CardNo, NodeID, SlotID, mode);
```

NOTE

伺服驅動器 A2F 機種，SLD 腳位為 DI3，對應參數為 P2-12。

DELTA CONFIDENTIAL

14.5 _DMC_01_set_sld_stop

■ **格式**

I16 PASCAL _DMC_01_set_sld_stop (U16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■ **目的**

啟動/關閉 SLD 模式設定。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	U16	選項單位	0：關閉 SLD 模式。 1：啟動 SLD 模式。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
U16 enable=1;　　//啟動 SLD 模式。

I16 status = _DMC_01_set_sld_stop (CardNo, NodeID, SlotID, enable);

DELTA CONFIDENTIAL

14.6 _DMC_01_set_sd_time

■ **格式**

I16 PASCAL _DMC_01_set_sd_time (U16 CardNo, U16 NodeID, U16 SlotID, F64 sd_dec)

■ **目的**

設定減速時間。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
sd_dec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 sd_dec=2

I16 status = _DMC_01_set_sd_time (CardNo, NodeID, SlotID, sd_dec);

DELTA CONFIDENTIAL

14.7　_DMC_01_set_scurve_rate

■　**格式**

I16 PASCAL _DMC_01_set_scurve_rate(U16 CardNo, U16 NodeID, U16 SlotID, U16 scurve_rate)

■　**目的**

設定加、減速段，S_Curve 模式的百分比。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
scurve_rate	U16	數值單位	以加、減速段中心點，往前/往後 T_Curve 模式的百分比(0~100)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo = 0;
U16 NodeID =1, SlotID = 0;
U16 scurve_rate = 60;
I16 status = _DMC_01_set_scurve_rate (CardNo, NodeID, SlotID, scurve_rate);
```

NOTE

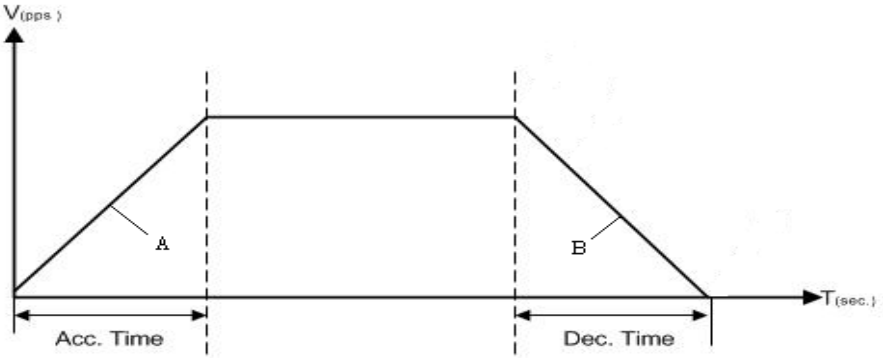


圖 13.3scurve_rate 設定

DELTA CONFIDENTIAL

14.8　_DMC_01_set_dio_input_level_emg

■　**格式**

I16 PASCAL _DMC_01_set_dio_input_level_emg (U16 CardNo, U16 Enable)

■　**目的**

啟動/關閉軸卡 DI 觸發緊急停止功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Enable	U16	選項單位	0：關閉功能。 1：啟動功能。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●	●	●	●				

■　**範例**

```
U16 CardNo = 0;
U16 Enable =1
I16 status =_DMC_01_set_dio_input_level_emg(CardNo, Enable);
```

DELTA CONFIDENTIAL

14.9　_DMC_01_set_dio_input_trigger_level

■　**格式**

I16 PASCAL _DMC_01_set_dio_input_trigger_level(U16 CardNo, U16 level_mode)

■　**目的**

設定 DI level 觸發準位

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
level_mode	U16	選項單位	0：Low 1：High

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●	●	●	●				

■　**範例**

U16 CardNo = 0;
U16 level_mode =1;(選擇 High 狀態)
I16 status = _DMC_01_set_dio_input_trigger_level (CardNo, level_mode);

DELTA CONFIDENTIAL

14.10　_DMC_01_set_dio_input_level_emg_clear

■ **格式**

I16 PASCAL _DMC_01_set_dio_input_level_emg_clear(U16 CardNo)

■ **目的**

解除 EMG 狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●	●	●	●				

■ **範例**

```
U16 CardNo = 0;
_DMC_01_set_dio_input_level_emg_clear( CardNo);
```

NOTE

假設選擇 High 狀態觸發緊急停止，觸發後若要解除狀態，需先將 DI 狀態復原至 Low 狀態，使用_DMC_01_set_dio_input_level_emg_clear 才能解除 EMG 狀態。

DELTA CONFIDENTIAL

14.11 _DMC_01_get_dio_input_level_emg_status

■ **格式**

I16 PASCAL _DMC_01_get_dio_input_level_emg_status(U16 CardNo, U16* status)

■ **目的**

取得 DI 觸發緊急停止功能狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
status	U16*	數值單位	(只有 bit0 ~ bit2 數值有定義) bit0: 0:緊急停止狀態解除 1:目前為緊急停止狀態 bit1: 0: 設定 DI level 觸發準位為 Low 1: 設定 DI level 觸發準位為 High bit2: 0: DI 觸發緊急停止功能為關閉狀態 1: DI 觸發緊急停止功能為啟動狀態

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo = 0;
U16* status;
I16 status = _DMC_01_get_dio_input_level_emg_status(CardNo, &status)
```

NOTE

假設取得狀態值為 7，二進制為 0111，代表

bit0=1，目前為緊急停止狀態。

bit1=1，_DMC_01_set_dio_input_trigger_level 設定的 DI level 觸發準位為 High。

bit2=1，DI 觸發緊急停止功能為啟動狀態。

DELTA CONFIDENTIAL

第十五章　運動狀態 API

此章節說明了如何使用運動狀態應用程式介面(API)，取得模組與伺服運動執行階段的運動訊息與狀態回授。

表 15.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_motion_done	傳回軸卡目前實行的命令執行狀態
A/B/F	_DMC_01_motion_status	傳回從站目前運動狀態
A/B/F	_DMC_01_set_01_ph_inposition	設定 01PH 到位訊號的區間範圍
A/B/F	_DMC_01_get_01_ph_inposition	取得目前 01PH 到位訊號設定的區間範圍

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

15.1 _DMC_01_motion_done

■ **格式**

I16 PASCAL _DMC_01_motion_done (U16 CardNo, U16 NodeID, U16 SlotID, U16* MC_status)

■ **目的**

傳回軸卡目前執行的運動階段。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MoSt	U16*	選項單位	0：運動位移動作停止 1：依加速時間進行運動位移 2：依最大速度進行運動位移 3：依減速時間進行運動位移 5：指令在 FIFO，尚未進入 Buffer(伺服或 04PI Md2) 6：Buffer 有運動指令(伺服或 04PI Md2) 12：回 Home 狀態中(伺服或 04PI Md2) 21：PC 與 DSP 的 Counter 沒對準(04PI Md1) 22：FIFO 中有指令(04PI Md1) 23：Buffer Full(04PI Md1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 MoSt;

I16 status = _DMC_01_motion_done (CardNo, NodeID, SlotID, &MoSt);
```

DELTA CONFIDENTIAL

NOTE

_DMC_01_motion_done 為軸卡運動狀態，並非實際伺服狀態，伺服狀態請參閱 _DMC_01_motion_status。

DELTA CONFIDENTIAL

15.2 _DMC_01_motion_status

■ **格式**

I16 PASCAL _DMC_01_motion_status (U16 CardNo, U16 NodeID, U16 SlotID, U32* MC_status)

■ **目的**

傳回軸卡目前的運動狀態。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>插槽 ID</td></tr>
<tr><td>MoSt</td><td>U32*</td><td>選項單位</td><td>運動狀態(bit0~bit15)
<table>
<tr><th>位元</th><th>標誌</th><th>描述</th></tr>
<tr><td>0 ~ 3</td><td>Mode0 ~ Mode3</td><td>啟用選擇模式</td></tr>
<tr><td>4</td><td>DI3</td><td>DI3（SLD）狀態映射</td></tr>
<tr><td>5</td><td>WR</td><td>警報訊息</td></tr>
<tr><td>6</td><td>DR</td><td>資料錯誤訊息</td></tr>
<tr><td>7</td><td>TG</td><td>觸發模式位元</td></tr>
<tr><td>8</td><td>PWRON</td><td>馬達激磁啟動位元</td></tr>
<tr><td>9</td><td>DriverErr</td><td>驅動器錯誤位元</td></tr>
<tr><td>10</td><td>Target</td><td>抵達目標位元</td></tr>
<tr><td>11</td><td>N/A</td><td>運動方向,1:正向 0:負向</td></tr>
<tr><td>12</td><td>MDS0</td><td>Mode specific</td></tr>
<tr><td>13</td><td>MDS1</td><td>Mode specific</td></tr>
<tr><td>14</td><td>PEL</td><td>正極限位元</td></tr>
<tr><td>15</td><td>MEL</td><td>負極限位元</td></tr>
</table></td></tr>
</table>

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U32 MoSt=0;

I16 status = _DMC_01_motion_status (CardNo, NodeID, SlotID, &MoSt);
```

NOTE

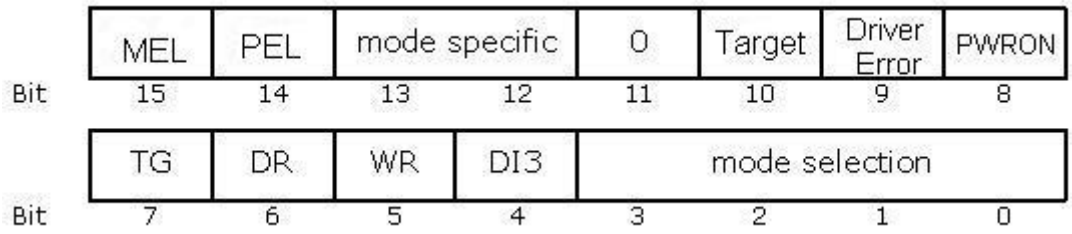


圖 15.1 運動狀態內容

關於「具體模式 (Mode specific)」位元 (位元 12 和位元 13) 相關敘述，請參考表 15.2。在模式選擇位元(位元 0~位元 3)中，僅有 2 種模式供使用者選擇，模式分別為 MODE1（位元 0 為 ON）和 MODE6（位元 1 與位元 2 為 ON）。

表 15.2 運動狀態的具體模式（Mode specific）位元敘述

模式項目	具體模式	
	位元 13 為 1	位元 12 為 1
原點復歸模式（MODE6）	復歸動作有誤	可實行復歸動作
DMCNET 模式（MODE1）	無定義	模式啟用

表 15.3 運動狀態的 Mode Selection 敘述

位元	0	1	2	3	模式說明
ON/OFF	ON	OFF	OFF	OFF	Profile Position mode
ON/OFF	ON	ON	OFF	OFF	Profile Velocity mode
ON/OFF	OFF	OFF	ON	OFF	Profile Torque mode
ON/OFF	OFF	ON	ON	OFF	Homing mode
ON/OFF	ON	ON	ON	ON	DMCNET mode

DELTA CONFIDENTIAL

15.3 _DMC_01_set_01_ph_inposition

■ **格式**

I16 PASCAL _DMC_01_set_01_ph_inposition (U16 CardNo, U16 NodeID,U16 SlotID , U16 poswindow_time,U32 poswindow)

■ **目的**

設定 01PH 模組 INP 到位訊號(單位:Pulse)的範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
poswindow_time	U16*	數值單位	設定比對次數，比對差值位於設定範圍幾次後，才將 Target 訊號變換
poswindow	U32	數值單位	設定 IN Position 訊號啟動的 Pulse 誤差量

■ **屬性**

軸卡	伺服	04PI Md1	GE01PH	MN	NT	DA&AD	RM64MN
			●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 poswindow_time=10;
U32 poswindow=100;

I16 status = _DMC_01_set_01_ph_inposition (CardNo, NodeID, SlotID, poswindow_time, poswindow);
```

DELTA CONFIDENTIAL

15.4 _DMC_01_get_01_ph_inposition

■ **格式**

I16 PASCAL _DMC_01_get_01_ph_inposition (U16 CardNo, U16 NodeID,U16 SlotID , U16* poswindow_time,U32* poswindow)

■ **目的**

取得目前 01PH 模組 INP 到位訊號(單位:Pulse)設定的範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
poswindow_time	U16*	數值單位	設定比對次數，比對差值位於設定範圍幾次後，才將 Target 訊號變換
poswindow	U32*	數值單位	設定 IN Position 訊號啟動的 Pulse 誤差量

■ **屬性**

軸卡	伺服	04PI Md1	GE01PH	MN	NT	DA&AD	RM64MN
			●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 poswindow_time;
U32 poswindow;

I16 status = _DMC_01_get_01_ph_inposition (CardNo, NodeID, SlotID, &poswindow_time,
&poswindow);
```

DELTA CONFIDENTIAL

第十六章　運動計數值 API

此章節說明了如何使用運動控制應用程式介面(API)，設定運動的命令(command)數值與目前位置(position)數值、取得運動的命令數值與目前位置數值，以及如何取得當前扭矩數值與速度的轉速設定。

表 16.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_command	取得軸卡 command 數值
A/B/F	_DMC_01_set_command	設置軸卡新的 command 數值
A/B/F	_DMC_01_get_position	取得從站模組目標位置數值
A/B/F	_DMC_01_set_position	設置從站模組目標位置數值，常用於執行清除 feedback
A/B/F	_DMC_01_get_target_pos	取得軸卡目的位置的設定值
A/B/F	_DMC_01_get_torque	取得馬達目前的轉矩數值(單位:電流百分比)
A/B/F	_DMC_01_get_current_speed	取得軸卡速度命令值(單軸分量)
A/B/F	_DMC_01_get_current_speed_rpm	取得馬達目前轉速 (單位:0.1RPM)

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

16.1 _DMC_01_get_command

■ **格式**

I16 PASCAL _DMC_01_get_command (U16 CardNo, U16 NodeID, U16 SlotID, I32* cmd)

■ **目的**

取得 command 計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
cmd	I32*	脈波數	目前 Command 的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;
I32 cmd = 0;

I16 status = _DMC_01_get_command (CardNo, NodeID, SlotID, &cmd);

DELTA CONFIDENTIAL

16.2 _DMC_01_set_command

■ **格式**

I16 PASCAL _DMC_01_set_command (U16 CardNo, U16 NodeID, U16 SlotID, I32 cmd)

■ **目的**

設置新的 command 計數值(下此指令之前應先執行 Set_Position 指令)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
cmd	I32	脈波數	欲設置的 Command 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 cmd=1000000;

I16 status = _DMC_01_set_command (CardNo, NodeID, SlotID, cmd);

DELTA CONFIDENTIAL

16.3 _DMC_01_get_position

■ **格式**

I16 PASCAL _DMC_01_get_position (U16 CardNo, U16 NodeID, U16 SlotID, I32* pos)

■ **目的**

取得目前 position 計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32*	脈波數	目前反饋的 position 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 pos;

I16 status = _DMC_01_get_position (CardNo, NodeID, SlotID, &pos);

DELTA CONFIDENTIAL

16.4 _DMC_01_set_position

■ **格式**

I16 PASCAL _DMC_01_set_position (U16 CardNo, U16 NodeID, U16 SlotID, I32 pos)

■ **目的**

設置新的 position 計數值(使用此 API 時，會切換為 DMC_NET 模式)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32	脈波數	設置 position 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 pos =500000;

I16 status = _DMC_01_set_position (CardNo, NodeID, SlotID, pos);

DELTA CONFIDENTIAL

16.5　_DMC_01_get_target_pos

■　**格式**

I16 PASCAL _DMC_01_get_target_pos (U16 CardNo, U16 NodeID, U16 SlotID, I32* pos)

■　**目的**

取得當前正在執行的運動指令之目標位置值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32*	脈波數	目的位置的 position 計數值(請參閱下方 note 說明)

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　範例

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 pos=0;

I16 status = _DMC_01_get_target_pos (CardNo, NodeID, &pos);
```

NOTE

※下達任一運動指令後，如果須依賴 _DMC_01_get_target_pos 指令的回傳值 pos 來判斷下一個步驟，請於下達運動指令後，延遲 10~100ms(此值依 CPU 性能與程式撰寫方式調整)後，再下達_DMC_01_get_target_pos 指令，以確保資料正確性。

DELTA CONFIDENTIAL

16.6 _DMC_01_get_torque

■ **格式**

I16 PASCAL _DMC_01_get_torque (U16 CardNo, U16 NodeID, U16 SlotID, U16* torque)

■ **目的**

取得傳回目前的轉矩計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
torque	U16*	數值	目前的轉矩值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 torque;

I16 status = _DMC_01_get_torque (CardNo, NodeID, SlotID, &torque);
```

DELTA CONFIDENTIAL

16.7　_DMC_01_get_current_speed

■　**格式**

I16 PASCAL _DMC_01_get_current_speed (U16 CardNo, U16 NodeID, U16 SlotID, I32* speed)

■　**目的**

取得目前的運動分量速度(只在 DMC_NET 模式有意義)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
speed	I32*	脈波數/秒	取得目前的運動分量速度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 speed;

I16 status = _DMC_01_get_current_speed (CardNo, NodeID, SlotID, &speed);
```

DELTA CONFIDENTIAL

16.8 _DMC_01_get_current_speed_rpm

■ **格式**

I16 PASCAL _DMC_01_get_current_speed_rpm (U16 CardNo, U16 NodeID, U16 SlotID, I32 *rpm)

■ **目的**

取得目前每分鐘的轉速乘上 10 倍的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I32*	轉速/秒	實際轉速值為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 rpm;

I16 status = _DMC_01_get_current_speed_rpm (CardNo, NodeID, SlotID, &rpm);
// 若您取得到的轉速值為 1000，實際轉速值為 100RPM.
```

DELTA CONFIDENTIAL

第十七章　軟體極限 API

此章節說明了如何使用軟體極限應用程式介面(API)，設定伺服軸的軟體極限狀態設定、關閉、停止方式。

表 17.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_soft_limit	設置軟體正/負極限的數值
A/B/F	_DMC_01_enable_soft_limit	設置開啟軟體極限和碰觸極限後的停止方式
A/B/F	_DMC_01_disable_soft_limit	關閉軟體極限
A/B/F	_DMC_01_get_soft_limit_status	取得運動期間軟體正/負極限的狀態

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

17.1 _DMC_01_set_soft_limit

■ **格式**

I16 PASCAL _DMC_01_set_soft_limit (U16 CardNo, U16 NodeID, U16 SlotID, I32 PLimit, I32 NLimit)

■ **目的**

設置軟體正/負極限的參考值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit	I32	數值	軟體正極限的值
NLimit	I32	數值	軟體負極限的值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
I32 PLimit = 8000;
I32 NLimit = -6000;

I16 status = _DMC_01_set_soft_limit (CardNo, NodeID, SlotID, PLimit, NLimit);
```

DELTA CONFIDENTIAL

17.2　_DMC_01_enable_soft_limit

■ **格式**

I16 PASCAL _DMC_01_enable_soft_limit (U16 CardNo, U16 NodeID, U16 SlotID, I16 Action)

■ **目的**

設置啟用極限開啟與否和碰觸極限後的停止方式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Action	I16	選項單位	1：碰觸極限後立即停止 2：碰觸極限後減速停止*1*2

*1. 減速時間可由 API _DMC_01_set_sd_time 進行設定。

*2. 在 DDA 模式僅顯示軟體正/負極限的狀態，所下達的位置命令仍會執行。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;

I16　Action=1; // 選擇碰觸極限後立即停止

I16 status = _DMC_01_enable_soft_limit (CardNo, NodeID, SlotID, Action);

DELTA CONFIDENTIAL

17.3　_DMC_01_disable_soft_limit

■ **格式**

I16 PASCAL _DMC_01_disable_soft_limit (U16 CardNo, U16 NodeID,U16 SlotID)

■ **目的**

關閉軟體極限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;

I16 status = _DMC_01_disable_soft_limit(CardNo, NodeID, SlotID);

DELTA CONFIDENTIAL

17.4　_DMC_01_get_soft_limit_status

■ **格式**

I16 PASCAL _DMC_01_get_soft_limit_status (U16 CardNo, U16 NodeID, U16 SlotID, U16* PLimit_sts,U16* NLimit_sts)

■ **目的**

取得運動期間軟體正/負極限的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit_sts	U16*	旗標	0：未碰觸到軟體正極限 1：已碰觸到軟體正極限
NLimit_sts	U16*	旗標	0：未碰觸到軟體負極限 1：已碰觸到軟體負極限

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 PLimit_sts, NLimit_sts;

I16 status = _DMC_01_get_soft_limit_status(CardNo, NodeID, SlotID, &PLimit_sts,
&NLimit_sts);
```

DELTA CONFIDENTIAL

第十八章　單軸運動控制 API

此章節說明了如何使用單軸運動應用程式介面(API)，控制單軸運動 T-Curve、S-Curve、位置參數配置、速度比率。

表 18.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move	相對運動，使用 T curve
A/B/F	_DMC_01_start_sr_move	相對運動，使用 S curve
A/B/F	_DMC_01_start_ta_move	絕對運動，使用 T curve
A/B/F	_DMC_01_start_sa_move	絕對運動，使用 S curve
A/B/F	_DMC_01_p_change	改變目標位置
A/B/F	_DMC_01_v_change	改變運動速度
A/B/F	_DMC_01_start_tr_move_2seg	兩段相對運動，使用 T curve 達成速度連續
A/B/F	_DMC_01_start_sr_move_2seg	兩段相對運動，使用 S curve 達成速度連續
A/B/F	_DMC_01_start_ta_move_2seg	兩段絕對運動，使用 T curve 達成速度連續
A/B/F	_DMC_01_start_sa_move_2seg	兩段絕對運動，使用 S curve 達成速度連續
A/B/F	_DMC_01_start_tr_move_2seg2	兩段相對運動，使用 T curve 達成速度連續
A/B/F	_DMC_01_start_sr_move_2seg2	兩段相對運動，使用 S curve 達成速度連續
A/B/F	_DMC_01_start_ta_move_2seg2	兩段絕對運動，使用 T curve 達成速度連續
A/B/F	_DMC_01_start_sa_move_2seg2	兩段絕對運動，使用 S curve 達成速度連續
A/B/F	_DMC_01_feedrate_overwrite	改變運動速度或速度比率
A/B/F	_DMC_01_start_v3_move	設定運動末速的單軸位移運動
A/B/F	_DMC_01_feedrate_overwrite_read	取得改變運動向量速度或向量速度比率設定值

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

18.1　_DMC_01_start_tr_move

■ **格式**

I16 PASCAL _DMC_01_start_tr_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行運動位移，請參閱敘述中圖 18.1 示意。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **敘述**

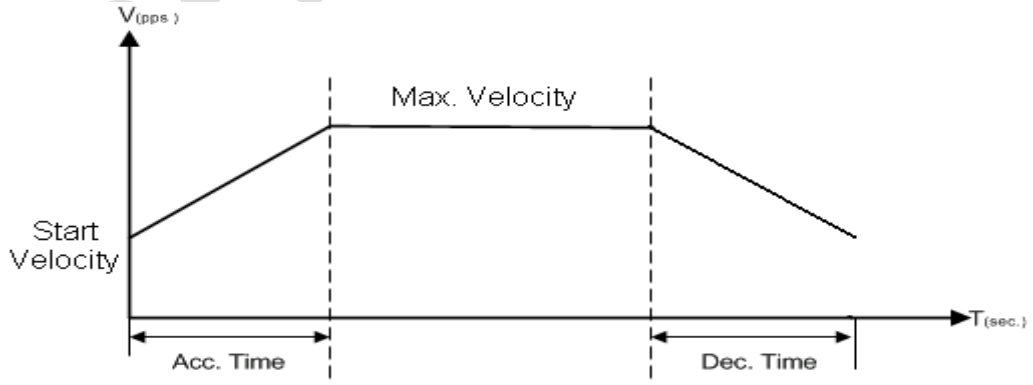


圖 18.1 T-curve 速度剖面下相對座標運動

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000, StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_tr_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

DELTA CONFIDENTIAL

18.2　_DMC_01_start_sr_move

■　**格式**

I16 PASCAL _DMC_01_start_sr_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行運動位移，請參閱敘述中圖 18.2 示意。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

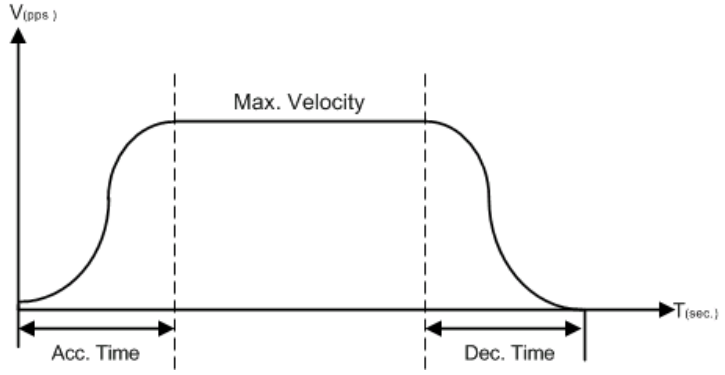


圖 18.2 S-curve 速度剖面下相對座標運動

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;

I32 Dist=100000, StrVel=0, MaxVel=30000;

F64 Tacc=0.1, Tdec=0.1;

16 status = _DMC_01_start_sr _move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

DELTA CONFIDENTIAL

18.3　_DMC_01_start_ta_move

■　**格式**

I16 PASCAL _DMC_01_start_ta_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

18.4　_DMC_01_start_sa_move

■　**格式**

I16 PASCAL _DMC_01_start_sa_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000;
I32 StrVel=0, MaxVel=30000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

18.5　_DMC_01_p_change

■ **格式**

I16 PASCAL _DMC_01_p_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewPos)

■ **目的**

以新的位置值替換目前的位置設置(p_change 只能對單軸運動指令下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewPos	I32	脈波數	欲替換的位置參數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0;U16 NodeID =1, SlotID=0;
I32 NewPos=100000;

I16 status = _DMC_01_p_change (CardNo, NodeID, SlotID, NewPos);

DELTA CONFIDENTIAL

18.6　_DMC_01_v_change

■ **格式**

I16 PASCAL _DMC_01_v_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewSpeed, F64 sec)

■ **目的**

以新的速度值設置替換目前的運動速度(v_change 不能在減速段下達即 motion done=3 時)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewSpeed	I32	脈波數/秒	欲替換速度參數
sec	F64	秒	速度轉換時，指定的加/減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0;U16 NodeID =1, SlotID=0;
I32 NewSpeed=3000;
F64 sec=0.1;
I16 status = _DMC_01_v_change (CardNo, NodeID, SlotID, NewSpeed, sec);

DELTA CONFIDENTIAL

NOTE

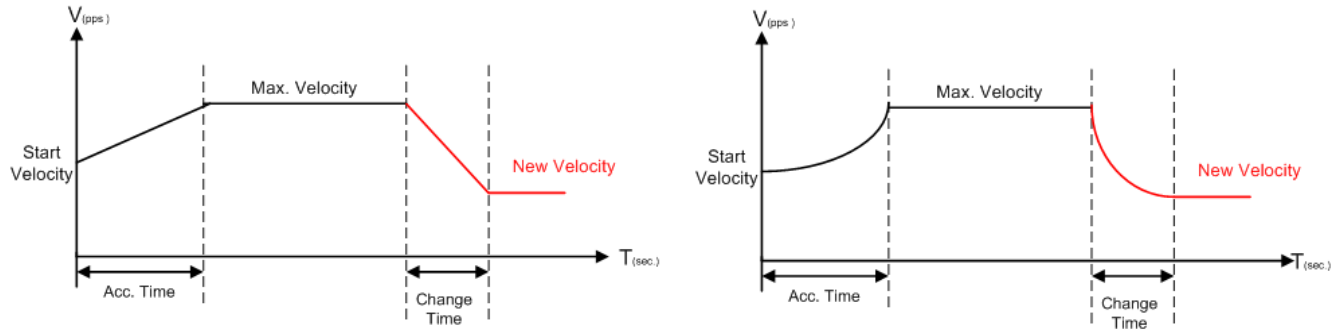


圖 18.3　更換新的速度參數

※在單軸及多軸運動下，都可以進行 V-change 的指令，多軸運動請對主控軸下達指令即可。

DELTA CONFIDENTIAL

18.7　_DMC_01_start_tr_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_tr_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行二段速度運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**描述**

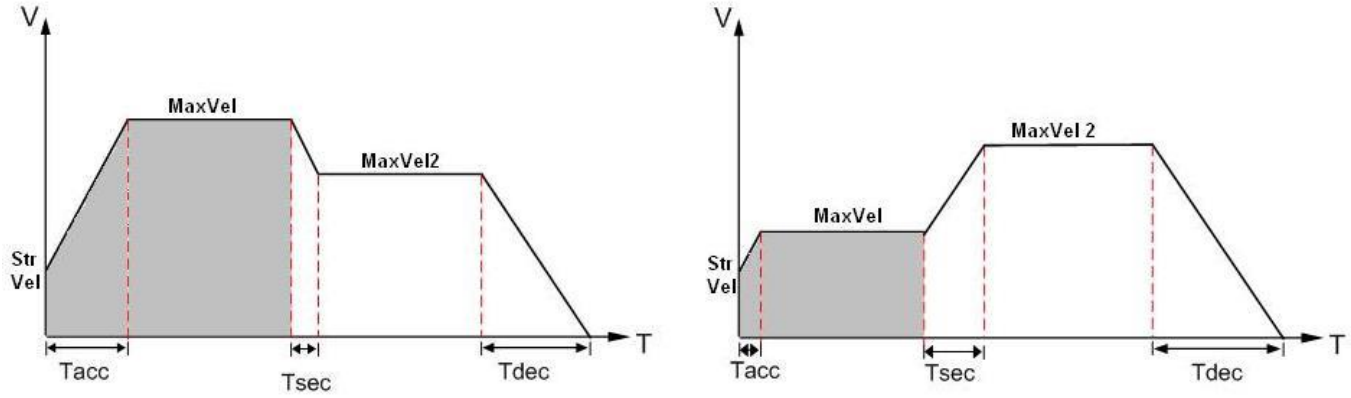


圖 18.4　T-curve 速度剖面下相對座標運動（灰色區塊表示為 Dist, 白色區塊則表示為 Dist2）

DELTA CONFIDENTIAL

■　**範例**

U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 Dist=500000;
I32 Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000;
I32 MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

NOTE

※請務必注意，此 API 參數中的 Dist 與 Dist2 必須「同向」，如圖 18.5 表示一個錯誤的設置。

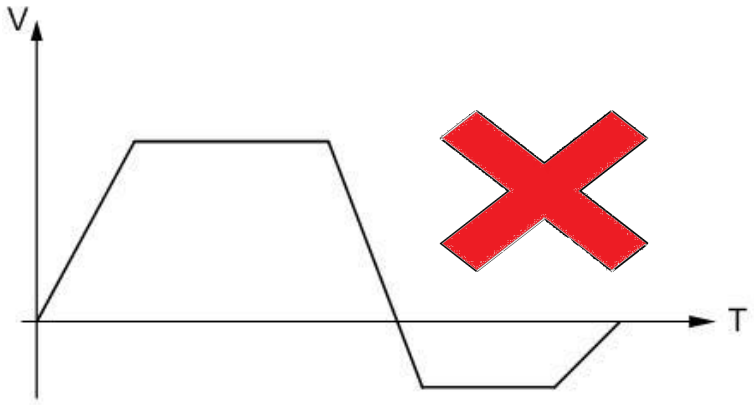


圖 18.5　錯誤圖例　-　Dist 與 Dist2 不同向

CONFIDENTIAL

18.8　_DMC_01_start_sr_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;

F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

DELTA CONFIDENTIAL

18.9　_DMC_01_start_ta_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_ta_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_ta_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

DELTA CONFIDENTIAL

18.10　_DMC_01_start_sa_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;

F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sa_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.11　_DMC_01_start_tr_move_2seg2

■ **格式**

I16 PASCAL _DMC_01_start_tr_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

NOTE

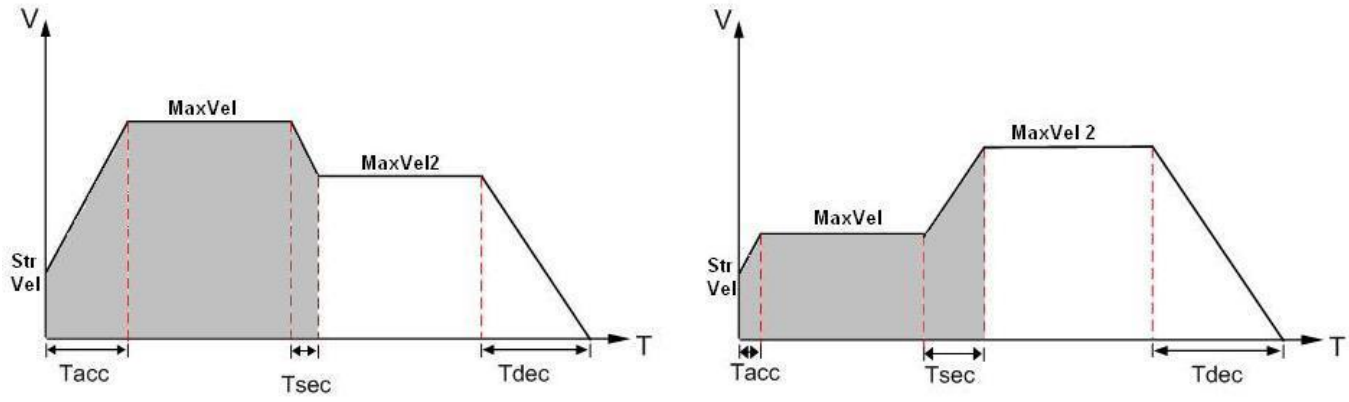


圖 18.5　T-curve 速度剖面下相對座標運動

（灰色區塊表示為 Dist, 白色區塊則表示為 Dist2）

DELTA CONFIDENTIAL

18.12　_DMC_01_start_sr_move_2seg2

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;

F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.13 _DMC_01_start_ta_move_2seg2

■ **格式**

I16 PASCAL _DMC_01_start_ta_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000

CONFIDENTIAL

```
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_ta_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
```

DELTA CONFIDENTIAL

18.14　_DMC_01_start_sa_move_2seg2

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;

F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_sa_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

DELTA CONFIDENTIAL

18.15　_DMC_01_feedrate_overwrite

■　**格式**

I16 PASCAL _DMC_01_feedrate_overwrite (U16 CardNo, U16 NodeID, U16 SlotID, U16 Mode, I32 New_Speed, F64 sec)

■　**目的**

改變運動向量速度或向量速度比率。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：與 V_Change 相同，在有運動執行下，改變當前運動的向量速度。 1：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度。 2：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度比例，範圍是 0% ~ 1000%。
NewSpeed	I32	脈波數/秒	Mode=0,1 時:欲替換速度參數 Mode=2 時:向量速度比例值範圍是 0% ~ 1000%
sec	F64	秒	速度轉換時，指定的加/減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　描述

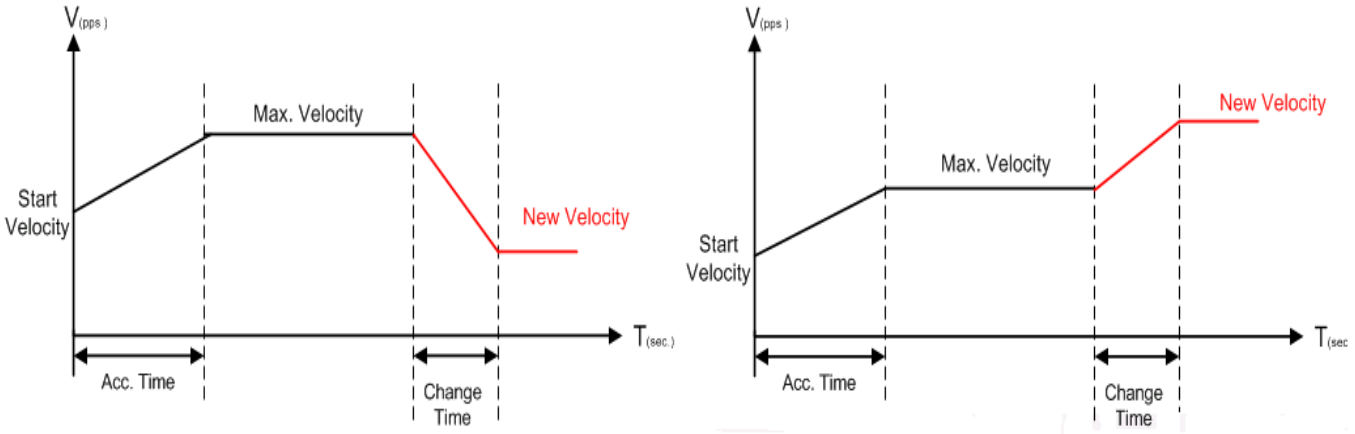


圖 18.7　Mode = 0 更換新的向量速度

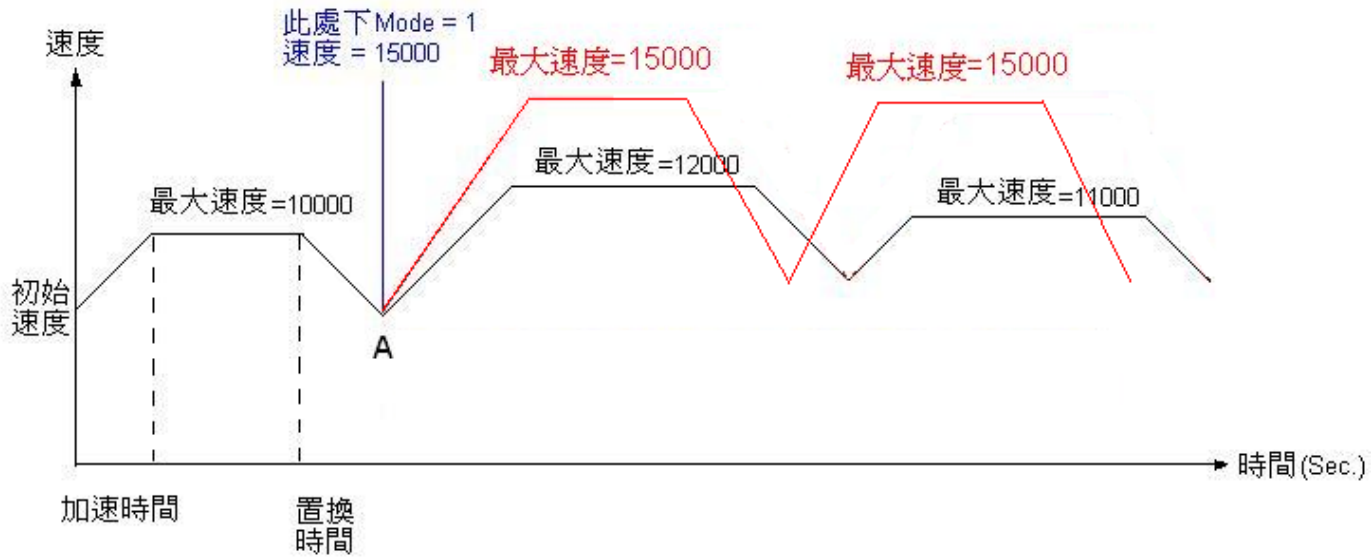


圖 18.8　Mode = 1 更換新的向量速度

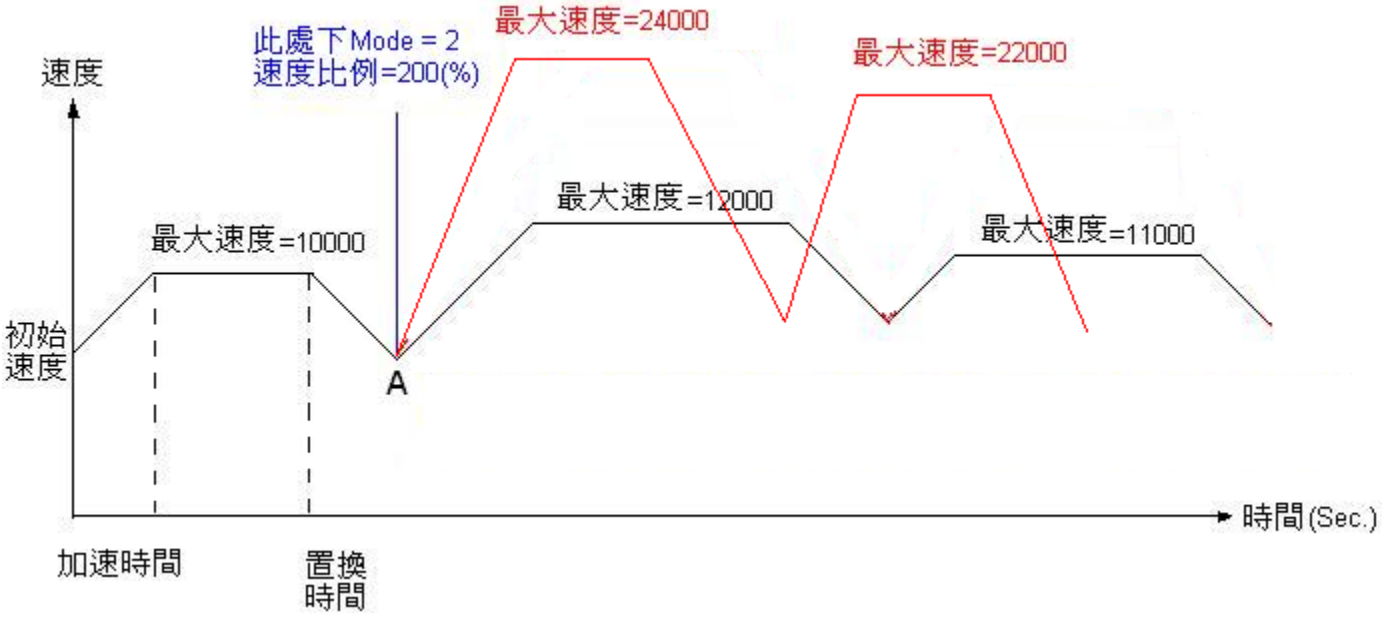


圖 18.9　Mode = 2 更換新的向量速度比例

範例

U16 CardNo=0; U16 NodeID =1, SlotID=0, Mode=0;
I32 NewSpeed=3000;
F64 sec=0.1;

I16 status = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID, Mode, NewSpeed, sec);

NOTE

使用 DMC_01_feedrate_overwrite　API 有幾點事項需注意。

若下 feedrate_overwrite 的時機剛好落在運動的加速段及最大速度段，那麼加、減速度及最大速度將依照您下的 NewSpeed 比例變化。

若下 feedrate_overwrite 的時機剛好落在運動的減速段，那麼 API 將回應錯誤值(9)，API 下達失敗，需要有 Check API Return Code 及重下機制。

DELTA CONFIDEN

18.16　_DMC_01_start_v3_move

■　**格式**

I16 PASCAL _DMC_01_start_v3_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■　**目的**

增加 EndVel 的單軸運動位移。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的運動行程
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
M_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
M_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

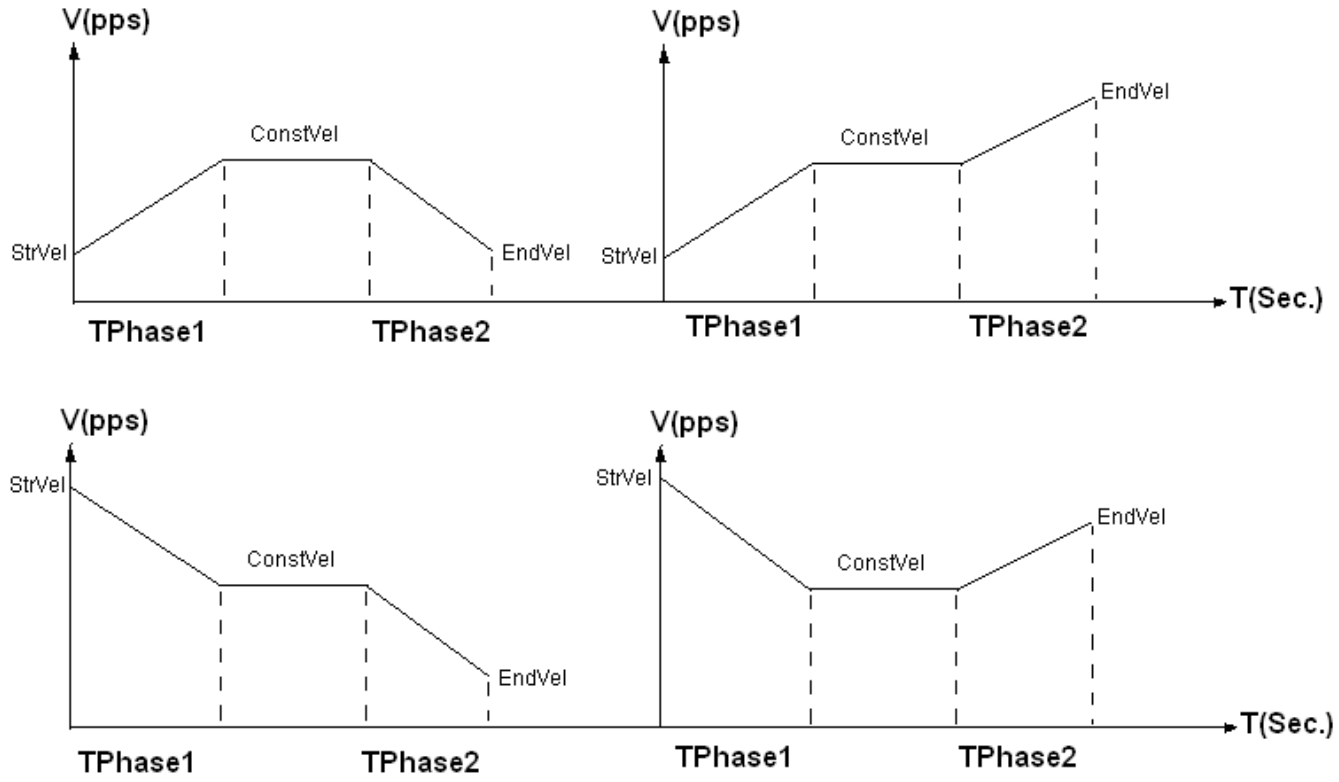
■　**範例**

U16 CardNo=0; U16 NodeID =1, SlotID=0;
I32 Dist=100000, StrVel=0, ConstVel=50000, EndVel=20000;
F64 TPhase1=0.2, TPhase2=0.1;

U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move (CardNo, NodeID, SlotID, Dist, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);

NOTE



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 18.10 TPhase1 與 TPhase2 說明

18.17　_DMC_01_multi_table_V3_move

■　**格式**

I16 PASCAL _DMC_01_multi_table_V3_move(U16 CardNo, U16 MotionNum, U16 NodeID, U16 SlotID, I32 *Dist,I32 StrVel, I32 *ConstVel, I32 EndVel , F64 *Tphase, F64 Tdec, U16 m_curve, U16 m_r_a);

■　**目的**

單軸使用 Table 的方式連續移動至不同位置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	群組卡號 GroupNo 為 1~4
MotionNum	U16	編號單位	運動指令數量(最大為 20 筆)
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32 *	脈波數陣列	對應運動指令數量的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32 *	脈波數/秒 陣列	對應運動指令數量的常態速度
EndVel	I32	脈波數/秒	運動結束速度
Tphase	F64	秒　陣列	對應運動指令數量的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

CONFIDENTIAL

■　**範例**

U16 CardNo = 0, MotionNum = 4, NodeID = 1, SlotID = 0;
I32Dist[4] = {5000,6000,4000,7000};
I32 StrVel = 2000, EndVel = 2000;
I32ConstVel[4] = {3000,2000,1000,5000};
F64Tphase[4] = {0.5,0.1,0.1,0.1};
F64 Tdec = 0.5;
U16 m_curve = 1, m_r_a = 0;

I16 status = _DMC_01_multi_table_V3_move(CardNo,MotionNum,NodeID,SlotID,Dist, StrVel, ConstVel, EndVel, Tphase, Tdec, m_curve, m_r_a);

NOTE

假設使用 4 筆運動指令，須設定 4 筆運動目標路徑，Tphase 為每筆運動指令的加速時間，且只有最後一筆指令有 Tdec 減速時間。
使用限制：指令是會近入 BUFFER，所以要連續使用此指令時，需確認 BUFFER 數量。

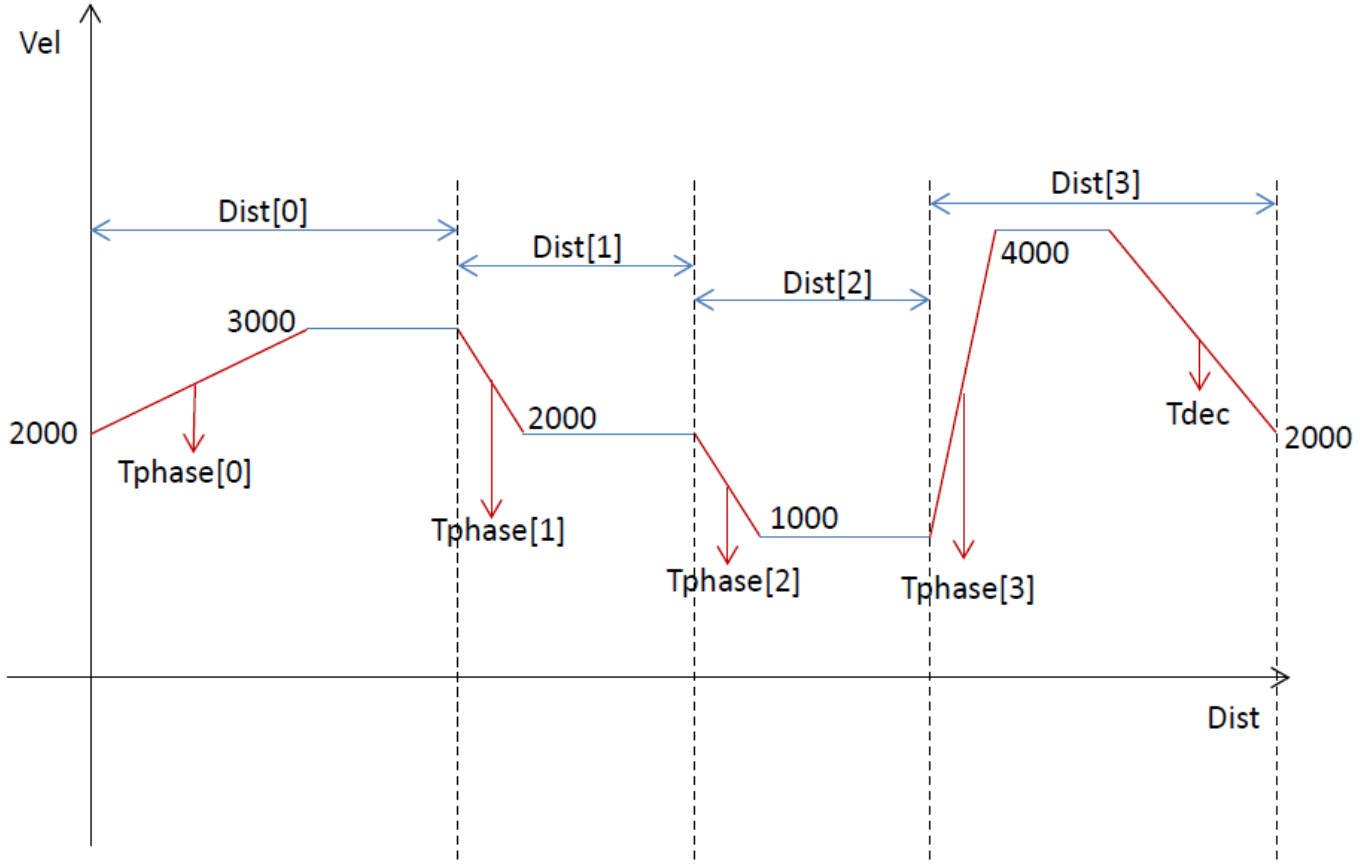


圖 18.10 TPhase1 與 TPhase2 說明

18.18　_DMC_01_feedrate_overwrite_read

■　**格式**

I16 PASCAL _DMC_01_feedrate_overwrite_read(U16 CardNo, U16 NodeID, U16 SlotID,U16* Mode, I32* New_Speed, F64* sec)

■　**目的**

取得改變運動向量速度或向量速度比率設定值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16*	選項單位	0：與 V_Change 相同，在有運動執行下，改變當前運動的向量速度。 1：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度。 2：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度比例，範圍是 0% ~ 1000%。
NewSpeed	I32*	脈波數/秒	Mode=0,1 時:欲替換速度參數 Mode=2 時:向量速度比例值範圍是 0% ~ 1000%
sec	F64*	秒	速度轉換時，指定的加/減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 Mode=0;

I32 NewSpeed;
F64 sec;

I16 status = _DMC_01_feedrate_overwrite_read (CardNo, NodeID, SlotID, &Mode, &NewSpeed, &sec);

DELTA CONFIDENTIAL

第十九章　二軸線性補間運動控制 API

此章節說明了如何使用二軸線性補間應用程式介面(API)，控制兩軸補間運動執行相對運動、絕對運動、加減速度運動在 T-Curve 與 S-Curve 兩種模式。

表 19.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move_xy	相對座標進行兩軸線性補間運動(T curve)
A/B/F	_DMC_01_start_sr_move_xy	相對座標進行兩軸線性補間運動(S curve)
A/B/F	_DMC_01_start_ta_move_xy	參絕對座標進行兩軸線性補間運動(T curve)
A/B/F	_DMC_01_start_sa_move_xy	絕對座標進行兩軸線性補間運動(S curve)
A/B/F	_DMC_01_start_v3_move_xy	設定運動末速的兩軸線性補間運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

19.1　_DMC_01_start_tr_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸線性補間運動。

※設置 StrVel 的值時，該值務必設定小於 MaxVel 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=30000;
I32 DisY=40000;
I32 StrVel=0;
I32 MaxVel=3000;
F64 Tacc=0.1;
F64 Tdec=0.1;
```

DELTA CONFIDENTIAL

I16 status = _DMC_01_start_tr_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);

// 當 MaxVel 設置為 3000RPM，則 X 軸速度為 1800RPM；Y 軸速度為 2400RPM

NOTE

如圖 19.1, 兩軸直線補間意指從 P0 到 P1 運動位移 XY 的位置。該兩軸會同時起始與終止，並且運動位移的路徑會在同一直線上。

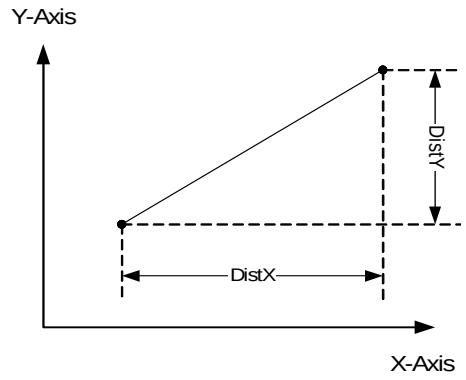


圖 19.1

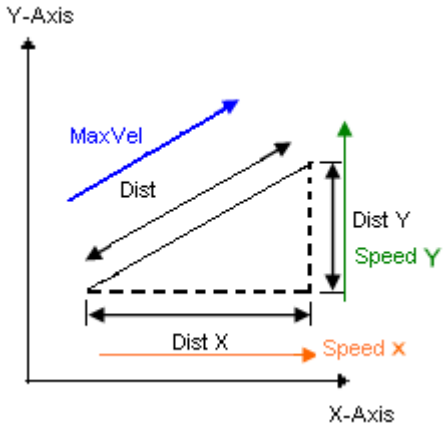


圖 19.2

沿著 X 軸與 Y 軸(DistX :DistY)分別的速率比與運動速度的關係為：

$$\frac{DistX}{Dist} = \frac{DistY}{Dist}$$

承上，從上圖 19.2 表示，您將可以得知最大速度(MaxVel)與 X 軸的速度(speed X)和 Y 軸的速度(speed Y)的關係如下：

$$Speed\ X = \frac{MaxVel \times Dist\ X}{Dist}, \quad Speed\ Y = \frac{MaxVel \times Dist\ Y}{Dist}$$

19.2　_DMC_01_start_sr_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_sr_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的相對座標進行兩軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

19.3　_DMC_01_start_ta_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;

I16 status = _DMC_01_start_ta_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

19.4　_DMC_01_start_sa_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_sa_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={0,1};
U16 SlotID[2]={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;

I16 status = _DMC_01_start_sa_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

19.5　_DMC_01_start_v3_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸線性補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的行程
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的行程
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={0,1};
U16 SlotID[2]={0, 0};
I32 DisX=50000;
I32 DisY= 100000;
I32 StrVel=0;
I32 ConstVel=50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel,
ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

第二十章　二軸圓弧補間運動控制 API

此章節說明了如何使用運動控制應用程式介面(API)，在 T-Curve 與 S-Curve 兩種模式下，控制兩軸圓弧補間運動執行相對運動、絕對運動、加減速度運動。

表 20.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_arc_xy	相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度) (T curve)
A/B/F	_DMC_01_start_sr_arc_xy	相對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度) (S curve)
A/B/F	_DMC_01_start_ta_arc_xy	絕對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度) (T curve)
A/B/F	_DMC_01_start_sa_arc_xy	絕對座標進行兩軸圓弧補間運動(已知條件：圓心座標、角度) (S curve)
A/B/F	_DMC_01_start_tr_arc2_xy	相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度) (T curve)
A/B/F	_DMC_01_start_sr_arc2_xy	相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度) (S curve)
A/B/F	_DMC_01_start_ta_arc2_xy	絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度) (T curve)
A/B/F	_DMC_01_start_sa_arc2_xy	絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度) (S curve)
A/B/F	_DMC_01_start_tr_arc3_xy	相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標) (T curve)
A/B/F	_DMC_01_start_sr_arc3_xy	相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標) (S curve)
A/B/F	_DMC_01_start_ta_arc3_xy	絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標) (T curve)
A/B/F	_DMC_01_start_sa_arc3_xy	絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標) (S curve)
A/B/F	_DMC_01_start_spiral_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標)
A/B/F	_DMC_01_start_spiral2_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
A/B/F	_DMC_01_start_v3_arc_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：圓心座標、角度)
A/B/F	_DMC_01_start_v3_arc2_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)

DELTA CONFIDENTIAL

A/B/F	_DMC_01_start_v3_arc3_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：圓心座標、終點座標)
A/B/F	_DMC_01_start_v3_spiral_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸圓心座標)
A/B/F	_DMC_01_start_v3_spiral2_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸圓心座標，X、Y 軸終點座標)
A/B	_DMC_01_start_multiarc	兩軸圓弧補間運動，支援多軸、arc、arc2、arc3，三種已知條件模式。
A/B	_DMC_01_Start_Move_Spiral_Circle	兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標)。
A/B	_DMC_01_Start_Move_Spiral2_Circle	兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。
A/B	_DMC_01_Start_Arc_follow	可指定任意兩軸作為圓弧運動的X和Y軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

20.1　_DMC_01_start_tr_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。
(限制只能同時下達四組 arc 指令。)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000;
I32 Center_Y =50000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

NOTE

_DMC_01_start_xx_arc_xy 可操作 360 度(倍數)的圓。

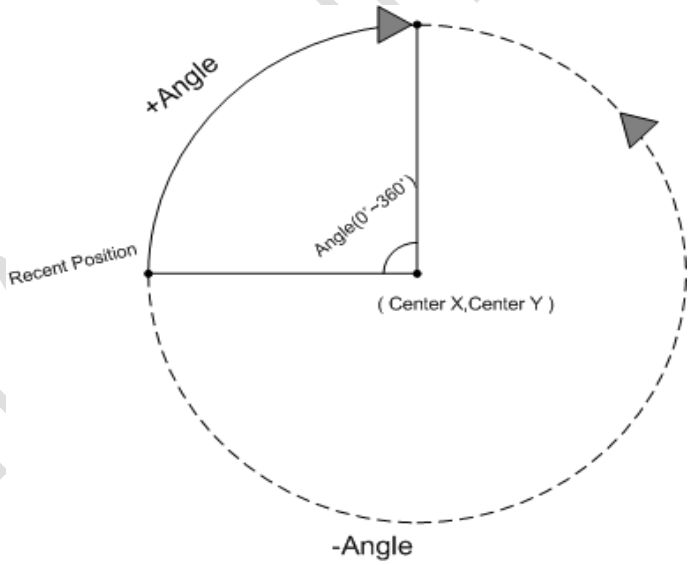


圖 20.1

DELTA CONFIDENTIAL

20.2　_DMC_01_start_sr_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_sr_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.3　_DMC_01_start_ta_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.4　_DMC_01_start_sa_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_sa_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sa_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.5　_DMC_01_start_tr_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之相對性終點位置 X
End_Y	I32	脈波數	指定軸對應之相對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000;
I32 End_Y =-50000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle,
StrVel, MaxVel, Tacc, Tdec);
```

NOTE

_DMC_01_start_xx_arc2_xy 無法操作 360 度(倍數)的圓。

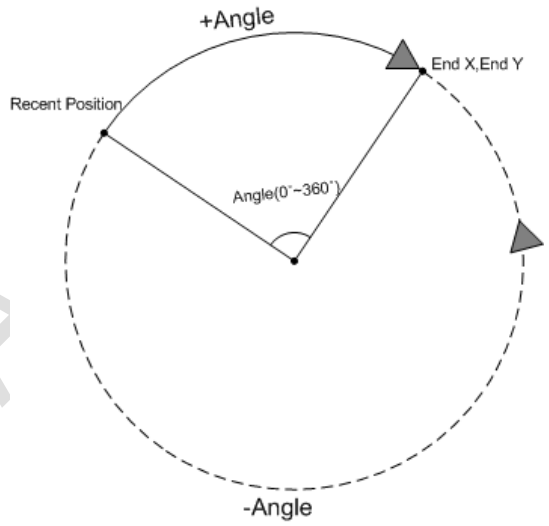


圖 20.2

DELTA CONFIDENTIAL

20.6　_DMC_01_start_sr_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_sr_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之相對性終點位置 X
End_Y	I32	脈波數	指定軸對應之相對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.7　_DMC_01_start_ta_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ 範例

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.8　_DMC_01_start_sa_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_sa_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sa_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.9　_DMC_01_start_tr_arc3_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

(限制只能同時下達三組 arc3 指令。)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性中心位置 Y
End_x	I32	脈波數	指定軸對應之相對性終點位置 X
End_y	I32	脈波數	指定軸對應之相對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000;
I32 Center_Y =25000;
I32 StrVel=0;
I32 MaxVel=50000;
I32 End_x=50000;
I32 End_y=50000;
I16 Dir=1; //此值為 1，表示此為逆時針方向圓弧補間
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

NOTE

1._DMC_01_start_xx_arc3_xy 只能操作 360 度以下的圓。

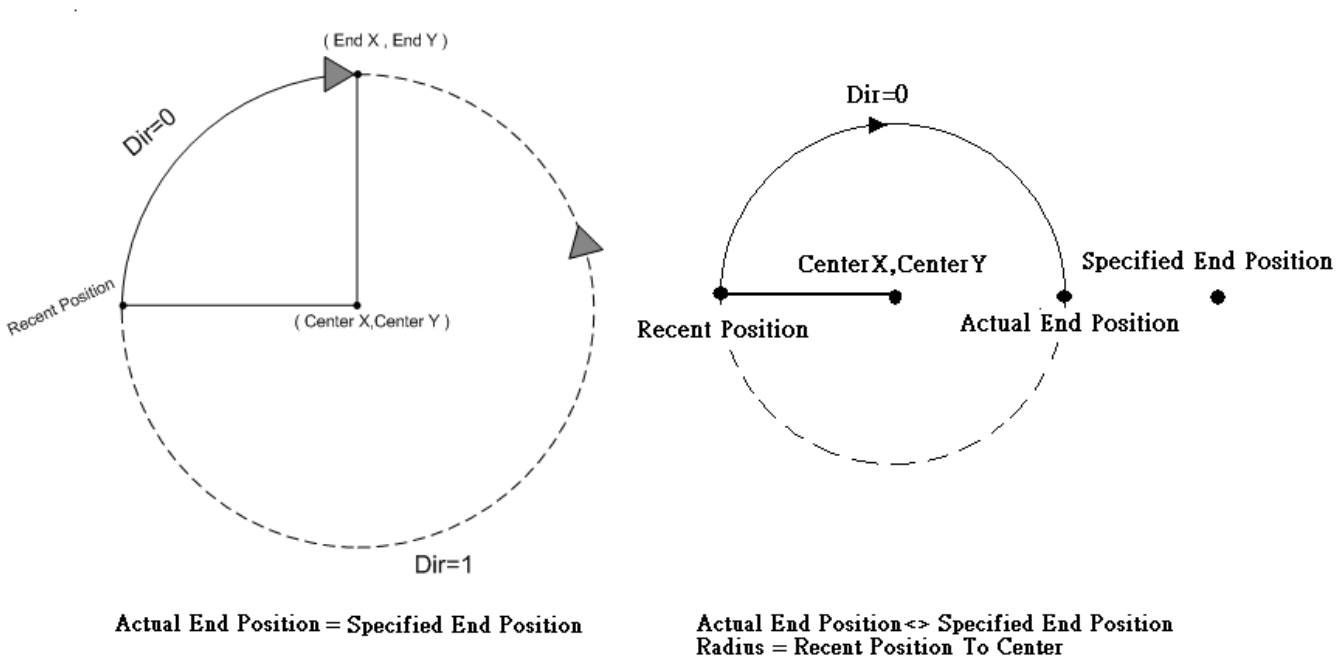


圖 20.3

CONFIDENTIAL

20.10　_DMC_01_start_sr_arc3_xy

■ **格式**

I16 PASCAL _DMC_01_start_sr_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性中心位置 Y
End_x	I32	脈波數	指定軸對應之相對性終點位置 X
End_y	I32	脈波數	指定軸對應之相對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1; //此值為 1，表示此為逆時針方向圓弧補間
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sr_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.11　_DMC_01_start_ta_arc3_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.12　_DMC_01_start_sa_arc3_xy

■ **格式**

I16 PASCAL _DMC_01_start_sa_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

20.13　_DMC_01_start_spiral_xy

■ **格式**

I16 PASCAL _DMC_01_start_spiral_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 Center_X, I32 Center_Y, I32 spiral_interval, I32 spiral_angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

進行兩軸螺旋運動(已知條件：X、Y 軸中心座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
spiral_interval	I32	脈波數	螺旋之間相對性位置距離
spiral_angle	I32	數值單位	螺旋運動之總角度(360 度一圈)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

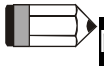
■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000,
I32 Spiral_interval= 20000,Spiral_angel=1800;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

I16 status = _DMC_01_start_spiral_xy (CardNo, NodeID, SlotID, Center_X, Center_Y,
Spiral_interval, Spiral_angel, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```



每 6 軸(1~6 及 7~12)只能同時下達一個 Spiral 指令，其他軸可下達如:直線插補運動(1、2、3 軸都可以)。

DELTA CONFIDENTIAL

20.14　_DMC_01_start_spiral2_xy

■ **格式**

I16 PASCAL _DMC_01_start_spiral2_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec, U16 m_curve, U16 m_r_a)

■ **目的**

進行兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =25000, center_y=25000;
I32 end_x =25000, end_y =25000;
U16 dir= 1,circelnum=5;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

I16 status = _DMC_01_start_spiral2_xy (CardNo, NodeID, SlotID, center_x, center_y,
end_x, end_y, dir, circelnuml, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

20.15　_DMC_01_start_v3_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32ConstVel, I32 EndVel, F64 TPhase1,
F64 TPhase2, U16 m_curve, U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、角度)。
※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

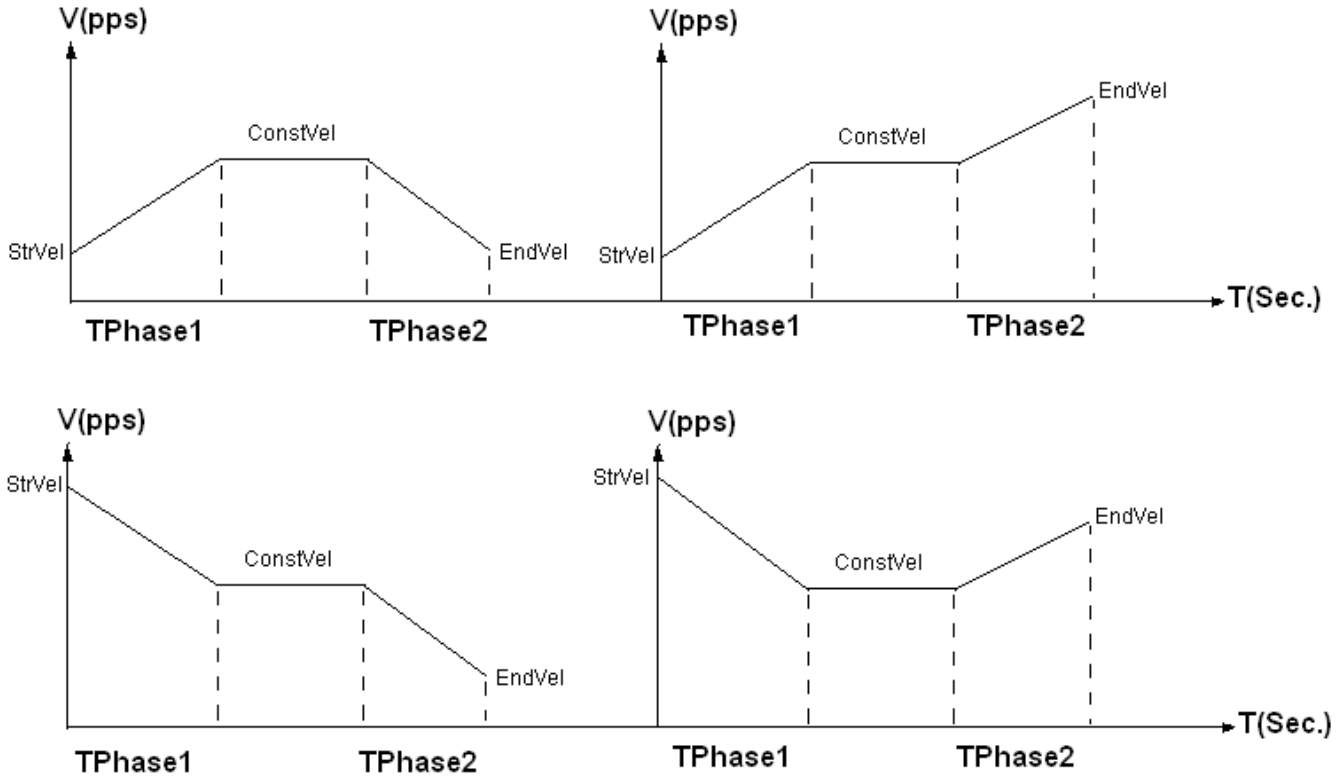
名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 20.4 TPhase1 與 TPhase2 說明

■ **範例**

U16 CardNo=0, NodeIDArray[2]={1,2}, SlotID[2]={0, 0};
I32 Center_X =50000;
I32 Center_Y =50000;
F64 Angle=180;
I32 StrVel=0;
I32 ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y, Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);

20.16　_DMC_01_start_v3_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000;
I32 End_Y =-50000;
I32 StrVel=0;
I32 ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y,
Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

20.17　_DMC_01_start_v3_arc3_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
I32 StrVel=0, ConstVel=50000 , EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

20.18　_DMC_01_start_v3_spiral_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_spiral_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 Center_X, I32 Center_Y, I32 spiral_interval, I32 spiral_angle, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
spiral_interval	I32	脈波數	螺旋之間相對性位置距離
spiral_angle	I32	數值單位	螺旋運動之總角度(360 度一圈)
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000,
I32 Spiral_interval= 20000, Spiral_angel=1800;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_spiral_xy (CardNo, NodeID, SlotID, Center_X, Center_Y,
Spiral_interval, Spiral_angel, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

20.19　_DMC_01_start_v3_spiral2_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_spiral2_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, 32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■ **目的**

增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =25000, center_y=25000;
I32 end_x =25000, end_y =25000;
U16 dir= 1, circelnum=5;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_spiral2_xy (CardNo, NodeID, SlotID, center_x, center_y,
end_x, end_y, dir, circelnuml, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

20.20　_DMC_01_start_multiarc

■ **格式**

I16 PASCAL _DMC_01_start_multiarc(U16 CardNo, U16 AxesNum, U16* NodeID, U16* SlotID, U16 ArcBit, U16 HelixBit, U16 ArcMode, I32 *CenterPoint, I32 *EndPoint, F64 Angle, I32 StrVel, I32 ConstVel, I32 EndVel, F64 Tacc, F64 Tdec, U16 Curve_mode, U16 AbsMode)

■ **目的**

兩軸圓弧補間運動，支援多軸、arc、arc2、arc3，三種已知條件模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxesNum	U16	編號單位	存取用來進行螺旋運動的站號 ID 數組 AxesNum [0] 存放第一組站號 AxesNum [1] 存放第二組站號
AxisNum	U16*	編號單位	軸數量(支援 12 軸)
SlotID	U16*	編號單位	插槽 ID
ArcBit	U16	選項單位	設置欲操作的軸 (二進制值，必須設置兩軸)，例: ArcBit = 3: 1、2 軸圓弧補間運動(0000 0000 0011) ArcBit = 6: 2、3 軸圓弧補間運動(0000 0000 0110) ArcBit = 5: 1、3 軸圓弧補間運動(0000 0000 0101) 以此類推。
HelixBit	U16	選項單位	保留
ArcMode	U16	選項單位	選擇模式 0: 已知條件：中心座標、角度。 1: 已知條件：終點座標、角度。 2: 已知條件：中心座標、終點座標。
CenterPoint	I32 *	脈波數	指定軸對應中心位置
EndPoint	I32 *	脈波數	指定軸對應終點位置
Angle	F64	度(°)	度(°) 一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數

DELTA CONFIDENTIAL

ConstVel	I32	脈波數/秒	切線速度參數
EndVel	I32	脈波數/秒	運動結束速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
Curve_mode	U16	選項單位	1：T-curve(Default) 2：S-curve
AbsMode	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 AxesNum=3;
U16 AxisNum[2]={1,2,3};
U16 SlotID={0};
U16 ArcBit=6;
U16 HelixBit=0;
U16 ArcMode=2;

I32 CenterPoint[2]={0,25000,25000};
I32 EndPoint[2]={0,50000,50000};

F64 Angle=180;
I32 StrVel=0;
I32 ConstVel=500000;
I32 EndVel=0;
F64 Tacc=0.1;
F64 Tdec=0.1;

U16 Curve_mode = 1;
U16 AbsMode = 0;
I16 status =_DMC_01_start_multiarc(CardNo, AxesNum, NodeID, SlotID, ArcBit, HelixBit,
ArcMode, CenterPoint, EndPoint, Angle, StrVel, ConstVel, EndVel, Tacc, Tdec, Curve_mode,
AbsMode);
```

DELTA CONFIDENTIAL

NOTE

若 ArcMode=0 或 1

Angle > 0:順時針方向

Angle < 0:逆時針方向

若 ArcMode=2

Angle > 0:逆時針方向

Angle < 0:順時針方向

DELTA CONFIDENTIAL

20.21　_DMC_01_Start_Move_Spiral_Circle

■ **格式**

I16 PASCAL _DMC_01_Start_Move_Spiral_Circle(U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 spiral_interval, U32 spiral_angle ,I32 StrVel, I32 MaxVel, I32 EndVel, F64 Tacc, F64 Tdec, I16 Curve_mode, I16 m_r_a)

■ **目的**

兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
spiral_interval	I32	脈波數	螺旋之間相對性位置距離
spiral_angle	I32	數值單位	螺旋運動之總角度(360 度一圈)
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

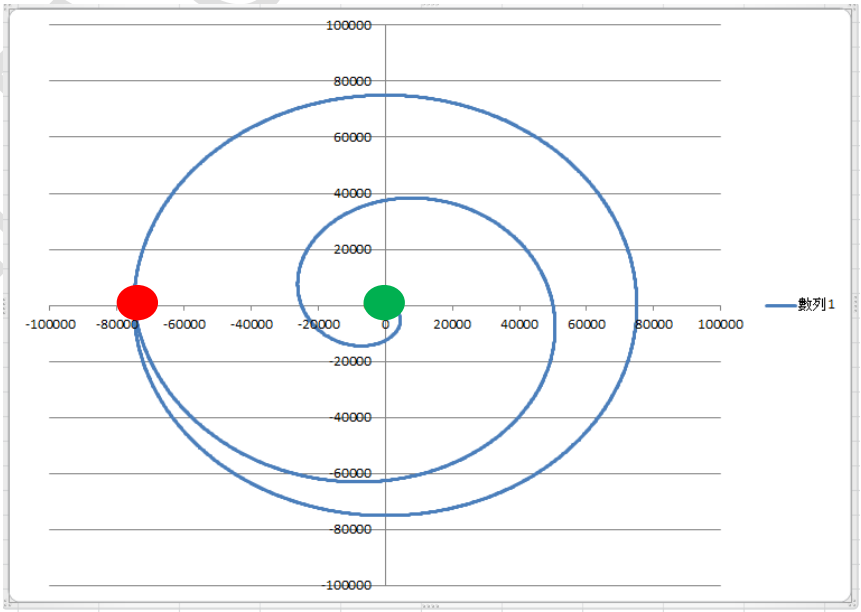
■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =0, Center_Y =0,
I32 Spiral_interval= 20000, Spiral_angel=540;
I32 StrVel=0, ConstVel =200000;
I32 EndVel=0;
F64 TPhase1=0.1;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_Start_Move_Spiral_Circle (CardNo, NodeID, SlotID, Center_X,
Center_Y,Spiral_interval, Spiral_angel, StrVel, ConstVel, EndVel, TPhase1, TPhase2,
m_curve, m_r_a);
```

NOTE

_DMC_01_Start_Move_Spiral_Circle 雙軸平面螺旋運動，從當前位置與指定的圓心，螺旋運動到指定角度。

起點　末點。



DELTA CONFIDENTIAL

20.22　_DMC_01_Start_Move_Spiral2_Circle

■ **格式**

I16 PASCAL _DMC_01_Start_Move_Spiral2_Circle(U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, I32 MaxVel, I32 EndVel, F64 Tacc, F64 Tdec, I16 Curve_mode, I16 m_r_a)

■ **目的**

兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

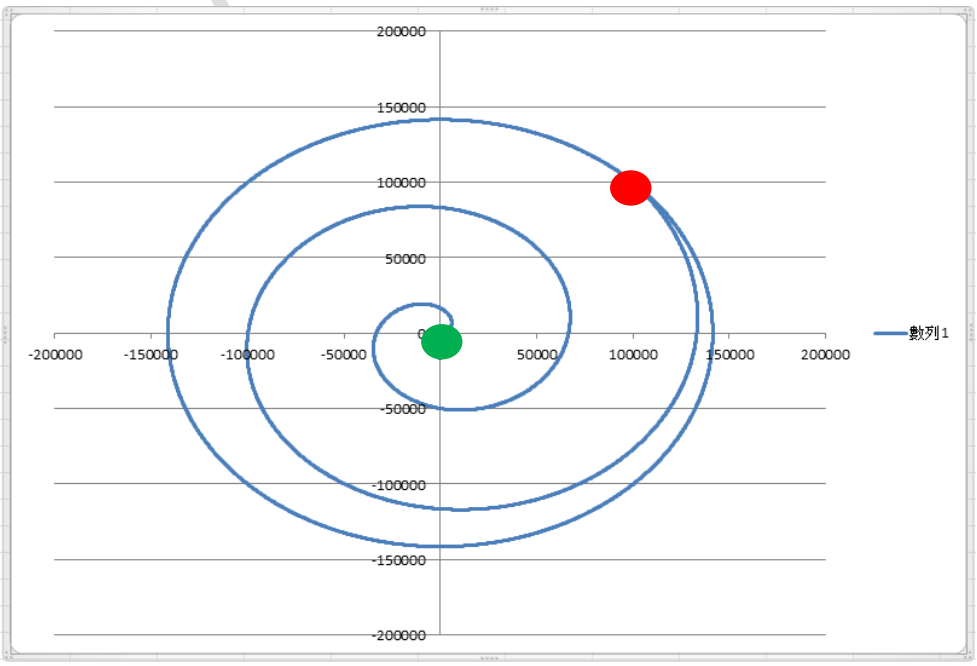
■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =0, center_y=0;
I32 end_x =100000, end_y =100000;
U16 dir= 1, circelnum=2;
I32 StrVel=0, ConstVel =200000;
I32 EndVel=0;
F64 TPhase1=0.1;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_Start_Move_Spiral2_Circle (CardNo, NodeID, SlotID, center_x,
center_y,end_x, end_y, dir, circelnuml, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve,
m_r_a);
```

NOTE

_DMC_01_Start_Move_Spiral2_Circle 雙軸平面螺旋運動，從當前位置與指定的圓心，螺旋運動指定圈數到指定的終點位置。

起點 末點。



DELTA CONFIDENTIAL

20.23　_DMC_01_Start_Move_Spiral2_Circle

■ **格式**

I16 PASCAL _DMC_01_Start_Move_Spiral2_Circle(U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, I32 MaxVel, I32 EndVel, F64 Tacc, F64 Tdec, I16 Curve_mode, I16 m_r_a)

■ **目的**

兩軸螺旋成圓運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

■ **參數**

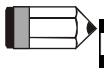
名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的軸卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =0, center_y=0;
I32 end_x =100000, end_y =100000;
U16 dir= 1, circelnum=2;
I32 StrVel=0, ConstVel =200000;
I32 EndVel=0;
F64 TPhase1=0.1;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_Start_Move_Spiral2_Circle (CardNo, NodeID, SlotID, center_x,
center_y,end_x, end_y, dir, circelnuml, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve,
m_r_a);
```



NOTE

_DMC_01_Start_Move_Spiral2_Circle 雙軸平面螺旋運動，從當前位置與指定的圓心，螺旋運

DELTA CONFIDENTIAL

20.24　_DMC_01_Start_Arc_follow

■ **格式**

I16 PASCAL _DMC_01_Start_Arc_follow(U16 CardNo, U16 AxesNum, U16 *AxisArray,U16* SlotID, U16 ArcMode, U16 *Arc_Axis_index, U16 master_axis_index , I32 *CenterPoint, I32 *EndPoint, F64 Angle, U16 Dir, I32 StrVel, I32 ConstVel, I32 EndVel, F64 Tacc, F64 Tdec, U16 Curve_mode, U16 AbsMode)

■ **目的**

可指定任意兩軸作為圓弧運動的 X 和 Y 軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxesNum	U16	編號單位	參與軸數（≧2）。
AxisArray	U16*	編號陣列	軸數組，數量等同於 AxisNum。
SlotID	U16*	編號陣列	插槽 ID
ArcMode	U16	選項單位	選擇 Arc 模式。 0：已知圓心、角度（Arc1）。 1：已知末點、角度（Arc2）。 2：已知圓心、末點（Arc3）。 3：已知中間點、末點（Arc4）。
Arc_Axis_index	U16*	編號陣列	指定軸數組內做 Arc 運動的軸的位置。 Ex:AxisArray={1,2,3,4},指定{2,3}軸做 Arc 運動，則 index 需填{1,2}。 Axis_index [0]≠Axis_index [1]且不能大於等於或少於 AxesNum。
master_axis_index	U16	編號單位	跟隨運動的主軸。此編號須和 Axis_index [0] or Axis_index [1] 其中一個一樣。
CenterPoint	I32*	脈波數	圓心位置 在 ArcMode=3 時，此參數定義為中間點位置。
EndPoint	I32*	脈波數	末點位置。
Angle	F64	度	圓弧的角度 正值代表往順時針方向旋轉。 負值代表往逆時針方向旋轉。

Dir	U16	選項單位	ArcMode=2 時使用，需填運動方向。 0：順時針選轉。 1：逆時針旋轉。
StrVel	I32	脈波數/秒	運動初始速度參數。
ConstVel	I32	脈波數/秒	運動等速度參數。
EndVel	I32	脈波數/秒	運動結束速度參數。
Tacc	F64	秒	初始速度到等速度的時間。
Tdec	F64	秒	等速度到結束速度的時間。
Curve_mode	U16	選項單位	0：T-curve(Default) 1：T-curve 2：S-curve
AbsMode	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
I16 CardNo = 0;
U16 AxisNum = 5;
U16 AxisArray[3]= { 1, 2, 3, 4 };
U16 SlotID{0,0,0,0};
U16 ArcMode = 0;
U16 Arc_index[2] = { 2, 3 };
U16 master_axis_index=3;
I32 CenterPoint[2] = { 200000, 200000};
I32 EndPoint[2] = { 0, 0 };
F64 Angle = 270;
U16 Dir = 0;
I32 StrVel = 0;
I32 MaxVel = 200000;
I32 EndVel = 0;
F64 Tacc = 0.5, Tdec = 0.5;
U16 Curve_mode = 0;
U16 AbsMode = 0;

I16 status = _DMC_01_Start_Arc_follow (CardNo, AxisNum, AxisArray, SlotID, ArcMode,
Arc_index, master_axis_index, CenterPoint, EndPoint, Angle, Dir, StrVel, MaxVel, EndVel,
Tacc, Tdec, Curve_mode, AbsMode);
```

NOTE

DELTA CONFIDENTIAL

第二十一章　三軸線性補間運動控制 API

此章節說明了如何使用三軸線性補間運動應用程式介面(API)，在 T-Curve 與 S-Curve 兩種模式下，控制三軸線性補間運動執行相對運動、絕對運動、加減速度運動。

表 21.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move_xyz	參照 T-curve 速度剖面相對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_sr_move_xyz	參照 S-curve 速度剖面相對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_ta_move_xyz	參照 T-curve 速度剖面絕對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_sa_move_xyz	參照 S-curve 速度剖面絕對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_v3_move_xyz	增加 EndVel 的三軸線性補間運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

21.1　_DMC_01_start_tr_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_tr_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
```

```
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_tr_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

21.2　_DMC_01_start_sr_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
```

```
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

21.3　_DMC_01_start_ta_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_ta_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
```

```
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

21.4　_DMC_01_start_sa_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
```

```
F64 Tacc=0.1,Tdec=0.1;
I16 status = _DMC_01_start_sa_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

21.5　_DMC_01_start_v3_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_v3_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的三軸線性補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

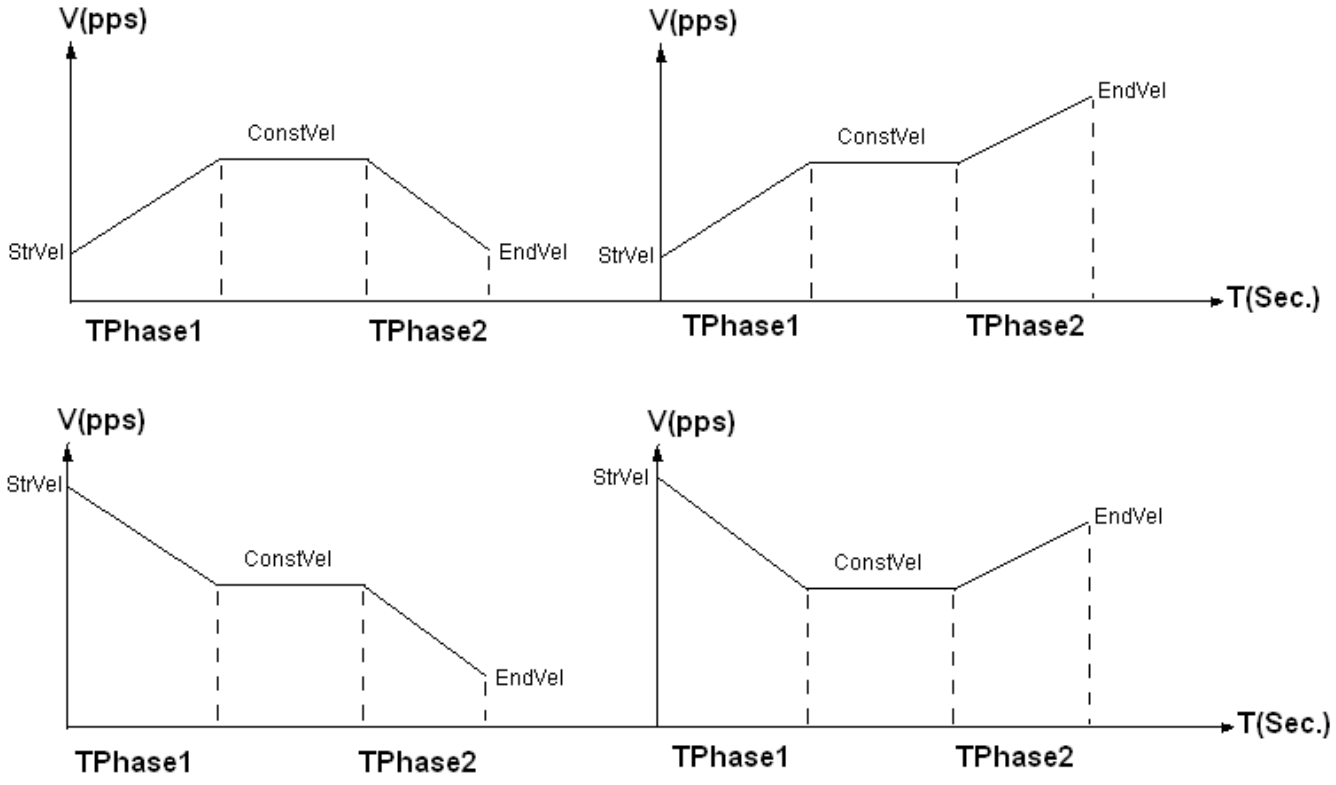
名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 21.4 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

第二十二章　三軸螺旋補間運動控制 API

此章節說明了如何使用三軸螺旋補間運動應用程式介面(API)，在 T-Curve 與 S-Curve 兩種模式下，控制三軸螺旋補間運動執行相對運動、絕對運動、螺旋補間運動、畫圓。

表 22.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_heli_xy	參照 T-curve 速度剖面相對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_sr_heli_xy	參照 S-curve 速度剖面相對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_ta_heli_xy	參照 T-curve 速度剖面絕對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_sa_heli_xy	參照 S-curve 速度剖面絕對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_v3_heli_xy	增加 EndVel 的三軸螺旋補間運動
A/B/F	_DMC_01_start_v3_sphere_xyz	三點成圓
A/B/F	_DMC_01_start_v3_heli_multi_axes_move	增加 EndVel 的多軸螺旋補間運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

22.1　_DMC_01_start_tr_heli_xy

■　**格式**

I16 PASCAL _DMC_01_start_tr_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行三軸螺旋補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　敘述

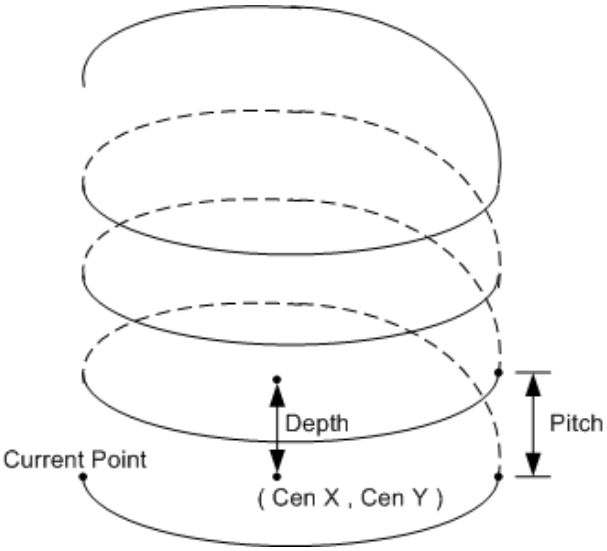


圖 22.1

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000;
I32 Center_Y =50000;
I32 Depth =10000;
I32 Pitch = 20000;

I16 Dir=1; //值為 1，表示此為逆時針方向的螺旋圓弧運動

I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

NOTE

這裡所下的最大速度是 X、Y 軸向量速度的依據，但 Z 軸的速度是依照 Depth 及 Pitch 的值所推算出來的。

DELTA CONFIDENTIAL

Z 的速度推算公式如下: (Pitch * Maxvel) /(Radius * 2 * π)。

DELTA CONFIDENTIAL

22.2　_DMC_01_start_sr_heli_xy

■　**格式**

I16 PASCAL _DMC_01_start_sr_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行三軸螺旋補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sr_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

22.3　_DMC_01_start_ta_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行三軸螺旋補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X,	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

22.4　_DMC_01_start_sa_heli_xy

■　**格式**

I16 PASCAL _DMC_01_start_sa_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行三軸螺旋補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

DELTA CONFIDENTIAL

22.5　_DMC_01_start_v3_heli_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, 32 ConstVel, I32 EndVel,
F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■　**目的**

增加 EndVel 的三軸螺旋補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

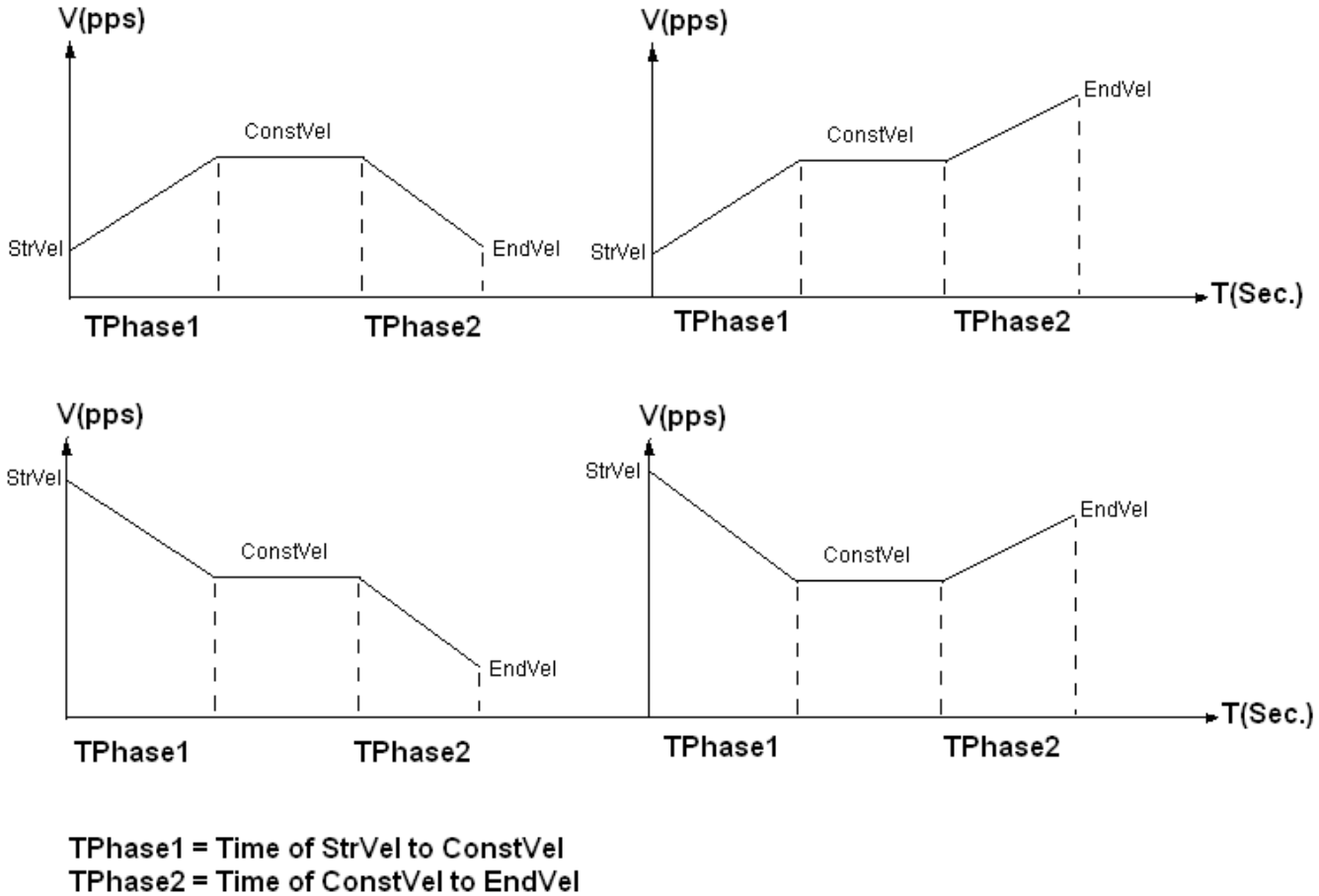
名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 22.2 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

22.6　_DMC_01_start_v3_sphere_xyz

■　**格式**

I16 PASCAL _DMC_01_start_v3_sphere_xyz (U16 CardNo, U16* NodeID, U16* SlotID,
I32 pos1_x, I32 pos1_y, I32 pos1_z, I32 pos2_x, I32 pos2_y, I32 pos2_z, I32 StrVel, I32 ConstVel,
I32 EndVel, F64 Tacc, F64 Tdec, U16 m_curve, U16 m_r_a)

■　**目的**

三點成圓(起始點與第一指定點與第二指定點，三點不能在同一直線上)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Pos1_x	I32	脈波數	第一指定點 X 軸位置
Pos1_y	I32	脈波數	第一指定點 Y 軸位置
Pos1_z	I32	脈波數	第一指定點 Z 軸位置
Pos2_x	I32	脈波數	第二指定點 X 軸位置
Pos2_y	I32	脈波數	第二指定點 Y 軸位置
Pos2_z	I32	脈波數	第二指定點 Z 軸位置
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 pos1_x =25000, pos1_y =50000, pos1_z =20000;
I32 pos2_x =95000, pos2_y =110000, pos2_z =60000;
I32 StrVel=0, ConstVel=50000, EndVel=20000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_sphere_xyz (CardNo, NodeIDArray, SlotID, pos1_x, pos1_y,
pos1_z, pos2_x, pos2_y, pos2_z, StrVel, ConstVel, EndVel,Tacc, Tdec, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

22.7　_DMC_01_start_v3_heli_multi_axes_move

■ **格式**

I16 PASCAL _DMC_01_start_v3_heli_multi_axes_move(U16 CardNo, U16 AxisNum,U16* NodeID, U16* SlotID, I32 Center_X, I32 Center_Y,I32 *Dist, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 ConstVel, I32 EndVel, F64 Tphase1, F64 Tphase2,U16 m_curve,U16 m_r_a)

■ **目的**

增加多軸螺旋補間運動。

使用 Heli 基本上會將前三軸設定為 Heli 運動，並配合 AxisNum 參數設定來決定有多少軸數需要配合移動，再由 DistArrary 陣列填入跟隨軸的深度數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16	軸數	跟隨的軸數目。設定數值: 0 ~ 8 。
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
DistArrary	I32*	編號單位	跟隨螺旋補間軸的陣列編號 DistArrary [0] 存放第一組跟隨軸的行程，對應的主要行程軸為 Node_4。 DistArrary [1] 存放第二組跟隨軸的行程，對應的主要行程軸為 Node_5。 DistArrary [2] 存放第三組跟隨軸的行程，對應的主要行程軸為 Node_6。 … DistArrary [8] 存放第九組跟隨軸的行程，對應的主要行程軸為 Node_12。

DELTA CONFIDENTIAL

Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度。
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數。
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ 敘述

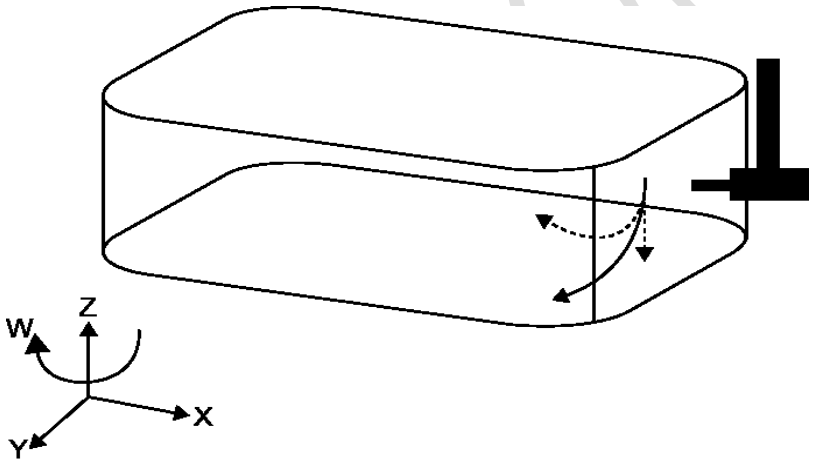


圖 22.7.1

■　**範例**

```
U16 CardNo = 0;
U16 AxisNum =1;
U16 NodeID[4] ={1, 2, 3, 4};//對應圖 22.7.1,   Node_1 為:X 軸
                                                Node_2 為:Y 軸
                                                Node_3 為:W 軸
                                                Node_4 為:Z 軸

U16 SlotID[4] = {0};
I32 Center_X = 25000 ;
I32 Center_Y = 50000 ;
I32 DistArray[1] ={40000};      //例一:Z 軸移動深度 DistArray[1] ={40000};代表一個跟隨軸，深度
                                為 40000。只要 Heli 的 XY 一轉動角度則跟隨的一軸則切入 40000。

                                //例二:DistArray[2] ={40000,40000};代表兩個跟隨軸，深度都為
                                40000。只要 Heli 的 XY 一轉動角度則 跟隨的兩軸則切入 40000。
I32 Depth = 10000;
I32 Pitch = 20000 ;
I16 Dir = 1;
I32 StrVel = 0;
I32 ConstVel = 50000;
I32 EndVel = 20000;
F64 Tphase1 =0.2;
F64 Tphase2 =0.1;
U16 m_curve = 1;
U16 m_r_a = 0;

I16 status = _DMC_01_start_v3_heli_multi_axes_move(CardNo,  AxisNum,  NodeID,  SlotID,
Center_X, Center_Y, DistArray, Depth, Pitch, Dir, StrVel, ConstVel, EndVel, Tphase1, Tphase2,
m_curve,   m_r_a);
```

DELTA CONFIDENTIAL

第二十三章　速度運動控制 API

此章節說明了如何使用速度運動應用程式介面(API)，在 T-Curve 與 S-Curve 兩種模式下，執行運動速度的控制。

表 23.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_tv_move	使用 T-curve 的速度運動控制
A/B/F	_DMC_01_sv_move	參照 S-curve 速度剖面下的速度運動控制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

23.1　_DMC_01_tv_move

■ **格式**

I16 PASCAL _DMC_01_tv_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 Dir)

■ **目的**

參照 T-curve 速度剖面下的速度運動控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數(最小值 5pps)
Tacc	F64	秒	指定的加速時間
Dir	I16	選項單位	0：往正方向 1：往負方向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **敘述**

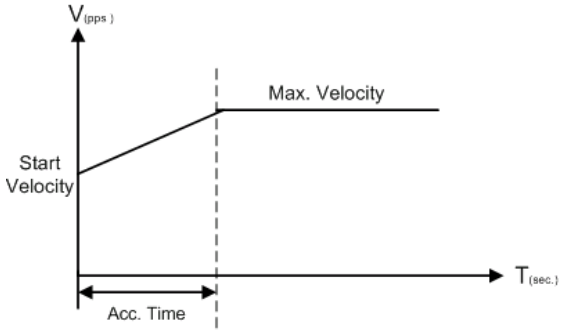


圖. 23.1 trapezoidal 運動模式速度常量表示

■　**範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
I32 StrVel=100, MaxVel=5000;
F64 Tacc=0.1;
I16 Dir =1;

I16 status = _DMC_01_tv_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
```

DELTA CONFIDENTIAL

23.2　_DMC_01_sv_move

■　**格式**

I16 PASCAL _DMC_01_sv_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 Dir)

■　**目的**

參照 S-curve 速度剖面下的速度運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數(最小值 5pps)
Tacc	F64	秒	指定的加速時間
Dir	I16	選項單位	0：往正方向 1：往負方向

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

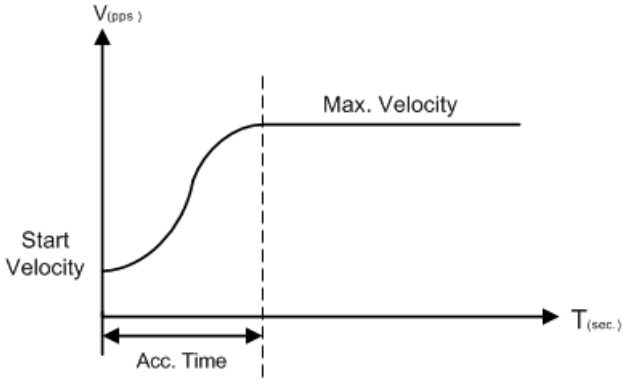


圖 23.2 S-Curve 運動模式速度常量表示

DELTA CONFIDENTIAL

■　**範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
I32 StrVel=100, MaxVel=5000;
F64 Tacc=0.1;
I16 Dir =1;

I16 status = _DMC_01_sv_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
```

DELTA CONFIDENTIAL

第二十四章　同步運動控制 API

此章節說明了如何使用同步運動應用程式介面(API)，Sync_Move_Config 指定需要加入同步運動的伺服與模組代號(NODEID)，Sync_Move 則對配置內的軸開始運行同步運動。

表 24.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_sync_move	設置開始運行同步運動
A/B/F	_DMC_01_sync_move_config	設置同步運動功能是否啟動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

24.1　_DMC_01_sync_move

■ **格式**

I16 PASCAL _DMC_01_sync_move(U16 CardNo)

■ **目的**

設置開始運行同步運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0;

I16 status = DMC_01_sync_move(CardNo);

DELTA CONFIDENTIAL

24.2　_DMC_01_sync_move_config

■ **格式**

I16 PASCAL _DMC_01_sync_move_config(U16 CardNo, U16 NodeID,U16 SlotID,I16 enable)

■ **目的**

設置同步運動功能是否啟動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	選項單位	0：關閉同動 1：啟動同動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;
I16 enable=1;

I16 status = _DMC_01_sync_move_config(CardNo,NodeID, SlotID,enable);

DELTA CONFIDENTIAL

第二十五章　遠端模組控制 API

此章節說明了如何使用遠端模組程式介面(API)，設定遠端模組 MN 與 NT 的輸出入控制，信號濾波，錯誤偵測與修正功能。

表 25.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_rm_input_value	取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
A/B/F	_DMC_01_set_rm_input_filter	設定遠端擴充模組輸入埠的軟體濾波等級
A/B/F	_DMC_01_set_rm_input_filter_enable	設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否
A/B/F	_DMC_01_set_rm_output_value	設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值
A/B/F	_DMC_01_set_rm_output_value_error_handle	設定遠端擴充模組發生錯誤時保持輸出狀態
A/B/F	_DMC_01_get_rm_output_value	取得遠端擴充模組的輸出值
A/B/F	_DMC_01_get_rm_output_value_error_handle	取得遠端擴充模組在發生錯誤時，系統是否將該值保留或重置的設定值
A/B/F	_DMC_01_set_rm_output_active	設定遠端擴充模組輸出
A/B/F	_DMC_01_get_rm_input_single_value	取得遠端擴充模組輸入埠單一 Bit 的值。
A/B/F	_DMC_01_set_rm_output_single_value	設定遠端擴充模組輸出埠單一 Bit 的值。
A/B/F	_DMC_01_get_rm_output_single_value	充模組的輸出埠單一 Bit 的值。

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DEMO CONFIDENTIAL

25.1　_DMC_01_get_rm_input_value

■　**格式**

I16 PASCAL _DMC_01_get_rm_input_value(U16 CardNo, U16 NodeID,U16 SlotID,
U16 port,U16 *value)

■　**目的**

取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
Value	U16*	數值	接收的資料

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			●

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

/* Remote 數位輸入模組  Port 0 所接收的資料  */

I16 status = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
```

DELTA CONFIDENTIAL

25.2　_DMC_01_get_rm_input_single_value

■　**格式**

I16 PASCAL _DMC_01_get_rm_input_single_value(U16 CardNo, U16 NodeID, U16 SlotID, U16 port, U16 bitno,U16 *value)

■　**目的**

取得遠端擴充模組輸入埠單一 Bit 的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
bitno	U16	選項單位	欲操作的 Bit(0~15) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
Value	U16*	數值	接收的資料(ON/OFF)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			●

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 biton=0;
U16 Value;
```

DELTA CONFIDENTIAL

/* Remote 數位輸入模組 Port 0 的 bit0 所接收的資料 */

I16 status = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port,biton, &Value);

DELTA CONFIDENTIAL

25.3 _DMC_01_set_rm_input_filter

■ **格式**

I16 PASCAL _DMC_01_set_rm_input_filter(U16 CardNo, U16 NodeID,U16 SlotID , U16 port,U16 value)

■ **目的**

設定遠端擴充模組輸入埠的軟體濾波等級。(當 value 變數的值為 1 時，軟體濾波時間為 1ms，值為 2 時，軟體濾波時間為 2ms，依此類推)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	數值	軟體濾波時間(1~255ms)，輸入 0 會視為 1. 值為 1 時，濾波時間為 1ms

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			

DELTA CONFIDENTIAL

■ **敘述**

Mode 參數設定示意

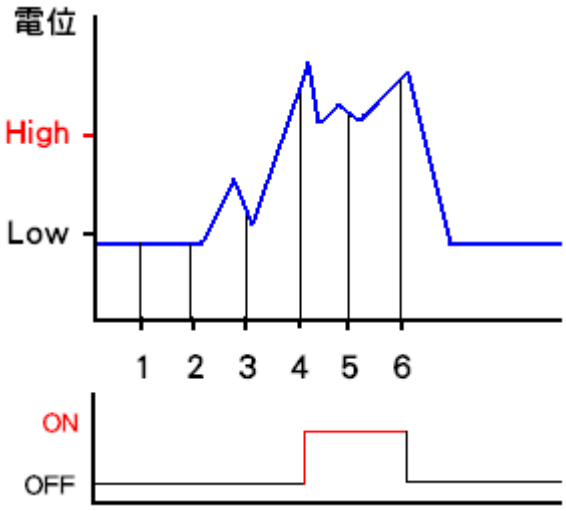


圖 25.1 Filter = 1 這個設定下，電位超過 High 連續 1 個 ms 就 ON 起來了，低於 High 連續 1 個 ms 就 OFF。

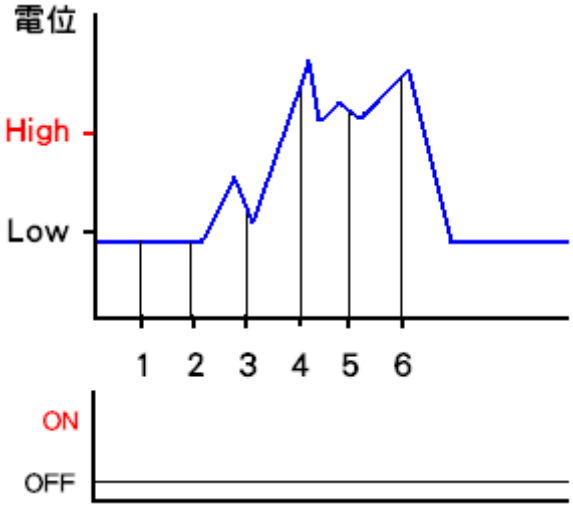


圖 25.2 Filter = 4 這個設定下，電位超過 High 連續 4 個 ms 才會 ON 起來，所以上圖都是 OFF 呈現。

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=2; //  此例值為 2，表示軟體濾波時間為 2ms

/*  設定 Remote 數位輸入模組 Port 0 的軟體濾波時間為 2ms */
I16 status = _DMC_01_set_rm_input_filter(CardNo, NodeID, SlotID, Port, Value);
```

DELTA CONFIDEN

25.4　_DMC_01_set_rm_input_filter_enable

■　**格式**

I16 PASCAL _DMC_01_set_rm_input_filter_enable(U16 CardNo, U16 NodeID,
U16 SlotID,U16 port,U16 enable)

■　**目的**

設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
enable	U16	數值	Port 0/1/2/3 bit0~bit15 濾波器遮罩(值：0~0xFFFF)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Enable=0xFF;

/*  設定 Remote 數位輸入模組 Port 0 濾波遮罩的 bit0~bit7 設置為 ON */

I16 status = _DMC_01_set_rm_input_filter_enable(CardNo, NodeID, SlotID, Port, Enable);
```

DELTA CONFIDENTIAL

25.5　_DMC_01_set_rm_output_value

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_value(U16 CardNo, U16 NodeID,U16 SlotID, U16 port,U16 value)

■ **目的**

設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	數值	Remote 數位輸出模組 Port 0/1/2/3 的 bit 0~bit 15 欲設定的值(ON/OFF)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=0xF;

/*  設定 Remote 數位輸入模組 Port 0 的 bit0~bit3 設置為 ON */
 I16 status = _DMC_01_set_rm_output_value(CardNo, NodeID, SlotID, Port, Value)
```

DELTA CONFIDENTIAL

25.6　_DMC_01_set_rm_output_single_value

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_single_value(U16 CardNo, U16 NodeID, U16 SlotID, U16 port, U16 bitno,U16 value)

■ **目的**

設定遠端擴充模組輸出埠單一 Bit 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
bitno	U16	選項單位	欲操作的 Bit(0~15) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
value	U16	數值	欲設定的值(ON/OFF)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 biton=0;
U16 Value=1;

/* 設定 Remote 數位輸入模組 Port 0 的 bit0 設置為 ON */
 I16 status = _DMC_01_set_rm_output_single_value (CardNo, NodeID, SlotID, Port, biton,
 Value)
```

DELTA CONFIDENTIAL

25.7　_DMC_01_set_rm_output_value_error_handle

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_value_error_handle(U16 CardNo, U16 NodeID, U16 SlotID,U16 port,U16 value)

■ **目的**

保存遠端擴充模組發生錯誤(離線)時的輸出值，直到解除 error_handle 或系統斷電。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	選項單位	0：錯誤發生時(離線)，輸出值將重置為 0 1：錯誤發生時(離線)，輸出值將被保存直到系統斷電

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=1;

I16 status = _DMC_01_set_rm_output_value_error_handle(CardNo, NodeID, SlotID, port, Value);
```

DELTA CONFIDENTIAL

25.8　_DMC_01_get_rm_output_value

■ **格式**

I16 PASCAL _DMC_01_get_rm_output_value (U16 CardNo, U16 NodeID, U16 SlotID, U16 port, U16* value)

■ **目的**

取得遠端擴充模組的輸出值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16*	數值	取得 Remote 數位輸出模組 Port 0/1/2/3 的 bit 0~bit 15 設定的值(ON/OFF)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

I16 status = _DMC_01_get_rm_output_value (CardNo, NodeID, SlotID, Port, &Value);
```

DELTA CONFIDENTIAL

25.9　_DMC_01_get_rm_output_single_value

■ **格式**

I16 PASCAL _DMC_01_get_rm_output_single_value(U16 CardNo, U16 NodeID, U16 SlotID, U16 port, U16 bitno, U16* value)

■ **目的**

取得遠端擴充模組的輸出埠單一 Bit 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
bitno	U16	選項單位	欲操作的 Bit(0~15) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
Value	U16*	數值	接收的資料(ON/OFF)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 biton=0;
U16 Value;
```

DELTA CONFIDENTIAL

I16 status = _DMC_01_get_rm_output_single_value (CardNo, NodeID, SlotID, Port,biton, &Value);

DELTA CONFIDENTIAL

25.10　_DMC_01_get_rm_output_value_error_handle

■ **格式**

I16 PASCAL _DMC_01_get_rm_output_value_error_handle (U16 CardNo, U16 NodeID, U16 SlotID,U16 port,U16* value)

■ **目的**

取得設定保存遠端擴充模組發生錯誤(離線)時的輸出值，直到解除 error_handle 或系統斷電。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16*	選項單位	取得 value 的值，當值為 0：錯誤發生時(離線)，輸出值將重置為 0 1：錯誤發生時(離線)，輸出值將被保存直到系統斷電

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

I16 status = _DMC_01_get_rm_output_value_error_handle(CardNo, NodeID, SlotID, port, &Value);
```

DELTA CONFIDENTIAL

25.11　_DMC_01_set_rm_output_active

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_active (U16 CardNo, U16 NodeID, U16 SlotID, U16 Enable)

■ **目的**

設定遠端擴充模組輸出是否開啟。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0：關閉輸出 1：啟動輸出，將設定的輸出值發送到目的端

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA	RM64MN
					●	●	

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Enable=1;

I16 status = _DMC_01_set_rm_output_active (CardNo, NodeID, SlotID, Enable);
```

DELTA CONFIDENTIAL

第二十六章　電子手輪與 JOG 操作 API

此章節說明了如何使用電子手輪與 JOG 模式程式介面(API)，設定電子手輪與 JOG 功能。電子手輪 API 提供介面讓使用者藉由模組端輸入 Pluse 提供馬達轉動的命令，軸卡端(上位機)只需針對馬達圈數比例參數，忠實的提供 command 值回傳給馬達，使馬達運轉即可。

表 26.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_rm_mpg_axes_enable	設定手輪運動控制。
A/B/F	_DMC_01_set_rm_mpg_axes_enable2	手輪運動控制（可設定馬達圈數比例分母值）。
A/B	_DMC_01_set_rm_mpg_axes_enable3	設定手輪運動控制(可設定馬達圈數比例之分母值)。
A/B/F	_DMC_01_set_rm_jog_axes_enable	設定 JOG 運動控制。
A02	_DMC_A02_set_mpg_axes_enable	設定手輪運動控制。
A02	_DMC_A02_mpg_get_io_state_enable	取得手輪的開和關狀態。
A02	_DMC_A02_mpg_fsm_get_lock_state	取得手輪的計數器是否為啟動狀態。
A02	_DMC_A02_mpg_get_poisition	取得目前手輪位置計數值。
A02	_DMC_A02_mpg_axis_get_io_state	取得目前所要控制的軸號。
A02	_DMC_A02_mpg_axis_get_ratio_state	取得手輪目前選擇的倍率。
A02	_DMC_A02_set_jog_axes_enable	設定手輪 JOG 硬體的控制模式。
A02	_DMC_A02_get_jog_io_state	讀取 JOG 輸出入狀態資料。
A02	_DMC_A02_set_mpg_axes_enable2	設定手輪運動控制(軟體模式)
A02	_DMC_A02_get_mpg_axes_enable2	取得手輪運動控制(軟體模式)設定狀態
A02	_DMC_A02_set_mpg_axes_extend	設定手輪運動控制(軟體模式)選擇 4 軸或 10 軸
A02	_DMC_A02_get_mpg_axes_extend	取得手輪運動控制(軟體模式)選擇 4 軸或 10 軸
A02	_DMC_A02_mpg_axis_get_io_state2	取得目前所控制的軸號
A02	_DMC_A02_mpg_axis_get_ratio_state	取得手輪目前選擇的倍率

CONFIDENTIAL

	2	
A02	_DMC_A02_set_mpg_axes_enable3	設定手輪運動控制(軟體模式)
A02	_DMC_01_Set_MPG_AMF	設定手輪濾波
B01	_DMC_Qep_set_mpg_axis_enable	設定手輪運動控制(軟體模式 MP1-A12D/PCI-DMC-B01 適用)
B01	_DMC_Qep_get_mpg_axis_enable	取得手輪運動控制設定(軟體模式 MP1-A12D/PCI-DMC-B01 適用)
B01	_DMC_Qep_get_mpg_axis_cnt	取得手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)
B01	_DMC_Qep_set_mpg_clear	清除手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)
MP1-P10D-15	_DMC_MP1_set_mpg_axis_enable	設定手輪運動控制(MP1-P10D-15 專用)
MP1-P10D-15	_DMC_MP1_get_mpg_axis_cnt	取得手輪計數值 (MP1-P10D-15 專用)
MP1-P10D-15	_DMC_MP1_set_mpg_axis_clear	清除手輪計數值 (MP1-P10D-15 專用)
MP1-P10D-15	_DMC_MP1_get_mpg_axis_enable	取得手輪運動控制設定 (MP1-P10D-15 專用)

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

26.1　_DMC_01_set_rm_mpg_axes_enable

■ **格式**

I16 PASCAL _DMC_01_set_rm_mpg_axes_enable (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable, U16 pulser_ratio, U32 *ratio, U32 *slop)

■ **目的**

啟動手輪運動控制(手輪啟動時伺服 P1-68 會調整成最大值 100，並記錄原始設定值，等待手輪關閉後寫回伺服驅動器端)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	選項單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_APIInputError	Rt = 310,　API 建構值索引參數輸入錯誤(超出限定值) 。
ERR_SVOFF	Rt = 322, 此動作需 Servo On 方可執行。

DELTA CONFIDENTIAL

■ **範例**

I16 rt = 0;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64 站號為 1
U16 NodeID[4] = {2, 3, 0, 0}, SlotID[4] = {0}; //使用兩個 ASD-A2F 伺服馬達, 站號分別為 2, 3
U16 enable = 1, pulser_ratio = 4; //設定手輪每格輸出 pulse 比例
U32 ratio[4] = {1, 1, 0, 0}; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值
U32 slop[4] = {100,100,0,0}; //設定接受手輪之最大速度為 100

rt = _DMC_01_set_rm_mpg_axes_enable(CardNo, MasterNodeID, MasterSlotID, NodeID, SlotID, enable, pulser_ratio, ratio, slop);

若同樣條件在步進馬達與 04PI，則結果如下：
手輪轉動一圈(100 格) = 脈波輸出值為 100*10=1000

例二：(ratio 的設定)
條件設定：假設手輪每轉 100 格， 手輪倍數 x10，pulser_ratio = 4，ratio=2， slop=1000
結果: 手輪轉動一圈(100 格) = 馬達轉動二圈(128000*10*2=2560000)
若同樣條件在步進馬達與 04PI，則結果如下：
手輪轉動一圈(100 格) = 脈波輸出值為 100*10=1000，ratio 值的設定與輸出脈波無關，只與手輪設定倍數有關。

例三：(slop 的設定)
條件設定：假設手輪每轉 100 格， 手輪倍數 x100，pulser_ratio = 4，ratio=1， slop=100
正確結果應該: 手輪轉動一圈(100 格) = 馬達轉動一圈(128000 * 100 * 1=12800000)
實際結果為: 手輪轉動一圈(100 格) = 馬達實際轉動(12531200)，由於設定接受手輪的最大速度為 100(PPS/秒)，所以手輪搖太快超出的部分被濾掉了。

例四：在步進馬達與 04PI，則結果如下：
條件設定：假設手輪每轉 100 格， 手輪倍數 X100，pulser_ratio = 4，ratio=1， slop=100
正確結果應該: 手輪轉動一圈(100 格) = 脈波輸出值應為 100 *100=10000
實際結果為: 手輪轉動一圈(100 格) = 脈波輸出為 3560，跟伺服一樣，速度超出的部分被濾掉了。

DELTA CONFIDENTIAL

26.2　_DMC_01_set_rm_mpg_axes_enable2

■ **格式**

I16 PASCAL _DMC_01_set_rm_mpg_axes_enable2 (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable,U16 pulser_ratio, U32 *ratio, U32 *slop, U16 *denominator)

■ **目的**

設定手輪運動控制（可設定馬達圈數比例之分母值）。
(手輪致能時伺服 P1-68 會調整成最大值 100，並記錄原值，等手輪除能時寫回伺服)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	選項單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
Ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分子)。
Slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。
Denominator	U16*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_APIInputError	Rt = 310,　API 建構值索引參數輸入錯誤(超出限定值) 。
ERR_SVOFF	Rt = 322, 此動作需 Servo On 方可執行。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt = 0;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64  站號為 1
U16 NodeID[4] = {2, 3, 0, 0}, SlotID[4] = {0}; //使用兩個 ASD-A2F  伺服馬達,  站號分別為 2, 3
U16 enable = 1, pulser_ratio = 4; //設定手輪每格輸出 pulse  比例
U32 ratio[4] = {10, 10, 0, 0}; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值
U32 slop[4] = {1000,1000,0,0}; //設定接受手輪之最大速度為  1000
U16 denominator [4] = {36,36, 36, 36}; //設定手輪每轉輸出比率(分母)

rt = _DMC_01_set_rm_mpg_axes_enable2(CardNo, MasterNodeID, MasterSlotID, NodeID,
SlotID, enable, pulser_ratio, ratio, slop, denominator);
```

DELTA CONFIDENTIAL

NOTE

經實際測試，若使用每轉一圈為 100pulse 的手輪，在高速轉動時，因受限於 RM64MN 的接收頻率，會發生無法接收之現象;建議客戶使用 25pulse/每轉，且有 4 倍頻的手輪，經測試，在高速轉動的情形下，不會發生 RM64MN 無法接收之現象。

以上述範例說明：

條件設定：

假設手輪轉 100 格, 手輪倍數 X10, ratio=10, Denominator=36, pulser_ratio =4, slop=1000, 電子齒輪比 P1.44/P1.45 = 1

手輪轉動 1 格 = 馬達轉動脈波數(**128000 * 10 / 100 * 10 / 36=355.55**) * **P1.45/P1.44**
* 正好可以控制馬達帶動的圓盤轉動 1 度(**1/360 * 1280000=355.55**)

公式:

手輪轉動一圈(100 格) =
馬達轉動脈波數(**128000 * 手輪倍數 * ratio / Denominator**) * **P1.45 / P1.44**。

換算比率參考:

手輪倍數	Ratio	Denominator	Slop	手輪轉數 (轉一圈)	伺服轉數
X1	1	1	1000	100	128000
X10	1	1	1000	1000	1280000
X100	1	1	1000	10000	12800000
X1	2	1	1000	100	256000
X1	1	2	1000	100	64000

與台達伺服 P1.44 及 P1.45 搭配範例: MPG 轉 X 格，伺服動 Y 個 Pulse

A: 基礎 P1.44 / P1.45 調整的比率值=640(1280000 / 2000)。
B: 基礎 et_rm_mpg_axes_enable2 中的 Ratio / Denominator 比率值=1。
C: 是目前 P1.44/p1.45 調整的比率值。
D: 待求值 set_rm_mpg_axes_enable2 中的 Ratio/Denominator 比率值。

演算式為: **D = B * C / A * Y / X → D = 0.5 * C / 640 * Y / X**

注意: 當您使用 Ratio/Denominator 比例值來調整模組輸出值時，
(手輪輸出一格 * 手輪倍數 * ratio / Denominator)得出模組輸出變化量，
其值將無條件刪去小數，僅得整數值輸出。

DELTA CONFIDENTIAL

26.3　_DMC_01_set_rm_mpg_axes_enable3

■ **格式**

I16 PASCAL _DMC_01_set_rm_mpg_axes_enable3 (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable,U16 pulser_ratio, U32 *ratio, U32 *slop, U32 *denominator)

■ **目的**

設定手輪運動控制（可設定馬達圈數比例之分母值）。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	選項單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
Ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分子)。
Slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒)。
Denominator	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_APIInputError	Rt = 310,　API 建構值索引參數輸入錯誤(超出限定值) 。
ERR_SVOFF	Rt = 322, 此動作需 Servo On 方可執行。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt = 0;
U16 CardNo = 0;
U16 MasterNodeID = 1;//RM-64  站號為 1
U16 MasterSlotID = 0;
U16 NodeID[4] = {1, 2, 3, 4}, SlotID[4] = {0}; //使用 ASD-A2F  伺服馬達
U16 enable = 1, pulser_ratio = 4; //設定手輪每格輸出 pulse  比例
U32 ratio[4] = {1, 1, 1, 1}; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值
U32 slop[4] = {1000,1000,1000,1000}; //設定接受手輪之最大速度為  1000
U32 denominator [4] = {1,1, 1, 1}; //設定手輪每轉輸出比率(分母)

rt = _DMC_01_set_rm_mpg_axes_enable3(CardNo, MasterNodeID, MasterSlotID, NodeID,
SlotID, enable, pulser_ratio, ratio, slop, denominator);

//此範例為手輪轉一格輸出 1 Pluse
```

NOTE

經實際測試，若使用每轉一圈為 100pulse 的手輪，在高速轉動時，因受限於 RM64MN 的接收頻率，會發生無法接收之現象;建議客戶使用 25pulse/每轉，且有 4 倍頻的手輪，經測試，在高速轉動的情形下，不會發生 RM64MN 無法接收之現象。

1. _DMC_01_set_rm_mpg_axes_enable3 調整手輪每轉與脈波輸出(馬達圈數)之倍數值 Denominator 為 U32

DELTA CONFIDENTIAL

DELTA CONFIDENTIAL

26.4　_DMC_01_set_rm_jog_axes_enable

■ **格式**

I16 PASCAL _DMC_01_set_rm_jog_axes_enable (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable, U16 jog_mode, I32 *jog_speed, F64 *sec)

■ **目的**

設定 JOG 運動控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸，站號不重複)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸,04PI,1 站四軸)
enable	U16	選項單位	0：關閉 JOG 功能 1：啟動 JOG 功能
jog_mode	U16	選項單位	JOG 軸選擇方式 0：RM 模式 1：MPG 模式
jog_speed	I32*	脈波數/秒	JOG 速度
sec	F64	脈波數/秒	JOG 加速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_APIInputError	Rt = 310,　API 建構值索引參數輸入錯誤(超出限定值) 。
ERR_SVOFF	Rt = 322, 此動作需 Servo On 方可執行。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64  站號為 1
U16 NodeID[4] = {2, 3, 4, 0}, SlotID[4] = {0}; //使用 3 個 ASD-A2F 伺服馬達，站號分別為 2, 3, 4
U16 enable = 1;
U16 jog_mode = 0; //使用 RM 模組 IO 點選擇 X, Y, Z 軸
I32 jog_speed[4] = {128000, 128000, 128000, 0}; //設定 JOG 速度為  128000 pps.
F64 sec = 0.1;

//馬達站號 2 為 X 軸，站號 3 為 Y 軸，站號 4 為 Z 軸，共計 3 軸
rt = _DMC_01_set_rm_jog_axes_enable (CardNo, MasterNodeID, MasterSlotID, NodeID,
SlotID, enable, jog_mode, jog_speed, sec);
```

DELTA CONFIDENTIAL

26.5　_DMC_A02_set_mpg_axes_enable

■　**格式**

I16 PASCAL _DMC_A02_set_mpg_axes_enable(U16 CardNo, U16* NodeID, U16* SlotID,U16 enable,U16 pulser_ratio,U32 *ratio,U32 *slop,U16 *denominator)

■　**目的**

設定手輪運動控制

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16*	編號單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16*	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U16	選項單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U16	選項單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。
Denominator	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值 (分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt = 0 ;
U16 slot[4]={0};
U16 enable = 1;
U16 PulseRatio = 4; //設定手輪每格輸出 pulse 比例。

U16 MPG_USE_gbNODE[4]={0,0,0,0};//使用的伺服馬達，最多 4 軸。

U32 ratio[4]={0,0,0,0} ; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值

U32 slop[4]={0,0,0,0};//設定接受手輪之最大速度為 100

U16 denominator[4]={0,0,0,0}; //調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

rt = _DMC_A02_set_mpg_axes_enable(gDMCCardNo, MPG_USE_gbNODE, slot, 1,
PulseRatio, ratio, slop, denominator );
```

對於資料結構定義如有疑問，請參閱 26.1 與 26.2 章節 有詳細參數說明與範例。

DELTA CONFIDENTIAL

26.6　_DMC_A02_mpg_get_io_state_enable

■ **格式**

I16 PASCAL _DMC_A02_mpg_get_io_state_enable(U16 CardNo, U8* nState);

■ **目的**

取得手輪的開和關狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U8*	編號單位	0:表示關閉 1:表示開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。

DELTA CONFIDENTIAL

26.7　_DMC_A02_mpg_fsm_get_lock_state

■ **格式**

I16 PASCAL _DMC_A02_mpg_fsm_get_lock_state(U16 CardNo, U8* nState);

■ **目的**

取得手輪的計數器是否為啟動狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U8*	編號單位	0:關閉狀態 1:開啟狀態

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。

DELTA CONFIDENTIAL

26.8　_DMC_A02_mpg_get_poisition

■　**格式**

I16 PASCAL _DMC_A02_mpg_get_poisition(U16 CardNo, U32* nValue);

■　**目的**

取得目前手輪位置計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nValue	U32*	編號單位	取得目前手輪位置計數值。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U32 nValue;

rt = _DMC_A02_mpg_get_poisition(U16 CardNo, &nValue);
```

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

NOTE

nValue 計數值型態為 U32 無號數，取正負值則需轉型為 I32 型態。

DELTA CONFIDENTIAL

26.9　_DMC_A02_mpg_axis_get_io_state

■ **格式**

I16 PASCAL _DMC_A02_mpg_axis_get_io_state(U16 CardNo, U8* nState);

■ **目的**

取得目前所要控制的軸號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	取得目前要控制的軸號，分別為 1、2、3、4 軸。對應的 X、Y、Z、W 等由使用者自行建立對應關係如下: [0]None [1]Axis#0 [2]Axis#1 [3]Axis#2 [4]Axis#3

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

26.10　_DMC_A02_mpg_axis_get_ratio_state

■ **格式**

I16 PASCAL　_DMC_A02_mpg_axis_get_ratio_state(U16 CardNo, U8* nState);

■ **目的**

取得手輪目前選擇的倍率。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U8*	編號單位	取得手輪目前選擇的倍率。 [0]None　[1]x1　[2]x10　[3]x100

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

26.11　_DMC_A02_set_jog_axes_enable

■　**格式**

I16 PASCAL _DMC_A02_set_jog_axes_enable(U16 CardNo, U16* NodeID, U16* SlotID,U16 enable,U16 jog_mode,I32 *jog_speed,F64 *sec);

■　**目的**

設定手輪 JOG 硬體的控制模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	編號單位	0：關閉手輪 JOG 功能 1：啟動手輪 JOG 功能
jog_mode	U16	編號單位	0: 6 個外部控制器觸發訊號。 1:配合手輪選擇軸與 X(+ 、-)方向 JOG 觸發訊號。
jog_speed	I32*	選項單位	設定脈波數
sec	F64*	選項單位	設定加速度時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

範例:

```
U16 rt = 0;

U16 slot[4]={0};//

U16 jog_mode =1;//由 6 個訊號源控制

I32 jog_speed[4]={0};//各軸 JOG 的脈波速度

F64 sec[4]={0.1, 0.1, 0.2, 0.2};//加速度時間

U16 JOG_USE_gbNODE[4]={0};//JOG 時使用的站號
```

```
rt = _DMC_A02_set_jog_axes_enable(gDMCCardNo, JOG_USE_gbNODE, slot, 1, jog_mode, jog_speed, sec);
```

DELTA CONFIDENTIAL

26.12　_DMC_A02_get_jog_io_state

■　**格式**

I16 PASCAL _DMC_A02_get_jog_io_state(U16 CardNo, U16* nState);

■　**目的**

讀取 JOG 輸出入狀態資料。(範例的軸向定義為自定義軸向，請自行參考實際應用軸)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U16*	編號單位	例: Bit0: X+,Bit1: X-, Bit2: Y+,Bit3: Y-, Bit4: Z+,Bit5: Z- (例:00001 = 1, 為 X+)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

26.13　_DMC_A02_set_mpg_axes_enable2

■　**格式**

I16 PASCAL _DMC_A02_set_mpg_axes_enable2(U16 CardNo, U16* NodeID, U16* SlotID,U16 enable,U16 pulser_ratio,U32 *ratio,U32 *slop,U16 *denominator)

■　**目的**

設定手輪運動控制(軟體模式)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16*	編號單位	操作的伺服站號(最多 10 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。
Denominator	U16*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值 (分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

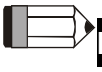
■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt = 0 ;
U16 slot[4]={0};
U16 enable = 1;
U16 PulseRatio = 4; //設定手輪每格輸出 pulse 比例。

U16 MPG_USE_gbNODE[4]={1,2,3,4};//使用的伺服馬達，最多 4 軸。

U32 ratio[4]={1,1,1,1} ; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值

U32 slop[4]={100,100,100,100};//設定接受手輪之最大速度為 100

U16 denominator[4]={1,1,1,1}; //調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

rt = _DMC_A02_set_mpg_axes_enable2(gDMCCardNo, MPG_USE_gbNODE, slot, 1,
PulseRatio, ratio, slop, denominator );
```



NOTE

_DMC_A02_set_mpg_axes_enable2 僅限 FPGA 1.0 版(2017/10)後使用

此 API 開啟後:
1.自動 Enable,硬體 Enable 腳位 don't care，可做一般 DI 使用
2.Ratio X1 硬體腳位 don't care，可做一般 DI 使用
3.若 Ratio X10 state=0,Ratio X100 state=0 ,此時默認為 Ratio X1
4.Jog 硬體 X+,X-, Y+,Y-, Z+,Z-,六支腳位切換為 Axis select,不具 Jog 功能

DELTA CONFIDENTIAL

26.14　_DMC_A02_get_mpg_axes_enable2

■　**格式**

I16 PASCAL _DMC_A02_get_mpg_axes_enable2(U16 CardNo, U16* Enable)

■　**目的**

取得手輪運動控制(軟體模式)設定狀態

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Enable	U16*	選項單位	0:已關閉 1:已啟動

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0
U16 Enable;

rt = _DMC_A02_get_mpg_axes_enable2( CardNo, &Enable);
```

NOTE

僅限 FPGA 1.0 版(2017/10)後使用

DELTA CONFIDENTIAL

26.15　_DMC_A02_set_mpg_axes_extend

■　**格式**

I16 PASCAL _DMC_A02_set_mpg_axes_extend(U16 CardNo, U16 AxisExtend)

■　**目的**

設定手輪運動控制(軟體模式)選擇 1 軸~10 軸

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisExtend	U16	數值單位	輸入範圍數值 0~10 0:default 4 軸 1:1 軸 2:2 軸 … 10:10 軸

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0
U16 AxisExtend=10;

rt = _DMC_A02_set_mpg_axes_extend(CardNo, AxisExtend)
```

NOTE

僅限 FPGA 1.0 版(2017/10)後使用

DELTA CONFIDENTIAL

26.16　_DMC_A02_get_mpg_axes_extend

■　**格式**

I16 PASCAL _DMC_A02_get_mpg_axes_extend(U16 CardNo, U16* AxisNum)

■　**目的**

取得手輪運動控制(軟體模式)選擇 1 軸~10 軸

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16*	數值單位	0:default 4 軸 1:1 軸 2:2 軸 … 10:10 軸

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0
U16 AxisNum;

rt = _DMC_A02_get_mpg_axes_extend(CardNo, &AxisNum)
```

NOTE

僅限 FPGA 1.0 版(2017/10)後使用

DELTA CONFIDENTIAL

26.17　_DMC_A02_mpg_axis_get_io_state2

■　**格式**

I16 PASCAL _DMC_A02_mpg_axis_get_io_state2(U16 CardNo, U8* nState);

■　**目的**

取得目前所控制的軸號。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U8*	選項單位	取得目前要控制的軸號，分別為 1~10 軸。 關係如下: [0]None [1]Axis#0 [2]Axis#1 [3]Axis#2 [4]Axis#3 [5]Axis#4 [6]Axis#5 [7]Axis#6 [8]Axis#7 [9]Axis#8 [10]Axis#9

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U8 nState = 0;

rt = _DMC_A02_mpg_axis_get_io_state2 (CardNo, &nState)
```

NOTE

僅限 FPGA 1.0 版(2017/10)後使用
若使用_DMC_A02_set_mpg_axes_enable2 啟動手輪，請搭配此 API 使用。

DELTA CONFIDENTIAL

26.18 _DMC_A02_mpg_axis_get_ratio_state2

■ **格式**

I16 PASCAL　_DMC_A02_mpg_axis_get_ratio_state2(U16 CardNo, U8* nState);

■ **目的**

取得手輪目前選擇的倍率。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
nState	U8*	編號單位	取得手輪目前選擇的倍率。 [[1]x1　[2]x10　[3]x100

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U8 nState = 0;

rt = _DMC_A02_mpg_axis_get_ratio_state2 (CardNo, &nState)
```

NOTE

僅限 FPGA 1.0 版(2017/10)後使用

若使用_DMC_A02_set_mpg_axes_enable2 啟動手輪，請搭配此 API 使用。

若 Ratio X10 state=0,Ratio X100 state=0 ,此時默認為 Ratio X1。

DELTA CONFIDENTIAL

26.19　_DMC_A02_set_mpg_axes_enable3

■　**格式**

I16 PASCAL _DMC_A02_set_mpg_axes_enable3(U16 CardNo, U16* NodeID, U16* SlotID,U16 enable,U16 pulser_ratio,U32* ratio,U32* slop,U32* denominator)

■　**目的**

設定手輪運動控制(軟體模式)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16*	編號單位	操作的伺服站號(最多 10 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒)。
Denominator	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值 (分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。
ERR_A02_MPG_Enable	Rt = 102,　A02 手輪已致能,無法重覆致能。
ERR_A02_JOG_Enable	Rt = 103,　A02 JOG 已致能,無法重覆致能。

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt = 0 ;
U16 slot[4]={0};
U16 enable = 1;
U16 PulseRatio = 4; //設定手輪每格輸出 pulse 比例。

U16 MPG_USE_gbNODE[4]={1,2,3,4};//使用的伺服馬達，最多 4 軸。

U32 ratio[4]={1,1,1,1} ; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值

U32 slop[4]={100,100,100,100};//設定接受手輪之最大速度為 100

U32 denominator[4]={1,1,1,1}; //調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

rt = _DMC_A02_set_mpg_axes_enable3(gDMCCardNo, MPG_USE_gbNODE, slot, 1,
PulseRatio, ratio, slop, denominator );
```

NOTE

_DMC_A02_set_mpg_axes_enable3 僅限 FPGA 1.0 版(2017/10)後使用
1. _DMC_01_set_rm_mpg_axes_enable3 調整手輪每轉與脈波輸出(馬達圈數)之倍數值 Denominator 為 U32

DELTA CONFIDENTIAL

26.20 _DMC_01_Set_MPG_AMF

■ **格式**

I16 PASCAL _DMC_01_Set_MPG_AMF(U16 CardNo, I16 AMFNo, I16 AMFNum, U16 FilterNum, U16 Enable);

■ **目的**

設定手輪濾波

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AMFNo	I16	編號單位	組數 0 ~ 32 手輪使用組數為 0
AMFNum	I16	數值單位	濾波次數 1~2
FilterNum	U16	數值單位	濾波時間 ms
Enable	U16	選項單位	0：關閉濾波功能 1：啟動濾波功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
I16 AMFNo = 0; //手輪使用第 0 組
I16 AMFNum = 2;
U16 FilterNum = 100;
U16 Enable = 1;

rt = _DMC_01_Set_MPG_AMF (CardNo, AMFNo, AMFNum, FilterNum, Enable);
```

NOTE

僅限 DLL 版本(2017/11)後，平滑使用手輪的震動情形。

DELTA CONFIDENTIAL

26.21　_DMC_Qep_set_mpg_axis_enable

■　**格式**

I16 PASCAL _DMC_Qep_set_mpg_axis_enable(U16 CardNo, U16 CompareChannel, U16 NodeID, U16 SlotID, U16 enable, U16 pulser_ratio, U32 ratio, U32 slop, U16 denominator)

■　**目的**

設定手輪運動控制(軟體模式 MP1-A12D/PCI-DMC-B01 適用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
CompareChannel	U16	選項單位	(PCI-DMC-B01) QEP Channel No 為 0~1
NodeID	U16	編號單位	操作的伺服站號(1~12 軸)
SlotID	U16	編號單位	操作的伺服插槽編號(目前未定義)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32	數值單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。
Denominator	U16	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U16 CompareChannel = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 enable = 1;
```

DELTA CONFIDENTIAL

U16 pulser_ratio = 4;
U32 ratio = 1;
U32 slop = 0;
Denominator = 1;

rt = _DMC_Qep_set_mpg_axis_enable(CardNo, CompareChannel, NodeID, SlotID, enable, pulser_ratio, ratio, slop, denominator);

DELTA CONFIDENTIAL

26.22　_DMC_Qep_get_mpg_axis_enable

■ **格式**

I16 PASCAL _DMC_Qep_get_mpg_axis_enable(U16 CardNo, U16* CompareChannel, U16* NodeID, U16* SlotID, U16* enable, U16* pulser_ratio, U32* ratio, U32* slop, U16* denominator)

■ **目的**

取得手輪運動控制設定(軟體模式 MP1-A12D/PCI-DMC-B01 適用)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
CompareChannel	U16*	選項單位	(PCI-DMC-B01) QEP Channel No 為 0~1
NodeID	U16*	編號單位	操作的伺服站號(1~12 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(目前未定義)
enable	U16*	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16*	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒，Max=1000)。
Denominator	U16*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U16 CompareChannel;
U16 NodeID;
U16 SlotID;
```

DELTA CONFIDENTIAL

U16 enable;
U16 pulser_ratio;
U32 ratio;
U32 slop;
Denominator;

rt = _DMC_Qep_set_mpg_axis_enable(CardNo, &CompareChannel, &NodeID, &SlotID, &enable, &pulser_ratio, &ratio, &slop, &denominator);

DELTA CONFIDENTIAL

26.23　_DMC_Qep_get_mpg_axis_cnt

■　**格式**

I16 PASCAL _DMC_Qep_get_mpg_axis_cnt(U16 CardNo, U16 compare_channel, I32* cnt)

■　**目的**

取得手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
CompareChannel	U16	選項單位	(PCI-DMC-B01) QEP Channel No 為 0~1
cnt	I32*	數值單位	手輪計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U16 CompareChannel=0;
I32 cnt;

rt = _DMC_Qep_get_mpg_axis_cnt(CardNo, compare_channel, &cnt);
```

DELTA CONFIDENTIAL

26.24　_DMC_Qep_set_mpg_clear

■　**格式**

I16 PASCAL _DMC_Qep_set_mpg_clear(U16 CardNo ,U16 compare_channel)

■　**目的**

清除手輪計數值 (軟體模式 MP1-A12D/PCI-DMC-B01 適用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
CompareChannel	U16*	選項單位	(PCI-DMC-B01) QEP Channel No 為 0~1

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U16 CompareChannel=0;

rt = _DMC_Qep_set_mpg_clear(U16 CardNo ,U16 compare_channel)
```

DELTA CONFIDENTIAL

26.25　_DMC_MP1_set_mpg_axis_enable

■　**格式**

I16 PASCAL _DMC_MP1_set_mpg_axis_enable(U16 CardNo, U16* NodeID, U16* SlotID, U16 enable, U16 pulser_ratio, U32* ratio, U32* slop, U32* denominator)

■　**目的**

設定手輪運動控制(MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0
NodeID	U16	編號單位	操作的伺服站號(最多 6 軸)
SlotID	U16	編號單位	操作的伺服插槽編號
enable	U16*	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16*	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U16	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U16	數值單位	設定接受手輪之最大速度 (PPS/秒)。
Denominator	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值 (分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_GPIO_PORTNO	Rt = 100,　FSM 是否未啟動。
ERR_JOG_AXISNO	Rt = 101,　軸選擇錯誤。

■　**範例**

DELTA CONFIDENTIAL

```
I16 rt = 0 ;
U16 slot[6]={0};
U16 enable = 1;
U16 PulseRatio = 4; //設定手輪每格輸出 pulse 比例。

U16 MPG_USE_gbNODE[6]={1,2,3,4,5,6};//使用的伺服馬達，最多 6 軸。

U32 ratio[6]={1,1,1,1,1,1} ; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值

U32 slop[6]={400,400,400,400,400,400};//設定接受手輪之最大速度

U32 denominator[6]={1,1,1,1,1,1}; //調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

rt = _DMC_MP1_set_mpg_axis_enable (gDMCCardNo, MPG_USE_gbNODE, slot, 1,
PulseRatio, ratio, slop, denominator);
```

//此範例為手輪轉一格輸出 1 Pluse

NOTE

僅限版號 0x73 MP1-P10D-15 機種使用

此 API 開啟後:
1.自動 Enable，硬體 Enable 腳位 don't care，可做一般 DI 使用。
2.Ratio X1 硬體腳位 don't care，可做一般 DI 使用。
3.若 Ratio X10 state=0,Ratio X100 state=0 ,此時默認為 Ratio X1。

DELTA CONFIDENTIAL

26.26　_DMC_MP1_get_mpg_axis_cnt

■　**格式**

I16 PASCAL _DMC_MP1_get_mpg_axis_cnt(U16 CardNo, I32* cnt)

■　**目的**

取得手輪計數值 (MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0
cnt	I32*	數值單位	手輪計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
I32 cnt;

rt = _DMC_MP1_get_mpg_axis_cnt (CardNo, &cnt);
```

NOTE

僅限版號 0x73 MP1-P10D-15 機種使用

DELTA CONFIDENTIAL

26.27　_DMC_MP1_set_mpg_axis_clear

■　**格式**

I16 PASCAL _DMC_MP1_set_mpg_axis_clear(U16 CardNo)

■　**目的**

清除手輪計數值 (MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;

rt = _DMC_MP1_set_mpg_axis_clear (U16 CardNo);
```

NOTE

僅限版號 0x73 MP1-P10D-15 機種使用

DELTA CONFIDENTIAL

26.28　_DMC_MP1_get_mpg_axis_enable

■　**格式**

I16 PASCAL _DMC_MP1_get_mpg_axis_enable(U16 CardNo, U16* NodeID, U16* SlotID, U16* enable, U16* pulser_ratio, U32* ratio, U32* slop, U32* denominator)

■　**目的**

取得手輪運動控制設定(MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0
NodeID	U16*	編號單位	操作的伺服站號(1~6 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(目前未定義)
enable	U16*	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16*	編號單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度 (PPS/秒)。
Denominator	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值 (分母)。

■　**屬性**

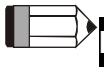
軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I16 rt = 0 ;
U16 CardNo = 0;
U16 NodeID[6] = {0};
U16 SlotID[6] = {0};
U16 enable;
U16 pulser_ratio;
U32 ratio[6] = {0};
```

DELTA CONFIDENTIAL

```
U32 slop[6] = {0};
U32 Denominator[6] = {0};

rt = _DMC_MP1_get_mpg_axis_enable( CardNo, NodeID, SlotID, &enable, &pulser_ratio,
ratio, slop, denominator);
```



僅限版號 0x73 MP1-P10D-15 機種使用

DELTA CONFIDENTIAL

第二十七章　四組脈波介面專用 API

此章節說明了如何使用四組脈波介面程式介面(API)，設定 RM04PI 與 GA01PH 模組脈波介面輸出入功能、模組的軸狀態控制。

表 27.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_rm_04pi_ipulser_mode	設定 RM04PI 脈波輸入模式
A/B/F	_DMC_01_set_rm_04pi_opulser_mode	設定 RM04PI 脈波輸出模式
A/B/F	_DMC_01_set_rm_04pi_svon_polarity	設定 POWER ON（SVON）的準位
A/B/F	_DMC_01_set_rm_04pi_DO2	設定 DO2 埠啟動
A/B/F	_DMC_01_set_rm_04pi_homing_ratio	設置原點復歸速度增量比 65535*(速度比例)
A/B/F	_DMC_01_rm_04PI_get_buffer	取得仍未執行的運動命令

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

27.1　_DMC_01_set_rm_04pi_ipulser_mode

■ **格式**

I16 PASCAL _DMC_01_set_rm_04pi_ipulser_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

使用 RM04PI 的模組當下可調整與設定脈波介面模組輸入相位模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：AB phase 1：CW / CCW

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //設置輸入相位為 AB phase

I16 status = _DMC_01_set_rm_04pi_ipulser_mode (CardNo, NodeID, SlotID, mode);
```

DELTA CONFIDENTIAL

27.2　_DMC_01_set_rm_04pi_opulser_mode

■ **格式**

I16 PASCAL _DMC_01_set_rm_04pi_opulser_mode (U16 CardNo, U16 NodeID, U16 SlotID ,U16 mode)

■ **目的**

使用 RM04PI 的模組當下可調整與設定脈波介面模組輸出相位模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：AB phase 1：CW / CCW 2：Pulse Direction DIR+ in low level 3：Pulse Direction DIR+ in high level

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **敘述**

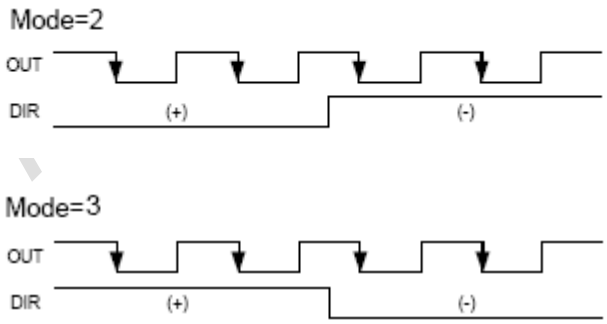


圖 27.1 參數 mode 功能示意

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //設置輸出相位為 AB phase
I16 status = _DMC_01_set_rm_04pi_opulser_mode (CardNo, NodeID, SlotID, mode);
```

DELTA CONFIDENTIAL

27.3　_DMC_01_set_rm_04pi_svon_polarity

■ **格式**

I16 PASCAL _DMC_01_set_rm_04pi_svon_polarity (U16 CardNo, U16 NodeID, U16 SlotID, U16 polarity)

■ **目的**

設定 POWER ON（SVON）的準位。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
polarity	U16	選項單位	0：Normal high 1：Normal low

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ON_OFF= 1;

U16 polarity = 0; //設置為 Normal high

/* 當驅動器為 Low active 時，此函式之 polarity 的值須設置為值 0（Normal high） */

status = _DMC_01_set_rm_04pi_svon_polarity(CardNo, NodeID, SlotID, polarity);

DELTA CONFIDENTIAL

27.4　_DMC_01_set_rm_04pi_DO2

■ **格式**

I16 PASCAL _DMC_01_set_rm_04pi_DO2 (U16 CardNo, U16 NodeID, U16 SlotID, U16 ON_OFF)

■ **目的**

啟動設定 DO2 埠。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ON_OFF	U16	選項單位	0：關閉 DO2 埠 1：啟動 DO2 埠

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ON_OFF=1; //  開啟 DO2 埠

I16 status = _DMC_01_set_rm_04pi_DO2 (CardNo, NodeID, SlotID, ON_OFF);
```

DELTA CONFIDENTIAL

27.5　_DMC_01_set_rm_04pi_homing_ratio

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_homing_ratio (U16 CardNo, U16 NodeID, U16 SlotID, U16 ratio)

■　**目的**

設置原點復歸速度增量比。

※使用 RM04PI 操作原點復歸，操作「_DMC_01_set_home_config」函式，該函式速度參數為 unsigned short 型別(16 bit)，最大速度僅能設定至 65535；故若欲使用大於 65535 速度值時，則必須使用此「_DMC_01_set_rm_04pi_homing_ratio」函式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ratio	U16	數值單位	回 Home 速度增量比

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ratio=100;

I16 status = _DMC_01_set_rm_04pi_homing_ratio( CardNo, NodeID, SlotID, ratio);
```

NOTE

欲運行原點復歸運動，操作「_DMC_01_set_home_config」函式，函式參數 lowSpeed 值為 200，highSpeed 值為 2000，此處 ratio 值為 100，則該 RM04PI 進行原復歸運動所使用的實際歸回原點的速度為：

DELTA CONFIDENTIAL

lowSpeed * ratio = 200 * 100 = 2000

highSpeed * ratio = 2000 * 100 = 200000

DELTA CONFIDENTIAL

27.6　_DMC_01_rm_04PI_get_buffer

■ **格式**

I16 PASCAL _DMC_01_rm_04PI_get_buffer(U16 CardNo,U16 NodeID, U16 SlotID, U16 *bufferLength)

■ **目的**

取得仍未執行的運動命令。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
bufferLength	U16*	選項單位	Buffer 中尚未執行的運動命令

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 bufferLength ;

I16 status = _DMC_01_rm_04PI_get_buffer( CardNo, NodeID, SlotID, &bufferLength);
```

DELTA CONFIDENTIAL

27.7　_DMC_01_set_rm_04pi_md2_feedback_mode

■　**格式**

I16 _stdcall _DMC_01_set_rm_04pi_md2_feedback_mode(U16 CardNo, U16 NodeID,U16 SlotID ,U16 mode)

■　**目的**

當 RM 模組 04PI md2 或是 01PH 無連接 Feedback 訊號時,模組發生警報時會將位置命令設為 0，直到警報解除。使用此 API 將 mode 設為 1 可使該站警報時將位置命令保持固定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	數值單位	Feedback 狀態 0:有連接　1:無連接

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 mode =50;
I16 status = DMC_01_set_rm_04pi_md2_feedback_mode(CardNo, NodeID, SlotID , mode)

DELTA CONFIDENTIAL

27.8 _DMC_01_set_rm_04pi_home_config_Tdec

■ **格式**

I16 _stdcall _DMC_01_set_rm_04pi_home_config_Tdec(U16 CardNo,U16 NodeID ,U16 SlotID ,U32 Tdec, U32 enable)

■ **目的**

當 RM 脈波模組原點複歸模式是選擇使用原點開關(ORG)作為觸發訊號時，在 DMC_01_set_home_move 指令下達前可使用此 API 設定由 High_Speed 切換至 Low_Speed 的減速時間。可減少複歸運動速度切換過於劇烈所產生的機構抖動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	U32	數值單位	ORG 觸發後的減速時間, 單位:ms
enable	U32	數值單位	enable 狀態 0: 無減速　1:有減速

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
U32 Tdec =1000, enable=1;
/* 設定原點復歸模式*/
I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed, highSpeed, acc);
I16 status = _DMC_01_set_rm_04pi_home_config_Tdec(CardNo, NodeID , SlotID , Tdec, enable);
/*啟用原點復歸*/
I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

DELTA CONFIDENTIAL

27.9 _DMC_01_set_rm_04pi_Home_counter_Delay_time

■ **格式**

I16 _stdcall _DMC_01_set_rm_04pi_Home_counter_Delay_time(U16 CardNo, U16 NodeID,U16 SlotID ,U16 Delay_time)

■ **目的**

當 RM 脈波模組原點複歸動作時，可由此 API 設定由 High_Speed 切換至 Low_Speed 且運動方向相反時的等待時間。觸發訊號為原點(ORG)或極限(PEL,MEL)皆有此功能。可減少複歸運動方向切換過於劇烈所產生的機構抖動。

(此為 SDO 指令寫入模組 0X622B 位置)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Delay_time	U16	數值單位	ORG 觸發後的減速時間 Default:10 數值範圍: 0~255 單位:ms

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
U26 Delay_time =250;
/* 設定原點復歸模式*/
I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed, highSpeed, acc);
I16 status = _DMC_01_set_rm_04pi_Home_counter_Delay_time(CardNo, NodeID, SlotID , Delay_time)
/*啟用原點復歸*/
I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

DELTA CONFIDENTIAL

27.10　_DMC_01_set_ref_counter

■ **格式**

I16 PASCAL _DMC_01_set_ref_counter (U16 CardNo, U16 NodeID,U16 SlotID , U16 mode)

■ **目的**

01PH 與 M04PI(模式 2)下，當模組重新連線後參考位置的選擇。

※使用此函式時，請確認是否有連結回授（Feedback）。

若有，mode 參數請設置為 1；反之如無接回授，則將該參數設置為 0。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：參考 command 計數值 1：參考 position 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //參考 command 計數值

I16 status =_DMC_01_set_ref_counter (CardNo, NodeID, SlotID, mode
```

DELTA CONFIDENTIAL

第二十八章　四組脈波介面（模式 1）運動控制 API

此章節說明了如何使用四組脈波介面（模式 1）運動程式介面(API)，設定 RM04PI（模式 1）模組的軸狀態控制，線性、圓弧、補間運動。

表 28.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04pi_md1_start_move	RM04PI 在模式 1 下，進行單軸運動位移
A/B/F	_DMC_01_rm_04pi_md1_v_move	RM04PI 在模式 1 下，進行速度運動控制
A/B/F	_DMC_01_rm_04pi_md1_start_line2	RM04PI 在模式 1 下，進行兩軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_line3	RM04PI 在模式 1 下，進行三軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_line4	RM04PI 在模式 1 下，進行四軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_arc	RM04PI 在模式 1 下，進行兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_rm_04pi_md1_start_arc2	RM04PI 在模式 1 下，進行兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_rm_04pi_md1_start_arc3	RM04PI 在模式 1 下，進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_rm_04pi_md1_start_heli	RM04PI 在模式 1 下，進行三軸螺旋補間運動
A/B/F	_DMC_01_rm_04pi_md1_p_change	RM04PI 在模式 1 下，運動中改變目標位置
A/B/F	_DMC_01_rm_04pi_md1_v_change	RM04PI 在模式 1 下，運動中改變速度
A/B/F	_DMC_01_rm_04pi_md1_set_gear	RM04PI 在模式 1 下，設定 Gear 功能啟動與其參數設置
A/B/F	_DMC_01_rm_04pi_md1_set_soft_limit	RM04PI 在模式 1 下，設定是否啟動軟體極限
A/B/F	_DMC_01_rm_04pi_md1_get_soft_limit_status	RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態
A/B/F	_DMC_01_rm_04pi_md1_set_sld	設定 SLD 埠（DI3）開啟與其相關配置
A/B/F	_DMC_01_rm_04pi_md1_get_mc_error_code	當 error code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息
A/B/F	_DMC_01_set_rm_04pi_ref_counter	RM04PI 在模式 1 下，選擇重新連線參考位置

CONFIDENTIAL

A/B/F	_DMC_01_Set_Stepping_Move_rate	設定脈波輸出解析放大的比率

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

28.1　_DMC_01_rm_04pi_md1_start_move

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行單軸運動位移。

※設置 StrVel 的值時，該值務必設定小於 MaxVel 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
Dist	I32	脈波數	指定的行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

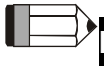
■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Dist = 1280000;
U16 StrVel = 1000, MaxVel =12800;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve = 1;U16 m_r_a = 0;

/* RM04PI MODE1 進行 T-curve 速度剖面相對座標位移運動 */
I16 status = _DMC_01_rm_04pi_md1_start_move (CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```



NOTE

RM04PI Md1 是一個站號下可以使用四個模組，通道只有一個，下指令時請特別檢查指令 _DMC_01_get_alm_code 的 alm_code 回傳值是否為 299， 如果回傳值為 299，請使用指令 _DMC_01_rm_04pi_md1_get_mc_error_code 檢查其回傳值 error_code 的值，error_code 回應值=127(代表 Buffer Full)。

DELTA CONFIDENTIAL

28.2　_DMC_01_rm_04pi_md1_v_move

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_v_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 dir,U16 m_curve)

■　**目的**

RM04PI 在模式 1 下，進行速度運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度(最小值 5pps)
Tacc	F64	秒	指定的加速時間
dir	I16	選項單位	0：進行正向速度位移 1：進行負向速度位移
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 StrVel = 1000, MaxVel =12800;
F64 Tacc=0.1;
I16 dir = 0; //  方向為正向
U16 m_curve = 1; //參照 T-curve 速度剖面

/* RM04PI MODE1  進行正向速度位移運動  */
I16 status = _DMC_01_rm_04pi_md1_v_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc,
dir, m_curve);
```

DELTA CONFIDENTIAL

28.3　_DMC_01_rm_04pi_md1_start_line2

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line2(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行兩軸線性補間運動。

※相關動作敘述說明，請參閱第 19 章「二軸線性補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID[2] ={0, 1};
I32 Dist[2]={30000, 40000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=2; //參照 S-curve 速度剖面
U16 m_r_a=1; //使用絕對座標位移

I16 status = _DMC_01_rm_04pi_md1_start_line2(CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

28.4　_DMC_01_rm_04pi_md1_start_line3

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line3(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行三軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數 對應 SlotID[2]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[3] ={0,1,2};
I32 Dist[3]={10000, 20000, 30000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1; //參照 T-curve 速度剖面
U16 m_r_a=1; //使用絕對座標位移

I16 status = _DMC_01_rm_04pi_md1_start_line3 (CardNo, NodeID, SlotID, Dist,StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.5　_DMC_01_rm_04pi_md1_start_line4

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line4(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行四軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	存取各種用來進行線性補間的站號 ID 數組
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID SlotID[3]存放第四組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數 對應 SlotID[2]運動的路徑參數 對應 SlotID[3]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[4] ={0,1,2,3};
I32 Dist[4]={10000, 20000, 30000, 40000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_line4 (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.6 _DMC_01_rm_04pi_md1_start_arc

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動。(已知條件：中心座標、角度)

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[2]={0,1};
I32 Center[2]={5000,5000}, StrVel=0, MaxVel=1000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_arc(CardNo, NodeID, SlotID, Center, Angle, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.7 _DMC_01_rm_04pi_md1_start_arc2

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc2(U16 CardNo, U16 NodeID,U16* SlotID, I32* End, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動。(已知條件：終點座標、角度)

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
End	I32*	脈波數	End[0]存放第一組終點位置 End[1]存放第二組終點位置
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[2]={0,1};
I32 End[2]={5000,5000}, StrVel=0, MaxVel=1000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_arc2(CardNo, NodeID, SlotID, End, Angle, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.8　_DMC_01_rm_04pi_md1_start_arc3

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc3(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center,I32* End, I16 dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve, U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置
End	I32*	脈波數	End[0]存放第一組終點位置 End[1]存放第二組終點位置
dir	I16	選項單位	指定的方向(值為 0 : 順時針方向 1 : 逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[2]={0,1}, m_curve=1, m_r_a=1;
I32 Center[2]={5000,2500}, End[2]={10000,5000}, StrVel=0, MaxVel=1000;
I16 dir=1; //表示往逆時針方向
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_rm_04pi_md1_start_arc3(CardNo, NodeID, SlotID, Center, End, dir, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.9　_DMC_01_rm_04pi_md1_start_heli

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_heli(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center,I32 Depth, I32 Pitch, I16 dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec, U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行三軸螺旋補間運動。

※相關動作敘述說明，請參閱第 22 章「三軸螺旋補間運動控制 API」說明。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置 Center[2]存放第三組圓心位置
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向的高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[3]={0,1,2}, m_curve=1, m_r_a=1;
I32 Center[3]={5000,2500,2500}, Depth = 2500, Pitch = 500, StrVel = 0, MaxVel = 1000;
I16 dir=1; //表示往逆時針方向
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_rm_04pi_md1_start_heli(CardNo, NodeID, SlotID, Center, Depth, Pitch, dir, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

DELTA CONFIDENTIAL

28.10　_DMC_01_rm_04pi_md1_p_change

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_p_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewPos)

■ **目的**

RM04PI 在模式 1 下，將設定的新位置值替換目前的位置(p_change 只能對單軸運動指令下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewPos	I32	脈波數	欲替換的位置參數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 NewPos=100000;

I16 status = _DMC_01_rm_04pi_md1_p_change (CardNo, NodeID, SlotID, NewPos);

DELTA CONFIDENTIAL

28.11　_DMC_01_rm_04pi_md1_v_change

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_v_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewSpeed, F64 sec)

■ **目的**

RM04PI 在模式 1 下，將設定的新速度值替換目前的速度(v_change 不能在減速段下達即 motion done=3 時)。

※關於其相關描述，請參閱第 18.6 節「_DMC_01_v_change」的描述項目。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewSpeed	I32	脈波數/秒	欲替換速度參數
sec	F64	秒	速度轉換時，指定的加/減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 NewSpeed=3000;
F64 sec=0.1;

I16 status = _DMC_01_rm_04pi_md1_v_change (CardNo, NodeID, SlotID, NewSpeed, sec)

DELTA CONFIDENTIAL

28.12　_DMC_01_rm_04pi_md1_set_gear

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_gear (U16 CardNo, U16 NodeID, U16 SlotID, I16 numerator,I16 denominator,U16 Enable)

■ **目的**

RM04PI 在模式 1 下，啟動設定 Gear 功能與其參數設置。

※由於步進驅動器無電子齒輪比功能，故可以此函式達到此功能需求。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
numberator	I16	數值單位	電子齒輪比分子
denominator	I16	數值單位	電子齒輪比分母
Enable	U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 numberator=1, denominator=2;
U16 Enable=1;

I16 status = _DMC_01_rm_04pi_md1_set_gear(CardNo, NodeID, SlotID, numerator, denominator, Enable);

DELTA CONFIDENTIAL

28.13　_DMC_01_rm_04pi_md1_set_soft_limit

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_soft_limit(U16 CardNo, U16 NodeID,U16 SlotID, I32 PLimit, I32 NLimit,U16 Enable)

■ **目的**

RM04PI 在模式 1 下，設定是否啟動軟體極限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit	I32	數值單位	軟體正極限設定值
NLimit	I32	數值單位	軟體負極限設定值
Enable	U16	選項單位	0：關閉軟體極限 1：啟動軟體極限

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0; U16 NodeID =1, SlotID=0;
I32 PLimit=5000, NLimit=-5000;
U16 Enable=1;

I16 status = _DMC_01_rm_04pi_md1_set_soft_limit (CardNo, NodeID, SlotID, PLimit, NLimit, Enable);

DELTA CONFIDENTIAL

28.14　_DMC_01_rm_04pi_md1_get_soft_limit_status

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_get_soft_limit_status (U16 CardNo, U16 NodeID, U16 SlotID ,U16* NLimit_status,U16* PLimit_status)

■ **目的**

RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>插槽 ID</td></tr>
<tr><td>NLimit_status</td><td>U16*</td><td>數值單位</td><td>傳回碰撞軟體負極限的狀態
<table><tr><th>數值</th><th>定義</th></tr><tr><td>1</td><td>Slot 0 碰撞軟體負極限</td></tr><tr><td>2</td><td>Slot 1 碰撞軟體負極限</td></tr><tr><td>4</td><td>Slot 2 碰撞軟體負極限</td></tr><tr><td>8</td><td>Slot 3 碰撞軟體負極限</td></tr><tr><td>….</td><td></td></tr><tr><td>15</td><td>Slot 0~3 都碰撞軟體負極限</td></tr></table></td></tr>
<tr><td>PLimit_status</td><td>U16*</td><td>數值單位</td><td>傳回碰撞軟體正極限的狀態
<table><tr><th>數值</th><th>定義</th></tr><tr><td>1</td><td>Slot 0 碰撞軟體正極限</td></tr><tr><td>2</td><td>Slot 1 碰撞軟體正極限</td></tr><tr><td>4</td><td>Slot 2 碰撞軟體正極限</td></tr><tr><td>8</td><td>Slot 3 碰撞軟體正極限</td></tr><tr><td>….</td><td></td></tr><tr><td>15</td><td>Slot 0~3 都碰撞軟體正極限</td></tr></table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0; U16 NodeID =1, SlotID=0;
U16 NLimit_status, PLimit_status;
I16 status = _DMC_01_rm_04pi_md1_get_soft_limit_status(CardNo, NodeID, SlotID ,
```

```
&NLimit_status, &PLimit_status);
```

DELTA CONFIDENTIAL

28.15　_DMC_01_rm_04pi_md1_set_sld

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_sld (U16 CardNo, U16 NodeID, U16 SlotID, I16 enable, I16 sd_logic,I16 mode)

■ **目的**

設定 SLD 埠（DI3）啟動與其相關配置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	選項單位	0：關閉 SLD 埠功能 1：啟動 SLD 埠功能
sd_logic	I16	選項單位	0：Normal high 1：Normal low
mode	I16	選項單位	1：使用 EMG stop（緊急停止） 2：使用 Slow down stop（減速停止）

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I16 enable=1;
I16 sd_logic=1;//埠操作極性為 Normal high
I16 mode=1; //使用的停止模式為 EMG stop

I16 status = _DMC_01_rm_04pi_md1_set_sld(CardNo, NodeID, SlotID, enable, sd_logic, mode);
```

DELTA CONFIDENTIAL

28.16　_DMC_01_rm_04pi_md1_get_mc_error_code

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_get_mc_error_code (U16 CardNo, U16 NodeID, U16 SlotID, U16 error_code)

■ **目的**

當 alarm code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_code	U16*	數值單位	運動控制錯誤訊息代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16* error_code;
U32* alm_code;

/* 取得 Slave 的錯誤訊息 */
I16 status = _DMC_01_rm_04pi_md1_get_mc_error_code (CardNo, NodeID, SlotID, &alm_code);
```

NOTE

當 alm_code 的值為 299 時，使用此函式。以取得 RM04PI 目前在運動控制位移中發生的錯誤訊息代碼。

_DMC_01_rm_04pi_md1_get_mc_error_code (CardNo, NodeID, SlotID, &error_code);

例：error_code 值為 127，表示" 運動指令 Buffer 已滿"（相關 Error Code 的詳細說明，請參閱 PCI_DMC_01_Err.h）

DELTA CONFIDENTIAL

28.17　_DMC_01_set_rm_04pi_ref_counter

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_ref_counter (U16 CardNo, U16 NodeID,U16 SlotID , U16 mode)

■　**目的**

M04PI 在模式 1 下，當模組重新連線後參考位置的選擇。

※使用此函式時，請確認是否有連結回授（Feedback）。

若有，mode 參數請設置為 1；反之如無接回授，則將該參數設置為 0。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：參考 command 計數值 1：參考 position 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //參考 command 計數值

I16 status =_DMC_01_set_rm_04pi_ref_counter (CardNo, NodeID, SlotID, mode);
```

DELTA CONFIDENTIAL

第二十九章　四組類比輸出遠端擴充模組 API

此章節說明了如何使用四組類比輸出程式介面(API)，設定 DA 模組的輸出訊號，DA 模組的腳位開關、輸出的範圍限制與狀態控制。

表 29.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04da_set_output_value	設定類比輸出輸出數值
A/B/F	_DMC_01_rm_04da_get_output_value	讀取類比輸出輸出數值
A/B/F	_DMC_01_rm_04da_get_return_code	讀取類比輸出狀態值
A/B/F	_DMC_01_rm_04da_set_output_range	設定類比輸出輸出範圍
A/B/F	_DMC_01_rm_04da_set_output_enable	設定腳位輸出開啟/關閉
A/B/F	_DMC_01_rm_04da_set_output_overrange	設定輸出範圍增加 10%
A/B/F	_DMC_01_rm_04da_set_output_error_clear	清除錯誤狀態
A/B/F	_DMC_01_rm_04da_set_output_error_handle	斷線保持原本設定狀態
A/B/F	_DMC_01_rm_04da_set_output_offset_value	設定類比輸出 offset 數值
A/B/F	_DMC_01_rm_04da_get_output_offset_value	讀取類比輸出 offset 數值

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

29.1　_DMC_01_rm_04da_set_output_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_value (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 Value)

■ **目的**

設定 DA 輸出數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Value	U16	數值單位	輸出值為 0 ~ 65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 Value = 0x5ff;

I16 status = _DMC_01_rm_04da_set_output_value (CardNo, NodeID, SlotID, ChannelNo, Value);

DELTA CONFIDENTIAL

29.2　_DMC_01_rm_04da_get_output_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_get_output_value(U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 *Value)

■ **目的**

讀取 DA 輸出數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Value	U16	數值單位	輸出值為 0 ~ 65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 Value = 0;

I16 status = _DMC_01_rm_04da_get_output_value(CardNo, NodeID, SlotID, ChannelNo,
&Value) ;
```

DELTA CONFIDENTIAL

29.3 _DMC_01_rm_04da_get_return_code

■ **格式**

I16 PASCAL _DMC_01_rm_04da_get_return_code (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16* ReturnCode)

■ **目的**

讀取 DA 狀態值。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>Data</td><td>U16</td><td>數值單位</td><td>取得傳回狀態參數
<table>
<tr><th>D3</th><th>D2</th><th>D1</th><th>D0</th></tr>
<tr><td>out enable</td><td colspan="3">out range mode</td></tr>
<tr><th>D7</th><th>D6</th><th>D5</th><th>D4</th></tr>
<tr><td colspan="2">return error code</td><td>error handle</td><td>over range</td></tr>
<tr><th>D11</th><th>D10</th><th>D9</th><th>D8</th></tr>
<tr><td colspan="4">return error code</td></tr>
<tr><th>D15</th><th>D14</th><th>D13</th><th>D12</th></tr>
<tr><td colspan="4">return error code</td></tr>
<tr><td colspan="4">Return Error Code:
0x2fe :HWFault　　0x2fc:OffsetOverRange
0x2fd:DAValueOverMax　　0x2fb: DAValueOverMin
0x2ef:OverTemp　　0x2ee:Slowactive
0x2ed:IOUTfault　　0x2e0:Multfault</td></tr>
</table></td></tr>
</table>

DELTA CONFIDENTIAL

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 1;
U16 ReturnCode = 0;

I16 status = _DMC_01_rm_04da_get_return_code (CardNo, NodeID, SlotID, ChannelNo,
&ReturnCode);
```

DELTA CONFIDENTIAL

29.4　_DMC_01_rm_04da_set_output_range

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16 Range)

■ **目的**

設定 DA 輸出範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Range	U16	數值單位	增益值為 0 ~ 7 數值 / 定義 0 / 輸出範圍: 0-5V (預設值) 1 / 輸出範圍: 0-10V 2 / 輸出範圍: ±5V 3 / 輸出範圍: ±10V 5 / 輸出範圍: 4-20mA 6 / 輸出範圍: 0-20mA 7 / 輸出範圍: 0-24mA

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 2;
U16 Range = 3; //選擇模式 3
I16 status = _DMC_01_rm_04da_set_output_range (CardNo, NodeID, SlotID, ChannelNo, range);
```

DELTA CONFIDENTIAL

29.5　_DMC_01_rm_04da_set_output_enable

■　格式

I16 PASCAL _DMC_01_rm_04da_set_output_enable (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 Enable)

■　目的

設定腳位輸出開啟/關閉。

■　參數

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Enable	U16	選項單位	0：輸出禁能 1：輸出致能。將設定的輸出值發送到目的端

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　範例

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 3;
U16 Enable = 1; //致能

I16 status = _DMC_01_rm_04da_set_output_enable (CardNo, NodeID, SlotID, ChannelNo, Enable);

DELTA CONFIDENTIAL

29.6　_DMC_01_rm_04da_set_output_overrange

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_overrange (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 On_Off)

■ **目的**

設定輸出範圍增加 10%。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	0：關閉 Overrange 1：啟動 Overrange

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 On_Off = 1; //啟動 OverRange。
// 輸出電壓範圍設定為-5V~5V 時，若啟動 OverRange，則輸出範圍擴大為 -5.5V~5.5V。

I16 status = _DMC_01_rm_04da_set_output_overrange (CardNo, NodeID, SlotID, ChannelNo, On_Off);

DELTA CONFIDENTIAL

29.7　_DMC_01_rm_04da_set_output_error_clear

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_error_clear (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 On_Off)

■ **目的**

清除錯誤狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	1：啟動 Clear Error

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 On_Off = 1; //Clear Error

I16 status = _DMC_01_rm_04da_set_output_error_clear (CardNo, NodeID, SlotID,
ChannelNo, On_Off);
```

DELTA CONFIDENTIAL

29.8　_DMC_01_rm_04da_set_output_error_handle

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_error_handle (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16 On_Off)

■ **目的**

斷線保持 DA 原本設定狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	0：關閉 ErrorHandle 1：啟動 ErrorHandle

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 ChannelNo = 3;
U16 On_Off = 1; //啟用 ErrorHandle

I16 status = _DMC_01_rm_04da_set_output_error_handle (CardNo, NodeID, SlotID,
ChannelNo, On_Off);
```

DELTA CONFIDENTIAL

29.9　_DMC_01_rm_04da_set_output_offset_value

■　**格式**

I16 PASCAL _DMC_01_rm_04da_set_output_offset_value (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, I16 Value)

■　**目的**

設定 DA offset 數值。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>SetValue</td><td>I16</td><td>選項單位</td><td>數值為 +127 ~ -128
■ 1 Step of Value = 38.14μv
■ Adjust the offset of the channel by -16 LSBs to +15.875 LSBs in increments of 1/8 LSB
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>127</td><td>Offset Adjustment: +15.875 LSBs (4.844mv)</td></tr>
<tr><td>126</td><td>Offset Adjustment: +15.75 LSBs (4.806mv)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>0</td><td>No Adjustment (default)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>-127</td><td>Offset Adjustment: -15.875 LSBs(- 4.844mv)</td></tr>
<tr><td>-128</td><td>Offset Adjustment: -16 LSBs(- 4.882mv)</td></tr>
</table></td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 ChannelNo = 0;
I16 Value = 0xf; //offset 設置為 0xf

I16 status = _DMC_01_rm_04da_set_output_offset_value (CardNo, NodeID, SlotID,
ChannelNo, Value);
```

DELTA CONFIDENTIAL

29.10 _DMC_01_rm_04da_get_output_offset_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_get_output_offset_value(U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, I16 *Value)

■ **目的**

讀取 DA offset 數值。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>SetValue</td><td>I16</td><td>選項單位</td><td>數值為 +127 ~ -128
■ 1 Step of Value = 38.14μv
■ Adjust the offset of the channel by -16 LSBs to +15.875 LSBs in increments of 1/8 LSB
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>127</td><td>Offset Adjustment: +15.875 LSBs (4.844mv)</td></tr>
<tr><td>126</td><td>Offset Adjustment: +15.75 LSBs (4.806mv)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>0</td><td>No Adjustment (default)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>-127</td><td>Offset Adjustment: -15.875 LSBs(- 4.844mv)</td></tr>
<tr><td>-128</td><td>Offset Adjustment: -16 LSBs(- 4.882mv)</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

DELTA CONFIDENTIAL

■ **範例**

U16 CardNo = 0, NodeID = 1, SlotId = 0;
U16 ChannelNo = 0;
I16 Value = 0;

I16 status = _DMC_01_rm_04da_get_output_offset_value (CardNo, NodeID, SlotID, ChannelNo, & Value);

DELTA CONFIDENTIAL

29.11 _DMC_01_rm_04da_read_data

■ **格式**

I16 PASCAL _DMC_01_rm_04da_read_data(U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* data)

■ **目的**

讀取 DA 輸出數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
data	U16*	數值單位	輸出值為 0 ~ 65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 data = 0;

I16 status = _DMC_01_rm_04da_read_data(CardNo, NodeID, SlotID, channelno, &data);
```

DELTA CONFIDENTIAL

29.12　_DMC_01_get_04da_Output_range

■ **格式**

I16 PASCAL _DMC_01_get_04da_Output_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 *range)

■ **目的**

取得目前 DA 的 Input 設定範圍。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>Range</td><td>U16</td><td>數值單位</td><td>增益值為 0 ~ 7
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>0</td><td>輸出範圍: 0-5V (預設值)</td></tr>
<tr><td>1</td><td>輸出範圍: 0-10V</td></tr>
<tr><td>2</td><td>輸出範圍: ±5V</td></tr>
<tr><td>3</td><td>輸出範圍: ±10V</td></tr>
<tr><td>5</td><td>輸出範圍: 4-20mA</td></tr>
<tr><td>6</td><td>輸出範圍: 0-20mA</td></tr>
<tr><td>7</td><td>輸出範圍: 0-24mA</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 range;

I16 status = _DMC_01_get_04da_Output_range (CardNo, NodeID, SlotID, channelno,
&range);
```

DELTA CONFIDENTIAL

第三十章　四組類比輸入遠端擴充模組 API

此章節說明了如何使用四組類比輸出程式介面(API)，設定 AD 模組的輸入訊號，AD 模組的轉換速度、平均電壓運算功能。

表 30.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_04ad_input_range	設定類比輸入的 Input 範圍
A/B/F	_DMC_01_get_04ad_input_range	取得目前 AD 的 Input 設定範圍
A/B/F	_DMC_01_set_04ad_conversion_time	設定 AD 轉換時間
A/B/F	_DMC_01_get_04ad_conversion_time	取得目前 AD 所設定的轉換時間
A/B/F	_DMC_01_get_04ad_data	讀取輸入之電壓數值
A/B/F	_DMC_01_set_04ad_average_mode	設定類比輸入平均功能之模式
A/B/F	_DMC_01_get_04ad_average_mode	讀取類比輸入平均功能之模式
A/B/F	_DMC_01_set_04ad_input_enable	設定 AD Channel 是否致能

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

30.1　_DMC_01_set_04ad_input_range

■ **格式**

I16 _DMC_01_set_04ad_input_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 range)

■ **目的**

設定 AD 的 Input 範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Range	U16	數值單位	輸入值為 0 ~ 5 數值 / 定義 0 / 輸入範圍:-10V~10V 1 / 輸入範圍:0V~10V 2 / 輸入範圍:-5V~5V 3 / 輸入範圍:0V~5V 4 / NULL 5 / 輸入範圍:0~20mA

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 0;
U16 range = 1; //選擇 Range -10 ~ 10V

I16 status = _DMC_01_set_04ad_input_range (CardNo, NodeID, SlotID, channelno, range);
```

DELTA CONFIDENTIAL

30.2　_DMC_01_get_04ad_input_range

■ **格式**

I16 _DMC_01_get_04ad_input_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 *range)

■ **目的**

取得目前 AD 的 Input 設定範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Range	U16	數值單位	取得回傳參數，值為 0 ~ 5 數值 / 定義 0 / 輸入範圍:-10V~10V 1 / 輸入範圍:0V~10V 2 / 輸入範圍:-5V~5V 3 / 輸入範圍:0V~5V 4 / NULL 5 / 輸入範圍:0~20mA

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 range;

I16 status = _DMC_01_get_04ad_input_range (CardNo, NodeID, SlotID, channelno, &range);
```

DELTA CONFIDENTIAL

30.3　_DMC_01_set_04ad_conversion_time

■ **格式**

I16 _DMC_01_set_04ad_conversion_time (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

設定 AD 轉換速度。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>Mode</td><td>U16</td><td>選項單位</td><td>輸入值為 0 ~ 6
<table>
<tr><th>數值</th><th>輸出比率(Hz)</th><th>-3 dB 頻率 (Hz)</th><th>RMS 雜訊 (μv)</th></tr>
<tr><td>0</td><td>372</td><td>200</td><td>9.6</td></tr>
<tr><td>1</td><td>1001</td><td>520</td><td>15.5</td></tr>
<tr><td>2</td><td>2005</td><td>1040</td><td>22.7</td></tr>
<tr><td>3</td><td>2534</td><td>1300</td><td>26.1</td></tr>
<tr><td>4</td><td>4826</td><td>2500</td><td>39.2</td></tr>
<tr><td>5</td><td>6041</td><td>3100</td><td>46.0</td></tr>
<tr><td>6</td><td>12166</td><td>6300</td><td>120.0</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 mode = 1;
I16 status = _DMC_01_set_04ad_conversion_time(CardNo, NodeID, SlotID, mode);
```

DELTA CONFIDENTIAL

30.4　_DMC_01_get_04ad_conversion_time

■ **格式**

I16 _DMC_01_get_04ad_conversion_time (U16 CardNo, U16 NodeID, U16 SlotID, U16* mode)

■ **目的**

取得目前 AD 所設定的轉換速度。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>Mode</td><td>U16</td><td>選項單位</td><td>輸出值為 0 ~ 6
<table>
<tr><th>數值</th><th>輸出比率(Hz)</th><th>-3 dB 頻率 (Hz)</th><th>RMS 雜訊 (μv)</th></tr>
<tr><td>0</td><td>372</td><td>200</td><td>9.6</td></tr>
<tr><td>1</td><td>1001</td><td>520</td><td>15.5</td></tr>
<tr><td>2</td><td>2005</td><td>1040</td><td>22.7</td></tr>
<tr><td>3</td><td>2534</td><td>1300</td><td>26.1</td></tr>
<tr><td>4</td><td>4826</td><td>2500</td><td>39.2</td></tr>
<tr><td>5</td><td>6041</td><td>3100</td><td>46.0</td></tr>
<tr><td>6</td><td>12166</td><td>6300</td><td>120.0</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 mode;
I16 status = _DMC_01_get_04ad_conversion_time(CardNo, NodeID, SlotID, &mode);
```

DELTA CONFIDENTIAL

30.5　_DMC_01_get_04ad_data

■ **格式**

I16 _DMC_01_get_04ad_data (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* value)

■ **目的**

讀取輸入之電壓數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
ReturnData	U16	數值單位	0~65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 channelno = 2;
U16 value;

I16 status = _DMC_01_get_04ad_data (CardNo, NodeID, SlotID, channelno, &value);
```

DELTA CONFIDENTIAL

30.6　_DMC_01_set_04ad_average_mode

■ **格式**

I16 _DMC_01_set_04ad_average_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 mode)

■ **目的**

設定 AD 平均功能之模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Mode	U16	數值單位	輸入數值 0~5 (輸入數值 N 表示 2^N 次數)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 mode = 1;

I16 status = _DMC_01_set_04ad_average_mode(CardNo, NodeID, SlotID, channelno, mode);
```

DELTA CONFIDENTIAL

30.7　_DMC_01_get_04ad_average_mode

■ **格式**

I16 _DMC_01_get_04ad_average_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* mode)

■ **目的**

讀取 AD 平均功能之模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Mode	U16	數值單位	讀取數值 0~5 (讀取數值 N 表示 2^N 次數)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1, mode;

I16 status = _DMC_01_get_04ad_average_mode (CardNo, NodeID, SlotID,
channelno,&mode);
```

DELTA CONFIDENTIAL

30.8　_DMC_01_set_04ad_input_enable

■ **格式**

I16 _DMC_01_set_04ad_input_enable (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 ON_OFF)

■ **目的**

設定 AD Channel 是否致能 Input。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	數值單位	0:Disable　1:Enable

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 ON_OFF=1;

I16 status = _DMC_01_set_04ad_input_enable (CardNo, NodeID, SlotID, channelno, ON_OFF);

DELTA CONFIDENTIAL

第三十一章 Slave 資訊 API

此章節說明了如何取得 Slave 資訊(API)，取得 Slave 裝置的類型、版本號與本次號。

表 31.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_devicetype	獲取 slave 裝置類型
A/B/F	_DMC_01_get_slave_version	獲取 slave 裝置韌體版本
A/B/F	_DMC_F01_get_slave_subversion	獲取 Salve 韌體版本次版號

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

DELTA CONFIDENTIAL

31.1 _DMC_01_get_devicetype

■ **格式**

I16 PASCAL _DMC_01_get_devicetype (U16 CardNo, U16 NodeID, U16 SlotID, U32 *DeviceType, U32 *IdentityObject)

■ **目的**

取得 slave 裝置類型。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
DeviceType	U32*	數值	Salve 裝置類型 （參考後頁 note 說明）
IdentityObject	U32*	數值	裝置的物件字典代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;
U32 DeviceType, IdentityObject;

I16 status = _DMC_01_get_devicetype (CardNo, NodeID, SlotID, &DeviceType, &IdentityObject);

DELTA CONFIDENTIAL

NOTE

※關於 DeviceType 變數代表裝置類型的代碼，請參考下表：

Device Type	Code	Device Type	Code
伺服驅動器 A2F 系列	0x04020192	伺服驅動器 M 系列	0x06020192
伺服驅動器 A2R 系列	0x08020192	伺服驅動器 S 系列	0x09020192
伺服驅動器 B2F 系列	0x05020192	伺服驅動器 A3F 系列	0x07020192
伺服驅動器 B3F 系列	0x0B020192	N/A	N/A
ASD-DMC-RM32MN	0x04110191	ASD-DMC-RM64MN	0x08110191
ASD-DMC-RM32NT	0x04120191	ASD-DMC-RM64NT	0x08120191
ASD-DMC-RM04PI(MD1)	0x1C100191	ASD-DMC-RM04AD	0x08140191
ASD-DMC-RM04PI(MD2)	0x14100191	ASD-DMC-RM04DA	0x08180191
ASD-DMC-RM32PT	0x04130191	ASD-DMC-GE01PH	0x11210191
ASD-DMC-GE16MN	0x08230191 【註 1】	ASD-DMC-GE16NT	0x08230191 【註 1】

【註 1】ASD-DMC-GE 的 IO 模組(GE16MN/GE16NT)，其為 64 點 IO 計為

1 個 Device type，即 1 個 Node。

※關於 IdentityObject 變數代表各類裝置物件字典代碼，請參考下表：

Object Dictionary	Code
伺服驅動器 A2F 系列	0x6000
伺服驅動器 A3F 系列	0x6010
伺服驅動器 M 系列	0x6030
伺服驅動器 A2R 系列	0x6040
伺服驅動器 S 系列	0x6050
伺服驅動器 B2F 系列	0x6070
伺服驅動器 B3F 系列	0x6080
Remote 模組	0x1000
Gateway 模組	0x2000

DELTA CONFIDENTIAL

31.2 _DMC_01_get_slave_version

■ **格式**

I16 PASCAL _DMC_01_get_slave_version(U16 CardNo, U16 NodeID,U16 SlotID, U16* version)

■ **目的**

取得 slave(模組或伺服)裝置韌體版本。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
version	U16*	版本編號	Salve 韌體版本

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 version;

I16 status = _DMC_01_get_slave_version(CardNo, NodeID, SlotID, &version);

DELTA CONFIDENTIAL

31.3 _DMC_01_get_slave_subversion

■ **格式**

I16 PASCAL _DMC_01_get_slave_subversion (U16 CardNo, U16 NodeID, U16 SlotID, U16 *SubVersion)

■ **目的**

取得 slave(模組或伺服)裝置版本次版號。(僅 F01 支援)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
SubVersion	U16*	版本編號	Salve 韌體版本次版號

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 SubVersion;

I16 status = _DMC_01_get_slave_subversion (CardNo, NodeID, SlotID ,&SubVersion);
```

DELTA CONFIDENTIAL

第三十二章 監控參數 API

此章節說明了如何取得監控參數(API)，可設定與監控 Slave 的設定命令狀態與取得使用者所設定的 Slave 狀態。

表 32.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_monitor	設定欲監控的參數
A/B/F	_DMC_01_get_monitor	取得監控參數的數值
A/B/F	_DMC_01_get_monitor_I32	取得監控參數的數值
A/B/F	_DMC_01_get_servo_command	取得伺服驅動器發送的命令值
A/B/F	_DMC_01_get_servo_DI	取得伺服驅動器 DI 訊號的值
A/B/F	_DMC_01_get_servo_DO	取得伺服驅動器 DO 訊號的值

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

DELTA CONFIDENTIAL

32.1 _DMC_01_set_monitor

■ **格式**

I16 PASCAL _DMC_01_set_monitor (U16 CardNo, U16 NodeID, U16 SlotID, U16 monitorw)

■ **目的**

設定欲監控的參數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
monitorw	U16	數值	欲監控的參數值(U8+U8)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

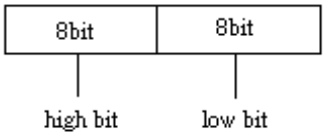
■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;
U16 monitorw=0x0101; //脈波命令輸入脈波數(電子齒輪比之後)—資料長度(32bit)

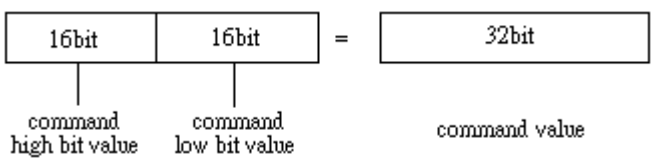
I16 status = _DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);

NOTE

在_DMC_01_set_monitor monitorw=0x0101 (脈波命令輸入脈波數**)** monitorw **說明如下:**



DMC_01_get_monitor value(32bit)說明如下



DELTA CONFIDENTIAL

※monitorw 的值請參閱下表 32.2

表 32.2

監控項目索引	監控項目	單位	資料長度	04PI Servo
0	馬達迴授脈波數(電子齒輪比之後)	使用者單位	32-bit[有號數]	■ ■
1	脈波命令輸入脈波數(電子齒輪比之後)	使用者單位	32-bit[有號數]	■ ■
2	控制命令脈波與迴授脈波誤差數	使用者單位	32-bit[有號數]	□ ■
3	馬達迴授脈波數	128 萬 Pulse/rev	32-bit[有號數]	□ ■
4	脈波命令輸入脈波數(電子齒輪比之前)	脈波數	32-bit[有號數]	□ ■
5	誤差脈波數(電子齒輪之後)	脈波數	32-bit[有號數]	□ ■
6	脈波命令輸入頻率	轉發封包數 K/秒	16-bit[有號數]	□ ■
7	馬達轉速	0.1 轉/分	32-bit[有號數]	□ ■
8	速度輸入命令	0.01 伏特	16-bit[有號數]	□ ■
9	速度輸入命令	0.1 轉/分	32-bit[有號數]	□ ■
10	扭矩輸入命令	0.01 伏特	16-bit[有號數]	□ ■
11	扭矩輸入命令	百分比(%)	16-bit[有號數]	□ ■
12	平均轉矩	百分比(%)	16-bit[有號數]	□ ■
13	峰值轉矩	百分比(%)	16-bit[有號數]	□ ■
14	主迴路電壓 (BUS 電壓)	伏特	16-bit[無號數]	□ ■

DELTA CONFIDENTIAL

15	負載/馬達慣性比	0.1 倍	16-bit[無號數]	□ ■
16	IGBT 溫度	^{o}C	16-bit[有號數]	□ ■
17	共振頻率	赫茲	16-bit[無號數]	□ ■
18	Z 相原點處的數值(脈波數)	-5000 ~ +5000	16-bit[有號數]	□ ■
19	映射參數#1：P0 - 25	數值	32-bit[無號數]	□ ■
20	映射參數#2：P0 - 26	數值	32-bit[無號數]	□ ■
21	映射參數#3：P0 - 27	數值	32-bit[無號數]	□ ■
22	映射參數#4：P0 - 28	數值	32-bit[無號數]	□ ■
23	監視變數#1：P0 - 09	數值	32-bit[有號數]	□ ■
24	監視變數#2：P0 - 10	數值	32-bit[有號數]	□ ■
25	監視變數#3：P0 - 11	數值	32-bit[有號數]	□ ■
26	監視變數#4：P0 – 12	數值	32-bit[有號數]	□ ■
27	Z 相原點處的數值(電子齒輪比之後)	-半圈 ~ +半圈	32-bit[有號數]	□ ■
28	異警碼	數值	32-bit[無號數]	■ ■
39	DI 狀態		16-bit[無號數]	■ ■
40	DO 狀態		16-bit[無號數]	■ ■
120	DMCNET 通訊狀態		16-bit[無號數]	□ ■
121	DMCNET 運轉時封包錯誤計數		32-bit[無號數]	□ ■
122	DMCNET 運轉時間	秒	16-bit[無號數]	□ ■

驅動器 A2F 輔助編碼器讀取方法:
可透過映射參數的方式由 set_monitor 取得,

(1).驅動器參數 P0-35 的參數內容會映射到 P0-25:
將 P5-17(輔助編碼器)設置到 P0-35,
(0x11 = 17)
P0-35 設置 0x05110511 (可參考驅動器 A2 手冊 P0-25 映射參數部分)

(2).下 API 指令
參數設置可參考(PCI-DMC_ProgrammingGuide_Traditional 32.1)
monitorw = 0x1313; (0x13 = 19)
_DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);

DELTA CONFIDENTIAL

32.2 _DMC_01_get_monitor

■ **格式**

I16 PASCAL _DMC_01_get_monitor (U16 CardNo, U16 NodeID, U16 SlotID, U32 *value)

■ **目的**

取得監控的參數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
value	U32*	數值	監控參數目前的數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
U16 monitorw=0x2728; //DI/DO 狀態—資料長度(DI-16bit-Hi Bit,DO-16bit-Low Bit)
U32 value;
I16 status = _DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);
status = _DMC_01_get_monitor (CardNo, NodeID, SlotID, &value);
```

DELTA CONFIDENTIAL

32.3 _DMC_01_get_monitor_I32

■ **格式**

I16 PASCAL _DMC_01_get_monitor_I32 (U16 CardNo, U16 NodeID, U16 SlotID, I32 *value)

■ **目的**

取得監控的參數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
value	I32*	數值	監控參數目前的數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

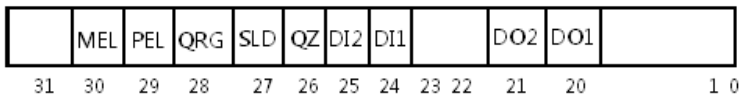
```
U16 CardNo=0, NodeID=1, SlotID=0;
U16 monitorw=0x2728; //DI/DO 狀態—資料長度(DI-16bit-Hi Bit,DO-16bit-Low Bit)
I32 value;
I16 status = _DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);
status = _DMC_01_get_monitor_I32 (CardNo, NodeID, SlotID, &value);
```

DELTA CONFIDENTIAL

NOTE

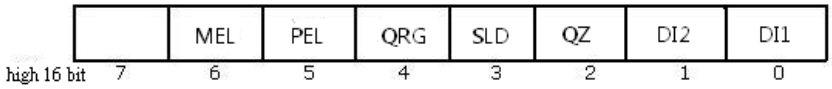
※在伺服下: _DMC_01_set_monitor monitorw=0x27(DI 狀態)或 0x28(DO 狀態) get_monitor Value 就會是伺服的 DI 或 DO 的值。

※在 RM04PI Model1 下：_DMC_01_set_monitor，monitorw=0x27(DI 狀態)或 0x28(DO 狀態) get_monitor Value 值說明如下(都會回 32bit Data):

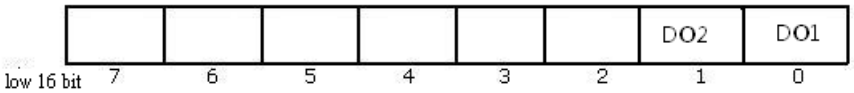


※在 RM04PI Model2(GE01PH)下

在_DMC_01_set_monitor，monitorw=0x27(DI 狀態) get_monitor Value 值說明如下:



在_DMC_01_set_monitor，monitorw=0x28(DO 狀態) get_monitor Value 值說明如下:



DELTA CONFIDENTIAL

32.4 _DMC_01_get_servo_command

■ **格式**

I16 PASCAL _DMC_01_get_servo_command (U16 CardNo, U16 NodeID, U16 SlotID, U32 *servo_cmd)

■ **目的**

取得伺服驅動器發送的命令值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_cmd	U32*	數值	驅動器命令值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 servo_cmd;

/* servo_cmd 的值為取回 servo 位置 command 值 */
I16 status = _DMC_01_get_servo_command (CardNo, NodeID, SlotID, &servo_cmd);
```

DELTA CONFIDENTIAL

32.5 _DMC_01_get_servo_DI

■ **格式**

I16 PASCAL _DMC_01_get_servo_DI (U16 CardNo, U16 NodeID, U16 SlotID, U16 *servo_DI)

■ **目的**

取得伺服驅動器(RM04PI_Md2)DI 訊號的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_DI	U16*	數值	驅動器上(RM04PI_Md2)DI1~DI8 訊號的值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 servo_DI;

I16 status = _DMC_01_get_servo_DI (CardNo, NodeID, SlotID, &servo_DI);
```

NOTE

04PI_Md2(01PH)、Servo、Servo_DI 說明

		MEL	PEL	QRG	SLD	QZ	DI2	DI1
Bit	7	6	5	4	3	2	1	0

DELTA CONFIDENTIAL

32.6　_DMC_01_get_servo_DO

■ **格式**

I16 PASCAL _DMC_01_get_servo_DO (U16 CardNo, U16 NodeID, U16 SlotID, U16 *servo_DO)

■ **目的**

取得伺服驅動器(04PI_Md2)DO 訊號的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_DO	U16*	數值	驅動器上(RM04PI_Md2)DO1~DO5 訊號的值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 servo_DO;

I16 status = _DMC_01_get_servo_DO (CardNo, NodeID, SlotID, &servo_DO);
```

NOTE

04PI_Md2(01PH)、Servo　Servo_DO 說明

DELTA CONFIDENTIAL

第三十三章　警報訊息 API

此章節說明了如何取得警報參數(API)，可設定與監控 Slave 的 ALARM 狀態，使用者可隨時偵測裝置是否有錯誤訊息，程式操作時一般都將 API 放置在 Timer 內掃描 slave 狀態。

表 33.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_ralm	輸出的伺服驅動器警示訊息重置
A/B/F	_DMC_01_get_alm_code	取得 Slave 的錯誤資訊
A/B/F	_DMC_01_master_alm_code	取得 Master Card 連線的錯誤資訊
A/B/F	_DMC_01_slave_error	取得 slave 通訊中連續錯誤次數
A/B/F	_DMC_01_set_ralm_servo_all	重置軸卡串接伺服驅動器之警示訊息
A/B/F	_DMC_01_set_limit_auto_ralm	重置軸卡串接之伺服驅動器(包含 RM04PI)由正反極限產生的警示訊息

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

33.1　_DMC_01_set_ralm

■ **格式**

I16 PASCAL _DMC_01_set_ralm (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

輸出的伺服驅動器(包含 RM04PI)警示訊息重置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;

I16 status = _DMC_01_set_ralm (CardNo, NodeID, SlotID);
```

NOTE

由於_DMC_01_set_ralm 這個指令執行，需要一些時間，如果習慣在下運動指令之前都去執行 _DMC_01_set_ralm，那麼常會造成系統反應不及時，請特別注意用法。

DELTA CONFIDENTIAL

33.2　_DMC_01_get_alm_code

■ **格式**

I16 PASCAL _DMC_01_get_alm_code (U16 CardNo, U16 NodeID, U16 SlotID,
U32 *alm_code)

■ **目的**

取得 Slave 的錯誤資訊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
alm_code	U32*	數值單位	Slave 的錯誤代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
U32 alm_code;

/* alm_code 的值，表示 Slave 的錯誤資訊，關於驅動器的錯誤碼，請參閱驅動器相關手冊 */
I16 status = _DMC_01_get_alm_code (CardNo, NodeID, SlotID, &alm_code);
```

NOTE

非驅動器部分，Error Code 說明如下：

錯誤代碼	錯誤敘述
0x09	**位置控制誤差量大於設定容許值時**
0x14	**負極限開關訊號觸發**
0x15	**正極限開關訊號觸發**
0x97	**外部 ALARM 訊號觸發(01PH)**
0x283	**軟體正極限訊號**
0x285	**軟體負極限訊號**
0x299	**請參照** 28.16 節 _DMC_01_rm_04pi_md1_get_mc_error_code **使用**

DELTA CONFIDENTIAL

33.3 _DMC_01_master_alm_code

■ **格式**

I16 PASCAL _DMC_01_master_alm_code(U16 CardNo,U16* alm_code)

■ **目的**

取得 Master Card 連線的錯誤資訊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
alm_code	U16*	數值	Master Card 連線失敗的錯誤代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 alm_code;

I16 status = _DMC_01_master_alm_code (CardNo , &alm_code);
```

NOTE

錯誤代碼一覽

錯誤代碼	錯誤敘述
0	正常
0X185 == 389	通訊不穩定

DELTA CONFIDENTIAL

33.4　_DMC_01_slave_error

■ **格式**

I16 PASCAL _DMC_01_slave_error (U16 CardNo, U16 NodeID,U16 SlotID,U16* alm_cnt)

■ **目的**

取得 slave 通訊中連續錯誤次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
alm_cnt	U16*	次數	通訊中 slave 連續錯誤的次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 alm_cnt;

I16 status = _DMC_01_slave_error (CardNo, NodeID, SlotID, &alm_cnt);
```

DELTA CONFIDENTIAL

33.5　_DMC_01_set_ralm_servo_all

■ **格式**

I16 PASCAL _DMC_01_set_ralm_servo_all (U16 CardNo)

■ **目的**

重置該卡串接伺服驅動器之警示訊息

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=1;

I16 status = _DMC_01_set_ralm_servo_all (CardNo);

DELTA CONFIDENTIAL

33.6 _DMC_01_set_limit_auto_ralm

■ **格式**

I16 _ PASCAL _DMC_01_set_limit_auto_ralm(U16 CardNo,U16 enable)

■ **目的**

在運動指令下達前,自動重置該卡串接之伺服驅動器(包含 RM04PI)由正反極限產生的警示訊息

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
enable	U16	編號單位	1: 啟動, 0:除能

:

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=1;

I16 status =_DMC_01_set_limit_auto_ralm(CardNo, 1)

DELTA CONFIDENTIAL

第三十四章　多軸運動控制 API

此章節說明了如何設置多軸運動控制 (API)，可設置多軸運動控制、多軸直線運動速度與 Endvel 的多軸運動控制。

表 34.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_multi_axes_move	設置 2 軸以上的運動控制
A/B/F	_DMC_01_liner_speed_master	多軸直線運動運動速度設定
A/B/F	_DMC_01_start_v3_multi_axes	增加 EndVel 的多軸(2 軸以上)運動控制
A/B/F	_DMC_01_start_v3_msbrline_move	指定 EndVel 的多軸(2 軸以上)運動控制。
A/B/F	_DMC_01_Start_Multi_Arc	多軸圓弧直線相對/絕對運動，可指定任意兩軸作為圓弧運動的 X 和 Y 軸，其餘軸做多軸直線運動。可選擇隨速(跟隨圓弧)或合速模式。

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

34.1 _DMC_01_multi_axes_move

■ **格式**

I16 PASCAL _DMC_01_multi_axes_move(U16 CardNo,U16 AxisNum, U16* NodeID, U16* SlotID, I32 *DistArrary, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve, U16 m_r_a)

■ **目的**

設置 2 軸以上的運動控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DELTA CONFIDENTIAL

■ **範例**

```
U16 CardNo=0, AxisNum=4;
U16 NodeID[4]={1,2,3,4}, SlotID[4]={0,0,0,0};
I32 DistArrary[4]={1000, 2000, 3000, 6000};
I32 StrVel=1000, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

/*設置為參照 T-curve 速度剖面下的相對座標進行多軸運動控制。*/
I16 status = _DMC_01_multi_axes_move(CardNo, AxisNum, NodeID, SlotID, DistArrary,
StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

34.2　_DMC_01_liner_speed_master

■　**格式**

I16 PASCAL _DMC_01_liner_speed_master (U16 CardNo, U16 NodeID, U16 SlotID, U16 Mode)

■　**目的**

多軸直線運動(Line2、Line3、Multi_Axis)運動速度設定，原本(Mode=0)運動速度的設定是向量速度，(Mode=1)的運動速度設定為行走距離最遠軸的最大分量速度(只對 Master 軸下達即可，其它軸速度會根據 Master 軸的值自動算出)。

■　**參數**

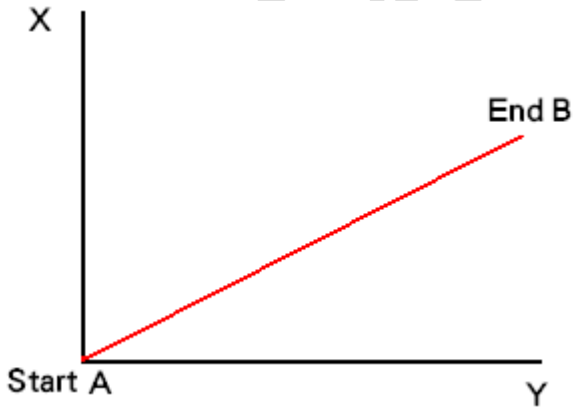
名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：向量速度(Default 設定)。 1：行走距離最遠軸的最大分量速度。 2：指定第一軸分量速度。

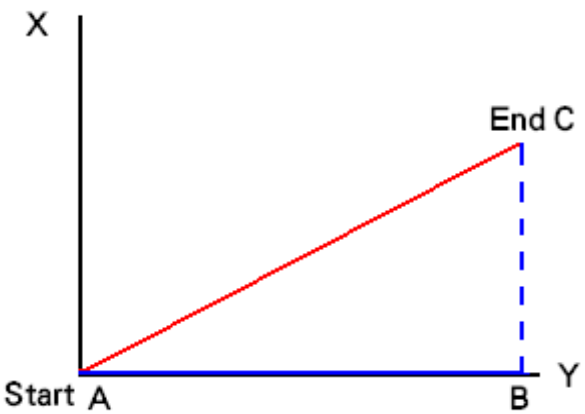
■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

Mode 參數設定示意





ADVANTECH CONFIDENTIAL

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Mode=1;

I16 status = _DMC_01_liner_speed_master (CardNo, NodeID, SlotID, Mode);
```

DELTA CONFIDENTIAL

34.3　_DMC_01_start_v3_multi_axes

■　**格式**

I16 PASCAL _DMC_01_start_v3_multi_axes(U16 CardNo,U16 AxisNum, U16* NodeID, U16* SlotID, I32 *DistArrary, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的多軸(2 軸以上)運動控制。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

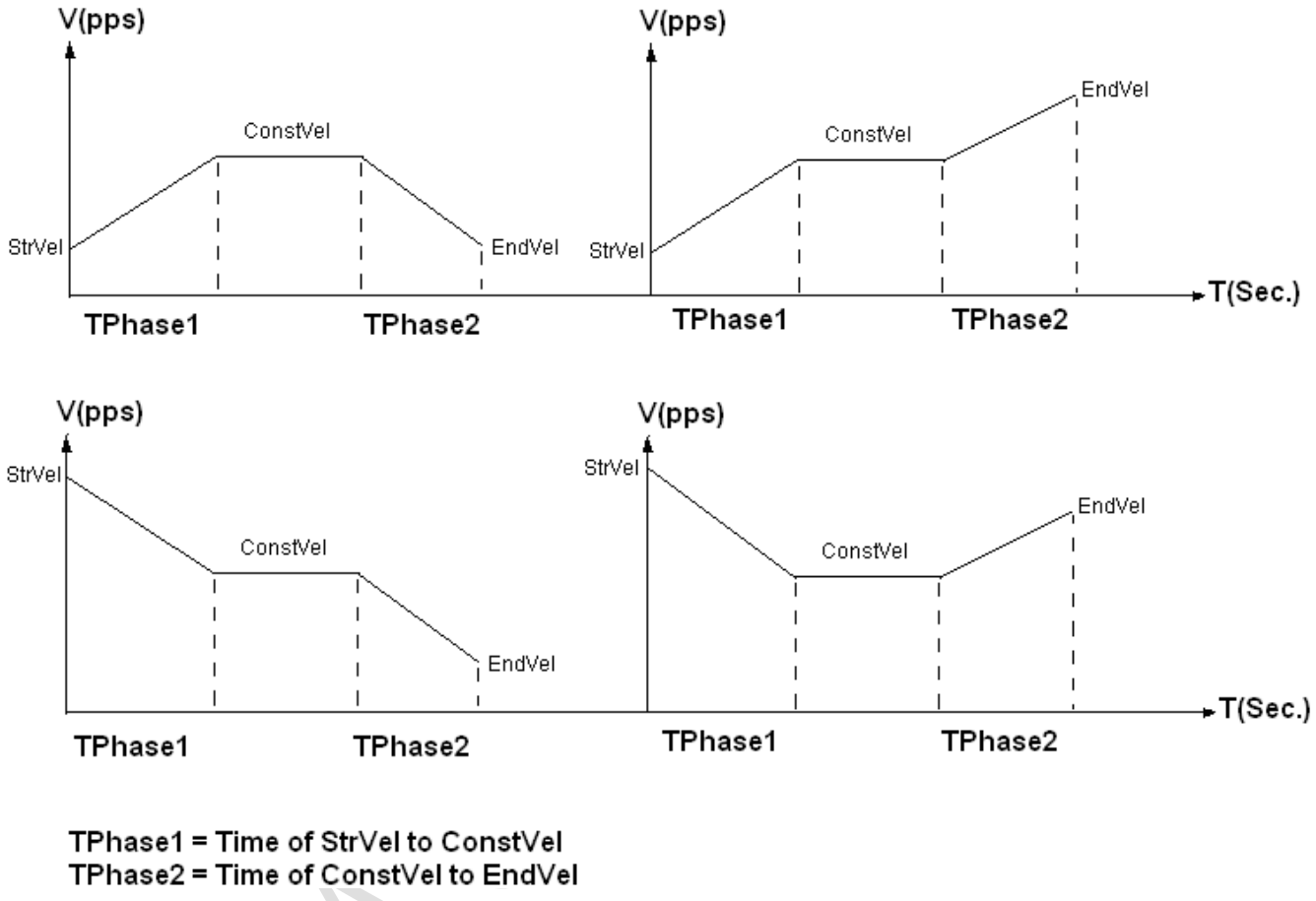
名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

DELTA CONFIDENTIAL

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 34.3 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0, AxisNum=4;
U16 NodeID[4]={1,2,3,4}, SlotID[4]={0,0,0,0};
I32 DistArrary[4]={1000, 2000, 3000, 6000};
I32 StrVel=1000, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_multi_axes (CardNo, AxisNum, NodeID, SlotID, DistArrary,
                StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

DELTA

34.4 _DMC_01_start_v3_msbrline_move

■ **格式**

I16 PASCAL _DMC_01_start_v3_msbrline_move(U16 CardNo,U16 AxisNum, U16* NodeID,U16* SlotID,U16 ArcNodeBit,I32* DistArray,I32* DistArray2,U16 Mode,F64 Parameter,F64 SpdRatio,F64 Angle1,F64 Angle2,I32 StrVel,I32 ConstVel,I32 EndVel,F64 Tphase1,F64 Tphase2,U16 m_curve,U16 m_r_a);

■ **目的**

指定 EndVel 的多軸(2 軸以上)運動控制。移動的路徑是由目前位置經過第一組目標位置到達第二組目標位置所形成的弧線。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
ArcNodeBit	U16*	編號單位	判斷銜接的運動是否為 Arc 預設值為 0 Bit0~7：代表 M1 的站號 1~8，如果是 Arc 軸則值為 1，否則 0，Bit8~15 代表 M2 同義
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
DistArrary2	I32*	脈波數陣列	DistArrary2 [0] 存放第一組運動的行程 DistArrary2 [1] 存放第二組運動的行程 DistArrary2 [2] 存放第三組運動的行程
Mode	U16	選項單位	0:M1、M2 均為直線, (Default) 1:M1 為直線、M2 為圓弧, 2:M1 為圓弧、M2 為直線, 3:M1、M2 為圓弧
Parameter	F64	選項單位	行程誤差量，誤差定義為與第一目標點的最短

CONFIDENTIAL

			距離。此值過大時，將會自動調整至合理範圍 (固定起始點以及第二目標點)
SpdRatio	F64	選項單位	M1 與 M2 的速度比率，相當於 SpdM2 / SpdM1。調整兩段運動上面的速度變化。 預設值為 0
Angle1	F64	選項單位	當 M1 為 Arc 時的角度， M1 為 Line 此值不會被使用。預設值為 0
Angle2	F64	選項單位	當 M2 為 Arc 時的角度， M2 Line，此值不會被使用。預設值為 0
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

DELTA CONFIDENTIAL

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**

可將兩個有折角的運動(Motion1(簡稱 m1) & Motion2(簡稱 m2))(無論圓或直線)合併成為一個運動(MSBR)的函式

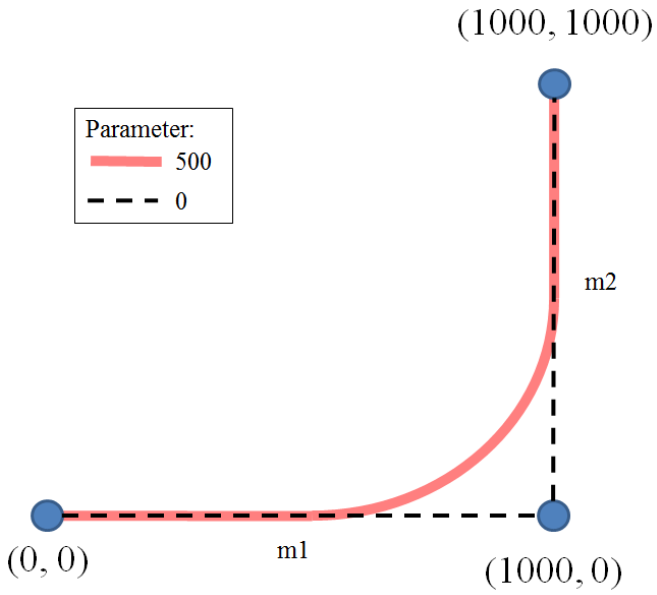


圖 34.40 Parameter 說明

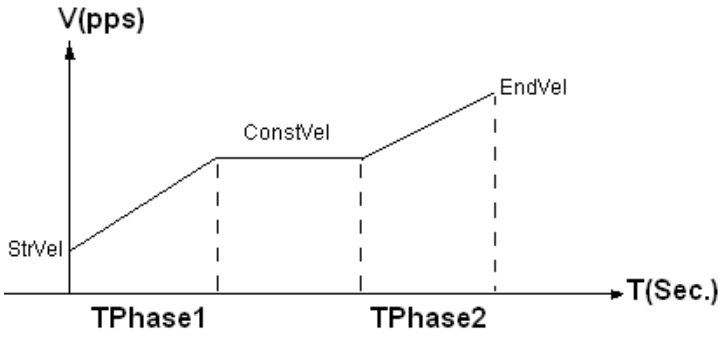


圖 34.41 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0, AxisNum=3;
U16 NodeID[3]={1,2,3}, SlotID[3]={0};
I32 DistArrary[12]={1000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
I32 DistArrary2[12]={1000, 1000, 1000, 0, 0, 0, 0, 0, 0, 0, 0, 0};
U16 Mode=0;
F64 Parameter=500, SpdRatio=0, Angle1=0, Angle2=0;
I32 StrVel=1000, ConstVel =50000, EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=1;

I16 status = _DMC_01_start_v3_msbrline_move(CardNo, AxisNum, NodeID, SlotID,
                ArcNodeBit, DistArray, DistArray2, Mode, Parameter, SpdRatio, Angle1, Angle2,
                StrVel, ConstVel, EndVel, Tphase1, Tphase2, m_curve, m_r_a);
```

DELTA CONFIDENTIAL

34.5 _DMC_01_p_change_multi_axes

■ **格式**

I16 PASCAL _DMC_01_p_change_multi_axes(U16 CardNo,U16 AxisNum ,U16* NodeID, U16* SlotID, I32 *position, I32 MaxVel, I32 EndVel, F64 Tacc, F64 Tdec, I16 Curve_mode)

■ **目的**

以新的位置值替換目前的位置設置(能對多軸運動指令下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxisNum	U16	軸數	多軸行程數量
NodeID	*U16	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	*U16	編號單位	插槽 ID
position	*I32	脈波數	各軸欲替換的位置參數(絕對位置) position [0] 存放第一組運動的行程 position [1] 存放第二組運動的行程 position [2] 存放第三組運動的行程
MaxVel	I32	脈波數/秒	最大速度
EndVel	I32	脈波數/秒	運動結束速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
Curve_mode	I16	選項單位	1：T-curve(Default) 2：S-curve

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

DIT CONFIDENTIAL

■ **範例**

```
U16 CardNo=0, AxisNum=3;
U16 NodeID[3]={1,2,3}, SlotID[3]={0,0,0 };
I32 position [3]={25000000, 15000000, 10000000 };
I32 MaxVel=4500000, EndVel =0;
F64 Tacc=0.1, Tdec=0.1;
I16 Curve_mode =1;

/*設置為參照 T-curve 速度剖面下的絕對座標多軸變位置功能。*/
I16 status = _DMC_01_p_change_multi_axes(CardNo, AxisNum , NodeID, SlotID, position,
MaxVel, EndVel, Tacc, Tdec, Curve_mode);
```

DELTA CONFIDENTIAL

34.6　_DMC_01_Start_Multi_Arc

■　**格式**

I16 PASCAL _DMC_01_Start_Multi_Arc(U16 CardNo, U16 AxesNum, U16* AxisArray, U16* SlotID, I32* DistArray, U16 SpeedMode, U16 ArcMode, U16 *Axis_index, I32 *CenterPoint, I32 *EndPoint, F64 Angle, U16 Dir, I32 StrVel, I32 ConstVel, I32 EndVel, F64 Tacc, F64 Tdec, U16 Curve_mode, U16 AbsMode)

■　**目的**

多軸圓弧直線相對/絕對運動，可指定任意兩軸作為圓弧運動的 X 和 Y 軸，其餘軸做多軸直線運動。可選擇隨速(跟隨圓弧)或合速模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
AxesNum	U16	數值單位	參與軸數（≧2）。Max:12
AxisArray	*U16	編號陣列	軸數組，數量等同於 AxisNum。 AxisArray[0] 存放第一組站號 AxisArray[1] 存放第二組站號 ．．．等。
DistArray	*I32	脈波數陣列	運行直線運動軸所需執行的行程陣列，數量等同於 AxesNum – 2 (不包含運行 Arc 的軸) 。
SpeedMode	U16	選項單位	0：隨速（直線跟隨 Arc 的合速）。 1：合速（直線和 Arc 做合速）。
ArcMode	U16	選項單位	選擇 Arc 模式。 0：已知圓心、角度（Arc1）。 1：已知末點、角度（Arc2）。 2：已知圓心、末點（Arc3）。 3：已知中間點、末點（Arc4）。
Axis_index	*U16	編號陣列	指定兩軸數組內做 Arc 運動的軸的位置。 Ex: AxisArray={1,2,3,4},指定{1,2}軸做 Arc 運動，則 index 需填{0,1}。 AxisArray={1,2,3,4},指定{2,3}軸做 Arc 運動，則 index 需填{1,2}。 Axis_index[0], Axis_index[1]不可相等。

TA CONFIDENTIAL

CenterPoint	*I32	脈波數	1.圓心位置 2.ArcMode=3 時此參數定義為中間點位置
EndPoint	*I32	脈波數	末點位置。
Angle	F64	度	圓弧的角度 正值代表往順時針方向旋轉。 負值代表往逆時針方向旋轉。
Dir	U16	選項單位	ArcMode=2 時使用，需填運動方向。 0：順時針選轉。 1：逆時針旋轉。
StrVel	I32	脈波數/秒	運動初始速度參數。
ConstVel	I32	脈波數/秒	運動等速度參數。
EndVel	I32	脈波數/秒	運動結束速度參數。
Tacc	F64	秒	初始速度到等速度的時間。
Tdec	F64	秒	等速度到結束速度的時間。
Curve_mode	U16	選項單位	0：T-curve(Default) 1：T-curve 2：S-curve
AbsMode	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
I16 CardNo = 0;
U16 AxisNum = 3;
I16 AxisArray[3]= { 1, 2, 3 };
I32 DistArray[0] = { 300000 };
U16 SpeedMode = 0;
U16 ArcMode = 0;
U16 Arc_index[2] = { 1, 2 };
I32 CenterPoint[2] = { 100000, 100000 };
I32 EndPoint[2] = { 0, 0 };
F64 Angle = 180;
```

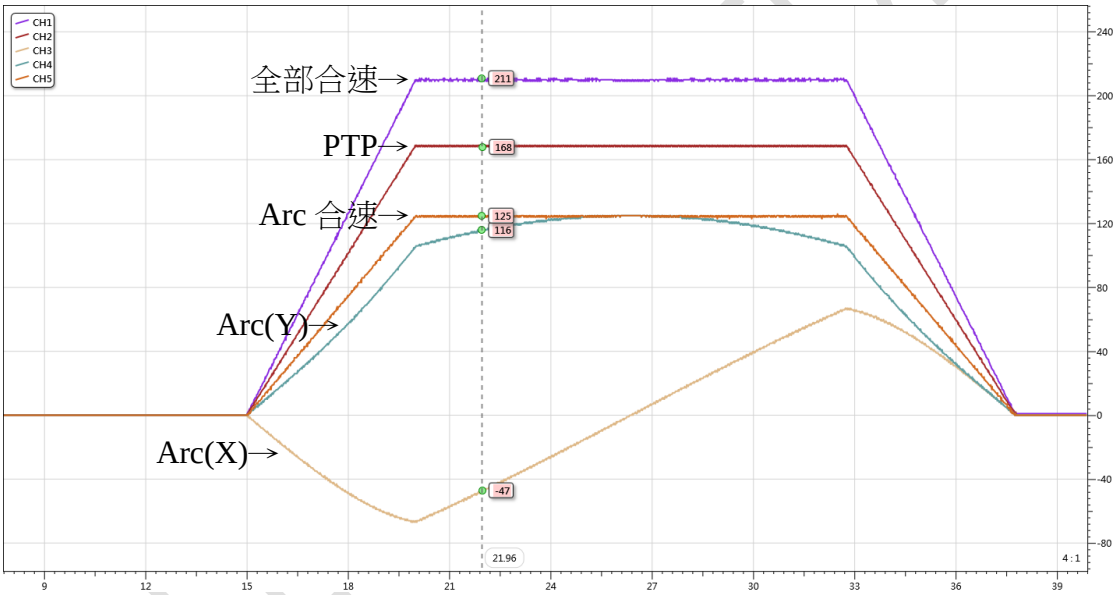
DELTA CONFIDENTIAL

U16 Dir = 0;
I32 StrVel = 0;
I32 MaxVel = 128000;
I32 EndVel = 0;
F64 Tacc = 0.5, Tdec = 0.5;
U16 Curve_mode = 1;
U16 AbsMode = 0;

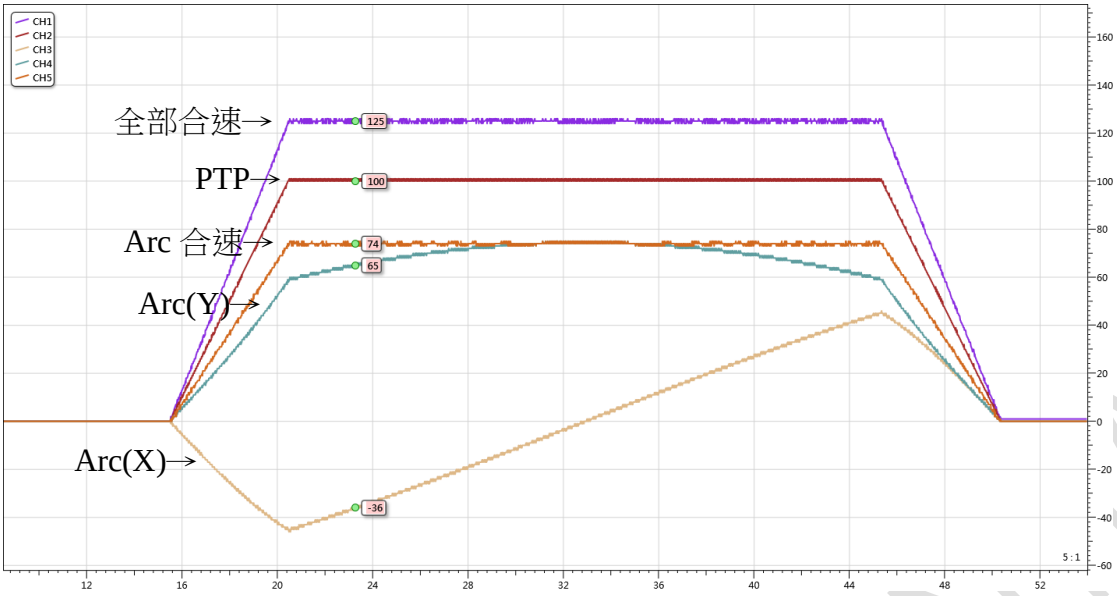
I16 status = _DMC_01_Start_Multi_Arc(CardNo, AxisNum, AxisArray, DistArray, SpeedMode, ArcMode, Arc_index, CenterPoint, EndPoint, Angle, Dir, StrVel, MaxVel, EndVel, Tacc, Tdec, Curve_mode, AbsMode);

NOTE

SpeedMode=0(隨速)下，PTP 跟隨 Arc 的合速做運動。



SpeedMode=1(合速)下，PTP 跟 Arc 做合速運動。



DELTA CONFIDENTIAL

第三十五章　緩衝區操作 API

此章節說明了如何使用伺服驅動器 DI3（SLD）觸發 Motion 指令(API)，透過啟動 enable 選項單位觸發 DI3（SLD）取得 Buffer 裏的 motion 指令。

表 35.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_trigger_buf_function	使用伺服驅動器 DI3(SLD)觸發 Motion 指令

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

35.1　_DMC_01_set_trigger_buf_function

■ **格式**

I16 PASCAL _DMC_01_set_trigger_buf_function (U16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■ **目的**

使用伺服驅動器 DI3（SLD）觸發 Motion 指令。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	U16	選項單位	0：buffer 只要有 motion 指令就執行 1：利用 DI 觸發 buffer 裏的 motion 指令

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0
U16 NodeID=1, SlotID=0;
U16 enable=1; //啟動此函式功能。將 DI3（SLD）觸發取得 Buffer 裏的 motion 指令

I16 status = _DMC_01_set_trigger_buf_function (CardNo, NodeID, SlotID, enable);
```

DELTA CONFIDENTIAL

第三十六章 Interrupt API

此章節說明如何設定中斷操作 (API)，提供 8 種中斷模式，可設定中斷致能與解除、讀取中斷達成次數、讀取中斷觸發的事件值及預設中斷處理程序功能。

表 36.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_int_factor	設定中斷模式，共有 8 種模式。
A/B	_DMC_01_int_enable	中斷設定致能
A/B	_DMC_01_int_disable	中斷設定解除
A/B	_DMC_01_get_int_count	中斷累計次數
A/B	_DMC_01_get_int_status	查詢目前中斷模式
A/B	_DMC_01_Link_interrupt	連結一處理程序，當設定之中斷發生就導入處理。
A/B	_DMC_01_set_dmc_cycle_int_count	使用中斷累加計數值後發出中斷

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

DELTA CONFIDENTIAL

36.1 _DMC_01_set_int_factor

■ **格式**

I16 PASCAL _DMC_01_set_int_factor (U16 CardNo, U16 NodeID, U16 int_factor)

■ **目的**

設定中斷的模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
int_factor	U16	選項單位	共有 8 種模式: 1. Normal Stop. 2. Next Buffer. 4. Acceleration End. 8. Deceleration Start. 16. Sdo Finish.(目前未開放) 32. DMC Cycle Start.(目前未開放) 64. RM04PI-FIFO.(目前未開放) 128. User Define.(目前未開放) 256. CAN Bus 硬體異常.(目前未開放) 512. EMGS 輸入緊急停止.(伺服 DI 碼 0x21)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 int_facter=5; //Normal Stop + Acceleration End.

I16 status = _DMC_01_set_int_factor (CardNo, NodeID, int_factor);
```

DELTA CONFIDENTIAL

36.2 _DMC_01_int_enable

■ **格式**

I16 PASCAL _DMC_01_int_enable (U16 CardNo, U16 NodeID)

■ **目的**

設定中斷致能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;

I16 status = _DMC_01_int_enable (CardNo, NodeID);
```

DELTA CONFIDENTIAL

36.3 _DMC_01_int_disable

■ **格式**

I16 PASCAL _DMC_01_int_disable (U16 CardNo, U16 NodeID)

■ **目的**

設定中斷禁能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;

I16 status = _DMC_01_int_disable (CardNo, NodeID);
```

DELTA CONFIDENTIAL

36.4 _DMC_01_get_int_count

■ **格式**

I16 PASCAL _DMC_01_get_int_count (U16 CardNo, U16 NodeID, U16 count)

■ **目的**

讀取中斷達成次數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
count	U16	中斷次數	達成中斷的次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16*count;

I16 status =_DMC_01_get_int_count (CardNo, NodeID, &count);
```

DELTA CONFIDENTIAL

36.5 _DMC_01_get_int_status

■ **格式**

I16 PASCAL _DMC_01_get_int_status (U16 CardNo, U16 NodeID, U16 event_int_status)

■ **目的**

讀取目前中斷觸發的事件值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
Event_int_status	U16	選項單位	目前中斷觸發的事件值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 event_int_status;//參考 36.1 模式設定。

I16 status = _DMC_01_get_int_status (CardNo, NodeID, & event_int_status);

DELTA CONFIDENTIAL

36.6 _DMC_01_link_interrupt

■ **格式**

I16 PASCAL _DMC_01_link_interrupt (U16 CardNo, void (__stdcall *callbackAddr) (U16 CardNo, U16 NodeID))

■ **目的**

設定一處理程序，當中斷條件發生就進入此程序處理。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
callbackAddr	U16	處理程序	預設之中斷處理程序

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
__stdcall * CallBack;

void _stdcall CallBack(U16 CardNo, U16 NodeID)
{
    U16 i;
    i = CardNo + NodeID;
}

I16 status = _DMC_01_link_interrupt (CardNo, CallBack);
```

DELTA CONFIDENTIAL

36.7 _DMC_01_set_dmc_cycle_int_count

■ **格式**

I16 PASCAL _DMC_01_set_dmc_cycle_int_count(U16 CardNo,U16 Count);

■ **目的**

根據當前 DSP Cycle time 取得循環時間，針對此循環時間，_DMC_01_set_dmc_cycle_int_count 下達檻值只要 Cycle time 達到此門檻值，則觸發中斷。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Count	U16	處理程序	預觸發中斷的中段次數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 Count_level =100;
U16*count;
I32 time;

/* 取得裝置的循環時間, time 的值必須小於 1000 */
I16 status = _DMC_01_get_cycle_time (CardNo, &time);

//只要 Cycle time 達到此門檻值，則觸發中斷
status =_DMC_01_set_dmc_cycle_int_count( CardNo,Count_level);
//判斷中斷是否觸發
If(time = Count_level)
{
status =_DMC_01_get_int_count (CardNo, NodeID, &count);
}
```

DELTA CONFIDENTIAL

36.8 _DMC_01_set_dio_input_trigger_mode

■ **格式**

I16 PASCAL _DMC_01_set_dio_input_trigger_mode(U16 CardNo, U16 trigger_mode)

■ **目的**

設定 DI 觸發準位。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
trigger_mode	U16	選項單位	0: 負緣觸發 Falling edge 1: 正緣觸發 Rising edge

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0;
U16 trigger_mode;

I16 status = _DMC_01_set_dio_input_trigger_mode(CardNo, trigger_mode)

DELTA CONFIDENTIAL

36.9 _DMC_01_set_dio_input_int_enable

■ **格式**

I16 PASCAL _DMC_01_set_dio_input_int_enable(U16 CardNo, U16 Enable)

■ **目的**

啟動/關閉 DI 觸發中斷功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Enable	U16	選項單位	0:關閉 1:啟動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0;

U16 Enable =1;(啟動 DI 觸發中斷功能)

I16 status = _DMC_01_set_dio_input_int_enable(U16 CardNo, U16 Enable);

DELTA CONFIDENTIAL

36.10 _DMC_01_get_dio_input_int_counter

■ **格式**

I16 PASCAL _DMC_01_get_dio_input_int_counter(U16 CardNo, U16* counter)

■ **目的**

取得 DI 觸發中斷事件發生次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
counter	U16*	數值單位	DI 觸發中斷事件發生次數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16* counter;

I16 status = _DMC_01_get_dio_input_int_counter(CardNo, &counter);
```

DELTA CONFIDENTIAL

36.11 _DMC_01_set_dio_input_int_counter_clear

■ **格式**

I16 PASCAL _DMC_01_set_dio_input_int_counter_clear(U16 CardNo)

■ **目的**

清除 DI 觸發中斷事件發生次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;

I16 status = _DMC_01_set_dio_input_int_counter_clear(CardNo)
```

DELTA CONFIDENTIAL

36.12 _DMC_01_get_dio_input_int_status

■ **格式**

I16 PASCAL _DMC_01_get_dio_input_int_status(U16 CardNo, U16* status)

■ **目的**

取得 DI 觸發中斷功能狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
status	U16*	數值單位	(只有 bit0 與 bit1 數值有定義) bit0: 0: 關閉 DI 觸發中斷功能 1: 啟動 DI 觸發中斷功能 bit1: 0: Falling edge 1: Rising edge

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16* status;

I16 status = _DMC_01_get_dio_input_int_status(CardNo, &status);
```

NOTE

假設取得 status = 3，二進制為 0011，代表

bit0=1，DI 觸發中斷功能啟動。

bit1=1，_DMC_01_set_dio_input_trigger_mode 設定的 DI 觸發準位為正緣觸發 Rising edge。

DELTA CONFIDENTIAL

第三十七章 Security API

此章節說明了如何操作主控(包含 Slave(04PI))軸卡安全機制(API)，提供讀取/寫入記憶體安全資料，讀/寫密碼與驗證碼，讀取產品序號功能。

表 37.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_read_security	讀取記憶體中的安全驗證資料
A/B	_DMC_01_read_security_status	獲取目前的記憶體是處於讀/寫的狀態
A/B	_DMC_01_write_security	將指定的安全資料寫入指定的記憶體區塊
A/B	_DMC_01_write_security_status	寫入安全資料前，將記憶體寫入功能致能
A/B/F	_DMC_01_check_userpassword	確認使用者對記憶體讀/寫的權限
A/B/F	_DMC_01_write_userpassword	變更密碼
A/B/F	_DMC_01_check_verifykey	驗證碼驗證
A/B/F	_DMC_01_write_verifykey	寫入驗證碼
A/B/F	_DMC_01_read_serialno	讀取軸卡產品序號
A/B/F	_misc_slave_check_userpassword	Slave(04PI)確認使用者對記憶體讀/寫的權限
A/B/F	_misc_slave_write_userpassword	Slave(04PI)變更密碼
A/B/F	_misc_slave_get_serialno	Slave(04PI)讀取 04PI 產品序號
A/B/F	_misc_security	使用者 Key+SerialNo 經加密產生驗證碼
A/B/F	_misc_slave_write_verifykey	Slave(04PI)寫入驗證碼
A/B/F	_misc_slave_check_verifykey	Slave(04PI)驗證碼驗證
A/B/F	_misc_slave_user_data_buffer_read	Slave(04PI)讀取記憶體中的資料
A/B/F	_misc_slave_user_data_buffer_write	Slave(04PI)將指定的資料寫入記憶體
A/B/F	_misc_slave_user_data_to_flash	Slave(04PI)將資料由 Bufeer 寫入快閃記憶體中

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

DELTA CONFIDENTIAL

37.1 _DMC_01_read_security

■ **格式**

I16 PASCAL _DMC_01_read_security (U16 CardNo, U16 page, U16 *array)

■ **目的**

讀取 Master Card 指定記憶體中的安全性資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Page	U16	編號單位	指定讀取的記憶體頁數編號
array	U16*	數值	由指定讀取的記憶體中讀回的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 Page=0;     // 0~9 共 10 頁
U16 array;

I16 status = _DMC_01_read_security ( CardNo, page ,&array);
```

DELTA CONFIDENTIAL

37.2 _DMC_01_read_security_status

■ **格式**

I16 PASCAL _DMC_01_read_security_status (U16 CardNo, U16 *status)

■ **目的**

讀取 Master Card 目前記憶體是處於讀或寫的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
status	U16*	旗標	4(Read Only) 或 6(Read/Write)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 status;

I16 status = _DMC_01_read_seacurity_status (CardNo, &status );
```

DELTA CONFIDENTIAL

37.3 _DMC_01_write_security

■ **格式**

I16 PASCAL _DMC_01_write_security (U16 CardNo, U16 page, U16 array)

■ **目的**

將安全資料寫入 Master Card 指定的記憶體區塊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Page	U16	編號單位	指定寫入的記憶體頁數編號
array	U16	數值	要寫入指定記憶體的安全資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 page=0; // 0~9 共 10 頁
U16 array={1,4,7,11,0a,ff,12,8,0b,10,3,5,c1,14,0d,6};

//執行此 API 前，須先執行_DMC_01_write_security_status (CardNo, 1)將讀/寫狀態變更為
  寫入狀態。
I16 status =_DMC_01_wtite_security( CardNo, page, array );
```

NOTE

注意:執行完_DMC_01_wtite_security 這個 API 後，Dll 會自動將讀/寫狀態變更為 Read Only 狀態，所以要再寫入資料，須重新使用_DMC_01_write_security_status (CardNo, 1)將讀/寫狀態變更為寫入狀態，然後再執行_DMC_01_wtite_security 寫入資料。

DELTA CONFIDENTIAL

37.4 _DMC_01_write_security_status

■ **格式**

I16 PASCAL _DMC_01_write_security_status (U16 CardNo, U16 status)

■ **目的**

Master Card 在寫入安全資料前，將記憶體寫入功能致能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
status	U16	旗標	將記憶體寫入功能致能(0:disable,1:enable)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 status =1;

I16 status = _DMC_01_write_security_status (CardNo, status);
```

DELTA CONFIDENTIAL

37.5 _DMC_01_check_userpassword

■ **格式**

I16 PASCAL _DMC_01_check_userpassword (U16 CardNo, U32 *password_data, U16 *password_state)

■ **目的**

Master Card 在讀/寫資料前，確認使用者對 Memory 讀/寫的權限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Password_data	U32*	數值	輸入 Check 的密碼
Password_state	U16*	旗標	Check 密碼後的回覆，0→Fail 1→OK

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 password_data[2]={0x12345678,0xabcdef};    //64bit data
U16 password_state;

I16 status = _DMC_01_check_userpassword ( CardNo, password_data, &password_state );
```

DELTA CONFIDENTIAL

37.6 _DMC_01_write_userpassword

■ **格式**

I16 PASCAL _DMC_01_write_userpassword (U16 CardNo, U32 password_data)

■ **目的**

Master Card 變更使用者密碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Password_data	U32	數值	輸入要變更的密碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 password_data[2]={0x12345678,0xabcdef};    //64bit data

I16 status = _DMC_01_write_userpassword ( CardNo, password_data );
```

DELTA CONFIDENTIAL

37.7 _DMC_01_check_verifykey

■ **格式**

I16 PASCAL _DMC_01_check_verifykey (U16 CardNo, U32 *Verifykey, U16*state)

■ **目的**

Master Card 將驗證碼帶入驗證是否相符。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Verifykey	U32*	數值	要驗證的驗證碼資料
State	U16*	旗標	Check 驗證碼後的回覆，0→Lock 1→Pass

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef};      //128bit data
U16 sate;

I16 status = _DMC_01_check_verifykey ( CardNo, Verifykey, &state );
```

DELTA CONFIDENTIAL

37.8 _DMC_01_write_verifykey

■ **格式**

I16 PASCAL _DMC_01_write_verifykey (U16 CardNo, U32 *Verifykey)

■ **目的**

將驗證碼寫入 Master Card。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Verifykey	U32*	數值	要寫入的驗證碼資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef};      //128bit data

I16 status = _DMC_01_write_verifykey (CardNo, Verifykey);
```

DELTA CONFIDENTIAL

37.9 _DMC_01_read_serialno

■ **格式**

I16 PASCAL _DMC_01_read_serialno (U16 CardNo, U32 *Serialno)

■ **目的**

讀取 Master Card 記憶體中的產品序號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Serialno	U32*	數值	存於記憶體中的產品序號資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Serialno[2];

I16 status = _DMC_01_read_serialno ( CardNo, &Serialno[0] );
```

DELTA CONFIDENTIAL

37.10 _misc_slave_check_userpassword

■ **格式**

I16 PASCAL _misc_slave_check_userpassword (U16 CardNo, U16 NodeID, U16 SlotID , U32 Password_data, U16 *Password_state)

■ **目的**

Slave(04PI)在讀/寫資料前，確認使用者對記憶體讀/寫的權限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Password_data	U32	數值	輸入 Check 的密碼
Password_state	U16*	旗標	Check 密碼後的回覆，0→Fail 1→OK

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Password_data[2]={0x12345678,0xabcdef};      //64bit data
U16 Password_state;

I16 status = _misc_slave_check_userpassword (CardNo, NodeID, SlotID, Password_data,
             &Password_state);
```

DELTA CONFIDENTIAL

37.11 _misc_slave_write_userpassword

■ **格式**

I16 PASCAL _misc_slave_write_userpassword (U16 CardNo, U16 NodeID, U16 SlotID ,U32 Password_data)

■ **目的**

Slave(04PI)寫入使用者密碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Password_data	U32	數值	要寫入記憶體的密碼變更資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Password_data[2]={0x12345678,0xabcdef};

I16 status = _misc_slave_write_userpasswaord (CardNo, NodeID, SlotID, Password_data);

DELTA CONFIDENTIAL

37.12 _misc_slave_get_serialno

■ **格式**

I16 PASCAL _misc_slave_get_serialno (U16 CardNo, U16 NodeID, U16 SlotID ,U32 *Serialno)

■ **目的**

讀取 Slave(04PI)產品序號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Serialno	U32*	數值	讀取記憶體中的產品序號資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Serialno[2];

I16 status = _misc_slave_get_serialno (CardNo, NodeID, SlotID, &Serialno);
```

DELTA CONFIDENTIAL

37.13 _misc_security

■ **格式**

I16 PASCAL _misc_security (U32 OtherWord0, U32 OtherWord1, U32 SyntekWord0, U32 SyntekWord1, U32 *Password0, U32 *Password1, U32 *Password2, U32 *Password3)

■ **目的**

Slave(04PI)由使用者自訂 64bit key 值與 64bit Serialno 帶入加密函數中得出 128bit 的驗證碼。

■ **參數**

名稱	資料型別	單位	描述
OtherWord0	U32	數值	32bit(0)使用者自訂 key 值
OtherWord1	U32	數值	32bit(1)使用者自訂 key 值
SyntekWord0	U32	數值	32bit (0)Serialno
SyntekWord1	U32	數值	32bit (1)Serialno
Password0	U32*	數值	加密函數得出 32bit(0)的驗證碼
Password1	U32*	數值	加密函數得出 32bit(1)的驗證碼
Password2	U32*	數值	加密函數得出 32bit(2)的驗證碼
Password3	U32*	數值	加密函數得出 32bit(3)的驗證碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U32 OtherWord0; //32bit
U32 OtherWord1; //32bit
U32 SyntekWord0; //32bit
U32 SyntekWord1; //32bit
U32 Password0[2];
U32 Password1[2];
U32 Password2[2];
U32 Password3[2];

I16 status = _misc_security (OtherWord0, OtherWord1, SyntekWord0, SyntekWord1,
        &Password0, &Password1, &Password2, &Password3);
```

DELTA CONFIDENTIAL

37.14 _misc_slave_write_verifykey

■ **格式**

I16 PASCAL _misc_slave_write_verifykey (U16 CardNo, U16 NodeID, U16 SlotID ,U32 Verifykey)

■ **目的**

Slave(04PI)寫入驗證碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Verifykey	U32	數值	將驗證碼寫入記憶體

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef };    //128bit data

I16 status = _misc_slave_write_verifykey (CardNo, NodeID, SlotID, Verifykey);
```

DELTA CONFIDENTIAL

37.15 _misc_slave_check_verifykey

■ **格式**

I16 PASCAL _misc_slave_check_verifykey (U16 CardNo, U16 NodeID, U16 SlotID , U32 Verifykey, U16 *Lock_state)

■ **目的**

將驗證碼帶入 Slave(04PI)驗證是否相符。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Verifykey	U32	數值	要驗證的驗證碼資料
Lock_state	U16*	旗標	Check 驗證碼後的回覆 0：LOCK 1：PASS

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef };      //128bit data
U16 Lock_state;

I16 status =_misc_slave_check_verifykey (CardNo, NodeID, SlotID, Verifykey, &Lock_state);
```

DELTA CONFIDENTIAL

37.16 _misc_slave_user_data_buffer_read

■ **格式**

I16 PASCAL _misc_slave_user_data_buffer_read (U16 CardNo, U16 NodeID, U16 SlotID U16 Address, U32* Data)

■ **目的**

讀取 Slave(04PI)指定 Page 的記憶體資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Address	U16	數值	要讀取的位置
Data	U32*	數值	記憶體儲存的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Address;
U32 Data[2];

I16 status = _misc_slave_user_data_buffer_read (CardNo, NodeID, SlotID, Address, &Data);
```

DELTA CONFIDENTIAL

37.17 _misc_slave_user_data_buffer_write

■ **格式**

I16 PASCAL _misc_slave_user_data_buffer_write (U16 CardNo, U16 NodeID, U16 SlotID U16 Address, U32 Data)

■ **目的**

將資料寫入暫存區中。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Address	U16	數值	要寫入的位置資料
Data	U32	數值	要寫入的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Address;
U32 Data;

I16 status = _misc_slave_user_data_buffer_write (CardNo, NodeID, SlotID, Address, Data);
```

DELTA CONFIDENTIAL

37.18 _misc_slave_user_data_to_flash

■ **格式**

I16 PASCAL _misc_slave_user_data_to_flash (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

將資料由暫存區寫入 Slave(04PI)指定位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;

I16 status = _misc_slave_user_data_to_flash (CardNo, NodeID, SlotID);
```

DELTA CONFIDENTIAL

第三十八章　極限反向 API

此章節說明了操作極限反向 (API)，可設定模組 RM04PI 正負極限正向/反向，取得正負極限狀態，GE01PH ALM 準位，RM04PI/GE01PH ORG 準位。

表 38.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04pi_set_MEL_polarity	設定 RM04PI 負極限反向
A/B/F	_DMC_01_rm_04pi_get_MEL_polarity	取得 RM04PI 負極限目前的狀態
A/B/F	_DMC_01_rm_04pi_set_PEL_polarity	設定 RM04PI 正極限反向
A/B/F	_DMC_01_rm_04pi_get_PEL_polarity	取得 RM04PI 正極限目前的狀態
A/B/F	_DMC_01_set_01ph_alm_polarity	設定 GE01PH ALM 準位反向
A/B/F	_DMC_01_set_01ph_org_polarity	設定 RM04PI/GE01PH ORG 準位反向

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

38.1　_DMC_01_rm_04pi_set_MEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_set_MEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 inverse)

■ **目的**

將 RM04PI 負極限反向，此設定在 RM04PI 模組斷電重開後將回復為預設值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Inverse	U16	選項資料	0：正向 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 inverse=1; //設定負極限反向

I16 status = _DMC_01_rm_04pi_set_MEL_polarity (CardNo, NodeID, SlotID, inverse);
```

DELTA CONFIDENTIAL

38.2　_DMC_01_rm_04pi_get_MEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_get_MEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 *data)

■ **目的**

取得 RM04PI 負極限目前的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Data	U16	選項資料	0：正向 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 data;

I16 status = _DMC_01_rm_04pi_get_MEL_polarity (CardNo, NodeID, SlotID, &data);
```

DELTA CONFIDENTIAL

38.3　_DMC_01_rm_04pi_set_PEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_set_PEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 inverse)

■ **目的**

將 RM04PI 正極限反向，此設定在 RM04PI 模組斷電重開後將回復為預設值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Inverse	U16	選項資料	0：正向 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 inverse=1;   //設定正極限反向

I16 status = _DMC_01_rm_04pi_set_PEL_polarity (CardNo, NodeID, SlotID, inverse);
```

DELTA CONFIDENTIAL

38.4　_DMC_01_rm_04pi_get_PEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_get_PEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 *data)

■ **目的**

取得 RM04PI 正極限目前的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Data	U16	旗標	0：正向; 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 data;

I16 status = _DMC_01_rm_04pi_get_PEL_polarity (CardNo, NodeID, SlotID, &data);
```

DELTA CONFIDENTIAL

38.5　_DMC_01_set_01ph_alm_polarity

■ **格式**

I16 PASCAL _DMC_01_set_01ph_alm_polarity(U16 CardNo, U16 NodeID, U16 SlotID, U16 inverse)

■ **目的**

設定 GE01PH ALM 準位，此設定在 GE01PH 模組斷電重開後將回復為預設值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
inverse	U16	選項單位	0：High 電位觸發 1：Low 電位觸發

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●GE01PH Only				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 inverse=0 ;// High 電位觸發。

I16 status = _DMC_01_set_01ph_alm_polarity(CardNo, NodeID, SlotID, inverse);
```

DELTA CONFIDENTIAL

38.6　_DMC_01_set_01ph_org_polarity

■　**格式**

I16 PASCAL _DMC_01_set_01ph_org_polarity(U16 CardNo, U16 NodeID, U16 SlotID, U16 inverse)

■　**目的**

設定 RM04PI/GE01PH ORG 準位，此設定在 RM04PI/GE01PH 模組斷電重開後將回復為預設值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
inverse	U16	選項單位	0：High 電位觸發 1：Low 電位觸發

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 inverse=0 ;// High 電位觸發。

I16 status = _DMC_01_set_01ph_org_polarity(CardNo, NodeID, SlotID, inverse);
```

DELTA CONFIDENTIAL

第三十九章 B01 Compare API

此章節說明了位置比較 Compare (API)，可設定/讀取 Position 計數值，Channe 脈波與 Channel_trigger 功能，設定 Compare 模式狀態與條件功能。

表 39.1

函數名稱	描述
_DMC_01_set_compare_channel_position	設定該 Channel 新的 Position 數值
_DMC_01_get_compare_channel_position	讀取該 Channel 目前的 Position 數值
_DMC_01_set_compare_ipulser_mode	設定脈波輸入模式
_DMC_01_set_compare_channel_direction	設定該 Channel 脈波方向
_DMC_01_set_compare_channel_trigger_time	設定觸發訊號時間
_DMC_01_set_compare_channel_one_shot	設定為執行一次觸發
_DMC_01_set_compare_channel_source	設定訊號來源
_DMC_01_channel0_position_cmp	執行固定區間的比對模式
_DMC_01_channel1_output_enable	不等區間的比對模式開啟/關閉
_DMC_01_channel1_output_mode	設定不等區間的比對模式輸出模式
_DMC_01_channel1_get_io_status	讀取不等區間的比對模式狀態
_DMC_01_channel1_set_gpio_out	設置 DMC-B01 軸卡輸出腳位狀態
_DMC_01_channel1_position_compare_table	設定不等距區間的位置比對資料
_DMC_01_channel1_position_compare_table_level	自定義輸出準位表格(Level 模式)
_DMC_01_channel1_position_compare_table_cnt	讀取已觸發次數
_DMC_01_set_compare_channel_polarity	設定觸發準位
_DMC_01_channel0_position_cmp_by_gpio	設定由外部 DI 啟動等距比較功能
_DMC_01_channel1_position_re_compare_table	重新執行一次 Channel1 的 Compare
_DMC_01_channel1_position_re_compare_table_level	重新執行一次 Channel1(Level 模式)的 Compare
_DMC_01_set_compare_channel_enable	設定 Compare 功能開啟/關閉
_DMC_01_channel1_get_fifo_status	取得目前 Compare Table 內資料的儲存狀態
_DMC_01_channel1_get_fifo_counter	取得目前 Compare Table 內資料的筆數

ADLINK CONFIDENTIAL

此表格 **API 僅 PCI_DMC_B01 可使用**

API名稱 xxxxxx _ channel0_xxxxx 用於 等距 Compare 模式

xxxxxx _ channel1_xxxxx 用於非等距 Compare 模式

DELTA CONFIDENTIAL

39.1 _DMC_01_set_compare_channel_position

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_position (U16 CardNo,
U16 compare_channel, I32 position)

■ **目的**

設定該 QEP Channel 新的計數值。(此數值與模組之 Position 意義並不相等)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_Channel	U16	編號單位	Channel No 為 0~1
Position	I32	數值單位	欲設定之 QEP Channel 新的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Compare_Channel = 0;
I32 position = 0;

I16 status = _DMC_01_set_compare_channel_position (CardNo, compare_channel, position);
```

DELTA CONFIDENTIAL

39.2 _DMC_01_get_compare_channel_position

■ **格式**

I16 PASCAL _DMC_01_get_compare_channel_position (U16 CardNo,
U16 compare_Channel, I32 *position)

■ **目的**

讀取該 QEP Channel 目前的計數值。(此數值與模組之 Position 意義並不相等)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_Channel	U16	編號單位	Channel No 為 0~1
Position	I32*	脈波數	讀取該 QEP Channel 目前的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Compare_channel = 0;
I32 position;

I16 status = _DMC_01_get_compare_channel_position (CardNo, compare_channel,
&position);
```

DELTA CONFIDENTIAL

39.3 _DMC_01_set_compare_ipulser_mode

■ **格式**

I16 PASCAL _DMC_01_set_compare_ipulser_mode (U16 CardNo, U16 mode)

■ **目的**

設定脈波介面模組輸入相位模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Mode	U16	數值單位	0：AB Phase 1：CW/CCW

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 mode = 0;       //AB Phase

I16 status = _DMC_01_set_compare_ipulser_mode (CardNo, mode);
```

DELTA CONFIDENTIAL

39.4 _DMC_01_set_compare_channel_direction

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_direction (U16 CardNo,
U16 compare_channel, U16 dir)

■ **目的**

設定該 QEP Channel 脈波方向。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Dir	U16	數值單位	0：Normal 1：Inverse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel = 0;
U16 dir = 1; //Inverse

I16 status = _DMC_01_set_compare_channel_direction (CardNo, compare_channel, dir);
```

DELTA CONFIDENTIAL

39.5 _DMC_01_set_compare_channel_trigger_time

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_trigger_time (U16 CardNo,
U16 compare_channel, U32 time_us)

■ **目的**

設定 Trigger 致能持續時間。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Time_us	U32	時間單位	輸入每個 Trigger 致能持續時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;
U16 compare_channel = 0;
U16 time_us = 20; //20us

I16 status = _DMC_01_set_compare_channel_trigger_time(CardNo, compare_channel, time_us);

NOTE

Compare Type0 狀態下，Trigger Time 設定值最小為 1us，若輸入值為 0 則顯示 0.8us。
Compare Type1 狀態下 Trigger Time 最小為 3us，若輸入值小於 3 則顯示 3us。

DELTA CONFIDENTIAL

39.6 _DMC_01_set_compare_channel_one_shot

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_one_shot (U16 CardNo,
U16 compare_channel)

■ **目的**

設定為執行一次 Trigger 致能狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel=0;

I16 status = _DMC_01_set_compare_channel_one_shot (CardNo, compare_channel);
```

DELTA CONFIDENTIAL

39.7 _DMC_01_set_compare_channel_source

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_source (U16 CardNo,
U16 compare_channel, U16 source)

■ **目的**

設定其比較來源。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Source	U16	編號單位	0：QEP1 1：QEP2

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel = 0;
U16 source = 0;

I16 status = _DMC_01_set_compare_channel_source (CardNo, compare_channel, source);
```

DELTA CONFIDENTIAL

39.8 _DMC_01_channel0_position_cmp

■ **格式**

I16 PASCAL _DMC_01_channel0_position_cmp (U16 CardNo, I32 satrt, U16 dir, U16 interval, U32 trigger_cnt)

■ **目的**

執行 Compare Type0。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Start	I32	數值單位	Compare 起始位置
Dir	U16	數值單位	0：正向 1：反向
Interval	U16	脈波數	脈波 Compare 間隔數
Trigger_cnt	U32	數值單位	Trigger Compare 總數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 start = 100000;
U16 dir = 0;
U16 Interval = 10; //10 pulse
U32 trigger_cnt = 50000;

I16 status = _DMC_01_channel0_position_cmp (CardNo, start, dir, interval, trigger_cnt);
```

DELTA CONFIDENTIAL

39.9 _DMC_01_channel1_output_enable

■ **格式**

I16 PASCAL _DMC_01_channel1_output_enable (U16 CardNo, U16 on_off)

■ **目的**

將 Compare Position 的存放陣列啟用，FPGA 內部的 Compare 功能才會開始以實體位置跟陣列的存放位置依序做比較。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_off	U16	數值單位	0：Off 1：On

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 on_off = 1;

I16 status = _DMC_01_channel1_output_enable (CardNo, on_off);
```

DELTA CONFIDENTIAL

39.10 _DMC_01_channel1_output_mode

■ **格式**

I16 PASCAL _DMC_01_channel1_output_mode (U16 CardNo, U16 Mode)

■ **目的**

Compare Type1 設定輸出模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Mode	U16	數值單位	0：Normal 模式 1：自訂模式

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 mode = 1;

I16 status = _DMC_01_channel1_output_mode (CardNo, mode);
```

NOTE

Normal 模式

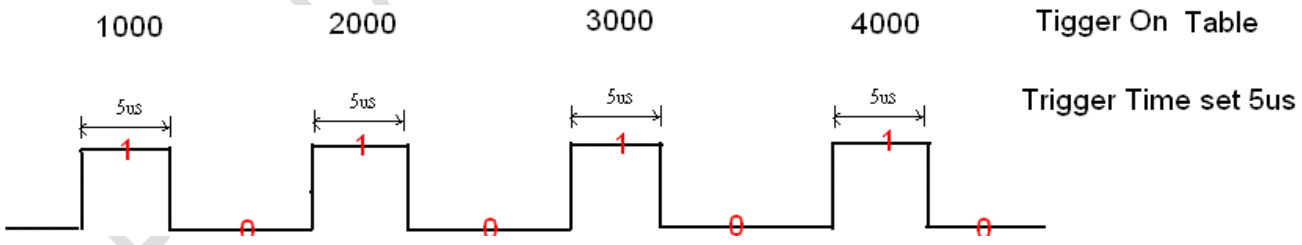


圖 39.1 Compare Output Normal 模式

※Trigger On 的位置表設定，是由 API「_DMC_01_channel1_position_compare_table」的"table_size"參數進行設定。

※Trigger 時間的設置，是由 API「_DMC_01_set_compare_channel_trigger_time」的"time_us"參數進行設定。

自訂模式

當設定為自訂模式，level table 的值設置為 0x88880000，如圖 39.2 所示。

TA CONFIDENTIAL

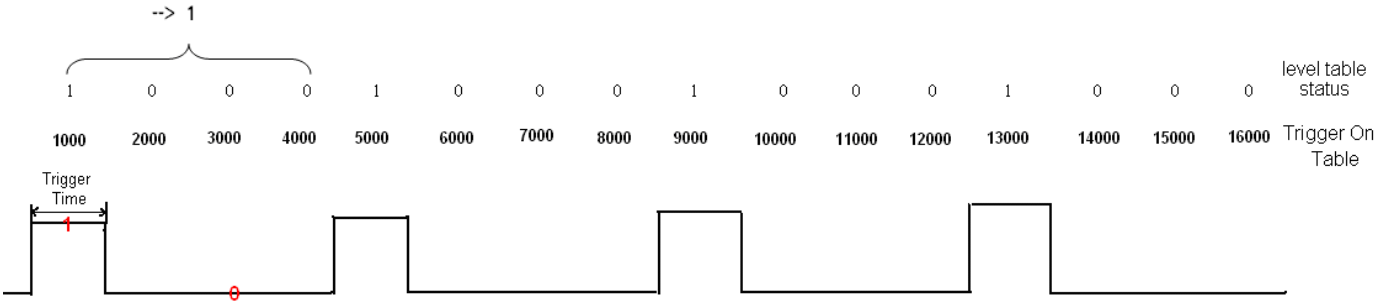


圖 39.2　Compare Output 使用者自訂模式

※自訂模式下 Trigger On 的位置表設定，

是由 API「_DMC_01_channel1_position_compare_table_level」的” level_table”參數進行設定。

※Trigger 時間則為到下一個位置所花費的時間（ex. 位置 1000 到位置 2000 花費的時間）。

DELTA CONFIDENTIAL

39.11 _DMC_01_channel1_get_io_status

■ **格式**

I16 PASCAL _DMC_01_channel1_get_io_status (U16 CardNo, U16* io_status)

■ **目的**

讀取 Compare Type1 狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Io_status	U16*	數值單位	Compare Type1 狀態 Bit0:GPIO IN 狀態 Bit1: 0:電源異常 1:5V 電源正確。 (Compare 硬體電源狀態,兩組 Compare 都是共用同一個電源)。 Bit2: 0: 自訂模式 1: Normal 模式 (目前規劃只有 Bit0 ~ Bit2 數值有意義)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 io_status;

I16 status = _DMC_01_channel1_get_io_status (CardNo,& io_status);
```

DELTA INITIAL

39.12 _DMC_01_channel1_set_gpio_out

■ **格式**

I16 PASCAL _DMC_01_channel1_set_gpio_out (U16 CardNo, U16 on_off)

■ **目的**

設置 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
On_off	U16	數值單位	0：Off 1：On

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 on_off = 1;

I16 status = _DMC_01_channel1_set_gpio_out (CardNo, on_off);
```

DELTA CONFIDENTIAL

39.13 _DMC_01_channel1_position_compare_table

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table (U16 CardNo, I32* pos_table, U32 table_size)

■ **目的**

設定 Compare Type1 的相關資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Pos_table	I32*	數值單位	依照資料計算出需 Compare 的 Pos_Table
Table_size	U32	數值單位	Pos_table 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 pos_table[4] = {1000,2000,3000,4000};
U32 table_size =4;

I16 status = _DMC_01_channel1_position_compare_table (CardNo, pos_table, table_size);
```

NOTE

Table_Size 最大設定值為 100000。

DELTA CONFIDENTIAL

39.14 _DMC_01_channel1_position_compare_table_level

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table_level (U16 CardNo,
I32* pos_table, U32* level_table, U32 table_size)

■ **目的**

設定 Compare Type1 的 Level 相關資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Pos_table	I32*	數值單位	依照資料計算出需 Compare 的 Pos_Table
Level_table	U32*	數值單位	設定自訂 Compare 的 Level_Table
Table_size	U32	數值單位	Pos_table 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 pos_table[100] ={1000,2000,3000,4000,…,100000};//假設共 100 筆
U32 table_size =100;
for (i = 0; i < table_size; i++)
{
    if ((i % 32)==0)
    level_table [i / 32] = 0x33333333;   //兩位置點之間 ON，兩位置點之間 OFF
}
I16 status = _DMC_01_channel1_position_compare_table_level (CardNo, pos_table,
            level_table,table_size);
```

DELTA CONFIDENTIAL

NOTE

Table_Size 最大設定值為 100000。

若 **pos_table** 有 **100** 筆，**LevelTable** 陣列第一筆則要填 **pos_table** 前 **32** 筆的 **level** 設定狀態**(**數值以 **16** 進位 **32bit** 表示**)**，第 **33** 筆**~64** 筆 **pos_table** 資料的 **level** 設定狀態要填在 **LevelTable** 第二個陣列，以此類推。

DELTA CONFIDENTIAL

39.15 _DMC_01_channel1_position_compare_table_cnt

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table_cnt (U16 CardNo, U32* cnt)

■ **目的**

讀取已 Trigger 的次數(Compare Type1 使用)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Cnt	U32*	數值單位	讀取已 Trigger 的次數(Compare Type1 使用)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U32 cnt;

I16 status = _DMC_01_channel1_position_compare_table_cnt (CardNo,& cnt);
```

DELTA CONFIDENTIAL

39.16 _DMC_01_set_compare_channel_polarity

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_polarity (U16 CardNo, U16 inverse)

■ **目的**

設定 Compare Trigger 的準位。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Inverse	U16	數值單位	0：Normal (0V) 1：Inverse (5V)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;
U16 inverse = 1;

I16 status = _DMC_01_set_compare_channel_polarity (CardNo, inverse);

DELTA CONFIDENTIAL

39.17 _DMC_01_channel0_position_cmp_by_gpio

■ **格式**

I16 PASCAL _DMC_01_channel0_position_cmp_by_gpio (U16 CardNo, U16 dir, U16 interval, I32 trigger_cnt)

■ **目的**

設定 CompareType0 的觸發條件由 GPIO 控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Dir	U16	數值單位	0：正向 1：反向
Interval	U16	數值單位	脈波 Compare 間隔數
Trigger _cnt	I32	數值單位	Trigger Compare 總數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Dir = 1;
U16 Interval = 10;
I32 trigger_cnt = 50000;

I16 status = _DMC_01_channel0_position_cmp_by_gpio (CardNo, Dir, Interval, trigger_cnt);
```

DELTA CONFIDENTIAL

39.18 _DMC_01_channel1_position_re_compare_table

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table (U16 CardNo)

■ **目的**

用上次設定的 Compare 條件，重新執行一次 Compare Type1。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;

I16 status = _DMC_01_channel1_position_re_compare_table (CardNo);

DELTA CONFIDENTIAL

39.19 _DMC_01_channel1_position_re_compare_table_level

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table_level (U16 CardNo)

■ **目的**

用上次設定的 Compare 條件，重新執行一次 Compare Type1(Level 模式)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_channel1_position_re_compare_table_level (CardNo);
```

DELTA CONFIDENTIAL

39.20 _DMC_01_set_compare_channel_enable

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_enable (U16 CardNo,
U16 compare_channel,U16 enable)

■ **目的**

設定 FPGA 內的 Compare 功能開啟/關閉，若未啟用比較陣列(Channel1_output_enable)，實體 Compare 因為沒有位置可以比較，所以依然不會觸發。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
enable	U16	數值單位	0：關閉 Compare 功能。 1：開啟 Compare 功能。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, ;
U16 compare_channel = 0;
U16 enable = 1;//開啟 Compare 功能

I16 status = _DMC_01_set_compare_channel_enable (CardNo, compare_channel, enable);

DELTA CONFIDENTIAL

39.21 _DMC_01_channel1_get_fifo_status

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table_level (U16 CardNo, U16* fifo_status)

■ **目的**

取得目前 Compare Table 內資料的存放狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
fifo_status	U16*	數值單位	Compare Table 內資料的存放狀態

數值	說明
0	Empty
1	Almost empty
2	Not empty or full
3	Almost full
4	Full

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, fifo_status = 0;

I16 status = _DMC_01_channel1_position_re_compare_table_level (CardNo, &fifo_status);

DELTA CONFIDENTIAL

39.22 _DMC_01_channel1_get_fifo_counter

■ **格式**

I16 PASCAL _DMC_01_channel1_get_fifo_counter (U16 CardNo, U16* fifo_cnt)

■ **目的**

取得目前 CompareTable 內存放比較資料的筆數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
fifo_cnt	U16*	數值單位	目前 CompareTable 內可填入剩餘筆數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, fifo_cnt = 0;

I16 status = _DMC_01_channel1_get_fifo_counter (CardNo, &fifo_cnt);

DELTA CONFIDENTIAL

第四十章　連續速度 API

此章節說明了設定連續速度 (API)，可設定致能、模式與連續速度合成比率。

表 40.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_speed_continue	開啟連續速度功能
A/B	_DMC_01_speed_continue_mode	設定連續速度模式
A/B	_DMC_01_speed_continue_combine_ratio	設定連續速度疊合比率

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

DELTA CONFIDENTIAL

40.1　_DMC_01_speed_continue

■ **格式**

I16 PASCAL _DMC_01_speed_continue (U16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■ **目的**

啟用/關閉連續速度功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0：Disable 連續速度功能 1：Enable 連續速度功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable=1;

I16 status = _DMC_01_speed_continue (CardNo, NodeID, SlotID, enable);

NOTE

當您啟用 _DMC_01_speed_continue 參數 enable=1，接下來所下的任何運動指令，其中的 StrVel 參數必須設定為 0，才能達成 Speed Continue 的效果。
加減速時間必須大於 2ms。
若您要在多軸指令中使用_DMC_01_speed_continue 指令，您必須逐軸下達，參數 enable=1

※若有運動指令正在運行，不可對連續速度功能進行操作。

DELTA CONFIDENTIAL

40.2　_DMC_01_speed_continue_mode

■ **格式**

I16 PASCAL _DMC_01_speed_continue_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

設定連續速度模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：等加速模式 1：加、減速模式 2：最大速度模式

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, U16 NodeID=1, U16 SlotID=0;
U16 mode=1;

I16 status = _DMC_01_speed_continue_mode (CardNo, NodeID, SlotID, mode);
```

DELTA CONFIDENTIAL

NOTE

Mode 值為 0

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 0 時，將會等比率縮減 MaxVel、Tacc 與 Tdec。以相等加速度完成。圖 41.1 所示，黑色為原規劃路徑，而紅色為實際路徑。

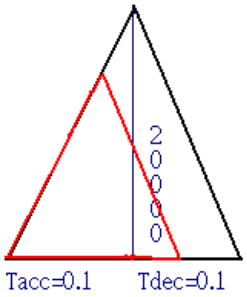


圖 40.1

Mode 值為 1

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 1 時，Tacc 與 Tdec 仍會維持在 0.1，但 MaxVel 值會自動縮減。圖 41.2 所示，黑色為原規劃路徑，而紅色為實際路徑。

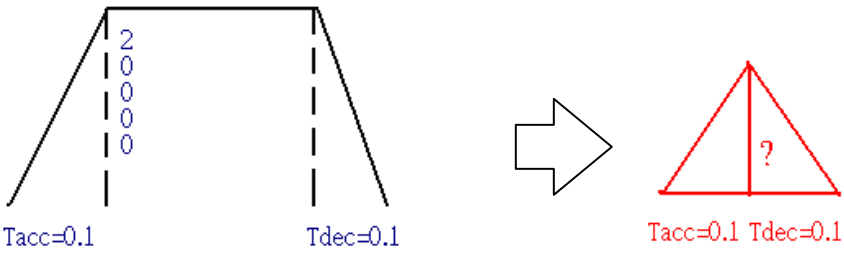


圖 40.2

Mode 值為 2

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 2 時，MaxVel 的值將維持不變，但 Tacc 與 Tdec 會依實際狀況變動。圖 41.3 所示，黑色為原規劃路徑，而紅色為實際路徑。

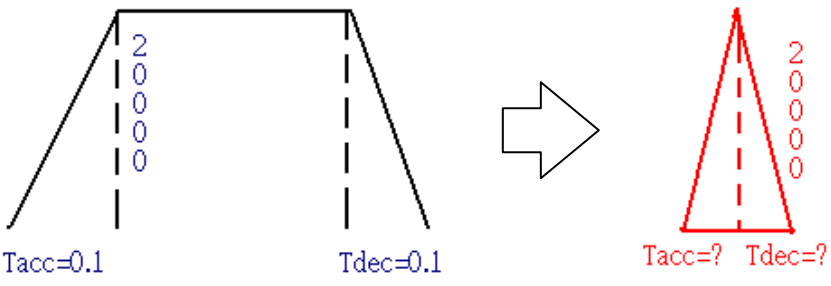


圖 40.3

DELTA CONFIDENTIAL

40.3　_DMC_01_speed_continue_combine_ratio

■ **格式**

I16 PASCAL _DMC_01_speed_continue_combine_ratio (U16 CardNo, U16 NodeID, U16 SlotID, U16 ratio)

■ **目的**

設定連續速度合成百分比。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Ratio	U16	數值單位	合成百分比(0~100)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, U16 NodeID=1, U16 SlotID=0;
U16 ratio=100;

I16 status = _DMC_01_speed_continue_combine_ratio (CardNo, NodeID, SlotID, ratio);

DELTA CONFIDENTIAL

NOTE

Ratio 的值為 100

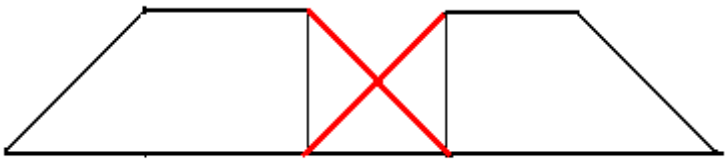


圖 40.4

Ratio 的值為 50

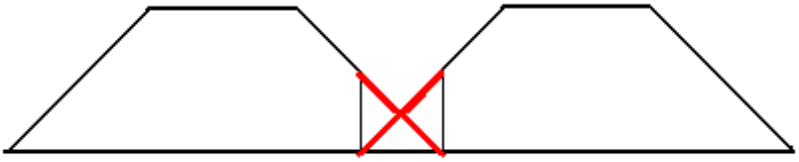


圖 40.5

※1~6 軸執行 Speed Continue 時，會運用到 7~12 軸的資源。
所以當 1~6 軸執行 Speed Continue Enable 時，7~12 軸不能執行 P2P 及 Continue 等動作。
必須將 Speed Continue Disable 後，7~12 軸方可進行後續動作。

※若有運動指令正在運行，不可任意變更 Ratio 值。

DELTA CONFIDENTIAL

第四十一章　特殊運動控制 API

此章節說明特殊運用 (API)，可設置電子凸輪間隙與背隙補償功能。

表 41.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_enable_electcam	開啟/關閉跟隨功能。
A/B	_DMC_01_set_cam_enable	開啟/關閉間隙補償功能
A/B	_DMC_01_set_cam_pitch	設定間隙補償間隔(單位:Pulse)
A/B	_DMC_01_set_cam_org	設定間隙補償開始原點位置
A/B	_DMC_01_set_cam_mode	設定間隙補償模式(單向/雙向)
A/B	_DMC_01_set_cam_table	建立間隙補償參數表
A/B	_DMC_01_set_cam_table2	建立間隙補償參數表資料並設定補償數量
A/B	_DMC_01_set_backlash	開啟/關閉背隙補償功能
A/B	_DMC_01_set_backlash_info	設定背隙補償參數
A/B	_DMC_01_set_ecam_repeat	開啟/關閉電子凸輪重複功能

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

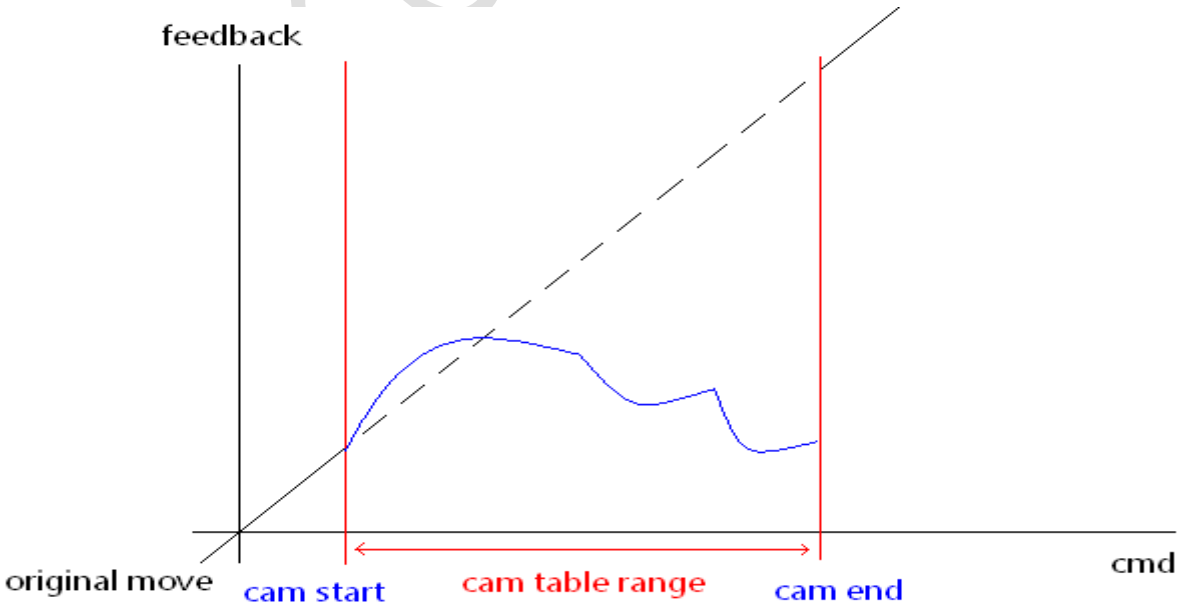


圖 41.1

說明:未使用電子凸輪前，其運動軌跡(cmd 與 feedback 相等)，進入電子凸輪後(cam table)，將依照上圖藍色路線進行，於 cam table 結束後，將恢復原運動軌跡，請注意由藍色路線

銜接原運動軌跡處，會因為追隨問題而導致暴衝現象，故 cam table 最好包含整個運動行程。

DELTA CONFIDENTIAL

41.1　_DMC_01_enable_electcam

■ **格式**

I16 PASCAL _DMC_01_enable_electcam(U16 CardNo, U16 NodeID, U16 SlotID, U16 enable,U16 axisbit,U16 mode)

■ **目的**

設定跟隨功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID(設定為主軸站號)
SlotID	U16	編號單位	插槽 ID
enable	U16	數值單位	0：關閉跟隨功能 1：開啟跟隨功能
axisbit	U16	數值單位	二進制值，設定為主軸的 bit 不能致能 Bit0:Node1 Bit1:Node2 Bit2:Node3 Bit3:Node4 Bit4:Node5 Bit5:Node6 Bit6:Node7 Bit7:Node8 Bit8:Node9 Bit9:Node10 Bit10:Node11 Bit11:Node12
mode	U16	數值單位	0：跟隨主軸 Command 1：跟隨主軸 Position

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;　//設定 Node1 為主軸
U16 enable =1; //開啟跟隨功能
U16 axisbit =14; //Node1 為主軸，Node2,3,4 設定為從軸 axisbit=0(2^0)+2(2^1)+4(2^2)+8(2^3)=14
U16 mode=0; //跟隨主軸 Command

I16 status = _DMC_01_enable_electcam(CardNo, NodeID, SlotID, enable, axisbit, mode);

DELTA CONFIDENTIAL

41.2　_DMC_01_set_cam_enable

■ **格式**

I16 PASCAL _DMC_01_set_cam_enable(U16 CardNo, U16 NodeID, U16 SlotID,U16 on_off)

■ **目的**

開啟/關閉間隙補償功能(不使用此功能時，請將間隙態補償 Disable)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
on_off	U16	數值單位	0：Disable 間隙態補償功能 1：Enable 間隙補償功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable=1; //開啟間隙補償功能

I16 status = _DMC_01_set_cam_enable(CardNo, NodeID, SlotID, on_off);

DELTA CONFIDENTIAL

41.3　_DMC_01_set_cam_pitch

■ **格式**

I16 PASCAL _DMC_01_set_cam_pitch (U16 CardNo, U16 NodeID, U16 SlotID,I32 pitch)

■ **目的**

設定間隙補償間隔(Pulse)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Pitch	I32	數值單位	設定間隙補償間隔(Pulse)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0;
I32 pitch =100000; //間隙補償間隔(Pulse)

//每 10000Pulse 間隔補償
I16 status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID, pitch);
```

DELTA CONFIDENTIAL

41.4　_DMC_01_set_cam_org

■ **格式**

I16 PASCAL _DMC_01_set_cam_org(U16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32 orgpos)

■ **目的**

設定間隙補償開始原點位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償原點 1：設定負向間隙補償原點
orgpos	I32	數值單位	間隙補償原點起始位置

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙態補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置

status = _DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = _DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

NOTE

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙**補償由 100000 這個位置開始，而負向的**間隙**補償，由 900000 這個位置開始。**

DELTA CONFIDENTIAL

41.5　_DMC_01_set_cam_mode

■ **格式**

I16 PASCAL _DMC_01_set_cam_mode (U16 CardNo, U16 NodeID, U16 SlotID,U16 Mode)

■ **目的**

設定間隙補償模式(單向/雙向)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	數值單位	0：設定雙向/雙表格間隙補償(來回用不同的參數表格資料做間隙補償) 1：設定雙向/單表格間隙補償(來回都用同一個參數表格資料做間隙補償)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 Mode=0; //設定為雙向/雙表格間隙補償

I16 status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, Mode);
```

DELTA CONFIDENTIAL

41.6　_DMC_01_set_cam_table

■ **格式**

I16 PASCAL _DMC_01_set_cam_table(U16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32* table)

■ **目的**

建立間隙補償參數表資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償 1：設定負向間隙補償
Table	I32*	數值單位	間隙補償參數資料

■ **屬性**

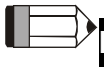
軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置
//設定正向間隙補償表資料,最大筆數 10000
I32 table0={10050,20030,30045,40040,50025,60020,70010,80045,90030};
//設定負向間隙補償表資料,最大筆數 10000
I32 table1={90032,80043,70015,60028,50020,40035,30040,20032,10045};

status = DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償

status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

DELTA CONFIDENTIAL

```
status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 0);//設定為雙向/雙表格模式
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir0, table0);
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir1, table1);

status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 1);//設定為雙向/單表格模式
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir0, table0);
```



因為是雙向都用同一個間隙補償資料表，所以只設定正向間隙補償資料表即可，負向部分會自動依正向動間隙資料表反算做補償

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙補償由 100000 這個位置開始，而負向的間隙補償，由 900000 這個位置開始。

DELTA CONFIDENTIAL

41.7　_DMC_01_set_cam_table2

■ **格式**

I16 PASCAL _DMC_01_set_cam_table2(U16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32* table,I32 Num)

■ **目的**

建立間隙補償參數表資料並設定間隙補償補償數量。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償 1：設定負向間隙補償
table	I32*	數值單位	間隙補償參數資料
Num	I32	數值單位	間隙補償數量

■ **屬性**

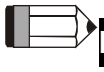
軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status = 0;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置
I32 table0={10050,20030,30045,40040,50025,60020,70010,80045,90030};
    //設定正向間隙補償表資料,最大筆數 10000
I32 table1={90032,80043,70015,60028,50020,40035,30040,20032,10045};
    //設定負向間隙補償表資料,最大筆數 10000
I32 Num0=9, Num1=8;

status = DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

DELTA CONFIDENTIAL

```
status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 0);//設定為雙向/雙表格模式
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir0, table0, Num0);
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir1, table1, Num1);

status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 1);//設定為雙向/單表格模式
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir0, table0, Num0);
```



NOTE

因為是雙向都用同一個間隙補償資料表，所以只設定正向間隙補償資料表即可，負向部分會自動依正向間隙資料表反算做補償

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙補償由 100000 這個位置開始，而負向的間隙補償，由 900000 這個位置開始。

DELTA CONFIDENTIAL

41.8　_DMC_01_set_backlash

■ **格式**

I16 PASCAL _DMC_01_set_backlash(U16 CardNo, U16 NodeID, U16 SlotID,I16 enable)

■ **目的**

開啟/關閉背隙補償功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	數值單位	0：關閉背隙補償功能 1：開啟背隙補償功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable =1; /開啟背隙補償功能

I16 status = _DMC_01_set_backlash(CardNo, NodeID, SlotID, enable);

DELTA CONFIDENTIAL

41.9　_DMC_01_set_backlash_info

■　**格式**

I16 PASCAL _DMC_01_set_backlash_info(U16 CardNo, U16 NodeID, U16 SlotID, I16 backlash,U16 accstep,U16 contstep,U16 decstep)

■　**目的**

設定背隙補償參數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
backlash	I16	數值單位	背隙補償值(Pulse)，設定 accstep,contstep, decstep 並依照圖 41.1 的方式完成背隙補償
accstep	U16	數值單位	設定圖 41.1 加速時間(ms)
contstep	U16	數值單位	設定圖 41.1 常速時間(ms)
decstep	U16	數值單位	設定圖 41.1 減速時間(ms)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

backlash 的值為 100

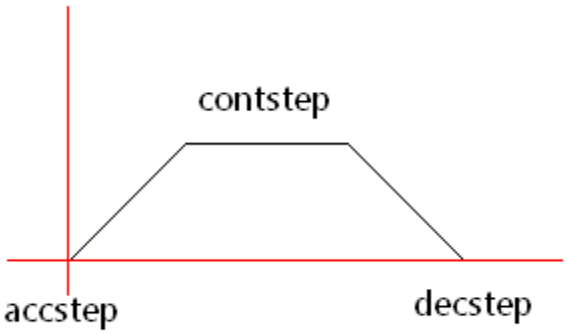


圖 41.1

I 16 CardNo=0, U16 NodeID=1, SlotID=0, status = 0;
U16 enable =1; //開啟背隙補償功能

I 16 backlash =100;
U16 accstep =10, contstep=10, decstep=10;
//在加速時間=10ms，常速時間=10ms，減速時間=10ms 的條件下算出以何種速度在上述條件內完成 100Pulse 的背隙補償;至於這三個參數的設定值，要如何不造成速度突增，需要嘗試。

```
status = _DMC_01_set_backlash_info(CardNo, NodeID, SlotID, backlash, accstep, contstep,
         decstep);
status = _DMC_01_set_backlash(CardNo, NodeID, SlotID, enable);
```

NOTE

背隙補償的 backlash 設定值若為正值，表示往正方向移動的背隙補償值，裝置若往負方向移動，則不做背隙補償; 反之 backlash 設定值若為負值，表示往負方向移動的背隙補償值，裝置若往正方向移動，則不做背隙補償。

Accstep+contstep+decstep 總和，不能超過 1000ms。

DELTA CONFIDENTIAL

41.10　_DMC_01_set_ecam_repeat

■ **格式**

I16 PASCAL _DMC_01_set_ecam_repeat (U16 CardNo, U16 NodeID, U16 SlotID,
I32 start_pos,I32 end_pos, U16 enable)

■ **目的**

開啟/關閉電子凸輪重複功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
start_pos	I32	數值單位	設定起始位置
end_pos	I32	數值單位	設定結束位置
enable	U16	選項單位	0：關閉電子凸輪重複功能 1：開啟電子凸輪重複功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0, NodeID=1, SlotID=0;
I32 start_pos =100000, end_pos =800000;
U16 enable =1; //開啟電子凸輪重複功能

I16 status = _DMC_01_set_ecam_repeat (CardNo, NodeID, SlotID, start_pos, end_pos, enable);

DELTA CONFIDENTIAL

第四十二章　軸卡特殊應用 API

此章節說明軸卡特殊應用(API)。

表 42.1

API 歸屬	函數名稱	描述
A/B/F	_misc_app_get_circle_endpoint	取得圓弧補間所需之終點座標(X, Y)
A/B/F	_misc_app_get_circle_center_point	取得圓弧補間所需之中心座標(X, Y)
A/B/F	_misc_set_record_debuging	Debug 記錄功能是否啟動
A/B/F	_misc_open_record_debuging_file	設置輸出 Debug 記錄存放位置
A/B/F	_DMC_01_enable_dda_mode	啟動 DDA 填表功能
A/B/F	_DMC_01_set_dda_data	填入 DDA Table 資料
A/B/F	_DMC_01_get_dda_cnt	取得目前 DDA Table 尚存的資料筆數
A/B/F	_DMC_01_clear_dda_data	將 DDA Table 中尚存的資料清除。
A/B/F	_DMC_01_enable_dda_mode_group	用 bit 指定要啟動的 NodeID。(新增 201601)
A/B/F	_DMC_01_set_dda_data_group	設定點跟點之間絕對移動的距離。(新增 201601)
A/B/F	_DMC_01_get_dda_cnt_group	取得 DDA Table 各群之間尚存的資料筆數。(新增 201601)
A/B/F	_DMC_01_clear_dda_data_group	將 DDA Table 各群之間尚存的資料清除。(新增 201601)
A/B/F	_DMC_01_save_device_info	將模組資料存為文字檔。
A/B/F	_DMC_01_compare_device_info	將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。
A/B/F	_DMC_01_enable_record_mode	開啟/關閉以毫秒為單位擷取 Monitor 所下參數的功能。
A/B/F	_DMC_01_get_record_data	讀取以毫秒為單位擷取 Monitor 所下參數 12 軸資料
A/B/F	_DMC_01_get_record_cnt	讀取以毫秒為單位擷取 Monitor 所下參數資料筆數
A/B	_DMC_01_set_mc_fifo_mode	FIFO 參數設定。
A/B	_DMC_01_set_function_time	設定每 1ms 最小的通訊時間以確保通訊不會異常。(詳細請參照 API 內說明)
A/B/F	_DMC_01_get_fifo_cnt	取得 FIFO 暫存區資料筆數。
A/B/F	_DMC_01_set_gear	啟動設定 Gear 功能與其參數設置

A/B/F	_DMC_01_get_gear	讀取 Gear 設定值。
A/B/F	_DMC_01_set_position_ref_source	設定座標的參考來源模式
A/B/F	_DMC_01_get_position_ref_source	取得目前座標的參考來源模式
A/B/F	_DMC_01_set_virtual_command	Set position 的快速模式
A/B/F	_DMC_01_get_virtual_command	取得實際 command、position 與 virtual_command 的差值
A/B/F	_DMC_01_set_virtual_position	將實際 Position 值設定為 Virtual Position 的值
A/B/F	_DMC_01_disable_virtual_pos	選擇重置 Command 與 Position 的連動性
A/B/F	_DMC_01_set_stepping_move_rate	設定解析放大的比率
MH1/MP1	_DMC_01_MRAM_read_word	讀取 MH1(MP1)記憶體 MRAM 的值 (單位:word)
MH1/MP1	_DMC_01_MRAM_write_word	寫值到 MH1(MP1)的記憶體 MRAM (單位:word)
MH1/MP1	_DMC_01_MRAM_read_dword	讀取 MH1(MP1)記憶體 MRAM 的值 (單位:double word)
MH1/MP1	_DMC_01_MRAM_write_dword	寫值到 MH1(MP1)的記憶體 MRAM (單位:double word)
A/B	_DMC_01_get_watchdog_cnt	讀取 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
A/B	_DMC_01_set_watchdog_cnt	修改 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
A/B	_DMC_01_get_watchdog_value	讀取 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
A/B	_DMC_01_set_watchdog_value	修改 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
A/B	_DMC_01_set_rm_04pi_md2_feedback_mode	設定脈波模組 Feedback 的連接狀態
A/B	_DMC_01_Security_Check_UserPassword	Master 在讀/寫資料前，確認使用者對 Memory 讀/寫的權限登入
A/B	_DMC_01_Security_Get_Check_UserPassword_State	讀取目前 Msater 檢查權限登入的狀態
A/B	_DMC_01_Security_Write_UserPassword	變更權限登入的密碼
A/B	_DMC_01_Security_Get_Write_UserPassword_State	檢查變更權限登入密碼的狀態
A/B	_DMC_01_Security_Check_Verifykey	Master 將驗證碼寫入驗證是否相符
A/B	_DMC_01_Security_Get_Check_Verifykey_State	讀取目前 Msater 檢查驗證的狀態
A/B	_DMC_01_Security_Write_Verifykey	變更驗證的密碼

A/B	_DMC_01_Security_Get_Write_Verifykey_State	檢查變更驗證密碼的狀態
A/B	_DMC_01_buffer_correlation_master_slave	設定總線內伺服軸的 MotionBuffer 關連群組
A/B	_DMC_01_buffer_correlation_status	讀取總線內伺服軸的 MotionBuffer 關連群組設定狀態
MP1	_DMC_01_GPIO_Set_Type	設定 RS-422/485 串列埠
MP1	_DMC_01_Get_GPIO_Input_Value	取得 MP1 GPIO 輸入腳位狀態
MP1	_DMC_01_Get_GPIO_Output_Value	取得 MP1 GPIO 輸出腳位狀態
MP1	_DMC_01_Set_GPIO_Output_Value	設定 MP1 GPIO 輸出腳位狀態
MP1	_DMC_01_Set_GPIO_Output_error_handle	設定 MP1 GPIO 保持輸出狀態
MP1	_DMC_01_Get_GPIO_Output_error_handle	取得 MP1 GPIO 保持輸出設定值
MP1-P10D-15	_DMC_01_get_mp1_dio_input	取得 MP1-P10D-15 8PIN 輸入腳位狀態 (MP1-P10D-15 專用)
MP1-P10D-15	_DMC_01_get_mp1_dio_single_input	取得 MP1-P10D-15　單一 bit 輸入腳位狀態(MP1-P10D-15 專用)

註 A:PCI_DMC_A01　B:PCI_DMC_B01　F:PCI_DMC_F01

DELTA CONFIDENTIAL

42.1　_misc_app_get_circle_endpoint

■　**格式**

I16 PASCAL _misc_app_get_circle_endpoint (I32 Start_X, I32 Start_Y, I32 Center_X, I32 Center_Y, F64 Angle, I32* End_x, I32* End_y)

■　**目的**

取得圓弧補間所需之終點座標(X, Y)。

■　**參數**

名稱	資料型別	單位	描述
Start_X	I32	脈波數	起始座標 X 軸
Start_Y	I32	脈波數	起始座標 Y 軸
Center_X	I32	脈波數	中心座標 X 軸
Center_Y	I32	脈波數	中心座標 Y 軸
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360o
End_x	I32*	脈波數	終點座標 X 軸
End_y	I32*	脈波數	終點座標 Y 軸

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I32 Start_X=0, Start_Y=0;
I32 Center_X=100000, Center_Y=0;
I32 End_x, End_y;
F64 Angle=180;

I16 status = _misc_app_get_circle_endpoint (Start_X, Start_Y, Center_X, Center_Y, Angle,
&End_x, &End_y);
```

DELTA CONFIDENTIAL

42.2　_misc_app_get_circle_center_point

■　**格式**

I16 PASCAL _misc_app_get_circle_center_point (I32 Start_X, I32 Start_Y, I32 End_x, I32 End_y, F64 Angle, I32* Center_X, I32* Center_Y)

■　**目的**

取得圓弧補間所需之中心座標(X, Y)。

■　**參數**

名稱	資料型別	單位	描述
Start_X	I32	脈波數	起始座標 X 軸
Start_Y	I32	脈波數	起始座標 Y 軸
End_x	I32	脈波數	終點座標 X 軸
End_y	I32	脈波數	終點座標 Y 軸
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360o
Center_X	I32*	脈波數	中心座標 X 軸
Center_Y	I32*	脈波數	中心座標 Y 軸

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I32 Start_X=0, Start_Y=0;
I32 End_x=100000, End_y0;
I32 Center_X, Center_Y;
F64 Angle=180;

I16 status = _misc_app_get_circle_center_point(Start_X, Start_Y , End_x, End_y, Angle,
&Center_X, &Center_Y);
```

DELTA CONFIDENTIAL

42.3 _misc_set_record_debuging

■ **格式**

I16 PASCAL _misc_set_record_debuging (U16 enable)

■ **目的**

Debug 記錄功能是否啟動。

■ **參數**

名稱	資料型別	單位	描述
Enable	U16	選項單位	0：關閉 Debug 記錄 1：啟動 Debug 記錄

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 Enable = 1;

I16 status = _misc_set_record_debuging (Enable);

DELTA CONFIDENTIAL

42.4 _misc_open_record_debuging_file

■ **格式**

I16 PASCAL _misc_open_record_debuging_file (U16 CardNo,Char* file_name, U16 open)

■ **目的**

設置輸出 Debug 記錄存放位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
file_name	Char*	字元陣列單位	Debug 記錄文件存放路徑
open	U16	選項單位	0：存放在 VC6 集成開發環境 debug window 1：記錄存放在 file_name 設定的路徑文件中

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
Char file_name[20]=”log_output.txt”;
U16 CardNo=0,
U16 open=1;

I16 status = _misc_open_record_debuging_file(CardNo ,file_name, open);
```

DELTA CONFIDENTIAL

42.5　_DMC_01_enable_dda_mode

■　**格式**

I16 PASCAL _DMC_01_enable_dda_mode (U16 CardNo, U16 enable)

■　**目的**

啟動 DDA 填表功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15。
Enable	U16	選項單位	0：關閉 DDA 填表功能。 1：啟動 DDA 填表功能。 Bit0:Node1 Bit1:Node2 Bit2:Node3 Bit3:Node4 Bit4:Node5 Bit5:Node6 Bit6:Node7 Bit7:Node8 Bit8:Node9 Bit9:Node10 Bit10:Node11 Bit11:Node12

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo = 0;
U16 Enable = 5;　//啟用 Node1,3

I16 status = _DMC_01_enable_dda_mode (CardNo, Enable);

DELTA

42.6　_DMC_01_set_dda_data

■　**格式**

I16 PASCAL _DMC_01_set_dda_data (U16 CardNo, U32* abs_pos)

■　**目的**

填入 DDA Table 資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Abs_Pos	U32*	數值資料	此欄位為一 12 軸資料的陣列

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo = 0;

//Abs_Pos 為一 12 軸的資料陣列，最大 Buffer 為 1000 筆，每一 ms 會執行一筆 DDA Table 的資料，要注意的是在 Buffer 執行完畢之前，不接受其他指令(例如:Sd_Stop 等)。
U32 Abs_Pos[0][11] = {1000,2000,0,0,0,0,0,0,0,0,0,0};

I16 status = _DMC_01_set_dda_data (CardNo, Abs_Pos);
//其他未啟用 DDA Table 功能的 Node 可以正常執行其他動作。

DELTA CONFIDENTIAL

42.7　_DMC_01_get_dda_cnt

■　**格式**

I16 PASCAL _DMC_01_get_dda_cnt (U16 CardNo, U16* dda_cnt)

■　**目的**

取得目前 DDA Table 尚存的資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
dda_cnt	U16*	數值資料	目前 DDA Table 尚存的資料筆數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 dda_cnt;

I16 status = _DMC_01_get_dda_cnt (CardNo, & dda_cnt);
```

DELTA CONFIDENTIAL

42.8 _DMC_01_clear_dda_data

■ **格式**

I16 PASCAL _DMC_01_clear_dda_data (U16 CardNo)

■ **目的**

將 DDA Table 中尚存的資料清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_clear_dda_data (CardNo);
```

DELTA CONFIDENTIAL

42.9 _DMC_01_enable_dda_mode_group

■ **格式**

I16 PASCAL _DMC_01_enable_dda_mode_group(U16 CardNo,U16 GroupNo, U16 enable)

■ **目的**

設定 bit 指定要啟動的 NodeID。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
GroupNo	U16	編號單位	GroupNo:0 第一群、1:第二群
enable	U16	數值	GroupNo 所設定的啟動站號: Bit:0000 0000 0001=1(10 進制)->第 1 站 Bit:0000 0000 0011=3(10 進制)->第 2 站 請填入(10 進制)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 GroupNo =3;
U16 enable =1;

I16 status = _DMC_01_enable_dda_mode_group(U16 CardNo,U16 GroupNo, U16 enable)
```

DELTA CONFIDENTIAL

42.10　_DMC_01_set_dda_data_group

■　**格式**

I16 PASCAL_DMC_01_set_dda_data_group(U16 CardNo,U16 GroupNo, U32* abs_pos)

■　**目的**

設定點跟點之間絕對移動的距離。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
GroupNo	U16	編號單位	GroupNo:0 第一群、1:第二群
abs_pos	U32*	數值	輸入 DDA 點之間的絕對移動位置。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo = 0;
U16 GroupNo =;
U32 abs_pos ;

I16 status = _DMC_01_set_dda_data_group(U16 CardNo,U16 GroupNo, &abs_pos);

DELTA CONFIDENTIAL

42.11　_DMC_01_get_dda_cnt_group

■　**格式**

I16 PASCAL _DMC_01_get_dda_cnt_group(U16 CardNo,U16 GroupNo, U16* dda_cnt)

■　**目的**

取得 DDA Table 各群之間尚存的資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
GroupNo	U16	編號單位	GroupNo: 0 第一群、1:第二群
dda_cnt	U16*	數值	取得該群剩餘的 DDA 筆數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo = 0;
U16 GroupNo =;
U16 dda_cnt =;

I16 status = _DMC_01_get_dda_cnt_group(U16 CardNo,U16 GroupNo, U16* dda_cnt);

DELTA CONFIDENTIAL

42.12　_DMC_01_clear_dda_data_group

■　**格式**

I16 PASCAL _DMC_01_clear_dda_data_group(U16 CardNo,U16 GroupNo)

■　**目的**

將 DDA Table 各群之間尚存的資料清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
GroupNo	U16	編號單位	選擇要清除的 GroupNo:0 第一群、1:第二群

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo = 0;

U16 GroupNo =;

I16 status = _DMC_01_clear_dda_data_group(U16 CardNo,U16 GroupNo);

DELTA CONFIDENTIAL

42.13　_DMC_01_save_device_info

■　**格式**

I16 PASCAL _DMC_01_save_device_info (U16 CardNo, char * path)

■　**目的**

將模組資料存為文字檔。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
path	Char*	字串	存檔路徑及檔名。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
char path = “C:\log\log.txt”;

16 status = _DMC_01_save_device_info (CardNo, path);
```

DELTA CONFIDENTIAL

42.14　_DMC_01_compare_device_info

■　**格式**

I16 PASCAL _DMC_01_compare_device_info (U16 CardNo, char *path, char *pathLog)

■　**目的**

將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
path	char*	字串	比對模組的檔案路徑及檔名。
path_log	char *	字串	與模組資料比對結果存檔路徑及檔名。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
char path[50] = “C:\log\DMC_NET.txt”;
char path_log=“C:\log\log.txt”;

I16 status = _DMC_01_get_dda_cnt (CardNo, path, path_log);
```

DELTA CONFIDENTIAL

42.15　_DMC_01_enable_record_mode

■　**格式**

I16 PASCAL _DMC_01_enable_record_mode (U16 CardNo, U16 enable, U16 mode)

■　**目的**

開啟/關閉以毫秒為單位擷取 Monitor 所下參數的功能。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>軸卡號 CardNo 為 0~15</td></tr>
<tr><td>enable</td><td>U16</td><td>選項單位</td><td>0：Default
1：所下 Monitor 參數有動作時記錄。
2：所下 Monitor 參數有、無動作皆會記錄。</td></tr>
<tr><td>mode</td><td>U16</td><td>數值單位</td><td>
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>1</td><td>Node1 開啟/關閉。</td></tr>
<tr><td>2</td><td>Node2 開啟/關閉。</td></tr>
<tr><td>4</td><td>Node3 開啟/關閉。</td></tr>
<tr><td>8</td><td>Node4 開啟/關閉。</td></tr>
<tr><td>….</td><td></td></tr>
<tr><td>2048</td><td>Node12 開啟/關閉。</td></tr>
</table>
</td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0, status;
U16 NodeID[11]={1,2,3,4,5,0,0,0,0,0,0,0}, SlotID=0;
U16 monitorw=0; //監控馬達回授脈波數請參照手冊表 32.2
U16 mode=2;   //馬達不動作時也記錄其回授波數

status = _DMC_01_set_monitor(CardNo, NodeID, 0, monitorw);

status = _DMC_01_enable_record_mode (CardNo, 0, mode);
```

DELTA CONFIDENTIAL

42.16　_DMC_01_get_record_data

■　**格式**

I16 PASCAL _DMC_01_get_record_data (U16 CardNo, U32* record_data)

■　**目的**

讀取以毫秒為單位，Monitor 所下參數的 12 軸資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
record_data	U32*	數值單位	讀取以毫秒為單位，Monitor 所下參數的 12 軸資料。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U32 PPos[12] = {0};

I16 status = _DMC_01_get_record_data (CardNo, &PPos [0]);
```

DELTA CONFIDENTIAL

42.17　_DMC_01_get_record_cnt

■　**格式**

I16 PASCAL _DMC_01_get_record_cnt (U16 CardNo, U16* Record_Cnt)

■　**目的**

讀取以毫秒為單位擷取 Monitor 所下參數的資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Record_Cnt	U16*	數值單位	讀取以毫秒為單位，Monitor 所下參數的 12 軸資料筆數。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Record_Cnt=0;

I16 status = _DMC_01_get_record_cnt (CardNo, &Record_Cnt);
```

DELTA CONFIDENTIAL

42.18　_DMC_01_set_mc_fifo_mode

■　**格式**

I16 PASCAL _DMC_01_set_mc_fifo_mode (U16 CardNo, U16 mode)

■　**目的**

FIFO 參數設定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
mode	U16	選項單位	0：Default 1：For PCB 2：set_function_time will active

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0, status = 0;
U16 mode=2;
U32 ms1000=480;

status = _DMC_01_set_mc_fifo_mode ( CardNo, mode);
```

DELTA CONFIDENTIAL

42.19　_DMC_01_set_function_time

■　**格式**

I16 PASCAL _DMC_01_set_function_time (U16 CardNo,U32 ms1000)

■　**目的**

設定每 1ms 最小的通訊時間以確保通訊不會異常。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
ms1000	U32	數值單位	Fifo mode=2 開啟 Default 通訊時間為 600us 可調整值區間為 390~800us。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0, status = 0;
U16 mode=2;
U32 ms1000=480;


status = _DMC_01_set_function_time(CardNo, ms1000);
```

NOTE

此 API 所設定的時間為填入通訊封包所需的時間，通訊封包所需時間分為兩部分，一部分是基本時間(30us/站)，若有 12 站，基本上需要 360us，另一部分則視所執行運動的性質。

DELTA CONFIDENTIAL

42.20　_DMC_01_get_fifo_cnt

■　**格式**

I16 PASCAL _DMC_01_get_fifo_cnt(U16 CardNo, U16* fifo_cnt)

■　**目的**

取得 FIFO 暫存區資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
fifo_cnt	U16*	數值單位	Fifo Counter

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, fifo_cnt;

I16 status = _DMC_01_get_fifo_cnt(CardNo, &fifo_cnt);

DELTA CONFIDENTIAL

42.21　_DMC_01_set_gear

■　**格式**

I16 PASCAL _DMC_01_set_gear(U16 CardNo, U16 NodeID, U16 SlotID, I16 Numerator, I16 Denominator, U16 On_Off)

■　**目的**

啟動設定 Gear 功能與其參數設置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Numberator	I16	數值單位	電子齒輪比分子
Denominator	I16	數值單位	電子齒輪比分母
On_Off	U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 Numberator=1, Denominator=2;
U16 On_Off =1;

I16 status = _DMC_01_set_gear(CardNo, NodeID, SlotID, Numerator, Denominator, On_Off);

DELTA CONFIDENTIAL

42.22　_DMC_01_get_gear

■　**格式**

I16 PASCAL _DMC_01_get_gear(U16 CardNo, U16 NodeID, U16 SlotID, I16* Numerator, I16* Denominator, U16* On_Off)

■　**目的**

讀取 Gear 設定值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Numberator	*I16	數值單位	電子齒輪比分子
Denominator	*I16	數值單位	電子齒輪比分母
On_Off	*U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 Numberator, Denominator;
U16 On_Off;

I16 status = _DMC_01_get_gear(CardNo, NodeID, SlotID, &Numerator, &Denominator, &On_Off);

DELTA CONFIDENTIAL

42.23　_DMC_01_set_position_ref_source

■　**格式**

I16 PASCAL _DMC_01_set_position_ref_source (U16 CardNo, U16 NodeID, U16 SlotID, U16 ref_source)

■　**目的**

設定座標的參考來源模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ref_source	U16	選項單位	0：參考 Command 位置 1：參考 Position 位置 2：參考 Command + Offset 位置 3：參考 Position + Offset 位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 ref_source =0;

I16 status = _DMC_01_set_position_ref_source (CardNo, NodeID, SlotID, ref_source);
```

DELTA CONFIDENTIAL

42.24　_DMC_01_get_position_ref_source

■　**格式**

I16 PASCAL _DMC_01_get_position_ref_source (U16 CardNo, U16 NodeID, U16 SlotID, U16 *ref_source)

■　**目的**

取得目前座標的參考來源模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ref_source	U16*	選項單位	0：參考 Command 位置 1：參考 Position 位置 2：參考 Command + Offset 位置 3：參考 Position + Offset 位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 ref_source;

I16 status = _DMC_01_get_position_ref_source (CardNo, NodeID, SlotID, &ref_source);
```

DELTA CONFIDENTIAL

42.25　_DMC_01_set_virtual_command

■　**格式**

I16 PASCAL _DMC_01_set_virtual_command (U16 CardNo, U16 NodeID, U16 SlotID,
I32 virtual_command)

■　**目的**

Set position 的快速模式(會將實際 command 與 position 改成 virtual_command 的值)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_command	I32	數值單位	欲設置的 virtual command 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo = 0, status = 0;
U16 NodeID = 1, SlotID = 0, on_off = 0;

//0: 將實際 command 與 position 都改成 virtual_command 的值
//1: 只將實際 command 改成 virtual_command 的值
I32 virtual_command = 0;

status = _DMC_01_disable_virtual_pos (CardNo, NodeID, SlotID, on_off);

status = _DMC_01_set_virtual_command (CardNo, NodeID, SlotID, virtual_command);
```

DELTA CONFIDENTIAL

42.26　_DMC_01_get_virtual_command

■　**格式**

I16 PASCAL _DMC_01_get_virtual_command (U16 CardNo, U16 NodeID, U16 SlotID,
I32 *virtual_command)

■　**目的**

取得實際 command、position 與 virtual_command 的差值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_command	I32*	數值單位	取得實際 command、position 與 virtual_command 的差值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1, SlotID=0;
I32 virtual_command ;

I16 status = _DMC_01_get_virtual_command (CardNo, NodeID, SlotID, &virtual_command);

DELTA CONFIDENTIAL

42.27　_DMC_01_set_virtual_position

■　**格式**

I16 PASCAL _DMC_01_set_virtual_position (U16 CardNo, U16 NodeID, U16 SlotID,
I32 virtual_Position)

■　**目的**

將實際 position 值設定為 virtual_position 的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_Position	I32	數值單位	欲設置的 virtual position 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1, SlotID=0;
I32 virtual_Position =0;

I16 status = _DMC_01_set_virtual_position (CardNo, NodeID, SlotID, virtual_Position);

DELTA CONFIDENTIAL

42.28　_DMC_01_disable_virtual_pos

■　**格式**

I16 PASCAL _DMC_01_disable_virtual_pos (U16 CardNo, U16 NodeID, U16 SlotID, U16 on_off)

■　**目的**

選擇重置 Command 與 Position 的連動性。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
on_off	U16	選項單位	使用_DMC_01_set_virtual_command 時 0：Command 與 Position 都清除 1：只清除 Command

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 on_off =1;

I16 status = _DMC_01_disable_virtual_pos (CardNo, NodeID, SlotID, on_off);
```

DELTA CONFIDENTIAL

42.29　_DMC_01_set_stepping_move_rate

■　**格式**

I16 PASCAL _DMC_01_set_stepping_move_rate (U16 CardNo, U16 NodeID, U16 SlotID, U16 MoveRatio)

■　**目的**

設定解析放大的比率。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MoveRatio	U16	數值單位	解析放大的比率

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 MoveRatio =50;
//例如:RM04PI Mode2 CW/CCW 模式下，最大的速度限制為 200K，若 MoveRatio 設定為 50，則最大速度限制為 4K，速度設定超過 4K 會出現 ALM59，與 ALM9 以資區別。

I16 status = _DMC_01_set_stepping_move_rate (CardNo, NodeID, SlotID, MoveRatio);

NOTE

※注意：
RM04PI Mode1，只須設定 NodeID 站號，所有的 Slot 就會跟著調整 MoveRatio
RM04PI Mode2，根據設定的 NodeID 站號，來設定 MoveRatio。

DELTA CONFIDENTIAL

42.30　_DMC_01_MRAM_read_word

■　**格式**

I16 PASCAL _DMC_01_MRAM_read_word (U16 offset, U16* word_value)

■　**目的**

讀取 MH1(MP1)記憶體 MRAM 的值(單位:word，MRAM 範圍:0x0000~0XFFFE)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFE(MRAM 相對位址)
word_value	U16*	數值單位	讀取位於 MRAM 相對位址的資料(單位:word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U16 word_value;

I16 status = _DMC_01_MRAM_read_word (offset, &word_value);
```

DELTA CONFIDENTIAL

42.31　_DMC_01_MRAM_write_word

■　**格式**

I16 PASCAL _DMC_01_MRAM_write_word (U16 offset, U16 word_value)

■　**目的**

寫值到 MH1(MP1)的記憶體 MRAM(單位:word，MRAM 範圍:0x0000~0XFFFE)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFE(MRAM 相對位址)
word_value	U16	數值單位	寫值到位於 MRAM 的相對位址(單位:word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U16 word_value=0x3f5d;

I16 status = _DMC_01_MRAM_write_word (offset, word_value);
```

DELTA CONFIDENTIAL

42.32　_DMC_01_MRAM_read_dword

■　**格式**

I16 PASCAL _DMC_01_MRAM_read_dword (U16 offset, U32* dword_value)

■　**目的**

讀取 MH1(MP1)記憶體 MRAM 的值(單位:double word，MRAM 範圍:0x0000~0XFFFC)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFC(MRAM 相對位址)
dword_value	U32*	數值單位	讀取位於 MRAM 相對位址的資料 (單位:double word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U32 dword_value;

I16 status = _DMC_01_MRAM_read_dword (offset, &dword_value);
```

DELTA CONFIDENTIAL

42.33　_DMC_01_MRAM_write_dword

■　**格式**

I16 PASCAL _DMC_01_MRAM_write_dword (U16 offset, U32 dword_value)

■　**目的**

寫值到 MH1(MP1)的記憶體 MRAM(單位:double word，MRAM 範圍:0x0000~0XFFFC)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFC(MRAM 相對位址)
dword_value	U32	數值單位	寫值到位於 MRAM 的相對位址 (單位:double word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U32 dword_value=0x3f5d16d2;

I16 status = _DMC_01_MRAM_write_dword (offset, dword_value);
```

NOTE

範例中寫入 0x3f5d16d2，用 read_word 讀取位址 0x0001 位址，得到 16d2 資料，讀取位址 0x0003 位址，得到 3f5d 資料。

DELTA CONFIDENTIAL

42.34　_DMC_01_get_watchdog_cnt

■　**格式**

I16 PASCAL _DMC_01_get_watchdog_cnt (U16 CardNo, U32 *watchdog_cnt)

■　**目的**

讀取軸卡 WatchDog 功能目前的累積計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
watchdog_cnt	U32*	數值單位	WatchDog 目前的累積計數值(單位:ms)*1

*1. 當 watchdog_value 非 0 時，watchdog_cnt 每 1ms 會進行加 1。

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_cnt;

I16 status = _DMC_01_get_watchdog_cnt (CardNo, & watchdog_cnt);
```

DELTA CONFIDENTIAL

42.35　_DMC_01_set_watchdog_cnt

■　**格式**

I16 PASCAL _DMC_01_set_watchdog_cnt (U16 CardNo, U32 watchdog_cnt)

■　**目的**

修改軸卡 WatchDog 功能目前的累積計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
watchdog_cnt	U32	數值單位	欲設置的 watchdog_cnt 計數值(單位:ms)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_cnt;

I16 status = _DMC_01_set_watchdog_cnt (CardNo, watchdog_cnt);
```

DELTA CONFIDENTIAL

42.36　_DMC_01_get_watchdog_value

■　**格式**

I16 PASCAL _DMC_01_get_watchdog_value(U16 CardNo, U32 *watchdog_value)

■　**目的**

讀取軸卡 WatchDog 功能的計數目標值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
watchdog_value	U32*	數值單位	WatchDog 目前的計數目標值(單位:ms)*1

*****1. watchdog_value 為 0 時，watchdog_cnt 不會進行累積計數。

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_value ;

I16 status = _DMC_01_get_watchdog_value (CardNo, &watchdog_value);
```

DELTA CONFIDENTIAL

42.37　_DMC_01_set_watchdog_value

■　**格式**

I16 PASCAL _DMC_01_set_watchdog_value(U16 CardNo, U32 watchdog_value)

■　**目的**

設定軸卡 WatchDog 功能的計數目標值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
watchdog_value	U32	數值單位	WatchDog 的計數目標值(單位:ms)*1

*1. watchdog_value 為 0 時，watchdog_cnt 不會進行累積計數。

■　**屬性**

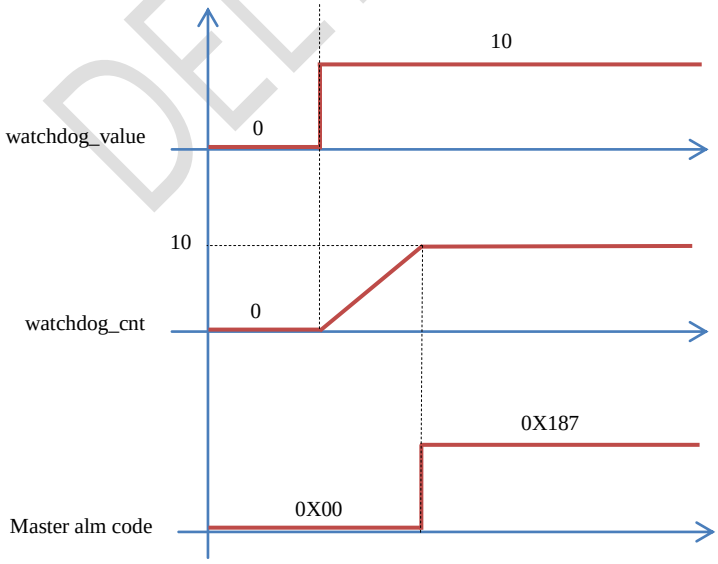
主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_value ;

I16 status = _DMC_01_set_watchdog_value (CardNo, watchdog_value);
```

NOTE

當 watchdog_value >0 且 watchdog_cnt = watchdog_value 時軸卡會觸發異警.
此時 master_alm_code 取回值為 0X187.　動作邏輯如下:



DELTA CONFIDENTIAL

42.38　_DMC_01_Security_Check_UserPassword

■　**格式**

I16 PASCAL　_DMC_01_Security_Check_UserPassword(U16 CardNo, U32 Password[8])

■　**目的**

Master 在讀/寫資料前，確認使用者對 Memory 讀/寫的權限登入。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Password[8]	U32	數值單位	輸入 Check 的密碼

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 Password[8]={0};

U16 status = _DMC_01_Security_Check_UserPassword(CardNo,Password);
```

NOTE

status 回傳值說明:
若 status =0，表示 API 動作完成。
若 status =352，表示操作中。
若 status =353，表示需再進行權限登入一次。
其他回傳值請參閱第二章錯誤傳回值一覽。

DELTA CONFIDENTIAL

42.39 _DMC_01_Security_Get_Check_UserPassword_State

■ **格式**

I16 PASCAL _DMC_01_Security_Get_Check_UserPassword_State(U16 CardNo, U16 *State)

■ **目的**

讀取目前 Msater 檢查權限登入的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
State	*U16	數值單位	0: Pass 1: Fail 2: Busy 3: No API command 4: Security communication error

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 State = 0;

U16 status = _DMC_01_Security_Get_Check_UserPassword_State(CardNo, &State);
```

NOTE

DELTA CONFIDENTIAL

42.39 _ DMC_01_set_watchdog_value

■ **格式**

I16 PASCAL _DMC_01_set_watchdog_value(U16 CardNo, U32 watchdog_value)

■ **目的**

設定軸卡 WatchDog 功能的計數目標值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
watchdog_value	U32	數值單位	WatchDog 的計數目標值(單位:ms)*1

*1. watchdog_value 為 0 時，watchdog_cnt 不會進行累積計數。

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 watchdog_value ;

I16 status = _DMC_01_set_watchdog_value (CardNo, watchdog_value);
```

NOTE

當 watchdog_value >0 且 watchdog_cnt = watchdog_value 時軸卡會觸發異警.
此時 master_alm_code 取回值為 0X187.　動作邏輯如下:

DELTA CONFIDENTIAL

42.40 _DMC_01_Security_Write_UserPassword

■ **格式**

I16 PASCAL _DMC_01_Security_Write_UserPassword(U16 CardNo, U32 Password[8])

■ **目的**

變更權限登入的密碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Password[8]	U32	數值單位	輸入要變更的密碼

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Password[8]={0};

U16 status = _DMC_01_Security_Write_UserPassword(CardNo,Password);
```

NOTE

status 回傳值說明:
若 status =0，表示 API 動作完成。
若 status =352，表示操作中。
若 status =353，表示需再進行權限登入一次。
其他回傳值請參閱第二章錯誤傳回值一覽。

DELTA CONFIDENTIAL

42.41 _DMC_01_Security_Get_Write_UserPassword_State

■ **格式**

I16 PASCAL _DMC_01_Security_Get_Write_UserPassword_State(U16 CardNo, U16 *State)

■ **目的**

檢查變更權限登入密碼的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
State	*U16	數值單位	0: Write_done 1: Fail 2: Busy 3: No API command 4: Security communication error

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 State= 0;

U16 status = _DMC_01_Security_Get_Write_UserPassword_State (CardNo,&State);
```

NOTE

DELTA CONFIDENTIAL

42.42　_DMC_01_Security_Check_Verifykey

■　**格式**

I16 PASCAL _DMC_01_Security_Check_Verifykey(U16 CardNo, U32 VerifyKey[8])

■　**目的**

Master 將驗證碼寫入驗證是否相符。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
VerifyKey[8]	U32	數值單位	輸入 Verifykey 的密碼

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 VerifyKey[8]={0};

U16 status = _DMC_01_Security_Check_Verifykey (CardNo, VerifyKey);
```

NOTE

status 回傳值說明:
若 status =0，表示 API 動作完成。
若 status =352，表示操作中。
若 status =353，表示需再進行權限登入一次。
其他回傳值請參閱第二章錯誤傳回值一覽。

DELTA CONFIDENTIAL

42.43 _DMC_01_Security_Get_Check_Verifykey_State

■ **格式**

I16 PASCAL _DMC_01_Security_Get_Check_Verifykey_State(U16 CardNo, U16 *State)

■ **目的**

讀取目前 Msater 檢查驗證的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
State	*U16	數值單位	0: Pass 1: Fail 2: Busy 3: No API command 4: Security communication error

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 State= 0;

U16 status = _DMC_01_Security_Get_Check_Verifykey_State(CardNo, &State);
```

NOTE

DELTA CONFIDENTIAL

42.44　_DMC_01_Security_Write_Verifykey

■　**格式**

I16 PASCAL _DMC_01_Security_Write_Verifykey(U16 CardNo, U32 VerifyKey[8])

■　**目的**

變更驗證的密碼。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
VerifyKey	U32	數值單位	輸入要變更的密碼

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 VerifyKey[8] ;

U16 status = _DMC_01_Security_Write_Verifykey(CardNo,VerifyKey);
```

NOTE

status 回傳值說明:

若 status =0，表示 API 動作完成。

若 status =352，表示操作中。

若 status =353，表示需再進行權限登入一次。

其他回傳值請參閱第二章錯誤傳回值一覽。

DELTA CONFIDENTIAL

42.45 _DMC_01_Security_Get_Write_Verifykey_State

■ **格式**

I16 PASCAL _DMC_01_Security_Get_Write_Verifykey_State(U16 CardNo, U16 *State)

■ **目的**

檢查變更驗證密碼的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
State	*U16	數值單位	0: Write_done 1: Fail 2: Busy 3: No API command 4: Security communication error

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 State= 0;

U16 status = _DMC_01_Security_Get_Write_Verifykey_State(CardNo, &State);
```

NOTE

DELTA CONFIDENTIAL

42.46 _DMC_01_buffer_correlation_master_slave

■ **格式**

I16 PASCAL _DMC_01_buffer_correlation_master_slave (U16 CardNo,U16 Slave_Node_Num,U16 Master_NodeID,U16* Slave_NodeID,U16 Slave_SlotID,I16 Enable)

■ **目的**

設定總線內伺服軸的 MotionBuffer 關連群組。

此 API 功能設定一個 master 軸為 Buffer 主軸，與多個 slave 軸，當使用者對任一 slave 軸(含主軸)下運動指令時，MotionBuffer 都會堆疊到主軸並依照主軸 Buffer 堆疊順序執行。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Slave_Node_Num	U16	編號單位	從軸數量(最多 11 軸)
Master_NodeID	U16	編號單位	主軸站號
Slave_NodeID	U16*	編號單位	存取用來從軸的站號 ID 數組 Slave_NodeID [0] 存放第一組站號 Slave_NodeID [1] 存放第二組站號 以此類推。
Slave_SlotID	U16*	編號單位	插槽 ID
Enable	I16	數值單位	0:關閉功能 1:啟動功能

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●	●						

■ **範例**

```
U16 CardNo=0;
U16 Slave_Node_Num = 3;
U16 Master_NodeID = 2;
U16 Slave_NodeID[3] = {1,3,4};
U16 Slave_SlotID[3] = {0}
I16 Enable = 1;
I16 status;
```

DELTA CONFIDENTIAL

status = _DMC_01_buffer_correlation_master_slave (CardNo,Slave_Node_Num, Master_NodeID,Slave_NodeID,Slave_SlotID,I16 Enable);

DELTA CONFIDENTIAL

42.47　_DMC_01_buffer_correlation_status

■　**格式**

I16 PASCAL _DMC_01_buffer_correlation_status (U16 CardNo,U16* Master_NodeID,U16* correlation_NodeID,U16* correlation_enable)

■　**目的**

讀取總線內伺服軸的 MotionBuffer 關連群組設定狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Master_NodeID	U16*	數值單位	目前設定的主軸站號
correlation_NodeID	U16*	數值單位	目前設定的關聯群組軸(含主軸) 例: master 設定為 2 slave 設定為 1、3、4 則 correlation_NodeID = 0xf (0xf 以二進位表示為 1111，bit0 代表第一軸，bit1 代表第二軸，以此類推。) 0:未被設為關聯軸。 1:設定為關聯軸。
correlation_enable	U16*	數值單位	0:功能為關閉狀態。 1:功能為啟動狀態。

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16* Master_NodeID;
U16* correlation_NodeID;
U16* correlation_enable;
I16 status;
```

status=_DMC_01_buffer_correlation_status(CardNo,&Master_NodeID,&correlation_NodeID,&correlation_enable);

DELTA CONFIDENTIAL

42.48　_DMC_01_GPIO_Set_Type

■　**格式**

I16 PASCAL　_DMC_01_GPIO_Set_Type(U16 CardNo, U16 PortNo, U16 Type);

■　**目的**

設定 RS-422/485 串列埠

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
PortNo	U16	編號單位	0->RS422_0 第一組 1->RS422_1 第二組
Type	U16	編號單位	0->RS422 1->RS485

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 PortNo = 0;
U16 Type = 0;// 0->RS422_0
I16 status;

status = _DMC_01_GPIO_Set_Type( CardNo, PortNo, Type);
```

DELTA CONFIDENTIAL

42.49　_DMC_01_Get_GPIO_Input_Value

■　**格式**

I16 PASCAL _DMC_01_Get_GPIO_Input_Value(U16 CardNo, U16 *Value);

■　**目的**

取得 MP1 GPIO 輸入腳位狀態

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Value	U16*	數值單位	0：關閉 1：開啟 MP1:12Input(ex:0xFFF, =bit0~bit11 為 ON 表示腳位有輸入)。

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Value;
I16 status;

status = _DMC_01_Get_GPIO_Input_Value(CardNo, &Value);
```

DELTA CONFIDENTIAL

42.50　_DMC_01_Get_GPIO_Output_Value

■　**格式**

I16 PASCAL　_DMC_01_Get_GPIO_Output_Value (U16 CardNo, U16 *Value);

■　**目的**

取得 MP1 GPIO 輸出腳位狀態

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Value	U16*	數值單位	0：關閉 1：開啟 MP1-A10D:4 output(ex:0xF, =bit0~bit3 為 ON 表示腳位有輸出)。 MP1-A12D:12 output(ex:0xFFF, =bit0~bit11 為 ON 表示腳位有輸出)。

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Value;
I16 status;

status = _DMC_01_Get_GPIO_Output_Value (CardNo, &Value);
```

DELTA CONFIDENTIAL

42.51　_DMC_01_Set_GPIO_Output_Value

■　**格式**

I16 PASCAL　_DMC_01_Set_GPIO_Output_Value (U16 CardNo, U16 Value);

■　**目的**

設定 MP1 GPIO 輸出腳位狀態

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Value	U16*	數值單位	0：關閉 1：開啟 MP1-A10D:4 output(ex:0xF, =bit0~bit3 為 ON 表示腳位輸出)。 MP1-A12D:12 output(ex:0xFFF, =bit0~bit11 為 ON 表示腳位輸出)。

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Value = 0xFF;
I16 status;

status = _DMC_01_Set_GPIO_Output_Value (CardNo, Value);
```

DELTA CONFIDENTIAL

42.52　_DMC_01_Set_GPIO_Output_error_handle

■ **格式**

I16 PASCAL　_DMC_01_Set_GPIO_Output_error_handle (U16 CardNo, U16 Value);

■ **目的**

設定 MP1 GPIO 保持輸出狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Value	U16	數值單位	0：關閉 1：開啟

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Value = 1;
I16 status;

status = _DMC_01_Set_GPIO_Output_error_handle (CardNo, Value);
```

NOTE

使用流程需在 _DMC_01_open 後設置啟動，功能開啟後關閉程式 GPIO 輸出狀態將會保持。

DELTA CONFIDENTIAL

42.53　_DMC_01_Get_GPIO_Output_error_handle

■　**格式**

I16 PASCAL　_DMC_01_Get_GPIO_Output_error_handle (U16 CardNo, U16 Value);

■　**目的**

取得 MP1 GPIO 保持輸出設定值

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
Value	U16*	數值單位	0：已關閉 1：已開啟

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Value;
I16 status;

status = _DMC_01_Get_GPIO_Output_error_handle (CardNo, &Value);
```

NOTE

DELTA CONFIDENTIAL

42.54　_DMC_01_get_mp1_dio_input

■　**格式**

I16 PASCAL _DMC_01_get_mp1_dio_input(U16 CardNo,U16 *On_Off)

■　**目的**

取得 MP1-P10D-15 8PIN 輸入腳位狀態(MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0
On_Off	U16*	數值單位	0：關閉 1：開啟 8 Input(ex:0xFF, = bit0~bit7 為 ON 表示腳位有輸入)。

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 On_Off;
I16 status;

status = _DMC_01_get_mp1_dio_input (CardNo, &On_Off);
```

NOTE

僅限版號 0x73 MP1-P10D-15 機種使用

DELTA CONFIDENTIAL

42.55　_DMC_01_get_mp1_dio_single_input

■　**格式**

I16 PASCAL _DMC_01_get_mp1_dio_single_input(U16 CardNo, U16 bitno, U16 *On_Off)

■　**目的**

取得 MP1-P10D-15　單一 bit 輸入腳位狀態(MP1-P10D-15 專用)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0
bitno	U16	選項單位	欲操作的 Bit(0~7) 如欲操作 Bit 0，bitno = 0 如欲操作 Bit 1，bitno = 1 如欲操作 Bit 2，bitno = 2 以此類推。
On_Off	U16*	數值單位	0：關閉 1：開啟

■　屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 bitno = 0;
U16 On_Off;
I16 status;

status = _DMC_01_get_mp1_dio_single_input (CardNo, bitno, &On_Off);
```

NOTE

僅限版號 0x73 MP1-P10D-15 機種使用

DELTA CONFIDENTIAL

第四十三章 B02 Compare

此章節說明了 B02 位置比較 Compare (API)，可設定/讀取 QEP Index 相位脈波模式與 Counter 功能，GPIO 的 CMP 功能，設定 ipcmp 與 tpcmp，設置 EXT GPIO 腳位輸出輸入狀態功能。

表 43

函數名稱	描述
_DMC_B02_qep_set_mode	設定 QEP Index 的脈波相位模式
_DMC_B02_qep_set_inverse	設定 QEP Index 脈波是否反向
_DMC_B02_qep_get_reload_counter	讀取 QEP Index 所設定的 Counter
_DMC_B02_qep_set_reload_counter	設定 QEP Index 的 Counter
_DMC_B02_qep_get_counter	讀取 QEP Index 實際的 Counter
_DMC_B02_gpio_software_control_status	讀取 CMP1~10 是否被設成 GPIO 輸出
_DMC_B02_gpio_software_control_setting	設置 CMP1~10 為 GPIO 輸出
_DMC_B02_ipcmp_initial	設置 ipcmp 的初始設定
_DMC_B02_ipcmp_oneshot_time	設置 ipcmp 執行一次 trigger 致能持續時間 (for oneshot and StartPosition use)
_DMC_B02_ipcmp_software_trigger	執行 ipcmp 一次 Trigger
_DMC_B02_ipcmp_compare_position	執行 ipcmp(StartPosition)
_DMC_B02_ipcmp_compare_trigger_count	讀取 ipcmp 已 Trigger 的次數
_DMC_B02_ipcmp_compare_trigger_count_clear	清除 ipcmp 已 Trigger 的次數
_DMC_B02_tpcmp_initial	設置 tpcmp(table)的初始設定
_DMC_B02_tpcmp_oneshot_time	設置 tpcmp 執行一次 trigger 致能持續時間
_DMC_B02_tpcmp_software_trigger	執行 tpcmp 一次 Trigger
_DMC_B02_tpcmp_software_trigger_level	執行 tpcmp 一次 Trigger Level
_DMC_B02_tpcmp_compare_position_table_reset	清除 compareTable 內的所有資料
_DMC_B02_tpcmp_compare_position_table	執行 tpcmp(table)
_DMC_B02_tpcmp_compare_position_table_level	執行 tpcmp(table level)
_DMC_B02_tpcmp_compare_trigger_count	讀取 tpcmp 已 Trigger 的次數
_DMC_B02_tpcmp_compare_trigger_count_clear	清除 tpcmp 已 Trigger 的次數
_DMC_B02_ext_io_get_input_value	取得 EXT GPIO 輸入腳位元狀態

CONFIDENTIAL

_DMC_B02_ext_io_get_output_value	取得 EXT GPIO 輸出腳位狀態
_DMC_B02_ext_io_set_output_value	設置 EXT GPIO 輸出腳位狀態
_DMC_B02_tpcmp_initial_2D	設置 2D tpcmp(table compare)的初始設定
_DMC_B02_tpcmp_oneshot_time_2D	設置 2D tpcmp(table compare)執行一次 trigger 致能持續時間
_DMC_B02_tpcmp_software_trigger_2D	執行 2D tpcmp(table compare)一次 Trigger
_DMC_B02_tpcmp_software_trigger_level_2D	執行 2D tpcmp(table compare)設定 Trigger Level
_DMC_B02_tpcmp_compare_position_table_2D	執行 2D tpcmp(table compare)，填入自訂 Table 表相關資料
_DMC_B02_tpcmp_compare_position_table_reset_2D	清除 2D compare table 內的所有資料
_DMC_B02_tpcmp_compare_position_table_level_2D	2D tpcmp(table compare)設定 Trigger Level
_DMC_B02_tpcmp_compare_axis_dist_2D	設置 2D tpcmp(table level compare)，X Y 軸誤差範圍
_DMC_B02_get_tpcmp_compare_axis_dist_2D	讀取設置的 2D tpcmp(table level compare)

此表格 API 僅 PCI_DMC_B02 可使用

API名稱 xxxxxx _ ipcmp _xxxxx 用於 等距 Compare 模式

xxxxxx _ tpcmp _xxxxx 用於非等距 Compare 模式

DELTA CONFIDENTIAL

43.1 _DMC_B02_qep_set_mode(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_qep_set_mode(U16 nCardNo, U8 nQepIndex, U8 nMode)

■ **目的**

設定 QEP Index(編碼器來源索引)的脈波相位模式。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nMode	U8	數值單位	0：CW/CCW 1：AB Phase

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
U8 nQepIndex=0;
U8 nMode = 0;     //CW/CCW

I16 status = _DMC_B02_qep_set_mode (nCardNo, nQepIndex ,nMode);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0, 無錯誤

DELTA CONFIDENTIAL

43.2 _DMC_B02_qep_set_inverse(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_qep_set_inverse(U16 nCardNo, U8 nQepIndex, U8 bEnable)

■ **目的**

設定 QEP Index(編碼器來源索引)脈波是否反向。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
bEnable	U8	數值單位	bEnable:0:Normal 1:Inverse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nQepIndex = 0;
U8 bEnable = 1; //Inverse

I16 status = _DMC_B02_qep_set_inverse (nCardNo, nQepIndex, bEnable);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.3 _DMC_B02_qep_get_reload_counter(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_qep_get_reload_counter(U16 nCardNo, U8 nQepIndex, I32* nCounter)

■ **目的**

讀取 QEP Index(編碼器來源索引)所設定的 Counter。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nCounter	I32*	數值單位	nQepIndex 的 Counter

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nQepIndex = 0;
I32 nCounter;

I16 status = _DMC_B02_qep_get_reload_counter (nCardNo, nQepIndex, &nCounter);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.4 _DMC_B02_qep_set_reload_counter(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_qep_set_reload_counter(U16 nCardNo, U8 nQepIndex, I32 nCounter)

■ **目的**

設定 QEP Index(編碼器來源索引)的 Counter。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nCounter	I32	數值單位	nQepIndex 的 Counter

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nQepIndex = 0;
I32 nCounter=0;

I16 status = _DMC_B02_qep_set_reload_counter (nCardNo, nQepIndex, nCounter);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.5 _DMC_B02_qep_get_counter(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_qep_get_counter(U16 nCardNo, U8 nQepIndex, I32* nCounter)

■ **目的**

讀取 QEP Index(編碼器來源索引)實際的 Counter。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nCounter	I32*	數值單位	nQepIndex 的 Counter

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nQepIndex = 0;
I32 nCounter;

I16 status = _DMC_B02_qep_get_counter (nCardNo, nQepIndex, &nCounter);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.6 _DMC_B02_gpio_software_control_status(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_gpio_software_control_status(U16 CardNo, U8 nPin, U8* nStatus)

■ **目的**

讀取 CMP1~10 GPIO 輸出的數值。一旦下達此 API 表示，CMP 腳位功能已事先切換為 GPIO 功能模式(已下達_DMC_B02_gpio_software_control_setting)，此時腳位則為 GPIO 功能腳 API 才有效。
註:如需切回 Compare 功能，則只需要再次設定_DMC_B02_ipcmp_initial 即可。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPin	U8	數值單位	nPin:0~9 選擇要觀察的腳位。
nStatus	U8*	數值單位	nStatus (說明:1:表示此腳位GPIO狀態為開啟) (說明:0:表示此腳位GPIO狀態為關閉)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nPin = 1 ; //觀察 CMP1+ 腳位物否開啟
U8 nStatus;

I16 status = _DMC_B02_gpio_software_control_status (nCardNo, nPin, &nStatus);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.7 _DMC_B02_gpio_software_control_setting(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_gpio_software_control_setting(U16 CardNo,U8 nPin, U8 nOnOff)

■ **目的**

設置 CMP1~10 為 GPIO 輸出狀態。
註:如需切回 Compare 功能，則只需要再次設定_DMC_B02_ipcmp_initial 即可。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPin	U8	數值單位	nPin: 0~9
nOnOff	U8	數值單位	nOnOff : 0(關閉) 、1(開啟)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nPin =1;//表示將 CMP1 設成 GPIO 輸出
U8 nOnOff =1; //開啟

I16 status = _DMC_B02_gpio_software_control_setting (nCardNo, nPin, nOnOff);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.8 _DMC_B02_ipcmp_initial

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_initial(U16 nCardNo, U8 nIpcmpIndex, U8 nQepIndex, U8 nGpioPin, U8 bInverse)

■ **目的**

設置 ipcmp(interval compare)的初始設定。一旦指定 IPCMP、訊號源(Encoder)與 GPIO 對應關係則會將資料 Bind 在一起。無法另外指定，除非重新 Initial 狀態，清除前次設定。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nGpioPin	U8	數值單位	nGpioPin(CMP1~10):0~9
bInverse	U8	數值單位	bInverse 0:Normal 1:Inverse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nIpcmpIndex = 0;
U8 nQepIndex = 0;
U8 nGpioPin = 1;
U8 bInverse = 1;

I16 status = _DMC_B02_ipcmp_initial (nCardNo, nIpcmpIndex , nQepIndex, nGpioPin,
                bInverse);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。
ERR_GPIO_OUTPUT_SIZE	Rt = 76, GPIO 索引值錯誤或者超出索引範圍。
ERR_IPCMP_INDEX	Rt = 77,ipcmp 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

範例程式:

```
short rt;
unsigned short l_nCardNo = g_nCardNo;
unsigned char l_nIpcmpIndex = g_nIpcmpIndex;
unsigned char l_nQepIndex = 0, l_nGpioPin = 0;
unsigned short l_nPulseWidth = (unsigned short)GetDlgItemInt(IDC_ED_One_Short, NULL, false);//脈波間隔
bool l_Linverse;

if(GetCheckValue(IDC_CHK_Inverse)) //脈波是否反向
{
      l_Linverse   = true;
}
else
{
      l_Linverse   = false;
}
     rt=   _DMC_B02_ipcmp_oneshot_time(l_nCardNo, l_nIpcmpIndex, l_nPulseWidth);

     rt=   _DMC_B02_ipcmp_initial(l_nCardNo,
           l_nIpcmpIndex,                    //CMPINDEX: 0~3
           l_nQepIndex,                      //QEP: 0~2
           l_nGpioPin,                       //GPIO: 1~10
           l_Linverse);                      //脈波準位 Inverse
```

DELTA CONFIDENTIAL

43.9 _DMC_ B02_ipcmp_oneshot_time

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_oneshot_time(U16 nCardNo, U8 nIpcmpIndex, U16 nPulseWidth)

■ **目的**

設置 ipcmp(interval compare)執行一次 trigger 致能持續時間(for oneshot and Interval use)。一旦完成上述設定即為 ipcomp_software_trigger 與 ipcmp_compare_position 都是依據此 nPulseWidth 執行。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3
nPulseWidth	U16	數值單位	nPulseWidth:0~4095 Trigger time公式:1.6*(nPulseWidth+1) 單位:us; Note1 :Trigger timer 設定若超過4095 則套用下列公式: ((X % 4096)*1.6) 單位:us; X:為非指定參數。 Note2 : (負值一律視為1個 nPulseWidth)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nIpcmpIndex = 0;
U16 nPulseWidth = 9; //16us

I16 status = _DMC_B02_ipcmp_oneshot_time (nCardNo, nIpcmpIndex , nPulseWidth);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。
ERR_IPCMP_INDEX	Rt = 77,ipcmp 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.10 _DMC_ B02_ipcmp_software_trigger

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_software_trigger(U16 nCardNo, U8 nIpcmpIndex)

■ **目的**

執行 ipcmp(interval compare)一次 SW Trigger，打出一次脈波訊號。通常用於測試脈波訊號是否正常。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3 0:CMP7 1:CMP8 2:CMP9 3:CMP10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 nCardNo = 0;
U8 nIpcmpIndex = 0;

I16 status = _DMC_B02_ipcmp_software_trigger(nCardNo, nIpcmpIndex);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。
ERR_IPCMP_INDEX	Rt = 77,ipcmp 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.11 _DMC_ B02_ipcmp_compare_position

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_compare_position(U16 nCardNo, U8 nIpcmpIndex, I32 nStartPosition, U16 nInterval, U32 nCount, U8 nDir)

■ **目的**

執行 ipcmp(interval compare)。與 B01 不同的設定方式，一旦下達此 API 則會將起使位置、interval、所需 count、脈波方向一併設定進去。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3
nStartPosition	I32	數值單位	Compare 起始位置
nInterval	U16	數值單位	脈波 Compare 間隔數
nCount	U32	數值單位	Trigger Compare 總數
nDir	U8	數值單位	0：CW　1：CCW

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nIpcmpIndex =0;
I32 nStartPosition = 100000;
U16 nInterval = 10; //10 pulse
U32 nCount = 50000;
U8 nDir = 0;

I16 status = _DMC_B02_ipcmp_compare_position (nCardNo, nIpcmpIndex, nStartPosition,
nInterval, nCount, nDir);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_IPCMP_INDEX	Rt = 77,ipcmp 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

測試項目: Internal Trigger(IPCMP)。

測試條件:04PI -12 軸同時作動 。

測試參數: Position[起使位置]: 1000, OneshotTime[Trigger 時間] : 1, Inverse[脈波間隔]:1。

✧ 4 組等間距 PCMP:QA_1[訊號源]->(7, 8, 9, 10)[GPIO 腳位]。

測試流程:依序遞減軸數，並提升頻寬，一看到 Trigger count 有停止的狀況，即判定此頻寬無效。並降低速度直到 ”可工作” 頻寬逼近臨界值。

✧ 可工作:判定軸運動正常,Trigger Count 計數正常。

Set_fun_timer:400	
軸數	頻寬
1	100400(100.4K)
2	100400 (100.4K)
3	100400 (100.4K)
4	100400 (100.4K)
5	100400 (100.4K)
6	100400 (100.4K)
7	100400 (100.4K)
8	100400 (100.4K)
9	100400 (100.4K)
10	100400 (100.4K)
11	100400 (100.3K)
12	100000 (100K)

Set_fun_timer:1000	
軸數	頻寬
1	100400 (100.4K)
2	100400 (100.4K)
3	100400 (100.4K)
4	100400 (100.4K)
5	100400(100.4K)
6	100400(100.4K)
7	100400(100.4K)
8	100400(100.4K)
9	100400(100.4K)
10	100400(100.4K)
11	100400(100.4K)
12	100000(100.4K)

DELTA CONFIDENTIAL

43.12 _DMC_ B02_ipcmp_compare_trigger_count

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_compare_trigger_count(U16 nCardNo, U8 nIpcmpIndex, U32* nCount);

■ **目的**

讀取 ipcmp(interval compare)已 Trigger 的次數。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3
nCount	U32*	數值單位	Trigger 的次數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nIpcmpIndex =0;
U32 nCount;

I16 status = _DMC_B02_ipcmp_compare_trigger_count (CardNo, nIpcmpIndex , &nCount);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。

DELTA CONFIDENTIAL

43.13 _DMC_ B02_ipcmp_compare_trigger_count_clear

■ **格式**

I16 PASCAL _DMC_B02_ipcmp_compare_trigger_count_clear(U16 nCardNo, U8 nIpcmpIndex)

■ **目的**

清除 ipcmp(interval compare)已 Trigger 的次數。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIpcmpIndex	U8	編號單位	nIpcmpIndex:0~3

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 nCardNo = 0;
U8 nIpcmpIndex = 0;

I16 status = _DMC_B02_ipcmp_compare_trigger_count_clear (nCardNo, nIpcmpIndex);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_IPCMP_INDEX	Rt = 77, ipcmp 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.14 _DMC_B02_tpcmp_initial

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_initial(U16 nCardNo, U8 nFifoIndex, U8 nQepIndex, U8 nGpioPin, U8 bInverse, U8 bInterrupt)

■ **目的**

設置 tpcmp(table compare)的初始設定。一旦指定 TPCMP、訊號源(Encoder)與 GPIO 對應關係則會將資料 Bind 在一起。無法另外指定，除非重新 Initial 狀態，清除前次設定。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
nQepIndex	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nGpioPin	U8	數值單位	nGpioPin(CMP1~10):0~9
bInverse	U8	數值單位	bInverse 0:Normal 1:Inverse
bInterrupt	U8	數值單位	保留

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex = 0;
U8 nQepIndex = 0;
U8 nGpioPin = 0;
U8 bInverse = 0; //此參數效果如同 Level
U8 bInterrupt = 0;

I16 status = _DMC_B02_tpcmp_initial (nCardNo, nFifoIndex,  nQepIndex, nGpioPin,
bInverse, bInterrupt);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75, Qep 索引值錯誤。
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。
ERR_TPCMP_QEP_SOURCE	Rt = 80, tpcmp qep 來源索引錯誤。

DELTA CONFIDENTIAL

完整範例:

```
short rt[12]; //回傳代碼暫存
unsigned short l_nCardNo = g_nCardNo; //卡號
unsigned char l_nTpcmpIndex[6] ={0,1,2,3,4,5};//預設定的 Channel
unsigned char l_nQepIndex[6] ={0,0,0,0,0,0}, l_nGpioPin[6] ={0,1,2,3,4,5 };
//l_nQepIndex 比較的訊號源                    //指定輸出的 PIN 腳位置

m_bInverse      = GetCheckValue(IDC_CHK_TAB_INVERSE);//脈波是否反向
m_bInterrupt    = GetCheckValue(IDC_CHK_TAB_INTUP);//暫不使用

m_nOneshotTime = (unsigned short)GetDlgItemInt(IDC_ED_TAB_ONE_SHORT, NULL, false);
//one shot time

//初始化 TPCMP 設定
for(int i=0    ; i<6 ; i++)
{
    rt[i] = _DMC_B02_tpcmp_oneshot_time(l_nCardNo, l_nTpcmpIndex[i], m_nOneshotTime);

    rt[i+6] =_DMC_B02_tpcmp_initial(l_nCardNo,
                        l_nTpcmpIndex[i],
                        l_nQepIndex[i],
                        l_nGpioPin[i],
                        m_bInverse,
                        m_bInterrupt);
}
```

DELTA CONFIDENTIAL

43.15 _DMC_B02_tpcmp_oneshot_time

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_oneshot_time(U16 nCardNo, U8 nFifoIndex, U16 nPulseWidth)

■ **目的**

設置 tpcmp(table compare)執行一次 trigger 致能持續時間。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
nPulseWidth	U16	數值單位	nPulseWidth:0~4095 Trigger time 公式:1.6*(nPulseWidth+1) 單位:us; Note1 :Trigger timer 設定若超過4095 則套用下列公式:((X % 4096)*1.6) 單位:us; X:為非法參數(例如:10000)。 Note2 : (負值一律視為 1 個 nPulseWidth)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

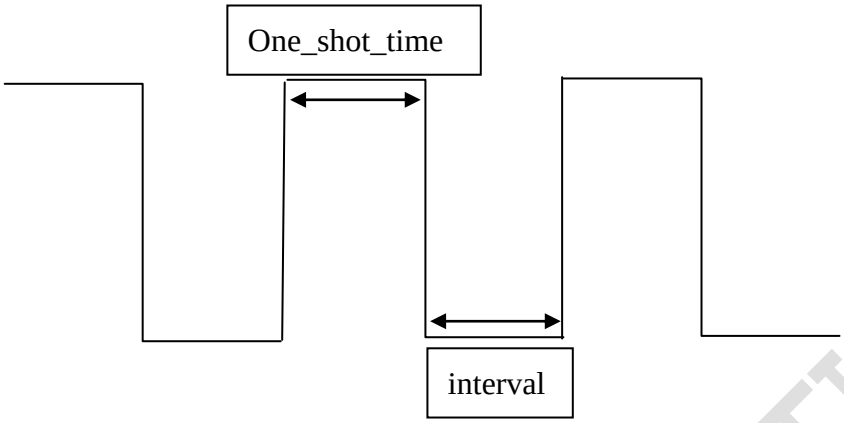
```
U16 nCardNo = 0;
U8 nFifoIndex = 0;
U16 nPulseWidth = 9; //16us

I16 status = _DMC_B02_tpcmp_oneshot_time (nCardNo, nFifoIndex, nPulseWidth);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

圖例說明:



DELTA CONFIDENTIAL

43.16 _DMC_B02_tpcmp_software_trigger

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_software_trigger(U16 nCardNo, U8 nFifoIndex)

■ **目的**

執行 tpcmp(table compare)一次 Trigger。打出一次脈波訊號。通常用於測試脈波訊號是否正常。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex = 0;

I16 status = _DMC_B02_tpcmp_software_trigger (nCardNo, nFifoIndex);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.17 _DMC_B02_tpcmp_software_trigger_level

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_software_trigger_level(U16 nCardNo, U8 nFifoIndex, U8 bActive)

■ **目的**

執行 tpcmp(table compare)設定 Trigger Level，觸發准位。H2L(5V)或者 L2H(0v)

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
bActive	U8	數值單位	bActive 0:low(0v) 1:high(5V)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex = 0;
U8 bActive=1;
I16 status = _DMC_B02_tpcmp_software_trigger_level (nCardNo, nFifoIndex, bActive);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.18 _DMC_B02_tpcmp_compare_position_table_reset

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table_reset(U16 nCardNo, U8 nFifoIndex)

■ **目的**

清除 compareTable 內的所有資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	數值單位	nFifoIndex:0~5

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex = 0;

I16 status = _DMC_B02_tpcmp_compare_position_table_reset (nCardNo, nFifoIndex);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.19 _DMC_B02_tpcmp_compare_position_table

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table(U16 nCardNo, U8 nFifoIndex, I32 nPositionTable[], U32 nTableSize)

■ **目的**

執行 tpcmp(table compare)，填入自訂 Table 表相關資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
nPositionTable[]	I32	數值單位	依照資料計算出需 Compare 的 PositionTable
nTableSize	U32	數值單位	PositionTable 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex = 0;
I32 nPositionTable[4] = {1000,2000,3000,4000};
U8 nTableSize =4;

I16 status = _DMC_B02_tpcmp_compare_position_table (nCardNo, nFifoIndex ,
nPositionTable, nTableSize);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_TABLE_NO_DATA	Rt = 81, tpcmp 內無 Table 資料。
ERR_TPCMP_TABLE_OUT_OF_RANGE	Rt =82, PositionTable 需 Compare 的值超過 255 或填寫的內容錯誤。
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。
ERR_TPCMP_TABLE_OVERFLOW	Rt = 83, FIFO 內容可填寫的內容值,超過容量大小。

DELTA CONFIDENTIAL

範例程式:

```
unsigned short l_nCardNo = g_nCardNo,i=0,j=0;
unsigned char l_nTpcmpIndex = g_nTpcmpIndex;
unsigned long l_nTableSize = 100000;
long* l_nPositionTable = new long[l_nTableSize];

 //針對每組設定 Table 表
 for(j=0 ; j<6 ; j++) //配置六組 PositionTable
 {
        //清除先前設定
        _DMC_B02_tpcmp_compare_trigger_count_clear(l_nCardNo, j);
        _DMC_B02_tpcmp_compare_position_table_reset(l_nCardNo, j);

        for (unsigned long I = 0; I < l_nTableSize;    ++I )
        {       //自動填入 Position Table  資料
                if (l_nPositionTable != NULL)
                {
                        l_nPositionTable[i] = 10*(i+1)+j;
                }
        }
        Sleep(50);

                // PositionTable
                _DMC_B02_tpcmp_compare_position_table(l_nCardNo, j, l_nPositionTable, l_nTableSize);

}
delete[] l_nPositionTable;
```

DELTA CONFIDENTIAL

測試項目: Table Trigger。

測試條件:04PI -12 軸同時作動 。

測試參數: Position: 10, OneshotTime: 10, Inverse: 10, Trigger Count: 100000。

✧ 測試訊號源:QA_1, GPIO =0;

✧ 6 組非等間距 TCMP

測試流程:依序遞減軸數，並提升測試頻寬(MaxVel)，一看到 trigger_count 有停止的狀況，即判定此測試頻寬無效。並降低速度直到"可工作"頻寬逼近臨界值。

✧ 可工作的判定:為軸運動正常,Trigger_Count 計數正常。

Set_fun_timer:400	
軸數	頻寬
1	62000(6.2K)
2	60000(6.0K)
3	56000(5.6K)
4	55000(5.5K)
5	53000(5.3K)
6	50000(5.0K)
7	48000(4.8K)
8	45000(4.5K)
9	43000(4.3K)
10	41000(4.1K)
11	38000(3.8K)
12	36000(3.6K)

Set_fun_timer:1000	
軸數	頻寬
1	59000(5.9K)
2	55000(5.5K)
3	54000(5.4K)
4	51000(5.1K)
5	49000(4.9K)
6	46000(4.6K)
7	44000(4.4K)
8	42000(4.2K)
9	39000(3.9K)
10	37000(3.7K)
11	34000(3.4K)
12	33000(3.3K)

DELTA CONFIDENTIAL

43.20 _DMC_B02_tpcmp_compare_position_table_level

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table_level(U16 nCardNo, U8 nFifoIndex, I32 nPositionTable[], U8 nLevelTable[], U32 nTableSize)

■ **目的**

執行 tpcmp(table level compare)，填入 Level Table 相關設定資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
nPositionTable[]	I32	數值單位	依照資料計算出需 Compare 的 PositionTable
nLevelTable[]	U8	數值單位	設定自訂 Compare 的 LevelTable
nTableSize	U32	數值單位	PositionTable 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

■ U16 nCardNo = 0;
U8 nFifoIndex = 0;
I32 nPositionTable[4] = {1000,2000,3000,4000};
U8 nLevelTable[4] ={1,0,0,0};//只有 1000 到了 High; 2000, 3000, 4000 到了都 low
U32 nTableSize = 4;

I16 status = _DMC_B02_tpcmp_compare_position_table_level (nCardNo, nFifoIndex , nPositionTable, nLevelTable , nTableSize);

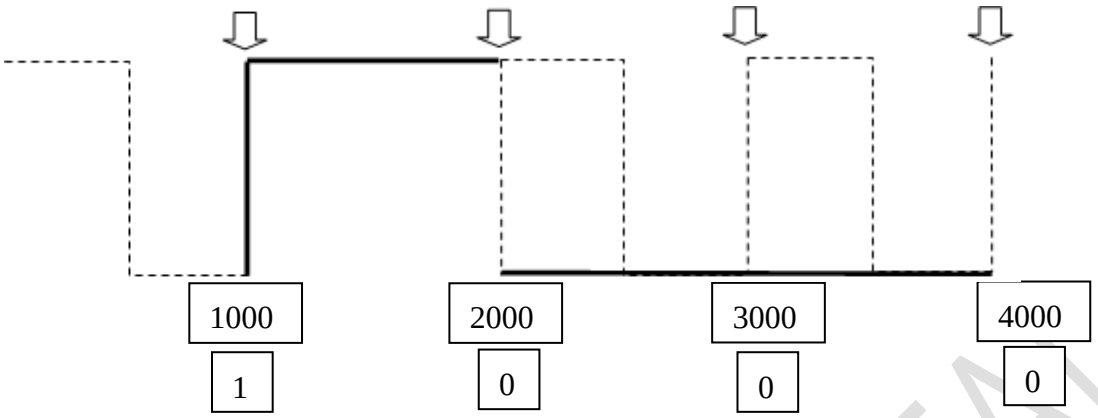
DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤。
ERR_TPCMP_TABLE_NO_DATA	Rt = 81, tpcmp 內無 Table 資料。
ERR_TPCMP_TABLE_OUT_OF_RANGE	Rt =82, PositionTable 需 Compare 的值超過 255 或填寫的內容錯誤。
ERR_TPCMP_FIFO_INDEX	Rt = 79, tpcmp FIFO 索引值錯誤或者超出索引範圍。
ERR_TPCMP_TABLE_OVERFLOW	Rt = 83, FIFO 內容可填寫的內容值,超過容量大小。

DELTA CONFIDENTIAL

圖例說明:



從圖示中可以看到 Level 未被設定為 1 的 LevelTable 輸出則為 low 。

DELTA CONFIDENTIAL

範例程式:

```
unsigned short l_nCardNo = g_nCardNo,i=0,j=0;
 unsigned char l_nTpcmpIndex = g_nTpcmpIndex;
 unsigned long l_nTableSize = 100000;
 long* l_nPositionTable = new long[l_nTableSize];
 unsigned char* l_nPositionLevel = new unsigned char[l_nTableSize];
                        //針對每組設定 Table 表
                        for(j=0;j<6;j++)
                        {
                            _DMC_B02_tpcmp_compare_trigger_count_clear(l_nCardNo, j);
                            _DMC_B02_tpcmp_compare_position_table_reset(l_nCardNo, j);
                            for (unsigned long i=0; i<l_nTableSize; ++i)
                            {
                                if (l_nPositionTable != NULL)
                                {
                                    l_nPositionTable[i] = 10*(i+1)+j;
                                }
                                if (l_nPositionLevel != NULL)
                                {
                                    l_nPositionLevel[i] = (i%2)?1:0;
                                }
                            }
                            Sleep(50);
  //設定 LevelTable
_DMC_B02_tpcmp_compare_position_table_level(l_nCardNo, j, l_nPositionTable, l_nPositionLevel, l_nTableSize);
                        }
 delete[] l_nPositionTable;
 delete[] l_nPositionLevel;
```

DELTA CONFIDENTIAL

測試項目: Table Level trigger。

測試條件:04PI -12 軸同時作動 。

測試參數: Position: 10, OneshotTime: 10, Inverse: 10, Trigger Count: 100000。

✧ 測試訊號源:QA_1, GPIO =0;

✧ 6 組非等間距 TCMP

測試流程:依序遞減軸數，並提升頻寬，一看到 Trigger count 有停止的狀況，即判定此頻寬無效。並降低速度直到 ”可工作” 頻寬逼近臨界值。

✧ 可工作:為軸運動正常,Trigger Count 計數正常。

Set_fun_timer:400	
軸數	頻寬(max_vel)
1	36000(3.6K)
2	35000(3.5K)
3	34000(3.4K)
4	32000(3.2K)
5	31000(3.1K)
6	30000(3.0K)
7	28000(2.8K)
8	26000(2.6K)
9	25000(2.5K)
10	24000(2.4K)
11	22000(2.2K)
12	21000(2.1K)

Set_fun_timer:1000	
軸數	頻寬(max_vel)
1	34000(3.4K)
2	33000(3.3K)
3	32000(3.2K)
4	30000(3.0K)
5	29000(2.9K)
6	27000(2.7K)
7	26000(2.6K)
8	25000(2.5K)
9	23000(2.3K)
10	22000(2.2K)
11	20000(2.0K)
12	19000(1.9K)

DELTA CONFIDENTIAL

43.21 _DMC_B02_tpcmp_compare_trigger_count(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_trigger_count(U16 nCardNo, U8 nFifoIndex, U32* nCount)

■ **目的**

讀取 tpcmp(table compare)已 Trigger 的次數。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U8	編號單位	nFifoIndex:0~5
nCount	U32*	數值單位	Trigger 的次數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nFifoIndex =0;
U32 nCount;

I16 status = _DMC_B02_tpcmp_compare_trigger_count(CardNo, nFifoIndex, &nCount);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.22 _DMC_B02_tpcmp_compare_trigger_count_clear(2D 共用)

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_trigger_count_clear(U16 nCardNo, U8 nFifoIndex);

■ **目的**

清除 tpcmp(table compare)已 Trigger 的次數。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nFifoIndex	U16*	編號單位	nFifoIndex:0~5

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 nCardNo = 0;
U8 nFifoIndex = 0;

I16 status = _DMC_B02_tpcmp_compare_trigger_count_clear(nCardNo, nFifoIndex);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤
ERR_TPCMP_FIFO_INDEX	Rt = 79,tpcmp FIFO 索引值錯誤或者超出索引範圍。

DELTA CONFIDENTIAL

43.23 _DMC_B02_ext_io_get_input_value

■ **格式**

I16 PASCAL _DMC_B02_ext_io_get_input_value(U16 nCardNo, U8* nIoState);

■ **目的**

取得 GPIO 輸入腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nIoState	U8*	編號單位	nIoState:0~7

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nIoState = 0;

I16 status = _DMC_B02_ext_io_get_input_value( nCardNo, &nIoState);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0, 無錯誤

DELTA CONFIDENTIAL

43.24 _DMC_B02_ext_io_get_output_value

■ **格式**

I16 PASCAL _DMC_B02_ext_io_get_output_value(U16 nCardNo, U8* nOutputValue);

■ **目的**

取得 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nOutputValue	U8*	編號單位	nOutputValue:0~3 (GPIO output :1~4 腳位)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nOutputValue = 0;

I16 status = _DMC_B02_ext_io_get_output_value(nCardNo, &nOutputValue);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

43.25 _DMC_B02_ext_io_set_output_value

■ **格式**

I16 PASCAL _DMC_B02_ext_io_set_output_value(U16 nCardNo, U8 nOutputValue);

■ **目的**

設置 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nOutputValue	U8	編號單位	nOutputValue:0~3

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nOutputValue = 0;

I16 status = _DMC_B02_ext_io_set_output_value(nCardNo, nOutputValue);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

43.26 _DMC_B02_tpcmp_initial_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_initial_2D(U16 nCardNo, U8 nQepIndex_X, U8 nQepIndex_Y, U8 nGpioPin, U8 bInverse, U8 bInterrupt, U8 Dimension);

■ **目的**

設置 2D tpcmp(table compare)的初始設定。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nQepIndex_X	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nQepIndex_Y	U8	編號單位	nQepIndex:0~2(編碼器來源索引)
nGpioPin	U8	編號單位	nGpioPin(CMP1~10):0~9
bInverse	U8	編號單位	bInverse 0:Normal 1:Inverse
bInterrupt	U8	編號單位	保留
Dimension	U8	編號單位	設置 table compare 模式 0: 1D 1: 2D

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 nCardNo = 0;
U8 nQepIndex_X = 0;
U8 nQepIndex_Y = 1;
U8 nGpioPin = 0;
U8 bInverse = 0;
U8 bInterrupt = 0;//(目前保留，無功能)
U8 Dimension = 1;

I16 status = _DMC_B02_tpcmp_initial_2D(nCardNo, nQepIndex_X, nQepIndex_Y, nGpioPin, bInverse, bInterrupt, Dimension);

DELTA CONFIDENTIAL

NOTE

2D table compare FifoIndex 只有 1 組，資料深度為 255

DELTA CONFIDENTIAL

43.27 _DMC_B02_tpcmp_oneshot_time_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_oneshot_time_2D(U16 nCardNo, U16 nPulseWidth);

■ **目的**

設置 2D tpcmp(table compare)執行一次 trigger 致能持續時間。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPulseWidth	U8	數值單位	nPulseWidth:0~4095 Trigger time 公式: (0.05us*(OneshotTime*32+31))

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nPulseWidth = 0;// (0.05us*(0*32+31)) = 1.55us

I16 status = _DMC_B02_tpcmp_oneshot_time_2D(nCardNo, nPulseWidth);
```

DELTA CONFIDENTIAL

43.28 _DMC_B02_tpcmp_software_trigger_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_software_trigger_2D(U16 nCardNo);

■ **目的**

執行 2D tpcmp(table compare)一次 Trigger。打出一次脈波訊號。通常用於測試脈波訊號是否正常。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;

I16 status = _DMC_B02_tpcmp_software_trigger_2D(nCardNo);
```

DELTA CONFIDENTIAL

43.29 _DMC_B02_tpcmp_software_trigger_level_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_software_trigger_level_2D(U16 CardNo, U8 bActive);

■ **目的**

執行 2D tpcmp(table compare)設定 Trigger Level，觸發准位。H2L(5V)或者 L2H(0v)

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
bActive	U8	編號單位	bActive 0:low(0v) 1:high(5V)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 bActive=1;
I16 status = _DMC_B02_tpcmp_software_trigger_level_2D(nCardNo, bActive);
```

DELTA CONFIDENTIAL

43.30 _DMC_B02_tpcmp_compare_position_table_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table_2D(U16 nCardNo, I32 nPositionTable_X[ ], I32 nPositionTable_Y[ ], U32 nTableSize);

■ **目的**

執行 2D tpcmp(table compare)，填入自訂 Table 表相關資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPositionTable_X[]	I32	數值單位	需 Compare 的 PositionTable X 軸
nPositionTable_Y[]	I32	數值單位	需 Compare 的 PositionTable Y 軸
nTableSize	U32	數值單位	PositionTable 資料筆數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
I32 nPositionTable_X [100000] = {100,200,300,400……10000000};
I32 nPositionTable_Y [100000] = {100,200,300,400……10000000};
U8 nTableSize =100000;

I16 status = _DMC_B02_tpcmp_compare_position_table_2D(nCardNo, nPositionTable_X,
nPositionTable_Y, nTableSize);
```

NOTE

X Y 兩軸最大可各填 100000 筆資料，TableSize 最大筆數為 100000 筆。

DELTA CONFIDENTIAL

43.31 _DMC_B02_tpcmp_compare_position_table_reset_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table_reset_2D(U16 nCardNo);

■ **目的**

清除 2D compare table 內的所有資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;

I16 status = _DMC_B02_tpcmp_compare_position_table_reset_2D(nCardNo);
```

DELTA CONFIDENTIAL

43.32 _DMC_B02_tpcmp_compare_position_table_level_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_position_table_level_2D(U16 nCardNo, I32 nPositionTable_X[ ], I32 nPositionTable_Y [ ], U8 nLevelTable[ ], U32 nTableSize);

■ **目的**

2D tpcmp(table level compare)，填入 Level Table 相關設定資料。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPositionTable_X[ ]	I32	數值單位	需 Compare 的 PositionTable X 軸
nPositionTable_Y[ ]	I32	數值單位	需 Compare 的 PositionTable Y 軸
nLevelTable[ ]	U8	數值單位	設定 Compare 的 LevelTable
nTableSize	U32	數值單位	PositionTable 資料筆數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
I32 nPositionTable_X [100000] = {100,200,300,400……10000000};
I32 nPositionTable_Y [100000] = {100,200,300,400……10000000};
U8 nLevelTable[100000] = {0,1,0,1……,1};//100000 筆

U8 nTableSize =100000;
I16 status = _DMC_B02_tpcmp_compare_position_table_level_2D (nCardNo,
nPositionTable_X, nPositionTable_Y, nTableSize);
```

NOTE

X Y 兩軸最大可各填 100000 筆資料，TableSize 最大筆數為 100000 筆，LevelTable 最大筆數為 100000 筆。

DELTA CONFIDENTIAL

43.33 _DMC_B02_tpcmp_compare_axis_dist_2D

■ **格式**

I16 PASCAL _DMC_B02_tpcmp_compare_axis_dist_2D(U16 nCardNo, I32 nPositionTable_X, I32 nPositionTable_Y);

■ **目的**

設置 2D tpcmp(table level compare)，X Y 軸誤差範圍。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPositionTable_X	I32	數值單位	X 軸誤差範圍(plus)
nPositionTable_Y	I32	數值單位	Y 軸誤差範圍(plus)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
I32 nPositionTable_X = 10;
I32 nPositionTable_Y = 10;

I16 status = _DMC_B02_tpcmp_compare_axis_dist_2D(nCardNo, nPositionTable_X,
nPositionTable_Y);
```

NOTE

DELTA CONFIDENTIAL

43.34 _DMC_B02_get_tpcmp_compare_axis_dist_2D

■ **格式**

I16 PASCAL _DMC_B02_get_tpcmp_compare_axis_dist_2D(U16 nCardNo, I32* nPositionTable_X, I32* nPositionTable_Y);

■ **目的**

讀取設置的 2D tpcmp(table level compare)，X Y 軸誤差範圍。

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nPositionTable_X	I32*	數值單位	X 軸誤差範圍(plus)
nPositionTable_Y	I32*	數值單位	Y 軸誤差範圍(plus)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
I32* nPositionTable_X;
I32* nPositionTable_Y;

I16 status = _DMC_B02_get_tpcmp_compare_axis_dist_2D(U16 nCardNo,
nPositionTable_X, nPositionTable_Y);
```

DELTA CONFIDENTIAL

第四十四章 TableMotion

此章節說明了 TableMotion(API)，可設置 TableMotion 循環模式運動資料，Table Motion federate 百分比，運動間混合百分比，速度規劃參數，計算轉角速度與後加減 FilterTime 功能。

表 44

API 歸屬	函數名稱	描述
A/B	_DMC_01_start_tablemotion	設置 TableMotion 循環模式的運動資料(各點間最大速度一致)
A/B	_DMC_01_start_tablemotion2	設置 TableMotion 循環模式的運動資料(可設定各點間最大速度，支援單步執行功能)
A/B	_DMC_01_start_tablemotion3	設置 TableMotion 循環模式的運動資料 (可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
A/B	_DMC_01_start_tablemotion_cyclemode	啟動 TableMotion 的循環模式
A/B	_DMC_01_start_tablemotion_buffer	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (各點間最大速度一致)
A/B	_DMC_01_start_tablemotion_buffer2	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (可設定各點間最大速度，支援單步執行功能)
A/B	_DMC_01_start_tablemotion_buffer3	Buffer Mode。於運動中使用此 API 可以一直新增運動資料 (可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓

DELTA CONFIDENTIAL

		路徑等功能)
A/B	_DMC_01_get_tablemotion_length	取得 Buffer 內可用的資料數量。
A/B	_DMC_01_tablemotion_feedrate_overwrite	設定 Table Motion federate 百分比
A/B	_DMC_01_tablemotion_feedhold	設定 FeedHold 是否開啟。
A/B	_DMC_01_set_tablemotion_blending_ratio	設定 Table Motion 運動間混合百分比。
A/B	_DMC_01_start_tablemotion_profile_planning_para	設定速度規劃模式的相關參數。
A/B	_DMC_01_start_tablemotion_profile_planning	根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。 (可設定各點間最大速度,支援單步執行功能)
A/B	_DMC_01_start_tablemotion_profile_planning2	根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。 (可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
A/B	_DMC_01_start_tablemotion_profilemode	啟動 TableMotion 速度規劃模式。
A/B	_DMC_01_set_axis_normalamf_enable	設定指定軸的後加減 FilterTime 與是否開啟
A/B	_DMC_01_set_tablemotion_ioctl_mapping	定義 Table 運動過程中每個運動間 IO 的輸出入狀態。
A/B	_DMC_01_set_normalamf_enable	啟用後加減功能
A/B	_DMC_01_get_tablemotion_current_speed	取得當前速度
A/B	_DMC_01_Execute_Single_Step	觸發啟動單步運轉功能
A/B	_DMC_01_Get_TableMotion_Now_MotionNo	取得當群運動的編號
A/B	_DMC_01_TableMotion_Single_Step	設定啟動單步運轉功能
A/B	_DMC_01_start_tableMotion_profileMode_data_flying_photography	兩軸過點飛拍功能，此 API 會根據設定的飛拍參數計算出新的運動軌跡及速度。

DELTA CONFIDENTIAL

44.1 _DMC_01_start_tablemotion

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData,I32 DataSize,I16 AxisNum, I32 MaxVel, F64 Tacc_Tdec, U16 AMFNum, U16 cycle, U32 *Motion_No, I32 *IOCtl)

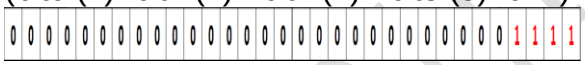
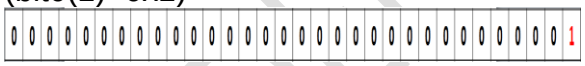
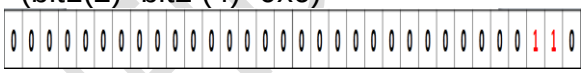
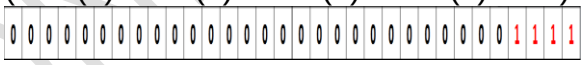
■ **目的**

設定 TableMotion 有循環次數模式。(各點間最大速度一致)
此 API 下了之後並不會馬上動作，需使用_DMC_01_start_tablemotion_cyclemode 才會開始動作。
此 API 的限制為無法一直填值(點位資料筆數)，一但使用者決定為循環模式的當下，就表示已經確定資料量小於 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數。
MaxVel	I32	數值單位	最大速度。(pulse/秒)
Tacc_Tdec	F64	數值單位	速度變化時間。(濾波時間:秒) 時間越長，轉角的實際移動誤差會增大，依照各個機構設計不同，使用者自行定義。
AMFNum	U16	數值單位	欲使用的濾波次數(AMFNum :0~2)。 建議值 2，數值大速度會較平滑，但是轉角部分的路線誤差會較大。

DELTA CONFIDENTIAL

cycle	U16	數值單位	循環次數。 Cycle = 0： 無限循環。 Cycle> 1： 循環次數。 當 Cycle = 0 時，末點座標須和起點座標相同。
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
IOCtl	U32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。 (bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。 (bit1(2)+bit2 (4)=0x6)  第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note ：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000

U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
```

DELTA CONFIDENTIAL

```
I32 DataSize = DATA_SIZE;          //此數值代表運動總點數，並不代表陣列長度。
I32 AxisNum =3;
I32 MaxVel = 1280000;
F64 Tacc_Tdec =0.1 ;
U16 AMFNum =0;
U16 Cycle =1;
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…..};
```

I16 status =_DMC_01_start_tablemotion (CardNo, NodeID,SlotID, 0, PositionData, DataSize, AxisNum, MaxVel, Tacc_Tdec, AMFNum, Cycle, Motion_No, IO_Table);

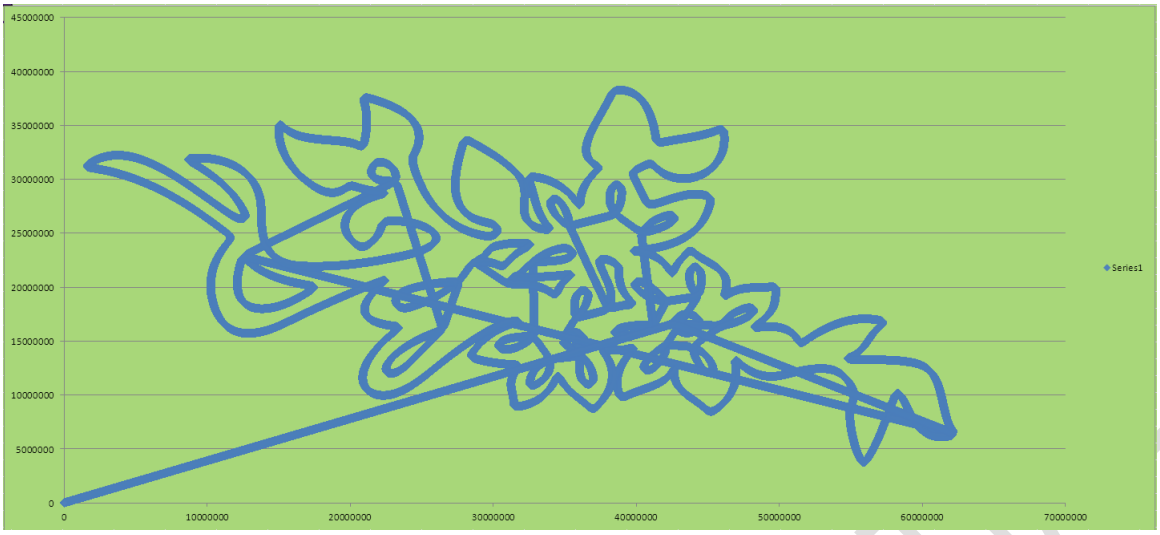
■ Error Code:

PCI_DMC_01_Err.h	Return Code
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用
ERR_PathError	Rt = 128, 移動距離為 0
ERR_TM_CYCLE_DATA	Rt =164，循環次數不等於 1 時，起始點和末點不同。
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限。
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_PROFILE_MODE_ENABLE	Rt = 200，已啟動速度規劃模式，因此無法使用此 API

應用範例(一):

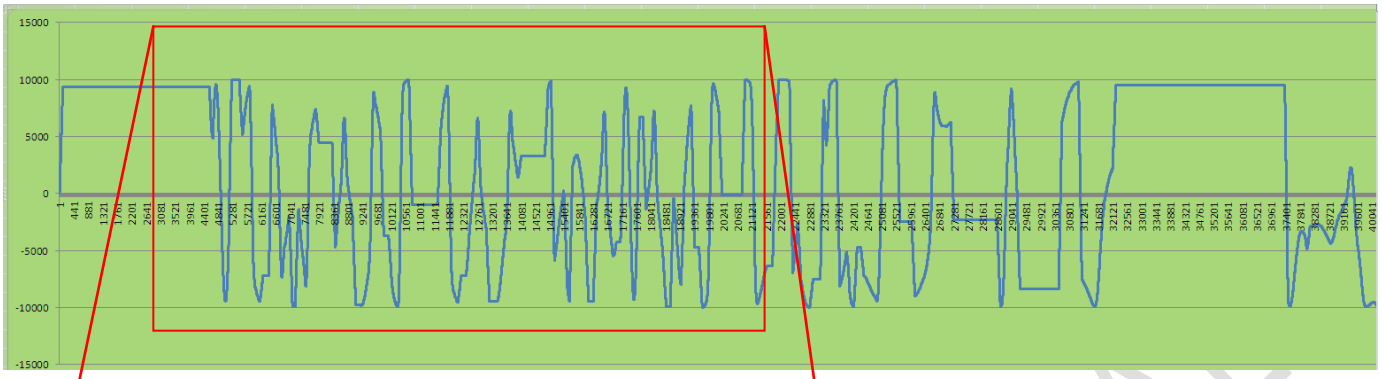
此範例圖(一)為實際應用加工內容生成的圖形

DELTA CONFIDENTIAL

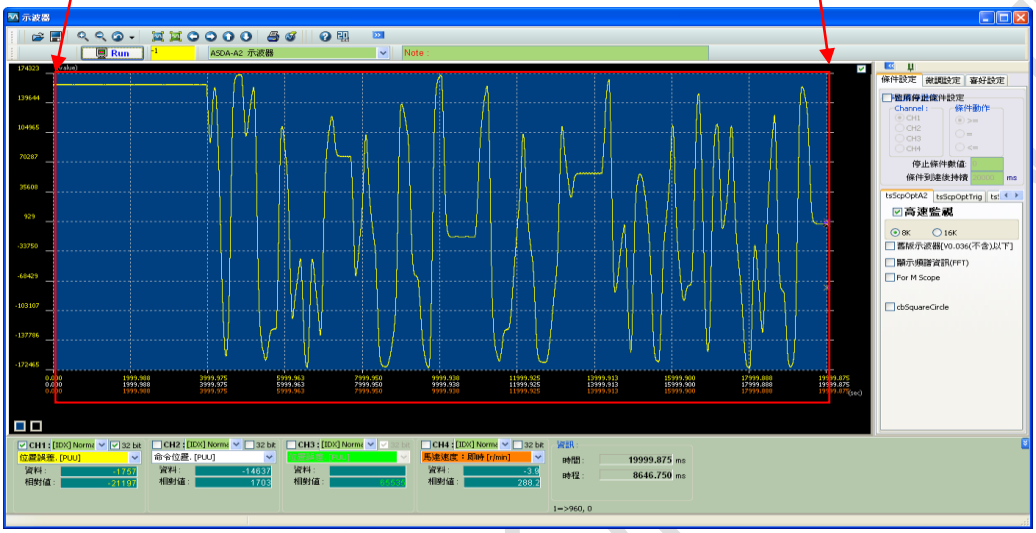


圖(一)命令點位圖

DELTA CONFIDENTIAL



圖(二)參照(圖一)實際伺服單軸速度曲線



圖(三) 實際伺服輸出速度曲線

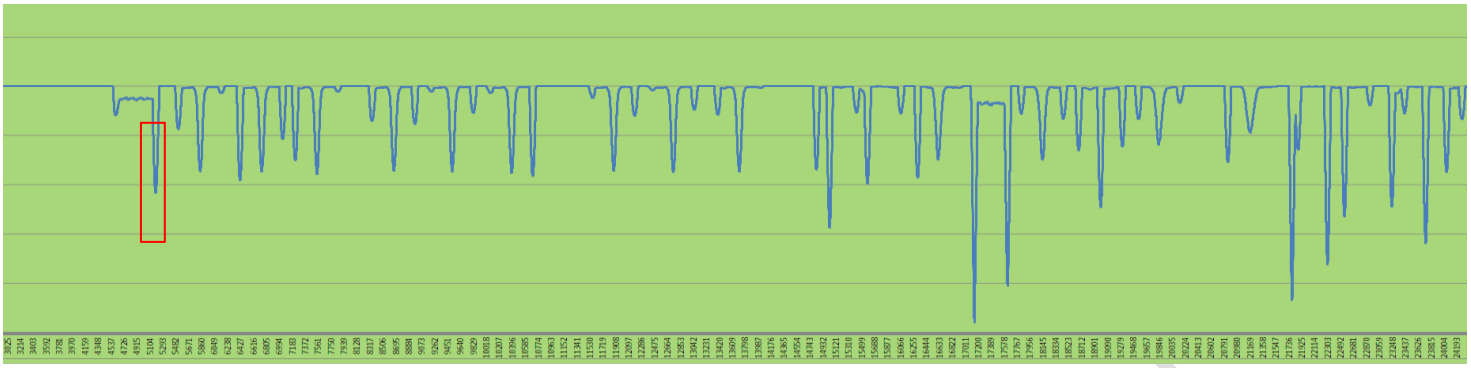
下列圖(四)為兩軸合速，可以由下圖看出，速度曲線振盪劇烈的部分為圖形加工的轉角轉折點，這部分跟實際應用上對於一般使用者跟加工精度需求有極大關聯。舉例:TableMotion 為使用者需要高加工精度的需求所以對於速度曲線(加工速度)不會有太大要求，反之今天如果需要提升加工速度，則不建議使用者使用此 API。

此時有兩個參照方案:

- 參考_DMC_01_start_tablemotion_profile_planning(44.9 節) 與 _DMC_01_start_tablemotion_profile_planning_para(44.10 節) 這兩個 API 來搭配使用，提升整體加工速度，一旦整體速度提升表示轉角加工的降速行為減少，隨之帶來的就是整體加工精度與圖面有些許誤差，誤差的多寡則依照使用者設定的參數。

- 參考_DMC_01_set_tablemotion_blending_ratio(44.8 節) 提升整個 Table 的轉角加工速度，與上述方法不同的地方在於，blending_ratio 可以將整體 Table 運動速度連貫起來，減少降速的動作產生。Profile 則個別針對轉角作設定。(在 Profile mode 無法支援 blending_ratio)。

DELTA CONFIDENTIAL



圖(四)

DELTA CONFIDENTIAL

44.2 _DMC_01_start_tablemotion2

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion2(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData,I32* MaxVelocity,I32 DataSize,I16 AxisNum, F64 Tacc_Tdec, U16 AMFNum, U16 cycle, U32 *Motion_No, I32 *IOCtl)

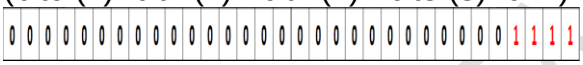
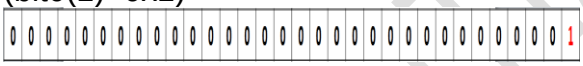
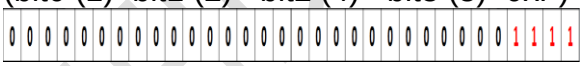
■ **目的**

設定 TableMotion 有循環次數模式。(可設定各點間最大速度，支援單步執行功能)
此 API 下了之後並不會馬上動作，需使用_DMC_01_start_tablemotion_cyclemode 才會開始動作。
此 API 的限制為無法一直填值(點位資料筆數)，一但使用者決定為循環模式的當下，就表示已經確定資料量小於 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
MaxVelocity	I32*	數值單位	各點間的最大速度。(pulse/秒)
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數。
Tacc_Tdec	F64	數值單位	速度變化時間。(濾波時間:秒) 時間越長，轉角的實際移動誤差會增大，依照各個機構設計不同，使用者自行定義。
AMFNum	U16	數值單位	欲使用的濾波次數(AMFNum :0~2)。 建議值 2，數值大速度會較平滑，但是轉角部分的路線誤差會較大。
cycle	U16	數值單位	循環次數。 Cycle = 0： 無限循環。 Cycle> 1： 循環次數。

DELTA CONFIDENTIAL

			當 Cycle = 0 時，末點座標須和起點座標相同。
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
IOCtl	U32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。(bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。(bit1(2)+bit2 (4)=0x6) 第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000

U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE;          //此數值代表運動總點數，並不代表陣列長度。
I32 AxisNum =3;
I32 MaxVelocity[MAX_DATA] ={v1,v2,v3,v4……};
F64 Tacc_Tdec =0.1 ;
```

DELTA CONFIDENTIAL

```
U16 AMFNum =0;
U16 Cycle =1;
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…..};

I16 status =_DMC_01_start_tablemotion2(CardNo, NodeID,SlotID, 0, PositionData, MaxVelocity,
DataSize, AxisNum, Tacc_Tdec, AMFNum, Cycle, Motion_No, IO_Table);
```

■ Error Code:

PCI_DMC_01_Err.h	Return Code
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用
ERR_PathError	Rt = 128, 移動距離為 0
ERR_TM_CYCLE_DATA	Rt =164，循環次數不等於 1 時，起始點和末點不同。
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限。
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_PROFILE_MODE_ENABLE	Rt = 200，已啟動速度規劃模式，因此無法使用此 API

DELTA CONFIDENTIAL

44.3 _DMC_01_start_tablemotion3

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion3(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData, I32 *Ref_Absolute_Pos,I32 *MaxVelocity,I32 DataSize,I16 AxisNum, F64 Tacc_Tdec, U16 AMFNum, U16 cycle,U32 *ArcBit, U32 *Motion_No, F64* Delay_CycleCount, U16 Abs_Mode,I32 *IOCtl);

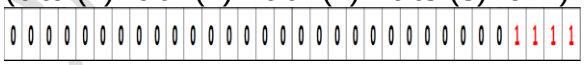
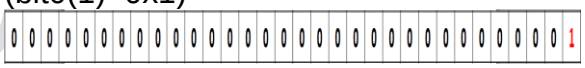
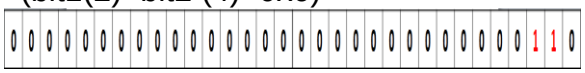
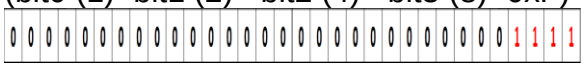
■ **目的**

設定 TableMotion 有循環次數模式。
(可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
此 API 下了之後並不會馬上動作，需使用_DMC_01_start_tablemotion_cyclemode 才會開始動作。
此 API 的限制為無法一直填值(點位資料筆數)，一但使用者決定為循環模式的當下，就表示已經確定資料量小於 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
Ref_Absolute_Pos	I32*	數值單位	相對模式時的參考位置陣列。
MaxVelocity	I32*	數值單位	各點間的最大速度。(pulse/秒)
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數。
Tacc_Tdec	F64	數值單位	速度變化時間。(濾波時間:秒) 時間越長，轉角的實際移動誤差會增大，依照各個機構設計不同，使用者自行定義。

DELTA CONFIDENTIAL

AMFNum	U16	數值單位	欲使用的濾波次數(AMFNum :0~2)。 建議值 2，數值大速度會較平滑，但是轉角部分的路線誤差會較大。
cycle	U16	數值單位	循環次數。 Cycle = 0： 無限循環。 Cycle> 1： 循環次數。 當 Cycle = 0 時，末點座標須和起點座標相同。
ArcBit	U32*	數值	此值為 0，則此單節為直線；如不為 0，則是用來標記此點是由哪幾軸做 Arc 運動。(註 1 :)
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
Delay_Cycle Count	U32*	數值	各單節運動完 Delay 的週期。
Abs_Mode	U16	選項	0： 相對模式 1： 絕對模式
IOCtl	U32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。(bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。(bit1(2)+bit2 (4)=0x6)  第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note ：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000

U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 Ref_Absolute_Pos[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE;          //此數值代表運動總點數，並不代表陣列長度。
I32 AxisNum =3;
I32 MaxVelocity[MAX_DATA] ={v1,v2,v3,v4.....…};
F64 Tacc_Tdec =0.1 ;
U16 AMFNum =0;
U16 Cycle =1;
U32 ArcBit[DATA_SIZE] = {3, 3, 3, 3,…..};
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U32 Delay_CycleCount[DATA_SIZE] = {1, 1, 1, 1,…..};
U16 Abs_Mode = 0 ;
U32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…..};

I16 status =_DMC_01_start_tablemotion2(CardNo, NodeID,SlotID, 0, PositionData, MaxVelocity,
DataSize, AxisNum, Tacc_Tdec, AMFNum, Cycle, Motion_No, IO_Table);
```

■ Error Code:

PCI_DMC_01_Err.h	Return Code
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用
ERR_PathError	Rt = 128, 移動距離為 0
ERR_TM_CYCLE_DATA	Rt =164，循環次數不等於 1 時，起始點和

DELTA CONFIDENTIAL

	末點不同。
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限。
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_PROFILE_MODE_ENABLE	Rt = 200，已啟動速度規劃模式，因此無法使用此 API

NOTE

ArcBit 使用說明

註 1 :

1.要連續兩個點的 ArcBit 不為 0，才能形成一個圓弧。

2.承上，連續三個點形成一個圓弧，ArcBit 需一樣，如以下說明

EX :

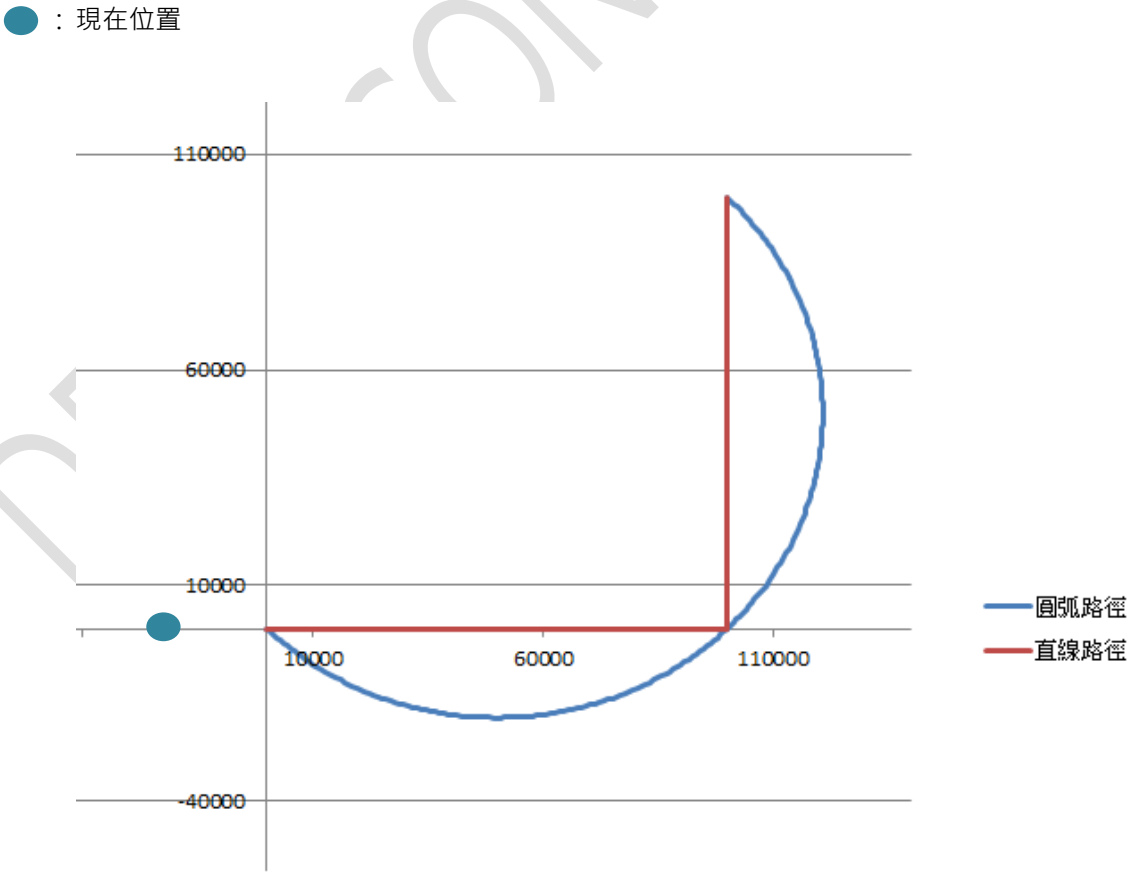
AxisNum = 2; DataSize = 3;

Position0 = {0, 0}, Position1 = {100000, 0}, Position2 = {100000, 100000}

現在位置 = Position0;

Ex1. 接下來 2 點 ArcBit 各為 3, 3, 則這 3 點都以第 0 軸、第 1 軸做 Arc 運動(藍色路徑)

Ex2. 接下來 2 點 ArcBit 各為 3, 5，因 ArcBit 非全一樣，所以此三點不能形成 Arc,會走直線(紅色路徑)



CONFIDENTIAL

3. 連續四個點的 ArcBit 不為 0可以形成兩個圓弧，說明如下 :
※綠色為現在位置的 Arcbit, 紅色為接下來2點的 ArcBit
Ex. ArcBit 各為 0, 3, 3, 3, 3, 前面3點都以第 0 軸、第1軸做 Arc 運動
0, 3, 3, 3, 3,後面3點以第 1 軸、第2軸要做 Arc 運動
以此類推，連續六個、八個……等。

4.如接下來 2 點 ArcBit 為 1, 1,
ArcBit = 1 , 表示 第 0 軸 要做 Arc 運動，單軸無法成圓弧，會走直線。
如接下來 2 點 ArcBit 為 15, 15
ArcBit = 15 , 表示 第 0 軸、第 1 軸、第 2 軸、第 3 軸 要做 Arc 運動，3 軸以上無法成圓弧，會走直線。

5.假使 TableMotion 使用軸數為 5 , 如接下來 2 點 ArcBit 為 3,3 ，
表示 第 0 軸、第 1 軸要做 Arc 運動，那麼其他軸的位置必須要一樣，否則會走直線。
(不允許 2 軸做圓弧，其他軸又做直線運動)
舉例如下 :

以下參數可以作用 ，會形成圓弧:
Position_1= {10000 ,0 ,0, 0, 0}, Position_2= {10000 ,10000 ,0, 0, 0}, Position_3= { 0,10000 , 0, 0, 0},

以下參數會做線性插補 :
Position_1= {10000 ,0 ,0, 0, 0}, Position_2= {10000 ,10000 ,0, 200, 0}, Position_3= {,0,10000 , 0, 0, 0},

6.如接下來 2 點 ArcBit 為 3, 3，以第 0 軸、第 1 軸做 Arc 運動，3 個點的位置如下
Position_1= {10000 ,10000 ,0,}, Position_2= {20000 ,20000 ,0, 0}, Position_3= {30000 ,30000 , 0},
因為 3 點共線無法形成圓弧，所以會做直線插補

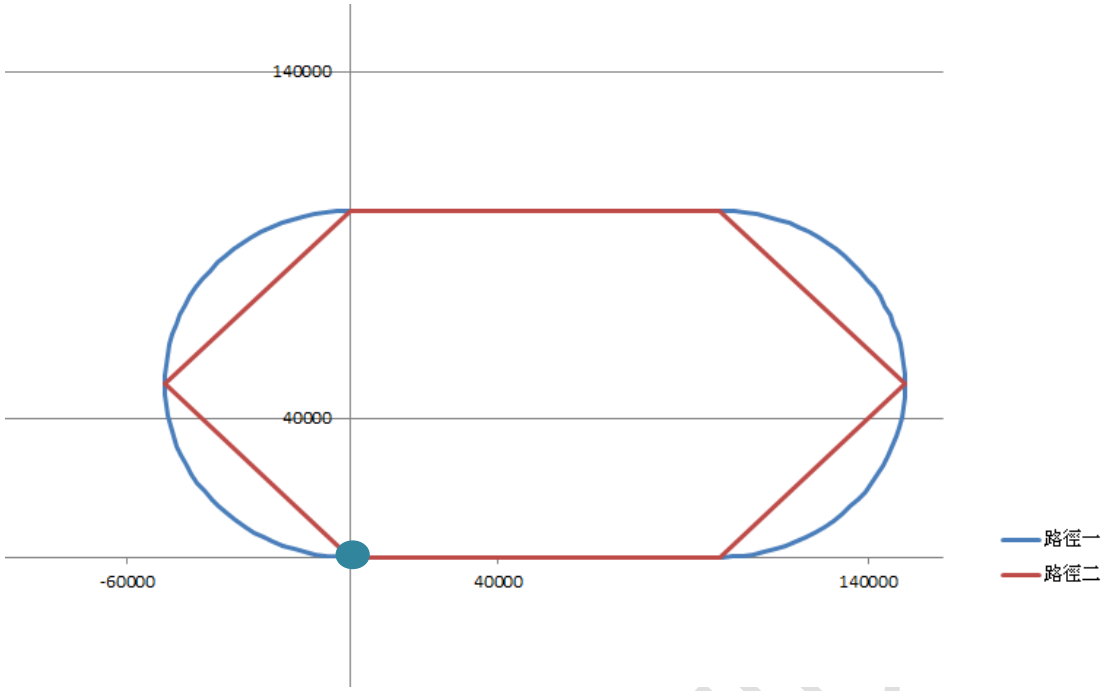
7.如接下來 2 點 ArcBit 為 3, 3 ，以第 0 軸、第 1 軸做 Arc 運動
如各點的 IO_Info 數值分別為 1 , 0, 2，則 IO 會以第三點為準(IO_Info[i] = 2)

8.如想要走直線路徑，則下一個點的 ArcBit 設為 0 即可
Example :

AxisNum = 2; DataSize = 7;
Position0 = {0, 0}, Position1 = {100000, 0}, Position2 = {150000, 50000},
Position3 = {100000, 100000}, Position4 = {0, 100000} , Position5 = {-50000, 50000}
Position6 = {0, 0}
ArcBit[7] = {0, 0, 3, 3, 0, 3, 3}
現在位置 = Position0;

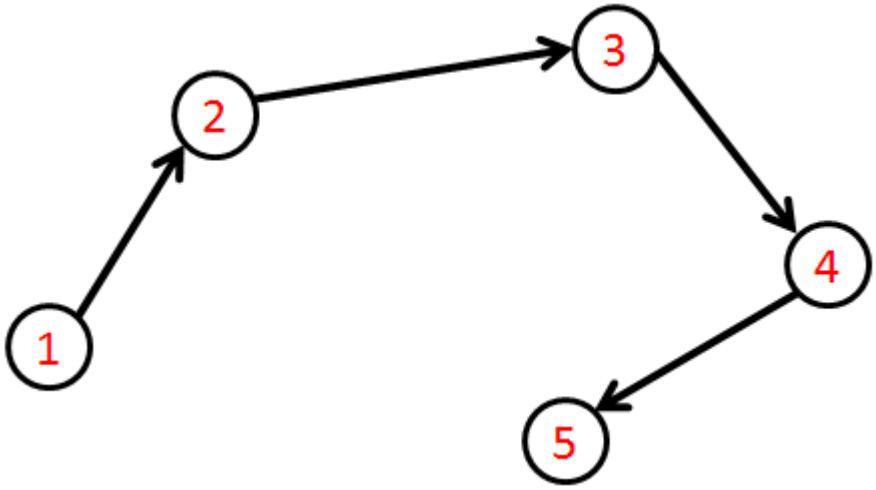
DELTA CONFIDENTIAL

: Position0

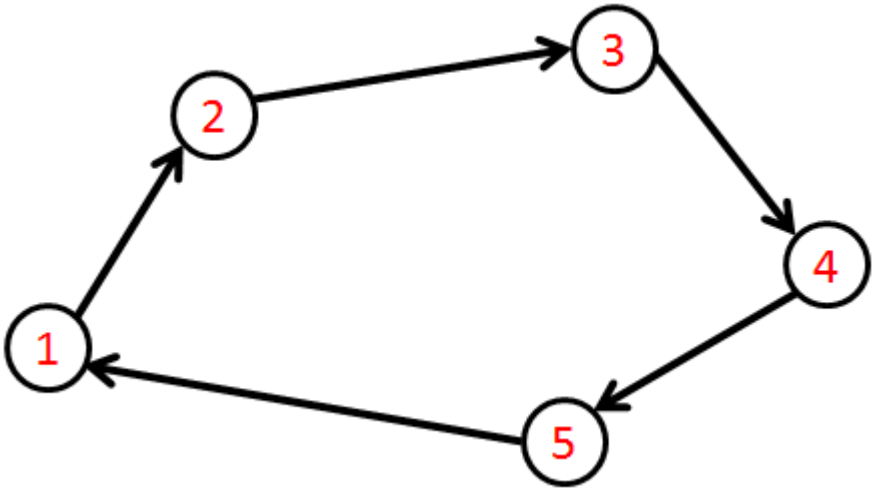


Cycle 使用說明

1. 當設定循環次數(Cycle = 1)時，運動會依照點位順序進行運動。



2. 當設定循環次數(Cycle 不等於 1)時，運動會依照點位順序進行循環運動。



DELTA CONFIDENTIAL

44.4 _DMC_01_start_tablemotion_cyclemode

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_cyclemode(U16 CardNo,I16 TableNo)

■ **目的**

一旦設定啟動 TableMotion 循環次數模式，當循環結束後，再次使用此 API 即可重複動作，不須再重下 Table。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	卡號
TableNo	I16	編號單位	Table 號碼(0 or 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16 TableNo =0;

I16 status =_DMC_01_start_tablemotion_cyclemode( CardNo,TableNo);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_NOT_TM_CYCLE_MODE	Rt = 201，尚未使用 CYCLE_MODE API
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

44.5 _DMC_01_start_tablemotion_buffer

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_buffer(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData,I32 DataSize,I16 AxisNum, I32 MaxVel, F64 Tacc_Tdec, U16 Single_Step_Enable, U32 *Motion_No, U16 AMFNum, I32 *IOCtl)

■ **目的**

設定 TableMotionBufferMode 模式。(各點間最大速度一致)
使用此 API 可以一直新增資料，但是以下參數在第一次使用_DMC_01_start_tablemotion_buffer 時就固定了：軸數、最大速、濾波時間、濾波次數。新增資料當下必須檢查 Buffer 是否超出剩餘可用數量，最多 2000 筆，且每次填值必須要於 2000。

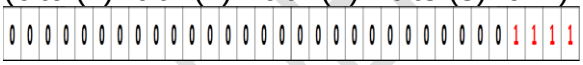
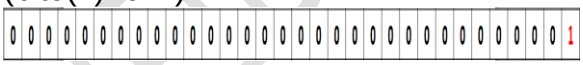
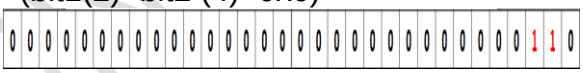
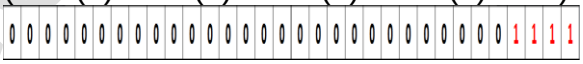
注意於實際使用方面必須考慮軸卡與 PC 溝通時間、通訊時間、伺服命令下達時間，所以建議值為每次填入 1500 筆資料，必須自行考慮當下配置各種因素。

假設當前預定運動比數超過 2000 筆，則可以在 Buffer 模式(**_DMC_01_start_tablemotion_buffer**) 底下執行。相對的 DMC_01_start_tablemotion 則無法超過 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	選項單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數
MaxVel	I32	數值單位	最大速度。(pulse/秒)
Tacc_Tdec	F64	數值單位	速度變化時間(濾波時間)
Single_Step_E	U16	選項單位	在 BufferMode 的模式下，控制單步運轉功能是否開啟。

DELTA CONFIDENTIAL

nable			此參數只有在 BufferMode 第一次執行時才有效，當 BufferMode 尚未結束時，再使用 BufferMode 填入新的位置資訊時，不參考使參數。 0：關閉單步運轉功能 1：開啟單步運轉功能
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
AMFNum	U16	數值單位	欲使用的濾波次數(0~2)。 建議值 2，速度會較平滑，但是轉角部分的誤差會大一點。
IOCtl	I32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。 (bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。 (bit1(2)+bit2 (4)=0x6)  第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000
U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
```

DELTA CONFIDENTIAL

```
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE; //此數值代表運動總點數，並不代表陣列長度。
I16 AxisNum =3;
I32 MaxVel = 1280000;
F64 Tacc_Tdec =0.1 ;
U16 Single_Step_Enable = 1 ;
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U16 AMFNum =0;
I32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…};

I16 status = _DMC_01_start_tablemotion_buffer (CardNo, NodeID,SlotID, 0, PositionData,
DataSize, AxisNum, MaxVel, Tacc_Tdec, Single_Step_Enable, Motion_No, AMFNum, IO_Table);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_OVER_TABLE_NUM	Rt = 203，超過最大 Table 數量
ERR_MOTION_DONE	Rt = 196，Buffer Mode 已結束，不能再下運動。
ERR_TM_INACTIVE	Rt = 167，Cycle Mode 正在作用
ERR_TM_BUFFER_FULL	Rt = 168，此次新增的運動數量超過剩餘可用的數量
ERR_PathError	Rt = 128, 移動距離為 0

DELTA CONFIDENTIAL

44.6 _DMC_01_start_tablemotion_buffer2

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_buffer2(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData,I32* MaxVelocity,I32 DataSize,I16 AxisNum, F64 Tacc_Tdec, U16 Single_Step_Enable, U32 *Motion_No, U16 AMFNum, I32 *IOCtl)

■ **目的**

設定 TableMotionBufferMode 模式。(可設定各點間最大速度，支援單步執行功能)
使用此 API 可以一直新增資料，但是以下參數在第一次使用_DMC_01_start_tablemotion_buffer 時就固定了 ： 軸數、最大速、濾波時間、濾波次數。新增資料當下必須檢查 Buffer 是否超出剩餘可用數量，最多 2000 筆，且每次填值必須要於 2000。

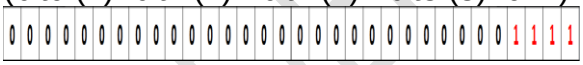
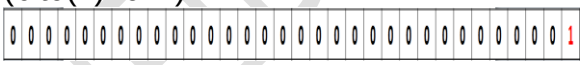
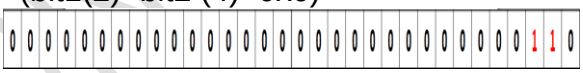
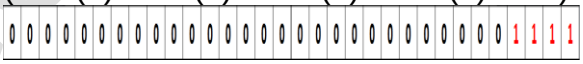
注意於實際使用方面必須考慮軸卡與 PC 溝通時間、通訊時間、伺服命令下達時間，所以建議值為每次填入 1500 筆資料，必須自行考慮當下配置各種因素。

假設當前預定運動比數超過 2000 筆，則可以在 Buffer 模式(**_DMC_01_start_tablemotion_buffer**) 底下執行。相對的 DMC_01_start_tablemotion 則無法超過 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	選項單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
MaxVelocity	I32*	數值單位	各點間的最大速度。(pulse/秒)
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數
Tacc_Tdec	F64	數值單位	速度變化時間(濾波時間)
Single_Step_E	U16	選項單位	在 BufferMode 的模式下，控制單步運轉功能是否開啟。

DELTA CONFIDENTIAL

nable			此參數只有在 BufferMode 第一次執行時才有效，當 BufferMode 尚未結束時，再使用 BufferMode 填入新的位置資訊時，不參考使參數。 0：關閉單步運轉功能 1：開啟單步運轉功能
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
AMFNum	U16	數值單位	欲使用的濾波次數(0~2)。 建議值 2，速度會較平滑，但是轉角部分的誤差會大一點。
IOCtl	I32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。 (bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。 (bit1(2)+bit2 (4)=0x6)  第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。 (bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000
U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
```

DELTA CONFIDENTIAL

```
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE; //此數值代表運動總點數，並不代表陣列長度。
I16 AxisNum =3;
I32 MaxVelocity[MAX_DATA] ={v1,v2,v3,v4.....…};
F64 Tacc_Tdec =0.1 ;
U16 Single_Step_Enable = 1 ;
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U16 AMFNum =0;
I32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…};

I16 status = _DMC_01_start_tablemotion_buffer2(CardNo, NodeID,SlotID, 0, PositionData,
MaxVelocity, DataSize, AxisNum, Tacc_Tdec, Single_Step_Enable, Motion_No, AMFNum,
IO_Table);
```

- **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_OVER_TABLE_NUM	Rt = 203，超過最大 Table 數量
ERR_MOTION_DONE	Rt = 196，Buffer Mode 已結束，不能再下運動。
ERR_TM_INACTIVE	Rt = 167，Cycle Mode 正在作用
ERR_TM_BUFFER_FULL	Rt = 168，此次新增的運動數量超過剩餘可用的數量
ERR_PathError	Rt = 128, 移動距離為 0

DELTA CONFIDENTIAL

DELTA CONFIDENTIAL

44.7 _DMC_01_start_tablemotion_buffer3

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_buffer3(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *PositionData, I32 *Ref_Absolute_Pos,I32 *MaxVelocity,I32 DataSize,I16 AxisNum, F64 Tacc_Tdec, U16 Single_Step_Enable, U32 *ArcBit, U32 *Motion_No, F64* Delay_CycleCount,U16 AMFNum, U16 Abs_Mode,I32 *IOCtl);

■ **目的**

設定 TableMotionBufferMode 模式。
(可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)
使用此 API 可以一直新增資料，但是以下參數在第一次使用_DMC_01_start_tablemotion_buffer 時就固定了 ： 軸數、最大速、濾波時間、濾波次數。新增資料當下必須檢查 Buffer 是否超出剩餘可用數量，最多 2000 筆，且每次填值必須要於 2000。

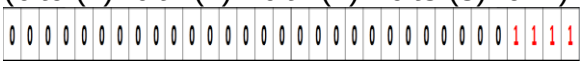
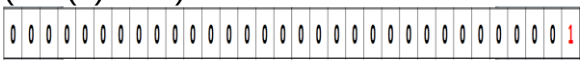
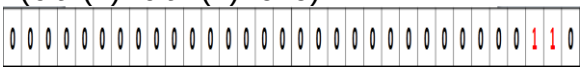
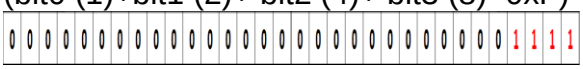
注意於實際使用方面必須考慮軸卡與 PC 溝通時間、通訊時間、伺服命令下達時間，所以建議值為每次填入 1500 筆資料，必須自行考慮當下配置各種因素。

假設當前預定運動比數超過 2000 筆，則可以在 Buffer 模式(**_DMC_01_start_tablemotion_buffer**) 底下執行。相對的 DMC_01_start_tablemotion 則無法超過 2000 筆。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	選項單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值單位	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
Ref_Absolute_Pos	I32*	數值單位	相對模式時的參考位置陣列。

DELTA CONFIDENTIAL

MaxVelocity	I32*	數值單位	各點間的最大速度。(pulse/秒)
DataSize	I32	數值單位	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值單位	使用的軸數
Tacc_Tdec	F64	數值單位	速度變化時間(濾波時間)
Single_Step_Enable	U16	選項單位	在 BufferMode 的模式下，控制單步運轉功能是否開啟。此參數只有在 BufferMode 第一次執行時才有效，當 BufferMode 尚未結束時，再使用 BufferMode 填入新的位置資訊時，不參考使參數。 0：關閉單步運轉功能 1：開啟單步運轉功能
ArcBit	U32*	數值	此值為 0，則此單節為直線；如不為 0，則是用來標記此點是由哪幾軸做 Arc 運動。(註 1:)
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合“44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
Delay_CycleCount	U32*	數值	各單節運動完 Delay 的週期。
AMFNum	U16	數值單位	欲使用的濾波次數(0~2)。 建議值 2，速度會較平滑，但是轉角部分的誤差會大一點。
Abs_Mode	U16	選項	0：相對模式 1：絕對模式
IOCtl	I32*	數值單位	PositionData 與 PositionData 資料點之間的 DO 資訊。 範例:IOControl[4] = {0xF, 0x1, 0x6, 0xF} 當前位置至第 1 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  第 1 點位置至第 2 點位置之前會觸發 0x1(1 點 DO 輸出)。(bit0(1)=0x1)  第 2 點位置至第 3 點位置之前會觸發 0x6(2 點 DO 輸出)。(bit1(2)+bit2 (4)=0x6)  第 3 點位置至第 4 點位置之前會觸發 0xF(4 點 DO 輸出)。(bit0 (1)+bit1 (2)+ bit2 (4)+ bit3 (8)=0xF)  Note：實際開啟的輸出裝置必須執行 _DMC_01_set_tablemotion_ioctl_mapping 來指定輸出的 DO 裝置。

DELTA CONFIDENTIAL

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAX_DATA 2000
#define DATA_SIZE 1000
U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 PositionData[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 Ref_Absolute_Pos[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE; //此數值代表運動總點數，並不代表陣列長度。
I16 AxisNum =3;
I32 MaxVelocity[MAX_DATA] ={v1,v2,v3,v4.....};
F64 Tacc_Tdec =0.1 ;
U16 Single_Step_Enable = 1 ;
U32 ArcBit[DATA_SIZE] = {3, 3, 3, 3,…..};
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U32 Delay_CycleCount[DATA_SIZE] = {1, 1, 1, 1,…..};
U16 AMFNum =0;
U16 Abs_Mode = 0 ;
I32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7,…};


I16 status = _DMC_01_start_tablemotion_buffer2(CardNo, NodeID,SlotID, 0, PositionData,
MaxVelocity, DataSize, AxisNum, Tacc_Tdec, Single_Step_Enable, Motion_No, AMFNum,
IO_Table);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_AMFNum	Rt = 166，AMFNum 超過 2 道或小於 1 道
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_OVER_TABLE_NUM	Rt = 203，超過最大 Table 數量
ERR_MOTION_DONE	Rt = 196，Buffer Mode 已結束，不能再下運動。
ERR_TM_INACTIVE	Rt = 167，Cycle Mode 正在作用
ERR_TM_BUFFER_FULL	Rt = 168，此次新增的運動數量超過剩餘可用的數量
ERR_PathError	Rt = 128, 移動距離為 0

DELTA CONFIDENTIAL

44.8 _DMC_01_get_tablemotion_length

■ **格式**

I16 PASCAL _DMC_01_get_tablemotion_length(U16 CardNo,I16 TableNo,I32 *cnt)

■ **目的**

回傳指定的 Table，Buffer 裡剩餘可以使用的資料數量。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)。
cnt	I32	數值單位	Buffer 可以使用的剩餘大小。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16 TableNo=0;
I32 cnt ;

I16 status =  _DMC_01_get_tablemotion_length( CardNo, TableNo,&cnt);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_OVER_TABLE_NUM	Rt = 103，TableNo 超過最大 Table 數量

DELTA CONFIDENTIAL

44.9 _DMC_01_tablemotion_feedrate_overwrite

■ **格式**

I16 PASCAL_DMC_01_tablemotion_feedrate_overwrite(U16 CardNo,I16 TableNo, I32 ratio,F64 sec)

■ **目的**

設定 TableMotion 的 FeedRate_OverWrite 達到運動當下變速的功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
ratio	I32	數值	設定 FeedRate 百分比。
sec	I16	編號單位	設定速度變化時間。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16 TableNo=0;
I32 ratio=100;(單位:百分比)
I16 sec =10;(單位:秒)

I16 status = _DMC_01_tablemotion_feedrate_overwrite( CardNo, TableNo,   ratio, sec);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_SPEED_RATIO	Rt = 95，ratio 超過預設最大百分比(3000%)
ERR_TM_FEEDRATE_OVERWRITE	Rt = 98，正在進行 FeedRate_OverWrite

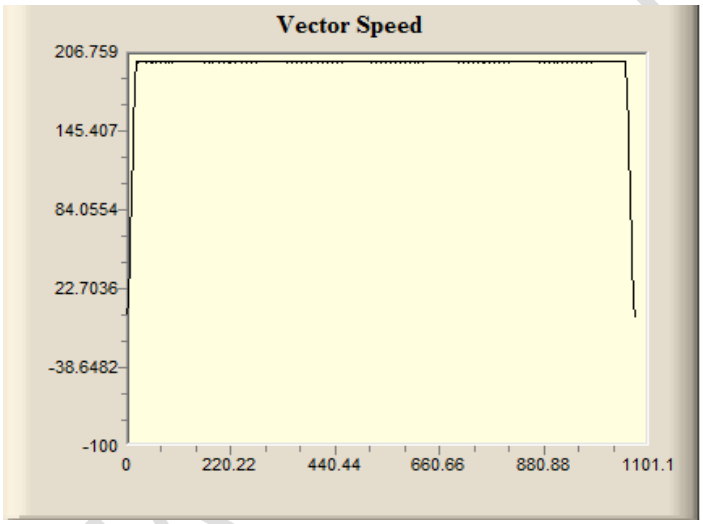
DELTA CONFIDENTIAL

程式範例:

測試說明:二軸，多段直線並在運動中改 FeedRate 的測試

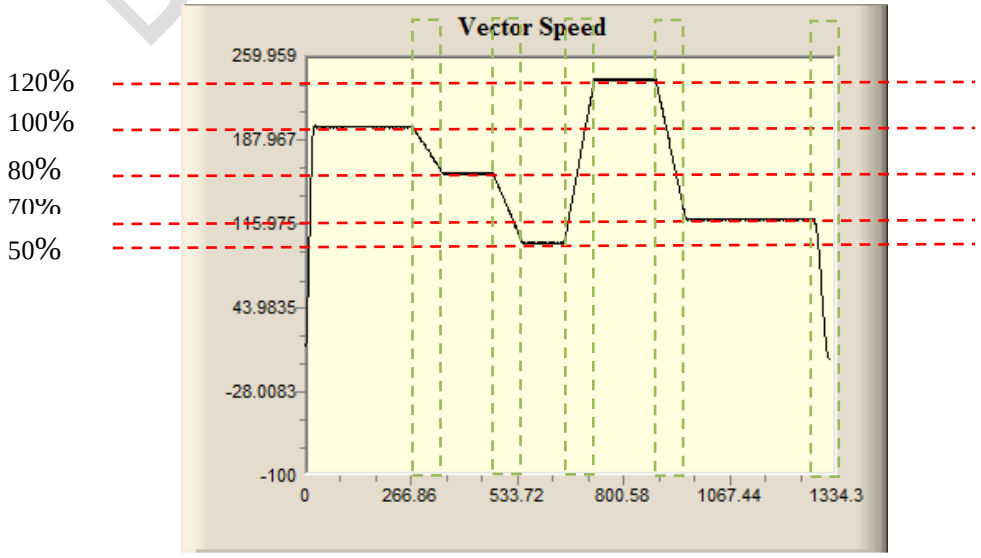
```
j = 70    //最多 70 點

for(i = 0; i < j; i++)
{
        PositionData[2*i] = 5000*i;        //點位資料
        PositionData[2*i+1] = 3000*i;      //點位資料
}
```

結果如下圖，整個 Table 在速度向量表示下為一個完整梯形。



使用_Path_TableMotion_FeedRate_OverWrite 更改 FeedRate 百分比，
則變更後的結果如下: 速度向量變化時間皆為 sec = 0. 3 秒(綠色虛線方塊內的變化時間)。

```
_DMC_01_tablemotion_feedrate_overwrite( CardNo, TableNo, 80 , 0.3 );
_DMC_01_tablemotion_feedrate_overwrite( CardNo, TableNo, 50 , 0.3 );
_DMC_01_tablemotion_feedrate_overwrite( CardNo, TableNo, 120 , 0.3 );
_DMC_01_tablemotion_feedrate_overwrite( CardNo, TableNo, 70 , 0.3 );
```



DELTA CONFIDENTIAL

44.10 _DMC_01_tablemotion_feedhold

■ **格式**

I16 PASCAL _DMC_01_tablemotion_feedhold(U16 CardNo,I16 TableNo, U16 On_Off,F64 sec)

■ **目的**

暫停當下速度變化。
設定 TableMotion 的 FeedHold，達到運動當下暫停的功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
On_Off	U16	數值單位	On_Off :1->關閉暫停 0:重新繼續
sec	F64	數值(單位:秒)	設定速度變化時間。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 nCardNo= 0;
I16 TableNo=0;
U16 On_Off=1;
I16 sec =10;(單位:秒)

I16 status = _DMC_01_tablemotion_feedhold (CardNo, TableNo, On_Off, sec);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75,Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

44.12 _DMC_01_set_tablemotion_blending_ratio

■ **格式**

I16 PASCAL _DMC_01_set_tablemotion_blending_ratio(U16 CardNo,I16 TableNo, U32 Ratio)

■ **目的**

設定 TableMotion 循環次數模式和 Buffermode 兩種運動的模式的混和百分比。
此 API 只適合使用在 Cycle mode 和 Buffermode 底下，Profile mode 不支援。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
Ratio	U32	數值單位	混和百分比: Ratio = 100 --> 100%，運動之間沒有等待，和原路徑誤差大。 Ratio = 0--> 0%，會等這個單節運動結束後再執行下個運動，基本上會疊合到運動間的端點位置。 Ratio 越小，路徑誤差越小，但轉角速度會降速。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16TableNo=0;
U32Ratio= 100;

I16 status = _DMC_01_set_tablemotion_blending_ratio( CardNo, TableNo, Ratio);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75,Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

NOTE

Ratio 的值為 100

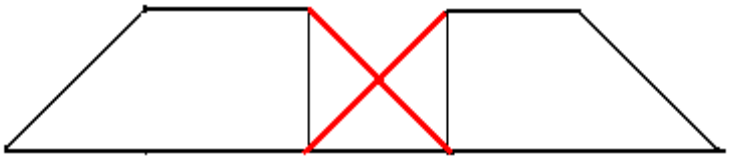


圖 44.1

Ratio 的值為 50

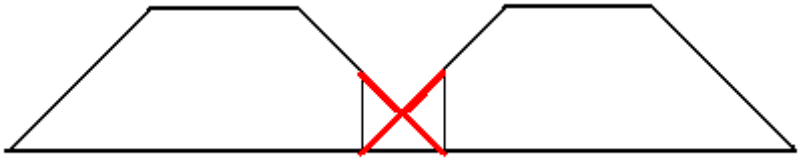


圖 44.2

DELTA CONFIDENTIAL

44.13 _DMC_01_start_tablemotion_profile_planning_para

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_profile_planning_para(U16 CardNo,I16 TableNo,I32 NanoSmoothDecectionLength,I32 RefTurnOnSpd ,I16 CurveDecectionFlag, I32 CurveDecectionLength, F64 CurveDecectionAngle, I32 ArcRefSpeed, I32 ArcRefRadius)

■ **目的**

設定 TableMotion 速度規劃模式的相關參數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
NanoSmoothDecectionLength	I32	數值單位	轉角判斷的累積長度。
RefTurnOnSpd	I32	數值單位	轉角參考速度。
CurveDecectionFlag	I16	數值單位	是否開啟曲線判斷。 1： 開啟 0:關閉
CurveDecectionLength	I32	數值單位	曲線判斷時的參考長度。
CurveDecectionAngle	F64	數值單位	曲線判斷時的參考角度。
ArcRefSpeed	I32	數值單位	曲線的參考速度。
ArcRefRadius	I32	數值單位	曲線的參考半徑。

詳細參數定義可參閱第 44.10 章節參數進階說明。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16 TableNo =0;
I32 NanoSmoothDecectionLength =500;
I32 RefTurnOnSpd =35000000 * 0.3;
I16 CurveDecectionFlag =0;
I32 CurveDecectionLength =4000;
F64 CurveDecectionAngle =15;
```

DELTA CONFIDENTIAL

I32 ArcRefSpeed =50000;
I32 ArcRefRadius=10000;

I16 status =_DMC_01_start_tablemotion_profile_planning_para(CardNo, TableNo, NanoSmoothDecectionLength, RefTurnOnSpd , CurveDecectionFlag, CurveDecectionLength, CurveDecectionAngle, ArcRefSpeed, ArcRefRadius);

- **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75,Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

44.14 _DMC_01_start_tablemotion_profile_planning

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_profile_planning(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *AccDec, I32 *PositionData,I32 DataSize,I16 AxisNum, I32 *MaxVel, F64 InitialVel_Ratio, I32 *IOCtl,U32 *Motion_No)

■ **目的**

此 API 會根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。
(可設定各點間最大速度，支援單步執行功能)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
AccDec	I32*	數值編號	各軸的加速度限制
PositionData	I32	數值編號	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
DataSize	I32	數值編號	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值編號	使用的軸數
MaxVel	I32*	數值編號	最大速度。(pulse/秒)
InitialVel_Ratio	F64	數值單位	整體初始速度百分比
IOCtl	U32*	數值編號	PositionData 與 PositionData 資料點之間的 IO 資訊。IO_Table[4] = { 0, 1, 3, 7}; 代表設定 IO 變化為 第 0 點位置到第 1 點位置 無 IO 啟動 第 1 點位置到第 2 點位置 設置第一點(bit1(1)=1)IO 啟動。 第 2 點位置到第 3 點位置 設置第一點與第二點 (bit1(1)+bit2 (2) =3) IO 啟動。 第 3 點位置到第 4 點位置設置第一點與第二點與第

DELTA CONFIDENTIAL

			三點 (bit1(1)+bit2 (2)+bit3(4)=7) IO 啟動。 上述為定義 IO 資料內容，實際開啟的輸出裝置必須執行_DMC_01_set_tablemotion_ioctl_mapping 此 API 來指定輸出的 DMC RM output 模組(NT)並給定代表輸出點位置。
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合 “44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

```
#define MAXDATA 20000
#define DATA_SIZE 1000
U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 AccDec[2] ={1280000,1280000,1280000};
I32 PositionData[MAXDATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4,……};
I32 DataSize = DATA_SIZE; //此數值代表每次可執行的點數，並不代表陣列長度。
I16 AxisNum =3;
I32 MaxVel [DATA_SIZE]={35000000,35000000,35000000,35000000,...};
F64 InitialVel_Ratio = 100;
I32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7 ,…}
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
```

I16 status =_DMC_01_start_tablemotion_profile_planning(CardNo, NodeID, SlotID, TableNo, AccDec, PositionData, DataSize, AxisNum, MaxVel, InitialVel_Ratio, IO_Table, Motion_No);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

參數進階說明:

功能說明:各軸的加速度限制。

***AccDec:** (最大速-初速) / 最大加速時間 = 加速度

舉例來說，如果某一軸從速度 0 加速度 10000 最快需要 0.1 秒，則加速度為

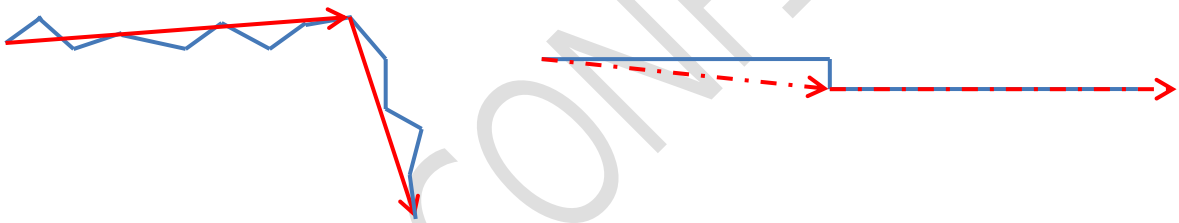
例:(10000 - 0) / 0.1 = 100000 (pulse/s^2)

NanoSmoothDecectionLength:

功能說明:判定轉角的累積長度。

為了設定短線段造成的轉角判定，因此在計算轉角時會累積足夠的長度才做轉角計算。(建議值:100nm 代表的 Pulse 量)。

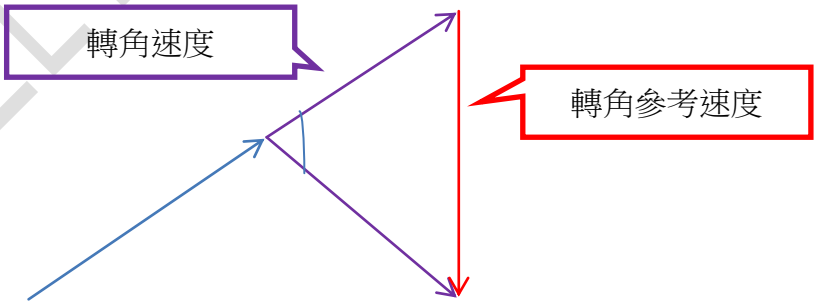
如左下圖，如依照藍線原始行為去判斷角度，會造成角度誤判一直有轉角，導致速度上不去。如右下圖，會導致角度誤判，會在該地方做不需要的降速。



RefTurnOnSpd:

功能說明:轉角參考速度。

如下圖圖例為參照設定轉角的速度和轉角角度。



舉例:如果轉角為 180 度，RefTurnOnSpd = 100000

即轉角速度不會超過 50000 (pulse/秒)

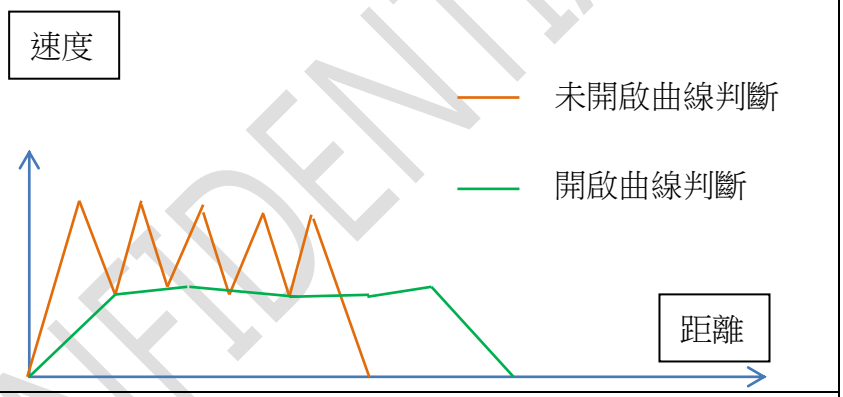
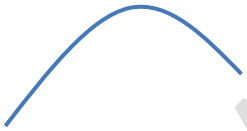
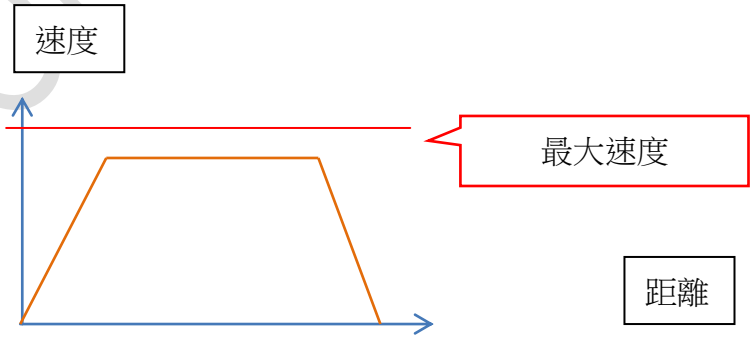
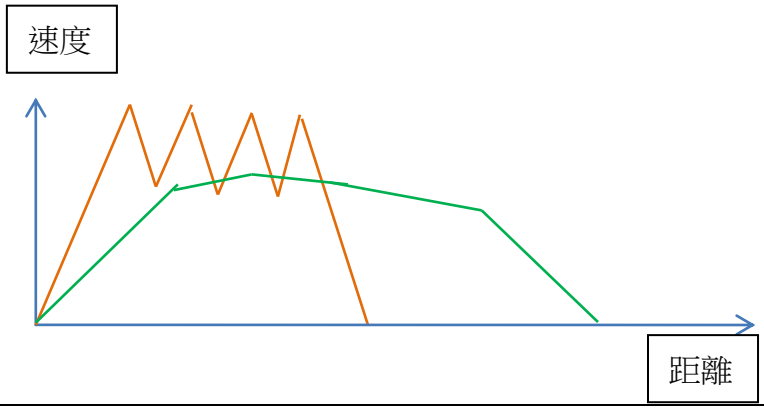
RefTurnOnSpd 的值越大，轉角速度越快，相對的震動量會比較大。

DELTA CONFIDENTIAL

CurveDecectionFlag:

功能說明:是否開啟曲線判斷。

於 NC 中，常常會把一段曲線切成很多短的直線單節。
舉例如下表中，如果是走一個圓形，那麼速度變化是比較平順的。但如果把圓切成很多小的直線變成多線段畫圓，也會希望它的速度曲線和圓的行為差不多，是平順的，也就是不要一直有加減速狀態。

運動路徑:	**速度規劃圖**
多小線段畫圓: 	
曲線: 	
曲線分段組合: 	

接續下頁

DELTA CONFIDENTIAL

因此，如果判斷成是曲線，則速度變化就會比較平滑，但相對地此運動所需的時間會變長。如果判斷成是直線，則會盡力加速到最大速，會一直有加減速。

判斷是否為曲線主要是依據下面 2 個參數。

CurveDecectionLength :

功能說明:曲線判斷時的參考長度。

CurveDecectionAngle :

功能說明:曲線判斷時的參考角度。

當此轉角角度小於"參考角度"且兩運動長度皆小於"參考長度"則此地方會被視為曲線，否則一律視為直線。

※建議值，參考角度:15~20 度，參考長度 ：1mm~2mm 所代表的 pulse 量
以上是經驗, 不代表所有狀況

如判斷為曲線，則轉角速度和以下兩參數有關:

ArcRefSpeed :

功能說明:曲線的參考速度。

ArcRefRadius：

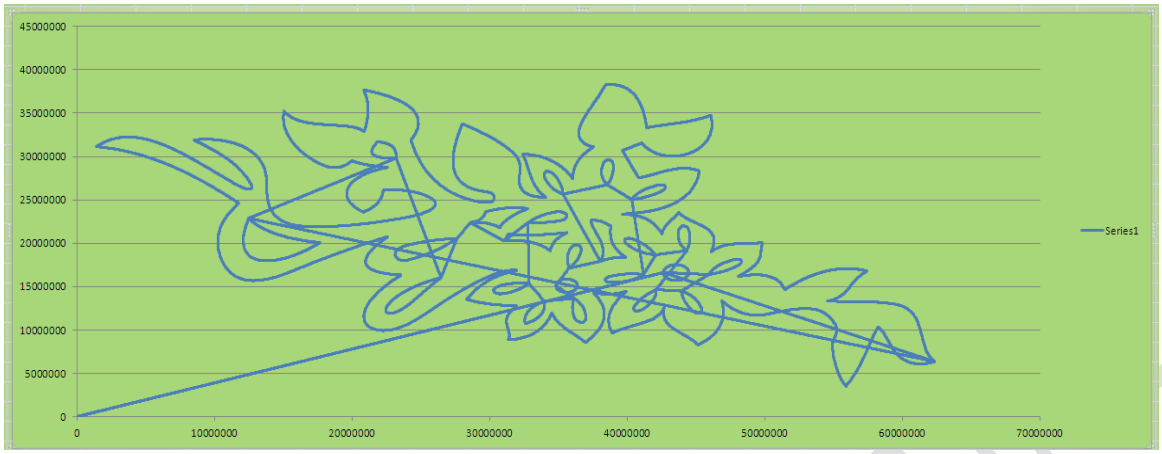
功能說明:曲線的參考半徑。

$$\text{轉角速度} \le \text{ArcRefSpeed} \times \sqrt{\frac{\text{此曲線的曲率半徑}}{\text{ArcRefRadius}}}$$

曲線速度和 ArcRefSpeed 成正比，和參考半徑的根號成反比。

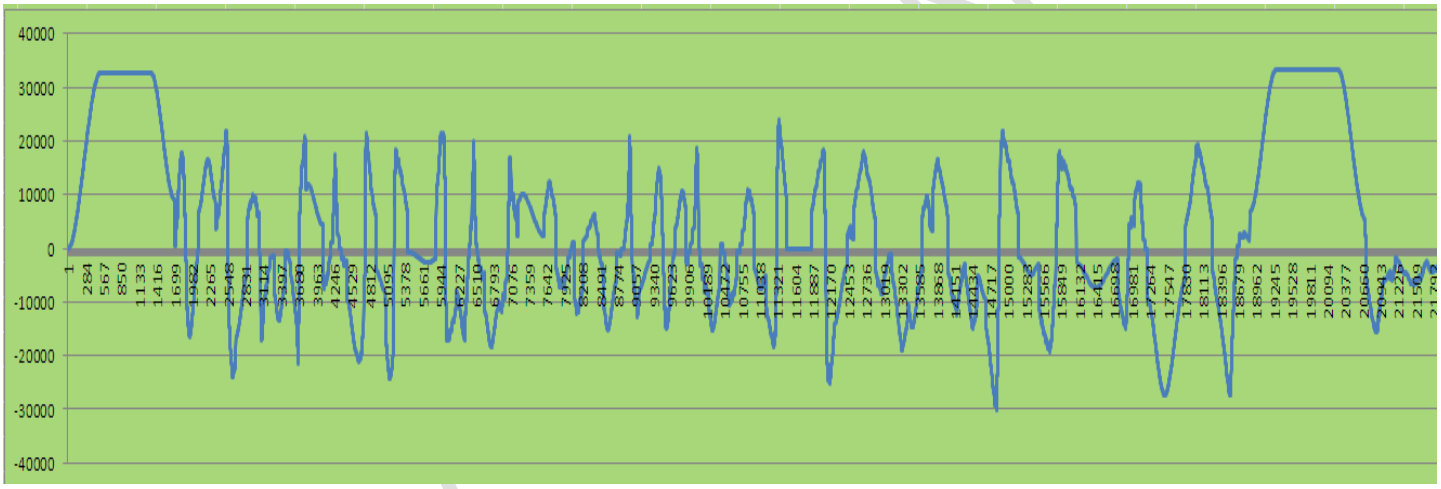
DELTA CONFIDENTIAL

範例為使用 Proflie mode:



(圖一、加工點位圖)

下(圖二)為觀察單軸速度曲線各速度段的描述

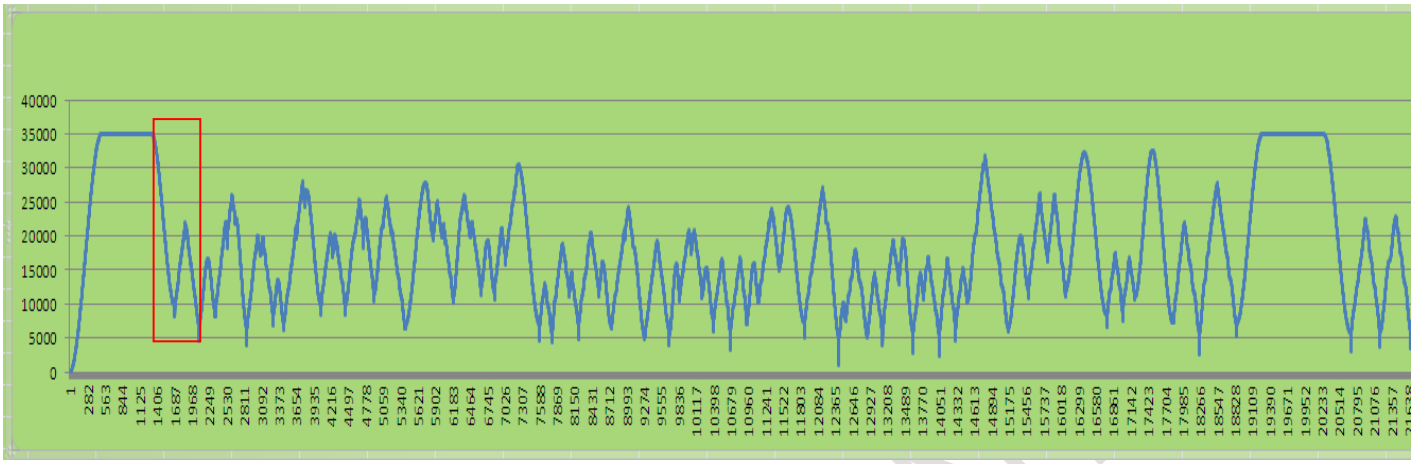


(圖二、單軸速度曲線圖 -X 軸)

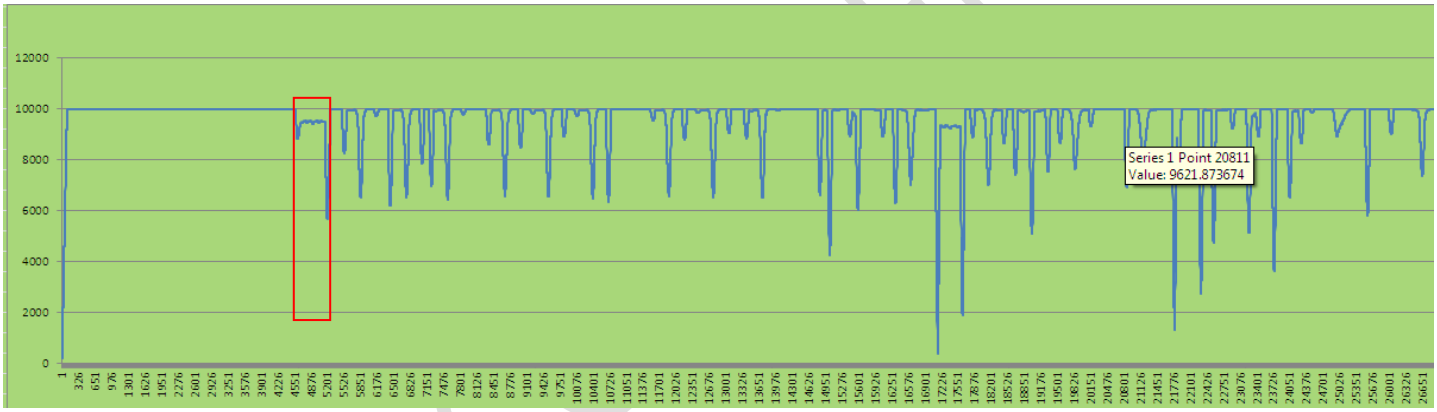
DELTA CONFIDENTIAL

下列圖(三)為兩軸合速圖(VT)，

可以看出在轉角速度會依照使用者設定，產生降速的效果。(紅色框內為單一轉角)



圖(三)　Profile mode 兩軸合速圖



圖(四)　Cycle mode 兩軸合速圖

由圖(三)與圖(四)比較 Profile mode 與 Cycle mode 的合速，結果可以看出 Profile mode 對於整體運動過程的轉角處理比較著重在轉角速度的提升，相對於 Cycle mode 的加工速度曲線來看是有明顯的差異。

DELTA CONFIDENTIAL

44.15 _DMC_01_start_tablemotion_profile_planning2

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_profile_planning2(U16 CardNo, U16 *NodeID, U16 *SlotID,I16 TableNo, I32 *AccDec, I32 *PositionData, I32 *Ref_Absolute_Pos, I32 DataSize,I16 AxisNum, I32 *MaxVel, F64 InitialVel_Ratio, I32 *IOCtl, U32 *ArcBit, U32 *Motion_No, F64 *Delay_CycleCount, U16 Abs_Mode)

■ **目的**

此 API 會根據設定的參數，計算出各運動間的加減速時間以及各運動間適當的轉角速度。
(可設定各點間最大速度，支援單步執行、絕對(相對)模式選擇、三點成圓路徑等功能)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
AccDec	I32*	數值編號	各軸的加速度限制
PositionData	I32	數值編號	各點的資料陣列。(註 1)必須先決定軸數。 假設此次為 TableMotion 為 3 軸運動。前面幾點資料分別是:(x1,y1,z1), (x2,y2,z2), (x3,y3,z3), (x4,y4,z4)……依序填入(軸 1、軸 2、軸 3)。 程式內部資料結構則為:PositionData[] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……} 依序塞入點位資料。
Ref_Absolute_Pos	I32*	數值單位	相對模式時的參考位置陣列。
DataSize	I32	數值編號	填入的點數最大數量，MAX 為 2000。
AxisNum	I32	數值編號	使用的軸數
MaxVel	I32*	數值編號	最大速度。(pulse/秒)
InitialVel_Ratio	F64	數值單位	整體初始速度百分比
IOCtl	U32*	數值編號	PositionData 與 PositionData 資料點之間的 IO 資訊。IO_Table[4] = { 0, 1, 3, 7}; 代表設定 IO 變化為 第 0 點位置到第 1 點位置 無 IO 啟動 第 1 點位置到第 2 點位置 設置第一點(bit1(1)=1)IO 啟動。

DELTA CONFIDENTIAL

			第 2 點位置到第 3 點位置 設置第一點與第二點 (bit1(1)+bit2 (2) =3) IO 啟動。 第 3 點位置到第 4 點位置設置第一點與第二點與第三點 (bit1(1)+bit2 (2)+bit3(4)=7) IO 啟動。 上述為定義 IO 資料內容，實際開啟的輸出裝置必須執行_DMC_01_set_tablemotion_ioctl_mapping 此 API 來指定輸出的 DMC RM output 模組(NT)並給定代表輸出點位置。
ArcBit	U32*	數值	此值為 0，則此單節為直線；如不為 0，則是用來標記此點是由哪幾軸做 Arc 運動。(註 1 :)
Motion_No	U32*	編號單位	各點的運動編號陣列，同編號的視為同一群的運動。配合 “44.21 單步運轉功能”，如單步運轉開啟，則運動會停在當群運動的結尾，此時如果單步運轉功能關閉，運動會再繼續進行。
Delay_CycleCount	U32*	數值	各單節運動完 Delay 的週期。
Abs_Mode	U16	選項	0：相對模式 1：絕對模式

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

```
#define MAXDATA 20000
#define DATA_SIZE 1000
U16 nCardNo= 0;
U16 NodeID[2] ={1,2,3};
U16 SlotID[2]={0};
I16 TableNo=0;
I32 AccDec[2] ={1280000,1280000,1280000};
I32 PositionData[MAXDATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4,……};
I32 Ref_Absolute_Pos[MAX_DATA] ={x1,y1,z1,x2,y2,z2,x3,y3,z3,x4,y4,z4……};
I32 DataSize = DATA_SIZE; //此數值代表每次可執行的點數，並不代表陣列長度。
I16 AxisNum =3;
I32 MaxVel [DATA_SIZE]={35000000,35000000,35000000,35000000,...};
F64 InitialVel_Ratio = 100;
I32 IO_Table[DATA_SIZE] = { 0, 1, 3, 7 ,…};
U32 ArcBit[DATA_SIZE] = {3, 3, 3, 3,…..};
U32 Motion_No[DATA_SIZE] = {1, 2, 3, 4,…..};
U32 Delay_CycleCount[DATA_SIZE] = {1, 1, 1, 1,…..};
U16 Abs_Mode = 0 ;
```

I16 status =_DMC_01_start_tablemotion_profile_planning2(CardNo, NodeID, SlotID,TableNo, AccDec, PositionData, Ref_Absolute_Pos, DataSize,AxisNum, MaxVel, InitialVel_Ratio, IO_Table, ArcBit, Motion_No, Delay_CycleCount, Abs_Mode);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

44.16 _DMC_01_start_tablemotion_profilemode

■ **格式**

I16 PASCAL _DMC_01_start_tablemotion_profilemode(U16 CardNo,I16 TableNo)

■ **目的**

啟動 TableMotion 速度規劃模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
I16 TableNo =1;

I16 status =_DMC_01_start_tablemotion_profilemode( CardNo, TableNo);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_PROFILE_MODE_ENABLE	Rt = 200，速度規劃模式已啟動
ERR_NONE_ANALYSIS	Rt = 202，尚未使用 profile_planning API
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_MotionBusy	Rt = 114，有運動正在作用
ERR_TM_PROFILE_RINGBUFFER_STATUS	Rt = 210，TableMotion ProfileMode 尚未重新造表

DELTA CONFIDENTIAL

44.17 _DMC_01_set_axis_normalamf_enable

■ **格式**

I16 PASCAL _DMC_01_set_axis_normalamf_enable(U16 CardNo, U16 NodeID, U16 SlotID, U16 filterTime, U16 enable)

■ **目的**

設定指定軸的 filtertime 以及是否開啟後加減功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15。
NodeID	I16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
filterTime	U16	數值	設定適當後加減的 filter 時間。
enable	U16	選項單位	0:關閉 1:啟動。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo= 0;
U16 NodeID =1;
U16 SlotID =0;
U16 filterTime =128;
U16 enable =1;

I16 status =_DMC_01_set_axis_normalamf_enable( CardNo, NodeID, SlotID, filterTime, enable);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_MotionBusy	Rt = 114，有運動正在作用
ERR_AMF_Time	Rt = 194，Filter 時間超過最大預設值(1 秒)

DELTA CONFIDENTIAL

44.18 _DMC_01_set_normalamf_enable

■ **格式**

I16 PASCAL _DMC_01_set_normalamf_enable(U16 CardNo, U16 afteramf_Num, U16 enable)

■ **目的**

設定後加減是否啟動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	軸卡號 nCardNo 為 0~15
afteramf_Num	U16	數值編號	設定後加減道數(1~2)
enable	U16	數值編號	0:關閉 1:啟動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
U16 afteramf_Num =1;
U16 enable =1;

I16 status =_DMC_01_set_normalamf_enable( CardNo, afteramf_Num, enable);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_AMF_ENABLE	Rt = 193，AMF 已啟動
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

44.19 _DMC_01_set_tablemotion_ioctl_mapping

■ **格式**

I16 PASCAL _DMC_01_set_tablemotion_ioctl_mapping(U16 CardNo,I16 TableNo,U16 ionum ,U16 NodeID, U16 SlotID,U16 iobit)

■ **目的**

設定 IO 資料表，控制每個總內線的 IO 模組狀態，定義 Table 運動過程中每個運動間 IO 的輸出入狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	數值編號	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
ionum	U16	數值編號	設定編號，由 0 號開始編碼。(最多 4 點)
NodeID	U16	數值編號	設定要控制的 Node 編號
SlotID	U16	數值編號	設定要控制的 Slot 插槽的編號:預設 0
iobit	U16	數值編號	設定 Node ID 接著設定欲輸出的 NT 模組。 緊接著選擇設定的模組輸出位置(十進制)，以 RM64NT 為例參數設定可以輸入 iobit = 5。 則模組 P0 的第五點會在 Table 設定的 Position 位置亮起第五點。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;
U16 TableNo =1;
U16 ionum = 0;//第 0 個狀態點
U16 NodeID =1; //第幾個 Node 是 IO 模組
U16 SlotID =0;//要控制 Node 底下第幾個 Slot;
U16 iobit = 0 ;//控制第 0 點 IO 輸出。
I16 status =_DMC_01_set_tablemotion_ioctl_mapping(CardNo, TableNo, ionum , NodeID, SlotID,

DELTA CONFIDENTIAL

iobit)

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_IO_NUMBER	Rt = 199，超過最大 IO 數量 4
ERR_AMF_ENABLE	Rt = 193，AMF 已啟動
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

44.20 _DMC_01_get_tablemotion_current_speed

■ **格式**

I16 PASCAL _DMC_01_get_tablemotion_current_speed(U16 CardNo,I16 TableNo, I32 *CurrentSpeed)

■ **目的**

設定取得指定 Table 當前速度。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	數值編號	Table 號碼(0 or 1)
CurrentSpeed	I32	數值單位	當前速度，(pulse/秒)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I16 TableNo =0;
I32 CurrentSpeed;
I16 status =  _DMC_01_get_tablemotion_current_speed(CardNo, TableNo, & CurrentSpeed);
```

DELTA CONFIDENTIAL

44.21 _DMC_01_Execute_Single_Step

■ **格式**

I16 PASCAL _DMC_01_Execute_Single_Step(U16 CardNo,I16 TableNo);

■ **目的**

觸發單步運轉功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) : 決定使用的 Table 編號(最大 2 組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I16 TableNo = 0;
I16 status = _DMC_01_Execute_Single_Step (CardNo, TableNo);// 觸發動作
```

NOTE

※如開啟單步執行功能，使用此 API 可以觸發動作，並會在運動群的結束點停止，等待下一次觸發。
※如單步執行功能關閉，使用此 API 也不會讓運動停止。

DELTA CONFIDENTIAL

44.22 _DMC_01_Get_TableMotion_Now_MotionNo

■ **格式**

I16 PASCAL _DMC_01_Get_TableMotion_Now_MotionNo(U16 CardNo,I16 TableNo, U32 *MotionNo)

■ **目的**

取得當群運動的編號

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) : 決定使用的 Table 編號(最大 2 組)
MotionNo	U32	編號單位	當 TableMotion (Cycle Mode, Buffer Mode, Profile Mode)運動執行時，取得當群運動的編號。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I16 TableNo =0;
U32* MotionNo;
I16 status =  _DMC_01_Get_TableMotion_Now_MotionNo (CardNo, TableNo, &MotionNo);
```

NOTE

※單步運動啟動後，當前的運動群執行完畢後運動會停止，此時取得的 MotionNo 值非目前停止的群，而是下一個準備啟動的群編號。

DELTA CONFIDENTIAL

44.23 _DMC_01_TableMotion_Single_Step

■ **格式**

I16 PASCAL _DMC_01_TableMotion_Single_Step (U16 CardNo,I16 TableNo, U16 Enable)

■ **目的**

設定啟動單步運轉功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) : 決定使用的 Table 編號(最大 2 組)
Enable	U16	選項單位	0：關閉單步運轉功能 1：開啟單步運轉功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I16 TableNo = 0;
U16 Enable = 1;
I16 status = _DMC_01_TableMotion_Single_Step (CardNo, TableNo, Enable);
```

DELTA CONFIDENTIAL

44.24 _DMC_01_start_tableMotion_profileMode_data_flying_photography

■ **格式**

I16 PASCAL _DMC_01_start_tableMotion_profileMode_data_flying_photography(U16 CardNo, U16 TableNo, I32 *PositionData, I32 DataSize, U16 *AxisArray, U16 *SlotArray, I32 *MaxVel, I32 *AccDecLimit, I32 Remain_Dist, I32 Stage2_Velocity, F64 InitialVel_Ratio, I32 *IOControl)

■ **目的**

兩軸過點飛拍功能，此 API 會根據設定的飛拍參數計算出新的運動軌跡及速度。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
TableNo	I16	編號單位	Table 號碼(0 or 1) :決定使用的 Table 編號(最大 2 組)
PositionData	I32*	數值編號	兩軸各點的資料陣列。 前面幾點資料分別是：(x1,y1)、(x2,y2)、(x3,y3)、(x4,y4)……依序填入 (軸 1、軸 2) 程式內部資料結構則為： PositionData[] = {x1,y1,x2,y2,x3,y3,x4,y4……} 依序塞入點位資料
DataSize	I32	數值編號	填入的點數最大數量，MAX 為 1000。
AxisArray	U16*	編號單位	站號 ID
SlotArray	U16*	編號單位	保留
MaxVel	I32*	數值編號	最大速度。(pulse/秒)
AccDecLimit	I32	數值單位	各軸的加速度限制
Remain_Dist	I32	數值單位	經過點前後要保留的距離
Stage2_Velocity	I32	數值單位	經過點時要維持的速度
InitialVel_Ratio	F64	數值單位	整體初始速度百分比
IOControl	I32*	數值編號	PositionData 與 PositionData 資料點之間的 IO 資訊。IO_Table[4] = { 0, 1, 3, 7}; 代表設定 IO 變化為 第 0 點位置到第 1 點位置 無 IO 啟動 第 1 點位置到第 2 點位置 設置第一點(bit1(1)=1)IO 啟動。 第 2 點位置到第 3 點位置 設置第一點與第二點

DELTA CONFIDENTIAL

			(bit1(1)+bit2 (2) =3) IO 啟動。 第 3 點位置到第 4 點位置設置第一點與第二點與第三點 (bit1(1)+bit2 (2)+bit3(4)=7) IO 啟動。 上述為定義 IO 資料內容，實際開啟的輸出裝置必須執行_DMC_01_set_tablemotion_ioctl_mapping 此 API 來指定輸出的 DMC RM output 模組(NT)並給定代表輸出點位置。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
#define MAXDATA   20000
#define DATA_SIZE 1000
U16 CardNo= 0;
U16 TableNo=0;
I32 PositionData[MAXDATA] ={x1,y1,x2,y2,x3,y3,x4,y4,……};
I32 DataSize = DATA_SIZE; //此數值代表每次可執行的點數，並不代表陣列長度。
U16 AxisArray[2] ={1,2};
U16 SlotID[2]={0};
I32 MaxVel [DATA_SIZE]={35000000,15000000,35000000,15000000,...};
I32 AccDecLimit [2] ={1280000,1280000 };
I32 Remain_Dist=1000;
I32 Stage2_Velocity = 5000000;
F64 InitialVel_Ratio = 100;
I32 IOControl [DATA_SIZE] = { 0, 1, 3, 7 ,…}

I16 status = _DMC_01_start_tableMotion_profileMode_data_flying_photography(CardNo,
TableNo, PositionData, DataSize, AxisArray, SlotID, MaxVel, AccDecLimit,   Remain_Dist,
Stage2_Velocity, InitialVel_Ratio, IOControl);
```

DELTA CONFIDENTIAL

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_TM_DATA_SIZE	Rt = 165，DataSize 超過資料個數上限
ERR_OVER_TABLE_NUM	Rt = 203，TableNo 超過最大 Table 數量
ERR_SpeedError	Rt = 116，MaxVel = 0, 陣列內容如有缺少資料也會跳此錯誤。
ERR_AccTimeError	Rt = 117，Tacc_Tdec 設置錯誤
ERR_RangeError	Rt = 112，NodeID 超出範圍
ERR_MotionBusy	Rt = 114，有運動正在作用

DELTA CONFIDENTIAL

第四十五章 PVT 模式

此章節說明了 PVT 模式(API)，可設置多組點位、時間、起始速度、結束速度與點位總數資料。

表 45

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_Start_PVTComplete	啟動 PVT Complete 運動
A/B/F	_DMC_01_Start_PVT	啟動 PVT 運動
A/B/F	_DMC_01_get_PVT_cnt	取得 PVT 點位個數

DELTA CONFIDENTIAL

45.1 _DMC_01_Start_PVTComplete

■ **格式**

I16 PASCAL _DMC_01_Start_PVTComplete(U16 CardNo, U16 NodeID, U16 SlotID, F64 *Position, F64 *Time, F64 Str_Vel, F64 End_Vel, I32 MotionCnt,U16 m_r_a);

■ **目的**

啟動 PVT Complete 運動。
輸入多組位置、時間資料。會於指定的時間到達指定的位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16	數值編號	站號 ID
SlotID	U16	數值編號	插槽 ID
Position	F64*	數值編號	點位資訊
Time	F64*	數值編號	時間資訊
Str_Vel	F64	數值編號	起始速度資訊
End_Vel	F64	數值編號	結束速度資訊
MotionCnt	I32	數值編號	點位總數
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo= 0;
U16 NodeID,=1;
U16 SlotID=0;
F64Position[4] ={0,100000,0,-100000,0};
F64Time [4]={0, 500, 1000, 1500, 2000};
F64 Str_Vel=0;
F64 End_Vel=128000;
I32 MotionCnt =3;
U16 m_r_a=0;
```

DELTA CONFIDENTIAL

I16 status =_DMC_01_Start_PVTComplete(CardNo, NodeID, SlotID,Position,Time, Str_Vel, End_Vel, MotionCnt,U16 m_r_a);

- **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_PVTDataError	Rt = 60，資料有錯誤，MotionCnt < 2 或是時間資訊不是遞增的值
ERR_PathError	Rt = 28，移動距離為 0

DELTA CONFIDENTIAL

45.2 _DMC_01_Start_PVT

■ **格式**

I16 PASCAL _DMC_01_Start_PVT(U16 CardNo, U16 NodeID, U16 SlotID, F64 *Position, F64 *Time, F64 *Velocity,I32 MotionCnt,U16 m_r_a);

■ **目的**

啟動 PVT 運動。輸入多組位置、速度、時間資料。會於指定的時間到達指定的位置、速度。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16	數值編號	站號 ID
SlotID	U16	數值編號	插槽 ID
Position	I32 *	數值編號	點位資訊。
Time	I32 *	數值編號	時間資訊。
Velocity	I32 *	數值編號	加速度資訊。
MotionCnt	F64	數值編號	點位總數。
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo= 0;
U16 NodeID =1;
U16 SlotID=0;
I32 Position[4] ={0,100000,0,-100000,0};
I32 Time[4]={0, 500, 1000, 1500, 2000};
I32 Velocity[4] ={0, 5000,10000,7500,0};
I32 MotionCnt =4;
U16 m_r_a=0;
I16 status = _DMC_01_Start_PVT(CardNo, NodeID, SlotID,Position,Time, Velocity, MotionCnt, m_r_a);
```

■ **Error Code:**

DELTA CONFIDENTIAL

PCI_DMC_01_Err.h	Return Code
ERR_RangeError	Rt = 12，NodeID 超出範圍
ERR_PathError	Rt = 28，移動距離為 0

DELTA CONFIDENTIAL

45.3 _DMC_01_get_PVT_cnt

■ **格式**

I16 PASCAL _DMC_01_get_PVT_cnt(U16 CardNo, U16 NodeID, U16 SlotID,U32 *count);

■ **目的**

取得 PVT 資料剩餘總數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	軸卡號 nCardNo 為 0~15
NodeID	U16	數值編號	站號 ID
SlotID	U16	數值編號	插槽 ID
count	U32*	數值編號	正在執行的運動點位。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 count;

I16 status =_DMC_01_get_PVT_cnt(CardNo, NodeID, SlotID,&count);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_QEP_INDEX	Rt = 75,Qep 索引值錯誤
ERR_B02_NO_ERROR	Rt = 0,無錯誤

DELTA CONFIDENTIAL

第四十六章 B02 Latch

表 46

API 歸屬	函數名稱	描述
B	_DMC_B02_ext_io_latch_initial	初始化 B02 Latch 功能
B	_DMC_B02_ext_io_latch_function	設定 B02 Latch 參數規則
B	_DMC_B02_ext_io_latch_enable	啟動 B02 Latch 功能
B	_DMC_B02_latch_fifo_reset	清除 Latch 資料功能
B	_DMC_B02_latch_fifo_status	取得 Latch FIFO 資訊值
B	_DMC_B02_latch_fifo_size	取得 FIFO 內 Latch 的資料筆數
B	_DMC_B02_latch_fifo_get_qep_counter	取得 FIFO 內 Latch 的 Encoder 資料(通用版適用)
B	_DMC_B02_latch_fifo_get_qep_counter2	取得 FIFO 內 Latch 的 Encoder 資料。(僅支援 2D 版本)
B	_DMC_B02_latch_fifo_dsp_get_data_counter	取得 Latch 的 Encoder 資料筆數。(僅支援 2D 版本)
B	_DMC_B02_latch_fifo_dsp_data_counter_reset	重置 Latch 的 Encoder 資料筆數為 0。(僅支援 2D 版本)
B	_DMC_B02_latch_fifo_dsp_data_clear	清除已從 FIFO 內存取的 Latch 資料。(僅支援 2D 版本)

此表格 API 僅 PCI_DMC_B02 可使用

DELTA CONFIDENTIAL

46.1 _DMC_B02_ext_io_latch_initial

■ **格式**

I16 PASCAL _DMC_B02_ext_io_latch_initial(U16 nCardNo, U8 nExternalIoNo, U8 nSource, U8 nFilter);

■ **目的**

初始化 B02 Latch 功能。(請參閱 NOTE 說明)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nExternalIoNo	U8	數值	選擇 latch 功能設定組: 0~7 代表 B02 軸卡 8 組功能設定
nSource	U8	數值	Latch Data Mode 設定 0~7 [0] Table TPCMP 0 OUT [1] Table TPCMP 1 OUT [2] Table TPCMP 2 OUT [3] Table TPCMP 3 OUT [4] Table TPCMP 4 OUT [5] Table TPCMP 5 OUT [6] External IN Enable [7] Software Enable
nFilter	U8	數值	Filter 為設定 External DI 觸發後 Filter time 1:0.1ms 2:0.5ms 3:1ms

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 nCardNo = 0;
U8 nExternalIoNo = 0;      //nExternalIoNo 0, 選擇設定第 1 組
U8 nSource = 0;            // Latch Mode:Table TPCMP 0
U8 nFilter = 0;(N/A)       //1: 0.1ms, 2: 0.5ms, 3: 1ms
```

DELTA CONFIDENTIAL

I16 status = _DMC_B02_ext_io_latch_initial(nCardNo, nExternalIoNo, nSource, nFilter);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。
ERR_EXTIO_RANGE	Rt = 84，超出指定的 IO 範圍。

NOTE

B02 軸卡 Latch 功能設定共有 0~7 組設定。
參數 nSource (Latch Data Mode) 設定說明:
Latch Mode 分為 CompareTable，External IN(DI)，Software Enable，三種觸發 Latch 資料模式，無法同時使用。

1.參數輸入 0~5 ，Table TPCMP n OUT :
當被設置的 CompareTable 比較到位觸發時 Latch QEP 資料。
(僅支援 B02 CompareTable 模式)

```
U16 nCardNo = 0;
U8 nExternalIoNo = 0;       //nExternalIoNo 0, 選擇設定第 0 組
U8 nSource = 0;             // Latch Mode:Table TPCMP 0
```

2.參數輸入 6 ，External IN Enable:
設置 B02 軸卡 DI1~DI8 (PIN 31 ~ 38)
DI 腳位與 nExternalIoNo 設置組別 Mapping，
例如設置 DI1(PIN 31)，則需設置 nExternalIoNo 為 0(第 1 組)，

```
U16 nCardNo = 0;
U8 nExternalIoNo = 0;       //nExternalIoNo 0, 選擇設定第 1 組
U8 nSource = 6;             // Latch Mode: External IN
```

若要設置 DI2(PIN 32)，則需設置 nExternalIoNo 為 1(第 2 組)。

```
U16 nCardNo = 0;
U8 nExternalIoNo = 1;       //nExternalIoNo 1, 選擇設定第 2 組
U8 nSource = 6;             // Latch Mode: External IN
```

DELTA CONFIDENTIAL

3.參數輸入 7，使用 Software Enable

(請參閱_DMC_B02_ext_io_latch_function NOTE 部分說明)

```
U16 nCardNo = 0;
U8 nExternalIoNo = 0;       //nExternalIoNo 0, 選擇設定第 0 組
U8 nSource = 7;             // Latch Mode: External IN
```

DELTA CONFIDENTIAL

46.2 _DMC_B02_ext_io_latch_function

■ **格式**

I16 PASCAL _DMC_B02_ext_io_latch_function(U16 CardNo, U8 nExternalIoNo, U8 bQep0, U8 bQep1, U8 bQep2, U8 bSw, U8 bH2L, U8 bL2H);

■ **目的**

設定 B02 Latch 參數規則。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nExternalIoNo	U8	編號單位	選擇 latch 功能設定組: 0~7 代表 B02 軸卡 0~7 組功能設定
bQep0	U8	數值	0:關閉 1:開啟，Latch 條件達成時，存取 QEP0 資料
bQep1	U8	數值	0:關閉 1:開啟，Latch 條件達成時，存取 QEP1 資料
bQep2	U8	數值	0:關閉 1:開啟，Latch 條件達成時，存取 QEP2 資料
bSw	U8	數值	Software Enable 0:關閉 1:開啟 (軟體觸發)
bH2L	U8	數值	falling edge 0:關閉 1:開啟
bL2H	U8	數值	Rising edge 0:關閉 1:開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

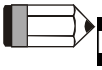
```
U16 CardNo = 0 ;
U8 nExternalIoNo= 0 ;     // nExternalIoNo 0, 選擇設定第 0 組
U8 bQep0= 1 ;             //開啟 Latch QEP0 資料
U8 bQep1= 0 ;
U8 bQep2= 0 ;
U8 bSw= 0 ;
U8 bH2L= 1 ;              //選擇外部 IO 為 falling edge 觸發
U8 bL2H= 0 ;
```

DELTA CONFIDENTIAL

I16 status = _DMC_B02_ext_io_latch_function(CardNo, nExternalIoNo, bQep0, bQep1, bQep2, bSw, bH2L, bL2H);

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。
ERR_EXTIO_RANGE	Rt = 84，超出指定的 IO 範圍。



NOTE

1.bQep0、bQep1、bQep2 可同時開啟，條件達成時一次 Latch 三個 QEP 資料。

2. 使用軟體觸發 Software Enable:
觸發模式 falling edge: bSw 狀態 1->0 時 Latch
U8 bSw= 1; // bSw 狀態輸入 1
_DMC_B02_ext_io_latch_function CardNo, nExternalIoNo, bQep0, bQep1, bQep2, bSw, bH2L, bL2H);
U8 bSw= 0; // bSw 狀態輸入 0
_DMC_B02_ext_io_latch_function CardNo, nExternalIoNo, bQep0, bQep1, bQep2, bSw, bH2L, bL2H);

觸發模式 Rising edge: bSw 狀態 0->1 時 Latch
U8 bSw= 0; // bSw 狀態輸入 0
_DMC_B02_ext_io_latch_function CardNo, nExternalIoNo, bQep0, bQep1, bQep2, bSw, bH2L, bL2H);
U8 bSw= 1; // bSw 狀態輸入 1
_DMC_B02_ext_io_latch_function CardNo, nExternalIoNo, bQep0, bQep1, bQep2, bSw, bH2L, bL2H);

DELTA CONFIDENTIAL

46.3 _DMC_B02_ext_io_latch_enable

■ **格式**

I16 PASCAL _DMC_B02_ext_io_latch_enable(U16 CardNo, U8 nExternalIoNo, U8 bEnable);

■ **目的**

啟動 B02 Latch 功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nExternalIoNo	U8	編號單位	選擇 Latch 功能設定組: 0~7 代表 B02 軸卡 0~7 組功能設定
bEnable	U8	數值	0:關閉 1:開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U8 nExternalIoNo = 0;   // nExternalIoNo 0, 選擇設定第 0 組
U8 bEnable = 1;

I16 status = _DMC_B02_ext_io_latch_enable(U16 CardNo, U8 nExternalIoNo, U8 bEnable);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。
ERR_EXTIO_RANGE	Rt = 84，超出指定的 IO 範圍。

DELTA CONFIDENTIAL

46.4 _DMC_B02_latch_fifo_reset

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_reset(U16 CardNo, U8 bEnable);

■ **目的**

清除 FIFO 內 Latch 的資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
bEnable	U8	數值	0:關閉 1:開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U8 bEnable  = 1;

I16 status = _DMC_B02_latch_fifo_reset(CardNo, bEnable);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。

NOTE

Enable 清除 Latch 資料功能開啟，開啟時會保持 fifo 是清除的狀態，若要進行 Latch 功能，需下 Disable 關閉清除 Latch 資料功能。

DELTA CONFIDENTIAL

46.5 _DMC_B02_latch_fifo_status

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_status(U16 CardNo, U8* nStatus);

■ **目的**

取得 Latch FIFO 資訊值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nStatus	U8*	編號單位	取得 Latch FIFO 狀態值。 bit 0: Empty bit 1: Full (僅 bit 0, bit 1 有定義)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U8 nStatus = 0;
I16 status = _DMC_B02_latch_fifo_status(CardNo, &nStatus);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。

DELTA CONFIDENTIAL

46.6 _DMC_B02_latch_fifo_size

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_size(U16 CardNo, U8* nSize);

■ **目的**

取得 FIFO 內 Latch 的資料筆數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nSize	U8*	編號單位	取得 FIFO 內 Latch 的資料筆數，最大 255 筆。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U8 nSize =0;

I16 status = _DMC_B02_latch_fifo_size(CardNo,  &nSize);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。

DELTA CONFIDENTIAL

46.7 _DMC_B02_latch_fifo_get_qep_counter

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_get_qep_counter(U16 CardNo, I32* nCounter, U8* nIoType, U8* nQepSource);

■ **目的**

取得 FIFO 內 Latch 的 Encoder 資料。(通用版適用)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nCounter	I32*	數值	取得一筆 Latch 的 Encoder(QEP)資料值(最大 255 筆)。
nIoType	U8*	數值	取得一筆 Latch 時 External DI edge (最大 255 筆)
nQepSource	U8*	編號單位	取得一筆 Latch 的 QEP 來源。(最大 255 筆)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 nCounter = 0;
U8 nIoType = 0;
U8 nQepSource = 0;

I16 status = _DMC_B02_latch_fifo_get_qep_counter(CardNo, &nCounter, &nIoType,
&nQepSource);
```

■ **Error Code:**

PCI_DMC_01_Err.h	Return Code
ERR_B02_NO_ERROR	Rt = 0，執行正常。
ERR_LATCH_FIFO_EMPTY	Rt = 86，B02 – 檢查 FIFO 為空值則拋出此錯誤

DELTA CONFIDENTIAL

NOTE

_DMC_B02_latch_fifo_get_qep_counter 取一次資料，fifo size 自動遞減 1。
Latch 資料筆數最大為 255 筆，由 3 組 QEP 共用。

DELTA CONFIDENTIAL

46.8 _DMC_B02_latch_fifo_get_qep_counter2

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_get_qep_counter2(U16 nCardNo, I32* nCounter, U8* nIoType, U8* nQepSource, U32* nSize);

■ **目的**

取得 FIFO 內 Latch 的 Encoder 資料。(僅支援 2D 版本)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nCounter	I32*	數值	取得 Latch 的 Encoder(QEP)資料值(最大 5000 筆)。
nIoType	U8*	數值	取得 Latch 時 External DI edge (最大 5000 筆)
nQepSource	U8*	編號單位	取得 Latch 的 QEP 來源。(最大 5000 筆)
nSize	U32*	數值	取得 Latch 資料總筆數。(最大 5000 筆)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 nCounter = 0;
U8 nIoType = 0;
U8 nQepSource = 0;
U32 nSize=0;

I16 status = _DMC_B02_latch_fifo_get_qep_counter2(CardNo, &nCounter, &nIoType,
&nQepSource, &nSize);
```

DELTA CONFIDENTIAL

NOTE

nIoType 數值說明:

nIoType	Description
15	External DI7 Rising
14	External DI6 Rising
13	External DI5 Rising
12	External DI4 Rising
11	External DI3 Rising
10	External DI2 Rising
9	External DI1 Rising
8	External DI0 Rising
7	External DI7 falling
6	External DI6 falling
5	External DI5 falling
4	External DI4 falling
3	External DI3 falling
2	External DI2 falling
1	External DI1 falling
0	External DI0 falling

Latch 資料筆數最大為 10000 筆，由 3 組 QEP 共用。

DELTA CONFIDENTIAL

46.9 _DMC_B02_latch_fifo_dsp_get_data_counter

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_dsp_get_data_counter(U16 nCardNo, U32* nCounter);

■ **目的**

取得 Latch 的 Encoder 資料筆數。(僅支援 2D 版本)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15
nCounter	U32*	編號單位	取得 Latch 的 Encoder 資料筆數(最大 5000 筆)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;
U32 nCounter = 0 ;

I16 status = _DMC_B02_latch_fifo_dsp_get_data_counter(CardNo, &nCounter);

DELTA CONFIDENTIAL

46.10 _DMC_B02_latch_fifo_dsp_data_counter_reset

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_dsp_data_counter_reset(U16 nCardNo);

■ **目的**

重置 Latch 的 Encoder 資料筆數為 0。(僅支援 2D 版本)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 nCardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;

I16 status = _DMC_B02_latch_fifo_dsp_data_counter_reset(CardNo);
```

DELTA CONFIDENTIAL

46.11 _DMC_B02_latch_fifo_dsp_data_clear

■ **格式**

I16 PASCAL _DMC_B02_latch_fifo_dsp_data_clear(U16 nCardNo);

■ **目的**

清除已從 FIFO 內存取的 Latch 資料。(僅支援 2D 版本)

■ **參數**

名稱	資料型別	單位	描述
nCardNo	U16	編號單位	軸卡號 nCardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;

I16 status = _DMC_B02_latch_fifo_dsp_data_clear(CardNo);

DELTA CONFIDENTIAL

第四十七章 Motion Buffer

表 47

API 歸屬	函數名稱	描述
A/B	_DMC_01_MotionBuf_Set_Crd_Parameters	設置坐標系參數
A/B	_DMC_01_MotionBuf_Get_Crd_Parameters	查詢坐標系參數
A/B	_DMC_01_MotionBuf_Line_MultiAxis	緩存區指令，多軸直線運動
A/B	_DMC_01_MotionBuf_Line_MultiAxis_G00	緩存區指令，多軸直線運動，終點速度始終為 0 (與SNC定義不同)
A/B	_DMC_01_MotionBuf_Arc_Radius	緩存區指令，2軸平面圓弧插補(以終點位置和半徑為輸入參數)
A/B	_DMC_01_MotionBuf_Arc_CenterPos	緩存區指令，2軸平面圓弧插補(以終點位置和中心點為輸入參數)
A/B	_DMC_01_MotionBuf_Set_LookAhead_Parameters	設置插補前瞻功能參數
A/B	_DMC_01_MotionBuf_Start	啟動插補運動
A/B	_DMC_01_MotionBuf_Stop	停止插補運動
A/B	_DMC_01_MotionBuf_Get_Position	查詢指定表格坐標系的當前坐標位置值
A/B	_DMC_01_MotionBuf_Get_Velocity	查詢指定表格坐標系的合成速度值
A/B	_DMC_01_MotionBuf_Feedrate_Override	設置插補運動目標合成速度倍率
A/B	_DMC_01_MotionBuf_Get_Motion_Status	查詢插補運動狀態
A/B	_DMC_01_MotionBuf_Set_Stop_Deceleration	設置插補運動平滑停止、急停合成加速度
A/B	_DMC_01_MotionBuf_Get_Stop_Deceleration	查詢插補運動平滑停止、急停合成加速度
A/B	_DMC_01_MotionBuf_SetUserSegNum	設置自定義插補段段號
A/B	_DMC_01_MotionBuf_GetUserSegNum	讀取自定義插補段段號
A/B	_DMC_01_MotionBuf_Clear_Buffer	清除插補緩存區內的插補數據
A/B	_DMC_01_MotionBuf_Single_Step	單步執行功能
A/B	_DMC_01_MotionBuf_BufGear	實現刀向跟隨功能，啟動某個軸跟隨運動
A/B	_DMC_01_MotionBuf_BufMove	實現刀向跟隨功能，啟動某個軸點位運動

A/B	_DMC_01_MotionBuf_IOControl	緩存區指令，IO 輸出功能
A/B	_DMC_01_MotionBuf_Advanced_IOControl	緩存區指令，IO 提前輸出功能
A/B	_DMC_01_MotionBuf_Delay	緩存區指令，延時功能
A/B	_DMC_01_MotionBuf_Get_Remain_Buffer_Size	查詢插補緩存區剩餘空間
A/B	_DMC_01_MotionBuf_SoftLimit_On	緩存區指令，緩存區內軟體極限開啟
A/B	_DMC_01_MotionBuf_SoftLimit_Off	緩存區指令，緩存區內軟體極限關閉
A/B	_DMC_01_MotionBuf_Clear_Crd	清除坐標系
A/B	_DMC_01_MotionBuf_Disable	緩存區指令，使用此功能可以在流程中關閉 Motion Buffer 功能。
A/B	_DMC_01_MotionBuf_Helix	緩存區指令，3軸立體螺旋插補(以螺旋深度跟間距與中心點為輸入參數)
A/B	_DMC_01_MotionBuf_Line_MultiAxis2	緩存區指令，多軸直線運動，可指定運動軸數。
A/B	_DMC_01_MotionBuf_Line_MultiAxis2_G00	緩存區指令，多軸直線運動，終點速度始終為 0(與 SNC 定義不同)，可指定運動軸數。
A/B	_DMC_01_MotionBuf_Arc_MidPos	緩存區指令，平面圓弧插補(以圓弧經過的中間點與目標點為輸入參數)
A/B	_DMC_01_MotionBuf_MultiAxis_Arc1	緩存區指令，圓弧插補(以圓弧的圓心與角度為輸入參數)做多軸同動
A/B	_DMC_01_MotionBuf_MultiAxis_Arc2	緩存區指令，圓弧插補(以圓弧的終點位置與角度為輸入參數)做多軸同動
A/B	_DMC_01_MotionBuf_MultiAxis_Arc3	緩存區指令，圓弧插補(以圓弧的終點位置與圓心位置為輸入參數)做多軸同動
A/B	_DMC_01_MotionBuf_MultiAxis_Arc4	緩存區指令，圓弧插補(以圓弧經過的中間點與目標點為輸入參數)做多軸同動
A/B	_DMC_01_MotionBuf_Arc_follow	緩存區指令，圓弧插補(可指定任意兩軸作為圓弧運動的X和Y軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動。)
A/B	_DMC_01_MotionBuf_Set_G00_Check	可設定比對 MultiAxis_G00 API 運行完成時驅動器 Feedback 是否達成目標範圍條件，達成條件後才執行下一筆指令，否則停止。(需建立坐標系後才能設定)
A/B	_DMC_01_MotionBuf_Get_G00_Check	取得比對條件設定值
A/B	_DMC_01_MotionBuf_Get_G00_Check_Flag	取得比對狀態

DELTA CONFIDENTIAL

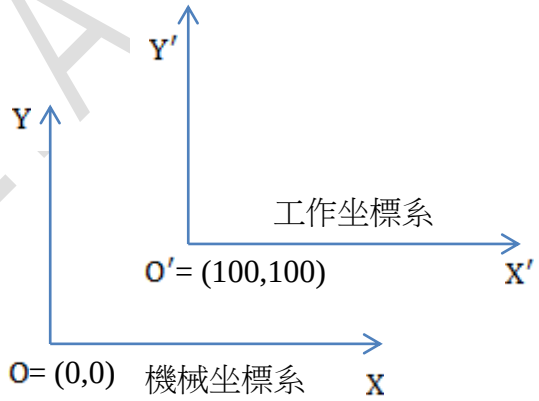
A/B	_DMC_01_MotionBuf_Get_G00_Check_match	取得比對成功時的 Feedback

此表格 API PCI_DMC_B01/A02/F02 可使用

MotionBuffer 可以實現多軸的協調運動，將數據段所描述曲線的起點、終點之過程的空間進行數據密化，根據密化後的數據向各個座標發出進給脈衝對應每個脈衝，機床在相應的坐標方向上移動一個脈衝當量的距離，從而將工件加工出所需要的輪廓形狀，因此在切銷加工等數控裝置中有廣泛的應用。

使用 MotionBuffer 需要至少兩階段操作:
1.建立座標系
2.向緩存區(FIFO)存入運動指令，才能開始執行運動，以下將依序介 MotionBuffer 使用步驟。

1.建立座標系
在運動控制器初始狀態下，還未使用過插補運動，所有的規劃軸都處於單軸運動模式下，尚未指定其座標軸的設定關係。所以進行插補運動時，首先需要建立坐標系，將規劃軸映射到相應的坐標系中。每個坐標系最多支持六軸(X-Y-Z-U-V-W)，用戶可以根據自己的需求，依照(X-Y-Z-U-V-W)順序設定坐標系的軸映射關係及描述運動軌跡。例如下圖即示範一個直線插補運動，其規劃軸映射了一個二維的坐標系系統，並在其描述運動軌跡。



2.向緩存區(FIFO)存入運動指令
插補運動模式採用緩存區運動方式，即用戶需要向插補緩存區中傳遞插補數據，然後，啟動插補運動，運動控制器則會依次執行用戶所傳遞的插補數據，直到所有的插補數據全部運動完成。向緩存區存入數據的指令分成直線插補（Line 相關）和平面圓弧插補指令（Arc 相關）兩種，本章節末段提供了一段直線插補與

DELTA CONFIDENTIAL

圓弧插補使用範例。

3. 以 G0 結尾的運動指令(與 SNC 定義不相同)

這類運動指令將會完成一個完整的加減速過程，即每段運動的合成速度都是從 0 開始，結束的時候也是 0。這類指令與其他插補運動指令(包括直線插補指令和圓弧插補指令)的區別在於：如果使用了前瞻預處理功能，調用其他插補運動指令，則用戶設置的終點速度將會無效，實際的終點速度是前瞻預處理模塊根據用戶設置的前瞻預處理參數和運動軌跡計算出來的一個合理的終點速度；但如果調用 G0 結尾的運動指令，終點速度仍然是 0，控制器不會優化或改變這類指令的終點速度。

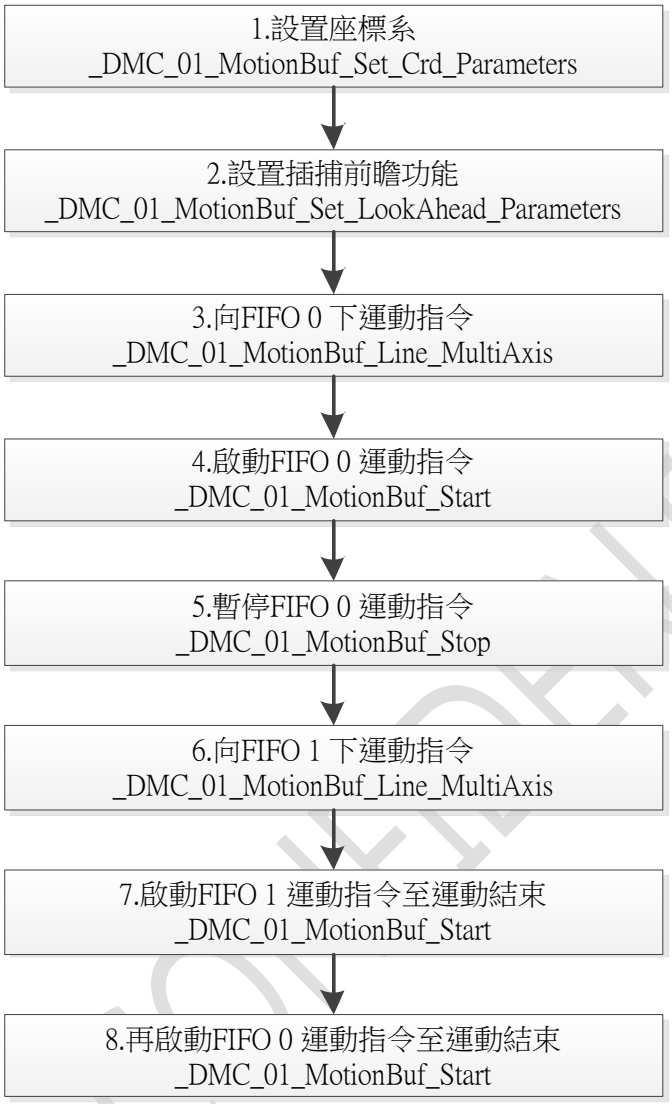
4. Motionbuffer路徑中換刀應用

在插補運動模式下，不能隨意切換到其他運動模式，否則會導致插補坐標系破壞，並且原來載入插補緩存區的數據會丟失。但是在實際應用中，經常會有機床在經過一段路徑的途中需要暫停下來，轉換新路徑做換刀或者移到安全的地方以查看加工效果類似的例子描述的情況，然後再回到暫停時的坐標繼續完成剩餘的路徑，可以透過 FIFO 的管理與切換完成這樣的案例。

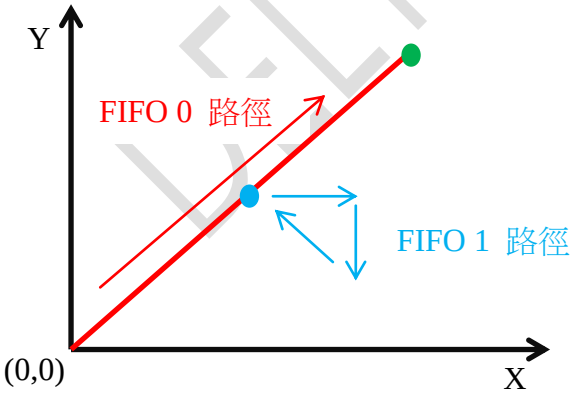
5. 缓存區FIFO的管理（運動暫停與恢復）

為了實現上述操作我們提供了兩個緩存區 FIFO0 和 FIFO1，FIFO0 是用戶的主要插補運動路徑，機床主要的運動指令應該放在 FIFO0 中，FIFO1 是輔助的插補運動路徑，換刀的動作可以放在 FIFO1 中輔助使用。執行中的 FIFO0 的插補運動可以被中斷，中斷後方可進行輔助 FIFO1 的插補運動，輔助 FIFO1 的插補運動完成後，需要將坐標系位置恢復到 FIFO0 主運動被打斷的位置，之後 FIFO0 可從斷點處繼續原來規畫的運動路徑。FIFO1 數據切換必須在 FIFO0 的運動停止的情況下才能輸入，如果 FIFO0 在運動，向 FIFO1 中傳遞插補數據，將會提示錯誤。FIFO1 的插補運動也可以暫停、恢復，但是，在 FIFO1 暫停時不可以進行 FIFO0 的插補運動，否則 FIFO1 緩存區將會被清空，不可以再恢復 FIFO1 的運動，插補主運動與輔助運動，實際的 API 使用執行可以參考下列流程，以下提供了一段機床運動中途換刀範例。

DELTA CONFIDENTIAL



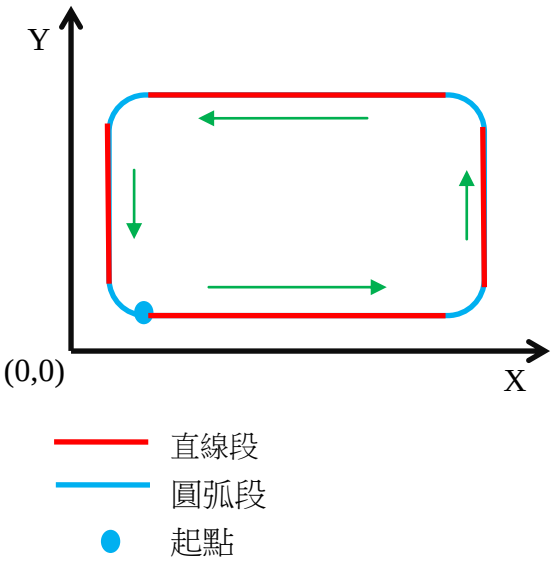
MotionBuffer 插補指令換刀範例流程



- 切換點，切換為 FIFO 1
- FIFO 0 終點

DELTA CONFIDENTIAL

6. 以下提供直線插補與圓弧使用範例



■ **範例**

```
U16 CardNo = 0;
I16 TableNo = 0;
I16 Node[3] = {1,2,3};
I32 i32_SynMaxVel = 6400000;
I32 i32_SynMaxAcc = 12800000;
U16 u16_Scurve_Time = 20;
I32 pi32_OrgPos[2] = {0, 0};
I16 FIFO_NO = 0;
U16 rt ;

rt = _DMC_01_MotionBuf_Set_Crd_Parameters(CardNo, TableNo, 2, NodeID,
        i32_SynMaxVel, i32_SynMaxAcc, u16_Scurve_Time, 0,     0, pi32_OrgPos);

I16 pu16_ArcAxis[2] = {1, 2};
I32 pi32_TargetPos[2] = {4000000, 0};

rt = _DMC_01_MotionBuf_Line_MultiAxis(CardNo, TableNo, pi32_TargetPos, i32_SynMaxVel,
                0, i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 5000000;
pi32_TargetPos[1] = 1000000;
```

DELTA CONFIDENTIAL

```
// arc
rt = _DMC_01_MotionBuf_Arc_Radius(CardNo, TableNo, pu16_ArcAxis, pi32_TargetPos, 1000000, 1,
i32_SynMaxVel,0, i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 5000000;
pi32_TargetPos[1] = 4000000;

rt = _DMC_01_MotionBuf_Line_MultiAxis(CardNo, TableNo, pi32_TargetPos, i32_SynMaxVel,0,
i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 4000000;
pi32_TargetPos[1] = 5000000;
// arc
rt = _DMC_01_MotionBuf_Arc_Radius(CardNo, TableNo, pu16_ArcAxis, pi32_TargetPos, 1000000, 1,
i32_SynMaxVel,0, i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 1000000;
pi32_TargetPos[1] = 5000000;

rt = _DMC_01_MotionBuf_Line_MultiAxis(CardNo, TableNo, pi32_TargetPos, i32_SynMaxVel,
0, i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 0;
pi32_TargetPos[1] = 4000000;

// arc
  rt = _DMC_01_MotionBuf_Arc_Radius(CardNo, TableNo, pu16_ArcAxis, pi32_TargetPos, 1000000, 1,
i32_SynMaxVel,0, i32_SynMaxAcc, 1, FIFO_NO);

pi32_TargetPos[0] = 1000000;
pi32_TargetPos[1] = 0;

// arc
rt = _DMC_01_MotionBuf_Arc_Radius(CardNo, TableNo, pu16_ArcAxis, pi32_TargetPos, 1000000, 1,
i32_SynMaxVel,0, i32_SynMaxAcc, 1, FIFO_NO);

rt=_DMC_01_MotionBuf_Set_LookAhead_Parameters(CardNo, TableNo, i32_SynMaxVel, 0, 100, 100,
FIFO_NO);
```

DELTA CONFIDENTIAL

47.1 _DMC_01_MotionBuf_Set_Crd_Parameters

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Set_Crd_Parameters(U16 CardNo, U16 TableNo, U16 AxisNum, U16 *AxisArray, I32 SycMaxVel, I32 SycMaxAcc, U16 S_Curve_Time, U16 Max_IO_Advance_Time, U16 SetOriginFlag, I32 *OriginPos);

■ **目的**

設置坐標系參數

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxisNum	U16	數值單位	此表格使用的軸數。(Max:9 軸)
AxisArray	U16*	數值單位	存取各種用來進行運動補間的從站編號數組 AxisArray[0] 存放第一組站號 AxisArray[1] 存放第二組站號……等
SycMaxVel	I32	編號單位	此坐標系最大合成速度
SycMaxAcc	I32	編號單位	此坐標系最大合成加速度
S_Curve_Time	U16	數值單位	S-Curve 濾波次數
Max_IO_Advance_Time	U16	數值單位	IO 提前最長時間。單位 ：ms(註 Max:200ms)
SetOriginFlag	U16	數值單位	表示是否需要指定坐標系的原點坐標的規劃位置，該參數可以方便用戶建立區別於機械坐標系的工作坐標系。 0：不需要指定原點坐標值，則坐標系的原點在當前規劃位置上。 1：需要指定原點坐標值，坐標系的原點在 originPos 指定的規劃位置上。
OriginPos	I32*	選項單位	指定的原點資料陣列

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

DELTA CONFIDENTIAL

I16 rt;
U16 CardNo=0,TableNo=0, AxisNum=2, AxisNoArray[2]={1,2},
//設置坐標系陣列 0,1 對應軸號 1,2
U16 S_Curve_Time=0,Max_IO_Advance_Time=0,
U16 SetOriginFlag=0;
I32 SycMaxVel=12800000, SycMaxAcc =128000000, OriginPos[2]={0,0}

I16 rt = _DMC_01_MotionBuf_Set_Crd_Parameters(CardNo, TableNo, AxisNum, AxisArray, SycMaxVel, SycMaxAcc, S_Curve_Time, Max_IO_Advance_Time, SetOriginFlag, OriginPos)

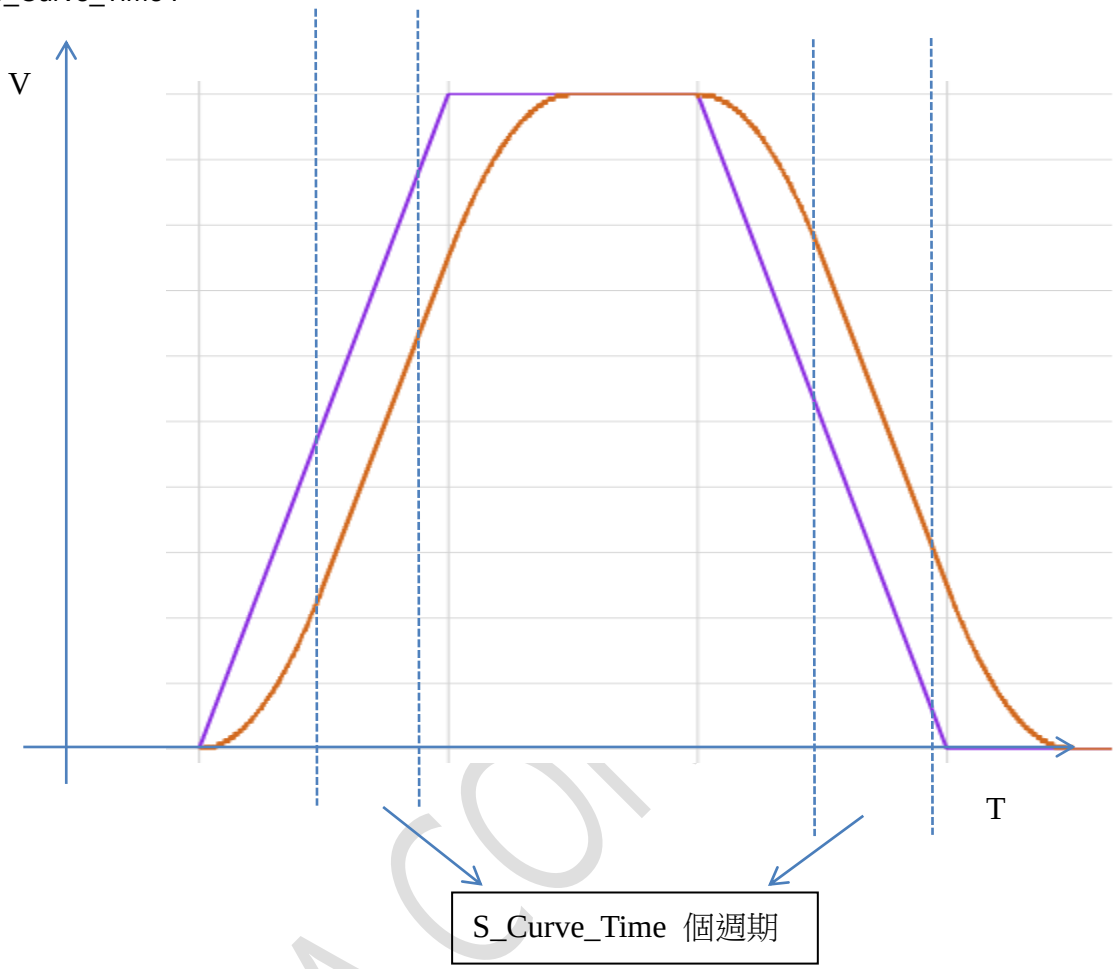
■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

DELTA CONFIDENTIAL

NOTE

1.S_Curve_Time :



未設置 S_Curve_Time 濾波次數

設置 S_Curve_Time 濾波次數

2.

設置坐標系陣列 0,1 對應軸號 1,2，後續所使用運動指令需填入坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.2 _DMC_01_MotionBuf_Get_Crd_Parameters

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Crd_Parameters(U16 CardNo, U16 TableNo, U16 *AxisNum, U16 *AxisArray, I32 *SycMaxVel, I32 *SycMaxAcc, U16 *S_Curve_Time, U16 *Max_IO_Advance_Time, U16 *SetOriginFlag, I32 *OriginPos)

■ **目的**

查詢坐標系參數

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxisNum	U16*	數值單位	此表格使用的軸數。(Max:9 軸)
AxisArray	U16*	數值單位	存取各種用來進行運動補間的從站編號數組 AxisArray[0] 存放第一組站號 AxisArray[1] 存放第二組站號……等
SycMaxVel	I32*	編號單位	此坐標系最大合成速度
SycMaxAcc	I32*	編號單位	此坐標系最大合成加速度
S_Curve_Time	U16*	數值單位	S-Curve 濾波次數
Max_IO_Advance_Time	U16*	數值單位	IO 提前最長時間。單位 ：ms(註 Max:200ms)
SetOriginFlag	U16*	數值單位	表示是否需要指定坐標系的原點坐標的規劃位置，該參數可以方便用戶建立區別於機械坐標系的工作坐標系。 0：不需要指定原點坐標值，則坐標系的原點在當前規劃位置上。 1：需要指定原點坐標值，坐標系的原點在 originPos 指定的規劃位置上。
OriginPos	I32*	選項單位	指定的原點資料陣列

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0, AxisNum, AxisNoArray[2]={1,2},
U16 S_Curve_Time=0,Max_IO_Advance_Time=0,
U16 SetOriginFlag=0;
I32 SycMaxVel=12800000, SycMaxAcc =128000000, OriginPos[2]={0,0}

I16 rt = _DMC_01_MotionBuf_Get_Crd_Parameters(CardNo, TableNo,&AxisNum, AxisArray,
&SycMaxVel, &SycMaxAcc, &S_Curve_Time,&Max_IO_Advance_Time, &SetOriginFlag,
&OriginPos)
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

DELTA CONFIDENTIAL

47.3 _DMC_01_MotionBuf_Line_MultiAxis

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Line_MultiAxis(U16 CardNo, U16 TableNo, I32 *TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，多軸直線運動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同指定表格坐標系軸數。 單位 ：Pulse。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 TargetPosition[2] = {10000000, 10000000};
I32 SynTargetVel = 1280000, SynEndVel = 0, SynAcc = 1280000;
```

DELTA CONFIDENTIAL

U16 AbsMode = 1;
U16 FifoNo = 0;

I16 rt = _DMC_01_MotionBuf_Line_MultiAxis(CardNo, TableNo, TargetPosition, SynTargetVel, SynEndVel, SynAcc, AbsMode, FifoNo)

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令 TargetPosition 會對應到_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列軸號。

DELTA CONFIDENTIAL

47.4 _DMC_01_MotionBuf_Line_MultiAxis_G00

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Line_MultiAxis_G00(U16 CardNo, U16 TableNo, I32 *TargetPosition, I32 SynTargetVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，多軸直線運動，終點速度始終為 0(與 SNC 定義不同)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 AxisNum 數值。 單位 ：Pulse。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 TargetPosition[2] = {10000000, 10000000};
I32 SynTargetVel = 1280000, SynAcc = 1280000;
U16 AbsMode = 1;
U16 FifoNo = 0;
```

DELTA CONFIDENTIAL

I16 rt = _DMC_01_MotionBuf_Line_MultiAxis(CardNo, TableNo, TargetPosition, SynTargetVel, SynAcc, AbsMode, FifoNo)

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令 TargetPosition 會對應到_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列軸號。

DELTA CONFIDENTIAL

47.5 _DMC_01_MotionBuf_Arc_Radius

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Arc_Radius(U16 CardNo, U16 TableNo, U16 *Arc_Axis, I32 *Arc_TargetPosition, I32 Radius, U16 CircleDir ,I32 SynTargetVel , I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，2 軸平面圓弧插補(以終點位置和半徑為輸入參數)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Arc_Axis	U16*	編號單位	此陣列大小為 2，輸入參與作圓弧的坐標系軸號。 Arc_No[0] = 0 ,表示圓弧的第一軸為坐標系 X 軸 Arc_No[1] = 1 ,表示圓弧的第二軸為坐標系 Y 軸
Arc_TargetPosition	I32*	數值單位	此圓弧插補運動的末點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
Radius	I32	數值單位	圓弧插補的圓弧半徑值。 半徑為正時，表示圓弧為小於等於 180°圓弧。半徑為負時，表示圓弧為大於 180°圓弧。半徑描述方式不能用來描述整圓。 單位 ：Pulse
CircleDir	U16	選項單位	圓弧的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式

DELTA CONFIDENTIAL

			1：絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

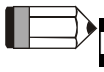
```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 Arc_Axis[2]={0, 1};
//運動指令需填入坐標系陣列才能對應正確軸號。

I32 TargetPosition[2]={10000000,10000000} ,Radius =256000,SynTargetVel=1280000,
SynEndVel=0, SynAcc=1280000;
U16 AbsMode=1;
U16 FifoNo =0, CircleDir=1 ;

rt = _DMC_01_MotionBuf_Arc_Radius(CardNo, TableNo, Arc_Axis, TargetPosition, Radius,
CircleDir ,SynTargetVel , SynEndVel, SynAcc, AbsMode, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯

DELTA CONFIDENTIAL

	誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

TargetPosition[2] 只需填入陣列 0,1

DELTA CONFIDENTIAL

47.6 _DMC_01_MotionBuf_Arc_CenterPos

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Arc_CenterPos(U16 CardNo, U16 TableNo, U16 *Arc_Axis, I32 *TargetPosition, I32 *CenterPos, U16 CircleDir ,I32 SynTargetVel , I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo);

■ **目的**

緩存區指令，2 軸平面圓弧插補(以終點位置和中心點為輸入參數)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Arc_Axis	U16*	編號單位	此陣列大小為 2，輸入參與作圓弧的坐標系軸號。 Arc_No[0] = 0 ,表示圓弧的第一軸為坐標系 X 軸 Arc_No[1] = 1 ,表示圓弧的第二軸為坐標系 Y 軸
TargetPosition	I32*	數值單位	此圓弧插補運動的末點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
CenterPos	I32*	數值單位	圓弧插補的圓心相對於起點位置的 2 軸偏移量，陣列大小為 2。 單位 ：Pulse
CircleDir	U16	選項單位	圓弧的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度。 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

DELTA CONFIDENTIAL

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

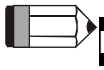
```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 Arc_Axis[2]={0, 1},
//運動指令需填入坐標系陣列才能對應正確軸號。

I32 TargetPos[2] = {0, 2000000};
I32 CenterPos[2] = {0, 1000000};

U16 CircleDir=1 ;
I32 SynTargetVel=1280000, SynEndVel=0, SynAcc=1280000;
U16 AbsMode =0;
U16 FifoNo =0;

rt =_DMC_01_MotionBuf_Arc_CenterPos(CardNo, TableNo, Arc_Axis, TargetPosition, CenterPos,
CircleDir ,SynTargetVel ,SynEndVel, SynAcc, AbsMode, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯

DELTA CONFIDENTIAL

	誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.7 _DMC_01_MotionBuf_Set_LookAhead_Parameters

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Set_LookAhead_Parameters(U16 CardNo, U16 TableNo, I32 RefSwerveSpd, U16 ArcSpdModify, I32 ArcRefSpeed, I32 ArcRefRadius,U16 FifoNo)

■ **目的**

設置插補前瞻功能參數。
前瞻預處理功能可以根據用戶的運動路徑計算出平滑的速度規劃，減少機床的衝擊，從而提高加工精度。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
RefSwerveSpd	I32	數值單位	轉角參考速度 單位 ：Pulse/s
ArcSpdModify	U16	數值單位	是否開啟圓弧速度修正功能。 0：關閉 1：開啟
ArcRefSpeed	I32	數值單位	圓弧參考速度。 單位 ：Pulse/s
ArcRefRadius	I32	數值單位	圓弧參考半徑。 單位 ：Pulse
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 RefSwerveSpd=1600000;
U16 ArcSpdModify=0;
I32 ArcRefSpeed=100;
```

DELTA CONFIDENTIAL

```
I32 ArcRefRadius=100;
U16 FifoNo=0;

rt = _DMC_01_MotionBuf_Set_LookAhead_Parameters (CardNo, RefSwerveSpd, ArcSpdModify,
ArcRefSpeed, ArcRefRadius, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

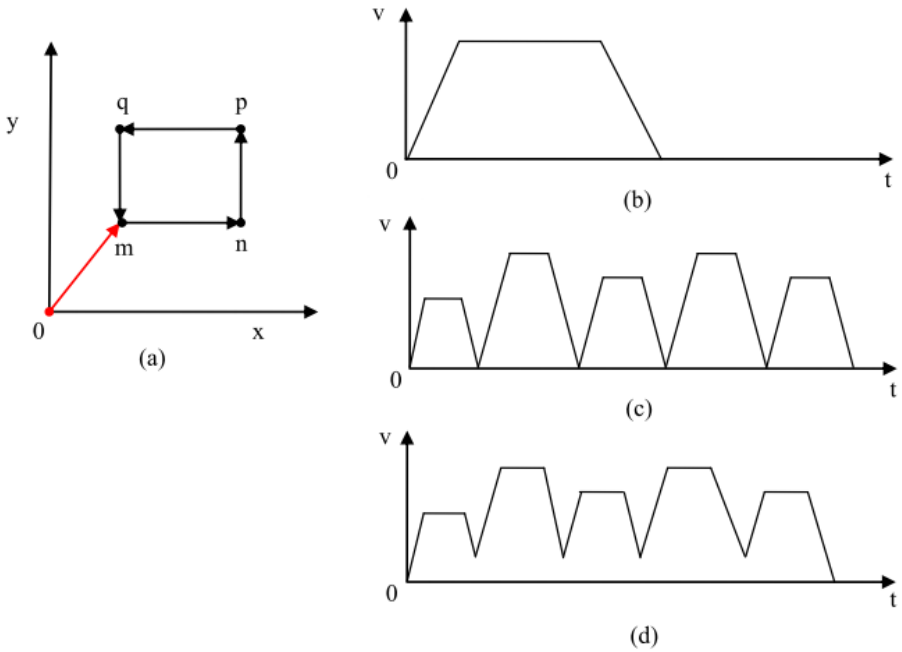
插補前瞻功能屬於設定用途 API，不會佔據 Buffer 空間。

在數控加工等應用中，要求數控系統對機床進行平滑的控制，以防止較大的衝擊影響零件的加工質量。前瞻預處理功能可以根據用戶的運動路徑計算出平滑的速度規劃，減少機床的衝擊，從而提高加工精度。

以下圖(a) 路徑規劃為案例，如果按照下圖(b)所示的速度規劃，即在拐角處不減速，則加工精度一定會較低，而且可能在拐彎時對刀具和零件造成較大衝擊。如果按照下圖(c)所示的速度規劃，即在拐角處減速為 0，可以最大限度保證加工精度，但加工速度就會慢下來。如果按照下圖(d)所示的速

DELTA CONFIDENTIAL

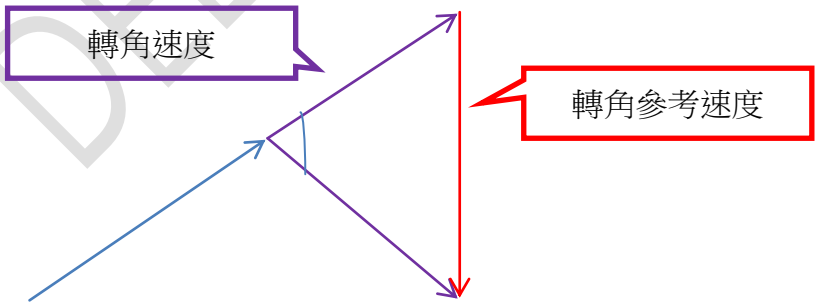
度規劃，在拐角處將速度減到一個合理值，既可以滿足加工精度又能提高加工速度，就是一個好的速度規劃。



為了實現類似上圖(d) 所示良好的速度規劃，前瞻預處理模塊不僅要知道當前運動的位置參數，還要提前知道後面規劃的運動位置參數，這就是所謂的前瞻。例如在對上圖(a) 中的軌跡做前瞻預處理時，我們設定控制器預先讀取 50 段運動軌跡到緩存區中，則它會自動分析出在第幾段將會出現拐點，並依據用戶設定的拐彎時間計算在拐彎處的終點速度。前瞻預處理模塊也會依照用戶設定座標的最大加速度值計算速度規劃，使任何加減速過程都不會超過這個值，防止對機械部分產生破壞性衝擊力。

RefSwerveSpd:

轉角參考速度。
轉角的速度和轉角角度之間有一個關係式。



DELTA CONFIDENTIAL

$$\text{轉角速度} \leq \frac{\text{RefTurnOnSpd}}{2 \times \sin\frac{\theta}{2}}$$

舉例來說，如果轉角為 180 度，RefTurnOnSpd = 100000

$$\text{轉角速度} \leq \frac{100000}{2 \times \sin\frac{180}{2}} = \frac{100000}{2} = 50000$$

即轉角速度不會超過 50000 (pulse/秒)

如轉角為 60 度，RefTurnOnSpd = 100000

$$\text{轉角速度} \leq \frac{100000}{2 \times \sin\frac{60}{2}} = \frac{100000}{1} = 100000$$

則轉角速度等於 100000 (pulse/秒)
亦即一連串的運動如最大速度為 100000，RefTurnOnSpd = 100000
則只要個運動間的夾角不超過 60 度，轉角就不會降速。

ArcSpdModify:

如開啟 ArcSpdModify 功能，則會修正圓弧速度
修正速度的算式如下

ArcRefSpeed：曲線的參考速度
ArcRefRadius：曲線的參考半徑

$$\text{圓弧速度} = \text{ArcRefSpeed} \times \sqrt{\frac{\text{此圓弧的曲率半徑}}{\text{ArcRefRadius}}}$$

圓弧速度和 ArcRefSpeed 成正比，和參考半徑的根號成反比。

DELTA CONFIDENTIAL

47.8 _DMC_01_MotionBuf_Start

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Start(U16 CardNo, U16 TableNo , U16 FifoNo)

■ **目的**

啟動插補運動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	欲啟動的插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

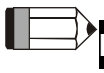
```
I16 rt;
U16 CardNo=0,TableNo=0,FifoNo =0;
rt = _DMC_01_MotionBuf_Start(CardNo, TableNo, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間

DELTA CONFIDENTIAL

	超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

如開啟單步執行功能，則插補運動只會執行一個緩衝區指令即停止，如欲繼續動作則須重複使用此 Function。

Feedhold 之後，使用此 Function 即可再次啟動插補運動。

DELTA CONFIDENTIAL

47.9 _DMC_01_MotionBuf_Stop

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Stop(U16 CardNo, U16 TableNo , U16 FifoNo, U16 Mode)

■ **目的**

停止插補運動(三種模式)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	欲啟動的表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	欲啟動的插補緩存區號碼。(0 ~ 1)
Mode	U16	選項單位	0 : Feedhold 1： 平滑停止 2： 緊急停止

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0, TableNo=0, FifoNo =0, Mode =2;
rt =_DMC_01_MotionBuf_Stop(CardNo, TableNo, FifoNo, Mode);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動

DELTA CONFIDENTIAL

ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

平滑停止 和 緊急停止 動作完成後，會清除 Motion Buffer 的資料，並解除相關軸的 Motion Buffer 狀態

DELTA CONFIDENTIAL

47.10 _DMC_01_MotionBuf_Get_Position

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Position(U16 CardNo, U16 TableNo ,I32 *Now_Position, I32 *Now_Position_org)

■ **目的**

查詢指定表格坐標系的當前坐標位置值。獲取的坐標值可能和規劃位置不一致，取決於建立坐標系的原點是否為零。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Now_Position	I32*	數值單位	回傳坐標系的當前位置，數組的元素個數取決於該坐標系的軸數。 單位 ：Pulse
Now_Position_org	I32*	數值單位	回傳工作坐標系。 (_DMC_01_MotionBuf_Set_Crd_Parameters 設置需要指定原點坐標值) 單位 ：Pulse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 Now_Position[2]={0, 0};
I32 Now_Position_org [2]={0, 0};
rt = _DMC_01_MotionBuf_Get_Position (CardNo, TableNo , Now_Position , Now_Position_org);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

Now_Position_org 與_DMC_01_MotionBuf_Set_Crd_Parameters 是否指定原點坐標相關

DELTA CONFIDENTIAL

47.11 _DMC_01_MotionBuf_Get_Velocity

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Velocity(U16 CardNo, U16 TableNo ,I32 *CurrentSpeed)

■ **目的**

查詢指定表格坐標系的合成速度值

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
CurrentSpeed	I32*	數值單位	回傳坐標系的當前合成速度 單位 ：Pulse/s

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

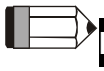
```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 CurrentSpeed;
rt = _DMC_01_MotionBuf_Get_Velocity (CardNo, TableNo,&CurrentSpeed);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動

DELTA CONFIDENTIAL

ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



DELTA CONFIDENTIAL

47.12 _DMC_01_MotionBuf_Feedrate_Override

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Feedrate_Override(U16 CardNo, I16 TableNo ,F64 SynVelRatio)

■ **目的**

設置插補運動目標合成速度倍率

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
SynVelRatio	F64	數值單位	設置的插補目標速度倍率 數值範圍 0~10，預設為 1

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

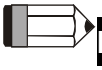
```
I16 rt;
U16 CardNo=0,TableNo=0;
F64 SynVelRatio=1;
rt = _DMC_01_MotionBuf_Feedrate_Override (CardNo, TableNo, SynVelRatio);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突

DELTA CONFIDENTIAL

ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

DELTA CONFIDENTIAL

47.13 _DMC_01_MotionBuf_Get_Motion_Status

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Motion_Status(U16 CardNo, U16 TableNo , U16 *Status,I32 *MD_BlockNum, U16 FifoNo)

■ **目的**

查詢插補運動狀態

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Status	U16*	數值單位	回傳插補運動狀態。 0 ：未啟動 1 ：啟動 2 ：停止
MD_BlockNum	I32*	數值單位	回傳當前已經完成的插補段數。
FifoNo	U16	數值單位	欲查詢的插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

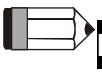
■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0 , FifoNo =0 ,Status;
I32 MD_BlockNum;
rt  = _DMC_01_MotionBuf_Get_Motion_Status (CardNo, TableNo, &Status,& MD_BlockNum ,FifoNo);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

DELTA CONFIDENTIAL

47.14 _DMC_01_MotionBuf_Set_Stop_Deceleration

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Set_Stop_Deceleration(U16 CardNo, U16 TableNo ,I32 DecSmoothStop, I32 DecAbruptStop)

■ **目的**

設置插補運動平滑停止、急停合成加速度

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
DecSmoothStop	I32	數值單位	設置的坐標系合成平滑停止加速度 單位 ：Pulse/s^2
DecAbruptStop	I32	數值單位	設置的坐標系合成急停加速度 單位 ：Pulse/s^2

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 DecSmoothStop=1, DecAbruptStop= 1;
rt = _DMC_01_MotionBuf_Set_Stop_Deceleration (CardNo, TableNo, DecSmoothStop, DecAbruptStop);
```

■ **Error Code:**

DSP Path.h	Return Code

DELTA CONFIDENTIAL

ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

DELTA CONFIDENTIAL

47.15 _DMC_01_MotionBuf_Get_Stop_Deceleration

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Stop_Deceleration(U16 CardNo, U16 TableNo ,I32 *DecSmoothStop, I32 *DecAbruptStop)

■ **目的**

查詢插補運動平滑停止、急停合成加速度

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
DecSmoothStop	I32*	數值單位	回傳坐標系合成平滑停止加速度 單位 ：Pulse/s^2
DecAbruptStop	I32*	數值單位	回傳坐標系合成急停加速度 單位 ：Pulse/s^2

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 DecSmoothStop, DecAbruptStop;
rt = _DMC_01_MotionBuf_Get_Stop_Deceleration (CardNo,TableNo,&DecSmoothStop,
&DecSmoothStop);
```

■ **Error Code:**

DSP Path.h	Return Code

DELTA CONFIDENTIAL

ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

DELTA CONFIDENTIAL

47.16 _DMC_01_MotionBuf_SetUserSegNum

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_SetUserSegNum(U16 CardNo, U16 TableNo ,I32 SegNum, U16 FifoNo)

■ **目的**

設置自定義插補段段號

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
SegNum	I32	數值單位	自定義插補段段號。
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

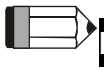
```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 SegNum=1;
U16 FifoNo =0
rt = _DMC_01_MotionBuf_SetUserSegNum (CardNo, TableNo, SegNum, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動

DELTA CONFIDENTIAL

ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command


NOTE

DELTA CONFIDENTIAL

47.17 _DMC_01_MotionBuf_GetUserSegNum

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_GetUserSegNum(U16 CardNo, U16 TableNo ,I32 *SegNum, U16 FifoNo)

■ **目的**

讀取自定義插補段段號

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
SegNum	I32*	數值單位	讀取當前運行到的自定義插補段段號。
FifoNo	U16	數值單位	欲讀取的插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

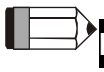
```
I16 rt;
U16 CardNo=0,TableNo=0;
I32 SegNum;
U16 FifoNo = 0;
rt = _DMC_01_MotionBuf_GetUserSegNum(CardNo, TableNo, &SegNum, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動

DELTA CONFIDENTIAL

ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

DELTA CONFIDENTIAL

47.18 _DMC_01_MotionBuf_Clear_Buffer

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Clear_Buffer(U16 CardNo, U16 TableNo , U16 FifoNo)

■ **目的**

清除插補緩存區內的插補數據

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	欲清除的插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

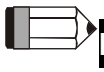
```
I16 rt;
U16 CardNo=0,TableNo=0,FifoNo =0;
rt = _DMC_01_MotionBuf_Clear_Buffer (CardNo, TableNo, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動

DELTA CONFIDENTIAL

	已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



DELTA CONFIDENTIAL

47.19 _DMC_01_MotionBuf_Single_Step

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Single_Step(U16 CardNo, U16 TableNo , U16 FifoNo, U16 Enable)

■ **目的**

單步執行功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)
Enable	U16	選項單位	0：關閉單步執行功能 1：開啟單步執行功能

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

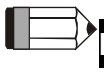
```
I16 rt;
U16 CardNo=0, TableNo=0, FifoNo =0, Enable=1;
rt = _DMC_01_MotionBuf_Single_Step (CardNo, TableNo, FifoNo, Enable);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐

DELTA CONFIDENTIAL

	標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

DELTA CONFIDENTIAL

47.20 _DMC_01_MotionBuf_BufGear

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_BufGear(U16 CardNo, U16 TableNo ,U16 GearAxisNo, I32 Distance, U16 FifoNo)

■ **目的**

實現刀向跟隨功能，啟動某個軸跟隨運動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
GearAxisNo	U16	數值單位	需要進行跟隨運動的軸號,該軸不能處於坐標系中。
Distance	I32	數值單位	跟隨運動的位移量，單位：pulse。
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0,GearAxisNo=3, FifoNo=0;
I32 Gear_Dist=320000;

rt = _DMC_01_MotionBuf_BufGear (CardNo, TableNo, GearAxisNo, Gear_Dist, FifoNo);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

在使用插補緩存區中的跟隨運動功能時，需要注意以下內容：

(1) BufferGear 必需選擇坐標系以外的軸號，否則API將無法正常執行

(2) BufferGear指令需要在所要跟隨的插補段之前，中間不要間隔其他種類的指令，但可以同時調用多個BufferGear指令來實現多個軸跟隨插補運動。

(3) 當暫停坐標系運動時，插補的合成速度會減速到 0，跟隨軸的速度也會減速為 0，如果希望重新恢復坐標系運動時，跟隨軸仍能夠實現跟隨，不要調用 BufferStop 指令停止跟隨軸的運動，否則重新啟動緩存區的運動時，跟隨軸將無法完成正在進行的跟隨運動。

DELTA CONFIDENTIAL

47.21 _DMC_01_MotionBuf_BufMove

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_BufMove(U16 CardNo, U16 TableNo ,U16 AxisNo, I32 Distance, I32 TargetVel , I32 Acceleration, U16 AbsMode, U16 BufMove_Mode, U16 FifoNo)

■ **目的**

實現刀向跟隨功能，啟動某個軸點位運動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxisNo	U16	數值單位	需要進行點位運動的軸號,該軸不能處於坐標系中。
Distance	I32	數值單位	點位運動的目標位置，單位：pulse。
TargetVel	I32	數值單位	點位運動的目標速度，單位：pulse/s。
Acceleration	I32	數值單位	點位運動的加速度，單位：pulse/s^2。
AbsMode	U16	選項單位	0：相對模式 1：絕對模式
BufMove_Mode	U16	選項單位	點位運動的模式。 0：該指令為非模態指令，即不阻塞後續的插補緩存區指令的執行。 1：該指令為模態指令，將會阻塞後續的插補緩存區指令的執行。
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0, AxisNo=3;
I32 Distance=160000;
```

DELTA CONFIDENTIAL

I32 TargetVel=1280000 , Acceleration =1280000;
U16 AbsMode=0;
U16 BufMove_Mode=0;
U16 FifoNo=0;

rt = _DMC_01_MotionBuf_BufMove (CardNo, TableNo, AxisNo, Distance, TargetVel, Acceleration, AbsMode, BufMove_Mode, FifoNo);

- **Error Code:**

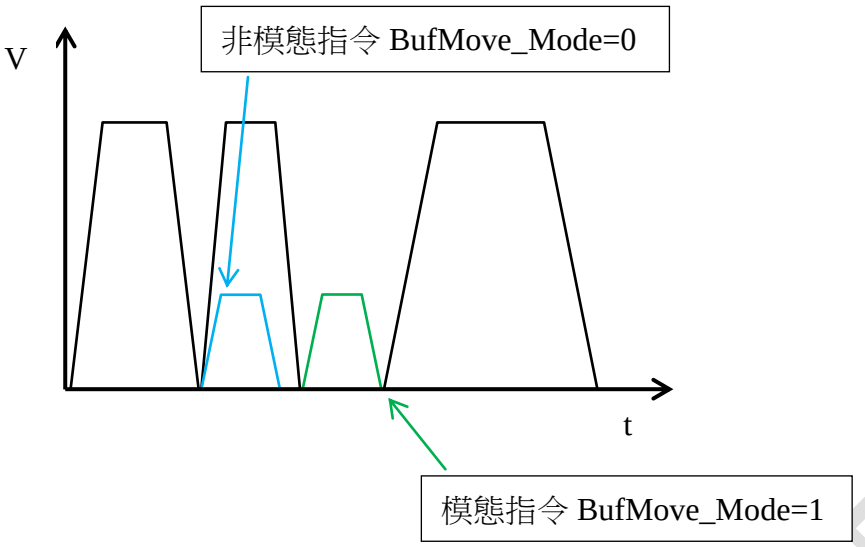
DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

屬於 Buffer 指令類功能 API。在插補運動過程中，如果需要坐標係以外的軸進行點位運動，則可以通過在緩存區中壓入 BufferMove 指令來實現。BufferMove 可以在插補運動的過程中插入模態和非模態的點位運動。模態指令的意義是，在進行該點位運動時，後續的插補緩存區中的指令將會被暫停執行，直到該指令執行完畢後，才執行下一條指令；非模態指令的意義是，該指令啟動了一個軸

DELTA CONFIDENTIAL

的點位運動後，立即取下一條緩存區中的指令執行，不會等待點位運動的結束。



如上圖所示，其中黑線為插補運動的合成速度，藍線、綠線為點位運動軸的速度值，藍線的第一個點位運動是非模態指令，與插補運動同時運動，而綠線第二個點位運動是模態指令，會阻塞插補運動(也可視為 BufferMove 被排入規畫的 Motion 的排程，只是跟隨軸非坐標系內的軸度)，只有點位運動結束之後，才接續進行插補運動。

在使用插補緩存區中的跟隨運動功能時，需要注意以下內容：
(1) BufferMove 必需選擇坐標系以外的軸號，否則 API 將無法正常執行
(2) 點位運動的目標位置是相對於機床原點的絕對位置。
(3) 如果在上一次的緩存區點位運動沒有運動完成，又發送了新的點位運動，則會按照新的點位運動指令進行規劃，即可以在插補緩存區中修改點位運動的目標位置和目標速度。
(4)如果在運動過程中停止插補緩存區的運動，則點位運動將不會停止，如果需要停止點位運動，則需要調用 Buffer_Stop 指令停止響應軸的運動。恢復緩存區運動時，用戶需自行保證之前點位運動的軸在合適的位置上。
(5)當在模態點位運動的過程中，進行點位運動的軸由於觸發限位等異常原因停止時，插補緩存區將不會再繼續運行，此時用戶需排查異常情況，重新設置相應參數，使系統正常後才可以工作。

(6)當模式為非模態指令運動且正在執行，此時若接著下_DMC_01_MotionBuf_BufGear 與 MotionBuffer 插補運動，將會使當前運動停止，需再_DMC_01_MotionBuf_Start，才能再啟動 BufGear 與 MotionBuffer 插補運動功能。

(7) 當模式為模態指令運動且正在執行，此時若接著下_DMC_01_MotionBuf_BufGear 與 MotionBuffer 插補運動，將等待模態指令運動完成後，接著啟動 BufGear 與 MotionBuffer 插補運動

DELTA CONFIDENTIAL

功能。

DELTA CONFIDENTIAL

47.22 _DMC_01_MotionBuf_IOControl

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_IOControl(U16 CardNo, U16 TableNo , U16 IO_Control_Num , IO_Set* IO_Array, U16 FifoNo)

■ **目的**

緩存區指令，IO 輸出功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
IO_Control_Num	U16	數值單位	IO 控制數量(Max:32)
IO_Array	IO_Set*	結構陣列	IO 資料陣列。(包含 IO 資訊及狀態) IO_Array[i].IO_Info.NodeID:設定控制站號 IO_Array[i].IO_Info.SlotID:保留 IO_Array[i].IO_Info.BitNo:設定控制的 Bit 點位 IO_Array[i].Value:設定 1 為 ON，0 為 OFF 1.一筆 IO_Array[i]資料只能設定控制 1 個 Node 中的 1 個 bit 2.若要使用 PCI-DMC-A02/F02 Local IO 則 IO_Array[i].IO_Info.NodeID 設定為 13
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

在 PCI_DMC.H 中會提供以下 struct

```
typedef struct
{
    U16 NodeID;
    U16 SlotID;
    U16 BitNo;

}IO_Information;    //MotionBuffer

typedef struct
{
    IO_Information IO_Info;
    U16 Value;

}IO_Set, *pIO_Set;          //MotionBuffer

//點位可由使用者自行規劃設定，
I16 rt ;
U16 CardNo=0,TableNo=0;
U16 FifoNo=0;
U16 i;
IO_Set IO_Array[32];
for(i=0;i<24;i++)
{
    IO_Array[i].IO_Info.NodeID = 13;    //A02/F02 LocalIO
    IO_Array[i].IO_Info.SlotID = 0;
    IO_Array[i].IO_Info.BitNo = i;
    IO_Array[i].Value = 1;// IO Control ON

}

rt = _DMC_01_MotionBuf_IOControl(CardNo, TableNo, 1, &IO_Array[0], FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿

DELTA CONFIDENTIAL

ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

DELTA CONFIDENTIAL

47.23 _DMC_01_MotionBuf_Advanced_IOControl

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Advanced_IOControl(U16 CardNo, U16 TableNo , U16 IO_Control_Num , IO_Set* IO_Array, U16 Time, U16 Mode, U16 FifoNo)

■ **目的**

緩存區指令，IO 提前輸出功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
IO_Control_Num	I32	數值單位	IO 控制數量(Max:32)
IO_Array	IO_Set*	結構陣列	IO 資料陣列。(包含 IO 資訊及狀態) IO_Array[i].IO_Info.NodeID:設定控制站號 IO_Array[i].IO_Info.SlotID:保留 IO_Array[i].IO_Info.BitNo:設定控制的 Bit 點位 IO_Array[i].Value:設定 1 為 ON，0 為 OFF 1.一筆資料只能設定控制 1 個 Node 中的 1 個 bit 2.若要使用 PCI-DMC-A02/F02 Local IO 則 IO_Array[i].IO_Info.NodeID 設定為 13
Time	U16	數值單位	IO 提前(延遲)輸出時間。 單位 ：ms 提前時間 Max: 200 ms
Mode	U16	選項單位	0： 延遲輸出 1： 提前輸出
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

在 PCI_DMC.H 中會提供以下 struct

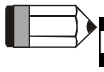
```
typedef struct
{
    U16 NodeID;
    U16 SlotID;
    U16 BitNo;

}IO_Information;    //MotionBuffer

typedef struct
{
    IO_Information IO_Info;
    U16 Value;

}IO_Set, *pIO_Set;          //MotionBuffer

//點位可由使用者自行規劃設定，
I16 rt;
U16 CardNo=0,TableNo=0;
U16 FifoNo=0;
U16 i;
IO_Set IO_Array[32];

IO_Array[i].IO_Info.NodeID = 1;
IO_Array[i].IO_Info.SlotID = 0;
IO_Array[i].IO_Info.BitNo = 0;
IO_Array[i].Value = 1;

rt = _DMC_01_MotionBuf_Advanced_IOControl(CardNo, TableNo, 1, &IO_Array[0], 100, 0,
FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動

DELTA CONFIDENTIAL

ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



DELTA CONFIDENTIAL

47.24 _DMC_01_MotionBuf_Delay

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Delay(U16 CardNo, U16 TableNo ,F64 DelayTime, U16 FifoNo);

■ **目的**

緩存區指令，延時功能

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
DelayTime	F64	數值單位	延時時間。 單位 ：s。
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0, TableNo=0, FifoNo =0;
F16 DelayTime= 1;
rt =_DMC_01_MotionBuf_Delay(CardNo, TableNo, DelayTime, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動

DELTA CONFIDENTIAL

ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

DELTA CONFIDENTIAL

47.25 _DMC_01_MotionBuf_Get_Remain_Buffer_Size

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_Remain_Buffer_Size(U16 CardNo, U16 TableNo ,I32 *RemainBufSize, U16 FifoNo)

■ **目的**

查詢插補緩存區剩餘空間

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
RemainBufSize	I32*	數值單位	回傳插補緩存區剩餘空間
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

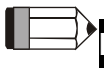
```
I16 rt;
U16 CardNo=0,TableNo=0, FifoNo =0;
I32 RemainBufSize;
rt = _DMC_01_MotionBuf_Get_Remain_Buffer_Size (CardNo, TableNo, &RemainBufSize ,FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐

DELTA CONFIDENTIAL

	標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

DELTA CONFIDENTIAL

47.26 _DMC_01_MotionBuf_SoftLimit_On

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_SoftLimit_On(U16 CardNo, U16 TableNo, U16 CD_AxisNo, I16 LimitType, U16 Mode, U16 FifoNo)

■ **目的**

緩存區指令，緩存區內軟體極限開啟

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
CD_AxisNo	U16	編號單位	需要將限位有效的座標系軸編號
LimitType	I16	數值單位	需要有效的限位類型。 0：需要將該軸的正限位設置為有效。 1：需要將該軸的負限位設置為有效。 -1：需要將該軸的正限位和負限位都設置為有效。
Mode	U16	數值單位	1：緊急停止 2：減速停止
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0, TableNo=0, CD_AxisNo=1, Mode =1, FifoNo =0;
I16 LimitType= -1;
rt = _DMC_01_set_soft_limit(CardNo, AxisNoArray[1], 0, 3000000, 2000000);
// AxisNoArray[1]為_DMC_01_MotionBuf_Set_Crd_Parameters所設置的坐標系陣列1

rt = _DMC_01_enable_soft_limit(CardNo, AxisNoArray[1], 0, 1);
rt =_DMC_01_MotionBuf_SoftLimit_On (CardNo, TableNo, CD_AxisNo, LimitType, Mode, FifoNo);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

軟體極限設置請參閱第十七章 軟體極限 API
指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.27 _DMC_01_MotionBuf_SoftLimit_Off

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_SoftLimit_Off(U16 CardNo, U16 TableNo, U16 CD_AxisNo, I16 LimitType, U16 FifoNo)

■ **目的**

緩存區指令，緩存區內軟體極限關閉

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
CD_AxisNo	U16	編號單位	需要將限位無效的座標系軸編號
LimitType	I16	數值單位	需要無效的限位類型。 0：需要將該軸的正限位設置為無效。 1：需要將該軸的負限位設置為無效。 -1：需要將該軸的正限位和負限位都設置為無效。
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0, TableNo=0, CD_AxisNo=1, Mode =1, FifoNo =0;
I16 LimitType= -1;
rt = _DMC_01_set_soft_limit(CardNo, AxisNoArray[1], 0, 3000000, 2000000);
// AxisNoArray[1]為_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列 1

rt = _DMC_01_enable_soft_limit(CardNo, AxisNoArray[1], 0, 1);
rt =_DMC_01_MotionBuf_SoftLimit_Off (CardNo, TableNo, CD_AxisNo, LimitType, FifoNo);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

軟體極限設置請參閱第十七章 軟體極限 API

指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.28 _DMC_01_MotionBuf_Clear_Crd

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Clear_Crd(U16 CardNo, U16 TableNo, U16 FifoNo)

■ **目的**

清除坐標系緩存

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

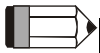
```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 FifoNo = 0;
rt = _DMC_01_MotionBuf_Clear_Crd(CardNo, TableNo, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突

DELTA CONFIDENTIAL

ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command



NOTE

_DMC_01_MotionBuf_Clear_Crd 會清除緩存並解除坐標系。

DELTA CONFIDENTIAL

47.29 _DMC_01_MotionBuf_Disable

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Disable(U16 CardNo, U16 TableNo, U16 FifoNo)

■ **目的**

緩存區指令，使用此功能可以在流程中關閉 Motion Buffer 功能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

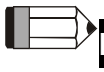
```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 FifoNo = 0;
rt = _DMC_01_MotionBuf_Disable (CardNo, TableNo, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動

DELTA CONFIDENTIAL

	已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command
ERR_MB_GEAR_CONTROL	Rt = 244，Motion Buffer BufGear 後面不可接緩存區指令
ERR_MB_DISABLE_COMMAND	Rt = 245，Motion Buffer 已使用 Disable command 後續指令無作用



一般用於 MotionBuffer 指令與 CSP 運動(例如單軸_DMC_01_start_tr_move)銜接處，在

MotionBuffer 指令與 CSP 之間下_DMC_01_MotionBuf_Disable，關閉 MotionBuffer，一旦下

_DMC_01_MotionBuf_Disable 會清空 MotionBuffer 緩存區，後續的 MotionBuffer 指令皆無作用，

必須重新建立坐標系_DMC_01_MotionBuf_Set_Crd_Parameters，再重新開始 MotionBuffer 指令。

※使用此 Function 後，如再使用緩存區指令，緩存區指令會有

Return code : ERR_PATH_MB_DISABLE_COMMAND

※可使用 _MotionBuf_Clear_Buffer 清除無法再使用緩存區指令的狀態

DELTA CONFIDENTIAL

47.30 _DMC_01_MotionBuf_Helix

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Helix(U16 CardNo, U16 TableNo, U16 *Helix_Axis, I32 *CenterPos, I32 Depth, I32 Pitch, I16 HelixDir, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，3 軸立體螺旋插補(以螺旋深度跟間距與中心點為輸入參數)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Helix_Axis	U16*	編號單位	此陣列大小為 3，輸入參與作螺旋的坐標系軸號 Helix_Axis [0] = 0 ,表示螺旋的第一軸為坐標系 X 軸 Helix_Axis [1] = 1 ,表示螺旋的第二軸為坐標系 Y 軸 Helix_Axis [2] = 2 ,表示螺旋的第三軸為坐標系 Z 軸
CenterPos	I32*	數值單位	螺旋插補的 XY 平面圓心相對於起點位置的 2 軸偏移量，陣列大小為 2。 單位 ：Pulse
Depth	I32	選項單位	螺旋插補運動的 Z 軸與起始位置的相對位移量。 單位 ：Pulse
Pitch	I32	數值單位	螺旋插補運動每一圈的間距。 註：螺旋運動 XY 最終位置會受參數 Depth 和 Pitch 影響。 Ex:起始位置(0,0),圓心(1000,0) Depth=1000,Pitch=400, 圈數=Depth/Pitch=1000/400=2.5 圈 最後 XY 平面座標會停在(2000,0)
HelixDir	I16	數值單位	螺旋的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。

DELTA CONFIDENTIAL

			單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 Helix_Axis [2]={0, 1};
//運動指令需填入坐標系陣列才能對應正確軸號。

I32 CenterPos[2] = {0, 20000};

I32 Depth = 100000; //此參數只能為相對位移量，不受 AbsMode 控制
I32 Pitch = 50000;
U16 HelixDir =0;
I32 SynTargetVel=1280000, SynEndVel=0, SynAcc=1280000;
U16 AbsMode =1;
U16 FifoNo =0;

rt =_DMC_01_MotionBuf_Helix(CardNo, TableNo, Helix_Axis, CenterPos, Depth, Pitch, HelixDir,
SynTargetVel, SynEndVel, SynAcc, AbsMode, FifoNo)
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動

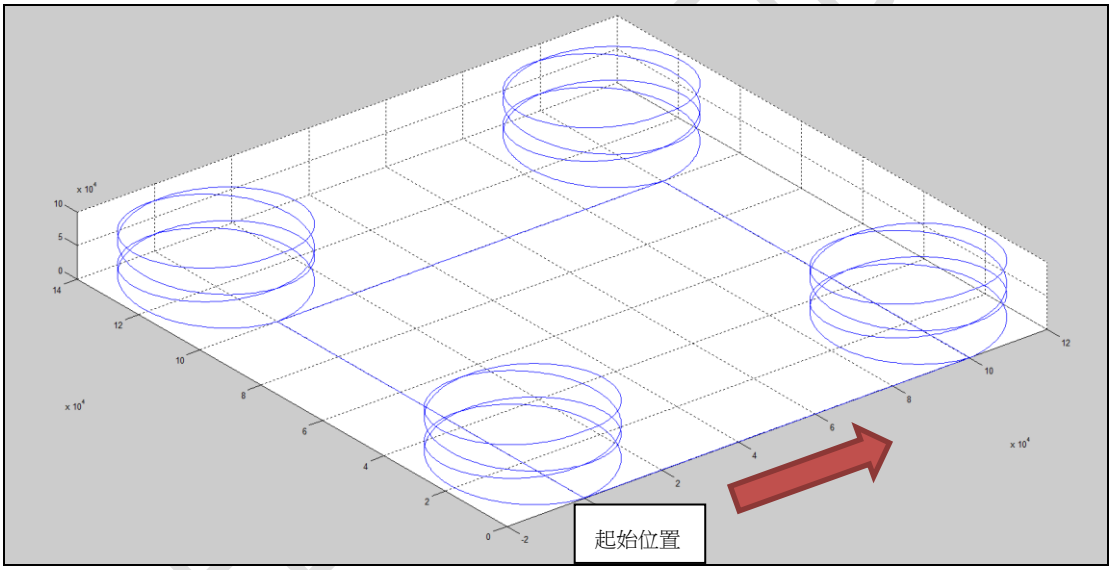
DELTA CONFIDENTIAL

	已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

XY 平面路徑示意圖(MotionBuffer Line 與 Helix 組合)



DELTA CONFIDENTIAL

47.31 _DMC_01_MotionBuf_Line_MultiAxis2

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Line_MultiAxis2(U16 CardNo, U16 TableNo, U16 AxisNum, U16 *Line_Axis, I32 *TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，多軸直線運動，可指定運動軸數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxisNum	U16	編號單位	軸數量，Max = 9
Line_Axis	U16*	編號單位	陣列大小同指定表格坐標系軸數，輸入參與多軸直線插補的坐標系軸號。 Line_Axis[0] = 0 ,表示直線的第一軸為坐標系 X 軸 Line_Axis[1] = 1 ,表示直線的第二軸為坐標系 Y 軸
TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同指定表格坐標系軸數。 單位 ：Pulse。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

DELTA CONFIDENTIAL

■ **範例**

```
I16 rt;
U16 CardNo=0,TableNo=0;
U16 AxisNum = 2;
U16 Line_Axis[2] = {0,1};
I32 TargetPosition[2] = {10000000, 10000000};
I32 SynTargetVel = 1280000, SynEndVel = 0, SynAcc = 1280000;
U16 AbsMode = 1;
U16 FifoNo = 0;

I16 rt = _DMC_01_MotionBuf_Line_MultiAxis2(CardNo, TableNo, AxisNum,  Line_Axis,
TargetPosition, SynTargetVel, SynEndVel, SynAcc,  AbsMode, FifoNo)
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

Line_Axis 陣列參數與運動指令 TargetPosition 會依輸入軸數 AxisNum 對應到 _DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列軸號。

DELTA CONFIDENTIAL

DELTA CONFIDENTIAL

47.32 _DMC_01_MotionBuf_Line_MultiAxis2_G00

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Line_MultiAxis2_G00(U16 CardNo, U16 TableNo, U16 AxisNum, U16 *Line_Axis, I32 *TargetPosition, I32 SynTargetVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，多軸直線運動，終點速度始終為 0(與 SNC 定義不同)，可指定運動軸數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxisNum	U16	編號單位	軸數量，Max = 9
Line_Axis	U16*	編號單位	陣列大小同指定表格坐標系軸數，輸入參與多軸直線插補的坐標系軸號。 Line_Axis[0] = 0 ,表示直線的第一軸為坐標系 X 軸 Line_Axis[1] = 1 ,表示直線的第二軸為坐標系 Y 軸
TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 AxisNum 數值。 單位 ：Pulse。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

I16 rt;

DELTA CONFIDENTIAL

```
U16 CardNo=0,TableNo=0;
U16 AxisNum = 2;
U16 Line_Axis[2] = {0,1};
I32 TargetPosition[2] = {10000000, 10000000};
I32 SynTargetVel = 1280000, SynAcc = 1280000;
U16 AbsMode = 1;
U16 FifoNo = 0;

I16 rt = _DMC_01_MotionBuf_Line_MultiAxis2_G00 (CardNo, TableNo, AxisNum,  Line_Axis,
TargetPosition, SynTargetVel, SynAcc, AbsMode, FifoNo)
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

Line_Axis 陣列參數與運動指令 TargetPosition 會依輸入軸數 AxisNum 對應到 _DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列軸號。

DELTA CONFIDENTIAL

47.33 _DMC_01_MotionBuf_Arc_MidPos

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Arc_MidPos(U16 CardNo, U16 TableNo, U16 *Arc_Axis, I32 *Arc_TargetPosition, I32 *Arc_MidTargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，平面圓弧插補(以圓弧經過的中間點與目標點為輸入參數)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Arc_Axis	U16*	編號單位	此陣列大小為 2，輸入參與作圓弧的坐標系軸號。 Arc_No[0] = 0 ,表示圓弧的第一軸為坐標系 X 軸 Arc_No[1] = 1 ,表示圓弧的第二軸為坐標系 Y 軸
Arc_TargetPosition	I32*	數值單位	此圓弧插補運動的末點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
Arc_MidTargetPosition	I32*	數值單位	此圓弧插補運動的中間點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
SynTargetVel	I32	數值單位	此運動的目標合成速度。
SynEndVel	I32	數值單位	單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的終點速度。
AbsMode	U16	選項單位	單位 ：Pulse/s
FifoNo	U16	數值單位	此運動的合成加速度

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

DELTA CONFIDENTIAL

```
I16 rt;
U16 CardNo=0;
U16 TableNo=0;
U16 Arc_Axis[4] = { 0, 1 };//運動指令需填入坐標系陣列才能對應正確軸號。
I32 Arc_TargetPosition[2] = { 120000, 20000 };
I32 Arc_MidTargetPosition[2] = { 80000, 0 };
I32 SynTargetVel = 128000 * 5;
I32 SynEndVel = 0;
I32 SynAcc = 1280000;
U16 AbsMode=1;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_Arc_MidPos(CardNo, TableNo , Axis_Array, Arc_TargetPosition,
Arc_MidTargetPosition, SynTargetVel, SynEndVel, SynAcc, AbsMode, FifoNo);
```

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

DELTA CONFIDENTIAL

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.34 _DMC_01_MotionBuf_MultiAxis_Arc1

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_MultiAxis_Arc1(U16 CardNo, U16 TableNo, U16 FollowAxisNum, U16 *Axis_Array, I32 *CenterPos, F64 Angle,U16 CircleDir, I32 *Follow_TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 SpeedMode, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，圓弧插補(以圓弧的圓心與角度為輸入參數)做多軸同動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FollowAxisNum	U16	編號單位	同動軸數。(不包含圓弧軸)
Axis_Array	U16*	數值單位	輸入參與圓弧多軸同動的坐標系軸號。 Axis_Array[0] = MOTION_BUF_AXIS_X ,表示圓弧的第一軸為坐標系 X 軸 Axis_Array[1] = MOTION_BUF_AXIS_Y ,表示圓弧的第二軸為坐標系 Y 軸。 Axis_Array[2] = MOTION_BUF_AXIS_A ,表示同步軸的第一軸為坐標系 A 軸。 …以此類推。
CenterPos	I32*	數值陣列	圓弧插補的 XY 平面圓心相對於起點位置的 2 軸偏移量，陣列大小為 2。 單位 ：Pulse
Angle	F64	數值單位	圓弧插補的 XY 平面的角度(0~360)，不可為零。 單位 ： 度
CircleDir	U16	數值單位	圓弧的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
Follow_Target Position	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 FollowAxisNum 參數。 單位 ：Pulse。

DELTA CONFIDENTIAL

SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
SpeedMode	U16	數值單位	0 : Arc 軸做合速，其餘相關軸跟隨。 1 : 所有相關軸做合速。
AbsMode	U16	選項單位	0 : 相對模式 1 : 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 FollowAxisNum=2;//2 個跟隨軸
U16 Axis_Array[4] = { 0, 1, 2, 3 };//軸 0,1 為圓弧, 軸 2,3 為跟隨
I32 Center_Pos[2] = { 0, 20000 };
F64 Angle = 270.0;
U16 CircleDir = 0;
I32 Follow_TargetPosition[2] = { 120000, 20000 };
I32 SynTargetVel = 128000 * 5;
I32 SynEndVel = 0;
I32 SynAcc = 1280000;
U16 SpeedMode = 0;
U16 AbsMode=1;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_MultiAxis_Arc1( CardNo, TableNo, FollowAxisNum, Axis_Array,
CenterPos, Angle, CircleDir, Follow_TargetPosition, SynTargetVel, SynEndVel, SynAcc,
SpeedMode, AbsMode, FifoNo)
```

■ **Error Code:**

DELTA CONFIDENTIAL

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.35 _DMC_01_MotionBuf_MultiAxis_Arc2

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_MultiAxis_Arc2(U16 CardNo, U16 TableNo, U16 FollowAxisNum, U16 *Axis_Array, I32 *Arc_TargetPosition, F64 Angle, U16 CircleDir, I32 *Follow_TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 SpeedMode, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，圓弧插補(以圓弧的終點位置與角度為輸入參數)做多軸同動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FollowAxisNum	U16	編號單位	同動軸數。(不包含圓弧軸)
Axis_Array	U16*	數值單位	輸入參與圓弧多軸同動的坐標系軸號。 Axis_Array[0] = MOTION_BUF_AXIS_X ,表示圓弧的第一軸為坐標系 X 軸 Axis_Array[1] = MOTION_BUF_AXIS_Y ,表示圓弧的第二軸為坐標系 Y 軸。 Axis_Array[2] = MOTION_BUF_AXIS_A ,表示同步軸的第一軸為坐標系 A 軸。 …以此類推。
Arc_TargetPosition	I32*	數值陣列	圓弧插補的 XY 平面的終點目標位置，陣列大小為 2。 單位 ：Pulse
Angle	F64	數值單位	圓弧插補的 XY 平面的角度(0~360)，不可為零。 單位 ： 度
CircleDir	U16	數值單位	圓弧的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
Follow_TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 FollowAxisNum 參數。 單位 ：Pulse。
SynTargetVel	I32	數值單位	此運動的目標合成速度。

DELTA CONFIDENTIAL

			單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
SpeedMode	U16	數值單位	0 : Arc 軸做合速，其餘相關軸跟隨。 1 : 所有相關軸做合速。
AbsMode	U16	選項單位	0 : 相對模式 1 : 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 FollowAxisNum=2;//2 個跟隨軸
U16 Axis_Array[4] = { 0, 1, 2, 3 };//軸 0,1 為圓弧, 軸 2,3 為跟隨
I32 Arc_TargetPosition[2] = { 120000, 20000 };
F64 Angle = 270.0;
U16 CircleDir = 0;
I32 Follow_TargetPosition[2] = { 120000, 20000 };
I32 SynTargetVel = 128000 * 5;
I32 SynEndVel = 0;
I32 SynAcc = 1280000;
U16 SpeedMode = 0;
U16 AbsMode=1;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_MultiAxis_Arc2 ( CardNo, TableNo, FollowAxisNum, Axis_Array,
Arc_TargetPosition, Angle, CircleDir, Follow_TargetPosition, SynTargetVel, SynEndVel, SynAcc,
SpeedMode, AbsMode, FifoNo)
```

■ **Error Code:**

DELTA CONFIDENTIAL

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.36 _DMC_01_MotionBuf_MultiAxis_Arc3

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_MultiAxis_Arc3(U16 CardNo, U16 TableNo, U16 FollowAxisNum, U16 *Axis_Array, I32 *Arc_TargetPosition, I32 *CenterPos, U16 CircleDir, I32 *Follow_TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 SpeedMode, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，圓弧插補(以圓弧的終點位置與圓心位置為輸入參數)做多軸同動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FollowAxisNum	U16	編號單位	同動軸數。(不包含圓弧軸)
Axis_Array	U16*	數值單位	輸入參與圓弧多軸同動的坐標系軸號。 Axis_Array[0] = MOTION_BUF_AXIS_X ,表示圓弧的第一軸為坐標系 X 軸 Axis_Array[1] = MOTION_BUF_AXIS_Y ,表示圓弧的第二軸為坐標系 Y 軸。 Axis_Array[2] = MOTION_BUF_AXIS_A ,表示同步軸的第一軸為坐標系 A 軸。 …以此類推。
Arc_TargetPosition	I32*	數值陣列	圓弧插補的 XY 平面的終點目標位置，陣列大小為 2。 單位 ：Pulse
CenterPos	I32*	數值陣列	圓弧插補的 XY 平面圓心相對於起點位置的 2 軸偏移量，陣列大小為 2。 單位 ：Pulse
CircleDir	U16	數值單位	圓弧的旋轉方向。 0：順時針圓弧。 1：逆時針圓弧。
Follow_TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 FollowAxisNum 參數。 單位 ：Pulse。

DELTA CONFIDENTIAL

SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
SpeedMode	U16	數值單位	0 : Arc 軸做合速，其餘相關軸跟隨。 1 : 所有相關軸做合速。
AbsMode	U16	選項單位	0 : 相對模式 1 : 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 FollowAxisNum=2;//2 個跟隨軸
U16 Axis_Array[4] = { 0, 1, 2, 3 };//軸 0,1 為圓弧, 軸 2,3 為跟隨
I32 Arc_TargetPosition[2] = { 120000, 20000 };
I32 Center_Pos[2] = { 0, 20000 };
U16 CircleDir = 1;
I32 Follow_TargetPosition[2] = { 120000, 20000 };
I32 SynTargetVel = 128000 * 5;
I32 SynEndVel = 0;
I32 SynAcc = 1280000;
U16 SpeedMode = 0;
U16 AbsMode=1;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_MultiAxis_Arc3 ( CardNo, TableNo, FollowAxisNum, Axis_Array,
Arc_TargetPosition, Center_Pos, CircleDir, Follow_TargetPosition, SynTargetVel, SynEndVel,
SynAcc, SpeedMode, AbsMode, FifoNo)
```

■ **Error Code:**

DELTA CONFIDENTIAL

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.37 _DMC_01_MotionBuf_MultiAxis_Arc4

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_MultiAxis_Arc4(U16 CardNo, U16 TableNo, U16 FollowAxisNum, U16 *Axis_Array, I32 *Arc_TargetPosition, I32 *Arc_MidPosition, I32 *Follow_TargetPosition, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 SpeedMode, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，圓弧插補(以圓弧經過的中間點與目標點為輸入參數)做多軸同動

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
FollowAxisNum	U16	編號單位	同動軸數。(不包含圓弧軸)
Axis_Array	U16*	數值單位	輸入參與圓弧多軸同動的坐標系軸號。 Axis_Array[0] = MOTION_BUF_AXIS_X ,表示圓弧的第一軸為坐標系 X 軸 Axis_Array[1] = MOTION_BUF_AXIS_Y ,表示圓弧的第二軸為坐標系 Y 軸。 Axis_Array[2] = MOTION_BUF_AXIS_A ,表示同步軸的第一軸為坐標系 A 軸。 …以此類推。
Arc_TargetPosition	I32*	數值陣列	此圓弧插補運動的末點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
Arc_MidTargetPosition	I32*	數值陣列	此圓弧插補運動的中間點位置，陣列大小為 2。 單位 ：Pulse 如為絕對模式，則此值為絕對位置 如為相對模式，則此值為相對位置
Follow_TargetPosition	I32*	數值單位	此多軸直線插補運動的末點位置。 陣列大小同 FollowAxisNum 參數。 單位 ：Pulse。

DELTA CONFIDENTIAL

SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
SpeedMode	U16	數值單位	0 : Arc 軸做合速，其餘相關軸跟隨。 1 : 所有相關軸做合速。
AbsMode	U16	選項單位	0 : 相對模式 1 : 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 FollowAxisNum=2;//2 個跟隨軸
U16 Axis_Array[4] = { 0, 1, 2, 3 };//軸 0,1 為圓弧, 軸 2,3 為跟隨
I32 Arc_TargetPosition[2] = { 120000, 20000 };
I32 Arc_MidTargetPosition[2] = { 80000, 0 };
I32 Follow_TargetPosition[2] = { 120000, 20000 };
I32 SynTargetVel = 128000 * 5;
I32 SynEndVel = 0;
I32 SynAcc = 1280000;
U16 SpeedMode = 0;
U16 AbsMode=1;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_MultiAxis_Arc4 ( CardNo, TableNo, FollowAxisNum, Axis_Array,
Arc_TargetPosition, Arc_MidTargetPosition, Follow_TargetPosition, SynTargetVel, SynEndVel,
SynAcc, SpeedMode, AbsMode, FifoNo)
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.38 _DMC_01_MotionBuf_Arc_follow

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Arc_follow(U16 CardNo, U16 TableNo, U16 AxesNum, U16 *AxisArray, U16 ArcMode, U16 *Arc_Axis_index, U16 master_axis_index, I32 *CenterPoint, I32 *EndPoint, F64 Angle, U16 Dir, I32 SynTargetVel, I32 SynEndVel, I32 SynAcc, U16 AbsMode, U16 FifoNo)

■ **目的**

緩存區指令，圓弧插補(可指定任意兩軸作為圓弧運動的X和Y軸，其餘軸跟隨圓弧中的其中一軸做跟隨運動。)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
AxesNum	U16	編號單位	參與軸數（≧2）。
AxisArray	U16*	數值單位	軸數組，數量等同於 AxisNum。 Ex: AxisNum=4; AxisArray[4] = { 0, 1, 2, 3 };
ArcMode	U16	數值陣列	選擇 Arc 模式。 0：已知圓心、角度（Arc1）。 1：已知末點、角度（Arc2）。 2：已知圓心、末點（Arc3）。 3：已知中間點、末點（Arc4）。
Arc_Axis_index	U16*	數值陣列	此陣列大小為 2，輸入參與作圓弧的坐標系軸號。 Arc_No[0] = 0 ,表示圓弧的第一軸為坐標系 X 軸 Arc_No[1] = 1 ,表示圓弧的第二軸為坐標系 Y 軸
master_axis_index	U16	數值單位	跟隨運動的主軸。此編號須和 Arc_Axis_index [0] or Arc_Axis_index [1] 其中一個一樣。
CenterPoint	I32*	數值單位	圓心位置 在 ArcMode=3 時，此參數定義為中間點位置。
EndPoint	I32*	數值單位	末點位置。
Angle	F64	數值單位	圓弧插補的 XY 平面的角度(0~360)，不可為零。 單位 ： 度

DELTA CONFIDENTIAL

Dir	U16	數值單位	0：順時針選轉。 1：逆時針旋轉。
SynTargetVel	I32	數值單位	此運動的目標合成速度。 單位 ：Pulse/s
SynEndVel	I32	數值單位	此運動的終點速度。 單位 ：Pulse/s
SynAcc	I32	數值單位	此運動的合成加速度 單位 ：Pulse/s^2
AbsMode	U16	選項單位	0： 相對模式 1： 絕對模式
FifoNo	U16	數值單位	插補緩存區號碼。(0 ~ 1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 AxesNum = 4;
U16 AxisArray[4] = { 0, 1, 2, 3 };
U16 ArcMode=0;// 已知圓心、角度（Arc1）。
U16 Arc_Axis_index[0] = 0;
U16 Arc_Axis_index[1] = 1;
U16 master_axis_index = 0;// 對應 AxisArray 陣列元素 2,3 跟隨 0 軸
I32 CenterPoint[0] = 200000;
I32 CenterPoint[1] = 200000;
I32 EndPoint[0] = 0;
I32 EndPoint[0] = 0;
F64 Angle=270;
U16 Dir=0;
I32 SynTargetVel = 128000 ;
I32 SynEndVel=0;
I32 SynAcc = 128000;
U16 AbsMode=0;
U16 FifoNo =0;

rt = _DMC_01_MotionBuf_Arc_follow(CardNo, TableNo, AxesNum, AxisArray, ArcMode,
```

DELTA CONFIDENTIAL

Arc_Axis_index, master_axis_index, CenterPoint, EndPoint, Angle, Dir, SynTargetVel, SynEndVel, SynAcc, AbsMode, FifoNo);

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.39 _DMC_01_MotionBuf_Set_G00_Check

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Set_G00_Check(U16 CardNo, U16 TableNo, U16 Check_Time, I32* Error_Range, U16 Enable)

■ **目的**

可設定比對 MultiAxis_G00 API 運行完成時驅動器 Feedback 是否達成目標範圍條件，達成條件後才執行下一筆指令，否則停止。(需建立坐標系後才能設定)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Check_Time	U16	數值單位	連續比對時間(ms)
Error_Range	I32*	數值陣列	Feedback 誤差範圍(Pluse)
Enable	U16	數值單位	0:關閉 1:啟動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 Check_Time = 5;
I32 Error_Range[9] = {100, 100, 100, 100, 100, 100, 100, 100, 100};
U16 Enable = 1;

rt = _DMC_01_MotionBuf_Set_G00_Check(CardNo, TableNo, Check_Time, &Error_Range[0], Enable);
// Error_Range 會依照_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系總軸數進行對應軸的比較運算。
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.40 _DMC_01_MotionBuf_Get_G00_Check

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_G00_Check(U16 CardNo, U16 TableNo, U16* Check_Time, I32* Error_Range, U16* Enable)

■ **目的**

取得比對條件設定值

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Check_Time	U16*	數值單位	取得設定的連續比對時間(ms)
Error_Range	I32*	數值陣列	取得設定的 Feedback 誤差範圍(Pluse)
Enable	U16*	數值單位	取得設定狀態 0:關閉 1:啟動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 Check_Time;
I32 Error_Range[9];
U16 Enable;

rt = _DMC_01_MotionBuf_Set_G00_Check(CardNo, TableNo, &Check_Time, &Error_Range[0], &Enable);
// Error_Range 會依照_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系總軸數進行對應軸取值。
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.41 _DMC_01_MotionBuf_Get_G00_Check_Flag

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_G00_Check_Flag(U16 CardNo, U16 TableNo, U16* Check_G00_Flag)

■ **目的**

取得比對狀態(MultiAxis_G00 API 運行完成時，才會開始進行 Feedback 比對)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
Check_G00_Flag	U16*	數值單位	取得比對狀態 1:MultiAxis_G00 API 運行完成時 0:比對完成 若結果保持為 1，表示比對不成功，無法進行下一筆指令。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
U16 Check_G00_Flag;


rt = _DMC_01_MotionBuf_Get_G00_Check_Flag (CardNo, TableNo, &Check_G00_Flag);
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL

47.42 _DMC_01_MotionBuf_Get_G00_Check_match

■ **格式**

I16 PASCAL _DMC_01_MotionBuf_Get_G00_Check_match(U16 CardNo, U16 TableNo, I32* feedback)

■ **目的**

取得比對成功時的 Feedback

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	軸卡號 CardNo 為 0~15
TableNo	U16	編號單位	表格號碼。(0 ,規格為一組 Table)
feedback	I32*	數值單位	取得比對成功時的 Feedback

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 rt=0;
U16 CardNo=0;
U16 TableNo=0;
I32 feedback[9];


rt = _DMC_01_MotionBuf_Get_G00_Check_match(CardNo, TableNo, &feedback[0]);
// feedback 會依照_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系總軸數進行對應軸取值。
```

DELTA CONFIDENTIAL

■ **Error Code:**

DSP Path.h	Return Code
ERR_MB_FIFO_FULL	Rt = 234，Motion Buffer FIFO 已滿
ERR_MB_ITP_START	Rt = 235，Motion Buffer 運動已啟動
ERR_FIFO_0_START	Rt = 236，Motion Buffer FIFO 0 已啟動
ERR_GEAR_AXISNO	Rt = 237，Motion Buffer BufGear 軸號和坐標系的軸號衝突
ERR_GEAR_SYNCHRONIZE_NUM	Rt = 238，Motion Buffer BufGear 同步運動已達上限
ERR_MB_IO_ADVANCE	Rt = 239，Motion Buffer IO 提前輸出時間超過坐標系設置時間
ERR_MB_NONE_INITIAL	Rt = 240，Motion Buffer 坐標系尚未建立
ERR_MB_CLEAR_BUFFER	Rt = 241，Motion Buffer 狀態在運動中，無法清除資料
ERR_MB_ARC_PARAMETER	Rt = 242，Motion Buffer Arc 運動參數錯誤，ex:半徑太短,起點和終點一樣
ERR_MB_SETCOMMAND	Rt = 243，Motion Buffer 狀態下無法使用 set command

NOTE

運動指令需填入_DMC_01_MotionBuf_Set_Crd_Parameters 所設置的坐標系陣列才能對應正確軸號。

DELTA CONFIDENTIAL